Last updated on 05/20/2025

Approved Verbs for PowerShell Commands

PowerShell uses a verb-noun pair for the names of cmdlets and for their derived .NET classes. The verb part of the name identifies the action that the cmdlet performs. The noun part of the name identifies the entity on which the action is performed. For example, the `Get-Command` cmdlet retrieves all the commands that are registered in PowerShell.

> **Note**
>
> PowerShell uses the term *verb* to describe a word that implies an action even if that word isn't a standard verb in the English language. For example, the term `New` is a valid PowerShell verb name because it implies an action even though it isn't a verb in the English language.

Each approved verb has a corresponding *alias prefix* defined. We use this alias prefix in aliases for commands using that verb. For example, the alias prefix for `Import` is `ip` and, accordingly, the alias for `Import-Module` is `ipmo`. This is a recommendation but not a rule; in particular, it need not be respected for command aliases mimicking well known commands from other environments.

Verb Naming Recommendations

The following recommendations help you choose an appropriate verb for your cmdlet, to ensure consistency between the cmdlets that you create, the cmdlets that are provided by PowerShell, and the cmdlets that are designed by others.

- Use one of the predefined verb names provided by PowerShell
- Use the verb to describe the general scope of the action, and use parameters to further refine the action of the cmdlet.
- Don't use a synonym of an approved verb. For example, always use `Remove`, never use `Delete` or `Eliminate`.
- Use only the form of each verb that's listed in this topic. For example, use `Get`, but don't use `Getting` or `Gets`.
- Don't use the following reserved verbs or aliases. The PowerShell language and a rare few cmdlets use these verbs under exceptional circumstances.
  - `ForEach` (`foreach`)
  - `Ping` (`pi`)
  - `Sort` (`sr`)
  - `Tee` (`te`)

- `Where` (`wh`)

You may get a complete list of verbs using the `Get-Verb` cmdlet.

Similar Verbs for Different Actions

The following similar verbs represent different actions.

`New` vs. `Add`

Use the `New` verb to create a new resource. Use the `Add` to add something to an existing container or resource. For example, `Add-Content` adds output to an existing file.

`New` vs. `Set`

Use the `New` verb to create a new resource. Use the `Set` verb to modify an existing resource, optionally creating it if it doesn't exist, such as the `Set-Variable` cmdlet.

`Find` vs. `Search`

Use the `Find` verb to look for an object. Use the `Search` verb to create a reference to a resource in a container.

`Get` vs. `Read`

Use the `Get` verb to obtain information about a resource (such as a file) or to obtain an object with which you can access the resource in future. Use the `Read` verb to open a resource and extract information contained within.

`Invoke` vs. `Start`

Use the `Invoke` verb to perform synchronous operations, such as running a command and waiting for it to end. Use the `Start` verb to begin asynchronous operations, such as starting an autonomous process.

`Ping` vs. `Test`

Use the `Test` verb.

Common Verbs

PowerShell uses the System.Management.Automation.VerbsCommon enumeration class to define generic actions that can apply to almost any cmdlet. The following table lists most of the defined verbs.

Expand table

Verb (alias)	Action	Synonyms to avoid
Add (`a`)	Adds a resource to a container, or attaches an item to another item. For example, the `Add-Content` cmdlet adds content to a file. This verb is paired with `Remove`.	`Append`, `Attach`, `Concatenate`, `Insert`
Clear (`cl`)	Removes all the resources from a container but doesn't delete the container. For example, the `Clear-Content` cmdlet removes the contents of a file but doesn't delete the file.	`Flush`, `Erase`, `Release`, `Unmark`, `Unset`, `Nullify`
Close (`cs`)	Changes the state of a resource to make it inaccessible, unavailable, or unusable. This verb is paired with `Open.`	
Copy (`cp`)	Copies a resource to another name or to another container. For example, the `Copy-Item` cmdlet copies an item (such as a file) from one location in the data store to another location.	`Duplicate`, `Clone`, `Replicate`, `Sync`
Enter (`et`)	Specifies an action that allows the user to move into a resource. For example, the `Enter-PSSession` cmdlet places the user in an interactive session. This verb is paired with `Exit`.	`Push`, `Into`
Exit (`ex`)	Sets the current environment or context to the most recently used context. For example, the `Exit-PSSession` cmdlet places the user in the session that was used to start the interactive session. This verb is paired with `Enter`.	`Pop`, `Out`
Find (`fd`)	Looks for an object in a container that's unknown, implied, optional, or specified.	`Search`
Format (`f`)	Arranges objects in a specified form or layout	
Get (`g`)	Specifies an action that retrieves a resource. This verb is paired with `Set`.	`Read`, `Open`, `Cat`, `Type`, `Dir`, `Obtain`, `Dump`, `Acquire`, `Examine`, `Find`, `Search`
Hide (`h`)	Makes a resource undetectable. For example, a cmdlet whose name includes the Hide verb might conceal a service from a user. This verb is paired with `Show`.	`Block`

Verb (alias)	Action	Synonyms to avoid
Join (`j`)	Combines resources into one resource. For example, the `Join-Path` cmdlet combines a path with one of its child paths to create a single path. This verb is paired with `Split`.	`Combine`, `Unite`, `Connect`, `Associate`
Lock (`lk`)	Secures a resource. This verb is paired with `Unlock`.	`Restrict`, `Secure`
Move (`m`)	Moves a resource from one location to another. For example, the `Move-Item` cmdlet moves an item from one location in the data store to another location.	`Transfer`, `Name`, `Migrate`
New (`n`)	Creates a resource. (The `Set` verb can also be used when creating a resource that includes data, such as the `Set-Variable` cmdlet.)	`Create`, `Generate`, `Build`, `Make`, `Allocate`
Open (`op`)	Changes the state of a resource to make it accessible, available, or usable. This verb is paired with `Close`.	
Optimize (`om`)	Increases the effectiveness of a resource.	
Pop (`pop`)	Removes an item from the top of a stack. For example, the `Pop-Location` cmdlet changes the current location to the location that was most recently pushed onto the stack.	
Push (`pu`)	Adds an item to the top of a stack. For example, the `Push-Location` cmdlet pushes the current location onto the stack.	
Redo (`re`)	Resets a resource to the state that was undone.	
Remove (`r`)	Deletes a resource from a container. For example, the `Remove-Variable` cmdlet deletes a variable and its value. This verb is paired with `Add`.	`Clear`, `Cut`, `Dispose`, `Discard`, `Erase`
Rename (`rn`)	Changes the name of a resource. For example, the `Rename-Item` cmdlet, which is used to access stored data, changes the name of an item in the data store.	`Change`
Reset (`rs`)	Sets a resource back to its original state.	
Resize (`rz`)	Changes the size of a resource.	
Search (`sr`)	Creates a reference to a resource in a container.	`Find`, `Locate`
Select (`sc`)	Locates a resource in a container. For example, the `Select-String` cmdlet finds text in strings and files.	`Find`, `Locate`

Verb (alias)	Action	Synonyms to avoid
Set (`s`)	Replaces data on an existing resource or creates a resource that contains some data. For example, the `Set-Date` cmdlet changes the system time on the local computer. (The `New` verb can also be used to create a resource.) This verb is paired with `Get`.	`Write`, `Reset`, `Assign`, `Configure`, `Update`
Show (`sh`)	Makes a resource visible to the user. This verb is paired with `Hide`.	`Display`, `Produce`
Skip (`sk`)	Bypasses one or more resources or points in a sequence.	`Bypass`, `Jump`
Split (`sl`)	Separates parts of a resource. For example, the `Split-Path` cmdlet returns different parts of a path. This verb is paired with `Join`.	`Separate`
Step (`st`)	Moves to the next point or resource in a sequence.	
Switch (`sw`)	Specifies an action that alternates between two resources, such as to change between two locations, responsibilities, or states.	
Undo (`un`)	Sets a resource to its previous state.	
Unlock (`uk`)	Releases a resource that was locked. This verb is paired with `Lock`.	`Release`, `Unrestrict`, `Unsecure`
Watch (`wc`)	Continually inspects or monitors a resource for changes.	

Communications Verbs

PowerShell uses the System.Management.Automation.VerbsCommunications class to define actions that apply to communications. The following table lists most of the defined verbs.

Expand table

Verb (alias)	Action	Synonyms to avoid
Connect (`cc`)	Creates a link between a source and a destination. This verb is paired with `Disconnect`.	`Join`, `Telnet`, `Login`
Disconnect (`dc`)	Breaks the link between a source and a destination. This verb is paired with `Connect`.	`Break`, `Logoff`
Read (`rd`)	Acquires information from a source. This verb is paired with `Write`.	`Acquire`, `Prompt`, `Get`

Verb (alias)	Action	Synonyms to avoid
Receive (`rc`)	Accepts information sent from a source. This verb is paired with `Send`.	`Read`, `Accept`, `Peek`
Send (`sd`)	Delivers information to a destination. This verb is paired with `Receive`.	`Put`, `Broadcast`, `Mail`, `Fax`
Write (`wr`)	Adds information to a target. This verb is paired with `Read`.	`Put`, `Print`

Data Verbs

PowerShell uses the System.Management.Automation.VerbsData class to define actions that apply to data handling. The following table lists most of the defined verbs.

Expand table

Verb Name (alias)	Action	Synonyms to avoid
Backup (`ba`)	Stores data by replicating it.	`Save`, `Burn`, `Replicate`, `Sync`
Checkpoint (`ch`)	Creates a snapshot of the current state of the data or of its configuration.	`Diff`
Compare (`cr`)	Evaluates the data from one resource against the data from another resource.	`Diff`
Compress (`cm`)	Compacts the data of a resource. Pairs with `Expand`.	`Compact`
Convert (`cv`)	Changes the data from one representation to another when the cmdlet supports bidirectional conversion or when the cmdlet supports conversion between multiple data types.	`Change`, `Resize`, `Resample`
ConvertFrom (`cf`)	Converts one primary type of input (the cmdlet noun indicates the input) to one or more supported output types.	`Export`, `Output`, `Out`
ConvertTo (`ct`)	Converts from one or more types of input to a primary output type (the cmdlet noun indicates the output type).	`Import`, `Input`, `In`
Dismount (`dm`)	Detaches a named entity from a location. This verb is paired with `Mount`.	`Unmount`, `Unlink`
Edit (`ed`)	Modifies existing data by adding or removing content.	`Change`, `Update`, `Modify`

Verb Name (alias)	Action	Synonyms to avoid
Expand (`en`)	Restores the data of a resource that has been compressed to its original state. This verb is paired with `Compress`.	`Explode`, `Uncompress`
Export (`ep`)	Encapsulates the primary input into a persistent data store, such as a file, or into an interchange format. This verb is paired with `Import`.	`Extract`, `Backup`
Group (`gp`)	Arranges or associates one or more resources	
Import (`ip`)	Creates a resource from data that's stored in a persistent data store (such as a file) or in an interchange format. For example, the `Import-Csv` cmdlet imports data from a comma-separated value (`CSV`) file to objects that can be used by other cmdlets. This verb is paired with `Export`.	`BulkLoad`, `Load`
Initialize (`in`)	Prepares a resource for use, and sets it to a default state.	`Erase`, `Init`, `Renew`, `Rebuild`, `Reinitialize`, `Setup`
Limit (`l`)	Applies constraints to a resource.	`Quota`
Merge (`mg`)	Creates a single resource from multiple resources.	`Combine`, `Join`
Mount (`mt`)	Attaches a named entity to a location. This verb is paired with `Dismount`.	`Connect`
Out (`o`)	Sends data out of the environment. For example, the `Out-Printer` cmdlet sends data to a printer.	
Publish (`pb`)	Makes a resource available to others. This verb is paired with `Unpublish`.	`Deploy`, `Release`, `Install`
Restore (`rr`)	Sets a resource to a predefined state, such as a state set by `Checkpoint`. For example, the `Restore-Computer` cmdlet starts a system restore on the local computer.	`Repair`, `Return`, `Undo`, `Fix`
Save (`sv`)	Preserves data to avoid loss.	
Sync (`sy`)	Assures that two or more resources are in the same state.	`Replicate`, `Coerce`, `Match`
Unpublish (`ub`)	Makes a resource unavailable to others. This verb is paired with `Publish`.	`Uninstall`, `Revert`, `Hide`
Update (`ud`)	Brings a resource up-to-date to maintain its state, accuracy, conformance, or compliance. For example, the `Update-FormatData` cmdlet updates and adds formatting files to the current PowerShell console.	`Refresh`, `Renew`, `Recalculate`, `Re-index`

Diagnostic Verbs

PowerShell uses the System.Management.Automation.VerbsDiagnostic class to define actions that apply to diagnostics. The following table lists most of the defined verbs.

Expand table

Verb (alias)	Action	Synonyms to avoid
Debug (db)	Examines a resource to diagnose operational problems.	Diagnose
Measure (ms)	Identifies resources that are consumed by a specified operation, or retrieves statistics about a resource.	Calculate, Determine, Analyze
Ping (pi)	Deprecated - Use the Test verb instead.	
Repair (rp)	Restores a resource to a usable condition	Fix, Restore
Resolve (rv)	Maps a shorthand representation of a resource to a more complete representation.	Expand, Determine
Test (t)	Verifies the operation or consistency of a resource.	Diagnose, Analyze, Salvage, Verify
Trace (tr)	Tracks the activities of a resource.	Track, Follow, Inspect, Dig

Lifecycle Verbs

PowerShell uses the System.Management.Automation.VerbsLifecycle class to define actions that apply to the lifecycle of a resource. The following table lists most of the defined verbs.

Expand table

Verb (alias)	Action	Synonyms to avoid
Approve (ap)	Confirms or agrees to the status of a resource or process.	
Assert (as)	Affirms the state of a resource.	Certify
Build (bd)	Creates an artifact (usually a binary or document) out of some set of input files (usually source code or declarative documents.) This verb was added in PowerShell 6.	

Verb (alias)	Action	Synonyms to avoid
Complete (`cp`)	Concludes an operation.	
Confirm (`cn`)	Acknowledges, verifies, or validates the state of a resource or process.	`Acknowledge`, `Agree`, `Certify`, `Validate`, `Verify`
Deny (`dn`)	Refuses, objects, blocks, or opposes the state of a resource or process.	`Block`, `Object`, `Refuse`, `Reject`
Deploy (`dp`)	Sends an application, website, or solution to a remote target[s] in such a way that a consumer of that solution can access it after deployment is complete. This verb was added in PowerShell 6.	
Disable (`d`)	Configures a resource to an unavailable or inactive state. For example, the `Disable-PSBreakpoint` cmdlet makes a breakpoint inactive. This verb is paired with `Enable`.	`Halt`, `Hide`
Enable (`e`)	Configures a resource to an available or active state. For example, the `Enable-PSBreakpoint` cmdlet makes a breakpoint active. This verb is paired with `Disable`.	`Start`, `Begin`
Install (`is`)	Places a resource in a location, and optionally initializes it. This verb is paired with `Uninstall`.	`Setup`
Invoke (`i`)	Performs an action, such as running a command or a method.	`Run`, `Start`
Register (`rg`)	Creates an entry for a resource in a repository such as a database. This verb is paired with `Unregister`.	
Request (`rq`)	Asks for a resource or asks for permissions.	
Restart (`rt`)	Stops an operation and then starts it again. For example, the `Restart-Service` cmdlet stops and then starts a service.	`Recycle`
Resume (`ru`)	Starts an operation that has been suspended. For example, the `Resume-Service` cmdlet starts a service that has been suspended. This verb is paired with `Suspend`.	
Start (`sa`)	Initiates an operation. For example, the `Start-Service` cmdlet starts a service. This verb is paired with `Stop`.	`Launch`, `Initiate`, `Boot`
Stop (`sp`)	Discontinues an activity. This verb is paired with `Start`.	`End`, `Kill`, `Terminate`, `Cancel`
Submit (`sb`)	Presents a resource for approval.	`Post`

Verb (alias)	Action	Synonyms to avoid
Suspend (`ss`)	Pauses an activity. For example, the `Suspend-Service` cmdlet pauses a service. This verb is paired with `Resume`.	`Pause`
Uninstall (`us`)	Removes a resource from an indicated location. This verb is paired with `Install`.	
Unregister (`ur`)	Removes the entry for a resource from a repository. This verb is paired with `Register`.	`Remove`
Wait (`w`)	Pauses an operation until a specified event occurs. For example, the `Wait-Job` cmdlet pauses operations until one or more of the background jobs are complete.	`Sleep`, `Pause`

Security Verbs

PowerShell uses the System.Management.Automation.VerbsSecurity class to define actions that apply to security. The following table lists most of the defined verbs.

Expand table

Verb (alias)	Action	Synonyms to avoid
Block (`bl`)	Restricts access to a resource. This verb is paired with `Unblock`.	`Prevent`, `Limit`, `Deny`
Grant (`gr`)	Allows access to a resource. This verb is paired with `Revoke`.	`Allow`, `Enable`
Protect (`pt`)	Safeguards a resource from attack or loss. This verb is paired with `Unprotect`.	`Encrypt`, `Safeguard`, `Seal`
Revoke (`rk`)	Specifies an action that doesn't allow access to a resource. This verb is paired with `Grant`.	`Remove`, `Disable`
Unblock (`ul`)	Removes restrictions to a resource. This verb is paired with `Block`.	`Clear`, `Allow`
Unprotect (`up`)	Removes safeguards from a resource that were added to prevent it from attack or loss. This verb is paired with `Protect`.	`Decrypt`, `Unseal`

Other Verbs

PowerShell uses the System.Management.Automation.VerbsOther class to define canonical verb names that don't fit into a specific verb name category such as the common, communications, data, lifecycle, or security verb names verbs.

Expand table

Verb (alias)	Action	Synonyms to avoid
Use (`u`)	Uses or includes a resource to do something.	

See Also

- System.Management.Automation.VerbsCommon
- System.Management.Automation.VerbsCommunications
- System.Management.Automation.VerbsData
- System.Management.Automation.VerbsDiagnostic
- System.Management.Automation.VerbsLifecycle
- System.Management.Automation.VerbsSecurity
- System.Management.Automation.VerbsOther
- Cmdlet Declaration
- Windows PowerShell Shell SDK

Last updated on 03/30/2026

Cmdlet Input Processing Methods

Cmdlets must override one or more of the input processing methods described in this topic to perform their work. These methods allow the cmdlet to perform operations of pre-processing, input processing, and post-processing. These methods also allow you to stop cmdlet processing. For a more detailed example of how to use these methods, see SelectStr Tutorial.

Pre-Processing Operations

Cmdlets should override the System.Management.Automation.Cmdlet.BeginProcessing method to add any preprocessing operations that are valid for all the records that will be processed later by the cmdlet. When PowerShell processes a command pipeline, PowerShell calls this method once for each instance of the cmdlet in the pipeline. For more information about how PowerShell invokes the command pipeline, see Cmdlet Processing Lifecycle.

The following code shows an implementation of the BeginProcessing method.

C#

```
protected override void BeginProcessing()
{
  // Replace the WriteObject method with the logic required by your cmdlet.
  WriteObject("This is a test of the BeginProcessing template.");
}
```

Input Processing Operations

Cmdlets can override the System.Management.Automation.Cmdlet.ProcessRecord method to process the input that is sent to the cmdlet. When PowerShell processes a command pipeline, PowerShell calls this method for each input record that is processed by the cmdlet. For more information about how PowerShell invokes the command pipeline, see Cmdlet Processing Lifecycle.

The following code shows an implementation of the ProcessRecord method.

C#

```
protected override void ProcessRecord()
{
  // Replace the WriteObject method with the logic required by your cmdlet.
```

```
  WriteObject("This is a test of the ProcessRecord template.");
}
```

Post-Processing Operations

Cmdlets should override the System.Management.Automation.Cmdlet.EndProcessing method to add any post-processing operations that are valid for all the records that were processed by the cmdlet. For example, your cmdlet might have to clean up object variables after it is finished processing.

When PowerShell processes a command pipeline, PowerShell calls this method once for each instance of the cmdlet in the pipeline. However, it is important to remember that the PowerShell runtime will not call the EndProcessing method if the cmdlet is canceled midway through its input processing or if a terminating error occurs in any part of the cmdlet. For this reason, a cmdlet that requires object cleanup should implement the complete System.IDisposable pattern, including a finalizer, so that the runtime can call both the EndProcessing and System.IDisposable.Dispose methods at the end of processing. For more information about how PowerShell invokes the command pipeline, see Cmdlet Processing Lifecycle.

The following code shows an implementation of the EndProcessing method.

C#

```
protected override void EndProcessing()
{
  // Replace the WriteObject method with the logic required by your cmdlet.
  WriteObject("This is a test of the EndProcessing template.");
}
```

See Also

System.Management.Automation.Cmdlet.BeginProcessing

System.Management.Automation.Cmdlet.ProcessRecord

System.Management.Automation.Cmdlet.EndProcessing

SelectStr Tutorial

System.IDisposable

Windows PowerShell Shell SDK

Last updated on 05/20/2025

Cmdlet Parameters

Cmdlet parameters provide the mechanism that allows a cmdlet to accept input. Parameters can accept input directly from the command line, or from objects passed to the cmdlet through the pipeline, The arguments (also known as *values*) of these parameters can specify the input that the cmdlet accepts, how the cmdlet should perform its actions, and the data that the cmdlet returns to the pipeline.

In This Section

Declaring Properties as Parameters Provides basic information you must understand before you declare the parameters of a cmdlet.

Types of Cmdlet Parameters Describes the different types of parameters that you can declare in cmdlets.

Cmdlet Parameter Name and Functionality Guidelines Discusses the names, recommended data type, and functionality of standard parameters.

Parameter Aliases Discusses the aliases that you can define for parameters.

Common Parameter Names This topic describes the parameters that Windows PowerShell adds to cmdlets.

Cmdlet Parameter Sets Discusses how parameter sets enable you to write a single cmdlet that can perform different actions for different scenarios.

Cmdlet Dynamic Parameters Discusses parameters that are available to the user under special conditions.

Supporting Wildcard Characters in Cmdlet Parameters Describes how to provide support for wildcard characters when you design a cmdlet that will be run against a group of resources.

Validating Parameter Input Describes how Windows PowerShell validates the arguments passed to cmdlet parameters.

Input Filter Parameters Discusses the `Filter`, `Include`, and `Exclude` parameters that filter the set of input objects that the cmdlet affects.

Related Sections

How to Validate Parameter Input

See Also

Parameter Attribute Declaration

Windows PowerShell Cmdlets

Last updated on 05/20/2025

Declaring Properties as Parameters

This topic provides basic information you must understand before you declare the parameters of a cmdlet.

To declare the parameters of a cmdlet within your cmdlet class, define the public properties that represent each parameter, and then add one or more Parameter attributes to each property. The Windows PowerShell runtime uses the Parameter attributes to identify the property as a cmdlet parameter. The basic syntax for declaring the Parameter attribute is `[Parameter()]`.

Here is an example of a property defined as a required parameter.

C#

```
[Parameter(Position = 0, Mandatory = true)]
public string UserName
{
  get { return userName; }
  set { userName = value; }
}
private string userName;
```

Here are some things to remember about parameters.

- A parameter must be explicitly marked as public. Parameters that are not marked as public default to internal and will not be found by the Windows PowerShell runtime.
- Parameters should be defined as Microsoft .NET Framework types to provide better parameter validation. For example, parameters that are restricted to one value out of a set of values should be defined as an enumeration type. Parameters that take a Uniform Resource Identifier (URI) value should be of type System.Uri.
- Avoid basic string parameters for all but free-form text properties.
- You can add a parameter to any number of parameter sets. For more information about parameter sets, see Cmdlet Parameter Sets.

Windows PowerShell also provides a set of common parameters that are automatically available to every cmdlet. For more information about these parameters and their aliases, see Cmdlet Common Parameters.

See Also

Cmdlet Common Parameters

Types of Cmdlet Parameter

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Types of Cmdlet Parameters

This topic describes the different types of parameters that you can declare in cmdlets. Cmdlet parameters can be positional, named, required, optional, or `[switch]` parameters.

Positional and Named Parameters

All cmdlet parameters are either named or positional parameters. A named parameter requires that you type the parameter name and argument when calling the cmdlet. A positional parameter requires only that you type the arguments in relative order. The system then maps the first unnamed argument to the first positional parameter. The system maps the second unnamed argument to the second unnamed parameter, and so on. By default, all cmdlet parameters are named parameters.

To define a named parameter, omit the `Position` keyword in the **Parameter** attribute declaration, as shown in the following parameter declaration.

C#

```
[Parameter(ValueFromPipeline=true)]
public string UserName
{
  get { return userName; }
  set { userName = value; }
}
private string userName;
```

To define a positional parameter, add the `Position` keyword in the Parameter attribute declaration, and then specify a position. In the following sample, the `UserName` parameter is declared as a positional parameter with position 0. This means that the first argument of the call is automatically bound to this parameter.

C#

```
[Parameter(Position = 0)]
public string UserName
{
  get { return userName; }
  set { userName = value; }
}
private string userName;
```

> ⓘ Note
>
> Good cmdlet design recommends that the most-used parameters be declared as positional parameters so that the user doesn't have to enter the parameter name when the cmdlet is run.

Positional and named parameters accept single arguments or multiple arguments separated by commas. Multiple arguments are allowed only if the parameter accepts a collection such as an array of strings. You may mix positional and named parameters in the same cmdlet. In this case, the system retrieves the named arguments first, and then attempts to map the remaining unnamed arguments to the positional parameters.

The following commands show the different ways in which you can specify single and multiple arguments for the parameters of the `Get-Command` cmdlet. Notice that in the last two samples, `-Name` doesn't need to be specified because the **Name** parameter is defined as a positional parameter.

PowerShell

```
Get-Command -Name Get-Service
Get-Command -Name Get-Service,Set-Service
Get-Command Get-Service
Get-Command Get-Service,Set-Service
```

Mandatory and Optional Parameters

You can also define cmdlet parameters as mandatory or optional parameters. (A mandatory parameter must be specified before the PowerShell runtime invokes the cmdlet.) By default, parameters are defined as optional.

To define a mandatory parameter, add the `Mandatory` keyword in the Parameter attribute declaration, and set it to `true`, as shown in the following parameter declaration.

C#

```
[Parameter(Position = 0, Mandatory = true)]
public string UserName
{
  get { return userName; }
  set { userName = value; }
```

```
}
private string userName;
```

To define an optional parameter, omit the `Mandatory` keyword in the **Parameter** attribute declaration, as shown in the following parameter declaration.

C#

```
[Parameter(Position = 0)]
public string UserName
{
  get { return userName; }
  set { userName = value; }
}
private string userName;
```

`[switch]` parameters

PowerShell provides a System.Management.Automation.SwitchParameter type that allows you to define a parameter whose default value `false` unless the parameter is specified when the cmdlet is called. Whenever possible, use `[switch]` parameters instead of Boolean parameters.

Consider the following example. Many PowerShell cmdlets return output. However, these cmdlets have a `PassThru` `[switch]` parameter that overrides the default behavior. When you use the `PassThru` parameter, the cmdlet returns output objects to the pipeline.

The following sample shows how to define a `[switch]` parameter:

C#

```
[Parameter()]
public SwitchParameter GoodBye
{
  get { return goodbye; }
  set { goodbye = value; }
}
private bool goodbye;
```

To make the cmdlet act on the parameter when it's specified, use the following structure within one of the input processing methods.

C#

```
protected override void ProcessRecord()
{
```

```
  WriteObject("Switch parameter test: " + userName + ".");
  if (goodbye)
  {
    WriteObject(" Goodbye!");
  }
} // End ProcessRecord
```

By default, `[switch]` parameters are excluded from positional parameters. You *can* override that in the **Parameter** attribute, but it can confuse users.

Design `[switch]` parameters so that using the parameter changes the default behavior of the command to a less common or more complicated mode. The simplest behavior of a command should be the default behavior that doesn't require the use of `[switch]` parameters. Base the behavior controlled by the `[switch]` parameter on the *value* of the parameter, not its *presence*.

There are several ways to test for the presence of a `[switch]` parameter:

- `MyInvocation.BoundParameters` contains the `[switch]` parameter name as a key
- `PSCmdlet.ParameterSetName` when the `[switch]` parameter defines a unique parameter set

For example, it's possible to provide an explicit value for the switch using `-MySwitch:$false` or splatting. If you only test for the presence of the parameter, the command behaves as if the switch value is `$true` instead of `$false`.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 04/08/2026

Standard Cmdlet Parameter Names and Types

Cmdlet parameter names should be consistent across the cmdlets that you design. The following topics list the parameter names that we recommend you use when you declare cmdlet parameters. The topics also describe the recommended data type and functionality of each parameter.

In This Section

Activity Parameters

Date and Time Parameters

Format Parameters

Property Parameters

Quantity Parameters

Resource Parameters

Security Parameters

Last updated on 05/20/2025

Activity Parameters

The following table lists the recommended names and functionality for activity parameters.

Expand table

Parameter	Functionality
Append Data type: SwitchParameter	Implement this parameter so that the user can add content to the end of a resource when the parameter is specified.
CaseSensitive Data type: SwitchParameter	Implement this parameter so the user can require case sensitivity when the parameter is specified.
Command Data type: String	Implement this parameter so the user can specify a command string to run.
CompatibleVersion Data type: System.Version object	Implement this parameter so the user can specify the semantics that the cmdlet must be compatible with for compatibility with previous versions.
Compress Data type: SwitchParameter	Implement this parameter so that data compression is used when the parameter is specified.
CompressionLevel Data type: Keyword	Implement this parameter so that the user can specify the algorithm to use for data compression.
Continuous Data type: SwitchParameter	Implement this parameter so that data is processed until the user terminates the cmdlet when the parameter is specified. If the parameter is not specified, the cmdlet processes a predefined amount of data and then terminates the operation.
Create Data type: SwitchParameter	Implement this parameter to indicate that a resource is created if one does not already exist when the parameter is specified.
Delete Data type: SwitchParameter	Implement this parameter so that resources are deleted when the cmdlet has completed its operation when the parameter is specified.
Drain Data type: SwitchParameter	Implement this parameter to indicate that outstanding work items are processed before the cmdlet processes new data when the parameter is specified. If the parameter is not specified, the work items are processed immediately.

Parameter	Functionality
Erase Data type: Int32	Implement this parameter so that the user can specify the number of times a resource is erased before it is deleted.
ErrorLevel Data type: Int32	Implement this parameter so that the user can specify the level of errors to report.
Exclude Data type: String[]	Implement this parameter so that the user can exclude something from an activity. For more information about how to use input filters, see Input Filter Parameters.
Filter Data type: Keyword	Implement this parameter so that the user can specify a filter that selects the resources upon which to perform the cmdlet action. For more information about how to use input filters, see Input Filter Parameters.
Follow Data type: SwitchParameter	Implement this parameter so that progress is tracked when the parameter is specified.
Force Data type: SwitchParameter	Implement this parameter to indicate that the user can perform an action even if restrictions are encountered when the parameter is specified. The parameter does not allow security to be compromised. For example, this parameter lets a user overwrite a read-only file.
Include Data type: String[]	Implement this parameter so that the user can include something in an activity. For more information about how to use input filters, see Input Filter Parameters.
Incremental Data type: SwitchParameter	Implement this parameter to indicate that processing is performed incrementally when the parameter is specified. For example, this parameter lets a user perform incremental backups that back up files only since the last backup.
InputObject Data type: Object	Implement this parameter when the cmdlet takes input from other cmdlets. When you define an **InputObject** parameter, always specify the **ValueFromPipeline** keyword when you declare the **Parameter** attribute. For more information about using input filters, see Input Filter Parameters.
Insert Data type: SwitchParameter	Implement this parameter so that the cmdlet inserts an item when the parameter is specified.
Interactive Data type: SwitchParameter	Implement this parameter so that the cmdlet works interactively with the user when the parameter is specified.
Interval Data type: HashTable	Implement this parameter so that the user can specify a hash table of keywords that contains the values. The following example shows sample values for the **Interval** parameter: `-interval @{ResumeScan=15; Retry=3}`.
Log Data type:	Implement this parameter audit the actions of the cmdlet when the parameter is specified.

Parameter	Functionality
SwitchParameter	
NoClobber Data type: SwitchParameter	Implement this parameter so that the resource will not be overwritten when the parameter is specified. This parameter generally applies to cmdlets that create new objects so that they can be prevented from overwriting existing objects with the same name.
Notify Data type: SwitchParameter	Implement this parameter so that the user will be notified that the activity is complete when the parameter is specified.
NotifyAddress Data type: Email address	Implement this parameter so that the user can specify the e-mail address to use to send a notification when the **Notify** parameter is specified.
Overwrite Data type: SwitchParameter	Implement this parameter so that the cmdlet overwrites any existing data when the parameter is specified.
Prompt Data type: String	Implement this parameter so that the user can specify a prompt for the cmdlet.
Quiet Data type: SwitchParameter	Implement this parameter so that the cmdlet suppresses user feedback during its actions when the parameter is specified.
Recurse Data type: SwitchParameter	Implement this parameter so that the cmdlet recursively performs its actions on resources when the parameter is specified.
Repair Data type: SwitchParameter	Implement this parameter so that the cmdlet will attempt to correct something from a broken state when the parameter is specified.
RepairString Data type: String	Implement this parameter so that the user can specify a string to use when the **Repair** parameter is specified.
Retry Data type: Int32	Implement this parameter so the user can specify the number of times the cmdlet will attempt an action.
Select Data type: Keyword array	Implement this parameter so that the user can specify an array of the types of items.
Stream Data type: SwitchParameter	Implement this parameter so the user can stream multiple output objects through the pipeline when the parameter is specified.
Strict Data type:	Implement this parameter so that all errors are handled as terminating errors when the parameter is specified.

Parameter	Functionality
SwitchParameter	
TempLocation Data type: String	Implement this parameter so the user can specify the location of temporary data that is used during the operation of the cmdlet.
Timeout Data type: Int32	Implement this parameter so that the user can specify the timeout interval (in milliseconds).
Truncate Data type: SwitchParameter	Implement this parameter so that the cmdlet will truncate its actions when the parameter is specified. If the parameter is not specified, the cmdlet performs another action.
Verify Data type: SwitchParameter	Implement this parameter so that the cmdlet will test to determine whether an action has occurred when the parameter is specified.
Wait Data type: SwitchParameter	Implement this parameter so that the cmdlet will wait for user input before continuing when the parameter is specified.
WaitTime Data type: Int32	Implement this parameter so that the user can specify the duration (in seconds) that the cmdlet will wait for user input when the **Wait** parameter is specified.

See Also

Cmdlet Parameters

Writing a Windows PowerShell Cmdlet

Windows PowerShell SDK

Last updated on 05/20/2025

Date and Time Parameters

The following table lists recommended names and functionality for parameters that handle date and time information. Date and time parameters are typically used to record when something is created or accessed.

Expand table

Parameter	Functionality
Accessed Data type: SwitchParameter	Implement this parameter so that when it is specified the cmdlet will operate on the resources that have been accessed based on the date and time specified by the **Before** and **After** parameters. If this parameter is specified, the **Created** and **Modified** parameters must be not be specified.
After Data type: DateTime	Implement this parameter to specify the date and time after which the cmdlet was used. For the **After** parameter to work, the cmdlet must also have an **Accessed**, **Created**, or **Modified** parameter. And, that parameter must be set to **true** when the cmdlet is called.
Before Data type: DateTime	Implement this parameter to specify the date and time before which the cmdlet was used. For the **Before** parameter to work, the cmdlet must also have an **Accessed**, **Created**, or **Modified** parameter. And, that parameter must be set to **true** when the cmdlet is called.
Created Data type: SwitchParameter	Implement this parameter so that when it is specified the cmdlet will operate on the resources that have been created based on the date and time specified by the **Before** and **After** parameters. If this parameter is specified, the **Accessed** and **Modified** parameters must not be specified.
Exact Data type: SwitchParameter	Implement this parameter so that when it is specified the resource term must match the resource name exactly. When the parameter is not specified the resource term and name do not need to match exactly.
Modified Data type: DateTime	Implement this parameter so that when it is specified the cmdlet will operate on resources that have been changed based on the date and time specified by the **Before** and **After** parameters. If this parameter is specified, the **Accessed** and **Created** parameters must not be specified.

See Also

Cmdlet Parameters

Writing a Windows PowerShell Cmdlet

Windows PowerShell SDK

Last updated on 05/20/2025

Format Parameters

The following table lists recommended names and functionality for parameters that are used to format or to generate data.

Expand table

Parameter	Functionality
As Data type: Keyword	Implement this parameter to specify the cmdlet output format. For example, possible values could be Text or Script.
Binary Data type: SwitchParameter	Implement this parameter to indicate that the cmdlet handles binary values.
Encoding Data type: Keyword	Implement this parameter to specify the type of encoding that is supported. For example, possible values could be ASCII, UTF8, Unicode, UTF7, BigEndianUnicode, Byte, and String.
NewLine Data type: SwitchParameter	Implement this parameter so that the newline characters are supported when the parameter is specified.
ShortName Data type: SwitchParameter	Implement this parameter so that short names are supported when the parameter is specified.
Width Data type: Int32	Implement this parameter so that the user can specify the width of the output device.
Wrap Data type: SwitchParameter	Implement this parameter so that text wrapping is supported when the parameter is specified.

See Also

Cmdlet Parameters

Writing a Windows PowerShell Cmdlet

Windows PowerShell SDK

Last updated on 05/20/2025

Property Parameters

The following table lists the recommended names and functionality for property parameters.

Expand table

Parameter	Functionality
Count Data type: Int32	Implement this parameter so that the user can specify the number of objects to be processed.
Description Data type: String	Implement this parameter so that the user can specify a description for a resource.
From Data type: String	Implement this parameter so that the user can specify the reference object to get information from.
Id Data type: Resource dependent	Implement this parameter so that the user can specify the identifier of a resource.
Input Data type: String	Implement this parameter so that the user can specify the input file specification.
Location Data type: String	Implement this parameter so that the user can specify the location of the resource.
LogName Data type: String	Implement this parameter so that the user can specify the name of the log file to process or use.
Name Data type: String	Implement this parameter so that the user can specify the name of the resource.
Output Data type: String	Implement this parameter so that the user can specify the output file.
Owner Data type: String	Implement this parameter so that the user can specify the name of the owner of the resource.
Property Data type: String	Implement this parameter so that the user can specify the name or the names of the properties to use.
Reason Data type: String	Implement this parameter so that the user can specify why this cmdlet is being invoked.
Regex Data type: SwitchParameter	Implement this parameter so that regular expressions are used when the parameter is specified. When this parameter is specified, wildcard characters are not resolved.

Parameter	Functionality
Speed Data type: Int32	Implement this parameter so that the user can specify the baud rate. The user sets this parameter to the speed of the resource.
State Data type: Keyword array	Implement this parameter so that the user can specify the names of states, such as KEYDOWN.
Value Data type: Object	Implement this parameter so that the user can specify a value to provide to the cmdlet.
Version Data type: String	Implement this parameter so that the user can specify the version of the property.

See Also

Cmdlet Parameters

Writing a Windows PowerShell Cmdlet

Windows PowerShell SDK

Last updated on 05/20/2025

Quantity Parameters

The following table lists the recommended names and functionality for quantity parameters.

Expand table

Parameter	Functionality
All Data type: Boolean	Implement this parameter so that `true` indicates that all resources should be acted upon instead of a default subset of resources. Implement this parameter so that `false` indicates a subset of the resources.
Allocation Data type: Int32	Implement this parameter so that the user can specify the number of items to allocate.
BlockCount Data type: Int64	Implement this parameter so that the user can specify the block count.
Count Data type: Int64	Implement this parameter so that the user can specify the count.
Scope Data type: Keyword	Implement this parameter so that the user can specify the scope to operate on.

See Also

Cmdlet Parameters

Writing a Windows PowerShell Cmdlet

Windows PowerShell SDK

Last updated on 05/20/2025

Resource Parameters

The following table lists the recommended names and functionality for resource parameters. For these parameters, the resources could be the assembly that contains the cmdlet class or the host application that is running the cmdlet.

Expand table

Parameter	Functionality
Application Data type: String	Implement this parameter so that the user can specify an application.
Assembly Data type: String	Implement this parameter so that the user can specify an assembly.
Attribute Data type: String	Implement this parameter so that the user can specify an attribute.
Class Data type: String	Implement this parameter so that the user can specify a Microsoft .NET Framework class.
Cluster Data type: String	Implement this parameter so that the user can specify a cluster.
Culture Data type: String	Implement this parameter so that the user can specify the culture in which to run the cmdlet.
Domain Data type: String	Implement this parameter so that the user can specify the domain name.
Drive Data type: String	Implement this parameter so that the user can specify a drive name.
Event Data type: String	Implement this parameter so that the user can specify an event name.

Parameter	Functionality
Interface Data type: String	Implement this parameter so that the user can specify a network interface name.
IpAddress Data type: String	Implement this parameter so that the user can specify an IP address.
Job Data type: String	Implement this parameter so that the user can specify a job.
LiteralPath Data type: String	Implement this parameter so that the user can specify the path to a resource when wildcard characters are not supported. (Use the **Path** parameter when wildcard characters are supported.)
Mac Data type: String	Implement this parameter so that the user can specify a media access controller (MAC) address.
ParentId Data type: String	Implement this parameter so that the user can specify the parent identifier.
Path Data type: String, String[]	Implement this parameter so that the user can indicate the paths to a resource when wildcard characters are supported. (Use the **LiteralPath** parameter when wildcard characters are not supported.) We recommend that you develop this parameter so that it supports the full `provider:path` syntax used by providers. We also recommend that you develop it so that it works with as many providers as possible.
Port Data type: Integer, String	Implement this parameter so that the user can specify an integer value for networking or a string value such as "biztalk" for other types of port.
Printer Data type: Integer, String	Implement this parameter so that the user can specify the printer for the cmdlet to use.
Size Data type: Int32	Implement this parameter so that the user can specify a size.
TID Data type: String	Implement this parameter so that the user can specify a transaction identifier (TID) for the cmdlet.
Type Data type: String	Implement this parameter so that the user can specify the type of resource on which to operate.

Parameter	Functionality
URL Data type: String	Implement this parameter so that the user can specify a Uniform Resource Locator (URL).
User Data type: String	Implement this parameter so that the user can specify their name or the name of another user.

See Also

Cmdlet Parameters

Writing a Windows PowerShell Cmdlet

Windows PowerShell SDK

Last updated on 05/20/2025

Security Parameters

The following table lists the recommended names and functionality for parameters used to provide security information for an operation, such as parameters that specify certificate key and privilege information.

Expand table

Parameter	Functionality
ACL Data type: String	Implement this parameter to specify the access control level of protection for a catalog or for a Uniform Resource Identifier (URI).
CertFile Data type: String	Implement this parameter so that the user can specify the name of a file that contains one of the following: - A Base64 or Distinguished Encoding Rules (DER) encoded x.509 certificate - A Public Key Cryptography Standards (PKCS) #12 file that contains at least one certificate and key
CertIssuerName Data type: String	Implement this parameter so that the user can specify the name of the issuer of a certificate or so that the user can specify a substring.
CertRequestFile Data type: String	Implement this parameter to specify the name of a file that contains a Base64 or DER-encoded PKCS #10 certificate request.
CertSerialNumber Data type: String	Implement this parameter to specify the serial number that was issued by the certification authority.
CertStoreLocation Data type: String	Implement this parameter so that the user can specify the location of the certificate store. The location is typically a file path.
CertSubjectName Data type: String	Implement this parameter so that the user can specify the issuer of a certificate or so that the user can specify a substring.
CertUsage Data type: String	Implement this parameter to specify the key usage or the enhanced key usage. The key can be represented as a bit mask, a bit, an object identifier (OID), or a string.
Credential Data type:	Implement this parameter so that the cmdlet will automatically prompt the user for a user name or

Parameter	Functionality
System.Management.Automation.PSCredential	password. A prompt for both is displayed if a full credential is not supplied directly.
CSPName Data type: String	Implement this parameter so that the user can specify the name of the certificate service provider (CSP).
CSPType Data type: Integer	Implement this parameter so that the user can specify the type of CSP.
Group Data type: String	Implement this parameter so that the user can specify a collection of principals for access. For more information, see the description of the **Principal** parameter.
KeyAlgorithm Data type: String	Implement this parameter so that the user can specify the key generation algorithm to use for security.
KeyContainerName Data type: String	Implement this parameter so that the user can specify the name of the key container.
KeyLength Data type: Integer	Implement this parameter so that the user can specify the length of the key in bits.
Operation Data type: String	Implement this parameter so that the user can specify an action that can be performed on a protected object.
Principal Data type: String	Implement this parameter so that the user can specify a unique identifiable entity for access.
Privilege Data type: String, String[]	Implement this parameter so that the user can specify the rights a cmdlet needs to perform an operation for a particular entity.
Role Data type: String	Implement this parameter so that the user can specify a set of operations that can be performed by an entity.
SaveCred Data type: SwitchParameter	Implement this parameter so that credentials that were previously saved by the user will be used when the parameter is specified.
Scope Data type: String	Implement this parameter so that the user can specify the group of protected objects for the cmdlet.
SID Data type: String	Implement this parameter so that the user can specify a unique identifier that represents a principal.
Trusted Data type: SwitchParameter	Implement this parameter so that trust levels are supported when the parameter is specified.

Parameter	Functionality
TrustLevel Data type: Keyword	Implement this parameter so that the user can specify the trust level that is supported. For example, possible values include internet, intranet, and fulltrust.

See Also

Cmdlet Parameters

Writing a Windows PowerShell Cmdlet

Windows PowerShell SDK

Last updated on 05/20/2025

Parameter Aliases

Cmdlet parameters can also have aliases. You can use the aliases instead of the parameter names when you type or specify the parameter in a command.

Benefits of Using Aliases

Adding aliases to parameters provides the following benefits.

- You can provide a shortcut so that the user does not have to use the complete parameter name when the cmdlet is called. For example, you could use the "CN" alias instead of the parameter name "ComputerName".
- You can define multiple aliases if you want to provide different names for the same parameter. You might want to define multiple aliases if you have to work with multiple user groups that refer to the same data in different ways.
- You can provide backwards compatibility for existing scripts if the name of a parameter changes.
- By using the Alias attribute along with the ValueFromPipelineByName attribute, you can define a parameter that allows your cmdlet to bind to different object types. For example, say you had two objects of different types and the first object had a writer property and the second object had an editor property. If your cmdlet had a parameter that had writer and editor aliases and the cmdlet accepted pipeline input based in property names, your cmdlet could bind to both objects using the two parameter aliases.

For more information about aliases that can be used with specific parameters, see Common Parameter Names.

Defining Parameter Aliases

To define an alias for a parameter, declare the Alias attribute, as shown in the following parameter declaration. In this example, multiple aliases are defined for the same parameter. (For more information, seeHow to Declare Cmdlet Parameters.)

```
C#
```

```
[Alias("UN","Writer","Editor")]
[Parameter()]
public string UserName
{
  get { return userName; }
  set { userName = value; }
}
private string userName;
```

See Also

Common Parameter Names

How to Declare Cmdlet Parameters

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Common Parameter Names

The parameters described in this topic are referred to as **common parameters**. They're added to cmdlets by the PowerShell runtime and can't be declared by the cmdlet.

> ⓘ **Note**
>
> These parameters are also added to provider cmdlets and to functions that are decorated with the `CmdletBinding` attribute.

General Common Parameters

The following parameters are added to all cmdlets and can be accessed whenever the cmdlet is run. These parameters are defined by the CommonParameters class.

Debug (alias: db)

Data type: **SwitchParameter**

This parameter specifies whether programmer-level debugging messages that can be displayed at the command line. These messages are intended for troubleshooting the operation of the cmdlet, and are generated by calls to the WriteDebug method. Debug messages don't need to be localized.

ErrorAction (alias: ea)

Data type: **Enumeration**

This parameter specifies what action should take place when an error occurs. The possible values for this parameter are defined by the ActionPreference enumeration.

ErrorVariable (alias: ev)

Data type: **String**

This parameter specifies the variable in which to place objects when an error occurs. To append to this variable, use *+varname* rather than clearing and setting the variable.

InformationAction (alias: infa)

Data type: **Enumeration**

This parameter specifies what action should take place when output is sent to the **Information** stream. The possible values for this parameter are defined by the ActionPreference enumeration.

InformationVariable (alias: iv)

Data type: **String**

This parameter specifies the variable in which to save output objects written to the **Information** stream. To append to this variable, use *+varname* rather than clearing and setting the variable.

OutBuffer (alias: ob)

Data type: **Int32**

This parameter defines the number of objects to store in the output buffer before any objects are passed down the pipeline. By default, objects are passed immediately down the pipeline.

OutVariable (alias: ov)

Data type: **String**

This parameter specifies the variable in which to place all output objects generated by the cmdlet. To append to this variable, use *+varname* rather than clearing and setting the variable.

PipelineVariable (alias: pv)

Data type: **String**

This parameter stores the value of the current pipeline element as a variable for any named command as it flows through the pipeline.

ProgressAction (alias: proga)

Data type: **Enumeration**

Determines how PowerShell responds to progress updates generated by a script, cmdlet, or provider, such as the progress bars generated by the `Write-Progress` cmdlet.

This parameter was added in PowerShell 7.4.

Verbose (alias: vb)

Data type: **SwitchParameter**

This parameter specifies whether the cmdlet writes explanatory messages that can be displayed at the command line. These messages are intended to provide additional help to the user, and are generated by calls to the WriteVerbose method.

WarningAction (alias: wa)

Data type: **Enumeration**

This parameter specifies what action should take place when the cmdlet writes a warning message. The possible values for this parameter are defined by the ActionPreference enumeration.

WarningVariable (alias: wv)

Data type: **String**

This parameter specifies the variable in which warning messages can be saved. To append to this variable, use *+varname* rather than clearing and setting the variable.

Risk-Mitigation Parameters

The following parameters are added to cmdlets that requests confirmation before they perform their action. For more information about confirmation requests, see Requesting Confirmation. These parameters are defined by the ShouldProcessParameters class.

Confirm (alias: cf)

Data type: **SwitchParameter**

This parameter specifies whether the cmdlet displays a prompt that asks if the user is sure that they want to continue.

WhatIf (alias: wi)

Data type: **SwitchParameter**

This parameter specifies whether the cmdlet writes a message that describes the effects of running the cmdlet without actually performing any action.

Transaction Parameters

The following parameter is added to cmdlets that support transactions. These parameters are defined by the TransactionParameters class.

Transaction support was introduced in PowerShell 3.0 and discontinued in PowerShell 6.0.

UseTransaction (alias: usetx)

Data type: **SwitchParameter**

This parameter specifies whether the cmdlet uses the current transaction to perform its action.

See Also

- about_CommonParameters
- System.Management.Automation.Internal.CommonParameters
- System.Management.Automation.Internal.ShouldProcessParameters
- System.Management.Automation.Internal.TransactionParameters
- Writing a Windows PowerShell Cmdlet
- Windows PowerShell SDK

Last updated on 05/20/2025

Cmdlet parameter sets

PowerShell uses parameter sets to enable you to write a single cmdlet that can do different actions for different scenarios. Parameter sets enable you to expose different parameters to the user. And, to return different information based on the parameters specified by the user.

Examples of parameter sets

For example, the PowerShell `Get-EventLog` cmdlet returns different information depending on whether the user specifies the **List** or **LogName** parameter. If the **List** parameter is specified, the cmdlet returns information about the log files themselves but not the event information they contain. If the **LogName** parameter is specified, the cmdlet returns information about the events in a specific event log. The **List** and **LogName** parameters identify two separate parameter sets.

Unique parameter

Each parameter set must have a unique parameter that the PowerShell runtime uses to expose the appropriate parameter set. If possible, the unique parameter should be a mandatory parameter. When a parameter is mandatory, the user must specify the parameter, and the PowerShell runtime uses that parameter to identify the parameter set. The unique parameter can't be mandatory if your cmdlet is designed to run without specifying any parameters.

Multiple parameter sets

In the following illustration, the left column shows three valid parameter sets. **Parameter A** is unique to the first parameter set, **parameter B** is unique to the second parameter set, and **parameter C** is unique to the third parameter set. In the right column, the parameter sets don't have a unique parameter.

Valid Parameter Sets	Invalid Parameter Sets
A D B D E C D	A B B C A C

Parameter set requirements

The following requirements apply to all parameter sets.

- Each parameter set must have at least one unique parameter. If possible, make this parameter a mandatory parameter.
- A parameter set that contains multiple positional parameters must define unique positions for each parameter. No two positional parameters can specify the same position.
- Only one parameter in a set can declare the `ValueFromPipeline` keyword with a value of `true`. Multiple parameters can define the `ValueFromPipelineByPropertyName` keyword with a value of `true`.
- If no parameter set is specified for a parameter, the parameter belongs to all parameter sets.

> **Note**
>
> For a cmdlet or function, there is a limit of 32 parameter sets.

Default parameter sets

When multiple parameter sets are defined, you can use the `DefaultParameterSetName` keyword of the **Cmdlet** attribute to specify the default parameter set. PowerShell uses the default parameter set if it can't determine the parameter set to use based on the information provided by the command. For more information about the **Cmdlet** attribute, see Cmdlet Attribute Declaration.

Declaring parameter sets

To create a parameter set, you must specify the `ParameterSetName` keyword when you declare the **Parameter** attribute for every parameter in the parameter set. For parameters that belong to multiple parameter sets, add a **Parameter** attribute for each parameter set. This attribute enables you to define the parameter differently for each parameter set. For example, you can define a parameter as mandatory in one set and optional in another. However, each parameter set must contain one unique parameter. For more information, see Parameter Attribute Declaration.

In the following example, the **UserName** parameter is the unique parameter of the `Test01` parameter set, and the **ComputerName** parameter is the unique parameter of the `Test02` parameter set. The **SharedParam** parameter belongs to both sets and is mandatory for the `Test01` parameter set but optional for the `Test02` parameter set.

C#

```
[Parameter(Position = 0, Mandatory = true, ParameterSetName = "Test01")]
public string UserName
{
  get { return userName; }
  set { userName = value; }
}
private string userName;

[Parameter(Position = 0, Mandatory = true, ParameterSetName = "Test02")]
public string ComputerName
{
  get { return computerName; }
  set { computerName = value; }
}
private string computerName;

[Parameter(Mandatory= true, ParameterSetName = "Test01")]
[Parameter(ParameterSetName = "Test02")]
public string SharedParam
{
    get { return sharedParam; }
    set { sharedParam = value; }
}
private string sharedParam;
```

Last updated on 05/20/2025

Cmdlet dynamic parameters

Cmdlets can define parameters that are available to the user under special conditions, such as when the argument of another parameter is a specific value. These parameters are added at runtime and are referred to as dynamic parameters because they're only added when needed. For example, you can design a cmdlet that adds several parameters only when a specific `[switch]` parameter is specified.

> **Note**
>
> Providers and PowerShell functions can also define dynamic parameters.

Dynamic parameters in PowerShell cmdlets

PowerShell uses dynamic parameters in several of its provider cmdlets. For example, the `Get-Item` and `Get-ChildItem` cmdlets add a **CodeSigningCert** parameter at runtime when the **Path** parameter specifies the **Certificate** provider path. If the **Path** parameter specifies a path for a different provider, the **CodeSigningCert** parameter isn't available.

The following examples show how the **CodeSigningCert** parameter is added at runtime when `Get-Item` is run.

In this example, the PowerShell runtime has added the parameter and the cmdlet is successful.

PowerShell

```powershell
Get-Item -Path Cert:\CurrentUser -CodeSigningCert
```

Output

```
Location   : CurrentUser
StoreNames : {SmartCardRoot, UserDS, AuthRoot, CA...}
```

In this example, a **FileSystem** drive is specified and an error is returned. The error message indicates that the **CodeSigningCert** parameter can't be found.

PowerShell

```
Get-Item -Path C:\ -CodeSigningCert
```

Output

```
Get-Item : A parameter cannot be found that matches parameter name
'CodeSigningCert'.
At line:1 char:37
+  Get-Item -Path C:\ -CodeSigningCert <<<<
--------
    CategoryInfo          : InvalidArgument: (:) [Get-Item],
ParameterBindingException
    FullyQualifiedErrorId :
NamedParameterNotFound,Microsoft.PowerShell.Commands.GetItemCommand
```

Support for dynamic parameters

To support dynamic parameters, the following elements must be included in the cmdlet code.

Interface

System.Management.Automation.IDynamicParameters. This interface provides the method that retrieves the dynamic parameters.

For example:

```
public class SendGreetingCommand : Cmdlet, IDynamicParameters
```

Method

System.Management.Automation.IDynamicParameters.GetDynamicParameters. This method retrieves the object that contains the dynamic parameter definitions.

For example:

C#

```
public object GetDynamicParameters()
{
  if (employee)
  {
    context= new SendGreetingCommandDynamicParameters();
    return context;
  }
  return null;
```

```
}
private SendGreetingCommandDynamicParameters context;
```

Class

A class that defines the dynamic parameters to be added. This class must include a **Parameter** attribute for each parameter and any optional **Alias** and **Validation** attributes that are needed by the cmdlet.

For example:

C#

```
public class SendGreetingCommandDynamicParameters
{
  [Parameter]
  [ValidateSet ("Marketing", "Sales", "Development")]
  public string Department
  {
    get { return department; }
    set { department = value; }
  }
  private string department;
}
```

For a complete example of a cmdlet that supports dynamic parameters, see How to Declare Dynamic Parameters.

See also

- System.Management.Automation.IDynamicParameters
- System.Management.Automation.IDynamicParameters.GetDynamicParameters
- How to Declare Dynamic Parameters
- Writing a Windows PowerShell Cmdlet

Last updated on 04/08/2026

Supporting Wildcard Characters in Cmdlet Parameters

Often, you will have to design a cmdlet to run against a group of resources rather than against a single resource. For example, a cmdlet might need to locate all the files in a data store that have the same name or extension. You must provide support for wildcard characters when you design a cmdlet that will be run against a group of resources.

> **Note**
>
> Using wildcard characters is sometimes referred to as *globbing*.

Windows PowerShell Cmdlets That Use Wildcards

Many Windows PowerShell cmdlets support wildcard characters for their parameter values. For example, almost every cmdlet that has a `Name` or `Path` parameter supports wildcard characters for these parameters. (Although most cmdlets that have a `Path` parameter also have a `LiteralPath` parameter that does not support wildcard characters.) The following command shows how a wildcard character is used to return all the cmdlets in the current session whose name contains the Get verb.

```
Get-Command get-*
```

Supported Wildcard Characters

Windows PowerShell supports the following wildcard characters.

Expand table

Wildcard	Description	Example	Matches	Does not match
*	Matches zero or more characters, starting at the specified position	`a*`	A, ag, Apple	
?	Matches any character at the specified position	`?n`	An, in, on	ran

Wildcard	Description	Example	Matches	Does not match
[ ]	Matches a range of characters	`[a-l]ook`	book, cook, look	nook, took
[ ]	Matches the specified characters	`[bn]ook`	book, nook	cook, look

When you design cmdlets that support wildcard characters, allow for combinations of wildcard characters. For example, the following command uses the `Get-ChildItem` cmdlet to retrieve all the .txt files that are in the C:\Techdocs folder and that begin with the letters "a" through "l."

```
Get-ChildItem C:\techdocs\[a-l]\*.txt
```

The previous command uses the range wildcard `[a-l]` to specify that the file name should begin with the characters "a" through "l" and uses the `*` wildcard character as a placeholder for any characters between the first letter of the filename and the **.txt** extension.

The following example uses a range wildcard pattern that excludes the letter "d" but includes all the other letters from "a" through "f."

```
Get-ChildItem C:\techdocs\[a-cef]\*.txt
```

Handling Literal Characters in Wildcard Patterns

If the wildcard pattern you specify contains literal characters that should not be interpreted as wildcard characters, use the backtick character (`` ` ``) as an escape character. When you specify literal characters int the PowerShell API, use a single backtick. When you specify literal characters at the PowerShell command prompt, use two backticks.

For example, the following pattern contains two brackets that must be taken literally.

When used in the PowerShell API use:

- "John Smith `[*`]"

When used from the PowerShell command prompt:

- "John Smith ``[*``]"

This pattern matches "John Smith [Marketing]" or "John Smith [Development]". For example:

```
```

```
PS> "John Smith [Marketing]" -like "John Smith ``[*``]"
True

PS> "John Smith [Development]" -like "John Smith ``[*``]"
True
```

Cmdlet Output and Wildcard Characters

When cmdlet parameters support wildcard characters, the operation usually generates an array output. Occasionally, it makes no sense to support an array output because the user might use only a single item. For example, the `Set-Location` cmdlet does not support array output because the user sets only a single location. In this instance, the cmdlet still supports wildcard characters, but it forces resolution to a single location.

See Also

Writing a Windows PowerShell Cmdlet

WildcardPattern Class

Last updated on 05/20/2025

Validating Parameter Input

PowerShell can validate the arguments passed to cmdlet parameters in several ways. PowerShell can validate the length, the range, and the pattern of the characters of the argument. It can validate the number of arguments available (the count). These validation rules are defined by validation attributes that are declared with the Parameter attribute on public properties of the cmdlet class.

To validate a parameter argument, the PowerShell runtime uses the information provided by the validation attributes to confirm the value of the parameter before the cmdlet is run. If the parameter input is not valid, the user receives an error message. Each validation parameter defines a validation rule that is enforced by PowerShell.

PowerShell enforces the validation rules based on the following attributes.

ValidateCount

Specifies the minimum and maximum number of arguments that a parameter can accept. For more information, see ValidateCount Attribute Declaration.

ValidateLength

Specifies the minimum and maximum number of characters in the parameter argument. For more information, see ValidateLength Attribute Declaration.

ValidatePattern

Specifies a regular expression that validates the parameter argument. For more information, see ValidatePattern Attribute Declaration.

ValidateRange

Specifies the minimum and maximum values of the parameter argument. For more information, see ValidateRange Attribute Declaration.

ValidateScript

Specifies the valid values for the parameter argument. For more information, see ValidateScript Attribute Declaration.

ValidateSet

Specifies the valid values for the parameter argument. For more information, see ValidateSet Attribute Declaration.

See Also

How to Validate Parameter Input

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Input Filter Parameters

A cmdlet can define `Filter`, `Include`, and `Exclude` parameters that filter the set of input objects that the cmdlet affects.

Typically, the set of input objects is specified by an `InputObject`, `Path`, or `Name` parameter. For example, a cmdlet can have a `Path` parameter that accepts multiple paths by using wildcard characters, and each path points to an input object. Used together, the `Filter`, `Include`, and `Exclude` parameters further qualify the paths the cmdlet works on each time it is invoked.

Include and Exclude Parameters

The `Include` and `Exclude` parameters identify the objects that are included or excluded from the set of input objects passed to the cmdlet. Use these parameters when the filter can be expressed in the standard wildcard language. (For more information about wildcard characters, see Supporting Wildcards in Cmdlet Parameters.) The `Include` parameter includes all the objects whose names match the inclusion filter. The `Exclude` parameter excludes all the objects whose names match the filter.

Filter Parameter

The `Filter` parameter specifies a filter that is not expressed in the standard wildcard language. For example, Active Directory Service Interfaces (ADSI) or SQL filters might be passed to the cmdlet through its `Filter` parameter. In the cmdlets provided by Windows PowerShell, these filters are specified by the Windows PowerShell providers that use the cmdlet to access a data store. Each provider typically defines its own filter.

Filtering If No Set of Input Objects Is Specified

If no set of input objects is specified, this typically means to filter against all objects. For more information, seeGet-Process.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Cmdlet Attributes

Windows PowerShell defines several attributes that you can use to add common functionality to your cmdlets without implementing that functionality within your own code. This includes the Cmdlet attribute that identifies a Microsoft .NET Framework class as a cmdlet class, the OutputType attribute that specifies the .NET Framework types returned by the cmdlet, the Parameter attribute that identifies public properties as cmdlet parameters, and more.

In This Section

Attributes in Cmdlet Code Describes the benefit of using attributes in cmdlet code.

Attribute Types Describes the different attributes that can decorate a cmdlet class.

Alias Attribute Declaration Describes how to define aliases for a cmdlet parameter name.

Cmdlet Attribute Declaration Describes how to define a .NET Framework class as a cmdlet.

Credential Attribute Declaration Describes how to add support for converting string input into a System.Management.Automation.PSCredential object.

OutputType attribute Declaration Describes how to specify the .NET Framework types returned by the cmdlet.

Parameter Attribute Declaration Describes how to define the parameters of a cmdlet.

ValidateCount Attribute Declaration Describes how to define how many arguments are allowed for a parameter.

ValidateLength Attribute Declaration Describes how to define the length (in characters) of a parameter argument.

ValidatePattern Attribute Declaration Describes how to define the valid patterns for a parameter argument.

ValidateRange Attribute Declaration Describes how to define the valid range for a parameter argument.

ValidateScript Attribute Declaration Describes how to define the possible values for a parameter argument.

ValidateSet Attribute Declaration Describes how to define the possible values for a parameter argument.

Reference

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Attributes in Cmdlet Code

To use the common functionality provided by Windows PowerShell, the classes and public properties defined in the cmdlet code are decorated with attributes. For example, the following class definition uses the Cmdlet attribute to identify the Microsoft .NET Framework class in which the **Get-Proc** cmdlet is implemented. (This cmdlet is used as an example in this document, and is similar to the `Get-Process` cmdlet provided by Windows PowerShell.)

C#

```
[Cmdlet(VerbsCommon.Get, "Proc")]
public class GetProcCommand : Cmdlet
```

These attributes are considered metadata because their implementation is separate from the implementation of the cmdlet code. When the Windows PowerShell runtime runs the cmdlet, it recognizes the attributes and then performs the appropriate action for each attribute.

Although you might want to implement your own version of the functionality provided by these attributes, a good cmdlet design uses these common functionalities.

For more information about the different attributes that can be declared in your cmdlets, see Attribute Types.

See Also

Attribute Types

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Attribute Types

Cmdlet attributes can be grouped by functionality. The following sections describe the available attributes and describe what the runtime does when the attribute is invoked.

Cmdlet Attributes

Cmdlet

Identifies a .NET Framework class as a cmdlet. This is the required base attribute. For more information, see Cmdlet Attribute Declaration.

Parameter Attributes

Parameter

Identifies a public property in the cmdlet class as a cmdlet parameter. For more information, see Parameter Attribute Declaration.

Alias

Specifies one or more aliases for a parameter. For more information, see Alias Attribute Declaration.

Argument Validation Attributes

ValidateCount

Specifies the minimum and maximum number of arguments that are allowed for a cmdlet parameter. For more information, see ValidateCount Attribute Declaration.

ValidateLength

Specifies a minimum and maximum number of characters for a cmdlet parameter argument. For more information, see ValidateLength Attribute Declaration.

ValidatePattern

Specifies a regular expression pattern that the cmdlet parameter argument must match. For more information, see ValidatePattern Attribute Declaration.

ValidateRange

Specifies the minimum and maximum values for a cmdlet parameter argument. For more information, see ValidateRange Attribute Declaration.

ValidateSet

Specifies a set of valid values for the cmdlet parameter argument. For more information, see ValidateSet Attribute Declaration.

See Also

Windows PowerShell SDK

Last updated on 05/20/2025

Alias Attribute Declaration

The **Alias** attribute allows the user to specify different names for a cmdlet or a cmdlet parameter. Aliases can be used to provide shortcuts for a parameter name, or they can provide different names that are appropriate for different scenarios.

Syntax

C#

```
[Alias(aliasNames)]
```

Parameters

`aliasNames` (String[]) Required. Specifies a set of comma-separated alias names for the cmdlet parameter.

Remarks

The **Alias** attribute is defined by the System.Management.Automation.AliasAttribute class.

Cmdlet aliases

- The **Alias** attribute is used with the cmdlet declaration. For more information about how to declare these attributes, see Cmdlet Aliases.
- Each parameter alias name must be unique. Windows PowerShell does not check for duplicate alias names.

Parameter aliases

- The **Alias** attribute is used with the **Parameter** attribute when you specify a cmdlet parameter. For more information about how to declare these attributes, see How to Declare Cmdlet Parameters.
- Each parameter alias name must be unique within a cmdlet. Windows PowerShell does not check for duplicate alias names.
- The Alias attribute is used once for each parameter in a cmdlet.

See Also

Cmdlet Aliases

Parameter Aliases

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Cmdlet Attribute Declaration

The Cmdlet attribute identifies a Microsoft .NET Framework class as a cmdlet and specifies the verb and noun used to invoke the cmdlet.

Syntax

C#

```
[Cmdlet("verbName", "nounName")]
[Cmdlet("verbName", "nounName", Named Parameters...)]
```

Parameters

`VerbName` (System.String) Required. Specifies the cmdlet verb. This verb specifies the action taken by the cmdlet. For more information about approved cmdlet verbs, see Cmdlet Verb Names and Required Development Guidelines.

`NounName` (System.String) Required. Specifies the cmdlet noun. This noun specifies the resource that the cmdlet acts upon. For more information about cmdlet nouns, see Cmdlet Declaration and Strongly Encouraged Development Guidelines.

`SupportsShouldProcess` (System.Boolean) Optional named parameter. `True` indicates that the cmdlet supports calls to the System.Management.Automation.Cmdlet.ShouldProcess method, which provides the cmdlet with a way to prompt the user before an action that changes the system is performed. `False`, the default value, indicates that the cmdlet does not support calls to the System.Management.Automation.Cmdlet.ShouldProcess method. For more information about confirmation requests, see Requesting Confirmation.

`ConfirmImpact` (System.Management.Automation.ConfirmImpact) Optional named parameter. Specifies when the action of the cmdlet should be confirmed by a call to the System.Management.Automation.Cmdlet.ShouldProcess method. System.Management.Automation.Cmdlet.ShouldProcess will only be called when the ConfirmImpact value of the cmdlet (by default, Medium) is equal to or greater than the value of the `$ConfirmPreference` variable. This parameter should be specified only when the `SupportsShouldProcess` parameter is specified.

`DefaultParameterSetName` (System.String) Optional named parameter. Specifies the default parameter set that the Windows PowerShell runtime attempts to use when it cannot determine which parameter set to use. Notice that this situation can be eliminated by making the unique parameter of each parameter set a mandatory parameter.

There is one case where Windows PowerShell cannot use the default parameter set even if a default parameter set name is specified. The Windows PowerShell runtime cannot distinguish between parameter sets based solely on object type. For example, if you have one parameter set that takes a string as the file path, and another set that takes a **FileInfo** object directly, Windows PowerShell cannot determine which parameter set to use based on the values passed to the cmdlet, nor does it use the default parameter set. In this case, even if you specify a default parameter set name, Windows PowerShell throws an ambiguous parameter set error message.

`SupportsTransactions` (System.Boolean) Optional named parameter. `True` indicates that the cmdlet can be used within a transaction. When `True` is specified, the Windows PowerShell runtime adds the `UseTransaction` parameter to the parameter list of the cmdlet. `False`, the default value, indicates that the cmdlet cannot be used within a transaction.

Remarks

- Together, the verb and noun are used to identify your registered cmdlet and to invoke your cmdlet within a script.
- When the cmdlet is invoked from the Windows PowerShell console, the command resembles the following command:

VerbName-NounName

- All cmdlets that change resources outside of Windows PowerShell should include the `SupportsShouldProcess` keyword when the Cmdlet attribute is declared, which allows the cmdlet to call the System.Management.Automation.Cmdlet.ShouldProcess method before the cmdlet performs its action. If the System.Management.Automation.Cmdlet.ShouldProcess call returns `false`, the action should not be taken. For more information about the confirmation requests generated by the System.Management.Automation.Cmdlet.ShouldProcess call, see Requesting Confirmation.

The `Confirm` and `WhatIf` cmdlet parameters are available only for cmdlets that support System.Management.Automation.Cmdlet.ShouldProcess calls.

Example

The following class definition uses the Cmdlet attribute to identify the .NET Framework class for a **Get-Proc** cmdlet that retrieves information about the processes running on the local computer.

C#

```
[Cmdlet(VerbsCommon.Get, "Proc")]
public class GetProcCommand : Cmdlet
```

For more information about the **Get-Proc** cmdlet, see GetProc Tutorial.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Credential Attribute Declaration

The Credential attribute is an optional attribute that can be used with credential parameters of type System.Management.Automation.PSCredential so that a string can also be passed as an argument to the parameter. When this attribute is added to a parameter declaration, Windows PowerShell converts the string input into a System.Management.Automation.PSCredential object. For example, the Get-Credential cmdlet uses this attribute to have Windows PowerShell generate the System.Management.Automation.PSCredential object that is returned by the cmdlet.

Syntax

C#

```
[Credential]
```

Remarks

- Typically this attribute is used by parameters of type System.Management.Automation.PSCredential so that a string can also be passed as an argument to the parameter. When a System.Management.Automation.PSCredential object is passed to the parameter, Windows PowerShell does nothing.
- When creating the System.Management.Automation.PSCredential object, Windows PowerShell uses the current Host to display the appropriate prompts to the user. For example, the default Host displays a prompt for a user name and password when this attribute is used. However, if a custom host is being used that defines a different prompt then that prompt would be displayed.
- This attribute is used with the Parameter attribute. For more information about that attribute, see Parameter Attribute Declaration.
- The credential attribute is defined by the System.Management.Automation.CredentialAttribute class.

See Also

Parameter Aliases

Parameter Attribute Declaration

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

OutputType Attribute Declaration

The `OutputType` attribute identifies the .NET Framework types returned by a cmdlet, function, or script.

Syntax

C#

```
[OutputType(params string[] type)]
[OutputType(params Type[] type)]
[OutputType(params string[] type, Named Parameters...)]
[OutputType(params Type[] type, Named Parameters...)]
```

Parameters

Type (`string[]` or `Type[]`) Required. Specifies the types returned by the cmdlet function, or script.

ParameterSetName (string[]) Optional. Specifies the parameter sets that return the types specified in the `type` parameter.

providerCmdlet Optional. Specifies the provider cmdlet that returns the types specified in the `type` parameter.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Parameter Attribute Declaration

The Parameter attribute identifies a public property of the cmdlet class as a cmdlet parameter.

Syntax

C#

```
[Parameter()]
[Parameter(Named Parameters...)]
```

Parameters

`Mandatory` (System.Boolean) Optional named parameter. `True` indicates the cmdlet parameter is required. If a required parameter is not provided when the cmdlet is invoked, Windows PowerShell prompts the user for a parameter value. The default is `false`.

`ParameterSetName` (System.String) Optional named parameter. Specifies the parameter set that this cmdlet parameter belongs to. If no parameter set is specified, the parameter belongs to all parameter sets.

`Position` (System.Int32) Optional named parameter. Specifies the position of the parameter within a Windows PowerShell command.

`ValueFromPipeline` (System.Boolean) Optional named parameter. `True` indicates that the cmdlet parameter takes its value from a pipeline object. Specify this keyword if the cmdlet accesses the complete object, not just a property of the object. The default is `false`.

`ValueFromPipelineByPropertyName` (System.Boolean) Optional named parameter. `True` indicates that the cmdlet parameter takes its value from a property of a pipeline object that has either the same name or the same alias as this parameter. For example, if the cmdlet has a `Name` parameter and the pipeline object also has a `Name` property, the value of the `Name` property is assigned to the `Name` parameter of the cmdlet. The default is `false`.

`ValueFromRemainingArguments` (System.Boolean) Optional named parameter. `True` indicates that the cmdlet parameter accepts all remaining arguments that are passed to the cmdlet. The default is `false`.

`HelpMessage` (System.String) Optional named parameter. Specifies a short description of the parameter. Windows PowerShell displays this message when a cmdlet is run and a mandatory parameter is not specified.

`HelpMessageBaseName` (System.String) Optional named parameter. Specifies the location where resource identifiers reside. For example, this parameter could specify a resource assembly that contains Help messages that you want to localize.

`HelpMessageResourceId` (System.String) Optional named parameter.Specifies the resource identifier for a Help message.

`DontShow` (System.Boolean) Optional named parameter. `True` indicates that the parameter is hidden from the user for tab expansion and IntelliSense. The default is `false`.

Remarks

- For more information about how to declare this attribute, see How to Declare Cmdlet Parameters.
- A cmdlet can have any number of parameters. However, for a better user experience, limit the number of parameters.
- Parameters must be declared on public non-static fields or properties. Parameters should be declared on properties. The property must have a public set accessor, and if the `ValueFromPipeline` or `ValueFromPipelineByPropertyName` keyword is specified, the property must have a public get accessor.
- When you specify positional parameters, limit the number of positional parameters in a parameter set to less than five. And, positional parameters do not have to be contiguous. Positions 5, 100, and 250 work the same as positions 0, 1, and 2.
- When the `Position` keyword is not specified, the cmdlet parameter must be referenced by its name.
- When you use parameter sets, note the following:
    - Each parameter set must have at least one unique parameter. Good cmdlet design indicates this unique parameter should also be mandatory if possible. If your cmdlet is designed to be run without parameters, the unique parameter cannot be mandatory.
    - No parameter set should contain more than one positional parameter with the same position.

    - Only one parameter in a parameter set should declare `ValueFromPipeline = true`.
    - Multiple parameters can define `ValueFromPipelineByPropertyName = true`.
- For more information about the guidelines for parameter names, see Cmdlet Parameter Names.
- The parameter attribute is defined by the System.Management.Automation.ParameterAttribute class.
- The `DontShow` parameter has the following side effects:
    - Affects all parameter sets for the associated parameter, even if there's a parameter set in which `DontShow` is unused.
    - Hides common parameters from tab completion and IntelliSense. `DontShow` doesn't hide the optional common parameters: **WhatIf**, **Confirm**, or **UseTransaction**.

See Also

- System.Management.Automation.ParameterAttribute
- Cmdlet Parameter Names
- Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

ValidateCount Attribute Declaration

The ValidateCount attribute specifies the minimum and maximum number of arguments allowed for a cmdlet parameter.

Syntax

C#

```
[ValidateCount(int minLength, int maxlength)]
```

Parameters

`MinLength` (System.Int32) Required. Specifies the minimum number of arguments.

`MaxLength`(System.Int32) Required. Specifies the maximum number of arguments.

Remarks

- For more information about how to declare this attribute, see How to Validate an Argument Count.
- When this attribute is not invoked, the corresponding cmdlet parameter can have any number of arguments.
- The Windows PowerShell runtime throws an error under the following conditions:
  - The `MinLength` and `MaxLength` attribute parameters are not of type System.Int32.
  - The value of the `MaxLength` attribute parameter is less than the value of the `MinLength` attribute parameter.
- The ValidateCount attribute is defined by the System.Management.Automation.ValidateCountAttribute class.

See Also

System.Management.Automation.ValidateCountAttribute

How to Validate an Argument Count

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

ValidateLength Attribute Declaration

The ValidateLength attribute specifies the minimum and maximum number of characters for a cmdlet parameter argument. This attribute can also be used by Windows PowerShell functions.

Syntax

```
C#
```

```
[ValidateLength(int minLength, int maxlength)]
```

Parameters

`MinLength` (System.Int32) Required. Specifies the minimum number of characters allowed.

`MaxLength` (System.Int32) Required. Specifies the maximum number of characters allowed.

Remarks

- For more information about how to declare this attribute, see How to Declare Input Validation Rules.
- When this attribute is not used, the corresponding parameter argument can be of any length.
- The Windows PowerShell runtime throws an error under the following conditions:
  - When the value of the `MaxLength` attribute parameter is less than the value of the `MinLength` attribute parameter.
  - When the `MaxLength` attribute parameter is set to 0.
  - When the argument is not a string.
- The ValidateLength attribute is defined by the System.Management.Automation.ValidateLengthAttribute class.

See Also

System.Management.Automation.ValidateLengthAttribute

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

ValidatePattern Attribute Declaration

The ValidatePattern attribute specifies a regular expression pattern that validates the argument of a cmdlet parameter. This attribute can also be used by Windows PowerShell functions.

When ValidatePattern is invoked within a cmdlet, the Windows PowerShell runtime converts the argument of the cmdlet parameter to a string and then compares that string to the pattern supplied by the ValidatePattern attribute. The cmdlet is run only if the converted string representation of the argument and the supplied pattern match. If they do not match, an error is thrown by the Windows PowerShell runtime.

Syntax

C#

```
[ValidatePattern(string regexString)]
[ValidatePattern(string regexString, Named Parameters)]
```

Parameters

`RegexString` (System.String) Required. Specifies a regular expression that validates the parameter argument.

Options (System.Text.RegularExpressions.RegexOptions) Optional named parameter. Specifies a bitwise combination of System.Text.RegularExpressions.RegexOptions flags that specify regular expression options.

Remarks

- This attribute can be used only once per parameter.
- You can use the `Option` parameter of the attribute to further define the pattern. For example, you can make the pattern case sensitive.
- If this attribute is applied to a collection, each element in the collection must match the pattern.
- The ValidatePattern attribute is defined by the System.Management.Automation.ValidatePatternAttribute class.

See Also

System.Management.Automation.ValidatePatternAttribute

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

ValidateRange Attribute Declaration

The ValidateRange attribute specifies the minimum and maximum values (the range) for the cmdlet parameter argument. This attribute can also be used by Windows PowerShell functions.

Syntax

C#

```
[ValidateRange(object minRange, object maxRange)]
```

Parameters

`MinRange` (System.Object) Required. Specifies the minimum value allowed.

`MaxRange` (System.Object) Required. Specifies the maximum value allowed.

Remarks

- The Windows PowerShell runtime throws a construction error when the value of the `MinRange` parameter is greater than the value of the `MaxRange` parameter.
- The Windows PowerShell runtime throws a validation error under the following conditions:
  - When the value of the argument is less than the `MinRange` limit or greater than the `MaxRange` limit.
  - When the argument is not of the same type as the `MinRange` and the `MaxRange` parameters.
- The ValidateRange attribute is defined by the System.Management.Automation.ValidateRangeAttribute class.

See Also

System.Management.Automation.ValidateRangeAttribute

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

ValidateScript Attribute Declaration

The `ValidateScript` attribute specifies a script that's used to validate a parameter or variable value. PowerShell pipes the value to the script, and generates an error if the script returns `$false` or if the script throws an exception.

When you use the `ValidateScript` attribute, the value that's being validated is mapped to the `$_` variable. You can use the `$_` variable to refer to the value in the script.

Syntax

C#

```
[ValidateScriptAttribute(ScriptBlock scriptBlock)]
[ValidateScriptAttribute(ScriptBlock scriptBlock, Named Parameters)]
```

Parameters

- `scriptBlock` - (System.Management.Automation.ScriptBlock) Required. The script block used to validate the input.
- `ErrorMessage` - Optional named parameter - The item being validated and the validating scriptblock are passed as the first and second formatting arguments.

> ⓘ Note
>
> The `ErrorMessage` argument was added in PowerShell 6.

Remarks

- This attribute can be used only once per parameter.
- If this attribute is applied to a collection, each element in the collection must match the pattern.
- The ValidateScript attribute is defined by the System.Management.Automation.ValidateScriptAttribute class.

See Also

System.Management.Automation.ValidateScriptAttribute

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

ValidateSet Attribute Declaration

The ValidateSetAttribute attribute specifies a set of possible values for a cmdlet parameter argument. This attribute can also be used by Windows PowerShell functions.

When this attribute is specified, the Windows PowerShell runtime determines whether the supplied argument for the cmdlet parameter matches an element in the supplied element set. The cmdlet is run only if the parameter argument matches an element in the set. If no match is found, an error is thrown by the Windows PowerShell runtime.

Syntax

C#

```
[ValidateSetAttribute(params string[] validValues)]
[ValidateSetAttribute(params string[] validValues, Named Parameters)]
```

Parameters

`ValidValues` (System.String) Required. Specifies the valid parameter element values. The following sample shows how to specify one element or multiple elements.

C#

```
[ValidateSetAttribute("Steve")]
[ValidateSetAttribute("Steve","Mary")]
```

`IgnoreCase` (System.Boolean) Optional named parameter. The default value of `true` indicates that case is ignored. A value of `false` makes the cmdlet case-sensitive.

Remarks

- This attribute can be used only once per parameter.
- If the parameter value is an array, every element of the array must match an element of the attribute set.
- The ValidateSetAttribute attribute is defined by the System.Management.Automation.ValidateSetAttribute class.

See Also

System.Management.Automation.ValidateSetAttribute

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Cmdlet Aliases

You can use cmdlet aliases to improve the cmdlet user experience. You can add aliases to frequently used cmdlets to reduce typing and to make it easier to complete tasks quickly. You can include built-in aliases in your cmdlets, or users can define their own custom aliases.

For example, the Get-Command cmdlet has a built-in `gcm` alias. You can also use aliases to add command names from other languages so that users do not have to learn new commands.

Alias Guidelines

Follow these guidelines when you create built-in aliases for your cmdlets:

- Before you assign aliases, start Windows PowerShell, and then run the Get-Alias cmdlet to see the aliases that are already used.
- Include an alias prefix that references the verb of the cmdlet name and an alias suffix that references the noun of the cmdlet name. For example, the alias for the `Import-Module` cmdlet is `ipmo`. For a list of all the verbs and their aliases, see Cmdlet Verbs.
- For cmdlets that have the same verb, include the same alias prefix. For example, the aliases for all the Windows PowerShell cmdlets that have the "Get" verb in their name use the "g" prefix.
- For cmdlets that have the same noun, include the same alias suffix. For example, the aliases for all the Windows PowerShell cmdlets that have the "Session" noun in their name use the "sn" suffix.
- For cmdlets that are equivalent to commands in other languages, use the name of the command.
- In general, make aliases as short as possible. Make sure the alias has at least one distinct character for the verb and one distinct character for the noun. Add more characters as needed to make the alias unique.
- For cmdlet written in C# (or any other compiled .NET language), the alias can be defined using the Alias attribute. For example:

  ```
  C#
  ```

```
[Cmdlet("Get", "SomeObject")]
[Alias("gso")]
public class GetSomeObjectCommand : Cmdlet
```

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Cmdlet Output

This section discusses the types of cmdlet output and the methods that cmdlets can call to generate output such as error messages and objects. This section also describes how to define the .NET Framework types that are returned by your cmdlets and how those objects are displayed.

In This Section

Types of Cmdlet Output Describes the types and output that cmdlets can generate and the methods that cmdlets call to generate the output.

Cmdlet Error Reporting Discusses cmdlet error reporting, a subset of cmdlet output.

Extending Output Objects Discusses how to use the types files (`.ps1xml`) to extend the .NET Framework objects that are returned by cmdlets, functions, and scripts.

PowerShell Formatting Files Describes the formatting files (`.format.ps1xml`) files that define the default display for a specific set of .NET Framework objects in Windows PowerShell.

Custom Formatting Files Describes how to create your own custom formatting files to overwrite the default display formats or to define the display of objects returned by your own commands.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Types of cmdlet output

PowerShell provides several methods that can be called by cmdlets to generate output. These methods use a specific operation to write their output to a specific data stream, such as the success data stream or the error data stream. This article describes the types of output and the methods used to generate them.

Types of output

Success output

Cmdlets can report success by returning an object that can be processed by the next command in the pipeline. After the cmdlet has successfully performed its action, the cmdlet calls the System.Management.Automation.Cmdlet.WriteObject method. We recommend that you call this method instead of the System.Console.WriteLine or System.Management.Automation.Host.PSHostUserInterface.WriteLine methods.

You can provide a **PassThru** `[switch]` parameter for cmdlets that do not typically return objects. When the **PassThru** `[switch]` parameter is specified at the command line, the cmdlet is asked to return an object. For an example of a cmdlet that has a **PassThru** parameter, see Add-History.

Error output

Cmdlets can report errors. When a terminating error occurs, the cmdlet throws an exception. When a non-terminating error occurs, the cmdlet calls the System.Management.Automation.Provider.CmdletProvider.WriteError method to send an error record to the error data stream. For more information about error reporting, see Error Reporting Concepts.

Verbose output

Cmdlets can provide useful information to you while the cmdlet is correctly processing records by calling the System.Management.Automation.Cmdlet.WriteVerbose method. The method generates verbose messages that indicate how the action is proceeding.

By default, verbose messages are not displayed. You can specify the **Verbose** parameter when the cmdlet is run to display these messages. **Verbose** is a common parameter that is available

to all cmdlets.

Progress output

Cmdlets can provide progress information to you when the cmdlet is performing tasks that take a long time to complete, such as copying a directory recursively. To display progress information the cmdlet calls the System.Management.Automation.Cmdlet.WriteProgress method.

Debug output

Cmdlets can provide debug messages that are helpful when troubleshooting the cmdlet code. To display debug information the cmdlet calls the System.Management.Automation.Cmdlet.WriteDebug method.

By default, debug messages are not displayed. You can specify the **Debug** parameter when the cmdlet is run to display these messages. **Debug** is a common parameter that is available to all cmdlets.

Warning output

Cmdlets can display warning messages by calling the System.Management.Automation.Cmdlet.WriteWarning method.

By default, warning messages are displayed. However, you can configure warning messages by using the `$WarningPreference` variable or by using the **Verbose** and **Debug** parameters when the cmdlet is called.

Displaying output

For all write-method calls, the content display is determined by specific runtime variables. The exception is the System.Management.Automation.Cmdlet.WriteObject method. By using these variables, you can make the appropriate write call at the correct place in your code and not worry about when or if the output should be displayed.

Accessing the output functionality of a host application

You can also design a cmdlet to directly access the output functionality of a host application through the PowerShell runtime. Using the host APIs provided by PowerShell instead of System.Console or System.Windows.Forms ensures that your cmdlet will work with a variety of hosts. For example: the `powershell.exe` console host, the `powershell_ise.exe` graphical host, the PowerShell remoting host, and third-party hosts.

See also

Error Reporting Concepts

Cmdlet Overview

Writing a Windows PowerShell Cmdlet

Last updated on 04/08/2026

Cmdlet error reporting

Cmdlets should report errors differently depending on whether the errors are terminating errors or non-terminating errors. Terminating errors are errors that cause the pipeline to be terminated immediately, or errors that occur when there's no reason to continue processing. Non-terminating errors are those errors that report a current error condition, but the cmdlet can continue to process input objects. With non-terminating errors, the user is typically notified of the problem, but the cmdlet continues to process the next input object.

Unless specified otherwise, all classes and methods mentioned in this document come from the System.Management.Automation namespace.

Terminating and non-terminating errors

The following guidelines can be used to determine if an error condition is a terminating error or a non-terminating error.

- Does the error condition prevent your cmdlet from successfully processing any further input objects? If so, this is a terminating error.
- Is the error condition related to a specific input object or a subset of input objects? If so, this is a non-terminating error.
- Does the cmdlet accept multiple input objects, such that processing may succeed on another input object? If so, this is a non-terminating error.
- Cmdlets that can accept multiple input objects should decide between what are terminating and non-terminating errors, even when a particular situation applies to only a single input object.
- Cmdlets can receive any number of input objects and send any number of success or error objects before throwing a terminating exception. There's no relationship between the number of input objects received and the number of success and error objects sent.
- Cmdlets that can accept only 0-1 input objects and generate only 0-1 output objects should treat errors as terminating errors and generate terminating exceptions.

Reporting non-terminating errors

The reporting of a non-terminating error should always be done within the cmdlet's implementation of the following methods:

- Cmdlet.BeginProcessing
- Cmdlet.ProcessRecord
- Cmdlet.EndProcessing

These types of errors are reported by calling the Cmdlet.WriteError method that in turn sends an error record to the error stream.

Reporting terminating errors

Terminating errors are reported by throwing exceptions or by calling the Cmdlet.ThrowTerminatingError method. Be aware that cmdlets can also catch and rethrow exceptions such as **OutOfMemory**, however, they aren't required to rethrow exceptions as the PowerShell runtime will catch them as well.

You can also define your own exceptions for issues specific to your situation, or add additional information to an existing exception using its error record.

Error records

PowerShell describes a non-terminating error condition with ErrorRecord objects. Each object provides error category information, an optional target object, and details about the error condition.

Error identifiers

The error identifier is a simple string that identifies the error condition within the cmdlet. PowerShell combines this identifier with a cmdlet identifier to create a fully qualified error identifier that can be used later when filtering error streams or logging errors, when responding to specific errors, or with other user-specific activities.

The following guidelines should be followed when specifying error identifiers:

- Assign different, highly specific, error identifiers to different code paths. Each code path that calls Cmdlet.WriteError or Cmdlet.ThrowTerminatingError should have its own error identifier.

- Error identifiers should be unique to Common Language Runtime (CLR) exception types for both terminating and non-terminating errors.

- Don't change the semantics of an error identifier between versions of your cmdlet or PowerShell provider. After the semantics of an error identifier is established, it should remain constant throughout the lifecycle of your cmdlet.
- For terminating errors, use a unique error identifier for a particular CLR exception type. If the exception type changes, use a new error identifier.
- For non-terminating errors, use a specific error identifier for a specific input object.
- Choose text for the identifier that tersely corresponds to the error being reported. Don't use white space or punctuation.
- Don't generate error identifiers that aren't reproducible. For example, don't generate identifiers that include a process identifier. Error identifiers are useful only when they correspond to identifiers that are seen by other users who are experiencing the same problem.

Error categories

Error categories are used to group errors for the user. PowerShell defines these categories and cmdlets and PowerShell providers must choose between them when generating the error record.

For a description of the error categories that are available, see the ErrorCategory enumeration. In general, you should avoid using **NoError**, **UndefinedError**, and **GenericError** whenever possible.

Users can view errors based on category when they set `$ErrorView` to **CategoryView**.

See also

- Cmdlet Overview
- Types of Cmdlet Output
- Windows PowerShell Reference

Last updated on 05/20/2025

Extending Output Objects

You can extend the .NET Framework objects that are returned by cmdlets, functions, and scripts by using types files (`.ps1xml`). Types files are XML-based files that let you add properties and methods to existing objects. For example, Windows PowerShell provides the Types.ps1xml file, which adds elements to several existing .NET Framework objects. The `Types.ps1xml` file is located in the Windows PowerShell installation directory (`$PSHOME`). You can create your own types file to further extend those objects or to extend other objects. When you extend an object by using a types file, any instance of the object is extended with the new elements.

Extending the System.Array Object

The following example shows how Windows PowerShell extends the System.Array object in the Types.ps1xml file. By default, System.Array objects have a `Length` property that lists the number of objects in the array. However, because the name "length" does not clearly describe the property, Windows PowerShell adds the `Count` alias property, which displays the same value as the `Length` property. The following XML adds the `Count` property to the System.Array type.

XML

```
<Type>
  <Name>System.Array</Name>
  <Members>
    <AliasProperty>
      <Name>Count</Name>
      <ReferencedMemberName>Length</ReferencedMemberName>
    </AliasProperty>
  </Members>
</Type>
```

To see this new alias property, use a Get-Member command on any array, as shown in the following example.

PowerShell

```
Get-Member -InputObject (1,2,3,4)
```

The command returns the following results.

Output

```
Name           MemberType    Definition
----           ----------    ----------
Count          AliasProperty Count = Length
Address        Method        System.Object& Address(Int32 )
Clone          Method        System.Object Clone()
CopyTo         Method        System.Void CopyTo(Array array, Int32 index):
Equals         Method        System.Boolean Equals(Object obj)
Get            Method        System.Object Get(Int32 )
...
Length         Property      System.Int32 Length {get;}
```

You can use either the `Count` property or the `Length` property to determine how many objects are in an array. For example:

PowerShell

```
PS> (1, 2, 3, 4).Count
```

Output

```
4
```

PowerShell

```
PS> (1, 2, 3, 4).Length
```

Output

```
4
```

Custom Types Files

To create a custom types file, start by copying an existing types file. The new file can have any name, but it must have a `.ps1xml` file name extension. When you copy the file, you can place the new file in any directory that is accessible to Windows PowerShell, but it is useful to place the files in the Windows PowerShell installation directory (`$PSHOME`) or in a subdirectory of the installation directory.

To add your own extended types to the file, add a types element for each object that you want to extend. The following topics provide examples.

- For more information about adding properties and property sets, see Extended Properties

- For more information about adding methods, see Extended Methods.
- For more information about adding member sets, see Extended Member Sets.

After you define your own extended types, use one of the following methods to make your extended objects available:

- To make your extended types file available to the current session, use the Update-TypeData cmdlet to add the new file. If you want your types to take precedence over the types that are defined in other types files (including the Types.ps1xml file), use the `PrependData` parameter of the Update-TypeData cmdlet.
- To make your extended types file available to all future sessions, add the types file to a module, export the current session, or add the Update-TypeData command to your Windows PowerShell profile.

Signing Types Files

Types files should be digitally signed to prevent tampering because the XML can include script blocks. For more information about adding digital signatures, see about_Signing

See Also

Defining Default Properties for Objects

Defining Default Methods for Objects

Defining Default Member Sets for Objects

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Extending Properties for Objects

When you extend .NET Framework objects, you can add alias properties, code properties, note properties, script properties, and property sets to the objects. The XML that defines these properties is described in the following sections.

> **Note**
>
> The examples in the following sections are from the default `Types.ps1xml` types file in the PowerShell installation directory (`$PSHOME`). For more information, see About Types.ps1xml.

Alias properties

An alias property defines a new name for an existing property.

In the following example, the **Count** property is added to the System.Array type. The AliasProperty element defines the extended property as an alias property. The Name element specifies the new name. And, the ReferencedMemberName element specifies the existing property that is referenced by the alias. You can also add the `AliasProperty` element to the members of the MemberSets element.

XML

```xml
<Type>
  <Name>System.Array</Name>
  <Members>
    <AliasProperty>
      <Name>Count</Name>
      <ReferencedMemberName>Length</ReferencedMemberName>
    </AliasProperty>
  </Members>
</Type>
```

Code properties

A code property references a static property of a .NET Framework object.

In the following example, the **Mode** property is added to the System.IO.DirectoryInfo type. The CodeProperty element defines the extended property as a code property. The Name element specifies the name of the extended property. And, the GetCodeReference element defines the static method that is referenced by the extended property. You can also add the `CodeProperty` element to the members of the MemberSets element.

XML

```xml
<Type>
  <Name>System.IO.DirectoryInfo</Name>
  <Members>
    <CodeProperty>
      <Name>Mode</Name>
      <GetCodeReference>
        <TypeName>Microsoft.PowerShell.Commands.FileSystemProvider</TypeName>
        <MethodName>Mode</MethodName>
      </GetCodeReference>
    </CodeProperty>
  </Members>
</Type>
```

Note properties

A note property defines a property that has a static value.

In the following example, the **Status** property, whose value is always **Success**, is added to the System.IO.DirectoryInfo type. The NoteProperty element defines the extended property as a note property. The Name element specifies the name of the extended property. The Value element specifies the static value of the extended property. The `NoteProperty` element can also be added to the members of the MemberSets element.

XML

```xml
<Type>
  <Name>System.IO.DirectoryInfo</Name>
  <Members>
    <NoteProperty>
      <Name>Status</Name>
      <Value>Success</Value>
    </NoteProperty>
  </Members>
</Type>
```

Script properties

A script property defines a property whose value is the output of a script.

In the following example, the **VersionInfo** property is added to the System.IO.FileInfo type. The ScriptProperty element defines the extended property as a script property. The Name element specifies the name of the extended property. And, the GetScriptBlock element specifies the script that generates the property value. You can also add the `ScriptProperty` element to the members of the MemberSets element.

XML

```xml
<Type>
  <Name>System.IO.FileInfo</Name>
  <Members>
    <ScriptProperty>
      <Name>VersionInfo</Name>
      <GetScriptBlock>
        [System.Diagnostics.FileVersionInfo]::GetVersionInfo($this.FullName)
      </GetScriptBlock>
    </ScriptProperty>
  </Members>
</Type>
```

Property sets

A property set defines a group of extended properties that can be referenced by the name of the set. For example, the Format-Table **Property** parameter can specify a specific property set to be displayed. When a property set is specified, only those properties that belong to the set are displayed.

There's no restriction on the number of property sets that can be defined for an object. However, the property sets used to define the default display properties of an object must be specified within the **PSStandardMembers** member set. In the `Types.ps1xml` types file, the default property set names include **DefaultDisplayProperty**, **DefaultDisplayPropertySet**, and **DefaultKeyPropertySet**. Any additional property sets that you add to the **PSStandardMembers** member set are ignored.

In the following example, the **DefaultDisplayPropertySet** property set is added to the **PSStandardMembers** member set of the System.ServiceProcess.ServiceController type. The PropertySet element defines the group of properties. The Name element specifies the name of the property set. And, the ReferencedProperties element specifies the properties of the set. You can also add the `PropertySet` element to the members of the Type element.

XML

```xml
<Type>
  <Name>System.ServiceProcess.ServiceController</Name>
  <Members>
    <MemberSet>
      <Name>PSStandardMembers</Name>
      <Members>
        <PropertySet>
           <Name>DefaultDisplayPropertySet</Name>
           <ReferencedProperties>
            <Name>Status</Name
            <Name>Name</Name>
            <Name>DisplayName</Name>
          </ReferencedProperties>
        </PropertySet>
      </Members>
    </MemberSet>
  </Members>
</Type>
```

See also

About Types.ps1xml

System.Management.Automation

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Defining Default Methods for Objects

When you extend .NET Framework objects, you can add code methods and script methods to the objects. The XML that is used to define these methods is described in the following sections.

> ⓘ **Note**
>
> The examples in the following sections are from the `Types.ps1xml` types file in the Windows PowerShell installation directory (`$PSHOME`). For more information, see About Types.ps1xml.

Code methods

A code method references a static method of a .NET Framework object.

In the following example, the **ToString** method is added to the System.Xml.XmlNode type. The PSCodeMethod element defines the extended method as a code method. The Name element specifies the name of the extended method. And, the CodeReference element specifies the static method. You can also add the PSCodeMethod element to the members of the PSMemberSets element.

XML

```
<Type>
  <Name>System.Xml.XmlNode</Name>
  <Members>
    <CodeMethod>
      <Name>ToString</Name>
      <CodeReference>
        <TypeName>Microsoft.PowerShell.ToStringCodeMethods</TypeName>
        <MethodName>XmlNode</MethodName>
      </CodeReference>
    </CodeMethod>
  </Members>
</Type>
```

Script methods

A script method defines a method whose value is the output of a script. In the following example, the **ConvertToDateTime** method is added to the System.Management.ManagementObject type. The PSScriptMethod element defines the extended method as a script method. The Name element specifies the name of the extended method. And, the Script element specifies the script that generates the method value. You can also add the PSScriptMethod element to the members of the PSMemberSets element.

XML

```
<Type>
  <Name>System.Management.ManagementObject</Name>
  <Members>
    <ScriptMethod>
      <Name>ConvertToDateTime</Name>
      <Script>
        [System.Management.ManagementDateTimeConverter]::ToDateTime($args[0])
      </Script>
    </ScriptMethod>
  </Members>
</Type>
```

See also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Defining Default Member Sets for Objects

The PSStandardMembers member set is used by Windows PowerShell to define the default property sets for an object. The default property sets can be used by commands such as the formatting cmdlets to display only those properties that are defined by the property set. The default property sets include DefaultDisplayProperty, DefaultDisplayPropertySet, and DefaultKeyPropertySet. Windows PowerShell ignores all other member sets and any other property sets added to the PSStandardMembers member set.

Member Set for System.Diagnostics.Process

In the following example, the PSStandardMembers member set defines the DefaultDisplayPropertySet property set for System.Diagnostics.Process objects. This property set is used by the Format-List cmdlet.

XML

```
<Type>
  <Name>System.Diagnostics.Process</Name>
  <Members>
    <MemberSet>
     <Name>PSStandardMembers</Name>
     <Members>
       <PropertySet>
         <Name>DefaultDisplayPropertySet</Name>
         <ReferencedProperties>
           <Name>Id</Name>
           <Name>Handles</Name>
           <Name>CPU</Name>
           <Name>Name</Name>
         </ReferencedProperties>
      </PropertySet>
    </Members>
  </MemberSet>
```

The following output shows the default properties returned by the Format-List cmdlet. Only the `Id`, `Handles`, `CPU`, and `Name` properties are returned for each process object.

PowerShell

```
Get-Process | Format-List
```

Output

```
Id      : 2036
Handles : 27
CPU     :
Name    : AEADISRV

Id      : 272
Handles : 38
CPU     :
Name    : agrsmsvc
...
```

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Custom Formatting Files

The display format for the objects returned by cmdlets, functions, and scripts are defined using formatting files (`format.ps1xml` files). Several of these files are provided by Windows PowerShell to define the default display format for those objects returned by Windows PowerShell cmdlets. However, you can also create your own custom formatting files to overwrite the default display formats or to define the display of objects returned by your own commands.

Windows PowerShell uses the data in these formatting files to determine what is displayed and how the data is formatted. The displayed data can include the properties of an object or the value of a script block. Script blocks are used if you want to display some value that is not available directly from the properties of an object. For example, you may want to add the value of two properties of an object and display the sum as a separate piece of data. When you write your own formatting file, you will need to define *views* for the objects that you want to display. You can define a single view for each object, you can define a single view for multiple objects, or you can define multiple views for the same object. There is no limit to the number of views that you can define.

> ⓘ **Important**
>
> Formatting files do not determine the elements of an object that are returned to the pipeline. When an object is returned to the pipeline, all members of that object are available.

Format Views

Formatting views can display objects in a table format, a list format, a wide format, and a custom format. For the most part, each formatting definition is described by a set of XML tags that describe a view. Each view contains the name of the view, the objects that use the view, and the elements of the view, such as the column and row information for a table view.

The following views are available.

Table view Lists the properties of an object or a script block value in one or more columns. Each column represents a property of the object or a script block value. You can define a table view that displays all the properties of an object, a subset of the properties of an object, or a

combination of properties and script block values. Each row of the table represents a returned object. For more information about this view, see Table View.

List view Lists the properties of an object or a script block value in a single column. Each row of the list displays an optional label or the property name followed by the value of the property or script block. For more information about this view, see List View.

Wide view Lists a single property of an object or a script block value in one or more columns. There is no label or header for this view. For more information about this view, see Wide View.

Custom view Displays a customizable view of object properties or script block values that does not adhere to the rigid structure of table views, list views, or wide views. You can define a standalone custom view, or you can define a custom view that is used by another view, such as a table view or list view. For more information about this view, see Custom View.

View XML Elements

The following example shows the XML tags used to define a table view that contains two columns. The ViewDefinitions element is the container element for all the views defined in the formatting file. The View element defines the specific table, list, wide, or custom view. Within each view, the Name element specifies the name of the view, the ViewSelectedBy element defines the objects that use the view, and the different control elements (such as the `TableControl` element) define the format of the view.

XML

```
ViewDefinitions
  <View>
    <Name>Name of View</Name>
    <ViewSelectedBy>
      <TypeName>Object to display using this view</TypeName>
      <TypeName>Object to display using this view</TypeName>
    </ViewSelectedBy>
    <TableControl>
      <TableHeaders>
        <TableColumnHeader>
          <Width></Width>
        </TableColumnHeader>
        <TableColumnHeader>
          <Width></Width>
        </TableColumnHeader>
      </TableHeaders>
      <TableRowEntries>
        <TableRowEntry>
          <TableColumnItems>
            <TableColumnItem>
```

```
              <PropertyName>Header for column 1</PropertyName>
            </TableColumnItem>
            <TableColumnItem>
              <PropertyName>Header for column 2</PropertyName>
            </TableColumnItem>
          </TableColumnItems>
        </TableRowEntry>
      </TableRowEntries>
    </TableControl)
  </View>
</ViewDefinitions>
```

See Also

Table View

List View

Wide View

Custom View

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Requesting Confirmation

Summarize this article for me

This section discusses confirmation messages that can be displayed before a cmdlet, function, or provider performs an action.

In This Section

Requesting Confirmation Process for Commands Discusses the process that cmdlets, functions, and providers must follow to request a confirmation before they make a change to the system.

Users Requesting Confirmation Discusses how users can make a cmdlet, function, or provider request confirmation when the System.Management.Automation.Cmdlet.ShouldProcess method is called.

Confirmation Messages Provides samples of the different confirmation messages that can be displayed.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 02/24/2026

Requesting Confirmation from Cmdlets

Cmdlets should request confirmation when they make a change to the system that's outside of the Windows PowerShell environment. For example, if a cmdlet is about to add a user account or stop a process, the cmdlet should require confirmation from the user before it proceeds. In contrast, if a cmdlet is about to change a Windows PowerShell variable, the cmdlet doesn't need to require confirmation.

In order to make a confirmation request, the cmdlet must indicate that it supports confirmation requests, and it must call the System.Management.Automation.Cmdlet.ShouldProcess and System.Management.Automation.Cmdlet.ShouldContinue (optional) methods to display a confirmation request message.

Supporting Confirmation Requests

To support confirmation requests, the cmdlet must set the `SupportsShouldProcess` parameter of the Cmdlet attribute to `true`. This enables the `Confirm` and `WhatIf` cmdlet parameters that are provided by Windows PowerShell. The `Confirm` parameter allows the user to control whether the confirmation request is displayed. The `WhatIf` parameter allows the user to determine whether the cmdlet should display a message or perform its action. Don't manually add the `Confirm` and `WhatIf` parameters to a cmdlet.

The following example shows a Cmdlet attribute declaration that supports confirmation requests.

C#

```
[Cmdlet(VerbsDiagnostic.Test, "RequestConfirmationTemplate1",
        SupportsShouldProcess = true)]
```

Calling the Confirmation request methods

In the cmdlet code, call the System.Management.Automation.Cmdlet.ShouldProcess method before the operation that changes the system is performed. Design the cmdlet so that if the call returns a value of `false`, the operation isn't performed, and the cmdlet processes the next operation.

Calling the ShouldContinue Method

Most cmdlets request confirmation using only the System.Management.Automation.Cmdlet.ShouldProcess method. However, some cases might require additional confirmation. For these cases, supplement the System.Management.Automation.Cmdlet.ShouldProcess call with a call to the System.Management.Automation.Cmdlet.ShouldContinue method. This allows the cmdlet or provider to more finely control the scope of the **Yes to all** response to the confirmation prompt.

If a cmdlet calls the System.Management.Automation.Cmdlet.ShouldContinue method, the cmdlet must also provide a `Force` `[switch]` parameter. If the user specifies `Force` when the user invokes the cmdlet, the cmdlet should still call System.Management.Automation.Cmdlet.ShouldProcess, but it should bypass the call to System.Management.Automation.Cmdlet.ShouldContinue.

System.Management.Automation.Cmdlet.ShouldContinue will throw an exception when it's called from a non-interactive environment where the user can't be prompted. Adding a `Force` parameter ensures that the command can still be performed when it's invoked in a non-interactive environment.

The following example shows how to call System.Management.Automation.Cmdlet.ShouldProcess and System.Management.Automation.Cmdlet.ShouldContinue.

C#

```
if (ShouldProcess (...) )
{
  if (Force || ShouldContinue(...))
  {
     // Add code that performs the operation.
  }
}
```

The behavior of a System.Management.Automation.Cmdlet.ShouldProcess call can vary depending on the environment in which the cmdlet is invoked. Using the previous guidelines will help ensure that the cmdlet behaves consistently with other cmdlets, regardless of the host environment.

Specify the Impact Level

When you create the cmdlet, specify the impact level (the severity) of the change. To do this, set the value of the `ConfirmImpact` parameter of the Cmdlet attribute to High, Medium, or Low. You can specify a value for `ConfirmImpact` only when you also specify the `SupportsShouldProcess` parameter for the cmdlet.

For most cmdlets, you don't have to explicitly specify `ConfirmImpact`. Instead, use the default setting of the parameter, which is Medium. If you set `ConfirmImpact` to High, the operation will be confirmed by default. Reserve this setting for highly disruptive actions, such as reformatting a hard-disk volume.

Calling Non-Confirmation Methods

If the cmdlet or provider must send a message but not request confirmation, it can call the following three methods. Avoid using the System.Management.Automation.Cmdlet.WriteObject method to send messages of these types because System.Management.Automation.Cmdlet.WriteObject output is intermingled with the normal output of your cmdlet or provider, which makes script writing difficult.

- To caution the user and continue with the operation, the cmdlet or provider can call the System.Management.Automation.Cmdlet.WriteWarning method.
- To provide additional information that the user can retrieve using the `Verbose` parameter, the cmdlet or provider can call the System.Management.Automation.Cmdlet.WriteVerbose method.
- To provide debugging-level detail for other developers or for product support, the cmdlet or provider can call the System.Management.Automation.Cmdlet.WriteDebug method. The user can retrieve this information using the `Debug` parameter.

Cmdlets and providers first call the following methods to request confirmation before they attempt to perform an operation that changes a system outside of Windows PowerShell:

- System.Management.Automation.Cmdlet.ShouldProcess
- System.Management.Automation.Provider.CmdletProvider.ShouldProcess

They do so by calling the System.Management.Automation.Cmdlet.ShouldProcess method, which prompts the user to confirm the operation based on how the user invoked the command.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 04/08/2026

Users Requesting Confirmation

When you specify a value of `true` for the `SupportsShouldProcess` parameter of the Cmdlet attribute declaration, the **Confirm** parameter is added to the parameters of the cmdlet.

In the default environment, users can specify the **Confirm** parameter so that confirmation is requested when the `ShouldProcess()` method is called. This forces confirmation regardless of the impact level setting.

If **Confirm** parameter is not used, the `ShouldProcess()` call requests confirmation if the `ConfirmImpact` setting is equal to or greater than the `$ConfirmPreference` preference variable. The default setting of `$ConfirmPreference` is **High**. Therefore, in the default environment, only cmdlets and providers that specify a high-impact action request confirmation.

If **Confirm** is explicitly set to false (`-Confirm:$false`), the cmdlet runs without prompting for confirmation and the `$ConfirmPreference` shell variable is ignored.

Remarks

- For cmdlets and providers that specify `SupportsShouldProcess`, but not `ConfirmImpact`, those actions are handled as `Medium` impact actions, and they will not prompt by default. Their impact level is less than the default setting of the `$ConfirmPreference` preference variable.
- If the user specifies the `Verbose` parameter, they will be notified of the operation even if they are not prompted for confirmation.

See Also

- Writing a Windows PowerShell Cmdlet
- System.Management.Automation.Cmdlet.ShouldProcess

Last updated on 05/20/2025

Confirmation Messages

Summarize this article for me

Here are different confirmation messages that can be displayed depending on the variants of the System.Management.Automation.Cmdlet.ShouldProcess and System.Management.Automation.Cmdlet.ShouldContinue methods that are called.

> **Important**
>
> For sample code that shows how to request confirmations, see Requesting Confirmation from Cmdlets.

Specifying the Resource

You can specify the resource that is about to be changed by calling the System.Management.Automation.Cmdlet.ShouldProcess method. In this case, you supply the resource by using the `target` parameter of the method, and the operation is added by Windows PowerShell. In the following message, the text "MyResource" is the resource acted on and the operation is the name of the command that makes the call.

Output

```
Confirm
Are you sure you want to perform this action?
Performing operation "Test-RequestConfirmationTemplate1" on Target "MyResource".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is
"Y"):
```

If the user selects **Yes** or **Yes to All** to the confirmation request (as shown in the following example), a call to the System.Management.Automation.Cmdlet.ShouldContinue method is made, which causes a second confirmation message to be displayed.

Output

```
Confirm
Are you sure you want to perform this action?
Performing operation "Test-RequestConfirmationTemplate1" on Target "MyResource".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is
"Y"): y

Confirm
Continue with this operation?
[Y] Yes  [N] No  [S] Suspend  [?] Help (default is "Y"):
```

Specifying the Operation and Resource

You can specify the resource that is about to be changed and the operation that the command is about to perform by calling the System.Management.Automation.Cmdlet.ShouldProcess method. In this case, you supply the resource by using the `target` parameter and the operation by using the `target` parameter. In the following message, the text "MyResource" is the resource acted on and "MyAction" is the operation to be performed.

Output

```
Confirm
Are you sure you want to perform this action?
Performing operation "MyAction" on Target "MyResource".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is
"Y"):
```

If the user selects **Yes** or **Yes to All** to the previous message, a call to the System.Management.Automation.Cmdlet.ShouldContinue method is made, which causes a second confirmation message to be displayed.

Output

```
Confirm
Are you sure you want to perform this action?
Performing operation "MyAction" on Target "MyResource".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is
"Y"): y

Confirm
Continue with this operation?
[Y] Yes  [N] No  [S] Suspend  [?] Help (default is "Y"):
```

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 02/24/2026

Windows PowerShell Error Reporting

The topics in this section discuss how cmdlets report errors.

In This Section

Error Reporting Concepts Describes the two mechanisms that cmdlets can use to report errors.

Terminating Errors Describes the method used to report terminating errors, where that method can be called from within the cmdlet, and exceptions that can be returned by the Windows PowerShell runtime when the method is called.

Non-Terminating Errors Describes the method used to report non-terminating errors and where that method can be called from within the cmdlet.

Displaying Error Information by Category Discusses the ways that users can display error.

Windows PowerShell Error Records Describes the components of an error record.

Interpreting ErrorRecord Objects Discusses how ErrorRecord objects are interpreted.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Error Reporting Concepts

Windows PowerShell provides two mechanisms for reporting errors: one mechanism for *terminating errors* and another mechanism for *non-terminating errors*. It is important for your cmdlet to report errors correctly so that the host application that is running your cmdlets can react in an appropriate manner.

Your cmdlet should call the System.Management.Automation.Cmdlet.ThrowTerminatingError* method when an error occurs that does not or should not allow the cmdlet to continue to process its input objects. Your cmdlet should call the System.Management.Automation.Cmdlet.WriteError method to report non-terminating errors when the cmdlet can continue processing the input objects. Both methods provide an error record that the host application can use to investigate the cause of the error.

Use the following guidelines to determine whether an error is a terminating or non-terminating error.

- An error is a terminating error if it prevents your cmdlet from continuing to process the current object or from successfully processing any further input objects, regardless of their content.
- An error is a terminating error if you do not want your cmdlet to continue processing the current object or any further input objects, regardless of their content.
- An error is a terminating error if it occurs in a cmdlet that does not accept or return an object or if it occurs in a cmdlet that accepts or returns only one object.
- An error is a non-terminating error if you want your cmdlet to continue processing the current object and any further input objects.
- An error is a non-terminating error if it is related to a specific input object or subset of input objects.

See Also

System.Management.Automation.Cmdlet.ThrowTerminatingError*

System.Management.Automation.Cmdlet.WriteError

Windows PowerShell Error Records

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Terminating Errors

This topic discusses the method used to report terminating errors. It also discusses how to call the method from within the cmdlet, and it discusses the exceptions that can be returned by the Windows PowerShell runtime when the method is called.

When a terminating error occurs, the cmdlet should report the error by calling the System.Management.Automation.Cmdlet.ThrowTerminatingError* method. This method allows the cmdlet to send an error record that describes the condition that caused the terminating error. For more information about error records, see Windows PowerShell Error Records.

When the System.Management.Automation.Cmdlet.ThrowTerminatingError* method is called, the Windows PowerShell runtime permanently stops the execution of the pipeline and throws a System.Management.Automation.PipelineStoppedException exception. Any subsequent attempts to call System.Management.Automation.Cmdlet.WriteObject, System.Management.Automation.Cmdlet.WriteError, or several other APIs causes those calls to throw a System.Management.Automation.PipelineStoppedException exception.

The System.Management.Automation.PipelineStoppedException exception can also occur if another cmdlet in the pipeline reports a terminating error, if the user has asked to stop the pipeline, or if the pipeline has been halted before completion for any reason. The cmdlet does not need to catch the System.Management.Automation.PipelineStoppedException exception unless it must clean up open resources or its internal state.

Cmdlets can write any number of output objects or non-terminating errors before reporting a terminating error. However, the terminating error permanently stops the pipeline, and no further output, terminating errors, or non-terminating errors can be reported.

Cmdlets can call System.Management.Automation.Cmdlet.ThrowTerminatingError* only from the thread that called the System.Management.Automation.Cmdlet.BeginProcessing, System.Management.Automation.Cmdlet.ProcessRecord, or System.Management.Automation.Cmdlet.EndProcessing input processing method. Do not attempt to call System.Management.Automation.Cmdlet.ThrowTerminatingError* or System.Management.Automation.Cmdlet.WriteError from another thread. Instead, errors must be communicated back to the main thread.

It is possible for a cmdlet to throw an exception in its implementation of the System.Management.Automation.Cmdlet.BeginProcessing,

System.Management.Automation.Cmdlet.ProcessRecord, or System.Management.Automation.Cmdlet.EndProcessing method. Any exception thrown from these methods (except for a few severe error conditions that stop the Windows PowerShell host) is interpreted as a terminating error which stops the pipeline, but not Windows PowerShell as a whole. (This applies only to the main cmdlet thread. Uncaught exceptions in threads spawned by the cmdlet, in general, halt the Windows PowerShell host.) We recommend that you use System.Management.Automation.Cmdlet.ThrowTerminatingError* rather than throwing an exception because the error record provides additional information about the error condition, which is useful to the end-user. Cmdlets should honor the managed code guideline against catching and handling all exceptions (`catch (Exception e)`). Convert only exceptions of known and expected types into error records.

See Also

System.Management.Automation.Cmdlet.BeginProcessing

System.Management.Automation.Cmdlet.EndProcessing

System.Management.Automation.Cmdlet.ProcessRecord

System.Management.Automation.PipelineStoppedException

System.Management.Automation.Cmdlet.ThrowTerminatingError*

System.Management.Automation.Cmdlet.WriteError

Windows PowerShell Error Records

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Non-Terminating Errors

This topic discusses the method used to report non-terminating errors. It also discusses how to call the method from within the cmdlet.

When a non-terminating error occurs, the cmdlet should report this error by calling the System.Management.Automation.Cmdlet.WriteError method. When the cmdlet reports a non-terminating error, the cmdlet can continue to operate on this input object and on further incoming pipeline objects. If the cmdlet calls the System.Management.Automation.Cmdlet.WriteError method, the cmdlet can write an error record that describes the condition that caused the non-terminating error. For more information about error records, see Windows PowerShell Error Records.

Cmdlets can call System.Management.Automation.Cmdlet.WriteError as necessary from within their input processing methods. However, cmdlets can call System.Management.Automation.Cmdlet.WriteError only from the thread that called the System.Management.Automation.Cmdlet.BeginProcessing, System.Management.Automation.Cmdlet.ProcessRecord, or System.Management.Automation.Cmdlet.EndProcessing input processing method. Do not call System.Management.Automation.Cmdlet.WriteError from another thread. Instead, communicate errors back to the main thread.

See Also

System.Management.Automation.Cmdlet.WriteError

System.Management.Automation.Cmdlet.BeginProcessing

System.Management.Automation.Cmdlet.ProcessRecord

System.Management.Automation.Cmdlet.EndProcessing

Windows PowerShell Error Records

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Displaying Error Information

This topic discusses the ways in which users can display error information.

When your cmdlet encounters an error, the presentation of the error information will, by default, resemble the following error output.

PowerShell

```
$ Stop-Service lanmanworkstation
You do not have sufficient permissions to stop the service Workstation.
```

However, users can view errors by category by setting the `$ErrorView` variable to `"CategoryView"`. Category view displays specific information from the error record rather than a free-text description of the error. This view can be useful if you have a long list of errors to scan. In category view, the previous error message is displayed as follows.

PowerShell

```
$ $ErrorView = "CategoryView"
$ Stop-Service lanmanworkstation
CloseError: (System.ServiceProcess.ServiceController:ServiceController) [Stop-
Service], ServiceCommandException
```

For more information about error categories, see Windows PowerShell Error Records.

See Also

Windows PowerShell Error Records

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Windows PowerShell Error Records

Cmdlets must pass an System.Management.Automation.ErrorRecord object that identifies the error condition for terminating and non-terminating errors.

The System.Management.Automation.ErrorRecord object contains the following information:

- The exception that describes the error. Often, this is an exception that the cmdlet caught and converted into an error record. Every error record must contain an exception.

If the cmdlet did not catch an exception, it must create a new exception and choose the exception class that best describes the error condition. However, you do not need to throw the exception because it can be accessed through the System.Management.Automation.ErrorRecord.Exception property of the System.Management.Automation.ErrorRecord object.

- An error identifier that provides a targeted designator that can be used for diagnostic purposes and by Windows PowerShell scripts to handle specific error conditions with specific error handlers. Every error record must contain an error identifier (see Error Identifier).

- An error category that provides a general designator that can be used for diagnostic purposes. Every error record must specify an error category (see Error Category).

- An optional replacement error message and a recommended action (see Replacement Error Message).

- Optional invocation information about the cmdlet that threw the error. This information is specified by Windows PowerShell (see Invocation Message).

- The target object that was being processed when the error occurred. This might be the input object, or it might be another object that your cmdlet was processing. For example, for the command `Remove-Item -Recurse C:\somedirectory`, the error might be an instance of a FileInfo object for "C:\somedirectory\lockedfile". The target object information is optional.

Error Identifier

When you create an error record, specify an identifier that designates the error condition within your cmdlet. Windows PowerShell combines the targeted identifier with the name of your cmdlet to create a fully qualified error identifier. The fully qualified error identifier can be accessed through the System.Management.Automation.ErrorRecord.FullyQualifiedErrorId property of the System.Management.Automation.ErrorRecord object. The error identifier is not available by itself. It is available only as part of the fully qualified error identifier.

Use the following guidelines to generate error identifiers when you create error records:

- Make error identifiers specific to an error condition. Target the error identifiers for diagnostic purposes and for scripts that handle specific error conditions with specific error handlers. A user should be able to use the error identifier to identify the error and its source. Error identifiers also enable reporting for specific error conditions from existing exceptions so that new exception subclasses are not required.

- In general, assign different error identifiers to different code paths. The end-user benefits from specific identifiers. Often, each code path that calls System.Management.Automation.Cmdlet.WriteError or System.Management.Automation.Cmdlet.ThrowTerminatingError* has its own identifier. As a rule, define a new identifier when you define a new template string for the error message, and vice-versa. Do not use the error message as an identifier.

- When you publish code using a particular error identifier, you establish the semantics of errors with that identifier for your complete product support lifecycle. Do not reuse it in a context that is semantically different from the original context. If the semantics of this error change, create and then use a new identifier.

- You should generally use a particular error identifier only for exceptions of a particular CLR type. If the type of the exception or the type of the target object changes, create and then use a new identifier.

- Choose text for your error identifier that concisely corresponds to the error that you are reporting. Use standard .NET Framework naming and capitalization conventions. Do not use white space or punctuation. Do not localize error identifiers.

- Do not dynamically generate error identifiers in a non-reproducible way. For example, do not incorporate error information such as a process ID. Error identifiers are useful only if they correspond to the error identifiers seen by other users who are experiencing the same error condition.

Error Category

When you create an error record, specify the category of the error using one of the constants defined by the System.Management.Automation.ErrorCategory enumeration. Windows PowerShell uses the error category to display error information when users set the `$ErrorView` variable to `"CategoryView"`.

Avoid using the System.Management.Automation.ErrorCategory **NotSpecified** constant. If you have any information about the error or about the operation that caused the error, choose the category that best describes the error or the operation, even if the category is not a perfect match.

The information displayed by Windows PowerShell is referred to as the category-view string and is built from the properties of the System.Management.Automation.ErrorCategoryInfo class. (This class is accessed through the error System.Management.Automation.ErrorRecord.CategoryInfo property.)

```
{Category}: ({TargetName}:{TargetType}):[{Activity}], {Reason}
```

The following list describes the information displayed:

- Category: A Windows PowerShell-defined System.Management.Automation.ErrorCategory constant.
- TargetName: By default, the name of the object the cmdlet was processing when the error occurred. Or, another cmdlet-defined string.
- TargetType: By default, the type of the target object. Or, another cmdlet-defined string.
- Activity: By default, the name of the cmdlet that created the error record. Or, some other cmdlet-defined string.
- Reason: By default, the exception type. Or, another cmdlet-defined string.

Replacement Error Message

When you develop an error record for a cmdlet, the default error message for the error comes from the default message text in the System.Exception.Message property. This is a read-only property whose message text is intended only for debugging purposes (according to the .NET

Framework guidelines). We recommend that you create an error message that replaces or augments the default message text. Make the message more user-friendly and more specific to the cmdlet.

The replacement message is provided by an System.Management.Automation.ErrorDetails object. Use one of the following constructors of this object because they provide additional localization information that can be used by Windows PowerShell.

- ErrorDetails(Cmdlet, String, String, Object[]): Use this constructor if your template string is a resource string in the same assembly in which the cmdlet is implemented or if you want to load the template string through an override of the System.Management.Automation.Cmdlet.GetResourceString method.
- ErrorDetails(Assembly, String, String, Object[]): Use this constructor if the template string is in another assembly and you do not load it through an override of System.Management.Automation.Cmdlet.GetResourceString.

The replacement message should conform to the .NET Framework design guidelines for writing exception messages with a small difference. The guidelines state that exception messages should be written for developers. These replacement messages should be written for the cmdlet user.

The replacement error message must be added before the System.Management.Automation.Cmdlet.WriteError or System.Management.Automation.Cmdlet.ThrowTerminatingError* methods are called. To add a replacement message, set the System.Management.Automation.ErrorRecord.ErrorDetails property of the error record. When this property is set, Windows PowerShell displays the System.Management.Automation.ErrorDetails.Message* property instead of the default message text.

Recommended Action Information

The System.Management.Automation.ErrorDetails object can also provide information about what actions are recommended when the error occurs.

Invocation information

When a cmdlet uses System.Management.Automation.Cmdlet.WriteError or System.Management.Automation.Cmdlet.ThrowTerminatingError* to report an error record, Windows PowerShell automatically adds information that describes the command that was

invoked when the error occurred. This information is provided by a System.Management.Automation.InvocationInfo object that contains the name of the cmdlet that was invoked by the command, the command itself, and information about the pipeline or script. This property is read-only.

See Also

System.Management.Automation.Cmdlet.WriteError

System.Management.Automation.Cmdlet.ThrowTerminatingError*

System.Management.Automation.ErrorCategory

System.Management.Automation.ErrorCategoryInfo

System.Management.Automation.ErrorRecord

System.Management.Automation.ErrorDetails

System.Management.Automation.InvocationInfo

Windows PowerShell Error Reporting

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Interpreting ErrorRecord Objects

In most cases, an System.Management.Automation.ErrorRecord object represents a non-terminating error generated by a command or script. Terminating errors can also specify the additional information in an ErrorRecord via the System.Management.Automation.IContainsErrorRecord interface.

If you want to write an error handler in your script or a host to handle specific errors that occur during command or script execution, you must interpret the System.Management.Automation.ErrorRecord object to determine whether it represents the class of error that you want to handle.

When a cmdlet encounters a terminating or non-terminating error, it should create an error record that describes the error condition. The host application must investigate these error records and perform whatever action will mitigate the error. The host application must also investigate error records for non-terminating errors that failed to process a record but were able to continue, and it must investigate error records for terminating errors that caused the pipeline to stop.

> **Note**
>
> For terminating errors, the cmdlet calls the System.Management.Automation.Cmdlet.ThrowTerminatingError method. For non-terminating errors, the cmdlet calls the System.Management.Automation.Cmdlet.WriteError method.

Error Record Design

Error records are designed to provide additional error information that is not available in exceptions while ensuring that the combined information in each error record is unique. This uniqueness allows the host application to inspect the different parts of the error record so that it can identify the error condition and the action the host must take.

Interpreting Error Records

You can review several parts of the error record to identify the error. These parts include the following:

- The error category
- The error exception
- The fully qualified error identifier (FQID)
- Other information

The Error Category

The error category of the error record is one of the predefined constants provided by the System.Management.Automation.ErrorCategory enumeration. This information is available through the System.Management.Automation.ErrorRecord.CategoryInfo property of the System.Management.Automation.ErrorRecord object.

The cmdlet can specify the CloseError, OpenError, InvalidType, ReadError, and WriteError categories, and other error categories. The host application can use the error category to capture groups of errors.

The Exception

The exception included in the error record is provided by the cmdlet and can be accessed through the System.Management.Automation.ErrorRecord.Exception property of the System.Management.Automation.ErrorRecord object.

Host applications can use the `is` keyword to identify that the exception is of a specific class or of a derived class. It is better to branch on the exception type, as shown in the following example.

PowerShell

```
`if (MyNonTerminatingError.Exception is AccessDeniedException)`
{
  ...
}
```

This way, you catch the derived classes. However, there are problems if the exception is deserialized.

The FQID

The FQID is the most specific information you can use to identify the error. It is a string that includes a cmdlet-defined identifier, the name of the cmdlet class, and the source that reported the error. In general, an error record is analogous to an event record of a Windows Event log. The FQID is analogous to the following tuple, which identifies the class of the event record: (*log name*, *source*, *event ID*).

The FQID is designed to be inspected as a single string. However, cases exist in which the error identifier is designed to be parsed by the host application. The following example is a well-formed fully qualified error identifier.

```
CommandNotFoundException,Microsoft.PowerShell.Commands.GetCommandCommand.
```

In the previous example, the first token is the error identifier, which is followed by the name of the cmdlet class. The error identifier can be a single token, or it can be a dot-separated identifier that allows for branching on inspection of the identifier. Do not use white space or punctuation in the error identifier. It is especially important not to use a comma; a comma is used by Windows PowerShell to separate the identifier and the cmdlet class name.

Other Information

The System.Management.Automation.ErrorRecord object might also provide information that describes the environment in which the error occurred. This information includes items such as error details, invocation information, and the target object that was being processed when the error occurred. Although this information might be useful to the host application, it is not typically used to identify the error. This information is available through the following properties:

- System.Management.Automation.ErrorRecord.ErrorDetails
- System.Management.Automation.ErrorRecord.InvocationInfo
- System.Management.Automation.ErrorRecord.TargetObject

See Also

- System.Management.Automation.ErrorRecord
- System.Management.Automation.ErrorCategory
- System.Management.Automation.ErrorCategoryinfo
- System.Management.Automation.Cmdlet.WriteError

- System.Management.Automation.Cmdlet.ThrowTerminatingError*
- Adding Non-Terminating Error Reporting to Your Cmdlet
- Windows PowerShell Error Reporting
- Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Background Jobs

Cmdlets can perform their action internally or as a Windows PowerShell*background job*. When a cmdlet runs as a background job, the work is done asynchronously in its own thread separate from the pipeline thread that the cmdlet is using. From the user perspective, when a cmdlet runs as a background job, the command prompt returns immediately even if the job takes an extended amount of time to complete, and the user can continue without interruption while the job runs.

Background Jobs, Child Jobs, and the Job Repository

The job object that is returned by the cmdlets that support background jobs defines the job. (The Start-Job cmdlet also returns a job object.) The name of the job, an identifier that is used to specify the job, the state information, and the child jobs are included in this definition. The job does not perform any of the work. Each background job has at least one child job because the child job performs the actual work. When you run a cmdlet so that the work is performed as a background job, the cmdlet must add the job and the child jobs to a common repository, referred to as the *job repository*.

For more information about how background jobs are handled at the command line, see the following:

- about_Jobs
- about_Job_Details
- about_Remote_Jobs

Writing a Cmdlet That Runs as a Background Job

To write a cmdlet that can be run as a background job, you must complete the following tasks:

- Define an `AsJob` `[switch]` parameter so that the user can decide whether to run the cmdlet as a background job.
- Create an object that derives from the System.Management.Automation.Job class. This object can be a custom job object or a job object provided by Windows PowerShell, such

as a System.Management.Automation.PSEventJob object.

- In a record processing method, add an `if` statement that detects whether the cmdlet should run as a background job.
- For custom job objects, implement the job class.
- Return the appropriate objects, depending on whether the cmdlet is run as a background job.

For a code example, see How to Support Jobs.

Background Job-Related APIs

The following APIs are provided by Windows PowerShell to manage background jobs.

System.Management.Automation.Job Derives custom job objects. This is an abstract class.

System.Management.Automation.JobRepository Manages and provides information about the current active background jobs.

System.Management.Automation.JobState Defines the state of the background job. States include Started, Running, and Stopped.

System.Management.Automation.JobStateInfo Provides information about the state of a background job and, if the last state change was caused by an error, the reason the job entered its current state.

System.Management.Automation.JobStateEventArgs Provides the arguments for an event that is raised when a background job changes state.

Windows PowerShell Job Cmdlets

The following cmdlets are provided by Windows PowerShell to manage background jobs.

Get-Job

Gets Windows PowerShell background jobs that are running in the current session.

Receive-Job

Gets the results of the Windows PowerShell background jobs in the current session.

Remove-Job

Deletes a Windows PowerShell background job.

Start-Job

Starts a Windows PowerShell background job.

Stop-Job

Stops a Windows PowerShell background job.

Wait-Job

Suppresses the command prompt until one or all of the Windows PowerShell background jobs running in the session are complete.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 04/08/2026

Invoking Cmdlets and Scripts Within a Cmdlet

A cmdlet can invoke other cmdlets and scripts from within the input processing method of the cmdlet. This allows you to add the functionality of existing cmdlets and scripts to your cmdlet without having to rewrite the code.

The Invoke Method

All cmdlets can invoke an existing cmdlet by calling the System.Management.Automation.Cmdlet.Invoke method from within an input processing method, such as System.Management.Automation.Cmdlet.BeginProcessing, that is overridden by the cmdlet. However, you can invoke only those cmdlets that derive directly from the System.Management.Automation.Cmdlet class. You cannot invoke a cmdlet that derives from the System.Management.Automation.PSCmdlet class.

The System.Management.Automation.Cmdlet.Invoke* method has the following variants.

System.Management.Automation.Cmdlet.Invoke This variant invokes the cmdlet object and returns a collection of "T" type objects.

System.Management.Automation.Cmdlet.Invoke This variant invokes the cmdlet object and returns a strongly typed enumerator. This variant allows the user to use the objects in the collection to perform custom operations.

Examples

Expand table

Example	Description
Invoking Cmdlets Within a Cmdlet	This example shows how to invoke a cmdlet from within another cmdlet.
Invoking Scripts Within a Cmdlet	This example shows how to invoke a script that is supplied to the cmdlet from within another cmdlet.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Cmdlet Sets

When you design your cmdlets, you might encounter cases in which you need to perform several actions on the same piece of data. For example, you might need to get and set data or start and stop a process. Although you will need to create separate cmdlets to perform each action, your cmdlet design should include a base class from which the classes for the individual cmdlets are derived.

Keep the following things in mind when implementing a base class.

- Declare any common parameters used by all the derived cmdlets in the base class.
- Add cmdlet-specific parameters to the appropriate cmdlet class.
- Override the appropriate input processing method in the base class.
- Declare the System.Management.Automation.CmdletAttribute attribute on all cmdlet classes, but do not declare it on the base class.
- Implement a System.Management.Automation.PSSnapIn or System.Management.Automation.CustomPSSnapIn class whose name and description reflects the set of cmdlets.

Example

The following example shows the implementation of a base class that is used by Get-Proc and Stop-Proc cmdlet that derive from the same base class.

C#

```
using System;
using System.Diagnostics;
using System.Management.Automation;             //Windows PowerShell namespace.

namespace Microsoft.Samples.PowerShell.Commands
{

  #region ProcessCommands

  /// <summary>
  /// This class implements a Stop-Proc cmdlet. The parameters
  /// for this cmdlet are defined by the BaseProcCommand class.
  /// </summary>
```

```
[Cmdlet(VerbsLifecycle.Stop, "Proc", SupportsShouldProcess = true)]
public class StopProcCommand : BaseProcCommand
{
  public override void ProcessObject(Process process)
  {
    if (ShouldProcess(process.ProcessName, "Stop-Proc"))
    {
      process.Kill();
    }
  }
}

/// <summary>
/// This class implements a Get-Proc cmdlet. The parameters
/// for this cmdlet are defined by the BaseProcCommand class.
/// </summary>

[Cmdlet(VerbsCommon.Get, "Proc")]
public class GetProcCommand : BaseProcCommand
{
  public override void ProcessObject(Process process)
  {
    WriteObject(process);
  }
}

/// <summary>
/// This class is the base class that defines the common
/// functionality used by the Get-Proc and Stop-Proc
/// cmdlets.
/// </summary>
public class BaseProcCommand : Cmdlet
{
  #region Parameters

  // Defines the Name parameter that is used to
  // specify a process by its name.
  [Parameter(
             Position = 0,
             Mandatory = true,
             ValueFromPipeline = true,
             ValueFromPipelineByPropertyName = true
  )]
  public string[] Name
  {
    get { return processNames; }
    set { processNames = value; }
  }
  private string[] processNames;

  // Defines the Exclude parameter that is used to
  // specify which processes should be excluded when
  // the cmdlet performs its action.
  [Parameter()]
  public string[] Exclude
```

```
    {
      get { return excludeNames; }
      set { excludeNames = value; }
    }
    private string[] excludeNames = new string[0];
    #endregion Parameters

    public virtual void ProcessObject(Process process)
    {
      throw new NotImplementedException("This method should be overridden.");
    }

    #region Cmdlet Overrides
    // <summary>
    // For each of the requested process names, retrieve and write
    // the associated processes.
    // </summary>
    protected override void ProcessRecord()
    {
      // Set up the wildcard characters used in resolving
      // the process names.
      WildcardOptions options = WildcardOptions.IgnoreCase |
                                WildcardOptions.Compiled;

      WildcardPattern[] include = new WildcardPattern[Name.Length];
      for (int i = 0; i < Name.Length; i++)
      {
        include[i] = new WildcardPattern(Name[i], options);
      }

      WildcardPattern[] exclude = new WildcardPattern[Exclude.Length];
      for (int i = 0; i < Exclude.Length; i++)
      {
        exclude[i] = new WildcardPattern(Exclude[i], options);
      }

      foreach (Process p in Process.GetProcesses())
      {
        foreach (WildcardPattern wIn in include)
        {
          if (wIn.IsMatch(p.ProcessName))
          {
            bool processThisOne = true;
            foreach (WildcardPattern wOut in exclude)
            {
              if (wOut.IsMatch(p.ProcessName))
              {
                processThisOne = false;
                break;
              }
            }
            if (processThisOne)
            {
              ProcessObject(p);
            }
```

```
            break;
          }
        }
      }
    }
    #endregion Cmdlet Overrides
  }
    #endregion ProcessCommands
}
```

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Windows PowerShell Session State

Session state refers to the current configuration of a Windows PowerShell session or module. A Windows PowerShell session is the operational environment that is used interactively by the command-line user or programmatically by a host application. The session state for a session is referred to as the global session state.

From a developer perspective, a Windows PowerShell session refers to the time between when a host application opens a Windows PowerShell runspace and when it closes the runspace. Looked at another way, the session is the lifetime of an instance of the Windows PowerShell engine that is invoked while the runspace exists.

Module Session State

Module session states are created whenever the module or one of its nested modules is imported into the session. When a module exports an element such as a cmdlet, function, or script, a reference to that element is added to the global session state of the session. However, when the element is run, it is executed within the session state of the module.

Session-State Data

Session state data can be public or private. Public data is available to calls from outside the session state while private data is available only to calls from within the session state. For example, a module can have a private function that can be called only by the module or only internally by a public element that has been exported. This is similar to the private and public members of a .NET Framework type.

Session-state data is stored by the current instance of the execution engine within the context of the current Windows PowerShell session. Session-state data consists of the following items:

- Path information
- Drive information
- Windows PowerShell provider information
- Information about the imported modules and references to the module elements (such as cmdlets, functions, and scripts) that are exported by the module. This information and

these references are for the global session state only.

- Session-state variable information

Accessing Session-State Data Within Cmdlets

Cmdlets can access session-state data either indirectly through the System.Management.Automation.PSCmdlet.SessionState* property of the cmdlet class or directly through the System.Management.Automation.SessionState class. The System.Management.Automation.SessionState class provides properties that can be used to investigate different types of session-state data.

See Also

System.Management.Automation.PSCmdlet.SessionState

System.Management.Automation.SessionState

Windows PowerShell Cmdlets

Writing a Windows PowerShell Cmdlet

Windows PowerShell Shell SDK

Last updated on 05/20/2025

Examples of Cmdlet Code

Summarize this article for me

This section contains examples of cmdlet code that you can use to start writing your own cmdlets.

> **Important**
>
> If you want step-by-step instructions for writing cmdlets, see Tutorials for Writing Cmdlets.

In This Section

- How to Write a Simple Cmdlet - This example shows the basic structure of cmdlet code.
- How to Declare Cmdlet Parameters - This example shows how to declare the different types of parameters.
- How to Declare Parameter Sets - This example shows how to declare sets of parameters that can change the action a cmdlet performs.
- How to Validate Parameter Input - These examples show how to validate parameter input.
- How to Declare Dynamic Parameters - This example shows how to declare a parameter that is added at runtime.
- How to Invoke Scripts Within a Cmdlet - This example shows how to invoke a script that is supplied to a cmdlet.
- How To Override Input Processing Methods - These examples show the basic structure used to override the BeginProcessing, ProcessRecord, and EndProcessing methods.
- How to Support ShouldProcess Calls - This example shows how the System.Management.Automation.Cmdlet.ShouldProcess and System.Management.Automation.Cmdlet.ShouldContinue methods should be called from within a cmdlet.
- How to Support Transactions - This example shows how to indicate that the cmdlet supports transactions and how to implement the action that is taken when the cmdlet is used within a transaction.
- How to Support Transactions - This example shows how to indicate that the cmdlet supports transactions and how to implement the action that is taken when the cmdlet is used within a transaction.
- How to Support Jobs - This example shows how to support jobs when you write cmdlets.
- How to Invoke a Cmdlet From Within a Cmdlet - This example shows how to invoke a cmdlet from within another cmdlet, which allows you to add the functionality of the invoked cmdlet to the cmdlet you are developing.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 02/24/2026

How to write a cmdlet

This article shows how to write a cmdlet. The `Send-Greeting` cmdlet takes a single user name as input and then writes a greeting to that user. Although the cmdlet does not do much work, this example demonstrates the major sections of a cmdlet.

Steps to write a cmdlet

1. To declare the class as a cmdlet, use the **Cmdlet** attribute. The **Cmdlet** attribute specifies the verb and the noun for the cmdlet name.

   For more information about the **Cmdlet** attribute, see CmdletAttribute Declaration.

2. Specify the name of the class.

3. Specify that the cmdlet derives from either of the following classes:

   - System.Management.Automation.Cmdlet
   - System.Management.Automation.PSCmdlet

4. To define the parameters for the cmdlet, use the **Parameter** attribute. In this case, only one required parameter is specified.

   For more information about the **Parameter** attribute, see ParameterAttribute Declaration.

5. Override the input processing method that processes the input. In this case, the System.Management.Automation.Cmdlet.ProcessRecord method is overridden.

6. To write the greeting, use the method System.Management.Automation.Cmdlet.WriteObject. The greeting is displayed in the following format:

   Output

   ```
   Hello <UserName>!
   ```

Example

C#

```
using System.Management.Automation;  // Windows PowerShell assembly.

namespace SendGreeting
{
  // Declare the class as a cmdlet and specify the
  // appropriate verb and noun for the cmdlet name.
  [Cmdlet(VerbsCommunications.Send, "Greeting")]
  public class SendGreetingCommand : Cmdlet
  {
    // Declare the parameters for the cmdlet.
    [Parameter(Mandatory=true)]
    public string Name
    {
      get { return name; }
      set { name = value; }
    }
    private string name;

    // Override the ProcessRecord method to process
    // the supplied user name and write out a
    // greeting to the user by calling the WriteObject
    // method.
    protected override void ProcessRecord()
    {
      WriteObject("Hello " + name + "!");
    }
  }
}
```

See also

System.Management.Automation.Cmdlet

System.Management.Automation.PSCmdlet

System.Management.Automation.Cmdlet.ProcessRecord

System.Management.Automation.Cmdlet.WriteObject

CmdletAttribute Declaration

ParameterAttribute Declaration

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Declare Cmdlet Parameters

These examples show how to declare named, positional, required, optional, and `[switch]` parameters. These examples also show how to define a parameter alias.

How to Declare a Named Parameter

- Define a public property as shown in the following code. When you add the Parameter attribute, omit the `Position` keyword from the attribute.

```
C#
```

```
[Parameter()]
public string UserName
{
  get { return userName; }
  set { userName = value; }
}
private string userName;
```

For more information about the Parameter attribute, see Parameter Attribute Declaration.

How to Declare a Positional Parameter

- Define a public property as shown in the following code. When you add the Parameter attribute, set the `Position` keyword to the argument position. A value of 0 indicates the first position.

```
C#
```

```
[Parameter(Position = 0)]
public string UserName
{
  get { return userName; }
  set { userName = value; }
}
private string userName;
```

For more information about the Parameter attribute, see Parameter Attribute Declaration.

How to Declare a Mandatory Parameter

- Define a public property as shown in the following code. When you add the Parameter attribute, set the `Mandatory` keyword to `true`.

```C#
[Parameter(Position = 0, Mandatory = true)]
public string UserName
{
  get { return userName; }
  set { userName = value; }
}
private string userName;
```

For more information about the Parameter attribute, see Parameter Attribute Declaration.

How to Declare an Optional Parameter

- Define a public property as shown in the following code. When you add the Parameter attribute, omit the `Mandatory` keyword.

```C#
[Parameter(Position = 0)]
public string UserName
{
  get { return userName; }
  set { userName = value; }
}
private string userName;
```

How to Declare a `[switch]` parameter

- Define a public property as type System.Management.Automation.SwitchParameter, and then declare the Parameter attribute.

```C#
[Parameter(Position = 1)]
public SwitchParameter GoodBye
{
  get { return goodbye; }
  set { goodbye = value; }
}
private bool goodbye;
```

For more information about the Parameter attribute, see Parameter Attribute Declaration.

How to Declare a Parameter with Aliases

- Define a public property as shown in the following code. Add an Alias attribute that lists the aliases for the parameter. In this example, three aliases are defined for the same parameter. The first alias provides a shortcut. The second and third aliases provide names you can use for different scenarios.

```
C#
```

```
[Alias("UN","Writer","Editor")]
[Parameter()]
public string UserName
{
  get { return userName; }
  set { userName = value; }
}
private string userName;
```

For more information about the Alias attribute, see Alias Attribute Declaration.

See Also

System.Management.Automation.SwitchParameter

Parameter Attribute Declaration

Alias Attribute Declaration

Writing a Windows PowerShell Cmdlet

Last updated on 04/08/2026

How to Declare Parameter Sets

This example shows how to define two parameter sets when you declare the parameters for a cmdlet. Each parameter set has both a unique parameter and a shared parameter that is used by both parameter sets. For more information about parameters sets, including how to specify the default parameter set, see Cmdlet Parameter Sets.

> **Important**
>
> Whenever possible, define the unique parameter of a parameter set as a required parameter. However, if you want your cmdlet to run without specifying any parameters, the unique parameter can be an optional parameter. For example, the unique parameter of the `Get-Command` cmdlet is optional.

How to Define Two Parameter Sets

1. Add the `ParameterSet` keyword to the Parameter attribute for the unique parameter of the first parameter set.

   C#

   ```
   [Parameter(Position = 0, Mandatory = true,
              ParameterSetName = "Test01")]
   public string UserName
   {
     get { return userName; }
     set { userName = value; }
   }
   private string userName;
   ```

2. Add the `ParameterSet` keyword to the Parameter attribute for the unique parameter of the second parameter set.

   C#

   ```
   [Parameter(Position = 0, Mandatory = true,
              ParameterSetName = "Test02")]
   public string ComputerName
   {
     get { return computerName; }
     set { computerName = value; }
   ```

```
}
private string computerName;
```

3. For the parameter that belongs to both parameter sets, add a Parameter attribute for each parameter set and then add the `ParameterSet` keyword to each set. In each Parameter attribute, you can specify how that parameter is defined. A parameter can be optional in one set and mandatory in another.

C#

```
[Parameter(Mandatory= true, ParameterSetName = "Test01")]
[Parameter(ParameterSetName = "Test02")]
public string SharedParam
{
    get { return sharedParam; }
    set { sharedParam = value; }
}
private string sharedParam;
```

See Also

Cmdlet Parameter Sets

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Validate Parameter Input

This section contains examples that show how to validate parameter input by using various attributes to implement validation rules.

In This Section

How to Validate an Argument with a Script Describes how to validate an argument set by using the ArgumentSet attribute.

How to Validate an Argument Set Describes how to validate an argument set by using the ArgumentSet attribute.

How to Validate an Argument Range Describes how to validate an argument range by using the ArgumentRange attribute.

How to Validate an Argument Pattern Describes how to validate an argument pattern by using the ArgumentPattern attribute.

How to Validate the Argument Length Describes how to validate the length of an argument by using the ArgumentLength attribute.

How to Validate an Argument Count Describes how to validate an argument count by using the ArgumentCount attribute.

The way a parameter is declared can affect validation. For more information, see How to Declare Cmdlet Parameters.

Reference

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to validate an argument using a script

This example shows how to specify a validation rule that uses a script to check the parameter argument before the cmdlet is run. The value of the parameter is piped to the script. The script must return `$true` for every value piped to it.

> ⓘ Note
>
> For more information about the class that defines this attribute, see System.Management.Automation.ValidateScriptAttribute.

To validate an argument using a script

- Add the ValidateScript attribute as shown in the following code. This example specifies a script to validate that the input value is an odd number.

  C#

```
[ValidateScript("$_ % 2", ErrorMessage = "The item '{0}' did not pass
validation of script '{1}'")]
[Parameter(Position = 0, Mandatory = true)]
public int32 OddNumber
{
   get { return oddNumber; }
   set { oddNumber = value; }
}

private int32 oddNumber;
```

For more information about how to declare this attribute, see ValidateScript Attribute Declaration.

See Also

System.Management.Automation.ValidateScriptAttribute

ValidateScript Attribute Declaration

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Validate an Argument Set

This example shows how to specify a validation rule that the Windows PowerShell runtime can use to check the parameter argument before the cmdlet is run. This validation rule provides a set of the valid values for the parameter argument.

> ⓘ Note
>
> For more information about the class that defines this attribute, see System.Management.Automation.ValidateSetAttribute.

To validate an argument set

- Add the ValidateSet attribute as shown in the following code. This example specifies a set of three possible values for the `UserName` parameter.

```
C#
```

```
[ValidateSet("Steve", "Mary", "Carl", IgnoreCase = true)]
[Parameter(Position = 0, Mandatory = true)]
public string UserName
{
  get { return userName; }
  set { userName = value; }
}

private string userName;
```

For more information about how to declare this attribute, see ValidateSet Attribute Declaration.

See Also

System.Management.Automation.ValidateSetAttribute

ValidateSet Attribute Declaration

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Validate an Argument Range

This example shows how to specify a validation rule that the Windows PowerShell runtime can use to check the minimum and maximum values of the parameter argument before the cmdlet is run. You set this validation rule by declaring the ValidateRange attribute.

> ⓘ Note
>
> For more information about the class that defines this attribute, see System.Management.Automation.ValidateRangeAttribute.

To validate an argument range

- Add the ValidateRange attribute as shown in the following code. This example specifies a range of 0 to 5 for the `InputData` parameter.

```
C#
```

```
[ValidateRange(0, 5)]
[Parameter(Position = 0, Mandatory = true)]
public int InputData
{
  get { return inputData; }
  set { inputData = value; }
}
private int inputData;
```

For more information about how to declare this attribute, see ValidateRange Attribute Declaration.

See Also

ValidateRange Attribute Declaration

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Validate an Argument Pattern

This example shows how to specify a validation rule that the Windows PowerShell runtime can use to check the character pattern of the parameter argument before the cmdlet is run. You set this validation rule by declaring the ValidatePattern attribute.

> **Note**
>
> For more information about the class that defines this attribute, see System.Management.Automation.ValidatePatternAttribute.

To validate an argument pattern

- Add the Validate attribute as shown in the following code. This example specifies a pattern of four digits, where each digit has a value of 0 through 9.

```
C#
```

```
[ValidatePattern("[0-9][0-9][0-9][0-9]")]
[Parameter(Position = 0, Mandatory = true)]
public int InputData
{
  get { return inputData; }
  set { inputData = value; }
}

private int inputData;
```

For more information about how to declare this attribute, see ValidatePattern Attribute Declaration.

See Also

ValidatePattern Attribute Declaration

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Validate the Argument Length

This example shows how to specify a validation rule that the Windows PowerShell runtime can use to check the number of characters (the length) of the parameter argument before the cmdlet is run. You set this validation rule by declaring the ValidateLength attribute.

> **Note**
>
> For more information about the class that defines this attribute, see System.Management.Automation.ValidateLengthAttribute.

To validate the argument length

- Add the Validate attribute as shown in the following code. This example specifies that the length of the argument should have a length of 0 to 10 characters.

```
C#
```

```
[ValidateLength(0, 10)]
[Parameter(Position = 0, Mandatory = true)]
public string UserName
{
  get { return userName; }
  set { userName = value; }
}
private string userName;
```

For more information about how to declare this attribute, see ValidateLength Attribute Declaration.

See Also

ValidateLength Attribute Declaration

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Validate an Argument Count

This example shows how to specify a validation rule that the Windows PowerShell runtime can use to check the number of arguments (the count) that a parameter accepts before the cmdlet is run. You set this validation rule by declaring the ValidateCount attribute.

> **Note**
>
> For more information about the class that defines this attribute, see System.Management.Automation.ValidateCountAttribute.

To validate an argument count

- Add the Validate attribute as shown in the following code. This example specifies that the parameter will accept one argument or as many as three arguments.

```
C#
```

```
[ValidateCount(1, 3)]
[Parameter(Position = 0, Mandatory = true)]
public string[] UserNames
{
  get { return userNames; }
  set { userNames = value; }
}

private string[] userNames;
```

For more information about how to declare this attribute, see ValidateCount Attribute Declaration.

See Also

ValidateCount Attribute Declaration

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Declare Dynamic Parameters

This example shows how to define dynamic parameters that are added to the cmdlet at runtime. In this example, the `Department` parameter is added to the cmdlet whenever the user specifies the `Employee` `[switch]` parameter. For more information about dynamic parameters, see Cmdlet Dynamic Parameters.

To define dynamic parameters

1. In the cmdlet class declaration, add the System.Management.Automation.IDynamicParameters interface as shown.

   C#

   ```
   public class SendGreetingCommand : Cmdlet, IDynamicParameters
   ```

2. Call the System.Management.Automation.IDynamicParameters.GetDynamicParameters* method, which returns the object in which the dynamic parameters are defined. In this example, the method is called when the `Employee` parameter is specified.

   C#

   ```
   public object GetDynamicParameters()
   {
       if (employee)
       {
         context= new SendGreetingCommandDynamicParameters();
         return context;
       }
       return null;
   }
   private SendGreetingCommandDynamicParameters context;
   ```

3. Declare a class that defines the dynamic parameters to be added. You can use the attributes that you used to declare the static cmdlet parameters to declare the dynamic parameters.

   C#

   ```
   public class SendGreetingCommandDynamicParameters
   {
     [Parameter]
   ```

```
    [ValidateSet ("Marketing", "Sales", "Development")]
    public string Department
    {
      get { return department; }
      set { department = value; }
    }
    private string department;
  }
```

Example

In this example, the `Department` parameter is added whenever the user specifies the `Employee` parameter. The `Department` parameter is an optional parameter, and the ValidateSet attribute is used to specify the allowed arguments.

C#

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Management.Automation;     // PowerShell assembly.

namespace SendGreeting
{
  // Declare the cmdlet class that supports the
  // IDynamicParameters interface.
  [Cmdlet(VerbsCommunications.Send, "Greeting")]
  public class SendGreetingCommand : Cmdlet, IDynamicParameters
  {
    // Declare the parameters for the cmdlet.
    [Parameter(Mandatory = true)]
    public string Name
    {
      get { return name; }
      set { name = value; }
    }
    private string name;

    [Parameter]
    [Alias ("FTE")]
    public SwitchParameter Employee
    {
      get { return employee; }
      set { employee = value; }
    }
    private Boolean employee;

    // Implement GetDynamicParameters to
    // retrieve the dynamic parameter.
    public object GetDynamicParameters()
```

```
    {
      if (employee)
      {
        context= new SendGreetingCommandDynamicParameters();
        return context;
      }
      return null;
    }
    private SendGreetingCommandDynamicParameters context;

    // Override the ProcessRecord method to process the
    // supplied user name and write out a greeting to
    // the user by calling the WriteObject method.
    protected override void ProcessRecord()
    {
      WriteObject("Hello " + name + "! ");
      if (employee)
      {
        WriteObject("Department: " + context.Department);
      }
    }
  }

  // Define the dynamic parameters to be added
  public class SendGreetingCommandDynamicParameters
  {
    [Parameter]
    [ValidateSet ("Marketing", "Sales", "Development")]
    public string Department
    {
      get { return department; }
      set { department = value; }
    }
    private string department;
  }
}
```

See Also

- System.Management.Automation.RuntimeDefinedParameterDictionary
- System.Management.Automation.IDynamicParameters.GetDynamicParameters*
- Cmdlet Dynamic Parameters
- Windows PowerShell SDK

Last updated on 04/08/2026

How to Invoke Scripts Within a Cmdlet

This example shows how to invoke a script that is supplied to a cmdlet. The script is executed by the cmdlet, and its results are returned to the cmdlet as a collection of System.Management.Automation.PSObject objects.

To invoke a script block

1. The command verifies that a script block was supplied to the cmdlet. If a script block was supplied, the command invokes the script block with its required parameters.

   C#

   ```
   if (script != null)
   {
        WriteDebug("Executing script block.");

        // Invoke the script block with the required arguments.
        Collection<PSObject> PSObjects = script.Invoke(
            line,
            simpleMatch,
            caseSensitive
        );
       // more code as needed...
   }
   ```

2. Then, the script iterates through the returned collection of System.Management.Automation.PSObject objects and perform the necessary operations.

   C#

   ```
   foreach (PSObject object in PSObjects)
   {
       if (LanguagePrimitives.IsTrue(object))
       {
            result = new MatchInfo();
            result.Line = line;
            result.IgnoreCase = !caseSensitive;
            break;
       }
   }
   ```

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Override Input Processing Methods

These examples show how to overwrite the input processing methods within a cmdlet. These methods are used to perform the following operations:

- The System.Management.Automation.Cmdlet.BeginProcessing method is used to perform one-time startup operations that are valid for all the objects processed by the cmdlet. The Windows PowerShell runtime calls this method only once.
- The System.Management.Automation.Cmdlet.ProcessRecord method is used to process the objects passed to the cmdlet. The Windows PowerShell runtime calls this method for each object passed to the cmdlet.
- The System.Management.Automation.Cmdlet.EndProcessing method is used to perform one-time post processing operations. The Windows PowerShell runtime calls this method only once.

To override the BeginProcessing method

- Declare a protected override of the System.Management.Automation.Cmdlet.BeginProcessing method.

The following class prints a sample message. To use this class, change the verb and noun in the Cmdlet attribute, change the name of the class to reflect the new verb and noun, and then add the functionality you require to the override of the System.Management.Automation.Cmdlet.BeginProcessing method.

C#

```
[Cmdlet(VerbsDiagnostic.Test, "BeginProcessingClass")]
public class TestBeginProcessingClassTemplate : Cmdlet
{
  // Override the BeginProcessing method to add preprocessing
  //operations to the cmdlet.
  protected override void BeginProcessing()
  {
    // Replace the WriteObject method with the logic required
    // by your cmdlet. It is used here to generate the following
    // output:
    // "This is a test of the BeginProcessing template."
    WriteObject("This is a test of the BeginProcessing template.");
```

```
  }
}
```

To override the ProcessRecord method

- Declare a protected override of the System.Management.Automation.Cmdlet.ProcessRecord method.

The following class prints a sample message. To use this class, change the verb and noun in the Cmdlet attribute, change the name of the class to reflect the new verb and noun, and then add the functionality you require to the override of the System.Management.Automation.Cmdlet.ProcessRecord method.

C#

```
[Cmdlet(VerbsDiagnostic.Test, "ProcessRecordClass")]
public class TestProcessRecordClassTemplate : Cmdlet
{
    // Override the ProcessRecord method to add processing
    //operations to the cmdlet.
    protected override void ProcessRecord()
    {
        // Replace the WriteObject method with the logic required
        // by your cmdlet. It is used here to generate the following
        // output:
        // "This is a test of the ProcessRecord template."
        WriteObject("This is a test of the ProcessRecord template.");
    }
}
```

To override the EndProcessing method

- Declare a protected override of the System.Management.Automation.Cmdlet.EndProcessing method.

The following class prints a sample. To use this class, change the verb and noun in the Cmdlet attribute, change the name of the class to reflect the new verb and noun, and then add the functionality you require to the override of the System.Management.Automation.Cmdlet.EndProcessing method.

C#

```
[Cmdlet(VerbsDiagnostic.Test, "EndProcessingClass")]
public class TestEndProcessingClassTemplate : Cmdlet
```

```
{
  // Override the EndProcessing method to add postprocessing
  //operations to the cmdlet.
  protected override void EndProcessing()
  {
    // Replace the WriteObject method with the logic required
    // by your cmdlet. It is used here to generate the following
    // output:
    // "This is a test of the BeginProcessing template."
    WriteObject("This is a test of the EndProcessing template.");
  }
}
```

See Also

System.Management.Automation.Cmdlet.BeginProcessing

System.Management.Automation.Cmdlet.EndProcessing

System.Management.Automation.Cmdlet.ProcessRecord

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Support Transactions

This example shows the basic code elements that add support for transactions to a cmdlet.

> **ⓘ Important**
>
> For more information about how Windows PowerShell handles transactions, see About Transactions.

To support transactions

1. When you declare the Cmdlet attribute, specify that the cmdlet supports transactions. When the cmdlet supports transactions, Windows PowerShell adds the `UseTransaction` parameter to the cmdlet when it is run.

   C#

   ```
   [Cmdlet(VerbsCommunications.Send, "GreetingTx",
           SupportsTransactions=true )]
   ```

2. Within one of the input processing methods, add an `if` block to determine if a transaction is available. If the `if` statement resolves to `true`, the actions within this statement can be performed within the context of the current transaction.

   C#

   ```
   if (TransactionAvailable())
   {
     using (CurrentPSTransaction)
     {
       WriteObject("Hello " + name + "  from within a transaction.");
     }
   }
   ```

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Support Jobs

This example shows how to support jobs when you write cmdlets. If you want users to run your cmdlet as a background job, you must include the code described in the following procedure. For more information about background jobs, see Background Jobs.

To support jobs

1. Define an `AsJob` `[switch]` parameter so that the user can decide whether to run the cmdlet as a job.

   The following example shows an AsJob parameter declaration.

   C#

   ```
   [Parameter()]
   public SwitchParameter AsJob
   {
     get { return asjob; }
     set { asjob = value; }
   }
   private bool asjob;
   ```

2. Create an object that derives from the System.Management.Automation.Job class. This object can be a custom job object or one of the job objects provided by Windows PowerShell, such a System.Management.Automation.PSEventJob object.

   The following example shows a custom job object.

   C#

   ```
   private SampleJob job = new SampleJob("Get-ProcAsJob");
   ```

3. In a record processing method, add an `if` statement to detect whether the cmdlet should run as a job. The following code uses the System.Management.Automation.Cmdlet.ProcessRecord method.

   C#

   ```
   protected override void ProcessRecord()
   {
     if (asjob)
   ```

```
  {
    // Add the job definition to the job repository,
    // return the job object, and then create the thread
    // used to run the job.
    JobRepository.Add(job);
    WriteObject(job);
    ThreadPool.QueueUserWorkItem(WorkItem);
  }
  else
  {
    job.ProcessJob();
    foreach (PSObject p in job.Output)
    {
      WriteObject(p);
    }
  }
}
```

4. For custom job objects, implement the job class.

C#

```
private class SampleJob : Job
{
  internal SampleJob(string command)
      : base(command)
  {
    SetJobState(JobState.NotStarted);
  }
  public override string StatusMessage
  {
    get { throw new NotImplementedException(); }
  }

  public override bool HasMoreData
  {
    get
    {
      return hasMoreData;
    }
  }
  private bool hasMoreData = true;

  public override string Location
  {
    get { throw new NotImplementedException(); }
  }

  public override void StopJob()
  {
    throw new NotImplementedException();
  }
```

```
  internal void ProcessJob()
  {
    SetJobState(JobState.Running);
    DoProcessLogic();
    SetJobState(JobState.Completed);
  }

  // Retrieve the processes of the local computer.
  void DoProcessLogic()
  {
    Process[] p = Process.GetProcesses();

    foreach (Process pl in p)
    {
      Output.Add(PSObject.AsPSObject(pl));
    }
    Output.Complete();
  } // End DoProcessLogic.
} // End SampleJob class.
```

5. If the cmdlet performs the work, call the System.Management.Automation.Cmdlet.WriteObject method to return a process object to the pipeline. If the work is performed as a job, add child job to the job.

C#

```
void DoProcessLogic(bool asJob)
{
  Process[] p = Process.GetProcesses();

  foreach (Process pl in p)
  {
    if (!asjob)
    {
      WriteObject(pl);
    }
    else
    {
      job.ChildJobs[0].Output.Add(PSObject.AsPSObject(pl));
    }
  }
} // End DoProcessLogic.
```

Example

The following sample code shows the code for a `Get-Proc` cmdlet that can retrieve processes internally or by using a background job.

C#

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Management.Automation;  // Windows PowerShell namespace.
using System.Threading;              // Thread pool namespace for posting work.
using System.Diagnostics;            // Diagnostics namespace for retrieving
                                     // process objects.

// This sample shows a cmdlet whose work can be done by the cmdlet or by using
// a background job. Background jobs are executed in their own thread,
// independent of the pipeline thread in which the cmdlet is executed.
//
// To load this cmdlet, create a module folder and copy the GetProcessSample06.dll
// assembly into the module folder. Make sure that the path to the module folder
// is added to the $PSModulePath environment variable.
// Module folder path:
//    user/documents/WindowsPowerShell/modules/GetProcessSample06
//
// To import the module, run the following command: Import-Module
GetProcessSample06.
// To test the cmdlet, run the following command: Get-Proc -Name <process name>
//

//
namespace Microsoft.Samples.PowerShell.Commands
{
  /// <summary>
  ///  This cmdlet retrieves process internally or returns
  ///  a job that retrieves the processes.
  /// </summary>
  [Cmdlet(VerbsCommon.Get, "Proc")]
  public sealed class GetProcCommand : PSCmdlet
  {

    #region Parameters
    /// <summary>
    /// Specify the Name parameter. This parameter accepts
    /// process names from the command line.
    /// </summary>
    [Parameter(
               Position = 0,
               ValueFromPipeline = true,
               ValueFromPipelineByPropertyName = true)]
    [ValidateNotNullOrEmpty]
    public string[] Name
    {
      get { return processNames; }
      set { processNames = value; }
    }
    private string[] processNames;

    /// <summary>
    /// Specify the AsJob parameter. This parameter indicates
```

```
    /// whether the cmdlet should retrieve the processes internally
    /// or return a Job object that retrieves the processes.
    /// </summary>
    [Parameter()]
    public SwitchParameter AsJob
    {
      get { return asjob; }
      set { asjob = value; }
    }
    private bool asjob;

    #endregion Parameters

    #region Cmdlet Overrides

    // Create a custom job object.
    private SampleJob job = new SampleJob("Get-ProcAsJob");

    /// <summary>
    /// Determines if the processes should be retrieved
    /// internally or if a Job object should be returned.
    /// </summary>
    protected override void ProcessRecord()
    {
      if (asjob)
      {
        // Add the job definition to the job repository,
        // return the job object, and then create the thread
        // used to run the job.
        JobRepository.Add(job);
        WriteObject(job);
        ThreadPool.QueueUserWorkItem(WorkItem);
      }
      else
      {
        job.ProcessJob();
        foreach (PSObject p in job.Output)
        {
          WriteObject(p);
        }
      }
    }
    #endregion Overrides

    // Implement a custom job that derives
    // from the System.Management.Automation.Job class.
    private class SampleJob : Job
    {
      internal SampleJob(string command)
          : base(command)
      {
        SetJobState(JobState.NotStarted);
      }
      public override string StatusMessage
      {
```

```
        get { throw new NotImplementedException(); }
      }

      public override bool HasMoreData
      {
        get
        {
          return hasMoreData;
        }
      }
      private bool hasMoreData = true;

      public override string Location
      {
        get { throw new NotImplementedException(); }
      }

      public override void StopJob()
      {
        throw new NotImplementedException();
      }

      internal void ProcessJob()
      {
        SetJobState(JobState.Running);
        DoProcessLogic();
        SetJobState(JobState.Completed);
      }

      // Retrieve the processes of the local computer.
      void DoProcessLogic()
      {
        Process[] p = Process.GetProcesses();

        foreach (Process pl in p)
        {
          Output.Add(PSObject.AsPSObject(pl));
        }
        Output.Complete();
      } // End DoProcessLogic.
    } // End SampleJob class.

    void WorkItem(object dummy)
    {
       job.ProcessJob();
    }

    // Display the results of the work. If not a job,
    // process objects are returned. If a job, the
    // output is added to the job as a child job.
    void DoProcessLogic(bool asJob)
    {
      Process[] p = Process.GetProcesses();

      foreach (Process pl in p)
```

```
      {
        if (!asjob)
        {
          WriteObject(pl);
        }
        else
        {
          job.ChildJobs[0].Output.Add(PSObject.AsPSObject(pl));
        }
      }
    } // End DoProcessLogic.
  } //End GetProcCommand
}
```

Last updated on 04/08/2026

How to Invoke a Cmdlet from Within a Cmdlet

This example shows how to invoke a binary cmdlet that derives from `[System.Management.Automation.Cmdlet]` directly from within another binary cmdlet, which allows you to add the functionality of the invoked cmdlet to the binary cmdlet you are developing. In this example, the `Get-Process` cmdlet is invoked to get the processes that are running on the local computer. The call to the `Get-Process` cmdlet is equivalent to the following command. This command retrieves all the processes whose names start with the characters "a" through "t".

PowerShell

```
Get-Process -Name [a-t]*
```

> **ⓘ Important**
>
> You can invoke only those cmdlets that derive directly from the System.Management.Automation.Cmdlet class. You can't invoke a cmdlet that derives from the System.Management.Automation.PSCmdlet class. For an example, see How to invoke a PSCmdlet from within a PSCmdlet.

To invoke a cmdlet from within a cmdlet

1. Ensure that the assembly that defines the cmdlet to be invoked is referenced and that the appropriate `using` statement is added. In this example, the following namespaces are added.

   C#

   ```
   using System.Diagnostics;
   using System.Management.Automation;   // PowerShell assembly.
   using Microsoft.PowerShell.Commands;  // PowerShell cmdlets assembly you want
   to call.
   ```

2. In the input processing method of the cmdlet, create a new instance of the cmdlet to be invoked. In this example, an object of type

Microsoft.PowerShell.Commands.GetProcessCommand is created along with the string that contains the arguments that are used when the cmdlet is invoked.

```
C#

GetProcessCommand gp = new GetProcessCommand();
gp.Name = new string[] { "[a-t]*" };
```

3. Call the System.Management.Automation.Cmdlet.Invoke* method to invoke the `Get-Process` cmdlet.

```
C#

  foreach (Process p in gp.Invoke<Process>())
  {
    Console.WriteLine(p.ToString());
  }
}
```

Example

In this example, the `Get-Process` cmdlet is invoked from within the System.Management.Automation.Cmdlet.BeginProcessing method of a cmdlet.

```
C#

using System;
using System.Diagnostics;
using System.Management.Automation;   // PowerShell assembly.
using Microsoft.PowerShell.Commands;  // PowerShell cmdlets assembly you want to
call.

namespace SendGreeting
{
  // Declare the class as a cmdlet and specify an
  // appropriate verb and noun for the cmdlet name.
  [Cmdlet(VerbsCommunications.Send, "GreetingInvoke")]
  public class SendGreetingInvokeCommand : Cmdlet
  {
    // Declare the parameters for the cmdlet.
    [Parameter(Mandatory = true)]
    public string Name { get; set; }

    // Override the BeginProcessing method to invoke
    // the Get-Process cmdlet.
    protected override void BeginProcessing()
    {
      GetProcessCommand gp = new GetProcessCommand();
```

```
      gp.Name = new string[] { "[a-t]*" };
      foreach (Process p in gp.Invoke<Process>())
      {
        WriteVerbose(p.ToString());
      }
    }

    // Override the ProcessRecord method to process
    // the supplied user name and write out a
    // greeting to the user by calling the WriteObject
    // method.
    protected override void ProcessRecord()
    {
      WriteObject("Hello " + Name + "!");
    }
  }
}
```

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to invoke a PSCmdlet from within a PSCmdlet

This example shows how to invoke a script based cmdlet or binary cmdlet inheriting from `[System.Management.Automation.PSCmdlet]` from within a binary cmdlet. In this example, the new cmdlet `Get-ClipboardReverse` calls `Get-Clipboard` to get the contents of the clipboard. The `Get-ClipboardReverse` reverses the order of the characters and returns the reversed string.

> **Note**
>
> The `[PSCmdlet]` class differs from the `[Cmdlet]` class. `[PSCmdlet]` implementations use runspace context information so you must invoke another cmdlet using the PowerShell pipeline API. In `[Cmdlet]` implementations you can call the cmdlet's .NET API directly. For an example, see How to invoke a Cmdlet from within a Cmdlet.

To invoke a cmdlet from within a PSCmdlet

1. Ensure that the namespace for the `[System.Management.Automation.PowerShell]` API is referenced. In this example, the following namespaces are added.

   C#

   ```
   using System.Management.Automation;   // PowerShell assembly.
   using System.Text;
   ```

2. To invoke a command from within another binary cmdlet you must use the `[PowerShell]` API to construct a new pipeline and add the cmdlet to be invoked. Call the System.Management.Automation.PowerShell.Invoke<T>() method to invoke the pipeline.

   C#

   ```
   using var ps = PowerShell.Create(RunspaceMode.CurrentRunspace);
   ps.AddCommand("Get-Clipboard").AddParameter("Raw");
   var output = ps.Invoke<string>();
   ```

Example

To invoke a script based cmdlet or binary cmdlet inheriting from `[PSCmdlet]` you must build a PowerShell pipeline with the command and parameters you want to execute, then invoke the pipeline.

C#

```
using System;
using System.Management.Automation;   // PowerShell assembly.
using System.Text;

namespace ClipboardReverse
{
    [Cmdlet(VerbsCommon.Get,"ClipboardReverse")]
    [OutputType(typeof(string))]
    public class ClipboardReverse : PSCmdlet
    {
        protected override void EndProcessing()
        {
            using var ps = PowerShell.Create(RunspaceMode.CurrentRunspace);
            ps.AddCommand("Get-Clipboard").AddParameter("Raw");
            var output = ps.Invoke<string>();
            if (ps.HadErrors)
            {
                WriteError(new ErrorRecord(ps.Streams.Error[0].Exception,
                           "Get-Clipboard Error", ErrorCategory.NotSpecified,
null));
            }
            else
            {
                var sb = new StringBuilder();
                foreach (var text in output)
                {
                    sb.Append(text);
                }

                var reversed = sb.ToString().ToCharArray();
                Array.Reverse(reversed);
                WriteObject(new string(reversed));
            }
        }
    }
}
```

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Tutorials for Writing Cmdlets

This section contains tutorials for writing cmdlets. These tutorials include the code needed to write the cmdlets, plus an explanation of why the code is needed. These topics will be very helpful for those who are just starting to write cmdlets.

> **Important**
>
> For those who want code examples with less description, see Cmdlet Samples.

In This Section

GetProc Tutorial - This tutorial describes how to define a cmdlet class and add basic functionality such as adding parameters and reporting errors. The cmdlet described in this tutorial is very similar to the Get-Process cmdlet provided by Windows PowerShell.

StopProc Tutorial - This tutorial describes how to define a cmdlet and add functionality such as user prompts, wildcard support, and the use of parameter sets. The cmdlet described here performs the same task as the Stop-Process cmdlet provided by Windows PowerShell.

SelectStr Tutorial - This tutorial describes how to define a cmdlet that accesses a data store. The cmdlet described here performs the same task as the Select-String cmdlet provided by Windows PowerShell.

See Also

GetProc Tutorial

StopProc Tutorial

SelectStr Tutorial

Windows PowerShell SDK

Last updated on 05/20/2025

GetProc Tutorial

This section provides a tutorial for creating a Get-Proc cmdlet that is very similar to the Get-Process cmdlet provided by Windows PowerShell. This tutorial provides fragments of code that illustrate how cmdlets are implemented, and an explanation of the code.

Topics in this Tutorial

The topics in this tutorial are designed to be read sequentially, with each topic building on what was discussed in the previous topic.

- **Creating a Cmdlet without Parameters**: This section describes how to create a cmdlet that retrieves information from the local computer without the use of parameters, and then writes the information to the pipeline.
- **Adding Parameters that Process Command-Line Input**: This section describes how to add a parameter to the Get-Proc cmdlet so that the cmdlet can process input based on explicit objects passed to the cmdlet. The implementation described here retrieves processes based on their name, and then writes the information to the pipeline.
- **Adding Parameters that Process Pipeline Input**: This section describes how to add a parameter to the Get-Proc cmdlet so that the cmdlet can process objects passed to it through the pipeline. The implementation cmdlet described here retrieves processes based on objects passed to the cmdlet, and then writes the information to the pipeline.
- **Adding Non-terminating Error Reporting to Your Cmdlet**: This section describes how to add non-terminating error reporting to a cmdlet. The implementation described here detects non-terminating errors that occur when processing input, and writes an error record to the error stream.

See Also

- Windows PowerShell SDK

Last updated on 05/20/2025

Creating a Cmdlet without Parameters

This section describes how to create a cmdlet that retrieves information from the local computer without the use of parameters, and then writes the information to the pipeline. The cmdlet described here is a Get-Proc cmdlet that retrieves information about the processes of the local computer, and then displays that information at the command line.

> ⓘ **Note**
>
> Be aware that when writing cmdlets, the Windows PowerShell® reference assemblies are downloaded onto disk (by default at C:\Program Files\Reference Assemblies\Microsoft\WindowsPowerShell\v1.0). They are not installed in the Global Assembly Cache (GAC).

Naming the Cmdlet

A cmdlet name consists of a verb that indicates the action the cmdlet takes and a noun that indicates the items that the cmdlet acts upon. Because this sample Get-Proc cmdlet retrieves process objects, it uses the verb "Get", defined by the System.Management.Automation.VerbsCommon enumeration, and the noun "Proc" to indicate that the cmdlet works on process items.

When naming cmdlets, do not use any of the following characters: # , () {} [] & - /\ $; : " '<> | ? @ `.

Choosing a Noun

You should choose a noun that is specific. It is best to use a singular noun prefixed with a shortened version of the product name. An example cmdlet name of this type is "`Get-SQLServer`".

Choosing a Verb

You should use a verb from the set of approved cmdlet verb names. For more information about the approved cmdlet verbs, see Cmdlet Verb Names.

Defining the Cmdlet Class

Once you have chosen a cmdlet name, define a .NET class to implement the cmdlet. Here is the class definition for this sample Get-Proc cmdlet:

C#

```
[Cmdlet(VerbsCommon.Get, "Proc")]
  public class GetProcCommand : Cmdlet
```

VB

```
<Cmdlet(VerbsCommon.Get, "Proc")> _
Public Class GetProcCommand
    Inherits Cmdlet
```

Notice that previous to the class definition, the System.Management.Automation.CmdletAttribute attribute, with the syntax `[Cmdlet(verb, noun, ...)]`, is used to identify this class as a cmdlet. This is the only required attribute for all cmdlets, and it allows the Windows PowerShell runtime to call them correctly. You can set attribute keywords to further declare the class if necessary. Be aware that the attribute declaration for our sample GetProcCommand class declares only the noun and verb names for the Get-Proc cmdlet.

> ⓘ **Note**
>
> For all Windows PowerShell attribute classes, the keywords that you can set correspond to properties of the attribute class.

When naming the class of the cmdlet, it is a good practice to reflect the cmdlet name in the class name. To do this, use the form "VerbNounCommand" and replace "Verb" and "Noun" with the verb and noun used in the cmdlet name. As is shown in the previous class definition, the sample Get-Proc cmdlet defines a class called GetProcCommand, which derives from the System.Management.Automation.Cmdlet base class.

> ⓘ **Important**
>
> If you want to define a cmdlet that accesses the Windows PowerShell runtime directly, your .NET class should derive from the System.Management.Automation.PSCmdlet base class. For more information about this class, see Creating a Cmdlet that Defines Parameter Sets.

> **Note**
>
> The class for a cmdlet must be explicitly marked as public. Classes that are not marked as public will default to internal and will not be found by the Windows PowerShell runtime.

Windows PowerShell uses the Microsoft.PowerShell.Commands namespace for its cmdlet classes. It is recommended to place your cmdlet classes in a Commands namespace of your API namespace, for example, xxx.PS.Commands.

Overriding an Input Processing Method

The System.Management.Automation.Cmdlet class provides three main input processing methods, at least one of which your cmdlet must override. For more information about how Windows PowerShell processes records, see How Windows PowerShell Works.

For all types of input, the Windows PowerShell runtime calls System.Management.Automation.Cmdlet.BeginProcessing to enable processing. If your cmdlet must perform some preprocessing or setup, it can do this by overriding this method.

> **Note**
>
> Windows PowerShell uses the term "record" to describe the set of parameter values supplied when a cmdlet is called.

If your cmdlet accepts pipeline input, it must override the System.Management.Automation.Cmdlet.ProcessRecord method, and optionally the System.Management.Automation.Cmdlet.EndProcessing method. For example, a cmdlet might override both methods if it gathers all input using System.Management.Automation.Cmdlet.ProcessRecord and then operates on the input as a whole rather than one element at a time, as the `Sort-Object` cmdlet does.

If your cmdlet does not take pipeline input, it should override the System.Management.Automation.Cmdlet.EndProcessing method. Be aware that this method is frequently used in place of System.Management.Automation.Cmdlet.BeginProcessing when the cmdlet cannot operate on one element at a time, as is the case for a sorting cmdlet.

Because this sample Get-Proc cmdlet must receive pipeline input, it overrides the System.Management.Automation.Cmdlet.ProcessRecord method and uses the default

implementations for System.Management.Automation.Cmdlet.BeginProcessing and System.Management.Automation.Cmdlet.EndProcessing. The System.Management.Automation.Cmdlet.ProcessRecord override retrieves processes and writes them to the command line using the System.Management.Automation.Cmdlet.WriteObject method.

C#

```
protected override void ProcessRecord()
{
  // Get the current processes
  Process[] processes = Process.GetProcesses();

  // Write the processes to the pipeline making them available
  // to the next cmdlet. The second parameter of this call tells
  // PowerShell to enumerate the array, and send one process at a
  // time to the pipeline.
  WriteObject(processes, true);
}
```

VB

```
Protected Overrides Sub ProcessRecord()

    '/ Get the current processes.
    Dim processes As Process()
    processes = Process.GetProcesses()

    '/ Write the processes to the pipeline making them available
    '/ to the next cmdlet. The second parameter of this call tells
    '/ PowerShell to enumerate the array, and send one process at a
    '/ time to the pipeline.
    WriteObject(processes, True)

End Sub 'ProcessRecord
```

Things to Remember About Input Processing

- The default source for input is an explicit object (for example, a string) provided by the user on the command line. For more information, see Creating a Cmdlet to Process Command Line Input.
- An input processing method can also receive input from the output object of an upstream cmdlet on the pipeline. For more information, see Creating a Cmdlet to Process Pipeline Input. Be aware that your cmdlet can receive input from a combination of command-line and pipeline sources.

- The downstream cmdlet might not return for a long time, or not at all. For that reason, the input processing method in your cmdlet should not hold locks during calls to System.Management.Automation.Cmdlet.WriteObject, especially locks for which the scope extends beyond the cmdlet instance.

> **Important**
>
> Cmdlets should never call **System.Console.Writeline*** or its equivalent.

- Your cmdlet might have object variables to clean up when it is finished processing (for example, if it opens a file handle in the System.Management.Automation.Cmdlet.BeginProcessing method and keeps the handle open for use by System.Management.Automation.Cmdlet.ProcessRecord). It is important to remember that the Windows PowerShell runtime does not always call the System.Management.Automation.Cmdlet.EndProcessing method, which should perform object cleanup.

For example, System.Management.Automation.Cmdlet.EndProcessing might not be called if the cmdlet is canceled midway or if a terminating error occurs in any part of the cmdlet. Therefore, a cmdlet that requires object cleanup should implement the complete System.IDisposable pattern, including the finalizer, so that the runtime can call both System.Management.Automation.Cmdlet.EndProcessing and System.IDisposable.Dispose* at the end of processing.

Code Sample

For the complete C# sample code, see GetProcessSample01 Sample.

Defining Object Types and Formatting

Windows PowerShell passes information between cmdlets using .NET objects. Consequently, a cmdlet might need to define its own type, or the cmdlet might need to extend an existing type provided by another cmdlet. For more information about defining new types or extending existing types, see Extending Object Types and Formatting.

Building the Cmdlet

After implementing a cmdlet, you must register it with Windows PowerShell through a Windows PowerShell snap-in. For more information about registering cmdlets, see How to Register Cmdlets, Providers, and Host Applications.

Testing the Cmdlet

When your cmdlet has been registered with Windows PowerShell, you can test it by running it on the command line. The code for our sample Get-Proc cmdlet is small, but it still uses the Windows PowerShell runtime and an existing .NET object, which is enough to make it useful. Let's test it to better understand what Get-Proc can do and how its output can be used. For more information about using cmdlets from the command line, see the Getting Started with Windows PowerShell.

1. Start Windows PowerShell, and get the current processes running on the computer.

   PowerShell

   ```
   Get-Proc
   ```

   The following output appears.

   Output

   ```
   Handles  NPM(K)  PM(K)  WS(K)  VS(M)  CPU(s)  Id   ProcessName
   -------  ------  -----  -----  -----  ------  --   -----------
   254      7       7664   12048  66     173.75  1200  QCTRAY
   32       2       1372   2628   31       0.04  1860  DLG
   271      6       1216   3688   33       0.03  3816  lg
   27       2       560    1920   24       0.01  1768  TpScrex
   ...
   ```

2. Assign a variable to the cmdlet results for easier manipulation.

   PowerShell

   ```
   $p=Get-Proc
   ```

3. Get the number of processes.

   PowerShell

   ```
   $p.Length
   ```

The following output appears.

Output

```
63
```

4. Retrieve a specific process.

PowerShell

```
$p[6]
```

The following output appears.

Output

```
Handles  NPM(K)  PM(K)  WS(K)  VS(M)  CPU(s)  Id    ProcessName
-------  ------  -----  -----  -----  ------  --    -----------
1033     3       2400   3336   35     0.53    1588  rundll32
```

5. Get the start time of this process.

PowerShell

```
$p[6].StartTime
```

The following output appears.

Output

```
Tuesday, July 26, 2005 9:34:15 AM
```

PowerShell

```
$p[6].StartTime.DayOfYear
```

Output

```
207
```

6. Get the processes for which the handle count is greater than 500, and sort the result.

PowerShell

```
$p | Where-Object {$_.HandleCount -gt 500 } | Sort-Object HandleCount
```

The following output appears.

Output

```
Handles  NPM(K)  PM(K)  WS(K)  VS(M)  CPU(s)  Id   ProcessName
-------  ------  -----  -----  -----  ------  --   -----------
568      14      2164   4972   39      5.55    824  svchost
716       7      2080   5332   28     25.38    468  csrss
761      21      33060  56608  440   393.56    3300 WINWORD
791      71      7412   4540   59      3.31    492  winlogon
...
```

7. Use the `Get-Member` cmdlet to list the properties available for each process.

PowerShell

```
$p | Get-Member -MemberType Property
```

Output

```
    TypeName: System.Diagnostics.Process
```

The following output appears.

Output

```
Name                       MemberType Definition
----                       ---------- ----------
BasePriority               Property   System.Int32 BasePriority {get;}
Container                  Property   System.ComponentModel.IContainer Conta...
EnableRaisingEvents        Property   System.Boolean EnableRaisingEvents {ge...
...
```

See Also

Creating a Cmdlet to Process Command Line Input

Creating a Cmdlet to Process Pipeline Input

How to Create a Windows PowerShell Cmdlet

Extending Object Types and Formatting

How Windows PowerShell Works

How to Register Cmdlets, Providers, and Host Applications

Windows PowerShell Reference

Cmdlet Samples

Last updated on 05/20/2025

Adding Parameters That Process Command-Line Input

One source of input for a cmdlet is the command line. This topic describes how to add a parameter to the `Get-Proc` cmdlet (which is described in Creating Your First Cmdlet) so that the cmdlet can process input from the local computer based on explicit objects passed to the cmdlet. The `Get-Proc` cmdlet described here retrieves processes based on their names, and then displays information about the processes at a command prompt.

Defining the Cmdlet Class

The first step in cmdlet creation is cmdlet naming and the declaration of the .NET Framework class that implements the cmdlet. This cmdlet retrieves process information, so the verb name chosen here is "Get." (Almost any sort of cmdlet that is capable of retrieving information can process command-line input.) For more information about approved cmdlet verbs, see Cmdlet Verb Names.

Here's the class declaration for the `Get-Proc` cmdlet. Details about this definition are provided in Creating Your First Cmdlet.

C#

```
[Cmdlet(VerbsCommon.Get, "proc")]
public class GetProcCommand: Cmdlet
```

VB

```
<Cmdlet(VerbsCommon.Get, "Proc")> _
Public Class GetProcCommand
    Inherits Cmdlet
```

Declaring Parameters

A cmdlet parameter enables the user to provide input to the cmdlet. In the following example, `Get-Proc` and `Get-Member` are the names of pipelined cmdlets, and `MemberType` is a parameter for the `Get-Member` cmdlet. The parameter has the argument "property."

PS> Get-Proc ; `Get-Member` -MemberType Property

To declare parameters for a cmdlet, you must first define the properties that represent the parameters. In the `Get-Proc` cmdlet, the only parameter is `Name`, which in this case represents the name of the .NET Framework process object to retrieve. Therefore, the cmdlet class defines a property of type string to accept an array of names.

Here's the parameter declaration for the `Name` parameter of the `Get-Proc` cmdlet.

C#

```
/// <summary>
/// Specify the cmdlet Name parameter.
/// </summary>
  [Parameter(Position = 0)]
  [ValidateNotNullOrEmpty]
  public string[] Name
  {
    get { return processNames; }
    set { processNames = value; }
  }
  private string[] processNames;

  #endregion Parameters
```

VB

```
<Parameter(Position:=0), ValidateNotNullOrEmpty()> _
Public Property Name() As String()
    Get
        Return processNames
    End Get

    Set(ByVal value As String())
        processNames = value
    End Set

End Property
```

To inform the Windows PowerShell runtime that this property is the `Name` parameter, a System.Management.Automation.ParameterAttribute attribute is added to the property definition. The basic syntax for declaring this attribute is `[Parameter()]`.

> ⓘ Note
>
> A parameter must be explicitly marked as public. Parameters that are not marked as public default to internal and are not found by the Windows PowerShell runtime.

This cmdlet uses an array of strings for the `Name` parameter. If possible, your cmdlet should also define a parameter as an array, because this allows the cmdlet to accept more than one item.

Things to Remember About Parameter Definitions

- Predefined Windows PowerShell parameter names and data types should be reused as much as possible to ensure that your cmdlet is compatible with Windows PowerShell cmdlets. For example, if all cmdlets use the predefined `Id` parameter name to identify a resource, user will easily understand the meaning of the parameter, regardless of what cmdlet they are using. Basically, parameter names follow the same rules as those used for variable names in the common language runtime (CLR). For more information about parameter naming, see Cmdlet Parameter Names.

- Windows PowerShell reserves a few parameter names to provide a consistent user experience. Do not use these parameter names: `WhatIf`, `Confirm`, `Verbose`, `Debug`, `Warn`, `ErrorAction`, `ErrorVariable`, `OutVariable`, and `OutBuffer`. Additionally, the following aliases for these parameter names are reserved: `vb`, `db`, `ea`, `ev`, `ov`, and `ob`.

- `Name` is a simple and common parameter name, recommended for use in your cmdlets. It is better to choose a parameter name like this than a complex name that is unique to a specific cmdlet and hard to remember.

- Parameters are case-insensitive in Windows PowerShell, although by default the shell preserves case. Case-sensitivity of the arguments depends on the operation of the cmdlet. Arguments are passed to a parameter as specified at the command line.

- For examples of other parameter declarations, see Cmdlet Parameters.

Declaring Parameters as Positional or Named

A cmdlet must set each parameter as either a positional or named parameter. Both kinds of parameters accept single arguments, multiple arguments separated by commas, and Boolean settings. A Boolean parameter, also called a *switch*, handles only Boolean settings. The switch is used to determine the presence of the parameter. The recommended default is `false`.

The sample `Get-Proc` cmdlet defines the `Name` parameter as a positional parameter with position 0. This means that the first argument the user enters on the command line is automatically inserted for this parameter. If you want to define a named parameter, for which the user must specify the parameter name from the command line, leave the `Position` keyword out of the attribute declaration.

> ⓘ Note
>
> Unless parameters must be named, we recommend that you make the most-used parameters positional so that users will not have to type the parameter name.

Declaring Parameters as Mandatory or Optional

A cmdlet must set each parameter as either an optional or a mandatory parameter. In the sample `Get-Proc` cmdlet, the `Name` parameter is defined as optional because the `Mandatory` keyword is not set in the attribute declaration.

Supporting Parameter Validation

The sample `Get-Proc` cmdlet adds an input validation attribute, System.Management.Automation.ValidateNotNullOrEmptyAttribute, to the `Name` parameter to enable validation that the input is neither `null` nor empty. This attribute is one of several validation attributes provided by Windows PowerShell. For examples of other validation attributes, see Validating Parameter Input.

```
[Parameter(Position = 0)]
[ValidateNotNullOrEmpty]
public string[] Name
```

Overriding an Input Processing Method

If your cmdlet is to handle command-line input, it must override the appropriate input processing methods. The basic input processing methods are introduced in Creating Your First Cmdlet.

The `Get-Proc` cmdlet overrides the System.Management.Automation.Cmdlet.ProcessRecord method to handle the `Name` parameter input provided by the user or a script. This method gets the processes for each requested process name, or all for processes if no name is provided. Notice that in System.Management.Automation.Cmdlet.ProcessRecord, the call to System.Management.Automation.Cmdlet.WriteObject is the output mechanism for sending output objects to the pipeline. The second parameter of this call, `enumerateCollection`, is set to

`true` to inform the Windows PowerShell runtime to enumerate the output array of process objects and write one process at a time to the command line.

C#

```
protected override void ProcessRecord()
{
  // If no process names are passed to the cmdlet, get all processes.
  if (processNames == null)
  {
    // Write the processes to the pipeline making them available
    // to the next cmdlet. The second argument of this call tells
    // PowerShell to enumerate the array, and send one process at a
    // time to the pipeline.
    WriteObject(Process.GetProcesses(), true);
  }
  else
  {
    // If process names are passed to the cmdlet, get and write
    // the associated processes.
    foreach (string name in processNames)
    {
      WriteObject(Process.GetProcessesByName(name), true);
    }
  }
}
```

VB

```
Protected Overrides Sub ProcessRecord()

    '/ If no process names are passed to the cmdlet, get all processes.
    If processNames Is Nothing Then
        Dim processes As Process()
        processes = Process.GetProcesses()
    End If

    '/ If process names are specified, write the processes to the
    '/ pipeline to display them or make them available to the next cmdlet.

    For Each name As String In processNames
        '/ The second parameter of this call tells PowerShell to enumerate the
        '/ array, and send one process at a time to the pipeline.
        WriteObject(Process.GetProcessesByName(name), True)
    Next

End Sub 'ProcessRecord
```

Code Sample

For the complete C# sample code, see GetProcessSample02 Sample.

Defining Object Types and Formatting

Windows PowerShell passes information between cmdlets by using .NET Framework objects. Consequently, a cmdlet might need to define its own type, or a cmdlet might need to extend an existing type provided by another cmdlet. For more information about defining new types or extending existing types, see Extending Object Types and Formatting.

Building the Cmdlet

After you implement a cmdlet, you must register it with Windows PowerShell by using a Windows PowerShell snap-in. For more information about registering cmdlets, see How to Register Cmdlets, Providers, and Host Applications.

Testing the Cmdlet

When your cmdlet is registered with Windows PowerShell, you can test it by running it on the command line. Here are two ways to test the code for the sample cmdlet. For more information about using cmdlets from the command line, see Getting Started with Windows PowerShell.

- At the Windows PowerShell prompt, use the following command to list the Internet Explorer process, which is named "IEXPLORE."

  PowerShell

  ```
  Get-Proc -Name iexplore
  ```

  The following output appears.

  Output

  ```
  Handles  NPM(K)  PM(K)   WS(K)  VS(M)  CPU(s)   Id   ProcessName
  -------  ------  -----   -----  -----   ------ --   -----------
      354      11  10036   18992    85   0.67   3284   iexplore
  ```

- To list the Internet Explorer, Outlook, and Notepad processes named "IEXPLORE," "OUTLOOK," and "NOTEPAD," use the following command. If there are multiple processes, all of them are displayed.

  PowerShell

```
Get-Proc -Name iexplore, outlook, notepad
```

The following output appears.

```
Handles  NPM(K)  PM(K)   WS(K)  VS(M)  CPU(s)   Id   ProcessName
-------  ------  -----   -----  -----  ------   --   -----------
    732      21  24696    5000    138   2.25  2288   iexplore
    715      19  20556   14116    136   1.78  3860   iexplore
   3917      62  74096   58112    468 191.56  1848   OUTLOOK
     39       2   1024    3280     30   0.09  1444   notepad
     39       2   1024     356     30   0.08  3396   notepad
```

See Also

Adding Parameters that Process Pipeline Input

Creating Your First Cmdlet

Extending Object Types and Formatting

How to Register Cmdlets, Providers, and Host Applications

Windows PowerShell Reference

Cmdlet Samples

Last updated on 05/20/2025

Adding Parameters that Process Pipeline Input

One source of input for a cmdlet is an object on the pipeline that originates from an upstream cmdlet. This section describes how to add a parameter to the Get-Proc cmdlet (described in Creating Your First Cmdlet) so that the cmdlet can process pipeline objects.

This Get-Proc cmdlet uses a `Name` parameter that accepts input from a pipeline object, retrieves process information from the local computer based on the supplied names, and then displays information about the processes at the command line.

Defining the Cmdlet Class

The first step in cmdlet creation is always naming the cmdlet and declaring the .NET class that implements the cmdlet. This cmdlet retrieves process information, so the verb name chosen here is "Get". (Almost any sort of cmdlet that is capable of retrieving information can process command-line input.) For more information about approved cmdlet verbs, see Cmdlet Verb Names.

The following is the definition for this Get-Proc cmdlet. Details of this definition are given in Creating Your First Cmdlet.

C#

```
[Cmdlet(VerbsCommon.Get, "proc")]
public class GetProcCommand : Cmdlet
```

VB

```
<Cmdlet(VerbsCommon.Get, "Proc")> _
Public Class GetProcCommand
    Inherits Cmdlet
```

Defining Input from the Pipeline

This section describes how to define input from the pipeline for a cmdlet. This Get-Proc cmdlet defines a property that represents the `Name` parameter as described in Adding Parameters that

Process Command Line Input. (See that topic for general information about declaring parameters.)

However, when a cmdlet needs to process pipeline input, it must have its parameters bound to input values by the Windows PowerShell runtime. To do this, you must add the `ValueFromPipeline` keyword or add the `ValueFromPipelineByProperty` keyword to the System.Management.Automation.ParameterAttribute attribute declaration. Specify the `ValueFromPipeline` keyword if the cmdlet accesses the complete input object. Specify the `ValueFromPipelineByProperty` if the cmdlet accesses only a property of the object.

Here is the parameter declaration for the `Name` parameter of this Get-Proc cmdlet that accepts pipeline input.

C#

```
[Parameter(
   Position = 0,
   ValueFromPipeline = true,
   ValueFromPipelineByPropertyName = true)]
[ValidateNotNullOrEmpty]
public string[] Name
{
   get { return this.processNames; }
   set { this.processNames = value; }
}
```

VB

```
<Parameter(Position:=0, ValueFromPipeline:=True, _
ValueFromPipelineByPropertyName:=True), ValidateNotNullOrEmpty()> _
Public Property Name() As String()
    Get
        Return processNames
    End Get

    Set(ByVal value As String())
        processNames = value
    End Set

End Property
```

The previous declaration sets the `ValueFromPipeline` keyword to `true` so that the Windows PowerShell runtime will bind the parameter to the incoming object if the object is the same type as the parameter, or if it can be coerced to the same type. The `ValueFromPipelineByPropertyName` keyword is also set to `true` so that the Windows PowerShell runtime will check the incoming object for a `Name` property. If the incoming object has such a

property, the runtime will bind the `Name` parameter to the `Name` property of the incoming object.

> ⓘ Note
>
> The setting of the `ValueFromPipeline` attribute keyword for a parameter takes precedence over the setting for the `ValueFromPipelineByPropertyName` keyword.

Overriding an Input Processing Method

If your cmdlet is to handle pipeline input, it needs to override the appropriate input processing methods. The basic input processing methods are introduced in Creating Your First Cmdlet.

This Get-Proc cmdlet overrides the System.Management.Automation.Cmdlet.ProcessRecord method to handle the `Name` parameter input provided by the user or a script. This method will get the processes for each requested process name or all processes if no name is provided. Notice that within System.Management.Automation.Cmdlet.ProcessRecord, the call to WriteObject(System.Object,System.Boolean) is the output mechanism for sending output objects to the pipeline. The second parameter of this call, `enumerateCollection`, is set to `true` to tell the Windows PowerShell runtime to enumerate the array of process objects, and write one process at a time to the command line.

C#

```
protected override void ProcessRecord()
{
  // If no process names are passed to the cmdlet, get all processes.
  if (processNames == null)
  {
      // Write the processes to the pipeline making them available
      // to the next cmdlet. The second argument of this call tells
      // PowerShell to enumerate the array, and send one process at a
      // time to the pipeline.
      WriteObject(Process.GetProcesses(), true);
  }
  else
  {
    // If process names are passed to the cmdlet, get and write
    // the associated processes.
    foreach (string name in processNames)
    {
      WriteObject(Process.GetProcessesByName(name), true);
    } // End foreach (string name...).
```

```
  }
}
```

VB

```
Protected Overrides Sub ProcessRecord()
    Dim processes As Process()

    '/ If no process names are passed to the cmdlet, get all processes.
    If processNames Is Nothing Then
        processes = Process.GetProcesses()
    Else

        '/ If process names are specified, write the processes to the
        '/ pipeline to display them or make them available to the next cmdlet.
        For Each name As String In processNames
            '/ The second parameter of this call tells PowerShell to enumerate the
            '/ array, and send one process at a time to the pipeline.
            WriteObject(Process.GetProcessesByName(name), True)
        Next
    End If

End Sub 'ProcessRecord
```

Code Sample

For the complete C# sample code, see GetProcessSample03 Sample.

Defining Object Types and Formatting

Windows PowerShell passes information between cmdlets using .NET objects. Consequently, a cmdlet may need to define its own type, or the cmdlet may need to extend an existing type provided by another cmdlet. For more information about defining new types or extending existing types, see Extending Object Types and Formatting.

Building the Cmdlet

After implementing a cmdlet it must be registered with Windows PowerShell through a Windows PowerShell snap-in. For more information about registering cmdlets, see How to Register Cmdlets, Providers, and Host Applications.

Testing the Cmdlet

When your cmdlet has been registered with Windows PowerShell, test it by running it on the command line. For example, test the code for the sample cmdlet. For more information about using cmdlets from the command line, see the Getting Started with Windows PowerShell.

- At the Windows PowerShell prompt, enter the following commands to retrieve the process names through the pipeline.

  PowerShell

  ```
  PS> type ProcessNames | Get-Proc
  ```

  The following output appears.

  ```
  Handles  NPM(K)  PM(K)   WS(K)  VS(M)  CPU(s)    Id  ProcessName
  -------  ------  -----   -----  -----  ------    --  -----------
      809      21  40856    4448    147    9.50  2288  iexplore
      737      21  26036   16348    144   22.03  3860  iexplore
       39       2   1024     388     30    0.08  3396  notepad
     3927      62  71836   26984    467  195.19  1848  OUTLOOK
  ```

- Enter the following lines to get the process objects that have a `Name` property from the processes called "IEXPLORE". This example uses the `Get-Process` cmdlet (provided by Windows PowerShell) as an upstream command to retrieve the "IEXPLORE" processes.

  PowerShell

  ```
  PS> Get-Process iexplore | Get-Proc
  ```

  The following output appears.

  ```
  Handles  NPM(K)  PM(K)   WS(K)  VS(M)  CPU(s)    Id  ProcessName
  -------  ------  -----   -----  -----  ------    --  -----------
      801      21  40720    6544    142    9.52  2288  iexplore
      726      21  25872   16652    138   22.09  3860  iexplore
      801      21  40720    6544    142    9.52  2288  iexplore
      726      21  25872   16652    138   22.09  3860  iexplore
  ```

See Also

Adding Parameters that Process Command Line Input

Creating Your First Cmdlet

Extending Object Types and Formatting

How to Register Cmdlets, Providers, and Host Applications

Windows PowerShell Reference

Cmdlet Samples

Last updated on 05/20/2025

Adding Non-Terminating Error Reporting to Your Cmdlet

Cmdlets can report non-terminating errors by calling the System.Management.Automation.Cmdlet.WriteError method and still continue to operate on the current input object or on further incoming pipeline objects. This section explains how to create a cmdlet that reports non-terminating errors from its input processing methods.

For non-terminating errors (as well as terminating errors), the cmdlet must pass an System.Management.Automation.ErrorRecord object identifying the error. Each error record is identified by a unique string called the "error identifier". In addition to the identifier, the category of each error is specified by constants defined by a System.Management.Automation.ErrorCategory enumeration. The user can view errors based on their category by setting the `$ErrorView` variable to "CategoryView".

For more information about error records, see Windows PowerShell Error Records.

Defining the Cmdlet

The first step in cmdlet creation is always naming the cmdlet and declaring the .NET class that implements the cmdlet. This cmdlet retrieves process information, so the verb name chosen here is "Get". (Almost any sort of cmdlet that is capable of retrieving information can process command-line input.) For more information about approved cmdlet verbs, see Cmdlet Verb Names.

The following is the definition for this `Get-Proc` cmdlet. Details of this definition are given in Creating Your First Cmdlet.

C#

```
[Cmdlet(VerbsCommon.Get, "proc")]
public class GetProcCommand: Cmdlet
```

VB

```
<Cmdlet(VerbsCommon.Get, "Proc")> _
Public Class GetProcCommand
    Inherits Cmdlet
```

Defining Parameters

If necessary, your cmdlet must define parameters for processing input. This Get-Proc cmdlet defines a **Name** parameter as described in Adding Parameters that Process Command-Line Input.

Here is the parameter declaration for the **Name** parameter of this Get-Proc cmdlet.

C#

```
[Parameter(
           Position = 0,
           ValueFromPipeline = true,
           ValueFromPipelineByPropertyName = true
)]
[ValidateNotNullOrEmpty]
public string[] Name
{
  get { return processNames; }
  set { processNames = value; }
}
private string[] processNames;
```

VB

```
<Parameter(Position:=0, ValueFromPipeline:=True, _
ValueFromPipelineByPropertyName:=True), ValidateNotNullOrEmpty()> _
Public Property Name() As String()
    Get
        Return processNames
    End Get

    Set(ByVal value As String())
        processNames = value
    End Set

End Property
```

Overriding Input Processing Methods

All cmdlets must override at least one of the input processing methods provided by the System.Management.Automation.Cmdlet class. These methods are discussed in Creating Your First Cmdlet.

> ⓘ Note

Your cmdlet should handle each record as independently as possible.

This Get-Proc cmdlet overrides the System.Management.Automation.Cmdlet.ProcessRecord method to handle the **Name** parameter for input provided by the user or a script. This method will get the processes for each requested process name or all processes if no name is provided. Details of this override are given in Creating Your First Cmdlet.

Things to Remember When Reporting Errors

The System.Management.Automation.ErrorRecord object that the cmdlet passes when writing an error requires an exception at its core. Follow the .NET guidelines when determining the exception to use. Basically, if the error is semantically the same as an existing exception, the cmdlet should use or derive from that exception. Otherwise, it should derive a new exception or exception hierarchy directly from the System.Exception class.

When creating error identifiers (accessed through the FullyQualifiedErrorId property of the ErrorRecord class) keep the following in mind.

- Use strings that are targeted for diagnostic purposes so that when inspecting the fully qualified identifier you can determine what the error is and where the error came from.
- A well formed fully qualified error identifier might be as follows.

  ```
  CommandNotFoundException,Microsoft.PowerShell.Commands.GetCommandCommand
  ```

Notice that in the previous example, the error identifier (the first token) designates what the error is and the remaining part indicates where the error came from.

- For more complex scenarios, the error identifier can be a dot separated token that can be parsed on inspection. This allows you too branch on the parts of the error identifier as well as the error identifier and error category.

The cmdlet should assign specific error identifiers to different code paths. Keep the following information in mind for assignment of error identifiers:

- An error identifier should remain constant throughout the cmdlet life cycle. Do not change the semantics of an error identifier between cmdlet versions.
- Use text for an error identifier that tersely corresponds to the error being reported. Do not use white space or punctuation.
- Have your cmdlet generate only error identifiers that are reproducible. For example, it should not generate an identifier that includes a process identifier. Error identifiers are

useful to a user only when they correspond to identifiers that are seen by other users experiencing the same problem.

Unhandled exceptions are not caught by PowerShell in the following conditions:

- If a cmdlet creates a new thread and code running in that thread throws an unhandled exception, PowerShell will not catch the error and will terminate the process.
- If an object has code in its destructor or Dispose methods that causes an unhandled exception, PowerShell will not catch the error and will terminate the process.

Reporting Non-terminating Errors

Any one of the input processing methods can report a non-terminating error to the output stream using the System.Management.Automation.Cmdlet.WriteError method.

Here is a code example from this Get-Proc cmdlet that illustrates the call to System.Management.Automation.Cmdlet.WriteError from within the override of the System.Management.Automation.Cmdlet.ProcessRecord method. In this case, the call is made if the cmdlet cannot find a process for a specified process identifier.

C#

```
protected override void ProcessRecord()
{
  // If no name parameter passed to cmdlet, get all processes.
  if (processNames == null)
  {
    WriteObject(Process.GetProcesses(), true);
  }
    else
    {
      // If a name parameter is passed to cmdlet, get and write
      // the associated processes.
      // Write a non-terminating error for failure to retrieve
      // a process.
      foreach (string name in processNames)
      {
        Process[] processes;

        try
        {
          processes = Process.GetProcessesByName(name);
        }
        catch (InvalidOperationException ex)
        {
          WriteError(new ErrorRecord(
                     ex,
```

```
                    "NameNotFound",
                    ErrorCategory.InvalidOperation,
                    name));
          continue;
        }

        WriteObject(processes, true);
      } // foreach (...
    } // else
  }
```

Things to Remember About Writing Non-terminating Errors

For a non-terminating error, the cmdlet must generate a specific error identifier for each specific input object.

A cmdlet frequently needs to modify the PowerShell action produced by a non-terminating error. It can do this by defining the `ErrorAction` and `ErrorVariable` parameters. If defining the `ErrorAction` parameter, the cmdlet presents the user options System.Management.Automation.ActionPreference, you can also directly influence the action by setting the `$ErrorActionPreference` variable.

The cmdlet can save non-terminating errors to a variable using the `ErrorVariable` parameter, which is not affected by the setting of `ErrorAction`. Failures can be appended to an existing error variable by adding a plus sign (+) to the front of the variable name.

Code Sample

For the complete C# sample code, see GetProcessSample04 Sample.

Define Object Types and Formatting

PowerShell passes information between cmdlets using .NET objects. Consequently, a cmdlet might need to define its own type, or the cmdlet might need to extend an existing type provided by another cmdlet. For more information about defining new types or extending existing types, see Extending Object Types and Formatting.

Building the Cmdlet

After implementing a cmdlet, you must register it with Windows PowerShell through a Windows PowerShell snap-in. For more information about registering cmdlets, see How to Register Cmdlets, Providers, and Host Applications.

Testing the Cmdlet

When your cmdlet has been registered with PowerShell, you can test it by running it on the command line. Let's test the sample Get-Proc cmdlet to see whether it reports an error:

- Start PowerShell, and use the Get-Proc cmdlet to retrieve the processes named "TEST".

  PowerShell

  ```
  Get-Proc -Name test
  ```

  The following output appears.

  ```
  Get-Proc : Operation is not valid due to the current state of the object.
  At line:1 char:9
  + Get-Proc  <<<< -Name test
  ```

See Also

- Adding Parameters that Process Pipeline Input
- Adding Parameters that Process Command-Line Input
- Creating Your First Cmdlet
- Extending Object Types and Formatting
- How to Register Cmdlets, Providers, and Host Applications
- Windows PowerShell Reference
- Cmdlet Samples

Last updated on 05/20/2025

StopProc Tutorial

This section provides a tutorial for creating the Stop-Proc cmdlet, which is very similar to the Stop-Process cmdlet provided by Windows PowerShell. This tutorial provides fragments of code that illustrate how cmdlets are implemented, and an explanation of the code.

Topics in this Tutorial

The topics in this tutorial are designed to be read sequentially, with each topic building on what was discussed in the previous topic.

- **Creating a Cmdlet that Modifies the System**: This section describes how to create a cmdlet that supports system modifications, such as stopping a process running on the computer.
- **Adding User Messages to Your Cmdlet**: This section describes how to add the ability to write user messages, debug messages, warning messages, and progress information to your cmdlet.
- **Adding Aliases, Wildcard Expansion, and Help to Cmdlet Parameters**: This section describes how to create a cmdlet that supports parameter aliases, Help, and wildcard expansion.
- **Adding Parameter Sets to Cmdlets**: This section describes how to add parameter sets to a cmdlet. Parameter sets allow the cmdlet to operate differently based on what parameters are specified by the user.

See Also

- Windows PowerShell SDK

Last updated on 05/20/2025

Creating a Cmdlet that Modifies the System

Sometimes a cmdlet must modify the running state of the system, not just the state of the Windows PowerShell runtime. In these cases, the cmdlet should allow the user a chance to confirm whether or not to make the change.

To support confirmation a cmdlet must do two things.

- Declare that the cmdlet supports confirmation when you specify the System.Management.Automation.CmdletAttribute attribute by setting the SupportsShouldProcess keyword to `true`.
- Call System.Management.Automation.Cmdlet.ShouldProcess during the execution of the cmdlet (as shown in the following example).

By supporting confirmation, a cmdlet exposes the `Confirm` and `WhatIf` parameters that are provided by Windows PowerShell, and also meets the development guidelines for cmdlets (For more information about cmdlet development guidelines, see Cmdlet Development Guidelines.).

Changing the System

The act of "changing the system" refers to any cmdlet that potentially changes the state of the system outside Windows PowerShell. For example, stopping a process, enabling or disabling a user account, or adding a row to a database table are all changes to the system that should be confirmed. In contrast, operations that read data or establish transient connections do not change the system and generally do not require confirmation. Confirmation is also not needed for actions whose effect is limited to inside the Windows PowerShell runtime, such as `Set-Variable`. Cmdlets that might or might not make a persistent change should declare `SupportsShouldProcess` and call System.Management.Automation.Cmdlet.ShouldProcess only if they are about to make a persistent change.

> ⓘ **Note**
>
> ShouldProcess confirmation applies only to cmdlets. If a command or script modifies the running state of a system by directly calling .NET methods or properties, or by calling applications outside of Windows PowerShell, this form of confirmation will not be available.

The StopProc Cmdlet

This topic describes a Stop-Proc cmdlet that attempts to stop processes that are retrieved using the Get-Proc cmdlet (described in Creating Your First Cmdlet).

Defining the Cmdlet

The first step in cmdlet creation is always naming the cmdlet and declaring the .NET class that implements the cmdlet. Because you are writing a cmdlet to change the system, it should be named accordingly. This cmdlet stops system processes, so the verb name chosen here is "Stop", defined by the System.Management.Automation.VerbsLifecycle class, with the noun "Proc" to indicate that the cmdlet stops processes. For more information about approved cmdlet verbs, see Cmdlet Verb Names.

The following is the class definition for this Stop-Proc cmdlet.

C#

```
[Cmdlet(VerbsLifecycle.Stop, "Proc",
        SupportsShouldProcess = true)]
public class StopProcCommand : Cmdlet
```

Be aware that in the System.Management.Automation.CmdletAttribute declaration, the `SupportsShouldProcess` attribute keyword is set to `true` to enable the cmdlet to make calls to System.Management.Automation.Cmdlet.ShouldProcess and System.Management.Automation.Cmdlet.ShouldContinue. Without this keyword set, the `Confirm` and `WhatIf` parameters will not be available to the user.

Extremely Destructive Actions

Some operations are extremely destructive, such as reformatting an active hard disk partition. In these cases, the cmdlet should set `ConfirmImpact` = `ConfirmImpact.High` when declaring the System.Management.Automation.CmdletAttribute attribute. This setting forces the cmdlet to request user confirmation even when the user has not specified the `Confirm` parameter. However, cmdlet developers should avoid overusing `ConfirmImpact` for operations that are just potentially destructive, such as deleting a user account. Remember that if `ConfirmImpact` is set to System.Management.Automation.ConfirmImpact **High**.

Similarly, some operations are unlikely to be destructive, although they do in theory modify the running state of a system outside Windows PowerShell. Such cmdlets can set `ConfirmImpact` to

System.Management.Automation.ConfirmImpact.Low. This will bypass confirmation requests where the user has asked to confirm only medium-impact and high-impact operations.

Defining Parameters for System Modification

This section describes how to define the cmdlet parameters, including those that are needed to support system modification. See Adding Parameters that Process CommandLine Input if you need general information about defining parameters.

The Stop-Proc cmdlet defines three parameters: `Name`, `Force`, and `PassThru`.

The `Name` parameter corresponds to the `Name` property of the process input object. Be aware that the `Name` parameter in this sample is mandatory, as the cmdlet will fail if it does not have a named process to stop.

The `Force` parameter allows the user to override calls to System.Management.Automation.Cmdlet.ShouldContinue. In fact, any cmdlet that calls System.Management.Automation.Cmdlet.ShouldContinue should have a `Force` parameter so that when `Force` is specified, the cmdlet skips the call to System.Management.Automation.Cmdlet.ShouldContinue and proceeds with the operation. Be aware that this does not affect calls to System.Management.Automation.Cmdlet.ShouldProcess.

The `PassThru` parameter allows the user to indicate whether the cmdlet passes an output object through the pipeline, in this case, after a process is stopped. Be aware that this parameter is tied to the cmdlet itself instead of to a property of the input object.

Here is the parameter declaration for the Stop-Proc cmdlet.

C#

```
[Parameter(
           Position = 0,
           Mandatory = true,
           ValueFromPipeline = true,
           ValueFromPipelineByPropertyName = true
)]
public string[] Name
{
  get { return processNames; }
  set { processNames = value; }
}
private string[] processNames;

/// <summary>
/// Specify the Force parameter that allows the user to override
```

```
/// the ShouldContinue call to force the stop operation. This
/// parameter should always be used with caution.
/// </summary>
[Parameter]
public SwitchParameter Force
{
  get { return force; }
  set { force = value; }
}
private bool force;

/// <summary>
/// Specify the PassThru parameter that allows the user to specify
/// that the cmdlet should pass the process object down the pipeline
/// after the process has been stopped.
/// </summary>
[Parameter]
public SwitchParameter PassThru
{
  get { return passThru; }
  set { passThru = value; }
}
private bool passThru;
```

Overriding an Input Processing Method

The cmdlet must override an input processing method. The following code illustrates the System.Management.Automation.Cmdlet.ProcessRecord override used in the sample Stop-Proc cmdlet. For each requested process name, this method ensures that the process is not a special process, tries to stop the process, and then sends an output object if the `PassThru` parameter is specified.

C#

```
protected override void ProcessRecord()
{
  foreach (string name in processNames)
  {
    // For every process name passed to the cmdlet, get the associated
    // process(es). For failures, write a non-terminating error
    Process[] processes;

    try
    {
      processes = Process.GetProcessesByName(name);
    }
    catch (InvalidOperationException ioe)
    {
      WriteError(new ErrorRecord(ioe,"Unable to access the target process by name",
                 ErrorCategory.InvalidOperation, name));
```

```
      continue;
    }

    // Try to stop the process(es) that have been retrieved for a name
    foreach (Process process in processes)
    {
      string processName;

      try
      {
        processName = process.ProcessName;
      }

      catch (Win32Exception e)
        {
          WriteError(new ErrorRecord(e, "ProcessNameNotFound",
                     ErrorCategory.ReadError, process));
          continue;
        }

        // Call Should Process to confirm the operation first.
        // This is always false if WhatIf is set.
        if (!ShouldProcess(string.Format("{0} ({1})", processName,
                           process.Id)))
        {
          continue;
        }
        // Call ShouldContinue to make sure the user really does want
        // to stop a critical process that could possibly stop the computer.
        bool criticalProcess =
             criticalProcessNames.Contains(processName.ToLower());

        if (criticalProcess &&!force)
        {
          string message = String.Format
                ("The process \"{0}\" is a critical process and should not be
stopped. Are you sure you wish to stop the process?",
                processName);

          // It is possible that ProcessRecord is called multiple times
          // when the Name parameter receives objects as input from the
          // pipeline. So to retain YesToAll and NoToAll input that the
          // user may enter across multiple calls to ProcessRecord, this
          // information is stored as private members of the cmdlet.
          if (!ShouldContinue(message, "Warning!",
                              ref yesToAll,
                              ref noToAll))
          {
            continue;
          }
        } // if (criticalProcess...
        // Stop the named process.
        try
        {
          process.Kill();
```

```
        }
        catch (Exception e)
        {
          if ((e is Win32Exception) || (e is SystemException) ||
              (e is InvalidOperationException))
          {
            // This process could not be stopped so write
            // a non-terminating error.
            string message = String.Format("{0} {1} {2}",
                             "Could not stop process \"", processName,
                             "\".");
            WriteError(new ErrorRecord(e, message,
                       ErrorCategory.CloseError, process));
                       continue;
          } // if ((e is...
          else throw;
        } // catch

        // If the PassThru parameter argument is
        // True, pass the terminated process on.
        if (passThru)
        {
          WriteObject(process);
        }
    } // foreach (Process...
  } // foreach (string...
} // ProcessRecord
```

Calling the ShouldProcess Method

The input processing method of your cmdlet should call the System.Management.Automation.Cmdlet.ShouldProcess method to confirm execution of an operation before a change (for example, deleting files) is made to the running state of the system. This allows the Windows PowerShell runtime to supply the correct "WhatIf" and "Confirm" behavior within the shell.

> **Note**
>
> If a cmdlet states that it supports should process and fails to make the System.Management.Automation.Cmdlet.ShouldProcess call, the user might modify the system unexpectedly.

The call to System.Management.Automation.Cmdlet.ShouldProcess sends the name of the resource to be changed to the user, with the Windows PowerShell runtime taking into account any command-line settings or preference variables in determining what should be displayed to the user.

The following example shows the call to System.Management.Automation.Cmdlet.ShouldProcess from the override of the System.Management.Automation.Cmdlet.ProcessRecord method in the sample Stop-Proc cmdlet.

C#

```
if (!ShouldProcess(string.Format("{0} ({1})", processName,
                   process.Id)))
{
  continue;
}
```

Calling the ShouldContinue Method

The call to the System.Management.Automation.Cmdlet.ShouldContinue method sends a secondary message to the user. This call is made after the call to System.Management.Automation.Cmdlet.ShouldProcess returns `true` and if the `Force` parameter was not set to `true`. The user can then provide feedback to say whether the operation should be continued. Your cmdlet calls System.Management.Automation.Cmdlet.ShouldContinue as an additional check for potentially dangerous system modifications or when you want to provide yes-to-all and no-to-all options to the user.

The following example shows the call to System.Management.Automation.Cmdlet.ShouldContinue from the override of the System.Management.Automation.Cmdlet.ProcessRecord method in the sample Stop-Proc cmdlet.

C#

```
if (criticalProcess &&!force)
{
  string message = String.Format
        ("The process \"{0}\" is a critical process and should not be stopped. Are
you sure you wish to stop the process?",
        processName);

  // It is possible that ProcessRecord is called multiple times
  // when the Name parameter receives objects as input from the
  // pipeline. So to retain YesToAll and NoToAll input that the
  // user may enter across multiple calls to ProcessRecord, this
  // information is stored as private members of the cmdlet.
  if (!ShouldContinue(message, "Warning!",
```

```
                        ref yesToAll,
                        ref noToAll))
  {
    continue;
  }
} // if (criticalProcess...
```

Stopping Input Processing

The input processing method of a cmdlet that makes system modifications must provide a way of stopping the processing of input. In the case of this Stop-Proc cmdlet, a call is made from the System.Management.Automation.Cmdlet.ProcessRecord method to the System.Diagnostics.Process.Kill* method. Because the `PassThru` parameter is set to `true`, System.Management.Automation.Cmdlet.ProcessRecord also calls System.Management.Automation.Cmdlet.WriteObject to send the process object to the pipeline.

Code Sample

For the complete C# sample code, see StopProcessSample01 Sample.

Defining Object Types and Formatting

Windows PowerShell passes information between cmdlets using .NET objects. Consequently, a cmdlet may need to define its own type, or the cmdlet may need to extend an existing type provided by another cmdlet. For more information about defining new types or extending existing types, see Extending Object Types and Formatting.

Building the Cmdlet

After implementing a cmdlet, it must be registered with Windows PowerShell through a Windows PowerShell snap-in. For more information about registering cmdlets, see How to Register Cmdlets, Providers, and Host Applications.

Testing the Cmdlet

When your cmdlet has been registered with Windows PowerShell, you can test it by running it on the command line. Here are several tests that test the Stop-Proc cmdlet. For more information about using cmdlets from the command line, see the Running commands in the shell.

- Start Windows PowerShell and use the Stop-Proc cmdlet to stop processing as shown below. Because the cmdlet specifies the `Name` parameter as mandatory, the cmdlet queries for the parameter.

  PowerShell

  ```
  PS> Stop-Proc
  ```

  The following output appears.

  ```

  Cmdlet Stop-Proc at command pipeline position 1
  Supply values for the following parameters:
  Name[0]:
  ```

- Now let's use the cmdlet to stop the process named "NOTEPAD". The cmdlet asks you to confirm the action.

  PowerShell

  ```
  PS> Stop-Proc -Name notepad
  ```

  The following output appears.

  ```

  Confirm
  Are you sure you want to perform this action?
  Performing operation "Stop-Proc" on Target "notepad (4996)".
  [Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default
  is "Y"): Y
  ```

- Use Stop-Proc as shown to stop the critical process named "WINLOGON". You are prompted and warned about performing this action because it will cause the operating system to reboot.

  PowerShell

  ```
  PS> Stop-Proc -Name Winlogon
  ```

  The following output appears.

  ```
  ```

Output

```
Confirm
Are you sure you want to perform this action?
Performing operation "Stop-Proc" on Target "winlogon (656)".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default
is "Y"): Y
Warning!
The process " winlogon " is a critical process and should not be stopped. Are
you sure you wish to stop the process?
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default
is "Y"): N
```

- Let's now try to stop the WINLOGON process without receiving a warning. Be aware that this command entry uses the `Force` parameter to override the warning.

  PowerShell

  ```
  PS> Stop-Proc -Name winlogon -Force
  ```

  The following output appears.

  Output

  ```
  Confirm
  Are you sure you want to perform this action?
  Performing operation "Stop-Proc" on Target "winlogon (656)".
  [Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default
  is "Y"): N
  ```

See Also

- Adding Parameters that Process Command-Line Input
- Extending Object Types and Formatting
- How to Register Cmdlets, Providers, and Host Applications
- Windows PowerShell SDK
- Cmdlet Samples

Last updated on 02/24/2026

Adding User Messages to Your Cmdlet

Cmdlets can write several kinds of messages that can be displayed to the user by the Windows PowerShell runtime. These messages include the following types:

- Verbose messages that contain general user information.
- Debug messages that contain troubleshooting information.
- Warning messages that contain a notification that the cmdlet is about to perform an operation that can have unexpected results.
- Progress report messages that contain information about how much work the cmdlet has completed when performing an operation that takes a long time.

There are no limits to the number of messages that your cmdlet can write or the type of messages that your cmdlet writes. Each message is written by making a specific call from within the input processing method of your cmdlet.

Defining the Cmdlet

The first step in cmdlet creation is always naming the cmdlet and declaring the .NET class that implements the cmdlet. Any sort of cmdlet can write user notifications from its input processing methods; so, in general, you can name this cmdlet using any verb that indicates what system modifications the cmdlet performs. For more information about approved cmdlet verbs, see Cmdlet Verb Names.

The Stop-Proc cmdlet is designed to modify the system; therefore, the System.Management.Automation.CmdletAttribute declaration for the .NET class must include the `SupportsShouldProcess` attribute keyword and be set to `true`.

The following code is the definition for this Stop-Proc cmdlet class. For more information about this definition, see Creating a Cmdlet that Modifies the System.

C#

```
[Cmdlet(VerbsLifecycle.Stop, "proc",
        SupportsShouldProcess = true)]
public class StopProcCommand : Cmdlet
```

Defining Parameters for System Modification

The Stop-Proc cmdlet defines three parameters: `Name`, `Force`, and `PassThru`. For more information about defining these parameters, see Creating a Cmdlet that Modifies the System.

Here is the parameter declaration for the Stop-Proc cmdlet.

C#

```
[Parameter(
           Position = 0,
           Mandatory = true,
           ValueFromPipeline = true,
           ValueFromPipelineByPropertyName = true
)]
public string[] Name
{
  get { return processNames; }
  set { processNames = value; }
}
private string[] processNames;

/// <summary>
/// Specify the Force parameter that allows the user to override
/// the ShouldContinue call to force the stop operation. This
/// parameter should always be used with caution.
/// </summary>
[Parameter]
public SwitchParameter Force
{
  get { return force; }
  set { force = value; }
}
private bool force;

/// <summary>
/// Specify the PassThru parameter that allows the user to specify
/// that the cmdlet should pass the process object down the pipeline
/// after the process has been stopped.
/// </summary>
[Parameter]
public SwitchParameter PassThru
{
  get { return passThru; }
  set { passThru = value; }
}
private bool passThru;
```

Overriding an Input Processing Method

Your cmdlet must override an input processing method, most often it will be System.Management.Automation.Cmdlet.ProcessRecord. This Stop-Proc cmdlet overrides the System.Management.Automation.Cmdlet.ProcessRecord input processing method. In this implementation of the Stop-Proc cmdlet, calls are made to write verbose messages, debug messages, and warning messages.

> ⓘ Note
>
> For more information about how this method calls the System.Management.Automation.Cmdlet.ShouldProcess and System.Management.Automation.Cmdlet.ShouldContinue methods, see Creating a Cmdlet that Modifies the System.

Writing a Verbose Message

The System.Management.Automation.Cmdlet.WriteVerbose method is used to write general user-level information that is unrelated to specific error conditions. The system administrator can then use that information to continue processing other commands. In addition, any information written using this method should be localized as needed.

The following code from this Stop-Proc cmdlet shows two calls to the System.Management.Automation.Cmdlet.WriteVerbose method from the override of the System.Management.Automation.Cmdlet.ProcessRecord method.

C#

```
message = String.Format("Attempting to stop process \"{0}\".", name);
WriteVerbose(message);
```

C#

```
message = String.Format("Stopped process \"{0}\", pid {1}.",
                        processName, process.Id);

WriteVerbose(message);
```

Writing a Debug Message

The System.Management.Automation.Cmdlet.WriteDebug method is used to write debug messages that can be used to troubleshoot the operation of the cmdlet. The call is made from

an input processing method.

> ⓘ Note
>
> Windows PowerShell also defines a `Debug` parameter that presents both verbose and debug information. If your cmdlet supports this parameter, it does not need to call System.Management.Automation.Cmdlet.WriteDebug in the same code that calls System.Management.Automation.Cmdlet.WriteVerbose.

The following two sections of code from the sample Stop-Proc cmdlet show calls to the System.Management.Automation.Cmdlet.WriteDebug method from the override of the System.Management.Automation.Cmdlet.ProcessRecord method.

This debug message is written immediately before System.Management.Automation.Cmdlet.ShouldProcess is called.

C#

```
message =
          String.Format("Acquired name for pid {0} : \"{1}\"",
                        process.Id, processName);
WriteDebug(message);
```

This debug message is written immediately before System.Management.Automation.Cmdlet.WriteObject is called.

C#

```
message =
         String.Format("Writing process \"{0}\" to pipeline",
         processName);
WriteDebug(message);
WriteObject(process);
```

Windows PowerShell automatically routes any System.Management.Automation.Cmdlet.WriteDebug calls to the tracing infrastructure and cmdlets. This allows the method calls to be traced to the hosting application, a file, or a debugger without your having to do any extra development work within the cmdlet. The following command-line entry implements a tracing operation.

PS> Trace-Expression Stop-Proc -File proc.log -Command Stop-Proc notepad

Writing a Warning Message

The System.Management.Automation.Cmdlet.WriteWarning method is used to write a warning when the cmdlet is about to perform an operation that might have an unexpected result, for example, overwriting a read-only file.

The following code from the sample Stop-Proc cmdlet shows the call to the System.Management.Automation.Cmdlet.WriteWarning method from the override of the System.Management.Automation.Cmdlet.ProcessRecord method.

C#

```
 if (criticalProcess)
 {
   message =
             String.Format("Stopping the critical process \"{0}\".",
                           processName);
   WriteWarning(message);
} // if (criticalProcess...
```

Writing a Progress Message

The System.Management.Automation.Cmdlet.WriteProgress is used to write progress messages when cmdlet operations take an extended amount of time to complete. A call to System.Management.Automation.Cmdlet.WriteProgress passes a System.Management.Automation.Progressrecord object that is sent to the hosting application for rendering to the user.

> ⓘ Note
>
> This Stop-Proc cmdlet does not include a call to the System.Management.Automation.Cmdlet.WriteProgress method.

The following code is an example of a progress message written by a cmdlet that is attempting to copy an item.

C#

```
int myId = 0;
string myActivity = "Copy-item: Copying *.* to C:\abc";
string myStatus = "Copying file bar.txt";
ProgressRecord pr = new ProgressRecord(myId, myActivity, myStatus);
```

```
WriteProgress(pr);

pr.RecordType = ProgressRecordType.Completed;
WriteProgress(pr);
```

Code Sample

For the complete C# sample code, see StopProcessSample02 Sample.

Define Object Types and Formatting

Windows PowerShell passes information between cmdlets using .NET objects. Consequently, a cmdlet might need to define its own type, or the cmdlet might need to extend an existing type provided by another cmdlet. For more information about defining new types or extending existing types, see Extending Object Types and Formatting.

Building the Cmdlet

After implementing a cmdlet, it must be registered with Windows PowerShell through a Windows PowerShell snap-in. For more information about registering cmdlets, see How to Register Cmdlets, Providers, and Host Applications.

Testing the Cmdlet

When your cmdlet has been registered with Windows PowerShell, you can test it by running it on the command line. Let's test the sample Stop-Proc cmdlet. For more information about using cmdlets from the command line, see the Getting Started with Windows PowerShell.

- The following command-line entry uses Stop-Proc to stop the process named "NOTEPAD", provide verbose notifications, and print debug information.

  PowerShell

  ```
  PS> Stop-Proc -Name notepad -Verbose -Debug
  ```

  The following output appears.

  ```
  VERBOSE: Attempting to stop process " notepad ".
  DEBUG: Acquired name for pid 5584 : "notepad"
  ```

```
Confirm
Continue with this operation?
[Y] Yes  [A] Yes to All  [H] Halt Command  [S] Suspend  [?] Help (default is
"Y"): Y

Confirm
Are you sure you want to perform this action?
Performing operation "Stop-Proc" on Target "notepad (5584)".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default
is "Y"): Y
VERBOSE: Stopped process "notepad", pid 5584.
```

See Also

Create a Cmdlet that Modifies the System

How to Create a Windows PowerShell Cmdlet

Extending Object Types and Formatting

How to Register Cmdlets, Providers, and Host Applications

Windows PowerShell SDK

Last updated on 05/20/2025

Adding Aliases, Wildcard Expansion, and Help to Cmdlet Parameters

This section describes how to add aliases, wildcard expansion, and Help messages to the parameters of the `Stop-Proc` cmdlet (described in Creating a Cmdlet that Modifies the System).

This `Stop-Proc` cmdlet attempts to stop processes that are retrieved using the `Get-Proc` cmdlet (described in Creating Your First Cmdlet).

Defining the Cmdlet

The first step in cmdlet creation is always naming the cmdlet and declaring the .NET class that implements the cmdlet. Because you are writing a cmdlet to change the system, it should be named accordingly. Because this cmdlet stops system processes, it uses the verb **Stop**, defined by the System.Management.Automation.VerbsLifecycle class, with the noun **Proc** to indicate process. For more information about approved cmdlet verbs, see Cmdlet Verb Names.

The following code is the class definition for this `Stop-Proc` cmdlet.

C#

```
[Cmdlet(VerbsLifecycle.Stop, "proc",
        SupportsShouldProcess = true)]
public class StopProcCommand : Cmdlet
```

Defining Parameters for System Modification

Your cmdlet needs to define parameters that support system modifications and user feedback. The cmdlet should define a **Name** parameter or equivalent so that the cmdlet will be able to modify the system by some sort of identifier. In addition, the cmdlet should define the **Force** and **PassThru** parameters. For more information about these parameters, see Creating a Cmdlet that Modifies the System.

Defining a Parameter Alias

A parameter alias can be an alternate name or a well-defined 1-letter or 2-letter short name for a cmdlet parameter. In both cases, the goal of using aliases is to simplify user entry from the

command line. Windows PowerShell supports parameter aliases through the System.Management.Automation.AliasAttribute attribute, which uses the declaration syntax `[Alias()]`.

The following code shows how an alias is added to the **Name** parameter.

C#

```
/// <summary>
/// Specify the mandatory Name parameter used to identify the
/// processes to be stopped.
/// </summary>
[Parameter(
           Position = 0,
           Mandatory = true,
           ValueFromPipeline = true,
           ValueFromPipelineByPropertyName = true,
           HelpMessage = "The name of one or more processes to stop. Wildcards are
permitted."
)]
[Alias("ProcessName")]
public string[] Name
{
  get { return processNames; }
  set { processNames = value; }
}
private string[] processNames;
```

In addition to using the System.Management.Automation.AliasAttribute attribute, the Windows PowerShell runtime performs partial name matching, even if no aliases are specified. For example, if your cmdlet has a **FileName** parameter and that is the only parameter that starts with `F`, the user could enter `Filename`, `Filenam`, `File`, `Fi`, or `F` and still recognize the entry as the **FileName** parameter.

Creating Help for Parameters

Windows PowerShell allows you to create Help for cmdlet parameters. Do this for any parameter used for system modification and user feedback. For each parameter to support Help, you can set the **HelpMessage** attribute keyword in the System.Management.Automation.ParameterAttribute attribute declaration. This keyword defines the text to display to the user for assistance in using the parameter. You can also set the **HelpMessageBaseName** keyword to identify the base name of a resource to use for the message. If you set this keyword, you must also set the **HelpMessageResourceId** keyword to specify the resource identifier.

The following code from this `Stop-Proc` cmdlet defines the **HelpMessage** attribute keyword for the **Name** parameter.

C#

```
/// <summary>
/// Specify the mandatory Name parameter used to identify the
/// processes to be stopped.
/// </summary>
[Parameter(
           Position = 0,
           Mandatory = true,
           ValueFromPipeline = true,
           ValueFromPipelineByPropertyName = true,
           HelpMessage = "The name of one or more processes to stop. Wildcards are
permitted."
)]
```

Overriding an Input Processing Method

Your cmdlet must override an input processing method, most often this will be System.Management.Automation.Cmdlet.ProcessRecord. When modifying the system, the cmdlet should call the System.Management.Automation.Cmdlet.ShouldProcess and System.Management.Automation.Cmdlet.ShouldContinue methods to allow the user to provide feedback before a change is made. For more information about these methods, see Creating a Cmdlet that Modifies the System.

Supporting Wildcard Expansion

To allow the selection of multiple objects, your cmdlet can use the System.Management.Automation.WildcardPattern and System.Management.Automation.WildcardOptions classes to provide wildcard expansion support for parameter input. Examples of wildcard patterns are `lsa*`, `*.txt`, and `[a-c]*`. Use the back-quote character (`` ` ``) as an escape character when the pattern contains a character that should be used literally.

Wildcard expansions of file and path names are examples of common scenarios where the cmdlet may want to allow support for path inputs when the selection of multiple objects is required. A common case is in the file system, where a user wants to see all files residing in the current folder.

You should need a customized wildcard pattern matching implementation only rarely. In this case, your cmdlet should support either the full POSIX 1003.2, 3.13 specification for wildcard expansion or the following simplified subset:

- **Question mark (`?`).** Matches any character at the specified location.
- **Asterisk (`*`).** Matches zero or more characters starting at the specified location.
- **Open bracket (`[`).** Introduces a pattern bracket expression that can contain characters or a range of characters. If a range is required, a hyphen (`-`) is used to indicate the range.
- **Close bracket (`]`).** Ends a pattern bracket expression.
- **Back-quote escape character (`` ` ``).** Indicates that the next character should be taken literally. Be aware that when specifying the back-quote character from the command line (as opposed to specifying it programmatically), the back-quote escape character must be specified twice.

> ⓘ Note
>
> For more information about wildcard patterns, see Supporting Wildcards in Cmdlet Parameters.

The following code shows how to set wildcard options and define the wildcard pattern used for resolving the **Name** parameter for this cmdlet.

C#

```
WildcardOptions options = WildcardOptions.IgnoreCase |
                          WildcardOptions.Compiled;
WildcardPattern wildcard = new WildcardPattern(name,options);
```

The following code shows how to test whether the process name matches the defined wildcard pattern. Notice that, in this case, if the process name does not match the pattern, the cmdlet continues on to get the next process name.

C#

```
if (!wildcard.IsMatch(processName))
{
  continue;
}
```

Code Sample

For the complete C# sample code, see StopProcessSample03 Sample.

Define Object Types and Formatting

Windows PowerShell passes information between cmdlets using .NET objects. Consequently, a cmdlet may need to define its own type, or the cmdlet may need to extend an existing type provided by another cmdlet. For more information about defining new types or extending existing types, see Extending Object Types and Formatting.

Building the Cmdlet

After implementing a cmdlet, it must be registered with Windows PowerShell through a Windows PowerShell snap-in. For more information about registering cmdlets, see How to Register Cmdlets, Providers, and Host Applications.

Testing the Cmdlet

When your cmdlet has been registered with Windows PowerShell, you can test it by running it on the command line. Let's test the sample Stop-Proc cmdlet. For more information about using cmdlets from the command line, see the Getting Started with Windows PowerShell.

- Start Windows PowerShell and use `Stop-Proc` to stop a process using the **ProcessName** alias for the **Name** parameter.

  PowerShell

  ```
  PS> Stop-Proc -ProcessName notepad
  ```

  The following output appears.

  ```
  Confirm
  Are you sure you want to perform this action?
  Performing operation "Stop-Proc" on Target "notepad (3496)".
  [Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default
  is "Y"): Y
  ```

- Make the following entry on the command line. Because the Name parameter is mandatory, you are prompted for it. Entering `!?` brings up the help text associated with the parameter.

PowerShell

```
PS> Stop-Proc
```

The following output appears.

```

Cmdlet Stop-Proc at command pipeline position 1
Supply values for the following parameters:
(Type !? for Help.)
Name[0]: !?
The name of one or more processes to stop. Wildcards are permitted.
Name[0]: notepad
```

- Now make the following entry to stop all processes that match the wildcard pattern `*note*`. You are prompted before stopping each process that matches the pattern.

PowerShell

```
PS> Stop-Proc -Name *note*
```

The following output appears.

```

Confirm
Are you sure you want to perform this action?
Performing operation "Stop-Proc" on Target "notepad (1112)".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default
is "Y"): Y
```

The following output appears.

```

Confirm
Are you sure you want to perform this action?
Performing operation "Stop-Proc" on Target "ONENOTEM (3712)".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default
is "Y"): N
```

The following output appears.

```
```

```
Confirm
Are you sure you want to perform this action?
Performing operation "Stop-Proc" on Target "ONENOTE (3592)".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default
is "Y"): N
```

See Also

- Create a Cmdlet that Modifies the System
- How to Create a Windows PowerShell Cmdlet
- Extending Object Types and Formatting
- How to Register Cmdlets, Providers, and Host Applications
- Supporting Wildcards in Cmdlet Parameters
- Windows PowerShell SDK

Last updated on 05/20/2025

Adding Parameter Sets to a Cmdlet

Things to Know About Parameter Sets

Windows PowerShell defines a parameter set as a group of parameters that operate together. By grouping the parameters of a cmdlet, you can create a single cmdlet that can change its functionality based on what group of parameters the user specifies.

An example of a cmdlet that uses two parameter sets to define different functionalities is the `Get-EventLog` cmdlet that is provided by Windows PowerShell. This cmdlet returns different information when the user specifies the `List` or `LogName` parameter. If the `LogName` parameter is specified, the cmdlet returns information about the events in a given event log. If the `List` parameter is specified, the cmdlet returns information about the log files themselves (not the event information they contain). In this case, the `List` and `LogName` parameters identify two separate parameter sets.

Two important things to remember about parameter sets is that the Windows PowerShell runtime uses only one parameter set for a particular input, and that each parameter set must have at least one parameter that is unique for that parameter set.

To illustrate that last point, this Stop-Proc cmdlet uses three parameter sets: `ProcessName`, `ProcessId`, and `InputObject`. Each of these parameter sets has one parameter that is not in the other parameter sets. The parameter sets could share other parameters, but the cmdlet uses the unique parameters `ProcessName`, `ProcessId`, and `InputObject` to identify which set of parameters that the Windows PowerShell runtime should use.

Declaring the Cmdlet Class

The first step in cmdlet creation is always naming the cmdlet and declaring the .NET class that implements the cmdlet. For this cmdlet, the lifecycle verb "Stop" is used because the cmdlet stops system processes. The noun name "Proc" is used because the cmdlet works on processes. In the declaration below, note that the cmdlet verb and noun name are reflected in the name of the cmdlet class.

> ⓘ **Note**

For more information about approved cmdlet verb names, see Cmdlet Verb Names.

The following code is the class definition for this Stop-Proc cmdlet.

C#

```
[Cmdlet(VerbsLifecycle.Stop, "Proc",
        DefaultParameterSetName = "ProcessId",
        SupportsShouldProcess = true)]
public class StopProcCommand : PSCmdlet
```

VB

```
<Cmdlet(VerbsLifecycle.Stop, "Proc", DefaultParameterSetName:="ProcessId", _
SupportsShouldProcess:=True)> _
Public Class StopProcCommand
    Inherits PSCmdlet
```

Declaring the Parameters of the Cmdlet

This cmdlet defines three parameters needed as input to the cmdlet (these parameters also define the parameter sets), as well as a `Force` parameter that manages what the cmdlet does and a `PassThru` parameter that determines whether the cmdlet sends an output object through the pipeline. By default, this cmdlet does not pass an object through the pipeline. For more information about these last two parameters, see Creating a Cmdlet that Modifies the System.

Declaring the Name Parameter

This input parameter allows the user to specify the names of the processes to be stopped. Note that the `ParameterSetName` attribute keyword of the System.Management.Automation.ParameterAttribute attribute specifies the `ProcessName` parameter set for this parameter.

C#

```
[Parameter(
   Position = 0,
   ParameterSetName = "ProcessName",
   Mandatory = true,
   ValueFromPipeline = true,
   ValueFromPipelineByPropertyName = true,
   HelpMessage = "The name of one or more processes to stop. Wildcards are
permitted."
)]
```

```
[Alias("ProcessName")]
public string[] Name
{
    get { return processNames; }
    set { processNames = value; }
}
private string[] processNames;
```

VB

```
<Parameter(Position:=0, ParameterSetName:="ProcessName", _
Mandatory:=True, _
ValueFromPipeline:=True, ValueFromPipelineByPropertyName:=True, _
HelpMessage:="The name of one or more processes to stop. " & _
    "Wildcards are permitted."), [Alias]("ProcessName")> _
Public Property Name() As String()
    Get
        Return processNames
    End Get
    Set(ByVal value As String())
        processNames = value
    End Set
End Property

Private processNames() As String
```

Note also that the alias "ProcessName" is given to this parameter.

Declaring the Id Parameter

This input parameter allows the user to specify the identifiers of the processes to be stopped. Note that the `ParameterSetName` attribute keyword of the System.Management.Automation.ParameterAttribute attribute specifies the `ProcessId` parameter set.

C#

```
[Parameter(
           ParameterSetName = "ProcessId",
           Mandatory = true,
           ValueFromPipelineByPropertyName = true,
           ValueFromPipeline = true
)]
[Alias("ProcessId")]
public int[] Id
{
  get { return processIds; }
  set { processIds = value; }
```

```
}
private int[] processIds;
```

VB

```
<Parameter(ParameterSetName:="ProcessId", _
Mandatory:=True, _
ValueFromPipelineByPropertyName:=True, _
ValueFromPipeline:=True), [Alias]("ProcessId")> _
Public Property Id() As Integer()
    Get
        Return processIds
    End Get
    Set(ByVal value As Integer())
        processIds = value
    End Set
End Property
Private processIds() As Integer
```

Note also that the alias "ProcessId" is given to this parameter.

Declaring the InputObject Parameter

This input parameter allows the user to specify an input object that contains information about the processes to be stopped. Note that the `ParameterSetName` attribute keyword of the System.Management.Automation.ParameterAttribute attribute specifies the `InputObject` parameter set for this parameter.

C#

```
[Parameter(
           ParameterSetName = "InputObject",
           Mandatory = true,
           ValueFromPipeline = true)]
public Process[] InputObject
{
  get { return inputObject; }
  set { inputObject = value; }
}
private Process[] inputObject;
```

VB

```
<Parameter(ParameterSetName:="InputObject", _
Mandatory:=True, ValueFromPipeline:=True)> _
Public Property InputObject() As Process()
    Get
        Return myInputObject
```

```
    End Get
    Set(ByVal value As Process())
        myInputObject = value
    End Set
End Property
Private myInputObject() As Process
```

Note also that this parameter has no alias.

Declaring Parameters in Multiple Parameter Sets

Although there must be a unique parameter for each parameter set, parameters can belong to more than one parameter set. In these cases, give the shared parameter a System.Management.Automation.ParameterAttribute attribute declaration for each set to which that the parameter belongs. If a parameter is in all parameter sets, you only have to declare the parameter attribute once and do not need to specify the parameter set name.

Overriding an Input Processing Method

Every cmdlet must override an input processing method, most often this will be the System.Management.Automation.Cmdlet.ProcessRecord method. In this cmdlet, the System.Management.Automation.Cmdlet.ProcessRecord method is overridden so that the cmdlet can process any number of processes. It contains a Select statement that calls a different method based on which parameter set the user has specified.

C#

```
protected override void ProcessRecord()
{
  switch (ParameterSetName)
  {
    case "ProcessName":
         ProcessByName();
         break;

    case "ProcessId":
         ProcessById();
         break;

    case "InputObject":
         foreach (Process process in inputObject)
         {
           SafeStopProcess(process);
         }
         break;

    default:
```

```
            throw new ArgumentException("Bad ParameterSet Name");
    } // switch (ParameterSetName...
} // ProcessRecord
```

VB

```
Protected Overrides Sub ProcessRecord()
    Select Case ParameterSetName
        Case "ProcessName"
            ProcessByName()

        Case "ProcessId"
            ProcessById()

        Case "InputObject"
            Dim process As Process
            For Each process In myInputObject
                SafeStopProcess(process)
            Next process

        Case Else
            Throw New ArgumentException("Bad ParameterSet Name")
    End Select

End Sub 'ProcessRecord ' ProcessRecord
```

The Helper methods called by the Select statement are not described here, but you can see their implementation in the complete code sample in the next section.

Code Sample

For the complete C# sample code, see StopProcessSample04 Sample.

Defining Object Types and Formatting

Windows PowerShell passes information between cmdlets using .NET objects. Consequently, a cmdlet might need to define its own type, or the cmdlet might need to extend an existing type provided by another cmdlet. For more information about defining new types or extending existing types, see Extending Object Types and Formatting.

Building the Cmdlet

After implementing a cmdlet, you must register it with Windows PowerShell through a Windows PowerShell snap-in. For more information about registering cmdlets, see How to Register Cmdlets, Providers, and Host Applications.

Testing the Cmdlet

When your cmdlet has been registered with Windows PowerShell, test it by running it on the command line. Here are some tests that show how the `ProcessId` and `InputObject` parameters can be used to test their parameter sets to stop a process.

- With Windows PowerShell started, run the Stop-Proc cmdlet with the `ProcessId` parameter set to stop a process based on its identifier. In this case, the cmdlet is using the `ProcessId` parameter set to stop the process.

```
PS> Stop-Proc -Id 444
Confirm
Are you sure you want to perform this action?
Performing operation "Stop-Proc" on Target "notepad (444)".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default
is "Y"): Y
```

- With Windows PowerShell started, run the Stop-Proc cmdlet with the `InputObject` parameter set to stop processes on the Notepad object retrieved by the `Get-Process` command.

```
PS> Get-Process notepad | Stop-Proc
Confirm
Are you sure you want to perform this action?
Performing operation "Stop-Proc" on Target "notepad (444)".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default
is "Y"): N
```

See Also

Creating a Cmdlet that Modifies the System

How to Create a Windows PowerShell Cmdlet

Extending Object Types and Formatting

How to Register Cmdlets, Providers, and Host Applications

Windows PowerShell SDK

Last updated on 05/20/2025

SelectStr Tutorial

This section provides a tutorial for creating the Select-Str cmdlet, which is very similar to the Select-String cmdlet provided by Windows PowerShell. This tutorial provides fragments of code that illustrate how cmdlets are implemented, and an explanation of the code.

Topic in this Tutorial

Creating a Cmdlet to Access a Data Store This section describes how to create a cmdlet that selects strings that are in a file or object.

See Also

Creating a Cmdlet to Access a Data Store

Windows PowerShell SDK

Last updated on 05/20/2025

Creating a Cmdlet to Access a Data Store

This section describes how to create a cmdlet that accesses stored data by way of a Windows PowerShell provider. This type of cmdlet uses the Windows PowerShell provider infrastructure of the Windows PowerShell runtime and, therefore, the cmdlet class must derive from the System.Management.Automation.PSCmdlet base class.

The Select-Str cmdlet described here can locate and select strings in a file or object. The patterns used to identify the string can be specified explicitly through the `Path` parameter of the cmdlet or implicitly through the `Script` parameter.

The cmdlet is designed to use any Windows PowerShell provider that derives from System.Management.Automation.Provider.IContentCmdletProvider. For example, the cmdlet can specify the FileSystem provider or the Variable provider that is provided by Windows PowerShell. For more information aboutWindows PowerShell providers, see Designing Your Windows PowerShell provider.

Defining the Cmdlet Class

The first step in cmdlet creation is always naming the cmdlet and declaring the .NET class that implements the cmdlet. This cmdlet detects certain strings, so the verb name chosen here is "Select", defined by the System.Management.Automation.VerbsCommon class. The noun name "Str" is used because the cmdlet acts upon strings. In the declaration below, note that the cmdlet verb and noun name are reflected in the name of the cmdlet class. For more information about approved cmdlet verbs, see Cmdlet Verb Names.

The .NET class for this cmdlet must derive from the System.Management.Automation.PSCmdlet base class, because it provides the support needed by the Windows PowerShell runtime to expose the Windows PowerShell provider infrastructure. Note that this cmdlet also makes use of the .NET Framework regular expressions classes, such as System.Text.RegularExpressions.Regex.

The following code is the class definition for this Select-Str cmdlet.

C#

```
[Cmdlet(VerbsCommon.Select, "Str", DefaultParameterSetName="PatternParameterSet")]
public class SelectStringCommand : PSCmdlet
```

This cmdlet defines a default parameter set by adding the `DefaultParameterSetName` attribute keyword to the class declaration. The default parameter set `PatternParameterSet` is used when the `Script` parameter is not specified. For more information about this parameter set, see the `Pattern` and `Script` parameter discussion in the following section.

Defining Parameters for Data Access

This cmdlet defines several parameters that allow the user to access and examine stored data. These parameters include a `Path` parameter that indicates the location of the data store, a `Pattern` parameter that specifies the pattern to be used in the search, and several other parameters that support how the search is performed.

> ⓘ Note
>
> For more information about the basics of defining parameters, see [Adding Parameters that Process Command Line Input][04].

Declaring the Path Parameter

To locate the data store, this cmdlet must use a Windows PowerShell path to identify the Windows PowerShell provider that is designed to access the data store. Therefore, it defines a `Path` parameter of type string array to indicate the location of the provider.

C#

```
[Parameter(
           Position = 0,
           ParameterSetName = "ScriptParameterSet",
           Mandatory = true)]
[Parameter(
           Position = 0,
           ParameterSetName = "PatternParameterSet",
           ValueFromPipeline = true,
           Mandatory = true)]
           [Alias("PSPath")]
public string[] Path
{
  get { return paths; }
  set { paths = value; }
}
private string[] paths;
```

Note that this parameter belongs to two different parameter sets and that it has an alias.

Two System.Management.Automation.ParameterAttribute attributes declare that the `Path` parameter belongs to the `ScriptParameterSet` and the `PatternParameterSet`. For more information about parameter sets, see Adding Parameter Sets to a Cmdlet.

The System.Management.Automation.AliasAttribute attribute declares a `PSPath` alias for the `Path` parameter. Declaring this alias is strongly recommended for consistency with other cmdlets that access Windows PowerShell providers. For more information aboutWindows PowerShell paths, see "PowerShell Path Concepts" in How Windows PowerShell Works.

Declaring the Pattern Parameter

To specify the patterns to search for, this cmdlet declares a `Pattern` parameter that is an array of strings. A positive result is returned when any of the patterns are found in the data store. Note that these patterns can be compiled into an array of compiled regular expressions or an array of wildcard patterns used for literal searches.

C#

```
[Parameter(
           Position = 1,
           ParameterSetName = "PatternParameterSet",
           Mandatory = true)]
public string[] Pattern
{
  get { return patterns; }
  set { patterns = value; }
}
private string[] patterns;
private Regex[] regexPattern;
private WildcardPattern[] wildcardPattern;
```

When this parameter is specified, the cmdlet uses the default parameter set `PatternParameterSet`. In this case, the cmdlet uses the patterns specified here to select strings. In contrast, the `Script` parameter could also be used to provide a script that contains the patterns. The `Script` and `Pattern` parameters define two separate parameter sets, so they are mutually exclusive.

Declaring Search Support Parameters

This cmdlet defines the following support parameters that can be used to modify the search capabilities of the cmdlet.

The `Script` parameter specifies a script block that can be used to provide an alternate search mechanism for the cmdlet. The script must contain the patterns used for matching and return a System.Management.Automation.PSObject object. Note that this parameter is also the unique parameter that identifies the `ScriptParameterSet` parameter set. When the Windows PowerShell runtime sees this parameter, it uses only parameters that belong to the `ScriptParameterSet` parameter set.

C#

```
[Parameter(
           Position = 1,
           ParameterSetName = "ScriptParameterSet",
           Mandatory = true)]
public ScriptBlock Script
{
  set { script = value; }
  get { return script; }
}
ScriptBlock script;
```

The `SimpleMatch` parameter is a `[switch]` parameter that indicates whether the cmdlet is to explicitly match the patterns as they are supplied. When the user specifies the parameter at the command line (`true`), the cmdlet uses the patterns as they are supplied. If the parameter is not specified (`false`), the cmdlet uses regular expressions. The default for this parameter is `false`.

C#

```
[Parameter]
public SwitchParameter SimpleMatch
{
  get { return simpleMatch; }
  set { simpleMatch = value; }
}
private bool simpleMatch;
```

The `CaseSensitive` parameter is a `[switch]` parameter that indicates whether a case-sensitive search is performed. When the user specifies the parameter at the command line (`true`), the cmdlet checks for the uppercase and lowercase of characters when comparing patterns. If the parameter is not specified (`false`), the cmdlet does not distinguish between uppercase and lowercase. For example "MyFile" and "myfile" would both be returned as positive hits. The default for this parameter is `false`.

C#

```
[Parameter]
public SwitchParameter CaseSensitive
{
  get { return caseSensitive; }
  set { caseSensitive = value; }
}
private bool caseSensitive;
```

The `Exclude` and `Include` parameters identify items that are explicitly excluded from or included in the search. By default, the cmdlet will search all items in the data store. However, to limit the search performed by the cmdlet, these parameters can be used to explicitly indicate items to be included in the search or omitted.

C#

```
[Parameter]
public SwitchParameter CaseSensitive
{
  get { return caseSensitive; }
  set { caseSensitive = value; }
}
private bool caseSensitive;
```

C#

```
[Parameter]
[ValidateNotNullOrEmpty]
public string[] Include
{
  get
  {
    return includeStrings;
  }
  set
  {
    includeStrings = value;

    this.include = new WildcardPattern[includeStrings.Length];
    for (int i = 0; i < includeStrings.Length; i++)
    {
      this.include[i] = new WildcardPattern(includeStrings[i],
WildcardOptions.IgnoreCase);
    }
  }
}

internal string[] includeStrings = null;
internal WildcardPattern[] include = null;
```

Declaring Parameter Sets

This cmdlet uses two parameter sets (`ScriptParameterSet` and `PatternParameterSet`, which is the default) as the names of two parameter sets used in data access. `PatternParameterSet` is the default parameter set and is used when the `Pattern` parameter is specified. `ScriptParameterSet` is used when the user specifies an alternate search mechanism through the `Script` parameter. For more information about parameter sets, see Adding Parameter Sets to a Cmdlet.

Overriding Input Processing Methods

Cmdlets must override one or more of the input processing methods for the System.Management.Automation.PSCmdlet class. For more information about the input processing methods, see Creating Your First Cmdlet.

This cmdlet overrides the System.Management.Automation.Cmdlet.BeginProcessing method to build an array of compiled regular expressions at startup. This increases performance during searches that do not use simple matching.

C#

```
protected override void BeginProcessing()
{
  WriteDebug("Validating patterns.");
  if (patterns != null)
  {
    foreach(string pattern in patterns)
    {
      if (pattern == null)
      ThrowTerminatingError(new ErrorRecord(
                            new ArgumentNullException(
                            "Search pattern cannot be null."),
                            "NullSearchPattern",
                            ErrorCategory.InvalidArgument,
                            pattern)
                            );
    }

    WriteVerbose("Search pattern(s) are valid.");

    // If a simple match is not specified, then
    // compile the regular expressions once.
    if (!simpleMatch)
    {
      WriteDebug("Compiling search regular expressions.");

      RegexOptions regexOptions = RegexOptions.Compiled;
```

```
      if (!caseSensitive)
         regexOptions |= RegexOptions.Compiled;
      regexPattern = new Regex[patterns.Length];

      for (int i = 0; i < patterns.Length; i++)
      {
        try
        {
          regexPattern[i] = new Regex(patterns[i], regexOptions);
        }
        catch (ArgumentException ex)
        {
          ThrowTerminatingError(new ErrorRecord(
                        ex,
                        "InvalidRegularExpression",
                        ErrorCategory.InvalidArgument,
                        patterns[i]
                    ));
        }
      } //Loop through patterns to create RegEx objects.

      WriteVerbose("Pattern(s) compiled into regular expressions.");
    }// If not a simple match.

    // If a simple match is specified, then compile the
    // wildcard patterns once.
    else
    {
      WriteDebug("Compiling search wildcards.");

      WildcardOptions wildcardOptions = WildcardOptions.Compiled;

      if (!caseSensitive)
      {
        wildcardOptions |= WildcardOptions.IgnoreCase;
      }

      wildcardPattern = new WildcardPattern[patterns.Length];
      for (int i = 0; i < patterns.Length; i++)
      {
        wildcardPattern[i] =
                    new WildcardPattern(patterns[i], wildcardOptions);
      }

      WriteVerbose("Pattern(s) compiled into wildcard expressions.");
    }// If match is a simple match.
  }// If valid patterns are available.
}// End of function BeginProcessing().
```

This cmdlet also overrides the System.Management.Automation.Cmdlet.ProcessRecord method to process the string selections that the user makes on the command line. It writes the results of string selection in the form of a custom object by calling a private **MatchString** method.

C#

```
protected override void ProcessRecord()
{
  UInt64 lineNumber = 0;
  MatchInfo result;
  ArrayList nonMatches = new ArrayList();

  // Walk the list of paths and search the contents for
  // any of the specified patterns.
  foreach (string psPath in paths)
  {
    // Once the filepaths are expanded, we may have more than one
    // path, so process all referenced paths.
    foreach(PathInfo path in
            SessionState.Path.GetResolvedPSPathFromPSPath(psPath)
           )
    {
      WriteVerbose("Processing path " + path.Path);

      // Check if the path represents one of the items to be
      // excluded. If so, continue to next path.
      if (!MeetsIncludeExcludeCriteria(path.ProviderPath))
         continue;

      // Get the content reader for the item(s) at the
      // specified path.
      Collection<IContentReader> readerCollection = null;
      try
      {
        readerCollection =
                    this.InvokeProvider.Content.GetReader(path.Path);
      }
      catch (PSNotSupportedException ex)
      {
        WriteError(new ErrorRecord(ex,
                   "ContentAccessNotSupported",
                    ErrorCategory.NotImplemented,
                    path.Path)
                   );
        return;
      }

      foreach(IContentReader reader in readerCollection)
      {
        // Reset the line number for this path.
        lineNumber = 0;

        // Read in a single block (line in case of a file)
        // from the object.
        IList items = reader.Read(1);

        // Read and process one block(line) at a time until
        // no more blocks(lines) exist.
```

```
        while (items != null && items.Count == 1)
        {
          // Increment the line number each time a line is
          // processed.
          lineNumber++;

          String message = String.Format("Testing line {0} : {1}",
                                         lineNumber, items[0]);

          WriteDebug(message);

          result = SelectString(items[0]);

          if (result != null)
          {
            result.Path = path.Path;
            result.LineNumber = lineNumber;

            WriteObject(result);
          }
          else
          {
            // Add the block(line) that did not match to the
            // collection of non matches , which will be stored
            // in the SessionState variable $NonMatches
            nonMatches.Add(items[0]);
          }

          // Get the next line from the object.
          items = reader.Read(1);

        }// While loop for reading one line at a time.
      }// Foreach loop for reader collection.
    }// Foreach loop for processing referenced paths.
  }// Foreach loop for walking of path list.

  // Store the list of non-matches in the
  // session state variable $NonMatches.
  try
  {
    this.SessionState.PSVariable.Set("NonMatches", nonMatches);
  }
  catch (SessionStateUnauthorizedAccessException ex)
  {
    WriteError(new ErrorRecord(ex,
               "CannotWriteVariableNonMatches",
               ErrorCategory.InvalidOperation,
               nonMatches)
              );
  }

}// End of protected override void ProcessRecord().
```

Accessing Content

Your cmdlet must open the provider indicated by the Windows PowerShell path so that it can access the data. The System.Management.Automation.SessionState object for the runspace is used for access to the provider, while the System.Management.Automation.PSCmdlet.InvokeProvider* property of the cmdlet is used to open the provider. Access to content is provided by retrieval of the System.Management.Automation.ProviderIntrinsics object for the provider opened.

This sample Select-Str cmdlet uses the System.Management.Automation.ProviderIntrinsics.Content* property to expose the content to scan. It can then call the System.Management.Automation.ContentCmdletProviderIntrinsics.GetReader* method, passing the required Windows PowerShell path.

Code Sample

The following code shows the implementation of this version of this Select-Str cmdlet. Note that this code includes the cmdlet class, private methods used by the cmdlet, and the Windows PowerShell snap-in code used to register the cmdlet. For more information about registering the cmdlet, see Building the Cmdlet.

C#

```
//
// Copyright (c) 2006 Microsoft Corporation. All rights reserved.
//
// THIS CODE AND INFORMATION IS PROVIDED "AS IS" WITHOUT WARRANTY OF
// ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO
// THE IMPLIED WARRANTIES OF MERCHANTABILITY AND/OR FITNESS FOR A
// PARTICULAR PURPOSE.
//
using System;
using System.Text.RegularExpressions;
using System.Collections;
using System.Collections.ObjectModel;
using System.Management.Automation;
using System.Management.Automation.Provider;
using System.ComponentModel;

namespace Microsoft.Samples.PowerShell.Commands
{
  #region SelectStringCommand
  /// <summary>
  /// This cmdlet searches through PSObjects for particular patterns.
```

```
  /// </summary>
  /// <remarks>
  /// This cmdlet can be used to search any object, such as a file or a
  /// variable, whose provider exposes methods for reading and writing
  /// content.
  /// </remarks>
  [Cmdlet(VerbsCommon.Select, "Str",
DefaultParameterSetName="PatternParameterSet")]
  public class SelectStringCommand : PSCmdlet
  {
    #region Parameters
    /// <summary>
    /// Declare a Path parameter that specifies where the data is stored.
    /// This parameter must specify a PowerShell that indicates the
    /// PowerShell provider that is used to access the objects to be
    /// searched for matching patterns. This parameter should also have
    /// a PSPath alias to provide consistency with other cmdlets that use
    /// PowerShell providers.
    /// </summary>
    /// <value>Path of the object(s) to search.</value>
    [Parameter(
               Position = 0,
               ParameterSetName = "ScriptParameterSet",
               Mandatory = true)]
    [Parameter(
               Position = 0,
               ParameterSetName = "PatternParameterSet",
               ValueFromPipeline = true,
               Mandatory = true)]
               [Alias("PSPath")]
    public string[] Path
    {
      get { return paths; }
      set { paths = value; }
    }
    private string[] paths;

    /// <summary>
    /// Declare a Pattern parameter that specifies the pattern(s)
    /// used to find matching patterns in the string representation
    /// of the objects. A positive result will be returned
    /// if any of the patterns are found in the objects.
    /// </summary>
    /// <remarks>
    /// The patterns will be compiled into an array of wildcard
    /// patterns for a simple match (literal string matching),
    /// or the patterns will be converted into an array of compiled
    /// regular expressions.
    /// </remarks>
    /// <value>Array of patterns to search.</value>
    [Parameter(
               Position = 1,
               ParameterSetName = "PatternParameterSet",
               Mandatory = true)]
    public string[] Pattern
```

```
    {
      get { return patterns; }
      set { patterns = value; }
    }
    private string[] patterns;
    private Regex[] regexPattern;
    private WildcardPattern[] wildcardPattern;

    /// <summary>
    /// Declare a Script parameter that specifies a script block
    /// that is called to perform the matching operations
    /// instead of the matching performed by the cmdlet.
    /// </summary>
    /// <value>Script block that will be called for matching</value>
    [Parameter(
               Position = 1,
               ParameterSetName = "ScriptParameterSet",
               Mandatory = true)]
    public ScriptBlock Script
    {
      set { script = value; }
      get { return script; }
    }
    ScriptBlock script;

    /// <summary>
    /// Declare a switch parameter that specifies if the pattern(s) are used
    /// literally. If not (default), searching is
    /// done using regular expressions.
    /// </summary>
    /// <value>If True, a literal pattern is used.</value>
    [Parameter]
    public SwitchParameter SimpleMatch
    {
      get { return simpleMatch; }
      set { simpleMatch = value; }
    }
    private bool simpleMatch;

    /// <summary>
    /// Declare a switch parameter that specifies if a case-sensitive
    /// search is performed.  If not (default), a case-insensitive search
    /// is performed.
    /// </summary>
    /// <value>If True, a case-sensitive search is made.</value>
    [Parameter]
    public SwitchParameter CaseSensitive
    {
      get { return caseSensitive; }
      set { caseSensitive = value; }
    }
    private bool caseSensitive;

    /// <summary>
    /// Declare an Include parameter that species which
```

```
    /// specific items are searched.  When this parameter
    /// is used, items that are not listed here are omitted
    /// from the search.
    /// </summary>
    [Parameter]
    [ValidateNotNullOrEmpty]
    public string[] Include
    {
      get
      {
        return includeStrings;
      }
      set
      {
        includeStrings = value;

        this.include = new WildcardPattern[includeStrings.Length];
        for (int i = 0; i < includeStrings.Length; i++)
        {
          this.include[i] = new WildcardPattern(includeStrings[i],
WildcardOptions.IgnoreCase);
        }
      }
    }

    internal string[] includeStrings = null;
    internal WildcardPattern[] include = null;

    /// <summary>
    /// Declare an Exclude parameter that species which
    /// specific items are omitted from the search.
    /// </summary>
    ///
    [Parameter]
    [ValidateNotNullOrEmpty]
    public string[] Exclude
    {
      get
      {
        return excludeStrings;
      }
      set
      {
        excludeStrings = value;

        this.exclude = new WildcardPattern[excludeStrings.Length];
        for (int i = 0; i < excludeStrings.Length; i++)
        {
          this.exclude[i] = new WildcardPattern(excludeStrings[i],
WildcardOptions.IgnoreCase);
        }
      }
    }
    internal string[] excludeStrings;
    internal WildcardPattern[] exclude;
```

```
#endregion Parameters

#region Overrides
/// <summary>
/// If regular expressions are used for pattern matching,
/// then build an array of compiled regular expressions
/// at startup. This increases performance during scanning
/// operations when simple matching is not used.
/// </summary>
protected override void BeginProcessing()
{
  WriteDebug("Validating patterns.");
  if (patterns != null)
  {
    foreach(string pattern in patterns)
    {
      if (pattern == null)
      ThrowTerminatingError(new ErrorRecord(
                           new ArgumentNullException(
                           "Search pattern cannot be null."),
                           "NullSearchPattern",
                           ErrorCategory.InvalidArgument,
                           pattern)
                           );
    }

    WriteVerbose("Search pattern(s) are valid.");

    // If a simple match is not specified, then
    // compile the regular expressions once.
    if (!simpleMatch)
    {
      WriteDebug("Compiling search regular expressions.");

      RegexOptions regexOptions = RegexOptions.Compiled;
      if (!caseSensitive)
         regexOptions |= RegexOptions.Compiled;
      regexPattern = new Regex[patterns.Length];

      for (int i = 0; i < patterns.Length; i++)
      {
        try
        {
          regexPattern[i] = new Regex(patterns[i], regexOptions);
        }
        catch (ArgumentException ex)
        {
          ThrowTerminatingError(new ErrorRecord(
                       ex,
                       "InvalidRegularExpression",
                       ErrorCategory.InvalidArgument,
                       patterns[i]
                    ));
        }
```

```
        } //Loop through patterns to create RegEx objects.

        WriteVerbose("Pattern(s) compiled into regular expressions.");
      }// If not a simple match.

      // If a simple match is specified, then compile the
      // wildcard patterns once.
      else
      {
        WriteDebug("Compiling search wildcards.");

        WildcardOptions wildcardOptions = WildcardOptions.Compiled;

        if (!caseSensitive)
        {
          wildcardOptions |= WildcardOptions.IgnoreCase;
        }

        wildcardPattern = new WildcardPattern[patterns.Length];
        for (int i = 0; i < patterns.Length; i++)
        {
          wildcardPattern[i] =
                      new WildcardPattern(patterns[i], wildcardOptions);
        }

        WriteVerbose("Pattern(s) compiled into wildcard expressions.");
      }// If match is a simple match.
    }// If valid patterns are available.
  }// End of function BeginProcessing().

  /// <summary>
  /// Process the input and search for the specified patterns.
  /// </summary>
  protected override void ProcessRecord()
  {
    UInt64 lineNumber = 0;
    MatchInfo result;
    ArrayList nonMatches = new ArrayList();

    // Walk the list of paths and search the contents for
    // any of the specified patterns.
    foreach (string psPath in paths)
    {
      // Once the filepaths are expanded, we may have more than one
      // path, so process all referenced paths.
      foreach(PathInfo path in
              SessionState.Path.GetResolvedPSPathFromPSPath(psPath)
             )
      {
        WriteVerbose("Processing path " + path.Path);

        // Check if the path represents one of the items to be
        // excluded. If so, continue to next path.
        if (!MeetsIncludeExcludeCriteria(path.ProviderPath))
           continue;
```

```
// Get the content reader for the item(s) at the
// specified path.
Collection<IContentReader> readerCollection = null;
try
{
  readerCollection =
              this.InvokeProvider.Content.GetReader(path.Path);
}
catch (PSNotSupportedException ex)
{
  WriteError(new ErrorRecord(ex,
             "ContentAccessNotSupported",
              ErrorCategory.NotImplemented,
              path.Path)
             );
  return;
}

foreach(IContentReader reader in readerCollection)
{
  // Reset the line number for this path.
  lineNumber = 0;

  // Read in a single block (line in case of a file)
  // from the object.
  IList items = reader.Read(1);

  // Read and process one block(line) at a time until
  // no more blocks(lines) exist.
  while (items != null && items.Count == 1)
  {
    // Increment the line number each time a line is
    // processed.
    lineNumber++;

    String message = String.Format("Testing line {0} : {1}",
                                   lineNumber, items[0]);

    WriteDebug(message);

    result = SelectString(items[0]);

    if (result != null)
    {
      result.Path = path.Path;
      result.LineNumber = lineNumber;

      WriteObject(result);
    }
    else
    {
      // Add the block(line) that did not match to the
      // collection of non matches , which will be stored
      // in the SessionState variable $NonMatches
```

```
              nonMatches.Add(items[0]);
            }

            // Get the next line from the object.
            items = reader.Read(1);

          }// While loop for reading one line at a time.
        }// Foreach loop for reader collection.
      }// Foreach loop for processing referenced paths.
    }// Foreach loop for walking of path list.

    // Store the list of non-matches in the
    // session state variable $NonMatches.
    try
    {
      this.SessionState.PSVariable.Set("NonMatches", nonMatches);
    }
    catch (SessionStateUnauthorizedAccessException ex)
    {
      WriteError(new ErrorRecord(ex,
                 "CannotWriteVariableNonMatches",
                 ErrorCategory.InvalidOperation,
                 nonMatches)
                );
    }

  }// End of protected override void ProcessRecord().
  #endregion Overrides

  #region PrivateMethods
  /// <summary>
  /// Check for a match using the input string and the pattern(s)
  /// specified.
  /// </summary>
  /// <param name="input">The string to test.</param>
  /// <returns>MatchInfo object containing information about
  /// result of a match</returns>
  private MatchInfo SelectString(object input)
  {
    string line = null;

    try
    {
      // Convert the object to a string type
      // safely using language support methods
      line = (string)LanguagePrimitives.ConvertTo(
                                                  input,
                                                  typeof(string)
                                                  );
      line = line.Trim(' ','\t');
    }
    catch (PSInvalidCastException ex)
    {
      WriteError(new ErrorRecord(
                 ex,
```

```
                "CannotCastObjectToString",
                ErrorCategory.InvalidOperation,
                input)
                );

    return null;
  }

  MatchInfo result = null;

  // If a scriptblock has been specified, call it
  // with the path for processing.  It will return
  // one object.
  if (script != null)
  {
    WriteDebug("Executing script block.");

    Collection<PSObject> psObjects =
                         script.Invoke(
                                      line,
                                      simpleMatch,
                                      caseSensitive
                                     );

    foreach (PSObject psObject in psObjects)
    {
      if (LanguagePrimitives.IsTrue(psObject))
      {
        result = new MatchInfo();
        result.Line = line;
        result.IgnoreCase = !caseSensitive;

        break;
      } //End of If.
    } //End ForEach loop.
  } // End of If if script exists.

  // If script block exists, see if this line matches any
  // of the match patterns.
  else
  {
    int patternIndex = 0;

    while (patternIndex < patterns.Length)
    {
      if ((simpleMatch &&
          wildcardPattern[patternIndex].IsMatch(line))
          || (regexPattern != null
          && regexPattern[patternIndex].IsMatch(line))
         )
      {
        result = new MatchInfo();
        result.IgnoreCase = !caseSensitive;
        result.Line = line;
        result.Pattern = patterns[patternIndex];
```

```
            break;
          }

          patternIndex++;

        }// While loop through patterns.
      }// Else for no script block specified.

      return result;

    }// End of SelectString

    /// <summary>
    /// Check whether the supplied name meets the include/exclude criteria.
    /// That is - it's on the include list if the include list was
    /// specified, and not on the exclude list if the exclude list was specified.
    /// </summary>
    /// <param name="path">path to validate</param>
    /// <returns>True if the path is acceptable.</returns>
    private bool MeetsIncludeExcludeCriteria(string path)
    {
      bool ok = false;

      // See if the file is on the include list.
      if (this.include != null)
      {
        foreach (WildcardPattern patternItem in this.include)
        {
          if (patternItem.IsMatch(path))
          {
            ok = true;
            break;
          }
        }
      }
      else
      {
        ok = true;
      }

      if (!ok)
         return false;

      // See if the file is on the exclude list.
      if (this.exclude != null)
      {
        foreach (WildcardPattern patternItem in this.exclude)
        {
          if (patternItem.IsMatch(path))
          {
            ok = false;
            break;
          }
        }
```

```
    }

    return ok;
  } //MeetsIncludeExcludeCriteria
  #endregion Private Methods

}// class SelectStringCommand

#endregion SelectStringCommand

#region MatchInfo

/// <summary>
/// Class representing the result of a pattern/literal match
/// that is passed through the pipeline by the Select-Str cmdlet.
/// </summary>
public class MatchInfo
{
  /// <summary>
  /// Indicates if the match was done ignoring case.
  /// </summary>
  /// <value>True if case was ignored.</value>
  public bool IgnoreCase
  {
    get { return ignoreCase; }
    set { ignoreCase = value; }
  }
  private bool ignoreCase;

  /// <summary>
  /// Specifies the number of the matching line.
  /// </summary>
  /// <value>The number of the matching line.</value>
  public UInt64 LineNumber
  {
    get { return lineNumber; }
    set { lineNumber = value; }
  }
  private UInt64 lineNumber;

  /// <summary>
  /// Specifies the text of the matching line.
  /// </summary>
  /// <value>The text of the matching line.</value>
  public string Line
  {
    get { return line; }
    set { line = value; }
  }
  private string line;

  /// <summary>
  /// Specifies the full path of the object(file) containing the
  /// matching line.
  /// </summary>
```

```
    /// <remarks>
    /// It will be "inputStream" if the object came from the input
    /// stream.
    /// </remarks>
    /// <value>The path name</value>
    public string Path
    {
      get { return path; }
      set
      {
        pathSet = true;
        path = value;
      }
    }
    private string path;
    private bool pathSet;

    /// <summary>
    /// Specifies the pattern that was used in the match.
    /// </summary>
    /// <value>The pattern string</value>
    public string Pattern
    {
      get { return pattern; }
      set { pattern = value; }
    }
    private string pattern;

    private const string MatchFormat = "{0}:{1}:{2}";

    /// <summary>
    /// Returns the string representation of this object. The format
    /// depends on whether a path has been set for this object or
    /// not.
    /// </summary>
    /// <remarks>
    /// If the path component is set, as would be the case when
    /// matching in a file, ToString() returns the path, line
    /// number and line text.  If path is not set, then just the
    /// line text is presented.
    /// </remarks>
    /// <returns>The string representation of the match object.</returns>
    public override string ToString()
    {
      if (pathSet)
         return String.Format(
         System.Threading.Thread.CurrentThread.CurrentCulture,
         MatchFormat,
         this.path,
         this.lineNumber,
         this.line
         );
      else
         return this.line;
    }
```

```
}// End class MatchInfo

#endregion

#region PowerShell snap-in

/// <summary>
/// Create a PowerShell snap-in for the Select-Str cmdlet.
/// </summary>
[RunInstaller(true)]
public class SelectStringPSSnapIn : PSSnapIn
{
  /// <summary>
  /// Create an instance of the SelectStrPSSnapin class.
  /// </summary>
  public SelectStringPSSnapIn()
         : base()
  {
  }

  /// <summary>
  /// Specify the name of the PowerShell snap-in.
  /// </summary>
  public override string Name
  {
    get
    {
      return "SelectStrPSSnapIn";
    }
  }

  /// <summary>
  /// Specify the vendor of the PowerShell snap-in.
  /// </summary>
  public override string Vendor
  {
    get
    {
      return "Microsoft";
    }
  }

  /// <summary>
  /// Specify the localization resource information for the vendor.
  /// Use the format: SnapinName,VendorName.
  /// </summary>
  public override string VendorResource
  {
    get
    {
      return "SelectStrSnapIn,Microsoft";
    }
  }

  /// <summary>
```

```
    /// Specify the description of the PowerShell snap-in.
    /// </summary>
    public override string Description
    {
      get
        {
          return "This is a PowerShell snap-in for the Select-Str cmdlet.";
        }
    }

    /// <summary>
    /// Specify the localization resource information for the description.
    /// Use the format: SnapinName,Description.

    /// </summary>
    public override string DescriptionResource
    {
      get
      {
          return "SelectStrSnapIn,This is a PowerShell snap-in for the Select-Str
cmdlet.";
        }
    }
  }
  #endregion PowerShell snap-in

} //namespace Microsoft.Samples.PowerShell.Commands;
```

Building the Cmdlet

After implementing a cmdlet, you must register it with Windows PowerShell through a Windows PowerShell snap-in. For more information about registering cmdlets, see How to Register Cmdlets, Providers, and Host Applications.

Testing the Cmdlet

When your cmdlet has been registered with Windows PowerShell, you can test it by running it on the command line. The following procedure can be used to test the sample Select-Str cmdlet.

1. Start Windows PowerShell, and search the Notes file for occurrences of lines with the expression ".NET". Note that the quotation marks around the name of the path are required only if the path consists of more than one word.

   PowerShell

   ```
   Select-Str -Path "notes" -Pattern ".NET" -SimpleMatch=$false
   ```

The following output appears.

Output

```
IgnoreCase   : True
LineNumber   : 8
Line         : Because Windows PowerShell works directly with .NET objects,
there is often a .NET object
Path         : C:\PowerShell-Progs\workspace\Samples\SelectStr\notes
Pattern      : .NET
IgnoreCase   : True
LineNumber   : 21
Line         : You should normally define the class for a cmdlet in a .NET
namespace
Path         : C:\PowerShell-Progs\workspace\Samples\SelectStr\notes
Pattern      : .NET
```

2. Search the Notes file for occurrences of lines with the word "over", followed by any other text. The `SimpleMatch` parameter is using the default value of `false`. The search is case-insensitive because the `CaseSensitive` parameter is set to `false`.

PowerShell

```powershell
Select-Str -Path notes -Pattern "over*" -SimpleMatch -CaseSensitive:$false
```

The following output appears.

Output

```
IgnoreCase   : True
LineNumber   : 45
Line         : Override StopProcessing
Path         : C:\PowerShell-Progs\workspace\Samples\SelectStr\notes
Pattern      : over*
IgnoreCase   : True
LineNumber   : 49
Line         : overriding the StopProcessing method
Path         : C:\PowerShell-Progs\workspace\Samples\SelectStr\notes
Pattern      : over*
```

3. Search the Notes file using a regular expression as the pattern. The cmdlet searches for alphabetical characters and blank spaces enclosed in parentheses.

PowerShell

```powershell
Select-Str -Path notes -Pattern "\([A-Za-z:blank:]" -SimpleMatch:$false
```

The following output appears.

Output

```
IgnoreCase   : True
LineNumber   : 1
Line         : Advisory Guidelines (Consider Following)
Path         : C:\PowerShell-Progs\workspace\Samples\SelectStr\notes
Pattern      : \([A-Za-z:blank:]
IgnoreCase   : True
LineNumber   : 53
Line         : If your cmdlet has objects that are not disposed of (written to
the pipeline)
Path         : C:\PowerShell-Progs\workspace\Samples\SelectStr\notes
Pattern      : \([A-Za-z:blank:]
```

4. Perform a case-sensitive search of the Notes file for occurrences of the word "Parameter".

PowerShell

```
Select-Str -Path notes -Pattern Parameter -CaseSensitive
```

The following output appears.

Output

```
IgnoreCase   : False
LineNumber   : 6
Line         : Support an InputObject Parameter
Path         : C:\PowerShell-Progs\workspace\Samples\SelectStr\notes
Pattern      : Parameter
IgnoreCase   : False
LineNumber   : 30
Line         : Support Force Parameter
Path         : C:\PowerShell-Progs\workspace\Samples\SelectStr\notes
Pattern      : Parameter
```

5. Search the Variable provider shipped with Windows PowerShell for variables that have numerical values from 0 through 9.

PowerShell

```
Select-Str -Path * -Pattern "[0-9]"
```

The following output appears.

Output

```
IgnoreCase   : True
LineNumber   : 1
Line         : 64
Path         : Variable:\MaximumHistoryCount
Pattern      : [0-9]
```

6. Use a script block to search the file SelectStrCommandSample.cs for the string "Pos". The `-cmatch` operator performs a case-insensitive pattern match.

PowerShell

```
Select-Str -Path "SelectStrCommandSample.cs" -Script {
    if ($args[0] -cmatch "Pos"){ return $true }
    return $false
}
```

The following output appears.

Output

```
IgnoreCase   : True
LineNumber   : 37
Line         :    Position = 0.
Path         : C:\PowerShell-
Progs\workspace\Samples\SelectStr\SelectStrCommandSample.cs
Pattern      :
```

See Also

How to Create a Windows PowerShell Cmdlet

Creating Your First Cmdlet

Creating a Cmdlet that Modifies the System

Design Your Windows PowerShell Provider

How Windows PowerShell Works

How to Register Cmdlets, Providers, and Host Applications)

Windows PowerShell SDK

Last updated on 04/08/2026

Cmdlet Samples

This section describes sample code that is provided in the Windows PowerShell 2.0 SDK.

In This Section

- **GetProcessSample01 Sample**: This sample shows how to write a cmdlet that retrieves the processes on the local computer.
- **GetProcessSample02 Sample**: This sample shows how to write a cmdlet that retrieves the processes on the local computer. It provides a Name parameter that can be used to specify the processes to be retrieved.
- **GetProcessSample03 Sample**: This sample shows how to write a cmdlet that retrieves the processes on the local computer. It provides a Name parameter that can accept an object from the pipeline or a value from a property of an object whose property name is the same as the parameter name.
- **GetProcessSample04 Sample**: This sample shows how to write a cmdlet that retrieves the processes on the local computer. It generates a non-terminating error if an error occurs while retrieving a process.
- **GetProcessSample05 Sample**: This sample shows a complete version of the Get-Proc cmdlet.
- **StopProcessSample01 Sample**: This sample shows how to write a cmdlet that requests feedback from the user before it attempts to stop a process, and how to implement a `PassThru` parameter that indicates that the user wants the cmdlet to return an object.
- **StopProcessSample02 Sample**: This sample shows how to write a cmdlet that writes debug, verbose, and warning messages while stopping processes on the local computer.
- **StopProcessSample03 Sample**: This sample shows how to write a cmdlet whose parameters have aliases and that support wildcard characters.
- **StopProcessSample04 Sample**: This sample shows how to write a cmdlet that declares parameter sets, specifies the default parameter set, and can accept an input object.
- **Events01 Sample**: This sample shows how to create a cmdlet that allows the user to register for events raised by System.IO.FileSystemWatcher. With this cmdlet users can, for

example, register an action to execute when a file is created under a specific directory. This sample derives from the Microsoft.PowerShell.Commands.ObjectEventRegistrationBase base class.

See Also

- Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

GetProcessSample01 Sample

This sample shows how to implement a cmdlet that retrieves the processes on the local computer. This cmdlet is a simplified version of the `Get-Process` cmdlet that is provided by Windows PowerShell 2.0.

How to build the sample by using Visual Studio

1. With the Windows PowerShell 2.0 SDK installed, navigate to the GetProcessSample01 folder. The default location is `C:\Program Files (x86)\Microsoft SDKs\Windows\v7.0\Samples\sysmgmt\WindowsPowerShell\csharp\GetProcessSample01`.
2. Double-click the icon for the solution (.sln) file. This opens the sample project in Microsoft Visual Studio.
3. In the **Build** menu, select **Build Solution** to build the library for the sample in the default `\bin` or `\bin\debug` folders.

How to run the sample

1. Open a Command Prompt window.
2. Navigate to the directory containing the sample .dll file.
3. Run `installutil "GetProcessSample01.dll"`.
4. Start Windows PowerShell.
5. Run the following command to add the snap-in to the shell.

   ```
   Add-PSSnapin GetProcPSSnapIn01
   ```

6. Enter the following command to run the cmdlet. `Get-Proc`

   ```
   Get-Proc
   ```

   This is a sample output that results from following these steps.

   Output

```
Id              Name            State      HasMoreData     Location
Command
--              ----            -----      -----------     --------
-------
1               26932870-d3b... NotStarted False
Write-Host "A f...
```

PowerShell

```
Set-Content $Env:TEMP\test.txt "This is a test file"
```

Output

```
A file was created in the TEMP directory
```

Requirements

This sample requires Windows PowerShell 1.0 or later.

Demonstrates

This sample demonstrates the following.

- Creating a basic sample cmdlet.
- Defining a cmdlet class by using the Cmdlet attribute.
- Creating a snap-in that works with both Windows PowerShell 1.0 and Windows PowerShell 2.0. Subsequent samples use modules instead of snap-ins so they require Windows PowerShell 2.0.

Example

This sample shows how to create a simple cmdlet and its snap-in.

C#

```
using System;
using System.Diagnostics;
using System.Management.Automation;             //Windows PowerShell namespace
using System.ComponentModel;

namespace Microsoft.Samples.PowerShell.Commands
```

```
{

   #region GetProcCommand

   /// <summary>
   /// This class implements the Get-Proc cmdlet.
   /// </summary>
   [Cmdlet(VerbsCommon.Get, "Proc")]
   public class GetProcCommand : Cmdlet
   {
      #region Cmdlet Overrides

      /// <summary>
      /// The ProcessRecord method calls the Process.GetProcesses
      /// method to retrieve the processes of the local computer.
      /// Then, the WriteObject method writes the associated processes
      /// to the pipeline.
      /// </summary>
      protected override void ProcessRecord()
      {
         // Retrieve the current processes.
         Process[] processes = Process.GetProcesses();

         // Write the processes to the pipeline to make them available
         // to the next cmdlet. The second argument (true) tells Windows
         // PowerShell to enumerate the array and to send one process
         // object at a time to the pipeline.
         WriteObject(processes, true);
      }

      #endregion Overrides

   } //GetProcCommand

   #endregion GetProcCommand

   #region PowerShell snap-in

   /// <summary>
   /// Create this sample as a PowerShell snap-in
   /// </summary>
   [RunInstaller(true)]
   public class GetProcPSSnapIn01 : PSSnapIn
   {
       /// <summary>
       /// Create an instance of the GetProcPSSnapIn01
       /// </summary>
       public GetProcPSSnapIn01()
           : base()
       {
       }

       /// <summary>
       /// Get a name for this PowerShell snap-in. This name will be used in
registering
```

```
        /// this PowerShell snap-in.
        /// </summary>
        public override string Name
        {
            get
            {
                return "GetProcPSSnapIn01";
            }
        }

        /// <summary>
        /// Vendor information for this PowerShell snap-in.
        /// </summary>
        public override string Vendor
        {
            get
            {
                return "Microsoft";
            }
        }

        /// <summary>
        /// Gets resource information for vendor. This is a string of format:
        /// resourceBaseName,resourceName.
        /// </summary>
        public override string VendorResource
        {
            get
            {
                return "GetProcPSSnapIn01,Microsoft";
            }
        }

        /// <summary>
        /// Description of this PowerShell snap-in.
        /// </summary>
        public override string Description
        {
            get
            {
                return "This is a PowerShell snap-in that includes the Get-Proc
cmdlet.";
            }
        }
    }

    #endregion PowerShell snap-in
}
```

See Also

- Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

GetProcessSample02 Sample

This sample shows how to write a cmdlet that retrieves the processes on the local computer. It provides a `Name` parameter that can be used to specify the processes to be retrieved. This cmdlet is a simplified version of the `Get-Process` cmdlet provided by Windows PowerShell 2.0.

How to build the sample using Visual Studio

1. With the Windows PowerShell 2.0 SDK installed, navigate to the GetProcessSample02 folder. The default location is `C:\Program Files (x86)\Microsoft SDKs\Windows\v7.0\Samples\sysmgmt\WindowsPowerShell\csharp\GetProcessSample02`.
2. Double-click the icon for the solution (.sln) file. This opens the sample project in Visual Studio.
3. In the **Build** menu, select **Build Solution** to build the library for the sample in the default `\bin` or `\bin\debug` folders.

How to run the sample

1. Create the following module folder:

   `[user]\Documents\WindowsPowerShell\Modules\GetProcessSample02`
2. Copy the sample assembly to the module folder.
3. Start Windows PowerShell.
4. Run the following command to load the assembly into Windows PowerShell:

   `Import-Module getprossessample02`
5. Run the following command to run the cmdlet:

   `Get-Proc`

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Declaring a cmdlet class using the Cmdlet attribute.
- Declaring a cmdlet parameter using the Parameter attribute.
- Specifying the position of the parameter.
- Declaring a validation attribute for the parameter input.

Example

This sample shows an implementation of the Get-Proc cmdlet that includes a `Name` parameter.

C#

```
namespace Microsoft.Samples.PowerShell.Commands
{
  using System;
  using System.Diagnostics;
  using System.Management.Automation;     // Windows PowerShell namespace

  #region GetProcCommand

  /// <summary>
  /// This class implements the Get-Proc cmdlet.
  /// </summary>
  [Cmdlet(VerbsCommon.Get, "Proc")]
  public class GetProcCommand : Cmdlet
  {
    #region Parameters

    /// <summary>
    /// The names of the processes retrieved by the cmdlet.
    /// </summary>
    private string[] processNames;

    /// <summary>
    /// Gets or sets the list of process names on which
    /// the Get-Proc cmdlet will work.
    /// </summary>
    [Parameter(Position = 0)]
    [ValidateNotNullOrEmpty]
    public string[] Name
    {
      get { return this.processNames; }
      set { this.processNames = value; }
    }
```

```
    #endregion Parameters

    #region Cmdlet Overrides

    /// <summary>
    /// The ProcessRecord method calls the Process.GetProcesses
    /// method to retrieve the processes specified by the Name
    /// parameter. Then, the WriteObject method writes the
    /// associated process objects to the pipeline.
    /// </summary>
    protected override void ProcessRecord()
    {
      // If no process names are passed to the cmdlet, get all
      // processes.
      if (this.processNames == null)
      {
        WriteObject(Process.GetProcesses(), true);
      }
      else
      {
        // If process names are passed to cmdlet, get and write
        // the associated processes.
        foreach (string name in this.processNames)
        {
          WriteObject(Process.GetProcessesByName(name), true);
        }
      } // End if (processNames...).
    } // End ProcessRecord.
    #endregion Cmdlet Overrides
  } // End GetProcCommand class.
  #endregion GetProcCommand
}
```

See Also

- Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

GetProcessSample03 Sample

This sample shows how to implement a cmdlet that retrieves the processes on the local computer. It provides a `Name` parameter that can accept an object from the pipeline or a value from a property of an object whose property name is the same as the parameter name. This cmdlet is a simplified version of the `Get-Process` cmdlet provided by Windows PowerShell 2.0.

How to build the sample using Visual Studio

1. With the Windows PowerShell 2.0 SDK installed, navigate to the GetProcessSample03 folder. The default location is `C:\Program Files (x86)\Microsoft SDKs\Windows\v7.0\Samples\sysmgmt\WindowsPowerShell\csharp\GetProcessSample03`.
2. Double-click the icon for the solution (.sln) file. This opens the sample project in Visual Studio.
3. In the **Build** menu, select **Build Solution** to build the library for the sample in the default `\bin` or `\bin\debug` folders.

How to run the sample

1. Create the following module folder:

   `[user]\Documents\WindowsPowerShell\Modules\GetProcessSample03`

2. Copy the sample assembly to the module folder.
3. Start Windows PowerShell.
4. Run the following command to load the assembly into Windows PowerShell:

   `Import-Module getprossessample03`

5. Run the following command to run the cmdlet:

   `Get-Proc`

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Declaring a cmdlet class using the Cmdlet attribute.
- Declaring a cmdlet parameter using the Parameter attribute.
- Specifying the position of the parameter.
- Specifying that the parameter takes input from the pipeline. The input can be taken from an object or a value from a property of an object whose property name is the same as the parameter name.
- Declaring a validation attribute for the parameter input.

Example

This sample shows an implementation of the Get-Proc cmdlet that includes a `Name` parameter that accepts input from the pipeline.

C#

```
namespace Microsoft.Samples.PowerShell.Commands
{
  using System;
  using System.Diagnostics;
  using System.Management.Automation;           // Windows PowerShell namespace
  #region GetProcCommand

  /// <summary>
  /// This class implements the Get-Proc cmdlet.
  /// </summary>
  [Cmdlet(VerbsCommon.Get, "Proc")]
  public class GetProcCommand : Cmdlet
  {
    #region Parameters

    /// <summary>
    /// The names of the processes retrieved by the cmdlet.
    /// </summary>
    private string[] processNames;

    /// <summary>
    /// Gets or sets the names of the
    /// process that the cmdlet will retrieve.
    /// </summary>
    [Parameter(
```

```
                Position = 0,
                ValueFromPipeline = true,
                ValueFromPipelineByPropertyName = true)]
    [ValidateNotNullOrEmpty]
    public string[] Name
    {
      get { return this.processNames; }
      set { this.processNames = value; }
    }

    #endregion Parameters

    #region Cmdlet Overrides

    /// <summary>
    /// The ProcessRecord method calls the Process.GetProcesses
    /// method to retrieve the processes specified by the Name
    /// parameter. Then, the WriteObject method writes the
    /// associated processes to the pipeline.
    /// </summary>
    protected override void ProcessRecord()
    {
      // If no process names are passed to the cmdlet, get all
      // processes.
      if (this.processNames == null)
      {
        WriteObject(Process.GetProcesses(), true);
      }
      else
      {
        // If process names are passed to the cmdlet, get and write
        // the associated processes.
        foreach (string name in this.processNames)
        {
          WriteObject(Process.GetProcessesByName(name), true);
        }
      } // End if (processNames ...)
    } // End ProcessRecord.

    #endregion Overrides
  } // End GetProcCommand.
  #endregion GetProcCommand
}
```

See Also

- Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

GetProcessSample04 Sample

This sample shows how to implement a cmdlet that retrieves the processes on the local computer. It generates a non-terminating error if an error occurs while retrieving a process. This cmdlet is a simplified version of the `Get-Process` cmdlet provided by Windows PowerShell 2.0.

How to build the sample using Visual Studio

1. With the Windows PowerShell 2.0 SDK installed, navigate to the GetProcessSample04 folder. The default location is `C:\Program Files (x86)\Microsoft SDKs\Windows\v7.0\Samples\sysmgmt\WindowsPowerShell\csharp\GetProcessSample04`.
2. Double-click the icon for the solution (.sln) file. This opens the sample project in Visual Studio.
3. In the **Build** menu, select **Build Solution** to build the library for the sample in the default `\bin` or `\bin\debug` folders.

How to run the sample

1. Create the following module folder:

   `[user]\Documents\WindowsPowerShell\Modules\GetProcessSample04`

2. Copy the sample assembly to the module folder.
3. Start Windows PowerShell.
4. Run the following command to load the assembly into Windows PowerShell:

   `Import-Module getprossessample04`

5. Run the following command to run the cmdlet:

   `Get-Proc`

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Declaring a cmdlet class using the Cmdlet attribute.
- Declaring a cmdlet parameter using the Parameter attribute.
- Specifying the position of the parameter.
- Specifying that the parameter takes input from the pipeline. The input can be taken from an object or a value from a property of an object whose property name is the same as the parameter name.
- Declaring a validation attribute for the parameter input.
- Trapping a non-terminating error and writing an error message to the error stream.

Example

This sample shows how to create a cmdlet that handles non-terminating errors and writes error messages to the error stream.

C#

```
namespace Microsoft.Samples.PowerShell.Commands
{
    using System;
    using System.Diagnostics;
    using System.Management.Automation;      // Windows PowerShell namespace.
   #region GetProcCommand

   /// <summary>
   /// This class implements the Get-Proc cmdlet.
   /// </summary>
   [Cmdlet(VerbsCommon.Get, "Proc")]
   public class GetProcCommand : Cmdlet
   {
      #region Parameters

       /// <summary>
       /// The names of the processes to act on.
       /// </summary>
       private string[] processNames;

      /// <summary>
      /// Gets or sets the list of process names on
      /// which the Get-Proc cmdlet will work.
```

```
        /// </summary>
        [Parameter(
           Position = 0,
           ValueFromPipeline = true,
           ValueFromPipelineByPropertyName = true)]
        [ValidateNotNullOrEmpty]
        public string[] Name
        {
           get { return this.processNames; }
           set { this.processNames = value; }
        }

        #endregion Parameters

        #region Cmdlet Overrides

        /// <summary>
        /// The ProcessRecord method calls the Process.GetProcesses
        /// method to retrieve the processes specified by the Name
        /// parameter. Then, the WriteObject method writes the
        /// associated processes to the pipeline.
        /// </summary>
        protected override void ProcessRecord()
        {
            // If no process names are passed to cmdlet, get all
            // processes.
            if (this.processNames == null)
            {
                WriteObject(Process.GetProcesses(), true);
            }
            else
            {
                // If process names are passed to the cmdlet, get and write
                // the associated processes.
                // If a non-terminating error occurs while retrieving processes,
                // call the WriteError method to send an error record to the
                // error stream.
                foreach (string name in this.processNames)
                {
                    Process[] processes;

                    try
                    {
                        processes = Process.GetProcessesByName(name);
                    }
                    catch (InvalidOperationException ex)
                    {
                        WriteError(new ErrorRecord(
                           ex,
                           "UnableToAccessProcessByName",
                           ErrorCategory.InvalidOperation,
                           name));
                        continue;
                    }
```

```
                WriteObject(processes, true);
            } // foreach (string name...
        } // else
    } // ProcessRecord

    #endregion Overrides
} // End GetProcCommand class.

#endregion GetProcCommand
}
```

See Also

- Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

GetProcessSample05 Sample

This sample shows a complete version of the Get-Proc cmdlet.

How to build the sample using Visual Studio.

1. Open Windows Explorer and navigate to the GetProcessSample05 directory under the Samples directory.

   With the Windows PowerShell 2.0 SDK installed, navigate to the GetProcessSample05 folder. The default location is `C:\Program Files (x86)\Microsoft SDKs\Windows\v7.0\Samples\sysmgmt\WindowsPowerShell\csharp\GetProcessSample05`.

2. Double-click the icon for the solution (.sln) file. This opens the sample project in Visual Studio.

3. In the **Build** menu, select **Build Solution** to build the library for the sample in the default `\bin` or `\bin\debug` folders.

How to run the sample

1. Create the following module folder:

   `[user]\Documents\WindowsPowerShell\Modules\GetProcessSample05`

2. Copy the sample assembly to the module folder.

3. Start Windows PowerShell.

4. Run the following command to load the assembly into Windows PowerShell:

   `Import-Module getprossessample05`

5. Run the following command to run the cmdlet:

   `Get-Proc`

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Declaring a cmdlet class using the Cmdlet attribute.
- Declaring a cmdlet parameter using the Parameter attribute.
- Specifying positions for parameters.
- Specifying that parameters can take input from the pipeline. The input can be taken from an object or a value from a property of an object whose property name is the same as the parameter name.
- Declaring a validation attribute for the parameter input.
- Handling errors and exceptions.
- Writing debug messages.

Example

This sample shows how to create a cmdlet that displays a list of specified processes.

C#

```
namespace Microsoft.Samples.PowerShell.Commands
{
    using System;
    using System.Collections.Generic;
    using System.Diagnostics;
    using System.Management.Automation;    // Windows PowerShell namespace.
    using System.Security.Permissions;
    using Win32Exception = System.ComponentModel.Win32Exception;
    #region GetProcCommand

    /// <summary>
   /// This class implements the Get-Proc cmdlet.
   /// </summary>
   [Cmdlet(VerbsCommon.Get, "Proc",
      DefaultParameterSetName = "ProcessName")]
   public class GetProcCommand : PSCmdlet
   {
       #region Fields
       /// <summary>
       /// The names of the processes to act on.
       /// </summary>
       private string[] processNames;
```

```
        /// <summary>
        /// The identifiers of the processes to act on.
        /// </summary>
        private int[] processIds;

        /// <summary>
        /// The process objects to act on.
        /// </summary>
        private Process[] inputObjects;

        #endregion Fields

        #region Parameters

    /// <summary>
    /// Gets or sets the list of process names on
    /// which the Get-Proc cmdlet will work.
    /// </summary>
    [Parameter(
       Position = 0,
       ParameterSetName = "ProcessName",
       ValueFromPipeline = true,
       ValueFromPipelineByPropertyName = true)]
    [ValidateNotNullOrEmpty]
    public string[] Name
    {
       get { return this.processNames; }
       set { this.processNames = value; }
    }

    /// <summary>
    /// Gets or sets the list of process identifiers on
    /// which the Get-Proc cmdlet will work.
    /// </summary>
    [Parameter(
       ParameterSetName = "Id",
       Mandatory = true,
       ValueFromPipeline = true,
       ValueFromPipelineByPropertyName = true,
       HelpMessage = "The unique id of the process to get.")]
    public int[] Id
    {
       get { return this.processIds; }
       set { this.processIds = value; }
    }

    /// <summary>
    /// Gets or sets Process objects directly. If the input is a
    /// stream of [collection of] Process objects, the cmdlet bypasses the
    /// ProcessName and Id parameters and reads the Process objects
    /// directly.  This allows the cmdlet to deal with processes that have
    /// wildcard characters in their name.
    /// <value>Process objects</value>
    /// </summary>
```

```
        [Parameter(
           ParameterSetName = "InputObject",
           Mandatory = true,
           ValueFromPipeline = true)]
        public Process[] Input
        {
           get { return this.inputObjects; }
           set { this.inputObjects = value; }
        }

        #endregion Parameters

        #region Cmdlet Overrides

        /// <summary>
        /// The ProcessRecord method calls the Process.GetProcesses
        /// method to retrieve the processes. Then, the WriteObject
        /// method writes the associated processes to the pipeline.
        /// </summary>
        protected override void ProcessRecord()
        {
           List<Process> matchingProcesses;

           WriteDebug("Obtaining the list of matching process objects.");

           switch (ParameterSetName)
           {
              case "Id":
                 matchingProcesses = this.GetMatchingProcessesById();
                 break;
              case "ProcessName":
                 matchingProcesses = this.GetMatchingProcessesByName();
                 break;
              case "InputObject":
                 matchingProcesses = this.GetProcessesByInput();
                 break;
              default:
                 ThrowTerminatingError(
                     new ErrorRecord(
                         new ArgumentException("Bad ParameterSetName"),
                         "UnableToAccessProcessList",
                         ErrorCategory.InvalidOperation,
                         null));
                 return;
           } // switch (ParameterSetName)

           WriteDebug("Outputting the matching process objects.");

           matchingProcesses.Sort(ProcessComparison);

           foreach (Process process in matchingProcesses)
           {
              WriteObject(process);
           }
        } // ProcessRecord
```

```
#endregion Overrides

#region protected Methods and Data

/// <summary>
/// Retrieves the list of all processes matching the ProcessName
/// parameter and generates a non-terminating error for each
/// specified process name which is not found even though the name
/// contains no wildcards.
/// </summary>
/// <returns>The matching processes.</returns>
[EnvironmentPermissionAttribute(
   SecurityAction.LinkDemand,
   Unrestricted = true)]
private List<Process> GetMatchingProcessesByName()
{
   new EnvironmentPermission(
      PermissionState.Unrestricted).Assert();

   List<Process> allProcesses =
      new List<Process>(Process.GetProcesses());

   // The keys dictionary is used for rapid lookup of
   // processes that are already in the matchingProcesses list.
   Dictionary<int, byte> keys = new Dictionary<int, byte>();

   List<Process> matchingProcesses = new List<Process>();

   if (null == this.processNames)
   {
       matchingProcesses.AddRange(allProcesses);
   }
   else
   {
       foreach (string pattern in this.processNames)
       {
           WriteVerbose("Finding matches for process name \""
              + pattern + "\".");

           // WildCard search on the available processes
           WildcardPattern wildcard =
              new WildcardPattern(
                  pattern,
                  WildcardOptions.IgnoreCase);

           bool found = false;

           foreach (Process process in allProcesses)
           {
               if (!keys.ContainsKey(process.Id))
               {
                   string processName = SafeGetProcessName(process);

                   // Remove the process from the allProcesses list
```

```
                    // so that it is not tested again.
                    if (processName.Length == 0)
                    {
                        allProcesses.Remove(process);
                    }

                    // Perform a wildcard search on this particular
                    // process name and check whether it matches the
                    // pattern specified.
                    if (!wildcard.IsMatch(processName))
                    {
                        continue;
                    }

                    WriteDebug("Found matching process id "
                       + process.Id + ".");

                    // A match is found.
                    found = true;

                    // Store the process identifier so that the same process
                    // is not added twice.
                    keys.Add(process.Id, 0);

                    // Add the process to the processes list.
                    matchingProcesses.Add(process);
                }
            } // foreach (Process...

            if (!found &&
              !WildcardPattern.ContainsWildcardCharacters(pattern))
            {
                WriteError(new ErrorRecord(
                   new ArgumentException("Cannot find process name "
                      + "\"" + pattern + "\"."),
                   "ProcessNameNotFound",
                   ErrorCategory.ObjectNotFound,
                   pattern));
            }
        } // foreach (string...
    } // if (null...

    return matchingProcesses;
} // GetMatchingProcessesByName

/// <summary>
/// Returns the name of a process.  If an error occurs, a blank
/// string is returned.
/// </summary>
/// <param name="process">The process whose name is
/// returned.</param>
/// <returns>The name of the process.</returns>
[EnvironmentPermissionAttribute(
   SecurityAction.LinkDemand, Unrestricted = true)]
protected static string SafeGetProcessName(Process process)
```

```
        {
            new EnvironmentPermission(PermissionState.Unrestricted).Assert();
            string name = String.Empty;

            if (process != null)
            {
                try
                {
                    return process.ProcessName;
                }
                catch (Win32Exception)
                {
                }
                catch (InvalidOperationException)
                {
                }
            }

            return name;
        } // SafeGetProcessName

        #endregion Cmdlet Overrides

        #region Private Methods

        /// <summary>
        /// Function to sort by process name first, and then by
        /// the process identifier.
        /// </summary>
        /// <param name="x">First process object.</param>
        /// <param name="y">Second process object.</param>
        /// <returns>
        /// Returns less than zero if x is less than y,
        /// greater than 0 if x is greater than y, and 0 if x == y.
        /// </returns>
        private static int ProcessComparison(Process x, Process y)
        {
            int diff = String.Compare(
                SafeGetProcessName(x),
                SafeGetProcessName(y),
                StringComparison.CurrentCultureIgnoreCase);

            if (0 != diff)
            {
                return diff;
            }
            else
            {
                return x.Id.CompareTo(y.Id);
            }
        }

        /// <summary>
        /// Retrieves the list of all processes matching the Id
        /// parameter and generates a non-terminating error for
```

```
        /// each specified process identifier which is not found.
        /// </summary>
        /// <returns>
        /// An array of processes that match the given identifier.
        /// </returns>
        [EnvironmentPermissionAttribute(
           SecurityAction.LinkDemand,
           Unrestricted = true)]
        private List<Process> GetMatchingProcessesById()
        {
           new EnvironmentPermission(
              PermissionState.Unrestricted).Assert();

           List<Process> matchingProcesses = new List<Process>();

           if (null != this.processIds)
           {
              // The keys dictionary is used for rapid lookup of the
              // processes already in the matchingProcesses list.
              Dictionary<int, byte> keys = new Dictionary<int, byte>();

              foreach (int processId in this.processIds)
              {
                 WriteVerbose("Finding match for process id "
                    + processId + ".");

                 if (!keys.ContainsKey(processId))
                 {
                    Process process;
                    try
                    {
                        process = Process.GetProcessById(processId);
                    }
                    catch (ArgumentException ex)
                    {
                       WriteError(new ErrorRecord(
                          ex,
                          "ProcessIdNotFound",
                          ErrorCategory.ObjectNotFound,
                          processId));
                       continue;
                    }

                    WriteDebug("Found matching process.");

                    matchingProcesses.Add(process);
                    keys.Add(processId, 0);
                 }
              }
           }

           return matchingProcesses;
        } // GetMatchingProcessesById

        /// <summary>
```

```
      /// Retrieves the list of all processes matching the InputObject
      /// parameter.
      /// </summary>
      /// <returns>The matching processes.</returns>
      [EnvironmentPermissionAttribute(
         SecurityAction.LinkDemand,
         Unrestricted = true)]
      private List<Process> GetProcessesByInput()
      {
         new EnvironmentPermission(
            PermissionState.Unrestricted).Assert();

         List<Process> matchingProcesses = new List<Process>();

         if (null != this.Input)
         {
            // The keys dictionary is used for rapid lookup of the
            // processes already in the matchingProcesses list.
            Dictionary<int, byte> keys = new Dictionary<int, byte>();

            foreach (Process process in this.Input)
            {
               WriteVerbose("Refreshing process object.");

               if (!keys.ContainsKey(process.Id))
               {
                  try
                  {
                      process.Refresh();
                  }
                  catch (Win32Exception)
                  {
                  }
                  catch (InvalidOperationException)
                  {
                  }

                  matchingProcesses.Add(process);
               }
            }
         }

         return matchingProcesses;
      } // GetProcessesByInput
      #endregion Private Methods
   } // End GetProcCommand class.

   #endregion GetProcCommand
}
```

See Also

- Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

StopProcessSample01 Sample

This sample shows how to write a cmdlet that requests feedback from the user before it attempts to stop a process, and how to implement a `PassThru` parameter indicating that the user wants the cmdlet to return an object. This cmdlet is similar to the `Stop-Process` cmdlet provided by Windows PowerShell 2.0.

How to build the sample by using Visual Studio

1. With the Windows PowerShell 2.0 SDK installed, navigate to the StopProcessSample01 folder. The default location is `C:\Program Files (x86)\Microsoft SDKs\Windows\v7.0\Samples\sysmgmt\WindowsPowerShell\csharp\StopProcessSample01`.

2. Double-click the icon for the solution (.sln) file. This opens the sample project in Microsoft Visual Studio.

3. In the **Build** menu, select **Build Solution** to build the library for the sample in the default `\bin` or `\bin\debug` folders.

How to run the sample

1. Create the following module folder:

   `[user]\Documents\WindowsPowerShell\Modules\StopProcessSample01`

2. Copy the sample assembly to the module folder.

3. Start Windows PowerShell.

4. Run the following command to load the assembly into Windows PowerShell:

   `Import-Module stopprossessample01`

5. Run the following command to run the cmdlet:

   `Stop-Proc`

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Declaring a cmdlet class by using the Cmdlet attribute.
- Declaring a cmdlet parameters by using the Parameter attribute.
- Calling the ShouldProcess method to request confirmation.
- Implementing a `PassThru` parameter that indicates if the user wants the cmdlet to return an object. By default, this cmdlet does not return an object to the pipeline.

Example

This sample shows how to implement a `PassThru` parameter that indicates that the user wants the cmdlet to return an object, and how to request user feedback by calls to the `ShouldProcess` and `ShouldContinue` methods.

C#

```
using System;
using System.Diagnostics;
using System.Collections;
using Win32Exception = System.ComponentModel.Win32Exception;
using System.Management.Automation;    // Windows PowerShell namespace
using System.Globalization;

namespace Microsoft.Samples.PowerShell.Commands
{
   #region StopProcCommand

    /// <summary>
   /// This class implements the Stop-Proc cmdlet.
   /// </summary>
   [Cmdlet(VerbsLifecycle.Stop, "Proc",
       SupportsShouldProcess = true)]
   public class StopProcCommand : Cmdlet
   {
       #region Parameters

      /// <summary>
      /// This parameter provides the list of process names on
      /// which the Stop-Proc cmdlet will work.
      /// </summary>
       [Parameter(
          Position = 0,
          Mandatory = true,
```

```
        ValueFromPipeline = true,
        ValueFromPipelineByPropertyName = true
    )]
    public string[] Name
    {
        get { return processNames; }
        set { processNames = value; }
    }
    private string[] processNames;

    /// <summary>
    /// This parameter overrides the ShouldContinue call to force
    /// the cmdlet to stop its operation. This parameter should always
    /// be used with caution.
    /// </summary>
    [Parameter]
    public SwitchParameter Force
    {
        get { return force; }
        set { force = value; }
    }
    private bool force;

    /// <summary>
    /// This parameter indicates that the cmdlet should return
    /// an object to the pipeline after the processing has been
    /// completed.
    /// </summary>
    [Parameter]
    public SwitchParameter PassThru
    {
        get { return passThru; }
        set { passThru = value; }
    }
    private bool passThru;

    #endregion Parameters

    #region Cmdlet Overrides

    /// <summary>
    /// The ProcessRecord method does the following for each of the
    /// requested process names:
    /// 1) Check that the process is not a critical process.
    /// 2) Attempt to stop that process.
    /// If no process is requested then nothing occurs.
    /// </summary>
    protected override void ProcessRecord()
    {
        foreach (string name in processNames)
        {
            // For every process name passed to the cmdlet, get the associated
            // processes.
            // Write a non-terminating error for failure to retrieve
            // a process.
```

```
                Process[] processes;

                try
                {
                    processes = Process.GetProcessesByName(name);
                }
                catch (InvalidOperationException ioe)
                {
                    WriteError(new ErrorRecord(ioe,"UnableToAccessProcessByName",
                        ErrorCategory.InvalidOperation, name));

                    continue;
                }

                // Try to stop the processes that have been retrieved.
                foreach (Process process in processes)
                {
                    string processName;

                    try
                    {
                        processName = process.ProcessName;
                    }
                    catch (Win32Exception e)
                    {
                       WriteError(new ErrorRecord(e, "ProcessNameNotFound",
                                                  ErrorCategory.ReadError, process));
                       continue;
                    }

                    // Confirm the operation with the user first.
                    // This is always false if the WhatIf parameter is set.
                    if (!ShouldProcess(string.Format(CultureInfo.CurrentCulture,"{0}
({1})", processName,
                                       process.Id)))
                    {
                        continue;
                    }

                    // Make sure that the user really wants to stop a critical
                    // process that could possibly stop the computer.
                    bool criticalProcess =

criticalProcessNames.Contains(processName.ToLower(CultureInfo.CurrentCulture));

                    if (criticalProcess &&!force)
                    {
                        string message = String.Format
                            (CultureInfo.CurrentCulture,
                                 "The process \"{0}\" is a critical process and
should not be stopped. Are you sure you wish to stop the process?",
                                    processName);

                        // It is possible that the ProcessRecord method is called
                        // multiple times when objects are received as inputs from
```

```
                    // the pipeline. So to retain YesToAll and NoToAll input
that
                    // the user may enter across multiple calls to this
function,
                    // they are stored as private members of the cmdlet.
                    if (!ShouldContinue(message, "Warning!",
                                        ref yesToAll, ref noToAll))
                    {
                        continue;
                    }
                } // if (criticalProcess...

                // Stop the named process.
                try
                {
                    process.Kill();
                }
                catch (Exception e)
                {
                    if ((e is Win32Exception) || (e is SystemException) ||
                       (e is InvalidOperationException))
                    {
                        // This process could not be stopped so write
                        // a non-terminating error.
                        WriteError(new ErrorRecord(e, "CouldNotStopProcess",
                                        ErrorCategory.CloseError, process));
                        continue;
                    } // if ((e is...
                    else throw;
                } // catch

                // If the PassThru parameter is
                // specified, return the terminated process.
                if (passThru)
                {
                    WriteObject(process);
                }
            } // foreach (Process...
        } // foreach (string...
    } // ProcessRecord

    #endregion Cmdlet Overrides

    #region Private Data

    private bool yesToAll, noToAll;

    /// <summary>
    /// Partial list of critical processes that should not be
    /// stopped.  Lower case is used for case insensitive matching.
    /// </summary>
    private ArrayList criticalProcessNames = new ArrayList(
       new string[] { "system", "winlogon", "spoolsv" }
    );
```

```
        #endregion Private Data

    } // StopProcCommand

    #endregion StopProcCommand
}
```

See Also

- Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

StopProcessSample02 Sample

This sample shows how to write a cmdlet that writes debug (WriteDebug), verbose (WriteVerbose), and warning (WriteWarning) messages while stopping processes on the local computer. This cmdlet is similar to the `Stop-Process` cmdlet provided by Windows PowerShell 2.0.

How to build the sample by using Visual Studio

1. Open Windows Internet Explorer and navigate to the StopProcessSample02 directory under the Samples directory.

   With the Windows PowerShell 2.0 SDK installed, navigate to the StopProcessSample02 folder. The default location is `C:\Program Files (x86)\Microsoft SDKs\Windows\v7.0\Samples\sysmgmt\WindowsPowerShell\csharp\StopProcessSample02`.

2. Double-click the icon for the solution (.sln) file. This opens the sample project in Microsoft Visual Studio.

3. In the **Build** menu, select **Build Solution** to build the library for the sample in the default `\bin` or `\bin\debug` folders.

How to run the sample

1. Create the following module folder:

   `[user]\Documents\WindowsPowerShell\Modules\StopProcessSample02`

2. Copy the sample assembly to the module folder.

3. Start Windows PowerShell.

4. Run the following command to load the assembly into Windows PowerShell:

   `Import-Module stopprossessample02`

5. Run the following command to run the cmdlet:

   `Stop-Proc`

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Declaring a cmdlet class by using the Cmdlet attribute.
- Declaring a cmdlet parameters by using the Parameter attribute.
- Writing verbose messages. For more information about the method used to write verbose messages, see System.Management.Automation.Cmdlet.WriteVerbose.
- Writing error messages. For more information about the method used to write error messages, see System.Management.Automation.Cmdlet.WriteError.
- Writing warning messages. For more information about the method used to write warning messages, see System.Management.Automation.Cmdlet.WriteWarning.

Example

This sample shows how to write debug, verbose, and warning messages by using the `WriteDebug`, `WriteVerbose`, and `WriteWarning` methods.

C#

```
using System;
using System.Diagnostics;
using System.Collections;
using Win32Exception = System.ComponentModel.Win32Exception;
using System.Management.Automation;             //Windows PowerShell namespace
using System.Globalization;

namespace Microsoft.Samples.PowerShell.Commands
{
   #region StopProcCommand

    /// <summary>
   /// This class implements the Stop-Proc cmdlet.
   /// </summary>
   [Cmdlet(VerbsLifecycle.Stop, "Proc",
       SupportsShouldProcess = true)]
   public class StopProcCommand : Cmdlet
   {
```

```
        #region Parameters

        /// <summary>
        /// This parameter provides the list of process names on
        /// which the Stop-Proc cmdlet will work.
        /// </summary>
        [Parameter(
            Position = 0,
            Mandatory = true,
            ValueFromPipeline = true,
            ValueFromPipelineByPropertyName = true
        )]
        public string[] Name
        {
            get { return processNames; }
            set { processNames = value; }
        }
        private string[] processNames;

        /// <summary>
        /// This parameter overrides the ShouldContinue call to force
        /// the cmdlet to stop its operation. This parameter should always
        /// be used with caution.
        /// </summary>
        [Parameter]
        public SwitchParameter Force
        {
            get { return force; }
            set { force = value; }
        }
        private bool force;

        /// <summary>
        /// This parameter indicates that the cmdlet should return
        /// an object to the pipeline after the processing has been
        /// completed.
        /// </summary>
        [Parameter]
        public SwitchParameter PassThru
        {
            get { return passThru; }
            set { passThru = value; }
        }
        private bool passThru;

        #endregion Parameters

        #region Cmdlet Overrides

        /// <summary>
        /// The ProcessRecord method does the following for each of the
        /// requested process names:
        /// 1) Check that the process is not a critical process.
        /// 2) Attempt to stop that process.
        /// If no process is requested then nothing occurs.
```

```
        /// </summary>
        protected override void ProcessRecord()
        {
            foreach (string name in processNames)
            {
                string message = null;

                // For every process name passed to the cmdlet, get the associated
                // processes.
                // Write a non-terminating error for failure to retrieve
                // a process.

                // Write a user-friendly verbose message to the pipeline. These
                // messages are intended to give the user detailed information
                // on the operations performed by the cmdlet. These messages will
                // appear with the -Verbose option.
                message = String.Format(CultureInfo.CurrentCulture,
                                "Attempting to stop process \"{0}\".", name);
                WriteVerbose(message);

                Process[] processes;

                try
                {
                    processes = Process.GetProcessesByName(name);
                }
                catch (InvalidOperationException ioe)
                {
                    WriteError(new ErrorRecord(ioe,
                                        "UnableToAccessProcessByName",
                                            ErrorCategory.InvalidOperation,
                                                name));
                    continue;
                }

                // Try to stop the processes that have been retrieved.
                foreach (Process process in processes)
                {
                    string processName;

                    try
                    {
                        processName = process.ProcessName;
                    }
                    catch (Win32Exception e)
                    {
                        WriteError(new ErrorRecord(e, "ProcessNameNotFound",
                                            ErrorCategory.ObjectNotFound, process));
                        continue;
                    }

                    // Write a debug message to the host that can be used when
                    // troubleshooting a problem. All debug messages will appear
                    // with the -Debug option.
                    message = String.Format(CultureInfo.CurrentCulture,
```

```
                                "Acquired name for pid {0} : \"{1}\"",
                                    process.Id, processName);
                    WriteDebug(message);

                    // Confirm the operation first.
                    // This is always false if the WhatIf parameter is specified.
                    if (!ShouldProcess(string.Format(CultureInfo.CurrentCulture,
                                    "{0} ({1})",
                                        processName, process.Id)))
                    {
                        continue;
                    }

                    // Make sure that the user really wants to stop a critical
                    // process that can possibly stop the computer.
                    bool criticalProcess =
criticalProcessNames.Contains(processName.ToLower(CultureInfo.CurrentCulture));

                    if (criticalProcess && !force)
                    {
                        message = String.Format(CultureInfo.CurrentCulture,
                                    "The process \"{0}\" is a critical process and
should not be stopped. Are you sure you wish to stop the process?",
                                    processName);

                        // It is possible that the ProcessRecord method is called
                        // multiple times when objects are received as inputs from
                        // the pipeline. So to retain YesToAll and NoToAll input
that
                        // the user may enter across multiple calls to this
function,
                        // they are stored as private members of the cmdlet.
                        if (!ShouldContinue(message, "Warning!",
                                    ref yesToAll, ref noToAll))
                        {
                            continue;
                        }
                    } // if (criticalProcess...

                    // Display a warning message if the cmdlet is stopping a
                    // critical process.
                    if (criticalProcess)
                    {
                        message = String.Format(CultureInfo.CurrentCulture,
                                    "Stopping the critical process \"{0}\".",
                                        processName);
                        WriteWarning(message);
                    } // if (criticalProcess...

                    // Stop the named process.
                    try
                    {
                        process.Kill();
                    }
                    catch (Exception e)
```

```
                {
                    if ((e is Win32Exception) || (e is SystemException) ||
                        (e is InvalidOperationException))
                    {
                        // This process could not be stopped so write
                        // a non-terminating error.
                        WriteError(new ErrorRecord(
                                        e,
                                        "CouldNotStopProcess",
                                        ErrorCategory.CloseError,
                                        process)
                                    );
                        continue;
                    } // if ((e is...
                        else throw;
                } // catch

                message = String.Format(CultureInfo.CurrentCulture,
                               "Stopped process \"{0}\", pid {1}.",
                                     processName, process.Id);

                WriteVerbose(message);

                // If the PassThru parameter is specified,
                // return the terminated process object to the pipeline.
                if (passThru)
                {
                    message = String.Format(CultureInfo.CurrentCulture,
                                  "Writing process \"{0}\" to pipeline",
                                       processName);
                    WriteDebug(message);
                    WriteObject(process);
                } // if (passThru...
            } // foreach (Process...
        } // foreach (string...
    } // ProcessRecord

    #endregion Cmdlet Overrides

    #region Private Data

    private bool yesToAll, noToAll;

    /// <summary>
    /// Partial list of critical processes that should not be
    /// stopped.  Lower case is used for case insensitive matching.
    /// </summary>
    private ArrayList criticalProcessNames = new ArrayList(
       new string[] { "system", "winlogon", "spoolsv" }
    );

    #endregion Private Data

} // StopProcCommand
```

```
    #endregion StopProcCommand
}
```

See Also

- Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

StopProcessSample03 Sample

This sample shows how to write a cmdlet whose parameters have aliases and whose parameters support wildcard characters. This cmdlet is similar to the `Stop-Process` cmdlet provided by Windows PowerShell 2.0.

How to build the sample by using Visual Studio

1. With the Windows PowerShell 2.0 SDK installed, navigate to the StopProcessSample03 folder. The default location is `C:\Program Files (x86)\Microsoft SDKs\Windows\v7.0\Samples\sysmgmt\WindowsPowerShell\csharp\StopProcessSample03`.
2. Double-click the icon for the solution (.sln) file. This opens the sample project in Microsoft Visual Studio.
3. In the **Build** menu, select **Build Solution** to build the library for the sample in the default `\bin` or `\bin\debug` folders.

How to run the sample

1. Create the following module folder:

   `[user]\Documents\WindowsPowerShell\Modules\StopProcessSample03`

2. Copy the sample assembly to the module folder.
3. Start Windows PowerShell.
4. Run the following command to load the assembly into Windows PowerShell:

   `Import-Module stopprossessample03`

5. Run the following command to run the cmdlet:

   `Stop-Proc`

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Declaring a cmdlet class by using the Cmdlet attribute.
- Declaring a cmdlet parameters by using the Parameter attribute.
- Adding aliases to parameter declarations..
- Adding wildcard support to parameters.

Example

This sample shows how to declare parameter aliases and support wildcards.

C#

```
using System;
using System.Diagnostics;
using System.Collections;
using Win32Exception = System.ComponentModel.Win32Exception;
using System.Management.Automation;             //Windows PowerShell namespace
using System.Globalization;

namespace Microsoft.Samples.PowerShell.Commands
{

   #region StopProcCommand

    /// <summary>
   /// This class implements the Stop-Proc cmdlet.
   /// </summary>
   [Cmdlet(VerbsLifecycle.Stop, "Proc",
       SupportsShouldProcess = true)]
   public class StopProcCommand : Cmdlet
   {
       #region Parameters

      /// <summary>
      /// This parameter provides the list of process names on
      /// which the Stop-Proc cmdlet will work.
      /// </summary>
       [Parameter(
          Position = 0,
          Mandatory = true,
          ValueFromPipeline = true,
          ValueFromPipelineByPropertyName = true,
          HelpMessage = "The name of one or more processes to stop. Wildcards are
permitted."
```

```
        )]
        [Alias("ProcessName")]
        public string[] Name
        {
            get { return processNames; }
            set { processNames = value; }
        }
        private string[] processNames;

        /// <summary>
        /// This parameter overrides the ShouldContinue call to force
        /// the cmdlet to stop its operation. This parameter should always
        /// be used with caution.
        /// </summary>
        [Parameter]
        public SwitchParameter Force
        {
           get { return force; }
           set { force = value; }
        }
        private bool force;

        /// <summary>
        /// This parameter indicates that the cmdlet should return
        /// an object to the pipeline after the processing has been
        /// completed.
        /// </summary>
        [Parameter(
           ValueFromPipelineByPropertyName = true,
           HelpMessage = "If set, the process(es) will be passed to the pipeline
after stopped."
        )]
        public SwitchParameter PassThru
        {
           get { return passThru; }
           set { passThru = value; }
        }
        private bool passThru;

        #endregion Parameters

        #region Cmdlet Overrides
        /// <summary>
        /// The ProcessRecord method does the following for each of the
        /// requested process names:
        /// 1) Check that the process is not a critical process.
        /// 2) Attempt to stop that process.
        /// If no process is requested then nothing occurs.
        /// </summary>
        protected override void ProcessRecord()
        {
            Process[] processes = null;

            try
            {
```

```
            processes = Process.GetProcesses();
        }
        catch (InvalidOperationException ioe)
        {
            base.ThrowTerminatingError(new ErrorRecord(ioe,
                      "UnableToAccessProcessList",
                          ErrorCategory.InvalidOperation,
                              null));
        }

        // For every process name passed to the cmdlet, get the associated
        // processes.
        // Write a non-terminating error for failure to retrieve
        // a process.
        foreach (string name in processNames)
        {
            // Write a user-friendly verbose message to the pipeline. These
            // messages are intended to give the user detailed information
            // on the operations performed by the cmdlet. These messages will
            // appear with the -Verbose option.
            string message = String.Format(CultureInfo.CurrentCulture,
                                 "Attempting to stop process \"{0}\".", name);
            WriteVerbose(message);

            // Validate the process name against a wildcard pattern.
            // If the name does not contain any wildcard patterns, it
            // will be treated as an exact match.
            WildcardOptions options = WildcardOptions.IgnoreCase |
                                      WildcardOptions.Compiled;
            WildcardPattern wildcard = new WildcardPattern(name,options);

            foreach (Process process in processes)
            {
                string processName;

                try
                {
                    processName = process.ProcessName;
                }
                catch (Win32Exception e)
                {
                    WriteError(new ErrorRecord(
                                            e, "ProcessNameNotFound",
                                              ErrorCategory.ObjectNotFound,
                                                process)
                               );
                    continue;
                }

                // Write a debug message to the host that can be used when
                // troubleshooting a problem. All debug messages will appear
                // with the -Debug option.
                message = String.Format(CultureInfo.CurrentCulture,
                             "Acquired name for pid {0} : \"{1}\"",
                                   process.Id, processName);
```

```
                    WriteDebug(message);

                    // Check to see if this process matches the current process
                    // name pattern. Skip this process if it does not.
                    if (!wildcard.IsMatch(processName))
                    {
                        continue;
                    }

                    // Stop the process.
                    SafeStopProcess(process);
                } // foreach (Process...
            } // foreach (string...
        } // ProcessRecord

        #endregion Cmdlet Overrides

        #region Helper Methods

        /// <summary>
        /// Safely stops a named process.  Used as standalone function
        /// to declutter the ProcessRecord method.
        /// </summary>
        /// <param name="process">The process to stop.</param>
        private void SafeStopProcess(Process process)
        {
            string processName = null;
            try
            {
                processName = process.ProcessName;
            }
            catch (Win32Exception e)
            {
                WriteError(new ErrorRecord(e, "ProcessNameNotFound",
                                    ErrorCategory.ObjectNotFound, process));
                return;
            }

            string message = null;

            // Confirm the operation first.
            // This is always false if the WhatIf parameter is specified.
            if (!ShouldProcess(string.Format(CultureInfo.CurrentCulture,
                     "{0} ({1})", processName, process.Id)))
            {
                return;
            }

            // Make sure that the user really wants to stop a critical
            // process that could possibly stop the computer.
            bool criticalProcess =
criticalProcessNames.Contains(processName.ToLower(CultureInfo.CurrentCulture));

            if (criticalProcess && !force)
            {
```

```
                message = String.Format(CultureInfo.CurrentCulture,
                            "The process \"{0}\" is a critical process and should
not be stopped. Are you sure you wish to stop the process?",
                                processName);

                // It is possible that ProcessRecord is called multiple
                // when objects are received as inputs from a pipeline.
                // So, to retain YesToAll and NoToAll input that the
                // user may enter across multiple calls to this
                // function, they are stored as private members of the
                // Cmdlet.
                if (!ShouldContinue(message, "Warning!",
                            ref yesToAll, ref noToAll))
                {
                    return;
                }
            } // if (criticalProcess...

            // Display a warning message if stopping a critical
            // process.
            if (criticalProcess)
            {
                message = String.Format(CultureInfo.CurrentCulture,
                            "Stopping the critical process \"{0}\".",
                                processName);
                WriteWarning(message);
            } // if (criticalProcess...

            try
            {
                // Stop the process.
                process.Kill();
            }
            catch (Exception e)
            {
                if ((e is Win32Exception) || (e is SystemException) ||
                    (e is InvalidOperationException))
                {
                    // This process could not be stopped so write
                    // a non-terminating error.
                    WriteError(new ErrorRecord(e, "CouldNotStopProcess",
                                    ErrorCategory.CloseError,
                                    process)
                                );
                    return;
                } // if ((e is...
                else throw;
            } // catch

            message = String.Format(CultureInfo.CurrentCulture,
                         "Stopped process \"{0}\", pid {1}.",
                               processName, process.Id);

            WriteVerbose(message);
```

```
            // If the PassThru parameter is specified,
            // return the terminated process to the pipeline.
            if (passThru)
            {
                message = String.Format(CultureInfo.CurrentCulture,
                            "Writing process \"{0}\" to pipeline",
                                processName);
                WriteDebug(message);
                WriteObject(process);
            } // if (passThru...
        } // SafeStopProcess

        #endregion Helper Methods

        #region Private Data

        private bool yesToAll, noToAll;

        /// <summary>
        /// Partial list of the critical processes that should not be
        /// stopped.  Lower case is used for case insensitive matching.
        /// </summary>
        private ArrayList criticalProcessNames = new ArrayList(
           new string[] { "system", "winlogon", "spoolsv" }
        );

        #endregion Private Data

    } // StopProcCommand

    #endregion StopProcCommand
} // namespace Microsoft.Samples.PowerShell.Commands
```

See Also

- Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

StopProcessSample04 Sample

This sample shows how to write a cmdlet that declares parameter sets, specifies the default parameter set, and can accept an input object. This cmdlet is similar to the `Stop-Process` cmdlet provided by Windows PowerShell 2.0.

How to build the sample by using Visual Studio

1. With the Windows PowerShell 2.0 SDK installed, navigate to the StopProcessSample04 folder. The default location is `C:\Program Files (x86)\Microsoft SDKs\Windows\v7.0\Samples\sysmgmt\WindowsPowerShell\csharp\StopProcessSample04`.

2. Double-click the icon for the solution (.sln) file. This opens the sample project in Microsoft Visual Studio.

3. In the **Build** menu, select **Build Solution** to build the library for the sample in the default `\bin` or `\bin\debug` folders.

How to run the sample

1. Create the following module folder:

   `[user]\Documents\WindowsPowerShell\Modules\StopProcessSample04`

2. Copy the sample assembly to the module folder.

3. Start Windows PowerShell.

4. Run the following command to load the assembly into Windows PowerShell:

   `Import-Module stopprossessample04`

5. Run the following command to run the cmdlet:

   `Stop-Proc`

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Declaring a cmdlet class by using the Cmdlet attribute.
- Declaring a cmdlet parameters by using the Parameter attribute.
- Adding a parameter that accepts input object.
- Adding parameters to parameter sets
- Specifying the default parameter set.

Example

The following code shows an implementation of the Stop-Proc cmdlet that declare parameter sets, specifies the default parameter set, and can accept an input object.

This sample shows the input object, how to declare parameter sets, and how to specify the default parameter set to use.

C#

```
using System;
using System.Diagnostics;
using System.Collections;
using Win32Exception = System.ComponentModel.Win32Exception;
using System.Management.Automation;             //Windows PowerShell namespace
using System.Globalization;

namespace Microsoft.Samples.PowerShell.Commands
{
   #region StopProcCommand

   /// <summary>
   /// This class implements the Stop-Proc cmdlet.
   /// </summary>
   [Cmdlet(VerbsLifecycle.Stop, "Proc",
       DefaultParameterSetName = "ProcessId",
       SupportsShouldProcess = true)]
   public class StopProcCommand : PSCmdlet
   {
       #region Parameters

      /// <summary>
      /// This parameter provides the list of process names on
      /// which the Stop-Proc cmdlet will work.
```

```
      /// </summary>
       [Parameter(
          Position = 0,
          ParameterSetName = "ProcessName",
          Mandatory = true,
          ValueFromPipeline = true,
          ValueFromPipelineByPropertyName = true,
          HelpMessage = "The name of one or more processes to stop. Wildcards are
permitted."
       )]
       [Alias("ProcessName")]
       public string[] Name
       {
           get { return processNames; }
           set { processNames = value; }
       }
       private string[] processNames;

       /// <summary>
       /// This parameter overrides the ShouldContinue call to force
       /// the cmdlet to stop its operation. This parameter should always
       /// be used with caution.
       /// </summary>
       [Parameter]
       public SwitchParameter Force
       {
           get { return force; }
           set { force = value; }
       }
       private bool force;

       /// <summary>
       /// This parameter indicates that the cmdlet should return
       /// an object to the pipeline after the processing has been
       /// completed.
       /// </summary>
       [Parameter(
          HelpMessage = "If set the process(es) will be passed to the pipeline
after stopped."
       )]
       public SwitchParameter PassThru
       {
           get { return passThru; }
           set { passThru = value; }
       }
       private bool passThru;

      /// This parameter provides the list of process identifiers on
      /// which the Stop-Proc cmdlet will work.
       [Parameter(
          ParameterSetName = "ProcessId",
          Mandatory = true,
          ValueFromPipelineByPropertyName = true,
          ValueFromPipeline = true
       )]
```

```
        [Alias("ProcessId")]
        public int[] Id
        {
            get { return processIds; }
            set { processIds = value; }
        }
        private int[] processIds;

        /// <summary>
        /// This parameter accepts an array of Process objects from the
        /// the pipeline. This object contains the processes to stop.
        /// </summary>
        /// <value>Process objects</value>
        [Parameter(
            ParameterSetName = "InputObject",
            Mandatory = true,
            ValueFromPipeline = true)]
        public Process[] InputObject
        {
            get { return inputObject; }
            set { inputObject = value; }
        }
        private Process[] inputObject;

        #endregion Parameters

        #region CmdletOverrides

        /// <summary>
        /// The ProcessRecord method does the following for each of the
        /// requested process names:
        /// 1) Check that the process is not a critical process.
        /// 2) Attempt to stop that process.
        /// If no process is requested then nothing occurs.
        /// </summary>
        protected override void ProcessRecord()
        {
            switch (ParameterSetName)
            {
                case "ProcessName":
                    ProcessByName();
                break;

                case "ProcessId":
                    ProcessById();
                    break;

                case "InputObject":
                    foreach (Process process in inputObject)
                    {
                        SafeStopProcess(process);
                    }
                    break;

                default:
```

```
                    throw new ArgumentException("Bad ParameterSet Name");
            } // switch (ParameterSetName...
        } // ProcessRecord

        #endregion Cmdlet Overrides

        #region Helper Methods

        /// <summary>
        /// Returns all processes with matching names.
        /// </summary>
        /// <param name="processName">
        /// The name of the processes to return.
        /// </param>
        /// <param name="allProcesses">An array of all
        /// computer processes.</param>
        /// <returns>An array of matching processes.</returns>
        internal ArrayList SafeGetProcessesByName(string processName,
                                  ref ArrayList allProcesses)
        {
            // Create and array to store the matching processes.
            ArrayList matchingProcesses = new ArrayList();

            // Create the wildcard for pattern matching.
            WildcardOptions options = WildcardOptions.IgnoreCase |
                                      WildcardOptions.Compiled;
            WildcardPattern wildcard = new WildcardPattern(processName, options);

            // Walk all of the machine processes.
            foreach(Process process in allProcesses)
            {
                string processNameToMatch = null;
                try
                {
                    processNameToMatch = process.ProcessName;
                }
                catch (Win32Exception e)
                {
                    // Remove the process from the list so that it is not
                    // checked again.
                    allProcesses.Remove(process);

                    string message =
                          String.Format(CultureInfo.CurrentCulture, "The process \"
{0}\" could not be found",
                                                processName);
                    WriteVerbose(message);
                    WriteError(new ErrorRecord(e, "ProcessNotFound",
                                     ErrorCategory.ObjectNotFound, processName));

                    continue;
                }

                if (!wildcard.IsMatch(processNameToMatch))
                {
```

```
                    continue;
                }

                matchingProcesses.Add(process);
            } // foreach(Process...

            return matchingProcesses;
        } // SafeGetProcessesByName

        /// <summary>
        /// Safely stops a named process.  Used as standalone function
        /// to declutter the ProcessRecord method.
        /// </summary>
        /// <param name="process">The process to stop.</param>
        private void SafeStopProcess(Process process)
        {
            string processName = null;

            try
            {
                processName = process.ProcessName;
            }
            catch (Win32Exception e)
            {
                WriteError(new ErrorRecord(e, "ProcessNotFound",
                                ErrorCategory.OpenError, processName));

                return;
            }

            // Confirm the operation first.
            // This is always false if the WhatIf parameter is specified.
            if (!ShouldProcess(string.Format(CultureInfo.CurrentCulture,
                     "{0} ({1})", processName, process.Id)))
            {
                return;
            }

            // Make sure that the user really wants to stop a critical
            // process that can possibly stop the computer.
            bool criticalProcess =
criticalProcessNames.Contains(processName.ToLower(CultureInfo.CurrentCulture));

            string message = null;
            if (criticalProcess && !force)
            {
                message = String.Format(CultureInfo.CurrentCulture,
                                        "The process \"{0}\" is a critical
process and should not be stopped. Are you sure you wish to stop the process?",
                                            processName);
                // It is possible that the ProcessRecord method is called
                // multiple times when objects are received as inputs from
                // the pipeline. So to retain YesToAll and NoToAll input that
                // the user may enter across multiple calls to this function,
                // they are stored as private members of the cmdlet.
```

```
            if (!ShouldContinue(message, "Warning!",
                        ref yesToAll, ref noToAll))
            {
                return;
            }
        } // if (criticalProcess...

        // Display a warning message if stopping a critical
        // process.
        if (criticalProcess)
        {
            message =
              String.Format(CultureInfo.CurrentCulture,
                            "Stopping the critical process \"{0}\".",
                                processName);
            WriteWarning(message);
        } // if (criticalProcess...

        try
        {
            // Stop the process.
            process.Kill();
        }
        catch (Exception e)
        {
            if ((e is Win32Exception) || (e is SystemException) ||
                (e is InvalidOperationException))
            {
                // This process could not be stopped so write
                // a non-terminating error.
                WriteError(new ErrorRecord(e, "CouldNotStopProcess",
                                ErrorCategory.CloseError,
                                process)
                           );

                return;
            } // if ((e is...
            else throw;
        } // catch

        // Write a user-level verbose message to the pipeline. These are
        // intended to give the user detailed information on the
        // operations performed by the cmdlet. These messages will
        // appear with the -Verbose option.
        message = String.Format(CultureInfo.CurrentCulture,
                                "Stopped process \"{0}\", pid {1}.",
                                    processName, process.Id);

        WriteVerbose(message);

        // If the PassThru parameter is specified, return the terminated
        // process to the pipeline.
        if (passThru)
        {
            // Write a debug message to the host that can be used
```

```
            // when troubleshooting a problem. All debug messages
            // will appear with the -Debug option
            message =
                String.Format(CultureInfo.CurrentCulture,
                              "Writing process \"{0}\" to pipeline",
                                  processName);
            WriteDebug(message);
            WriteObject(process);
        } // if (passThru..
    } // SafeStopProcess

    /// <summary>
    /// Stop processes based on their names (using the
    /// ParameterSetName as ProcessName)
    /// </summary>
    private void ProcessByName()
    {
        ArrayList allProcesses = null;

        // Get a list of all processes.
        try
        {
            allProcesses = new ArrayList(Process.GetProcesses());
        }
        catch (InvalidOperationException ioe)
        {
            base.ThrowTerminatingError(new ErrorRecord(
                 ioe, "UnableToAccessProcessList",
                 ErrorCategory.InvalidOperation, null));
        }

        // If a process name is passed to the cmdlet, get
        // the associated processes.
        // Write a non-terminating error for failure to
        // retrieve a process.
        foreach (string name in processNames)
        {
            // The allProcesses array list is passed as a reference because
            // any process whose name cannot be obtained will be removed
            // from the list so that its not compared the next time.
            ArrayList processes =
                SafeGetProcessesByName(name, ref allProcesses);

            // If no processes were found write a non-
            // terminating error.
            if (processes.Count == 0)
            {
                WriteError(new ErrorRecord(
                    new Exception("Process not found."),
                    "ProcessNotFound",
                    ErrorCategory.ObjectNotFound,
                    name));
            } // if (processes...
            // Otherwise terminate all processes in the list.
            else
```

```
                {
                    foreach (Process process in processes)
                    {
                        SafeStopProcess(process);
                    } // foreach (Process...
                } // else
            } // foreach (string...
        } // ProcessByName

        /// <summary>
        /// Stop processes based on their identifiers (using the
        /// ParameterSetName as ProcessIds)
        /// </summary>
        internal void ProcessById()
        {
            foreach (int processId in processIds)
            {
                Process process = null;
                try
                {
                    process = Process.GetProcessById(processId);

                    // Write a debug message to the host that can be used
                    // when troubleshooting a problem. All debug messages
                    // will appear with the -Debug option
                    string message =
                        String.Format(CultureInfo.CurrentCulture,
                                      "Acquired process for pid : {0}",
                                          process.Id);
                    WriteDebug(message);
                }
                catch (ArgumentException ae)
                {
                    string
                        message = String.Format(CultureInfo.CurrentCulture,
                                          "The process id {0} could not be
found",
                                              processId);
                    WriteVerbose(message);
                    WriteError(new ErrorRecord(ae, "ProcessIdNotFound",
                                    ErrorCategory.ObjectNotFound, processId));
                    continue;
                }

                SafeStopProcess(process);
            } // foreach (int...
        } // ProcessById

        #endregion Helper Methods

        #region Private Data

        private bool yesToAll, noToAll;

        /// <summary>
```

```
        /// Partial list of critical processes that should not be
        /// stopped.  Lower case is used for case insensitive matching.
        /// </summary>
        private ArrayList criticalProcessNames = new ArrayList(
           new string[] { "system", "winlogon", "spoolsv", "calc" }
        );

        #endregion Private Data

    } // StopProcCommand

    #endregion StopProcCommand
}
```

See Also

- Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Events01 Sample

This sample shows how to create a cmdlet that allows the user to register for events that are raised by System.IO.FileSystemWatcher. With this cmdlet, users can register an action to execute when a file is created under a specific directory. This sample derives from the Microsoft.PowerShell.Commands.ObjectEventRegistrationBase base class.

How to build the sample by using Visual Studio

1. With the Windows PowerShell 2.0 SDK installed, navigate to the Events01 folder. The default location is `C:\Program Files (x86)\Microsoft SDKs\Windows\v7.0\Samples\sysmgmt\WindowsPowerShell\csharp\Events01`.

2. Double-click the icon for the solution (.sln) file. This opens the sample project in Microsoft Visual Studio.

3. In the **Build** menu, select **Build Solution** to build the library for the sample in the default `\bin` or `\bin\debug` folders.

How to run the sample

1. Create the following module folder:

   `[user]\Documents\WindowsPowerShell\Modules\events01`

2. Copy the library file for the sample to the module folder.

3. Start Windows PowerShell.

4. Run the following command to load the cmdlet into Windows PowerShell:

   PowerShell

   ```
   Import-Module events01
   ```

5. Use the Register-FileSystemEvent cmdlet to register an action that will write a message when a file is created under the TEMP directory.

   PowerShell

```
Register-FileSystemEvent $Env:TEMP Created -Filter "*.txt" -Action { Write-
Host "A file was created in the TEMP directory" }
```

6. Create a file under the TEMP directory and note that the action is executed (the message is displayed).

This is a sample output that results by following these steps.

Output

```
Id              Name            State      HasMoreData     Location
Command
--              ----            -----      -----------     --------             ---
----
1               26932870-d3b... NotStarted False
Write-Host "A f...
```

PowerShell

```
Set-Content $Env:TEMP\test.txt "This is a test file"
```

Output

```
A file was created in the TEMP directory
```

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

How to write a cmdlet for event registration

The cmdlet derives from the Microsoft.PowerShell.Commands.ObjectEventRegistrationBase class, which provides support for parameters common to the `Register-*Event` cmdlets. Cmdlets that are derived from Microsoft.PowerShell.Commands.ObjectEventRegistrationBase need only to define their particular parameters and override the `GetSourceObject` and `GetSourceObjectEventName` abstract methods.

Example

This sample shows how to register for events raised by System.IO.FileSystemWatcher.

C#

```
namespace Sample
{
    using System;
    using System.IO;
    using System.Management.Automation;
    using System.Management.Automation.Runspaces;
    using Microsoft.PowerShell.Commands;

    [Cmdlet(VerbsLifecycle.Register, "FileSystemEvent")]
    public class RegisterObjectEventCommand : ObjectEventRegistrationBase
    {
        /// <summary>The FileSystemWatcher that exposes the events.</summary>
        private FileSystemWatcher fileSystemWatcher = new FileSystemWatcher();

        /// <summary>Name of the event to which the cmdlet registers.</summary>
        private string eventName = null;

        /// <summary>
        /// Gets or sets the path that will be monitored by the FileSystemWatcher.
        /// </summary>
        [Parameter(Mandatory = true, Position = 0)]
        public string Path
        {
            get
            {
                return this.fileSystemWatcher.Path;
            }

            set
            {
                this.fileSystemWatcher.Path = value;
            }
        }

        /// <summary>
        /// Gets or sets the name of the event to which the cmdlet registers.
        /// <para>
        /// Currently System.IO.FileSystemWatcher exposes 6 events: Changed,
Created,
        /// Deleted, Disposed, Error, and Renamed. Check the documentation of
        /// FileSystemWatcher for details on each event.
        /// </para>
        /// </summary>
        [Parameter(Mandatory = true, Position = 1)]
        public string EventName
        {
            get
```

```
            {
                return this.eventName;
            }

            set
            {
                this.eventName = value;
            }
        }

        /// <summary>
        /// Gets or sets the filter that will be user by the FileSystemWatcher.
        /// </summary>
        [Parameter(Mandatory = false)]
        public string Filter
        {
            get
            {
                return this.fileSystemWatcher.Filter;
            }

            set
            {
                this.fileSystemWatcher.Filter = value;
            }
        }

        /// <summary>
        /// Derived classes must implement this method to return the object that
generates
        /// the events to be monitored.
        /// </summary>
        /// <returns> This sample returns an instance of
System.IO.FileSystemWatcher</returns>
        protected override object GetSourceObject()
        {
            return this.fileSystemWatcher;
        }

        /// <summary>
        /// Derived classes must implement this method to return the name of the
event to
        /// be monitored. This event must be exposed by the input object.
        /// </summary>
        /// <returns> This sample returns the event specified by the user with the
-EventName parameter.</returns>
        protected override string GetSourceObjectEventName()
        {
            return this.eventName;
        }
    }
}
```

See Also

- Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Writing a Windows PowerShell Module

This document is written for administrators, script developers, and cmdlet developers who need to package and distribute their Windows PowerShell cmdlets. By using Windows PowerShell modules, you can package and distribute your Windows PowerShell solutions without using a compiled language.

Windows PowerShell modules enable you to partition, organize, and abstract your Windows PowerShell code into self-contained, reusable units. With these reusable units, you can easily share your modules directly with others. If you are a script developer, you can also repackage third-party modules to create custom script-based applications. Modules, similar to modules in other scripting languages such as Perl and Python, enable production-ready scripting solutions that use reusable, redistributable components, with the added benefit of enabling you to repackage and abstract multiple components to create custom solutions.

At their most basic, Windows PowerShell will treat any valid Windows PowerShell script code saved in a `.psm1` file as a module. PowerShell will also automatically treat any binary cmdlet assembly as a module. However, you can also use a module (or more specifically, a module manifest) to bundle an entire solution together. The following scenarios describe typical uses for Windows PowerShell modules.

Libraries

Modules can be used to package and distribute cohesive libraries of functions that perform common tasks. Typically, the names of these functions share one or more nouns that reflect the common task that they are used for. These functions can also be similar to .NET Framework classes in that they can have public and private members. For example, a library can contain a set of functions for file transfers. In this case, the noun reflecting the common task might be "file."

Configuration

Modules can be used to customize your environment by adding specific cmdlets, providers, functions, and variables.

Compiled Code Development and Distribution

Cmdlet and provider developers can use modules to test and distribute their compiled code without needing to create snap-ins. They can import the assembly that contains the compiled code as a module (a binary module) without needing to create and register snap-ins.

See Also

Understanding a Windows PowerShell Module

How to Write a PowerShell Script Module

How to Write a PowerShell Binary Module

How to Write a PowerShell Module Manifest

about_PSModulePath

Importing a PowerShell Module

Installing a PowerShell Module

Last updated on 05/20/2025

Understanding a Windows PowerShell Module

A *module* is a set of related Windows PowerShell functionalities, grouped together as a convenient unit (usually saved in a single directory). By defining a set of related script files, assemblies, and related resources as a module, you can reference, load, persist, and share your code much easier than you would otherwise.

The main purpose of a module is to allow the reuse and abstraction of Windows PowerShell code. For example, the most basic way of creating a module is to simply save a Windows PowerShell script as a `.psm1` file. Doing so allows you to control which functions and variables are exposed to users variables in the script are public or private. Saving the script as a `.psm1` file also allows you to control the scope of certain variables. Finally, you can also use cmdlets such as Install-Module to organize, install, and use your script as a building block for larger solutions.

Module Components and Types

A module is made up of four basic components:

1. Some sort of code file - usually either a PowerShell script or a managed cmdlet assembly.
2. Anything else that the above code file may need, such as additional assemblies, help files, or scripts.
3. A manifest file that describes the above files, as well as stores metadata such as author and versioning information.
4. A directory that contains all of the above content, and is located where PowerShell can reasonably find it.

> **Note**
>
> None of these components, by themselves, are actually necessary. For example, a module can technically be only a script stored in a `.psm1` file. You can also have a module that's nothing but a manifest file, which is used mainly for organizational purposes. You can also write a script that dynamically creates a module, and as such doesn't actually need a directory to store anything in. The following sections describe

the types of modules you can get by mixing and matching the different possible parts of a module together.

Script Modules

As the name implies, a *script module* is a file (`.psm1`) that contains any valid Windows PowerShell code. Script developers and administrators can use this type of module to create modules whose members include functions, variables, and more. At heart, a script module is simply a Windows PowerShell script with a different extension, which allows administrators to use import, export, and management functions on it.

In addition, you can use a manifest file to include other resources in your module, such as data files, other dependent modules, or runtime scripts. Manifest files are also useful for tracking metadata such as authoring and versioning information.

Finally, a script module, like any other module that isn't dynamically created, needs to be saved in a folder that PowerShell can reasonably discover. Usually, this is on the PowerShell module path; but if necessary you can explicitly describe where your module is installed. For more information, see How to Write a PowerShell Script Module.

Binary Modules

A **binary module** is a .NET Framework assembly (`.dll`) that contains compiled code, such as C#. Cmdlet developers can use this type of module to share cmdlets, providers, and more. Existing snap-ins can also be used as binary modules. Compared to a script module, a binary module allows you to create cmdlets that are faster or use features (such as multithreading) that aren't as easy to code in Windows PowerShell scripts.

As with script modules, you can include a manifest file to describe additional resources that your module uses, and to track metadata about your module. Similarly, you probably should install your binary module in a folder somewhere along the PowerShell module path. For more information, see How to How to Write a PowerShell Binary Module.

Manifest Modules

A **manifest module** is a module that uses a manifest file to describe all of its components, but doesn't have any sort of core assembly or script. Formally, a manifest module leaves the `ModuleToProcess` or `RootModule` element of the manifest empty. However, you can still use the other features of a module, such as the ability to load up dependent assemblies or automatically

run certain pre-processing scripts. You can also use a manifest module as a convenient way to package up resources that other modules will use, such as nested modules, assemblies, types, or formats. For more information, see How to Write a PowerShell Module Manifest.

Dynamic Modules

A **dynamic module** is a module that isn't loaded from, or saved to, a file. Instead, they're created dynamically by a script, using the New-Module cmdlet. This type of module enables a script to create a module on demand that doesn't need to be loaded or saved to persistent storage. By its nature, a dynamic module is intended to be short-lived, and therefore can't be accessed by the `Get-Module` cmdlet. Similarly, they usually don't need module manifests, nor do they likely need permanent folders to store their related assemblies.

Module Manifests

A **module manifest** is a `.psd1` file that contains a hash table. The keys and values in the hash table do the following things:

- Describe the contents and attributes of the module.
- Define the prerequisites.
- Determine how the components are processed.

Manifests aren't required for a module. Modules can reference script files (`.ps1`), script module files (`.psm1`), manifest files (`.psd1`), formatting and type files (`.ps1xml`), cmdlet and provider assemblies (`.dll`), resource files, Help files, localization files, or any other type of file or resource that's bundled as part of the module. For an internationalized script, the module folder also contains a set of message catalog files. If you add a manifest file to the module folder, you can reference the multiple files as a single unit by referencing the manifest.

The manifest itself describes the following categories of information:

- Metadata about the module, such as the module version number, the author, and the description.
- Prerequisites needed to import the module, such as the Windows PowerShell version, the common language runtime (CLR) version, and the required modules.
- Processing directives, such as the scripts, formats, and types to process.
- Restrictions on the members of the module to export, such as the aliases, functions, variables, and cmdlets to export.

For more information, see How to Write a PowerShell Module Manifest.

Storing and Installing a Module

Once you have created a script, binary, or manifest module, you can save your work in a location that others may access it. For example, your module can be stored in the system folder where Windows PowerShell is installed, or it can be stored in a user folder.

Generally speaking, you can determine where you should install your module by using one of the paths stored in the `$Env:PSModulePath` variable. Using one of these paths means that PowerShell can automatically find and load your module when a user makes a call to it in their code. If you store your module somewhere else, you can explicitly let PowerShell know by passing in the location of your module as a parameter when you call `Install-Module`.

Regardless, the path of the folder is referred to as the **base** of the module (ModuleBase), and the name of the script, binary, or manifest module file should be the same as the module folder name, with the following exceptions:

- Dynamic modules that are created by the `New-Module` cmdlet can be named using the `Name` parameter of the cmdlet.
- Modules imported from assembly objects by the `Import-Module -Assembly` command are named according to the following syntax: `"dynamic_code_module_" + assembly.GetName()`.

For more information, see Installing a PowerShell Module and about_PSModulePath.

Module Cmdlets and Variables

The following cmdlets and variables are provided by Windows PowerShell for the creation and management of modules.

- New-Module creates a new dynamic module that exists only in memory. The module is created from a script block, and its exported members, such as its functions and variables, are immediately available in the session and remain available until the session is closed.
- New-ModuleManifest creates a new module manifest (`.psd1`) file, populates its values, and saves the manifest file to the specified path. This cmdlet can also be used to create a module manifest template that can be filled in manually.
- Import-Module adds one or more modules to the current session.
- Get-Module retrieves information about the modules that have been or that can be imported into the current session.
- Export-ModuleMember specifies the module members (such as cmdlets, functions, variables, and aliases) that are exported from a script module (`.psm1`) file or from a dynamic

module created by using the `New-Module` cmdlet.

- Remove-Module removes modules from the current session.
- Test-ModuleManifest verifies that a module manifest accurately describes the components of a module by verifying that the files that are listed in the module manifest file (`.psd1`) actually exist in the specified paths.
- `$PSScriptRoot` contains the directory from which the script module is being executed. It enables scripts to use the module path to access other resources.
- `$Env:PSModulePath` contains a list of the directories in which Windows PowerShell modules are stored. Windows PowerShell uses the value of this variable when importing modules automatically and updating Help topics for modules.

See Also

Writing a Windows PowerShell Module

Last updated on 06/16/2026

How to Write a PowerShell Script Module

A script module is any valid PowerShell script saved in a `.psm1` extension. This extension allows the PowerShell engine to use rules and module cmdlets on your file. Most of these capabilities are there to help you install your code on other systems, as well as manage scoping. You can also use a module manifest file, which describes more complex installations and solutions.

Writing a PowerShell script module

To create a script module, save a valid PowerShell script to a `.psm1` file. The script and the directory where it's stored must use the same name. For example, a script named `MyPsScript.psm1` is stored in a directory named `MyPsScript`.

The module's directory needs to be in a path specified in `$Env:PSModulePath`. The module's directory can contain any resources that are needed to run the script, and a module manifest file that describes to PowerShell how your module works.

Create a basic PowerShell module

The following steps describe how to create a PowerShell module.

1. Save a PowerShell script with a `.psm1` extension. Use the same name for the script and the directory where the script is saved.

   Saving a script with the `.psm1` extension means that you can use the module cmdlets, such as Import-Module. The module cmdlets exist primarily so that you can import and export your code onto other user's systems. The alternate solution would be to load your code on other systems and then dot-source it into active memory, which isn't a scalable solution. For more information, see Understanding a Windows PowerShell Module. By default, when users import your `.psm1` file, all functions in your script are accessible, but variables aren't.

   An example PowerShell script, entitled `Show-Calendar`, is available at the end of this article.

   PowerShell

   ```
   function Show-Calendar {
   param(
   ```

```
    [datetime] $Start = [datetime]::Today,
    [datetime] $End = $Start,
    $FirstDayOfWeek,
    [int[]] $HighlightDay,
    [string[]] $HighlightDate = [datetime]::Today.ToString('yyyy-MM-dd')
    )

    #actual code for the function goes here see the end of the topic for the
complete code sample
}
```

2. To control user access to certain functions or variables, call Export-ModuleMember at the end of your script.

   The example code at the bottom of the article has only one function, which by default would be exposed. However, it's recommended you explicitly call out which functions you wish to expose, as described in the following code:

   PowerShell

   ```
   function Show-Calendar {
         }
   Export-ModuleMember -Function Show-Calendar
   ```

   You can restrict what's imported using a module manifest. For more information, see Importing a PowerShell Module and How to Write a PowerShell Module Manifest.

3. If you have modules that your own module needs to load, you can use `Import-Module`, at the top of your module.

   The `Import-Module` cmdlet imports a targeted module onto a system, and can be used at a later point in the procedure to install your own module. The sample code at the bottom of this article doesn't use any import modules. But if it did, they would be listed at the top of the file, as shown in the following code:

   PowerShell

   ```
   Import-Module GenericModule
   ```

4. To describe your module to the PowerShell Help system, you can either use standard help comments inside the file, or create an additional Help file.

   The code sample at the bottom of this article includes the help information in the comments. You could also write expanded XML files that contain additional help content.

For more information, see Writing Help for Windows PowerShell Modules.

5. If you have additional modules, XML files, or other content you want to package with your module, you can use a module manifest.

   A module manifest is a file that contains the names of other modules, directory layouts, versioning numbers, author data, and other pieces of information. PowerShell uses the module manifest file to organize and deploy your solution. For more information, see How to write a PowerShell module manifest.

6. To install and run your module, save the module to one of the appropriate PowerShell paths, and use `Import-Module`.

   The paths where you can install your module are located in the `$Env:PSModulePath` global variable. For example, a common path to save a module on a system would be `%SystemRoot%/users/<user>/Documents/PowerShell/Modules/<moduleName>`. Be sure to create a directory for your module that uses the same name as the script module, even if it's only a single `.psm1` file. If you didn't save your module to one of these paths, you would have to specify the module's location in the `Import-Module` command. Otherwise, PowerShell wouldn't be able to find the module.

   > **Note**
   >
   > Starting with PowerShell 3.0, if you've placed your module in one of the PowerShell module paths, you don't need to explicitly import it. Your module is automatically loaded when a user calls your function. For more information about the module path, see Importing a PowerShell Module and about_PSModulePath.

7. To remove a module from active service in the current PowerShell session, use Remove-Module.

   > **Note**
   >
   > `Remove-Module` removes a module from the current PowerShell session, but doesn't uninstall the module or delete the module's files.

Show-Calendar code example

The following example is a script module that contains a single function named `Show-Calendar`. This function displays a visual representation of a calendar. The sample contains the PowerShell Help strings for the synopsis, description, parameter values, and code. When the module is imported, the `Export-ModuleMember` command ensures that the `Show-Calendar` function is exported as a module member.

PowerShell

```
<#
 .SYNOPSIS
  Displays a visual representation of a calendar.

 .DESCRIPTION
  Displays a visual representation of a calendar. This function supports multiple
months
  and lets you highlight specific date ranges or days.

 .PARAMETER Start
  The first month to display.

 .PARAMETER End
  The last month to display.

 .PARAMETER FirstDayOfWeek
  The day of the month on which the week begins.

 .PARAMETER HighlightDay
  Specific days (numbered) to highlight. Used for date ranges like (25..31).
  Date ranges are specified by the Windows PowerShell range syntax. These dates are
  enclosed in square brackets.

 .PARAMETER HighlightDate
  Specific days (named) to highlight. These dates are surrounded by asterisks.

 .EXAMPLE
   # Show a default display of this month.
   Show-Calendar

 .EXAMPLE
   # Display a date range.
   Show-Calendar -Start "March, 2010" -End "May, 2010"

 .EXAMPLE
   # Highlight a range of days.
   Show-Calendar -HighlightDay (1..10 + 22) -HighlightDate "2008-12-25"
#>
function Show-Calendar {
param(
    [datetime] $Start = [datetime]::Today,
    [datetime] $End = $Start,
    $FirstDayOfWeek,
    [int[]] $HighlightDay,
```

```
    [string[]] $HighlightDate = [datetime]::Today.ToString('yyyy-MM-dd')
    )

## Determine the first day of the start and end months.
$Start = New-Object DateTime $Start.Year,$Start.Month,1
$End = New-Object DateTime $End.Year,$End.Month,1

## Convert the highlighted dates into real dates.
[datetime[]] $HighlightDate = [datetime[]] $HighlightDate

## Retrieve the DateTimeFormat information so that the
## calendar can be manipulated.
$dateTimeFormat  = (Get-Culture).DateTimeFormat
if($FirstDayOfWeek)
{
    $dateTimeFormat.FirstDayOfWeek = $FirstDayOfWeek
}

$currentDay = $Start

## Process the requested months.
while($Start -le $End)
{
    ## Return to an earlier point in the function if the first day of the month
    ## is in the middle of the week.
    while($currentDay.DayOfWeek -ne $dateTimeFormat.FirstDayOfWeek)
    {
        $currentDay = $currentDay.AddDays(-1)
    }

    ## Prepare to store information about this date range.
    $currentWeek = New-Object PsObject
    $dayNames = @()
    $weeks = @()

    ## Continue processing dates until the function reaches the end of the month.
    ## The function continues until the week is completed with
    ## days from the next month.
    while(($currentDay -lt $Start.AddMonths(1)) -or
        ($currentDay.DayOfWeek -ne $dateTimeFormat.FirstDayOfWeek))
    {
        ## Determine the day names to use to label the columns.
        $dayName = "{0:ddd}" -f $currentDay
        if($dayNames -notcontains $dayName)
        {
            $dayNames += $dayName
        }

        ## Pad the day number for display, highlighting if necessary.
        $displayDay = " {0,2} " -f $currentDay.Day

        ## Determine whether to highlight a specific date.
        if($HighlightDate)
        {
            $compareDate = New-Object DateTime $currentDay.Year,
```

```
                $currentDay.Month,$currentDay.Day
            if($HighlightDate -contains $compareDate)
            {
                $displayDay = "*" + ("{0,2}" -f $currentDay.Day) + "*"
            }
        }

        ## Otherwise, highlight as part of a date range.
        if($HighlightDay -and ($HighlightDay[0] -eq $currentDay.Day))
        {
            $displayDay = "[" + ("{0,2}" -f $currentDay.Day) + "]"
            $null,$HighlightDay = $HighlightDay
        }

        ## Add the day of the week and the day of the month as note properties.
        $currentWeek | Add-Member NoteProperty $dayName $displayDay

        ## Move to the next day of the month.
        $currentDay = $currentDay.AddDays(1)

        ## If the function reaches the next week, store the current week
        ## in the week list and continue.
        if($currentDay.DayOfWeek -eq $dateTimeFormat.FirstDayOfWeek)
        {
            $weeks += $currentWeek
            $currentWeek = New-Object PsObject
        }
    }

    ## Format the weeks as a table.
    $calendar = $weeks | Format-Table $dayNames -AutoSize | Out-String

    ## Add a centered header.
    $width = ($calendar.Split("`n") | Measure-Object -Maximum Length).Maximum
    $header = "{0:MMMM yyyy}" -f $Start
    $padding = " " * (($width - $header.Length) / 2)
    $displayCalendar = " `n" + $padding + $header + "`n " + $calendar
    $displayCalendar.TrimEnd()

    ## Move to the next month.
    $Start = $Start.AddMonths(1)

}
}
Export-ModuleMember -Function Show-Calendar
```

Last updated on 05/20/2025

How to Write a PowerShell Binary Module

06/12/2025

A binary module can be any assembly (`.dll`) that contains cmdlet classes. By default, all the cmdlets in the assembly are imported when the binary module is imported. However, you can restrict the cmdlets that are imported by creating a module manifest whose root module is the assembly. (For example, the **CmdletsToExport** key of the manifest can be used to export only those cmdlets that are needed.) In addition, a binary module can contain additional files, a directory structure, and other pieces of useful management information that a single cmdlet cannot.

The following procedure describes how to create and install a PowerShell binary module.

How to create and install a PowerShell binary module

1. Create a binary PowerShell solution (such as a cmdlet written in C#), with the capabilities you need, and ensure that it runs properly.

   From a code perspective, the core of a binary module is a cmdlet assembly. In fact, PowerShell treats a single cmdlet assembly as a module for loading and unloading, with no additional effort on the part of the developer. For more information about writing a cmdlet, see Writing a Windows PowerShell Cmdlet.

2. If necessary, create the rest of your solution: (additional cmdlets, XML files, and so on) and describe them with a module manifest.

   In addition to describing the cmdlet assemblies in your solution, a module manifest can describe how you want your module exported and imported, what cmdlets will be exposed, and what additional files will go into the module. As stated previously however, PowerShell can treat a binary cmdlet like a module with no additional effort. As such, a module manifest is useful mainly for combining multiple files into a single package, or for explicitly controlling publication for a given assembly. For more information, see How to Write a PowerShell Module Manifest.

   The following code is a simplified C# example that contains three cmdlets in the same file that can be used as a module.

   ```
   C#

   using System.Management.Automation;           // Windows PowerShell
   namespace.
   ```

```
namespace ModuleCmdlets
{
  [Cmdlet(VerbsDiagnostic.Test,"BinaryModuleCmdlet1")]
  public class TestBinaryModuleCmdlet1Command : Cmdlet
  {
    protected override void BeginProcessing()
    {
      WriteObject("BinaryModuleCmdlet1 exported by the ModuleCmdlets
module.");
    }
  }

  [Cmdlet(VerbsDiagnostic.Test, "BinaryModuleCmdlet2")]
  public class TestBinaryModuleCmdlet2Command : Cmdlet
  {
      protected override void BeginProcessing()
      {
          WriteObject("BinaryModuleCmdlet2 exported by the ModuleCmdlets
module.");
      }
  }

  [Cmdlet(VerbsDiagnostic.Test, "BinaryModuleCmdlet3")]
  public class TestBinaryModuleCmdlet3Command : Cmdlet
  {
      protected override void BeginProcessing()
      {
          WriteObject("BinaryModuleCmdlet3 exported by the ModuleCmdlets
module.");
      }
  }

}
```

3. Package your solution, and save the package to somewhere in the PowerShell module path.

   The `$env:PSModulePath` global environment variable describes the default paths that PowerShell uses to locate your module. For example, a common path to save a module on a system would be `%SystemRoot%\Users\<user>\Documents\WindowsPowerShell\Modules\<moduleName>`. If you don't use the default paths, you need to explicitly state the location of your module during installation. Be sure to create a folder to save your module in, as you may need the folder to store multiple assemblies and files for your solution.

   Technically, you don't need to install your module anywhere on the `$env:PSModulePath` - those are simply the default locations that PowerShell will look for your module. However, it's considered best practice to do so, unless you have a good reason for storing your

module somewhere else. For more information, see Installing a PowerShell Module and about_PSModulePath.

4. Import your module into PowerShell with a call to Import-Module.

   Calling to Import-Module loads your module into active memory. If you are using PowerShell 3.0 and later, invoking a command from your module in code also imports it. For more information, see Importing a PowerShell Module.

Module initialization and cleanup code

If your module needs to do something upon import or removal such as a discovery task or initialization, you can implement the IModuleAssemblyInitializer and IModuleAssemblyCleanup interfaces.

> **Note**
>
> This pattern is discouraged unless absolutely necessary. To keep PowerShell performant, you should lazily load things at the point your commands are called rather than on import.

Importing snap-in assemblies as modules

Cmdlets and providers that exist in snap-in assemblies can be loaded as binary modules. When the snap-in assemblies are loaded as binary modules, the cmdlets and providers in the snap-in are available to the user, but the snap-in class in the assembly is ignored, and the snap-in isn't registered. As a result, the snap-in cmdlets provided by Windows PowerShell can't detect the snap-in even though the cmdlets and providers are available to the session.

In addition, any formatting or types files that are referenced by the snap-in can't be imported as part of a binary module. To import the formatting and types files you must create a module manifest. See, How to Write a PowerShell Module Manifest.

See Also

- Writing a Windows PowerShell Module

How to write a PowerShell module manifest

After you've written your PowerShell module, you can add an optional module manifest that includes information about the module. For example, you can describe the author, specify files in the module (such as nested modules), run scripts to customize the user's environment, load type and formatting files, define system requirements, and limit the members that the module exports.

Creating a module manifest

A **module manifest** is a PowerShell data file (`.psd1`) that describes the contents of a module and determines how a module is processed. The manifest file is a text file that contains a hash table of keys and values. You link a manifest file to a module by naming the manifest the same as the module, and storing the manifest in the module's root directory.

For simple modules that contain only a single `.psm1` or binary assembly, a module manifest is optional. But, the recommendation is to use a module manifest whenever possible, as they're useful to help you organize your code and maintain versioning information. And, a module manifest is required to export an assembly that is installed in the Global Assembly Cache. A module manifest is also required for modules that support the Updatable Help feature. Updatable Help uses the **HelpInfoUri** key in the module manifest to find the Help information (HelpInfo XML) file that contains the location of the updated help files for the module. For more information about Updatable Help, see Supporting Updatable Help.

To create and use a module manifest

1. The best practice to create a module manifest is to use the New-ModuleManifest cmdlet. You can use parameters to specify one or more of the manifest's default keys and values. The only requirement is to name the file. `New-ModuleManifest` creates a module manifest with your specified values, and includes the remaining keys and their default values. If you need to create multiple modules, use `New-ModuleManifest` to create a module manifest template that can be modified for your different modules. For an example of a default module manifest, see the Sample module manifest.

   ```
   New-ModuleManifest -Path C:\myModuleName.psd1 -ModuleVersion "2.0" -Author
   "YourNameHere"
   ```

An alternative is to manually create the module manifest's hash table using the minimal information required, the **ModuleVersion**. You save the file with the same name as your module and use the `.psd1` file extension. You can then edit the file and add the appropriate keys and values.

2. Add any additional elements that you want in the manifest file.

   To edit the manifest file, use any text editor you prefer. But, the manifest file is a script file that contains code, so you may wish to edit it in a scripting or development environment, such as Visual Studio Code. All elements of a manifest file are optional, except for the **ModuleVersion** number.

   For more information, see the parameter descriptions in the New-ModuleManifest cmdlet. For descriptions of the keys and values you can include in a module manifest, see about_Module_Manifests.

3. To address any scenarios that might not be covered by the base module manifest elements, you have the option to add additional code to your module manifest.

   For security concerns, PowerShell only runs a small subset of the available operations in a module manifest file. Generally, you can use the `if` statement, arithmetic and comparison operators, and the basic PowerShell data types.

4. After you've created your module manifest, you can test it to confirm that any paths described in the manifest are correct. To test your module manifest, use Test-ModuleManifest.

   ```
   Test-ModuleManifest myModuleName.psd1
   ```

5. Be sure that your module manifest is located in the top level of the directory that contains your module.

   When you copy your module onto a system and import it, PowerShell uses the module manifest to import your module.

6. Optionally, you can directly test your module manifest with a call to Import-Module by dot-sourcing the manifest itself.

   ```
   Import-Module .\myModuleName.psd1
   ```

Sample module manifest

The following sample module manifest was created with `New-ModuleManifest` in PowerShell 7 and contains the default keys and values. For a detailed description of each element in a module manifest, see about_Module_Manifests.

PowerShell

```
#
# Module manifest for module 'SampleModuleManifest'
#
# Generated by: User01
#
# Generated on: 10/15/2019
#

@{

# Script module or binary module file associated with this manifest.
# RootModule = ''

# Version number of this module.
ModuleVersion = '0.0.1'

# Supported PSEditions
# CompatiblePSEditions = @()

# ID used to uniquely identify this module
GUID = 'b632e90c-df3d-4340-9f6c-3b832646bf87'

# Author of this module
Author = 'User01'

# Company or vendor of this module
CompanyName = 'Unknown'

# Copyright statement for this module
Copyright = '(c) User01. All rights reserved.'

# Description of the functionality provided by this module
# Description = ''

# Minimum version of the PowerShell engine required by this module
# PowerShellVersion = ''

# Name of the PowerShell host required by this module
# PowerShellHostName = ''

# Minimum version of the PowerShell host required by this module
# PowerShellHostVersion = ''

# Minimum version of Microsoft .NET Framework required by this module. This
prerequisite is valid for the PowerShell Desktop edition only.
# DotNetFrameworkVersion = ''
```

```
# Minimum version of the common language runtime (CLR) required by this module. This
prerequisite is valid for the PowerShell Desktop edition only.
# CLRVersion = ''

# Processor architecture (None, X86, Amd64) required by this module
# ProcessorArchitecture = ''

# Modules that must be imported into the global environment prior to importing this
module
# RequiredModules = @()

# Assemblies that must be loaded prior to importing this module
# RequiredAssemblies = @()

# Script files (.ps1) that are run in the caller's environment prior to importing this
module.
# ScriptsToProcess = @()

# Type files (.ps1xml) to be loaded when importing this module
# TypesToProcess = @()

# Format files (.ps1xml) to be loaded when importing this module
# FormatsToProcess = @()

# Modules to import as nested modules of the module specified in
RootModule/ModuleToProcess
# NestedModules = @()

# Functions to export from this module, for best performance, do not use wildcards and
do not delete the entry, use an empty array if there are no functions to export.
FunctionsToExport = @()

# Cmdlets to export from this module, for best performance, do not use wildcards and
do not delete the entry, use an empty array if there are no cmdlets to export.
CmdletsToExport = @()

# Variables to export from this module
VariablesToExport = '*'

# Aliases to export from this module, for best performance, do not use wildcards and
do not delete the entry, use an empty array if there are no aliases to export.
AliasesToExport = @()

# DSC resources to export from this module
# DscResourcesToExport = @()

# List of all modules packaged with this module
# ModuleList = @()

# List of all files packaged with this module
# FileList = @()

# Private data to pass to the module specified in RootModule/ModuleToProcess. This may
also contain a PSData hashtable with additional module metadata used by PowerShell.
PrivateData = @{
```

```
    PSData = @{

        # Tags applied to this module. These help with module discovery in online
galleries.
        # Tags = @()

        # A URL to the license for this module.
        # LicenseUri = ''

        # A URL to the main website for this project.
        # ProjectUri = ''

        # A URL to an icon representing this module.
        # IconUri = ''

        # ReleaseNotes of this module
        # ReleaseNotes = ''

        # Prerelease string of this module
        # Prerelease = ''

        # Flag to indicate whether the module requires explicit user acceptance for
install/update/save
        # RequireLicenseAcceptance = $false

        # External dependent modules of this module
        # ExternalModuleDependencies = @()

    } # End of PSData hashtable

} # End of PrivateData hashtable

# HelpInfo URI of this module
# HelpInfoURI = ''

# Default prefix for commands exported from this module. Override the default prefix
using Import-Module -Prefix.
# DefaultCommandPrefix = ''

}
```

See also

- about_Comparison_Operators
- about_If
- Global Assembly Cache
- Import-Module
- New-ModuleManifest
- Test-ModuleManifest

- Update-ModuleManifest
- Writing a Windows PowerShell Module

Last updated on 05/04/2026

Installing a PowerShell Module

After you have created your PowerShell module, you will likely want to install the module on a system, so that you or others may use it. Generally speaking, this consists of copying the module files (ie, the `.psm1`, or the binary assembly, the module manifest, and any other associated files) onto a directory on that computer. For a very small project, this may be as simple as copying and pasting the files with Windows Explorer onto a single remote computer; however, for larger solutions you may wish to use a more sophisticated installation process. Regardless of how you get your module onto the system, PowerShell can use a number of techniques that will let users find and use your modules. Therefore, the main issue for installation is ensuring that PowerShell will be able to find your module. For more information, see Importing a PowerShell Module.

Rules for Installing Modules

The following information pertains to all modules, including modules that you create for your own use, modules that you get from other parties, and modules that you distribute to others.

Install Modules in PSModulePath

Whenever possible, install all modules in a path that is listed in the **PSModulePath** environment variable or add the module path to the **PSModulePath** environment variable value.

The **PSModulePath** environment variable (`$Env:PSModulePath`) contains the locations of Windows PowerShell modules. Cmdlets rely on the value of this environment variable to find modules.

By default, the **PSModulePath** environment variable value contains the following system and user module directories, but you can add to and edit the value.

- `$PSHOME\Modules` (`%windir%\System32\WindowsPowerShell\v1.0\Modules`)

  > ⚠ **Warning**
  >
  > This location is reserved for modules that ship with Windows. Do not install modules to this location.

- `$HOME\Documents\WindowsPowerShell\Modules` (`%HOMEDRIVE%%HOMEPATH%\Documents\WindowsPowerShell\Modules`)
- `$Env:ProgramFiles\WindowsPowerShell\Modules` (`%ProgramFiles%\WindowsPowerShell\Modules`)

  To get the value of the **PSModulePath** environment variable, use either of the following commands.

  PowerShell

  ```
  $Env:PSModulePath
  [Environment]::GetEnvironmentVariable("PSModulePath")
  ```

  To add a module path to value of the **PSModulePath** environment variable value, use the following command format. This format uses the **SetEnvironmentVariable** method of the **System.Environment** class to make a session-independent change to the **PSModulePath** environment variable.

  PowerShell

  ```
  #Save the current value in the $p variable.
  $p = [Environment]::GetEnvironmentVariable("PSModulePath")

  #Add the new path to the $p variable. Begin with a semi-colon separator.
  $p += ";C:\Program Files (x86)\MyCompany\Modules\"

  #Add the paths in $p to the PSModulePath value.
  [Environment]::SetEnvironmentVariable("PSModulePath",$p)
  ```

  > **ⓘ Important**
  >
  > Once you have added the path to **PSModulePath**, you should broadcast an environment message about the change. Broadcasting the change allows other applications, such as the shell, to pick up the change. To broadcast the change, have your product installation code send a **WM_SETTINGCHANGE** message with `lParam` set to the string "Environment". Be sure to send the message after your module installation code has updated **PSModulePath**.

Use the Correct Module Directory Name

A well-formed module is a module that is stored in a directory that has the same name as the base name of at least one file in the module directory. If a module is not well-formed, Windows PowerShell does not recognize it as a module.

The "base name" of a file is the name without the file name extension. In a well-formed module, the name of the directory that contains the module files must match the base name of at least one file in the module.

For example, in the sample Fabrikam module, the directory that contains the module files is named "Fabrikam" and at least one file has the "Fabrikam" base name. In this case, both Fabrikam.psd1 and Fabrikam.dll have the "Fabrikam" base name.

```
C:\Program Files
  Fabrikam Technologies
    Fabrikam Manager
      Modules
        Fabrikam
          Fabrikam.psd1 (module manifest)
          Fabrikam.dll (module assembly)
```

Effect of Incorrect Installation

If the module is not well-formed and its location is not included in the value of the **PSModulePath** environment variable, basic discovery features of Windows PowerShell, such as the following, do not work.

- The Module Auto-Loading feature cannot import the module automatically.
- The `ListAvailable` parameter of the Get-Module cmdlet cannot find the module.
- The Import-Module cmdlet cannot find the module. To import the module, you must provide the full path to the root module file or module manifest file.

  Additional features, such as the following, do not work unless the module is imported into the session. In well-formed modules in the **PSModulePath** environment variable, these features work even when the module is not imported into the session.
- The Get-Command cmdlet cannot find commands in the module.
- The Update-Help and Save-Help cmdlets cannot update or save help for the module.
- The Show-Command cmdlet cannot find and display the commands in the module.

The commands in the module are missing from the `Show-Command` window in Windows PowerShell Integrated Scripting Environment (ISE).

Where to Install Modules

This section explains where in the file system to install Windows PowerShell modules. The location depends on how the module is used.

Installing Modules for a Specific User

If you create your own module or get a module from another party, such as a Windows PowerShell community website, and you want the module to be available for your user account only, install the module in your user-specific Modules directory.

```
$HOME\Documents\WindowsPowerShell\Modules\<Module Folder>\<Module Files>
```

The user-specific Modules directory is added to the value of the **PSModulePath** environment variable by default.

Installing Modules for all Users in Program Files

If you want a module to be available to all user accounts on the computer, install the module in the Program Files location.

```
$Env:ProgramFiles\WindowsPowerShell\Modules\<Module Folder>\<Module Files>
```

> ⓘ **Note**
>
> The Program Files location is added to the value of the PSModulePath environment variable by default in Windows PowerShell 4.0 and later. For earlier versions of Windows PowerShell, you can manually create the Program Files location (%ProgramFiles%\WindowsPowerShell\Modules) and add this path to your PSModulePath environment variable as described above.

Installing Modules in a Product Directory

If you are distributing the module to other parties, use the default Program Files location described above, or create your own company-specific or product-specific subdirectory of the %ProgramFiles% directory.

For example, Fabrikam Technologies, a fictitious company, is shipping a Windows PowerShell module for their Fabrikam Manager product. Their module installer creates a Modules subdirectory in the Fabrikam Manager product subdirectory.

```
C:\Program Files
  Fabrikam Technologies
    Fabrikam Manager
      Modules
        Fabrikam
          Fabrikam.psd1 (module manifest)
          Fabrikam.dll (module assembly)
```

To enable the Windows PowerShell module discovery features to find the Fabrikam module, the Fabrikam module installer adds the module location to the value of the **PSModulePath** environment variable.

PowerShell

```
$p = [Environment]::GetEnvironmentVariable("PSModulePath")
$p += ";C:\Program Files\Fabrikam Technologies\Fabrikam Manager\Modules\"
[Environment]::SetEnvironmentVariable("PSModulePath",$p)
```

Installing Modules in the Common Files Directory

If a module is used by multiple components of a product or by multiple versions of a product, install the module in a module-specific subdirectory of the %ProgramFiles%\Common Files\Modules subdirectory.

In the following example, the Fabrikam module is installed in a Fabrikam subdirectory of the `%ProgramFiles%\Common Files\Modules` subdirectory. Note that each module resides in its own subdirectory in the Modules subdirectory.

```
C:\Program Files
  Common Files
    Modules
      Fabrikam
        Fabrikam.psd1 (module manifest)
        Fabrikam.dll (module assembly)
```

Then, the installer assures the value of the **PSModulePath** environment variable includes the path of the `Common Files\Modules` subdirectory.

PowerShell

```
$m = $Env:ProgramFiles + '\Common Files\Modules'
$p = [Environment]::GetEnvironmentVariable("PSModulePath")
$q = $p -split ';'
if ($q -notcontains $m) {
    $q += ";$m"
}
$p = $q -join ';'
[Environment]::SetEnvironmentVariable("PSModulePath", $p)
```

Installing Multiple Versions of a Module

To install multiple versions of the same module, use the following procedure.

1. Create a directory for each version of the module. Include the version number in the directory name.
2. Create a module manifest for each version of the module. In the value of the **ModuleVersion** key in the manifest, enter the module version number. Save the manifest file (`.psd1`) in the version-specific directory for the module.
3. Add the module root folder path to the value of the **PSModulePath** environment variable, as shown in the following examples.

To import a particular version of the module, the end-user can use the `MinimumVersion` or `RequiredVersion` parameters of the Import-Module cmdlet.

For example, if the Fabrikam module is available in versions 8.0 and 9.0, the Fabrikam module directory structure might resemble the following.

```
C:\Program Files
Fabrikam Manager
 Fabrikam8
   Fabrikam
     Fabrikam.psd1 (module manifest: ModuleVersion = "8.0")
     Fabrikam.dll (module assembly)
 Fabrikam9
   Fabrikam
     Fabrikam.psd1 (module manifest: ModuleVersion = "9.0")
     Fabrikam.dll (module assembly)
```

The installer adds both of the module paths to the **PSModulePath** environment variable value.

PowerShell

```
$p = [Environment]::GetEnvironmentVariable("PSModulePath")
$p += ";C:\Program Files\Fabrikam\Fabrikam8;C:\Program Files\Fabrikam\Fabrikam9"
[Environment]::SetEnvironmentVariable("PSModulePath",$p)
```

When these steps are complete, the **ListAvailable** parameter of the Get-Module cmdlet gets both of the Fabrikam modules. To import a particular module, use the `MinimumVersion` or `RequiredVersion` parameters of the Import-Module cmdlet.

If both modules are imported into the same session, and the modules contain cmdlets with the same names, the cmdlets that are imported last are effective in the session.

Handling Command Name Conflicts

Command name conflicts can occur when the commands that a module exports have the same name as commands in the user's session.

When a session contains two commands that have the same name, Windows PowerShell runs the command type that takes precedence. When a session contains two commands that have the same name and the same type, Windows PowerShell runs the command that was added to the session most recently. To run a command that is not run by default, users can qualify the command name with the module name.

For example, if the session contains a `Get-Date` function and the `Get-Date` cmdlet, Windows PowerShell runs the function by default. To run the cmdlet, preface the command with the module name, such as:

PowerShell

```
Microsoft.PowerShell.Utility\Get-Date
```

To prevent name conflicts, module authors can use the **DefaultCommandPrefix** key in the module manifest to specify a noun prefix for all commands exported from the module.

Users can use the **Prefix** parameter of the `Import-Module` cmdlet to use an alternate prefix. The value of the **Prefix** parameter takes precedence over the value of the **DefaultCommandPrefix** key.

Supporting paths on non-Windows systems

Non-Windows platforms use the colon (`:`) character as a path separator and a forward-slash (`/`) character as a directory separator. The `[System.IO.Path]` class has static members that can be used to make your code work on any platform:

- `[System.IO.Path]::PathSeparator` - returns the character used to separate paths in a PATH environment variable for the host platform
- `[System.IO.Path]::DirectorySeparatorChar` - returns the character used to separate directory names with a path for the host platform

Use these static properties to in place of the `;` and `\` characters when you are constructing path strings.

See Also

about_Command_Precedence

Writing a Windows PowerShell Module

Last updated on 05/20/2025

Registering Cmdlets

The topics in this section describe modules and snap-ins and how to use modules and snap-ins to make cmdlets available in a Windows PowerShell session.

In This Section

Modules and Snap-ins Describes the differences between registering cmdlets through modules and through snap-ins.

How to Register Cmdlets using Modules Describes how to register cmdlets using modules.

How to Create a Windows PowerShell Snap-in Describes how to register cmdlets using snap-ins.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Modules and Snap-ins

Cmdlets can be added to a session using modules (introduced by Windows PowerShell 2.0) or snap-ins. Once the cmdlet is added to the session it can be run programmatically by a host application or interactively at the command line.

We recommend that you use modules as the delivery method for adding cmdlets to a session for the following reasons:

- Modules allow you to add cmdlets by loading the assembly where the cmdlet is defined. There is no need to implement a snap-in class.
- Modules allow you to add other resources, such as variables, functions, scripts, types and formatting files, and more.
- Snap-ins can be used only to add cmdlets and providers to the session.

See Also

Writing a Windows PowerShell Module

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Import Cmdlets Using Modules

This article describes how to import cmdlets to a PowerShell session by using a binary module.

> **Note**
>
> The members of modules can include cmdlets, providers, functions, variables, aliases, and much more. Snap-ins can contain only cmdlets and providers.

How to load cmdlets using a module

1. Create a module folder that has the same name as the assembly file in which the cmdlets are implemented. In this procedure, the module folder is created in the Windows `system32` folder.

   ```
   %SystemRoot%\system32\WindowsPowerShell\v1.0\Modules\mymodule
   ```

2. Make sure that the `PSModulePath` environment variable includes the path to your new module folder. By default, the system folder is already added to the `PSModulePath` environment variable. To view the `PSModulePath`, type: `$Env:PSModulePath`.

3. Copy the cmdlet assembly into the module folder.

4. Add a module manifest file (`.psd1`) in the module's root folder. PowerShell uses the module manifest to import your module. For more information, see How to Write a PowerShell Module Manifest.

5. Run the following command to add the cmdlets to the session:

   ```
   Import-Module [Module_Name]
   ```

   This procedure can be used to test your cmdlets. It adds all the cmdlets in the assembly to the session. For more information about modules, see Writing a Windows PowerShell Module.

See also

How to Write a PowerShell Module Manifest

Importing a PowerShell Module

Import-Module

Installing Modules

about_PSModulePath

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Create a Windows PowerShell Snap-in

A Windows PowerShell snap-in provides a mechanism for registering sets of cmdlets and another Windows PowerShell provider with the shell, thus extending the functionality of the shell. A Windows PowerShell snap-in can register all the cmdlets and providers in a single assembly, or it can register a specific list of cmdlets and providers.

Snap-in assemblies should be installed in a protected directory, just as they would be with other operating systems. Otherwise, malicious users can replace an assembly with unsafe code.

Windows PowerShell Snap-in Classes

All Windows PowerShell snap-in classes derive from the System.Management.Automation.PSSnapIn or System.Management.Automation.CustomPSSnapIn classes.

Examples

Writing a Windows PowerShell Snap-in: This example shows how to create a snap-in that is used to register all the cmdlets and providers in an assembly.

Writing a Custom Windows PowerShell Snap-in: This example shows how to create a custom snap-in that is used to register a specific set of cmdlets and providers that might or might not exist in a single assembly.

See Also

System.Management.Automation.PSSnapIn

System.Management.Automation.CustomPSSnapIn

Registering Cmdlets

Windows PowerShell Shell SDK

Last updated on 05/20/2025

Writing a Windows PowerShell Snap-in

This example shows how to write a Windows PowerShell snap-in that can be used to register all the cmdlets and Windows PowerShell providers in an assembly.

With this type of snap-in, you do not select which cmdlets and providers you want to register. To write a snap-in that allows you to select what is registered, see Writing a Custom Windows PowerShell Snap-in.

Writing a Windows PowerShell Snap-in

1. Add the RunInstallerAttribute attribute.
2. Create a public class that derives from the System.Management.Automation.PSSnapIn class.

   In this example, the class name is "GetProcPSSnapIn01".
3. Add a public property for the name of the snap-in (required). When naming snap-ins, do not use any of the following characters: `#`, `.`, `,`, `(`, `)`, `{`, `}`, `[`, `]`, `&`, `-`, `/`, `\`, `$`, `;`, `:`, `"`, `'`, `<`, `>`, `|`, `?`, `@`, `` ` ``, `*`

   In this example, the name of the snap-in is "GetProcPSSnapIn01".
4. Add a public property for the vendor of the snap-in (required).

   In this example, the vendor is "Microsoft".
5. Add a public property for the vendor resource of the snap-in (optional).

   In this example, the vendor resource is "GetProcPSSnapIn01,Microsoft".
6. Add a public property for the description of the snap-in (required).

   In this example, the description is "This is a Windows PowerShell snap-in that registers the Get-Proc cmdlet".
7. Add a public property for the description resource of the snap-in (optional).

   In this example, the vendor resource is "GetProcPSSnapIn01,This is a Windows PowerShell snap-in that registers the Get-Proc cmdlet".

Example

This example shows how to write a Windows PowerShell snap-in that can be used to register the Get-Proc cmdlet in the Windows PowerShell shell. Be aware that in this example, the complete assembly would contain only the GetProcPSSnapIn01 snap-in class and the `Get-Proc` cmdlet class.

C#

```
[RunInstaller(true)]
public class GetProcPSSnapIn01 : PSSnapIn
{
  /// <summary>
  /// Create an instance of the GetProcPSSnapIn01 class.
  /// </summary>
  public GetProcPSSnapIn01()
         : base()
  {
  }

  /// <summary>
  /// Specify the name of the PowerShell snap-in.
  /// </summary>
  public override string Name
  {
    get
    {
      return "GetProcPSSnapIn01";
    }
  }

  /// <summary>
  /// Specify the vendor for the PowerShell snap-in.
  /// </summary>
  public override string Vendor
  {
    get
    {
      return "Microsoft";
    }
  }

  /// <summary>
  /// Specify the localization resource information for the vendor.
  /// Use the format: resourceBaseName,VendorName.
  /// </summary>
  public override string VendorResource
  {
    get
    {
      return "GetProcPSSnapIn01,Microsoft";
```

```
    }
  }

  /// <summary>
  /// Specify a description of the PowerShell snap-in.
  /// </summary>
  public override string Description
  {
    get
    {
      return "This is a PowerShell snap-in that includes the Get-Proc cmdlet.";
    }
  }

  /// <summary>
  /// Specify the localization resource information for the description.
  /// Use the format: resourceBaseName,Description.
  /// </summary>
  public override string DescriptionResource
  {
    get
    {
      return "GetProcPSSnapIn01,This is a PowerShell snap-in that includes the Get-
Proc cmdlet.";
    }
  }
}
```

See Also

How to Register Cmdlets, Providers, and Host Applications

Windows PowerShell Shell SDK

Last updated on 05/20/2025

Writing a Custom Windows PowerShell Snap-in

This example shows how to write a Windows PowerShell snap-in that registers specific cmdlets.

With this type of snap-in, you specify which cmdlets, providers, types, or formats to register. For more information about how to write a snap-in that registers all the cmdlets and providers in an assembly, see Writing a Windows PowerShell Snap-in.

To write a Windows PowerShell Snap-in that registers specific cmdlets.

1. Add the RunInstallerAttribute attribute.
2. Create a public class that derives from the System.Management.Automation.CustomPSSnapIn class.

   In this example, the class name is "CustomPSSnapinTest".
3. Add a public property for the name of the snap-in (required). When naming snap-ins, do not use any of the following characters: `#`, `.`, `,`, `(`, `)`, `{`, `}`, `[`, `]`, `&`, `-`, `/`, `\`, `$`, `;`, `:`, `"`, `'`, `<`, `>`, `|`, `?`, `@`, `` ` ``, `*`

   In this example, the name of the snap-in is "CustomPSSnapInTest".
4. Add a public property for the vendor of the snap-in (required).

   In this example, the vendor is "Microsoft".
5. Add a public property for the vendor resource of the snap-in (optional).

   In this example, the vendor resource is "CustomPSSnapInTest,Microsoft".
6. Add a public property for the description of the snap-in (required).

   In this example, the description is: "This is a custom Windows PowerShell snap-in that includes the `Test-HelloWorld` and `Test-CustomSnapinTest` cmdlets".
7. Add a public property for the description resource of the snap-in (optional).

In this example, the vendor resource is:

> CustomPSSnapInTest, This is a custom Windows PowerShell snap-in that includes the Test-HelloWorld and Test-CustomSnapinTest cmdlets".

8. Specify the cmdlets that belong to the custom snap-in (optional) using the System.Management.Automation.Runspaces.CmdletConfigurationEntry class. The information added here includes the name of the cmdlet, its .NET type, and the cmdlet Help file name (the format of the cmdlet Help file name should be `name.dll-help.xml`).

   This example adds the Test-HelloWorld and TestCustomSnapinTest cmdlets.

9. Specify the providers that belong to the custom snap-in (optional).

   This example does not specify any providers.

10. Specify the types that belong to the custom snap-in (optional).

    This example does not specify any types.

11. Specify the formats that belong to the custom snap-in (optional).

    This example does not specify any formats.

Example

This example shows how to write a Custom Windows PowerShell snap-in that can be used to register the `Test-HelloWorld` and `Test-CustomSnapinTest` cmdlets. Be aware that in this example, the complete assembly could contain other cmdlets and providers that would not be registered by this snap-in.

C#

```
[RunInstaller(true)]
public class CustomPSSnapinTest : CustomPSSnapIn
{
  /// <summary>
  /// Creates an instance of CustomPSSnapInTest class.
  /// </summary>
  public CustomPSSnapinTest()
          : base()
  {
  }

  /// <summary>
  /// Specify the name of the custom PowerShell snap-in.
```

```
  /// </summary>
  public override string Name
  {
    get
    {
      return "CustomPSSnapInTest";
    }
  }

  /// <summary>
  /// Specify the vendor for the custom PowerShell snap-in.
  /// </summary>
  public override string Vendor
  {
    get
    {
      return "Microsoft";
    }
  }

  /// <summary>
  /// Specify the localization resource information for the vendor.
  /// Use the format: resourceBaseName,resourceName.
  /// </summary>
  public override string VendorResource
  {
    get
    {
        return "CustomPSSnapInTest,Microsoft";
    }
  }

  /// <summary>
  /// Specify a description of the custom PowerShell snap-in.
  /// </summary>
  public override string Description
  {
    get
    {
      return "This is a custom PowerShell snap-in that includes the Test-HelloWorld
and Test-CustomSnapinTest cmdlets.";
    }
  }

  /// <summary>
  /// Specify the localization resource information for the description.
  /// Use the format: resourceBaseName,Description.
  /// </summary>
  public override string DescriptionResource
  {
    get
    {
        return "CustomPSSnapInTest,This is a custom PowerShell snap-in that
includes the Test-HelloWorld and Test-CustomSnapinTest cmdlets.";
    }
```

```
  }

  /// <summary>
  /// Specify the cmdlets that belong to this custom PowerShell snap-in.
  /// </summary>
  private Collection<CmdletConfigurationEntry> _cmdlets;
  public override Collection<CmdletConfigurationEntry> Cmdlets
  {
    get
    {
      if (_cmdlets == null)
      {
        _cmdlets = new Collection<CmdletConfigurationEntry>();
        _cmdlets.Add(new CmdletConfigurationEntry("test-customsnapintest",
typeof(TestCustomSnapinTest), "TestCmdletHelp.dll-help.xml"));
        _cmdlets.Add(new CmdletConfigurationEntry("test-helloworld",
typeof(TestHelloWorld), "HelloWorldHelp.dll-help.xml"));
      }

      return _cmdlets;
    }
  }

  /// <summary>
  /// Specify the providers that belong to this custom PowerShell snap-in.
  /// </summary>
  private Collection<ProviderConfigurationEntry> _providers;
  public override Collection<ProviderConfigurationEntry> Providers
  {
    get
    {
      if (_providers == null)
      {
        _providers = new Collection<ProviderConfigurationEntry>();
      }

      return _providers;
    }
  }

  /// <summary>
  /// Specify the types that belong to this custom PowerShell snap-in.
  /// </summary>
  private Collection<TypeConfigurationEntry> _types;
  public override Collection<TypeConfigurationEntry> Types
  {
    get
    {
      if (_types == null)
      {
        _types = new Collection<TypeConfigurationEntry>();
      }

      return _types;
    }
```

```
  }

  /// <summary>
  /// Specify the formats that belong to this custom PowerShell snap-in.
  /// </summary>
  private Collection<FormatConfigurationEntry> _formats;
  public override Collection<FormatConfigurationEntry> Formats
  {
    get
    {
      if (_formats == null)
      {
        _formats = new Collection<FormatConfigurationEntry>();
      }

      return _formats;
    }
  }
}
```

For more information about registering snap-ins, see How to Register Cmdlets, Providers, and Host Applications in the Windows PowerShell Programmer's Guide.

See Also

How to Register Cmdlets, Providers, and Host Applications

Windows PowerShell Shell SDK

Last updated on 11/20/2025

Importing a PowerShell Module

Once you have installed a module on a system, you will likely want to import the module. Importing is the process that loads the module into active memory, so that a user can access that module in their PowerShell session. In PowerShell 2.0, you can import a newly-installed PowerShell module with a call to Import-Module cmdlet. In PowerShell 3.0, PowerShell is able to implicitly import a module when one of the functions or cmdlets in the module is called by a user. Note that both versions assume that you install your module in a location where PowerShell is able to find it; for more information, see Installing a PowerShell Module. You can use a module manifest to restrict what parts of your module are exported, and you can use parameters of the `Import-Module` call to restrict what parts are imported.

Importing a Snap-In (PowerShell 1.0)

Modules did not exist in PowerShell 1.0: instead, you had to register and use snap-ins. However, it is not recommended that you use this technology at this point, as modules are generally easier to install and import. For more information, see How to Create a Windows PowerShell Snap-in.

Importing a Module with Import-Module (PowerShell 2.0)

PowerShell 2.0 uses the appropriately-named Import-Module cmdlet to import modules. When this cmdlet is run, Windows PowerShell searches for the specified module within the directories specified in the `PSModulePath` variable. When the specified directory is found, Windows PowerShell searches for files in the following order: module manifest files (`.psd1`), script module files (`.psm1`), binary module files (.dll). For more information about adding directories to the search, see about_PSModulePath. The following code describes how to import a module:

PowerShell

```
Import-Module myModule
```

Assuming that myModule was located in the `PSModulePath`, PowerShell would load myModule into active memory. If myModule was not located on a `PSModulePath` path, you could still explicitly tell PowerShell where to find it:

PowerShell

```
Import-Module -Name C:\myRandomDirectory\myModule -Verbose
```

You can also use the `-Verbose` parameter to identify what is being exported out of the module, and what is being imported into active memory. Both exports and imports restrict what is exposed to the user: the difference is who is controlling the visibility. Essentially, exports are controlled by code within the module. In contrast, imports are controlled by the `Import-Module` call. For more information, see **Restricting Members That Are Imported**, below.

Implicitly Importing a Module (PowerShell 3.0)

Beginning in Windows PowerShell 3.0, modules are imported automatically when any cmdlet or function in the module is used in a command. This feature works on any module in a directory that is included in the value of the **PSModulePath** environment variable. If you do not save your module on a valid path however, you can still load them using the explicit Import-Module option, described above.

The following actions trigger automatic importing of a module, also known as "module auto-loading."

- Using a cmdlet in a command. For example, typing `Get-ExecutionPolicy` imports the Microsoft.PowerShell.Security module that contains the `Get-ExecutionPolicy` cmdlet.
- Using the Get-Command cmdlet to get the command. For example, typing `Get-Command Get-JobTrigger` imports the **PSScheduledJob** module that contains the `Get-JobTrigger` cmdlet. A `Get-Command` command that includes wildcard characters is considered to be discovery and does not trigger importing of a module.
- Using the Get-Help cmdlet to get help for a cmdlet. For example, typing `Get-Help Get-WinEvent` imports the Microsoft.PowerShell.Diagnostics module that contains the `Get-WinEvent` cmdlet.

To support automatic importing of modules, the `Get-Command` cmdlet gets all cmdlets and functions in all installed modules, even if the module is not imported into the session. For more information, see the help topic for the Get-Command cmdlet.

The Importing Process

When a module is imported, a new session state is created for the module, and a System.Management.Automation.PSModuleInfo object is created in memory. A session-state is created for each module that is imported (this includes the root module and any nested modules). The members that are exported from the root module, including any members that were exported to the root module by any nested modules, are then imported into the caller's session state.

The metadata of members that are exported from a module have a ModuleName property. This property is populated with the name of the module that exported them.

> ⚠ **Warning**
>
> If the name of an exported member uses an unapproved verb or if the name of the member uses restricted characters, a warning is displayed when the **Import-Module** cmdlet is run.

By default, the Import-Module cmdlet does not return any objects to the pipeline. However, the cmdlet supports a **PassThru** parameter that can be used to return a System.Management.Automation.PSModuleInfo object for each module that is imported. To send output to the host, users should run the Write-Host cmdlet.

Restricting the Members That Are Imported

When a module is imported by using the Import-Module cmdlet, by default, all exported module members are imported into the session, including any commands exported to the module by a nested module. By default, variables and aliases are not exported. To restrict the members that are exported, use a module manifest. To restrict the members that are imported, use the following parameters of the `Import-Module` cmdlet.

- **Function**: This parameter restricts the functions that are exported. (If you are using a module manifest, see the FunctionsToExport key.)
- **`Cmdlet**: This parameter restricts the cmdlets that are exported (If you are using a module manifest, see the CmdletsToExport key.)
- **Variable**: This parameter restricts the variables that are exported (If you are using a module manifest, see the VariablesToExport key.)

- **Alias**: This parameter restricts the aliases that are exported (If you are using a module manifest, see the AliasesToExport key.)

See Also

Writing a Windows PowerShell Module

Last updated on 05/20/2025

Windows PowerShell Provider Quickstart

This topic explains how to create a Windows PowerShell provider that has basic functionality of creating a new drive. For general information about providers, see Windows PowerShell Provider Overview. For examples of providers with more complete functionality, see Provider Samples.

Writing a basic provider

The most basic functionality of a Windows PowerShell provider is to create and remove drives. In this example, we implement the System.Management.Automation.Provider.DriveCmdletProvider.NewDrive* and System.Management.Automation.Provider.DriveCmdletProvider.RemoveDrive* methods of the System.Management.Automation.Provider.DriveCmdletProvider class. You will also see how to declare a provider class.

When you write a provider, you can specify default drives-drives that are created automatically when the provider is available. You also define a method to create new drives that use that provider.

The examples provided in this topic are based on the AccessDBProviderSample02 sample, which is part of a larger sample that represents an Access database as a Windows PowerShell drive.

Setting up the project

In Visual Studio, create a Class Library project named AccessDBProviderSample. Complete the following steps to configure your project so that Windows PowerShell will start, and the provider will be loaded into the session, when you build and start your project.

Configure the provider project

1. Add the System.Management.Automation assembly as a reference to your project.
2. Click **Project** > **AccessDBProviderSample Properties** > **Debug**. In **Start project**, click **Start external program**, and navigate to the Windows PowerShell executable (typically C:\Windows\System32\WindowsPowerShell\v1.0\.powershell.exe).

3. Under **Start Options**, enter the following into the **Command line arguments** box: `-NoExit -Command "[Reflection.Assembly]::LoadFrom(AccessDBProviderSample.dll' ) | Import-Module"`

Declaring the provider class

Our provider derives from the System.Management.Automation.Provider.DriveCmdletProvider class. Most providers that provide real functionality (accessing and manipulating items, navigating the data store, and getting and setting content of items) derive from the System.Management.Automation.Provider.NavigationCmdletProvider class.

In addition to specifying that the class derives from System.Management.Automation.Provider.DriveCmdletProvider, you must decorate it with the System.Management.Automation.Provider.CmdletProviderAttribute as shown in the example.

C#

```
namespace Microsoft.Samples.PowerShell.Providers
{
  using System;
  using System.Data;
  using System.Data.Odbc;
  using System.IO;
  using System.Management.Automation;
  using System.Management.Automation.Provider;

  #region AccessDBProvider

  [CmdletProvider("AccessDB", ProviderCapabilities.None)]
  public class AccessDBProvider : DriveCmdletProvider
  {

}
}
```

Implementing NewDrive

The System.Management.Automation.Provider.DriveCmdletProvider.NewDrive* method is called by the Windows PowerShell engine when a user calls the Microsoft.PowerShell.Commands.NewPSDriveCommand cmdlet specifying the name of your provider. The PSDriveInfo parameter is passed by the Windows PowerShell engine, and the method returns the new drive to the Windows PowerShell engine. This method must be declared within the class created above.

The method first checks to make sure both the drive object and the drive root that were passed in exist, returning `null` if either of them do not. It then uses a constructor of the internal class AccessDBPSDriveInfo to create a new drive and a connection to the Access database the drive represents.

C#

```
protected override PSDriveInfo NewDrive(PSDriveInfo drive)
    {
      // Check if the drive object is null.
      if (drive == null)
      {
        WriteError(new ErrorRecord(
                   new ArgumentNullException("drive"),
                   "NullDrive",
                   ErrorCategory.InvalidArgument,
                   null));

        return null;
      }

      // Check if the drive root is not null or empty
      // and if it is an existing file.
      if (String.IsNullOrEmpty(drive.Root) || (File.Exists(drive.Root) == false))
      {
        WriteError(new ErrorRecord(
                   new ArgumentException("drive.Root"),
                   "NoRoot",
                   ErrorCategory.InvalidArgument,
                   drive));

        return null;
      }

      // Create a new drive and create an ODBC connection to the new drive.
      AccessDBPSDriveInfo accessDBPSDriveInfo = new AccessDBPSDriveInfo(drive);
      OdbcConnectionStringBuilder builder = new OdbcConnectionStringBuilder();

      builder.Driver = "Microsoft Access Driver (*.mdb)";
      builder.Add("DBQ", drive.Root);

      OdbcConnection conn = new OdbcConnection(builder.ConnectionString);
      conn.Open();
      accessDBPSDriveInfo.Connection = conn;

      return accessDBPSDriveInfo;
    }
```

The following is the AccessDBPSDriveInfo internal class that includes the constructor used to create a new drive, and contains the state information for the drive.

C#

```
internal class AccessDBPSDriveInfo : PSDriveInfo
  {
    /// <summary>
    /// A reference to the connection to the database.
    /// </summary>
    private OdbcConnection connection;

    /// <summary>
    /// Initializes a new instance of the AccessDBPSDriveInfo class.
    /// The constructor takes a single argument.
    /// </summary>
    /// <param name="driveInfo">Drive defined by this provider</param>
    public AccessDBPSDriveInfo(PSDriveInfo driveInfo)
           : base(driveInfo)
    {
    }

    /// <summary>
    /// Gets or sets the ODBC connection information.
    /// </summary>
    public OdbcConnection Connection
    {
        get { return this.connection; }
        set { this.connection = value; }
    }
  }
```

Implementing RemoveDrive

The System.Management.Automation.Provider.DriveCmdletProvider.RemoveDrive* method is called by the Windows PowerShell engine when a user calls the Microsoft.PowerShell.Commands.RemovePSDriveCommand cmdlet. The method in this provider closes the connection to the Access database.

C#

```
protected override PSDriveInfo RemoveDrive(PSDriveInfo drive)
    {
      // Check if drive object is null.
      if (drive == null)
      {
        WriteError(new ErrorRecord(
                   new ArgumentNullException("drive"),
                   "NullDrive",
                   ErrorCategory.InvalidArgument,
                   drive));

        return null;
      }
```

```
    // Close the ODBC connection to the drive.
    AccessDBPSDriveInfo accessDBPSDriveInfo = drive as AccessDBPSDriveInfo;

    if (accessDBPSDriveInfo == null)
    {
       return null;
    }

    accessDBPSDriveInfo.Connection.Close();

    return accessDBPSDriveInfo;
  }
```

Last updated on 08/15/2025

Windows PowerShell Provider Overview

A Windows PowerShell provider allows any data store to be exposed like a file system as if it were a mounted drive. For example, the built-in Registry provider allows you to navigate the registry like you would navigate the `c` drive of your computer. A provider can also override the `Item` cmdlets (for example, `Get-Item`, `Set-Item`, etc.) such that the data in your data store can be treated like files and directories are treated when navigating a file system. For more information about providers and drives, and the built-in providers in Windows PowerShell, see about_Providers.

Providers and Drives

A Provider defines the logic that is used to access, navigate, and edit a data store, while a drive specifies a specific entry point to a data store (or a portion of a data store) that is of the type defined by the provider. For example, the Registry provider allows you to access hives and keys in a registry, and the HKLM and HKCU drives specify the corresponding hives within the registry. The HKLM and HKCU drives both use the Registry provider.

When you write a provider, you can specify default drives-drives that are created automatically when the provider is available. You also define a method to create new drives that use that provider.

Type of Providers

There are several types of providers, each of which provides a different level of functionality. A provider is implemented as a class that derives from one of the descendants of the System.Management.Automation.SessionStateCategory **CmdletProvider** class. For information about the different types of providers, see Provider types.

Provider cmdlets

Providers can implement methods that correspond to cmdlets, creating custom behaviors for those cmdlets when used in a drive for that provider. Depending on the type of provider, different sets of cmdlets are available. For a complete list of the cmdlets available for customization in providers, see Provider cmdlets.

Provider paths

Users navigate provider drives like file systems. Because of this, they expect the syntax of paths to correspond to the paths used in file system navigation. When a user runs a provider cmdlet, they specify a path to the item to be accessed. The path that is specified can be interpreted in several ways. A provider should support one or more of the following path types.

Drive-qualified paths

A drive-qualified path is a combination of the item name, the container and subcontainers in which the item is located, and the Windows PowerShell drive through which the item is accessed. (Drives are defined by the provider that is used to access the data store. This path starts with the drive name followed by a colon (:). For example: `Get-ChildItem C:`

Provider-qualified paths

To allow the Windows PowerShell engine to initialize and uninitialize your provider, the provider must support a provider-qualified path. For example, the user can initialize and uninitialize the FileSystem provider because it defines the following provider-qualified path: `FileSystem::\\uncshare\abc\bar`.

Provider-direct paths

To allow remote access to your Windows PowerShell provider, it should support a provider-direct path to pass directly to the Windows PowerShell provider for the current location. For example, the registry Windows PowerShell provider can use `\\server\regkeypath` as a provider-direct path.

Provider-internal paths

To allow the provider cmdlet to access data using non-Windows PowerShell application programming interfaces (APIs), your Windows PowerShell provider should support a provider-internal path. This path is indicated after the "::" in the provider-qualified path. For example, the provider-internal path for the FileSystem Windows PowerShell provider is `\\uncshare\abc\bar`.

Overriding cmdlet parameters

The behavior of some provider-specific cmdlets can be overridden by a provider. For a list of parameters that can be overridden, and how to override them in your provider class, see

Provider cmdlet parameters

Dynamic parameters

Providers can define dynamic parameters that are added to a provider cmdlet when the user specifies a certain value for one of the static parameters of the cmdlet. A provider does this by implementing one or more dynamic parameter methods. For a list of cmdlet parameters that can be used to add dynamic parameter, and the methods used to implement them, see Provider cmdlet dynamic parameters.

Provider capabilities

The System.Management.Automation.Provider.ProviderCapabilities enumeration defines a number of capabilities that providers can support. These include the ability to use wildcards, filter items, and support transactions. To specify capabilities for a provider, add a list of values of the System.Management.Automation.Provider.ProviderCapabilities enumeration, combined with a logical `OR` operation, as the System.Management.Automation.Provider.CmdletProviderAttribute.ProviderCapabilities* property (the second parameter of the attribute) of the System.Management.Automation.Provider.CmdletProviderAttribute attribute for your provider class. For example, the following attribute specifies that the provider supports the System.Management.Automation.Provider.ProviderCapabilities **ShouldProcess** and System.Management.Automation.Provider.ProviderCapabilities **Transactions** capabilities.

C#

```
[CmdletProvider(RegistryProvider.ProviderName, ProviderCapabilities.ShouldProcess |
ProviderCapabilities.Transactions)]
```

Provider cmdlet help

When writing a provider, you can implement your own Help for the provider cmdlets that you support. This includes a single help topic for each provider cmdlet or multiple versions of a help topic for cases where the provider cmdlet acts differently based on the use of dynamic parameters. To support provider cmdlet-specific help, your provider must implement the System.Management.Automation.Provider.ICmdletProviderSupportsHelp interface.

The Windows PowerShell engine calls the System.Management.Automation.Provider.ICmdletProviderSupportsHelp.GetHelpMaml* method to display the Help topic for your provider cmdlets. The engine provides the name of the cmdlet that the user specified when running the `Get-Help` cmdlet and the current path of the user. The current path is required if your provider implements different versions of the same provider cmdlet for different drives. The method must return a string that contains the XML for the cmdlet Help.

The content for the Help file is written using PSMAML XML. This is the same XML schema that is used for writing Help content for stand-alone cmdlets. Add the content for your custom cmdlet Help to the Help file for your provider under the `CmdletHelpPaths` element. The following example shows the `command` element for a single provider cmdlet, and it shows how you specify the name of the provider cmdlet that your provider. supports

XML

```
<CmdletHelpPaths>
  <command:command>
    <command:details>
      <command:name>ProviderCmdletName</command:name>
      <command:verb>Verb</command:verb>
      <command:noun>Noun</command:noun>
    <command:details>
  </command:command>
<CmdletHelpPath>
```

See Also

Windows PowerShell Provider Functionality

Provider Cmdlets

Writing a Windows PowerShell Provider

Last updated on 05/20/2025

Provider types

Providers define their basic functionality by changing how the provider cmdlets, provided by PowerShell, perform their actions. For example, providers can use the default functionality of the `Get-Item` cmdlet, or they can change how that cmdlet operates when retrieving items from the data store. The provider functionality described in this document includes functionality defined by overwriting methods from specific provider base classes and interfaces.

> **Note**
>
> For provider features that are pre-defined by PowerShell, see Provider capabilities.

Drive-enabled providers

Drive-enabled providers specify the default drives available to the user and allow the user to add or remove drives. In most cases, providers are drive-enabled providers because they require some default drive to access the data store. However, when writing your own provider you might or might not want to allow the user to create and remove drives.

To create a drive-enabled provider, your provider class must derive from the System.Management.Automation.Provider.DriveCmdletProvider class or another class that derives from that class. The **DriveCmdletProvider** class defines the following methods for implementing the default drives of the provider and supporting the `New-PSDrive` and `Remove-PSDrive` cmdlets. In most cases, to support a provider cmdlet you must overwrite the method that the PowerShell engine calls to invoke the cmdlet, such as the `NewDrive` method for the `New-PSDrive` cmdlet, and optionally you can overwrite a second method, such as `NewDriveDynamicParameters`, for adding dynamic parameters to the cmdlet.

- The InitializeDefaultDrives method defines the default drives that are available to the user whenever the provider is used.
- The NewDrive and NewDriveDynamicParameters methods defines how your provider supports the `New-PSDrive` provider cmdlet. This cmdlet allows the user to create drives to access the data store.
- The RemoveDrive method defines how your provider supports the `Remove-PSDrive` provider cmdlet. This cmdlet allows the user to remove drives from the data store.

Item-enabled providers

Item-enabled providers allow the user to get, set, or clear the items in the data store. An "item" is an element of the data store that the user can access or manage independently. To create an item-enabled provider, your provider class must derive from the System.Management.Automation.Provider.ItemCmdletProvider class or another class that derives from that class.

The **ItemCmdletProvider** class defines the following methods for implementing specific provider cmdlets. In most cases, to support a provider cmdlet you must overwrite the method that the PowerShell engine calls to invoke the cmdlet, such as the `ClearItem` method for the `Clear-Item` cmdlet, and optionally you can overwrite a second method, such as `ClearItemDynamicParameters`, for adding dynamic parameters to the cmdlet.

- The ClearItem and ClearItemDynamicParameters methods define how your provider supports the `Clear-Item` provider cmdlet. This cmdlet allows the user to remove of the value of an item in the data store.
- The GetItem and GetItemDynamicParameters methods define how your provider supports the `Get-Item` provider cmdlet. This cmdlet allows the user to retrieve data from the data store.
- The SetItem and SetItemDynamicParameters methods define how your provider supports the `Set-Item` provider cmdlet. This cmdlet allows the user to update the values of items in the data store.
- The InvokeDefaultAction and InvokeDefaultActionDynamicParameters methods define how your provider supports the `Invoke-Item` provider cmdlet. This cmdlet allows the user to perform the default action specified by the item.
- The ItemExists and ItemExistsDynamicParameters methods define how your provider supports the `Test-Path` provider cmdlet. This cmdlet allows the user to determine if all the elements of a path exist.

In addition to the methods used to implement provider cmdlets, the **ItemCmdletProvider** class also defines the following methods:

- The ExpandPath method allows the user to use wildcards when specifying the provider path.

- The IsValidPath is used to determine if a path is syntactically and semantically valid for the provider.

Container-enabled providers

Container-enabled providers allow the user to manage items that are containers. A container is a group of child items under a common parent item. To create a container-enabled provider, your provider class must derive from the System.Management.Automation.Provider.ContainerCmdletProvider class or another class that derives from that class.

> **Important**
>
> Container-enabled providers can't access data stores that contain nested containers. If a child item of a container is another container, you must implement a navigation-enabled provider.

The **ContainerCmdletProvider** class defines the following methods for implementing specific provider cmdlets. In most cases, to support a provider cmdlet you must overwrite the method that the PowerShell engine calls to invoke the cmdlet, such as the `CopyItem` method for the `Copy-Item` cmdlet, and optionally you can overwrite a second method, such as `CopyItemDynamicParameters`, for adding dynamic parameters to the cmdlet.

- The CopyItem and CopyItemDynamicParameters methods define how your provider supports the `Copy-Item` provider cmdlet. This cmdlet allows the user to copy an item from one location to another.
- The GetChildItems and GetChildItemsDynamicParameters methods define how your provider supports the `Get-ChildItem` provider cmdlet. This cmdlet allows the user to retrieve the child items of the parent item.
- The GetChildNames and GetChildNamesDynamicParameters methods define how your provider supports the `Get-ChildItem` provider cmdlet if its `Name` parameter is specified.
- The NewItem and NewItemDynamicParameters methods define how your provider supports the `New-Item` provider cmdlet. This cmdlet allows the user to create new items in the data store.

- The RemoveItem and RemoveItemDynamicParameters methods define how your provider supports the `Remove-Item` provider cmdlet. This cmdlet allows the user to remove items from the data store.

- The RenameItem and RenameItemDynamicParameters methods define how your provider supports the `Rename-Item` provider cmdlet. This cmdlet allows the user to rename items in the data store.

In addition to the methods used to implement provider cmdlets, the **ContainerCmdletProvider** class also defines the following methods:

- The HasChildItems method can be used by the provider class to determine whether an item has child items.

- The ConvertPath method can be used by the provider class to create a new provider-specific path from a specified path.

Navigation-enabled providers

Navigation-enabled providers allow the user to move items in the data store. To create a navigation-enabled provider, your provider class must derive from the System.Management.Automation.Provider.NavigationCmdletProvider class.

The **NavigationCmdletProvider** class defines the following methods for implementing specific provider cmdlets. In most cases, to support a provider cmdlet you must overwrite the method that the PowerShell engine calls to invoke the cmdlet, such as the `MoveItem` method for the `Move-Item` cmdlet, and optionally you can overwrite a second method, such as `MoveItemDynamicParameters`, for adding dynamic parameters to the cmdlet.

- The MoveItem and MoveItemDynamicParameters methods define how your provider supports the `Move-Item` provider cmdlet. This cmdlet allows the user to move an item from one location in the store to another location.

- The MakePath method defines how your provider supports the `Join-Path` provider cmdlet. This cmdlet allows the user to combine a parent and child path segment to create a provider-internal path.

In addition to the methods used to implement provider cmdlets, the **NavigationCmdletProvider** class also defines the following methods:

- The GetChildName method extracts the name of the child node of a path.

- The GetParentPath method extracts the parent part of a path.
- The IsItemContainer method determines whether the item is a container item. In this context, a container is a group of child items under a common parent item.
- The NormalizeRelativePath method returns a path to an item that's relative to a specified base path.

Content-enabled providers

Content-enabled providers allow the user to clear, get, or set the content of items in a data store. For example, the FileSystem provider allows you to clear, get, and set the content of files in the file system. To create a content enabled provider, your provider class must implement the methods of the System.Management.Automation.Provider.IContentCmdletProvider interface.

The **IContentCmdletProvider** interface defines the following methods for implementing specific provider cmdlets. In most cases, to support a provider cmdlet you must overwrite the method that the PowerShell engine calls to invoke the cmdlet, such as the `ClearContent` method for the `Clear-Content` cmdlet, and optionally you can overwrite a second method, such as `ClearContentDynamicParameters`, for adding dynamic parameters to the cmdlet.

- The ClearContent and ClearContentDynamicParameters methods define how your provider supports the `Clear-Content` provider cmdlet. This cmdlet allows the user to delete the content of an item without deleting the item.
- The GetContentReader and GetContentReaderDynamicParameters methods define how your provider supports the `Get-Content` provider cmdlet. This cmdlet allows the user to retrieve the content of an item. The `GetContentReader` method returns an System.Management.Automation.Provider.IContentReader interface that defines the methods used to read the content.
- The GetContentWriter and GetContentWriterDynamicParameters methods define how your provider supports the `Set-Content` provider cmdlet. This cmdlet allows the user to update the content of an item. The `GetContentWriter` method returns an System.Management.Automation.Provider.IContentWriter interface that defines the methods used to write the content.

Property-enabled providers

Property-enabled providers allow the user to manage the properties of the items in the data store. To create a property-enabled provider, your provider class must implement the methods of the System.Management.Automation.Provider.IPropertyCmdletProvider and System.Management.Automation.Provider.IDynamicPropertyCmdletProvider interfaces. In most cases, to support a provider cmdlet you must overwrite the method that the PowerShell engine calls to invoke the cmdlet, such as the `ClearProperty` method for the Clear-Property cmdlet, and optionally you can overwrite a second method, such as `ClearPropertyDynamicParameters`, for adding dynamic parameters to the cmdlet.

The **IPropertyCmdletProvider** interface defines the following methods for implementing specific provider cmdlets:

- The ClearProperty and ClearPropertyDynamicParameters methods define how your provider supports the `Clear-ItemProperty` provider cmdlet. This cmdlet allows the user to delete the value of a property.
- The GetProperty and GetPropertyDynamicParameters methods define how your provider supports the `Get-ItemProperty` provider cmdlet. This cmdlet allows the user to retrieve the property of an item.
- The SetProperty and SetPropertyDynamicParameters methods define how your provider supports the `Set-ItemProperty` provider cmdlet. This cmdlet allows the user to update the properties of an item.

The **IDynamicPropertyCmdletProvider** interface defines the following methods for implementing specific provider cmdlets:

- The CopyProperty and CopyPropertyDynamicParameters methods define how your provider supports the `Copy-ItemProperty` provider cmdlet. This cmdlet allows the user to copy a property and its value from one location to another.
- The MoveProperty and MovePropertyDynamicParameters methods define how your provider supports the `Move-ItemProperty` provider cmdlet. This cmdlet allows the user to move a property and its value from one location to another.
- The NewProperty and NewPropertyDynamicParameters methods define how your provider supports the `New-ItemProperty` provider cmdlet. This cmdlet allows the user to create a new property and set its value.
- The RemoveProperty and RemovePropertyDynamicParameters methods define how your provider supports the `Remove-ItemProperty` cmdlet. This cmdlet allows the user to delete

a property and its value.

- The RenameProperty and RenamePropertyDynamicParameters methods define how your provider supports the `Rename-ItemProperty` cmdlet. This cmdlet allows the user to change the name of a property.

See also

about_Providers

Writing a Windows PowerShell Provider

Last updated on 08/25/2025

Provider cmdlets

The cmdlets that the user can run to manage a data store are referred to as provider cmdlets. To support these cmdlets, you need to overwrite some of the methods defined by the base provider classes and interfaces.

Here are the provider cmdlets that can be run by the user:

PSDrive cmdlets

`Get-PSDrive`

This cmdlet returns the PowerShell drives in the current session. You do not need to overwrite any methods to support this cmdlet.

`New-PSDrive`

This cmdlet allows the user to create PowerShell drives to access the data store. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.DriveCmdletProvider class:

- NewDrive
- NewDriveDynamicParameters

`Remove-PSDrive`

This cmdlet allows the user to remove PowerShell drives that access the data store. To support this cmdlet, overwrite the System.Management.Automation.Provider.DriveCmdletProvider.RemoveDrive method.

Item cmdlets

`Clear-Item`

This cmdlet allows the user to remove the value of an item in the data store. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.ItemCmdletProvider class:

- ClearItem
- ClearItemDynamicParameters

`Copy-Item`

This cmdlet allows the user to copy an item from one location to another. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.ContainerCmdletProvider class:

- CopyItem
- CopyItemDynamicParameters

`Get-Item`

This cmdlet allows the user to retrieve data from the data store. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.ItemCmdletProvider class:

- GetItem
- GetItemDynamicParameters

`Get-ChildItem`

This cmdlet allows the user to retrieve the child items of the parent item. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.ContainerCmdletProvider class:

- GetChildItems
- GetChildItemsDynamicParameters
- GetChildNames
- GetChildNamesDynamicParameters

`Invoke-Item`

This cmdlet allows the user to perform the default action specified by the item. To support this cmdlet, overwrite the System.Management.Automation.Provider.ItemCmdletProvider.InvokeDefaultAction method.

`Move-Item`

This cmdlet allows the user to move an item from one location to another location. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.NavigationCmdletProvider class:

- MoveItem
- MoveItemDynamicParameters

`New-Item`

This cmdlet allows the user to create a new item in the data store. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.ContainerCmdletProvider class:

- NewItem
- NewItemDynamicParameters

`Remove-Item`

This cmdlet allows the user to remove items from the data store. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.ContainerCmdletProvider class:

- RemoveItem
- RemoveItemDynamicParameters

`Rename-Item`

This cmdlet allows the user to rename items in the data store. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.ContainerCmdletProvider class:

- RenameItem
- RenameItemDynamicParameters

`Set-Item`

This cmdlet allows the user to update the values of items in the data store. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.ItemCmdletProvider class:

- SetItem

- SetItemDynamicParameters

Item content cmdlets

`Add-Content`

This cmdlet allows the user to add content to an item.

`Clear-Content`

This cmdlet allows the user to delete content from an item without deleting the item. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.IContentCmdletProvider interface:

- ClearContent
- ClearContentDynamicParameters

`Get-Content`

This cmdlet allows the user to retrieve the content of an item. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.IContentCmdletProvider interface:

- GetContentReader
- GetContentReaderDynamicParameters

The GetContentReader method returns an System.Management.Automation.Provider.IContentReader interface that defines the methods used to read the content.

`Set-Content`

This cmdlet allows the user to update the content of an item. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.IContentCmdletProvider interface:

- GetContentWriter
- GetContentWriterDynamicParameters

The GetContentWriter method returns an System.Management.Automation.Provider.IContentWriter interface that defines the methods

used to write the content.

Item property cmdlets

`Clear-ItemProperty`

This cmdlet allows the user to delete the value of a property. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.IPropertyCmdletProvider interface:

- ClearProperty
- ClearPropertyDynamicParameters

`Copy-ItemProperty`

This cmdlet allows the user to copy a property and its value from one location to another. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.IDynamicPropertyCmdletProvider interface:

- CopyProperty
- CopyPropertyDynamicParameters

`Get-ItemProperty`

This cmdlet retrieves the properties of an item. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.IPropertyCmdletProvider interface:

- GetProperty
- GetPropertyDynamicParameters

`Move-ItemProperty`

This cmdlet allows the user to move a property and its value from one location to another. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.IDynamicPropertyCmdletProvider interface:

- MoveProperty
- MovePropertyDynamicParameters

`New-ItemProperty`

This cmdlet allows the user to create a new property and set its value. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.IDynamicPropertyCmdletProvider interface:

- NewProperty
- NewPropertyDynamicParameters

`Remove-ItemProperty`

This cmdlet allows the user to delete a property and its value. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.IDynamicPropertyCmdletProvider interface:

- RemoveProperty
- RemovePropertyDynamicParameters

`Rename-ItemProperty`

This cmdlet allows the user to change the name of a property. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.IDynamicPropertyCmdletProvider interface:

- RenameProperty
- RenamePropertyDynamicParameters

`Set-ItemProperty`

This cmdlet allows the user to update the properties of an item. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.IPropertyCmdletProvider interface:

- SetProperty
- SetPropertyDynamicParameters

Location cmdlets

`Get-Location`

Retrieves information about the current working location. You do not need to overwrite any methods to support this cmdlet.

`Pop-Location`

This cmdlet changes the current location to the location most recently pushed onto the stack. You do not need to overwrite any methods to support this cmdlet.

`Push-Location`

This cmdlet adds the current location to the top of a list of locations (a "stack"). You do not need to overwrite any methods to support this cmdlet.

`Set-Location`

This cmdlet sets the current working location to a specified location. You do not need to overwrite any methods to support this cmdlet.

Path cmdlets

`Join-Path`

This cmdlet allows the user to combine a parent and child path segment to create a provider-internal path. To support this cmdlet, overwrite the System.Management.Automation.Provider.NavigationCmdletProvider.MakePath method.

`Convert-Path`

This cmdlet converts a path from a PowerShell path to a PowerShell provider path.

`Split-Path`

Returns the specified part of a path.

`Resolve-Path`

Resolves the wildcard characters in a path, and displays the path contents.

`Test-Path`

This cmdlet determines whether all elements of a path exist. To support this cmdlet, overwrite the following methods of System.Management.Automation.Provider.ItemCmdletProvider class:

- ItemExists

- ItemExistsDynamicParameters

PSProvider cmdlets

`Get-PSProvider`

This cmdlet returns information about the providers available in the session. You do not need to overwrite any methods to support this cmdlet.

Last updated on 12/19/2025

Provider cmdlet parameters

Provider cmdlets come with a set of static parameters that are available to all providers that support the cmdlet, as well as dynamic parameters that are added when the user specifies a certain value for certain static parameters of the provider cmdlet.

Provider Cmdlet Static Parameters

Static parameters are defined by Windows PowerShell. A large set of these parameters is implemented by Windows PowerShell to provide consistency across all the providers and to provide a simpler development experience. Examples of these parameters include the `LiteralPath`, `Exclude`, and `Include` parameters of the `Get-Item` cmdlet. A smaller set of these parameters can be overwritten to provide actions that are specific to your provider. Examples of these parameters include the `Path` and `Value` parameter of the `Set-Item` cmdlet. Here is a list of the parameters that can be overwritten for the provider cmdlets.

`Clear-Content` cmdlet You can define how your provider will use the values passed to the `Path` parameter of the `Clear-Content` cmdlet by implementing the System.Management.Automation.Provider.IContentCmdletProvider.ClearContent* method.

`Clear-Item` cmdlet You can define how your provider will use the values passed to the `Path` parameter of the `Clear-Item` cmdlet by implementing the System.Management.Automation.Provider.ItemCmdletProvider.ClearItem* method.

`Clear-ItemProperty` cmdlet You can define how your provider will use the values passed to the `Path` and `Name` parameters of the `Clear-ItemProperty` cmdlet by implementing the System.Management.Automation.Provider.IPropertyCmdletProvider.ClearProperty* method.

`Copy-Item` cmdlet You can define how your provider will use the values passed to the `Path`, `Destination`, and `Recurse` parameters of the `Copy-Item` cmdlet by implementing the System.Management.Automation.Provider.ContainerCmdletProvider.CopyItem method.

Get-ChildItems cmdlet You can define how your provider will use the values passed to the `Path` and `Recurse` parameters of the `Get-ChildItem` cmdlet by implementing the System.Management.Automation.Provider.ContainerCmdletProvider.GetChildItems* and System.Management.Automation.Provider.ContainerCmdletProvider.GetChildNames* methods.

`Get-Content` cmdlet You can define how your provider will use the values passed to the `Path` parameter of the `Get-Content` cmdlet by implementing the System.Management.Automation.Provider.IContentCmdletProvider.GetContentReader* method.

`Get-Item` cmdlet You can define how your provider will use the values passed to the `Path` parameter of the `Get-Item` cmdlet by implementing the System.Management.Automation.Provider.ItemCmdletProvider.GetItem* method.

`Get-ItemProperty` cmdlet You can define how your provider will use the values passed to the `Path` and `Name` parameters of the `Get-ItemProperty` cmdlet by implementing the System.Management.Automation.Provider.IPropertyCmdletProvider.GetProperty* method.

`Invoke-Item` cmdlet You can define how your provider will use the values passed to the `Path` parameter of the `Invoke-Item` cmdlet by implementing the System.Management.Automation.Provider.ItemCmdletProvider.InvokeDefaultAction* method.

`Move-Item` cmdlet You can define how your provider will use the values passed to the `Path` and `Destination` parameters of the `Move-Item` cmdlet by implementing the System.Management.Automation.Provider.NavigationCmdletProvider.MoveItem* method.

`New-Item` cmdlet You can define how your provider will use the values passed to the `Path`, `ItemType`, and `Value` parameters of the `New-Item` cmdlet by implementing the System.Management.Automation.Provider.ContainerCmdletProvider.NewItem* method.

`New-ItemProperty` cmdlet You can define how your provider will use the values passed to the `Path`, `Name`, `PropertyType`, and `Value` parameters of the `New-ItemProperty` cmdlet by implementing the Microsoft.PowerShell.Commands.RegistryProvider.NewProperty* method.

`Remove-Item` You can define how your provider will use the values passed to the `Path` and `Recurse` parameters of the `Remove-Item` cmdlet by implementing the System.Management.Automation.Provider.ContainerCmdletProvider.RemoveItem* method.

`Remove-ItemProperty` You can define how your provider will use the values passed to the `Path` and `Name` parameters of the `Remove-ItemProperty` cmdlet by implementing the System.Management.Automation.Provider.IDynamicPropertyCmdletProvider.RemoveProperty* method.

`Rename-Item` cmdlet You can define how your provider will use the values passed to the `Path` and `NewName` parameters of the `Rename-Item` cmdlet by implementing the System.Management.Automation.Provider.ContainerCmdletProvider.RenameItem* method.

`Rename-ItemProperty` You can define how your provider will use the values passed to the `Path`, `NewName`, and `Name` parameters of the `Rename-ItemProperty` cmdlet by implementing the System.Management.Automation.Provider.IDynamicPropertyCmdletProvider.RenameProperty* method.

`Set-Content` cmdlet You can define how your provider will use the values passed to the `Path` parameter of the `Set-Content` cmdlet by implementing the System.Management.Automation.Provider.IContentCmdletProvider.GetContentWriter* method.

`Set-Item` cmdlet You can define how your provider will use the values passed to the `Path` and `Value` parameters of the `Set-Item` cmdlet by implementing the System.Management.Automation.Provider.ItemCmdletProvider.SetItem* method.

`Set-ItemProperty` cmdlet You can define how your provider will use the values passed to the `Path` and `Value` parameters of the `Set-Item` cmdlet by implementing the System.Management.Automation.Provider.IPropertyCmdletProvider.SetProperty* method.

`Test-Path` cmdlet You can define how your provider will use the values passed to the `Path` parameter of the `Test-Path` cmdlet by implementing the System.Management.Automation.Provider.ItemCmdletProvider.InvokeDefaultAction* method.

In addition, you cannot specify the characteristics of these parameters, such as whether they are optional or required, nor can you give these parameters an alias or specify any of the validation attributes. In contrast, you can specify parameter characteristics in stand-alone cmdlets by using attributes such as the `Parameters` attribute.

Provider Cmdlet Dynamic Parameters

Dynamic parameters for cmdlet providers are similar to dynamic providers for stand-alone cmdlets. In both cases, the parameters are added to the cmdlet when the user specifies a certain value for one of the default parameters, such as the `path` parameter. However, not all of the static parameters can be used to trigger the addition of dynamic parameters. For more information about dynamic parameters, see Provider Cmdlet Dynamic Parameters.

See Also

Provider Cmdlet Dynamic Parameters

Writing a Windows PowerShell Provider

Last updated on 05/20/2025

Provider cmdlet dynamic parameters

Providers can define dynamic parameters that are added to a provider cmdlet when the user specifies a certain value for one of the static parameters of the cmdlet. For example, a provider can add different dynamic parameters based on what path the user specifies when they call the `Get-Item` or `Set-Item` provider cmdlets.

Dynamic Parameter Methods

Dynamic parameters are defined by implementing one of the dynamic parameter methods, such as the System.Management.Automation.Provider.ItemCmdletProvider.GetItemDynamicParameters* and System.Management.Automation.Provider.SetItemDynamicParameters.SetItemDynamicParameters* methods. These methods return an object that has public properties that are decorated with attributes similar to those of stand-alone cmdlets. Here is an example of an implementation of the System.Management.Automation.Provider.ItemCmdletProvider.GetItemDynamicParameters* method taken from the Certificate provider:

C#

```
protected override object GetItemDynamicParameters(string path)
{
    return new CertificateProviderDynamicParameters();
}
```

Unlike the static parameters of provider cmdlets, you can specify the characteristics of these parameters in the same way that parameters are defined in stand-alone cmdlets. Here is an example of a dynamic parameter class taken from the Certificate provider:

C#

```
internal sealed class CertificateProviderDynamicParameters
{
  /// <summary>
  /// Dynamic parameter the controls whether we only return
  /// code signing certs.
  /// </summary>
  [Parameter()]
  public SwitchParameter CodeSigningCert
```

```
  {
    get
    {
      {
        return codeSigningCert;
      }
    }

    set
    {
      {
        codeSigningCert = value;
      }
    }
  }

    private SwitchParameter codeSigningCert = new SwitchParameter();
}
```

Dynamic Parameters

Here is a list of the static parameters that can be used to add dynamic parameters.

- `Clear-Content` cmdlet - You can define dynamic parameters that are triggered by the `Path` parameter of the Clear-Clear cmdlet by implementing the System.Management.Automation.Provider.IContentCmdletProvider.ClearContentDynamicParameters* method.

- `Clear-Item` cmdlet - You can define dynamic parameters that are triggered by the `Path` parameter of the `Clear-Item` cmdlet by implementing the System.Management.Automation.Provider.ItemCmdletProvider.ClearItemDynamicParameters* method.

- `Clear-ItemProperty` cmdlet - You can define dynamic parameters that are triggered by the `Path` parameter of the `Clear-ItemProperty` cmdlet by implementing the System.Management.Automation.Provider.IPropertyCmdletProvider.ClearPropertyDynamicParameters* method.

- `Copy-Item` cmdlet - You can define dynamic parameters that are triggered by the `Path`, `Destination`, and `Recurse` parameters of the `Copy-Item` cmdlet by implementing the System.Management.Automation.Provider.ContainerCmdletProvider.CopyItemDynamicParameters* method.

- `Get-ChildItem` cmdlet - You can define dynamic parameters that are triggered by the `Path` and `Recurse` parameters of the `Get-ChildItem` cmdlet by implementing the

System.Management.Automation.Provider.ContainerCmdletProvider.GetChildItemsDynamicParameters* and System.Management.Automation.Provider.ContainerCmdletProvider.GetChildNamesDynamicParameters* methods.

- `Get-Content` cmdlet - You can define dynamic parameters that are triggered by the `Path` parameter of the `Get-Content` cmdlet by implementing the System.Management.Automation.Provider.IContentCmdletProvider.GetContentReaderDynamicParameters* method.

- `Get-Item` cmdlet - You can define dynamic parameters that are triggered by the `Path` parameter of the `Get-Item` cmdlet by implementing the System.Management.Automation.Provider.ItemCmdletProvider.GetItemDynamicParameters* method.

- `Get-ItemProperty` cmdlet - You can define dynamic parameters that are triggered by the `Path` and `Name` parameters of the `Get-ItemProperty` cmdlet by implementing the System.Management.Automation.Provider.IPropertyCmdletProvider.GetPropertyDynamicParameters* method.

- `Invoke-Item` cmdlet - You can define dynamic parameters that are triggered by the `Path` parameter of the `Invoke-Item` cmdlet by implementing the System.Management.Automation.Provider.ItemCmdletProvider.InvokeDefaultActionDynamicParameters* method.

- `Move-Item` cmdlet - You can define dynamic parameters that are triggered by the `Path` and `Destination` parameters of the `Move-Item` cmdlet by implementing the System.Management.Automation.Provider.NavigationCmdletProvider.MoveItemDynamicParameters* method.

- `New-Item` cmdlet - You can define dynamic parameters that are triggered by the `Path`, `ItemType`, and `Value` parameters of the `New-Item` cmdlet by implementing the System.Management.Automation.Provider.ContainerCmdletProvider.NewItemDynamicParameters* method.

- `New-ItemProperty` cmdlet - You can define dynamic parameters that are triggered by the `Path`, `Name`, `PropertyType`, and `Value` parameters of the `New-ItemProperty` cmdlet by implementing the System.Management.Automation.Provider.IDynamicPropertyCmdletProvider.NewPropertyDynamicParameters* method.

- `New-PSDrive` cmdlet - You can define dynamic parameters that are triggered by the System.Management.Automation.PSDriveInfo object returned by the `New-PSDrive` cmdlet by implementing the System.Management.Automation.Provider.DriveCmdletProvider.NewDriveDynamicParameters* method.

- `Remove-Item` cmdlet - You can define dynamic parameters that are triggered by the `Path` and `Recurse` parameters of the `Remove-Item` cmdlet by implementing the System.Management.Automation.Provider.ContainerCmdletProvider.RemoveItemDynamicParameters* method.

- `Remove-ItemProperty` cmdlet - You can define dynamic parameters that are triggered by the `Path` and `Name` parameters of the `Remove-ItemProperty` cmdlet by implementing the System.Management.Automation.Provider.IDynamicPropertyCmdletProvider.RemovePropertyDynamicParameters* method.

- `Rename-Item` cmdlet - You can define dynamic parameters that are triggered by the `Path` and `NewName` parameters of the `Rename-Item` cmdlet by implementing the System.Management.Automation.Provider.ContainerCmdletProvider.RenameItemDynamicParameters* method.

- `Rename-ItemProperty` - You can define dynamic parameters that are triggered by the `Path`, `Name`, and `NewName` parameters of the `Rename-ItemProperty` cmdlet by implementing the System.Management.Automation.Provider.IDynamicPropertyCmdletProvider.RenamePropertyDynamicParameters* method.

- `Set-Content` cmdlet - You can define dynamic parameters that are triggered by the `Path` parameter of the `Set-Content` cmdlet by implementing the System.Management.Automation.Provider.IContentCmdletProvider.GetContentWriterDynamicParameters* method.

- `Set-Item` cmdlet - You can define dynamic parameters that are triggered by the `Path` and `Value` parameters of the `Set-Item` cmdlet by implementing the System.Management.Automation.Provider.ItemCmdletProvider.SetItemDynamicParameters* method.

- `Set-ItemProperty` cmdlet - You can define dynamic parameters that are triggered by the `Path` and `Value` parameters of the `Set-Item` cmdlet by implementing the

System.Management.Automation.Provider.IPropertyCmdletProvider.SetPropertyDynamicParameters* method.

- `Test-Path` cmdlet - You can define dynamic parameters that are triggered by the `Path` parameter of the `Test-Path` cmdlet by implementing the System.Management.Automation.Provider.ItemCmdletProvider.InvokeDefaultActionDynamicParameters* method.

See Also

Writing a Windows PowerShell Provider

Last updated on 08/25/2025

Writing an item provider

This topic describes how to implement the methods of a Windows PowerShell provider that access and manipulate items in the data store. To be able to access items, a provider must derive from the System.Management.Automation.Provider.ItemCmdletProvider class.

The provider in the examples in this topic uses an Access database as its data store. There are several helper methods and classes that are used to interact with the database. For the complete sample that includes the helper methods, see AccessDBProviderSample03

For more information about Windows PowerShell providers, see Windows PowerShell Provider Overview.

Implementing item methods

The System.Management.Automation.Provider.ItemCmdletProvider class exposes several methods that can be used to access and manipulate the items in a data store. For a complete list of these methods, see ItemCmdletProvider Methods. In this example, we will implement four of these methods.
System.Management.Automation.Provider.ItemCmdletProvider.GetItem* gets an item at a specified path. System.Management.Automation.Provider.ItemCmdletProvider.SetItem* sets the value of the specified item.
System.Management.Automation.Provider.ItemCmdletProvider.ItemExists* checks whether an item exists at the specified path.
System.Management.Automation.Provider.ItemCmdletProvider.IsValidPath* checks a path to see if it maps to a location in the data store.

> **Note**
>
> This topic builds on the information in Windows PowerShell Provider QuickStart. This topic does not cover the basics of how to set up a provider project, or how to implement the methods inherited from the System.Management.Automation.Provider.DriveCmdletProvider class that create and remove drives.

Declaring the provider class

Declare the provider to derive from the System.Management.Automation.Provider.ItemCmdletProvider class, and decorate it with the System.Management.Automation.Provider.CmdletProviderAttribute.

C#

```
[CmdletProvider("AccessDB", ProviderCapabilities.None)]

   public class AccessDBProvider : ItemCmdletProvider
   {

  }
```

Implementing GetItem

The System.Management.Automation.Provider.ItemCmdletProvider.GetItem* is called by the PowerShell engine when a user calls the Microsoft.PowerShell.Commands.GetItemCommand cmdlet on your provider. The method returns the item at the specified path. In the Access database example, the method checks whether the item is the drive itself, a table in the database, or a row in the database. The method sends the item to the PowerShell engine by calling the System.Management.Automation.Provider.CmdletProvider.WriteItemObject* method.

C#

```
protected override void GetItem(string path)
      {
          // check if the path represented is a drive
          if (PathIsDrive(path))
          {
              WriteItemObject(this.PSDriveInfo, path, true);
              return;
          }// if (PathIsDrive...

           // Get table name and row information from the path and do
           // necessary actions
           string tableName;
           int rowNumber;

           PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

           if (type == PathType.Table)
           {
               DatabaseTableInfo table = GetTable(tableName);
               WriteItemObject(table, path, true);
           }
```

```
            else if (type == PathType.Row)
            {
                DatabaseRowInfo row = GetRow(tableName, rowNumber);
                WriteItemObject(row, path, false);
            }
            else
            {
                ThrowTerminatingInvalidPathException(path);
            }

        }
```

Implementing SetItem

The System.Management.Automation.Provider.ItemCmdletProvider.SetItem* method is called by the PowerShell engine calls when a user calls the Microsoft.PowerShell.Commands.SetItemCommand cmdlet. It sets the value of the item at the specified path.

In the Access database example, it makes sense to set the value of an item only if that item is a row, so the method throws NotSupportedException when the item is not a row.

C#

```
protected override void SetItem(string path, object values)
        {
            // Get type, table name and row number from the path specified
            string tableName;
            int rowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            if (type != PathType.Row)
            {
                WriteError(new ErrorRecord(new NotSupportedException(
                      "SetNotSupported"), "",
                   ErrorCategory.InvalidOperation, path));

                return;
            }

            // Get in-memory representation of table
            OdbcDataAdapter da = GetAdapterForTable(tableName);

            if (da == null)
            {
                return;
            }
            DataSet ds = GetDataSetForTable(da, tableName);
            DataTable table = GetDataTable(ds, tableName);
```

```
            if (rowNumber >= table.Rows.Count)
            {
                // The specified row number has to be available. If not
                // NewItem has to be used to add a new row
                throw new ArgumentException("Row specified is not available");
            } // if (rowNum...

            string[] colValues = (values as string).Split(',');

            // set the specified row
            DataRow row = table.Rows[rowNumber];

            for (int i = 0; i < colValues.Length; i++)
            {
                row[i] = colValues[i];
            }

            // Update the table
            if (ShouldProcess(path, "SetItem"))
            {
                da.Update(ds, tableName);
            }

        }
```

Implementing ItemExists

The System.Management.Automation.Provider.ItemCmdletProvider.ItemExists* method is called by the PowerShell engine when a user calls the Microsoft.PowerShell.Commands.TestPathCommand cmdlet. The method determines whether there is an item at the specified path. If the item does exist, the method passes it back to the PowerShell engine by calling System.Management.Automation.Provider.CmdletProvider.WriteItemObject*.

C#

```
protected override bool ItemExists(string path)
        {
            // check if the path represented is a drive
            if (PathIsDrive(path))
            {
                return true;
            }

            // Obtain type, table name and row number from path
            string tableName;
            int rowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);
```

```
            DatabaseTableInfo table = GetTable(tableName);

            if (type == PathType.Table)
            {
                // if specified path represents a table then DatabaseTableInfo
                // object for the same should exist
                if (table != null)
                {
                    return true;
                }
            }
            else if (type == PathType.Row)
            {
                // if specified path represents a row then DatabaseTableInfo should
                // exist for the table and then specified row number must be within
                // the maximum row count in the table
                if (table != null && rowNumber < table.RowCount)
                {
                    return true;
                }
            }

            return false;

        }
```

Implementing IsValidPath

The System.Management.Automation.Provider.ItemCmdletProvider.IsValidPath* method checks whether the specified path is syntactically valid for the current provider. It does not check whether an item exists at the path.

C#

```
protected override bool IsValidPath(string path)
        {
            bool result = true;

            // check if the path is null or empty
            if (String.IsNullOrEmpty(path))
            {
                result = false;
            }

            // convert all separators in the path to a uniform one
            path = NormalizePath(path);

            // split the path into individual chunks
            string[] pathChunks = path.Split(pathSeparator.ToCharArray());
```

```
            foreach (string pathChunk in pathChunks)
            {
                if (pathChunk.Length == 0)
                {
                    result = false;
                }
            }
            return result;
        }
```

Next steps

A typical real-world provider is capable of supporting items that contain other items, and of moving items from one path to another within the drive. For an example of a provider that supports containers, see Writing a container provider. For an example of a provider that supports moving items, see Writing a navigation provider.

See Also

Writing a container provider

Writing a navigation provider

Windows PowerShell Provider Overview

Last updated on 05/20/2025

Writing a container provider

This topic describes how to implement the methods of a Windows PowerShell provider that support items that contain other items, such as folders in the FileSystem provider. To be able to support containers, a provider must derive from the System.Management.Automation.Provider.ContainerCmdletProvider class.

The provider in the examples in this topic uses an Access database as its data store. There are several helper methods and classes that are used to interact with the database. For the complete sample that includes the helper methods, see AccessDBProviderSample04.

For more information about Windows PowerShell providers, see Windows PowerShell Provider Overview.

Implementing container methods

The System.Management.Automation.Provider.ContainerCmdletProvider class implements methods that support containers, and create, copy, and remove items. For a complete list of these methods, see System.Management.Automation.Provider.ContainerCmdletProvider.

> **Note**
>
> This topic builds on the information in Windows PowerShell Provider QuickStart. This topic does not cover the basics of how to set up a provider project, or how to implement the methods inherited from the System.Management.Automation.Provider.DriveCmdletProvider class that create and remove drives. This topic also does not cover how to implement methods exposed by the System.Management.Automation.Provider.ItemCmdletProvider class. For an example that shows how to implement item cmdlets, see Writing an item provider.

Declaring the provider class

Declare the provider to derive from the System.Management.Automation.Provider.ContainerCmdletProvider class, and decorate it with the System.Management.Automation.Provider.CmdletProviderAttribute.

C#

```
[CmdletProvider("AccessDB", ProviderCapabilities.None)]
   public class AccessDBProvider : ContainerCmdletProvider
   {

   }
```

Implementing GetChildItems

The PowerShell engine calls the System.Management.Automation.Provider.ContainerCmdletProvider.GetChildItems* method when a user calls the Microsoft.PowerShell.Commands.GetChildItemCommand cmdlet. This method gets the items that are the children of the item at the specified path.

In the Access database example, the behavior of the System.Management.Automation.Provider.ContainerCmdletProvider.GetChildItems* method depends on the type of the specified item. If the item is the drive, then the children are tables, and the method returns the set of tables from the database. If the specified item is a table, then the children are the rows of that table. If the item is a row, then it has no children, and the method returns that row only. All child items are sent back to the PowerShell engine by the System.Management.Automation.Provider.CmdletProvider.WriteItemObject* method.

C#

```
protected override void GetChildItems(string path, bool recurse)
       {
           // If path represented is a drive then the children in the path are
           // tables. Hence all tables in the drive represented will have to be
           // returned
           if (PathIsDrive(path))
           {
               foreach (DatabaseTableInfo table in GetTables())
               {
                   WriteItemObject(table, path, true);

                   // if the specified item exists and recurse has been set then
                   // all child items within it have to be obtained as well
                   if (ItemExists(path) && recurse)
                   {
                       GetChildItems(path + pathSeparator + table.Name, recurse);
                   }
               } // foreach (DatabaseTableInfo...
           } // if (PathIsDrive...
           else
           {
               // Get the table name, row number and type of path from the
               // path specified
               string tableName;
```

```
                int rowNumber;

                PathType type = GetNamesFromPath(path, out tableName, out
rowNumber);

                if (type == PathType.Table)
                {
                    // Obtain all the rows within the table
                    foreach (DatabaseRowInfo row in GetRows(tableName))
                    {
                        WriteItemObject(row, path + pathSeparator + row.RowNumber,
                                false);
                    } // foreach (DatabaseRowInfo...
                }
                else if (type == PathType.Row)
                {
                    // In this case the user has directly specified a row, hence
                    // just give that particular row
                    DatabaseRowInfo row = GetRow(tableName, rowNumber);
                    WriteItemObject(row, path + pathSeparator + row.RowNumber,
                                false);
                }
                else
                {
                    // In this case, the path specified is not valid
                    ThrowTerminatingInvalidPathException(path);
                }
            } // else
        }
```

Implementing GetChildNames

The System.Management.Automation.Provider.ContainerCmdletProvider.GetChildNames* method is similar to the System.Management.Automation.Provider.ContainerCmdletProvider.GetChildItems* method, except that it returns only the name property of the items, and not the items themselves.

C#

```
protected override void GetChildNames(string path,
                                      ReturnContainers returnContainers)
        {
            // If the path represented is a drive, then the child items are
            // tables. get the names of all the tables in the drive.
            if (PathIsDrive(path))
            {
                foreach (DatabaseTableInfo table in GetTables())
                {
                    WriteItemObject(table.Name, path, true);
                } // foreach (DatabaseTableInfo...
            } // if (PathIsDrive...
```

```
            else
            {
                // Get type, table name and row number from path specified
                string tableName;
                int rowNumber;

                PathType type = GetNamesFromPath(path, out tableName, out
rowNumber);

                if (type == PathType.Table)
                {
                    // Get all the rows in the table and then write out the
                    // row numbers.
                    foreach (DatabaseRowInfo row in GetRows(tableName))
                    {
                        WriteItemObject(row.RowNumber, path, false);
                    } // foreach (DatabaseRowInfo...
                }
                else if (type == PathType.Row)
                {
                    // In this case the user has directly specified a row, hence
                    // just give that particular row
                    DatabaseRowInfo row = GetRow(tableName, rowNumber);

                    WriteItemObject(row.RowNumber, path, false);
                }
                else
                {
                    ThrowTerminatingInvalidPathException(path);
                }
            } // else
        }
```

Implementing NewItem

The System.Management.Automation.Provider.ContainerCmdletProvider.NewItem* method creates a new item of the specified type at the specified path. The PowerShell engine calls this method when a user calls the Microsoft.PowerShell.Commands.NewItemCommand cmdlet.

In this example, the method implements logic to determine that the path and type match. That is, only a table can be created directly under the drive (the database), and only a row can be created under a table. If the specified path and item type don't match in this way, the method throws an exception.

C#

```
protected override void NewItem(string path, string type,
                                object newItemValue)
        {
            string tableName;
```

```
            int rowNumber;

            PathType pt = GetNamesFromPath(path, out tableName, out rowNumber);

            if (pt == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(path);
            }

            // Check if type is either "table" or "row", if not throw an
            // exception
            if (!String.Equals(type, "table", StringComparison.OrdinalIgnoreCase)
                && !String.Equals(type, "row", StringComparison.OrdinalIgnoreCase))
            {
                WriteError(new ErrorRecord
                                (new ArgumentException("Type must be either a
table or row"),
                                    "CannotCreateSpecifiedObject",
                                       ErrorCategory.InvalidArgument,
                                            path
                                    )
                           );

                throw new ArgumentException("This provider can only create items of
type \"table\" or \"row\"");
            }

            // Path type is the type of path of the container. So if a drive
            // is specified, then a table can be created under it and if a table
            // is specified, then a row can be created under it. For the sake of
            // completeness, if a row is specified, then if the row specified by
            // the path does not exist, a new row is created. However, the row
            // number may not match as the row numbers only get incremented based
            // on the number of rows

            if (PathIsDrive(path))
            {
                if (String.Equals(type, "table",
StringComparison.OrdinalIgnoreCase))
                {
                    // Execute command using ODBC connection to create a table
                    try
                    {
                        // create the table using an sql statement
                        string newTableName = newItemValue.ToString();

                        if (!TableNameIsValid(newTableName))
                        {
                            return;
                        }
                        string sql = "create table " + newTableName
                                               + " (ID INT)";

                        // Create the table using the Odbc connection from the
                        // drive.
```

```
                        AccessDBPSDriveInfo di = this.PSDriveInfo as
AccessDBPSDriveInfo;

                        if (di == null)
                        {
                            return;
                        }
                        OdbcConnection connection = di.Connection;

                        if (ShouldProcess(newTableName, "create"))
                        {
                            OdbcCommand cmd = new OdbcCommand(sql, connection);
                            cmd.ExecuteScalar();
                        }
                    }
                    catch (Exception ex)
                    {
                        WriteError(new ErrorRecord(ex, "CannotCreateSpecifiedTable",
                                  ErrorCategory.InvalidOperation, path)
                                  );
                    }
                } // if (String...
                else if (String.Equals(type, "row",
StringComparison.OrdinalIgnoreCase))
                {
                    throw new
                        ArgumentException("A row cannot be created under a database,
specify a path that represents a Table");
                }
            }// if (PathIsDrive...
            else
            {
                if (String.Equals(type, "table",
StringComparison.OrdinalIgnoreCase))
                {
                    if (rowNumber < 0)
                    {
                        throw new
                            ArgumentException("A table cannot be created within
another table, specify a path that represents a database");
                    }
                    else
                    {
                        throw new
                            ArgumentException("A table cannot be created inside a
row, specify a path that represents a database");
                    }
                } //if (String.Equals....
                // if path specified is a row, create a new row
                else if (String.Equals(type, "row",
StringComparison.OrdinalIgnoreCase))
                {
                    // The user is required to specify the values to be inserted
                    // into the table in a single string separated by commas
                    string value = newItemValue as string;
```

```
                    if (String.IsNullOrEmpty(value))
                    {
                        throw new
                            ArgumentException("Value argument must have comma
separated values of each column in a row");
                    }
                    string[] rowValues = value.Split(',');

                    OdbcDataAdapter da = GetAdapterForTable(tableName);

                    if (da == null)
                    {
                        return;
                    }

                    DataSet ds = GetDataSetForTable(da, tableName);
                    DataTable table = GetDataTable(ds, tableName);

                    if (rowValues.Length != table.Columns.Count)
                    {
                        string message =
                             String.Format(CultureInfo.CurrentCulture,
                                           "The table has {0} columns and the
value specified must have so many comma separated values",
                                               table.Columns.Count);

                        throw new ArgumentException(message);
                    }

                    if (!Force && (rowNumber >=0 && rowNumber < table.Rows.Count))
                    {
                        string message = String.Format(CultureInfo.CurrentCulture,
                                                       "The row {0} already
exists. To create a new row specify row number as {1}, or specify path to a table,
or use the -Force parameter",
                                                           rowNumber,
table.Rows.Count);

                        throw new ArgumentException(message);
                    }

                    if (rowNumber > table.Rows.Count)
                    {
                        string message = String.Format(CultureInfo.CurrentCulture,
                                           "To create a new row specify row number
as {0}, or specify path to a table",
                                               table.Rows.Count);

                        throw new ArgumentException(message);
                    }

                    // Create a new row and update the row with the input
                    // provided by the user
                    DataRow row = table.NewRow();
```

```
                for (int i = 0; i < rowValues.Length; i++)
                {
                    row[i] = rowValues[i];
                }
                table.Rows.Add(row);

                if (ShouldProcess(tableName, "update rows"))
                {
                    // Update the table from memory back to the data source
                    da.Update(ds, tableName);
                }

            }// else if (String...
        }// else ...

    }
```

Implementing CopyItem

The System.Management.Automation.Provider.ContainerCmdletProvider.CopyItem copies the specified item to the specified path. The PowerShell engine calls this method when a user calls the Microsoft.PowerShell.Commands.CopyItemCommand cmdlet. This method can also be recursive, copying all of the items children in addition to the item itself.

Similarly to the System.Management.Automation.Provider.ContainerCmdletProvider.NewItem* method, this method performs logic to make sure that the specified item is of the correct type for the path to which it is being copied. For example, if the destination path is a table, the item to be copied must be a row.

C#

```
protected override void CopyItem(string path, string copyPath, bool recurse)
    {
        string tableName, copyTableName;
        int rowNumber, copyRowNumber;

        PathType type = GetNamesFromPath(path, out tableName, out rowNumber);
        PathType copyType = GetNamesFromPath(copyPath, out copyTableName, out
copyRowNumber);

        if (type == PathType.Invalid)
        {
            ThrowTerminatingInvalidPathException(path);
        }

        if (type == PathType.Invalid)
        {
            ThrowTerminatingInvalidPathException(copyPath);
        }
```

```
            // Get the table and the table to copy to
            OdbcDataAdapter da = GetAdapterForTable(tableName);
            if (da == null)
            {
                return;
            }

            DataSet ds = GetDataSetForTable(da, tableName);
            DataTable table = GetDataTable(ds, tableName);

            OdbcDataAdapter cda = GetAdapterForTable(copyTableName);
            if (cda == null)
            {
                return;
            }

            DataSet cds = GetDataSetForTable(cda, copyTableName);
            DataTable copyTable = GetDataTable(cds, copyTableName);

            // if source represents a table
            if (type == PathType.Table)
            {
                // if copyPath does not represent a table
                if (copyType != PathType.Table)
                {
                    ArgumentException e = new ArgumentException("Table can only be
copied on to another table location");

                    WriteError(new ErrorRecord(e, "PathNotValid",
                        ErrorCategory.InvalidArgument, copyPath));

                    throw e;
                }

                // if table already exists then Force parameter should be set
                // to force a copy
                if (!Force && GetTable(copyTableName) != null)
                {
                    throw new ArgumentException("Specified path already exists");
                }

                for (int i = 0; i < table.Rows.Count; i++)
                {
                    DataRow row = table.Rows[i];
                    DataRow copyRow = copyTable.NewRow();

                    copyRow.ItemArray = row.ItemArray;
                    copyTable.Rows.Add(copyRow);
                }
            } // if (type == ...
            // if source represents a row
            else
            {
                if (copyType == PathType.Row)
```

```
            {
                if (!Force && (copyRowNumber < copyTable.Rows.Count))
                {
                    throw new ArgumentException("Specified path already
exists.");
                }

                DataRow row = table.Rows[rowNumber];
                DataRow copyRow = null;

                if (copyRowNumber < copyTable.Rows.Count)
                {
                    // copy to an existing row
                    copyRow = copyTable.Rows[copyRowNumber];
                    copyRow.ItemArray = row.ItemArray;
                    copyRow[0] = GetNextID(copyTable);
                }
                else if (copyRowNumber == copyTable.Rows.Count)
                {
                    // copy to the next row in the table that will
                    // be created
                    copyRow = copyTable.NewRow();
                    copyRow.ItemArray = row.ItemArray;
                    copyRow[0] = GetNextID(copyTable);
                    copyTable.Rows.Add(copyRow);
                }
                else
                {
                    // attempting to copy to a nonexistent row or a row
                    // that cannot be created now - throw an exception
                    string message = String.Format(CultureInfo.CurrentCulture,
                                                   "The item cannot be specified to the
copied row. Specify row number as {0}, or specify a path to the table.",
                                                   table.Rows.Count);

                    throw new ArgumentException(message);
                }
            }
            else
            {
                // destination path specified represents a table,
                // create a new row and copy the item
                DataRow copyRow = copyTable.NewRow();
                copyRow.ItemArray = table.Rows[rowNumber].ItemArray;
                copyRow[0] = GetNextID(copyTable);
                copyTable.Rows.Add(copyRow);
            }
        }

        if (ShouldProcess(copyTableName, "CopyItems"))
        {
            cda.Update(cds, copyTableName);
        }

    } //CopyItem
```

Implementing RemoveItem

The System.Management.Automation.Provider.ContainerCmdletProvider.RemoveItem* method removes the item at the specified path. The PowerShell engine calls this method when a user calls the Microsoft.PowerShell.Commands.RemoveItemCommand cmdlet.

C#

```
protected override void RemoveItem(string path, bool recurse)
       {
           string tableName;
           int rowNumber = 0;

           PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

           if (type == PathType.Table)
           {
               // if recurse flag has been specified, delete all the rows as well
               if (recurse)
               {
                   OdbcDataAdapter da = GetAdapterForTable(tableName);
                   if (da == null)
                   {
                       return;
                   }

                   DataSet ds = GetDataSetForTable(da, tableName);
                   DataTable table = GetDataTable(ds, tableName);

                   for (int i = 0; i < table.Rows.Count; i++)
                   {
                       table.Rows[i].Delete();
                   }

                   if (ShouldProcess(path, "RemoveItem"))
                   {
                       da.Update(ds, tableName);
                       RemoveTable(tableName);
                   }
               }//if (recurse...
               else
               {
                   // Remove the table
                   if (ShouldProcess(path, "RemoveItem"))
                   {
                       RemoveTable(tableName);
                   }
               }
           }
           else if (type == PathType.Row)
```

```
        {
            OdbcDataAdapter da = GetAdapterForTable(tableName);
            if (da == null)
            {
                return;
            }

            DataSet ds = GetDataSetForTable(da, tableName);
            DataTable table = GetDataTable(ds, tableName);

            table.Rows[rowNumber].Delete();

            if (ShouldProcess(path, "RemoveItem"))
            {
                da.Update(ds, tableName);
            }
        }
        else
        {
            ThrowTerminatingInvalidPathException(path);
        }

    }
```

Next steps

A typical real-world provider is capable of moving items from one path to another within the drive. For an example of a provider that supports moving items, see Writing a navigation provider.

See Also

Writing a navigation provider

Windows PowerShell Provider Overview

Last updated on 05/20/2025

Writing a navigation provider

This topic describes how to implement the methods of a Windows PowerShell provider that support nested containers (multi-level data stores), moving items, and relative paths. A navigation provider must derive from the System.Management.Automation.Provider.NavigationCmdletProvider class.

The provider in the examples in this topic uses an Access database as its data store. There are several helper methods and classes that are used to interact with the database. For the complete sample that includes the helper methods, see AccessDBProviderSample05.

For more information about Windows PowerShell providers, see Windows PowerShell Provider Overview.

Implementing navigation methods

The System.Management.Automation.Provider.NavigationCmdletProvider class implements methods that support nested containers, relative paths, and moving items. For a complete list of these methods, see NavigationCmdletProvider Methods.

> **Note**
>
> This topic builds on the information in Windows PowerShell Provider QuickStart. This topic does not cover the basics of how to set up a provider project, or how to implement the methods inherited from the System.Management.Automation.Provider.DriveCmdletProvider class that create and remove drives. This topic also does not cover how to implement methods exposed by the System.Management.Automation.Provider.ItemCmdletProvider or System.Management.Automation.Provider.ContainerCmdletProvider classes. For an example that shows how to implement item cmdlets, see Writing an item provider. For an example that shows how to implement container cmdlets, see Writing a container provider.

Declaring the provider class

Declare the provider to derive from the System.Management.Automation.Provider.NavigationCmdletProvider class, and decorate it

with the System.Management.Automation.Provider.CmdletProviderAttribute.

```
[CmdletProvider("AccessDB", ProviderCapabilities.None)]
   public class AccessDBProvider : NavigationCmdletProvider
   {

   }
```

Implementing IsItemContainer

The System.Management.Automation.Provider.NavigationCmdletProvider.IsItemContainer* method checks whether the item at the specified path is a container.

C#

```
protected override bool IsItemContainer(string path)
      {
         if (PathIsDrive(path))
         {
             return true;
         }

         string[] pathChunks = ChunkPath(path);
         string tableName;
         int rowNumber;

         PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

         if (type == PathType.Table)
         {
            foreach (DatabaseTableInfo ti in GetTables())
            {
                if (string.Equals(ti.Name, tableName,
StringComparison.OrdinalIgnoreCase))
                {
                    return true;
                }
            } // foreach (DatabaseTableInfo...
         } // if (pathChunks...

         return false;
      }
```

Implementing GetChildName

The System.Management.Automation.Provider.NavigationCmdletProvider.GetChildName* method gets the name property of the child item at the specified path. If the item at the specified path is not a child of a container, then this method should return the path.

C#

```
protected override string GetChildName(string path)
       {
           if (PathIsDrive(path))
           {
               return path;
           }

           string tableName;
           int rowNumber;

           PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

           if (type == PathType.Table)
           {
               return tableName;
           }
           else if (type == PathType.Row)
           {
               return rowNumber.ToString(CultureInfo.CurrentCulture);
           }
           else
           {
               ThrowTerminatingInvalidPathException(path);
           }

           return null;
       }
```

Implementing GetParentPath

The System.Management.Automation.Provider.NavigationCmdletProvider.GetParentPath* method gets the path of the parent of the item at the specified path. If the item at the specified path is the root of the data store (so it has no parent), then this method should return the root path.

C#

```
protected override string GetParentPath(string path, string root)
       {
           // If root is specified then the path has to contain
           // the root. If not nothing should be returned
           if (!String.IsNullOrEmpty(root))
           {
```

```
                if (!path.Contains(root))
                {
                    return null;
                }
            }

            return path.Substring(0, path.LastIndexOf(pathSeparator,
StringComparison.OrdinalIgnoreCase));
        }
```

Implementing MakePath

The System.Management.Automation.Provider.NavigationCmdletProvider.MakePath* method joins a specified parent path and a specified child path to create a provider-internal path (for information about path types that providers can support, see Windows PowerShell Provider Overview. The PowerShell engine calls this method when a user calls the Microsoft.PowerShell.Commands.JoinPathCommand cmdlet.

C#

```
protected override string MakePath(string parent, string child)
        {
            string result;

            string normalParent = NormalizePath(parent);
            normalParent = RemoveDriveFromPath(normalParent);
            string normalChild = NormalizePath(child);
            normalChild = RemoveDriveFromPath(normalChild);

            if (String.IsNullOrEmpty(normalParent) &&
String.IsNullOrEmpty(normalChild))
            {
                result = String.Empty;
            }
            else if (String.IsNullOrEmpty(normalParent) &&
!String.IsNullOrEmpty(normalChild))
            {
                result = normalChild;
            }
            else if (!String.IsNullOrEmpty(normalParent) &&
String.IsNullOrEmpty(normalChild))
            {
                if (normalParent.EndsWith(pathSeparator,
StringComparison.OrdinalIgnoreCase))
                {
                    result = normalParent;
                }
                else
                {
                    result = normalParent + pathSeparator;
```

```
            }
        } // else if (!String...
        else
        {
            if (!normalParent.Equals(String.Empty) &&
                !normalParent.EndsWith(pathSeparator,
StringComparison.OrdinalIgnoreCase))
            {
                result = normalParent + pathSeparator;
            }
            else
            {
                result = normalParent;
            }

            if (normalChild.StartsWith(pathSeparator,
StringComparison.OrdinalIgnoreCase))
            {
                result += normalChild.Substring(1);
            }
            else
            {
                result += normalChild;
            }
        } // else

        return result;
    }
```

Implementing NormalizeRelativePath

The System.Management.Automation.Provider.NavigationCmdletProvider.NormalizeRelativePath* method takes `path` and `basepath` parameters, and returns a normalized path that is equivalent to the `path` parameter and relative to the `basepath` parameter.

C#

```
protected override string NormalizeRelativePath(string path,
                                                 string basepath)
    {
        // Normalize the paths first
        string normalPath = NormalizePath(path);
        normalPath = RemoveDriveFromPath(normalPath);
        string normalBasePath = NormalizePath(basepath);
        normalBasePath = RemoveDriveFromPath(normalBasePath);

        if (String.IsNullOrEmpty(normalBasePath))
        {
            return normalPath;
        }
```

```
            else
            {
                if (!normalPath.Contains(normalBasePath))
                {
                    return null;
                }

                return normalPath.Substring(normalBasePath.Length +
pathSeparator.Length);
            }
        }
```

Implementing MoveItem

The System.Management.Automation.Provider.NavigationCmdletProvider.MoveItem* method moves an item from the specified path to the specified destination path. The PowerShell engine calls this method when a user calls the Microsoft.PowerShell.Commands.MoveItemCommand cmdlet.

C#

```
protected override void MoveItem(string path, string destination)
        {
            // Get type, table name and rowNumber from the path
            string tableName, destTableName;
            int rowNumber, destRowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            PathType destType = GetNamesFromPath(destination, out destTableName,
                                    out destRowNumber);

            if (type == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(path);
            }

            if (destType == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(destination);
            }

            if (type == PathType.Table)
            {
                ArgumentException e = new ArgumentException("Move not supported for
tables");

                WriteError(new ErrorRecord(e, "MoveNotSupported",
                    ErrorCategory.InvalidArgument, path));

                throw e;
```

```
            }
            else
            {
                OdbcDataAdapter da = GetAdapterForTable(tableName);
                if (da == null)
                {
                    return;
                }

                DataSet ds = GetDataSetForTable(da, tableName);
                DataTable table = GetDataTable(ds, tableName);

                OdbcDataAdapter dda = GetAdapterForTable(destTableName);
                if (dda == null)
                {
                    return;
                }

                DataSet dds = GetDataSetForTable(dda, destTableName);
                DataTable destTable = GetDataTable(dds, destTableName);
                DataRow row = table.Rows[rowNumber];

                if (destType == PathType.Table)
                {
                    DataRow destRow = destTable.NewRow();

                    destRow.ItemArray = row.ItemArray;
                }
                else
                {
                    DataRow destRow = destTable.Rows[destRowNumber];

                    destRow.ItemArray = row.ItemArray;
                }

                // Update the changes
                if (ShouldProcess(path, "MoveItem"))
                {
                    WriteItemObject(row, path, false);
                    dda.Update(dds, destTableName);
                }
            }
        }
```

See Also

Writing a container provider

Windows PowerShell Provider Overview

Last updated on 05/20/2025

Provider Samples

This section includes samples of providers that access a Microsoft Access database. These samples include provider classes that derive from all the base provider classes.

In This Section

This section includes the following topics:

AccessDBProviderSample01 Sample This sample shows how to declare the provider class that derives directly from the System.Management.Automation.Provider.CmdletProvider class. It is included here only for completeness.

AccessDBProviderSample02 This sample shows how to overwrite the System.Management.Automation.Provider.DriveCmdletProvider.NewDrive* and System.Management.Automation.Provider.DriveCmdletProvider.RemoveDrive* methods to support calls to the `New-PSDrive` and `Remove-PSDrive` cmdlets. The provider class in this sample derives from the System.Management.Automation.Provider.DriveCmdletProvider class.

AccessDBProviderSample03 This sample shows how to overwrite the System.Management.Automation.Provider.ItemCmdletProvider.GetItem* and System.Management.Automation.Provider.ItemCmdletProvider.SetItem* methods to support calls to the `Get-Item` and `Set-Item` cmdlets. The provider class in this sample derives from the System.Management.Automation.Provider.ItemCmdletProvider class.

AccessDBProviderSample04 This sample shows how to overwrite container methods to support calls to the `Copy-Item`, `Get-ChildItem`, `New-Item`, and `Remove-Item` cmdlets. These methods should be implemented when the data store contains items that are containers. A container is a group of child items under a common parent item. The provider class in this sample derives from the System.Management.Automation.Provider.ContainerCmdletProvider class.

AccessDBProviderSample05 This sample shows how to overwrite container methods to support calls to the `Move-Item` and `Join-Path` cmdlets. These methods should be implemented when the user needs to move items within a container and if the data store contains nested containers. The provider class in this sample derives from the System.Management.Automation.Provider.NavigationCmdletProvider class.

AccessDBProviderSample06 This sample shows how to overwrite content methods to support calls to the `Clear-Content`, `Get-Content`, and `Set-Content` cmdlets. These methods should be implemented when the user needs to manage the content of the items in the data store. The provider class in this sample derives from the System.Management.Automation.Provider.NavigationCmdletProvider class, and it implements the System.Management.Automation.Provider.IContentCmdletProvider interface.

See Also

Writing a Windows PowerShell Provider

Last updated on 05/20/2025

AccessDBProviderSample01

This sample shows how to declare a provider class that derives directly from the System.Management.Automation.Provider.CmdletProvider class. It is included here only for completeness.

Demonstrates

> **Important**
>
> Your provider class will most likely derive from one of the following classes and possibly implement other provider interfaces:
>
> - System.Management.Automation.Provider.ItemCmdletProvider class. See AccessDBProviderSample03.
> - System.Management.Automation.Provider.ContainerCmdletProvider class. See AccessDBProviderSample04.
> - System.Management.Automation.Provider.NavigationCmdletProvider class. See AccessDBProviderSample05.
>
> For more information about choosing which provider class to derive from based on provider features, see Designing Your Windows PowerShell Provider.

This sample demonstrates the following:

- Declaring the `CmdletProvider` attribute.
- Defining a provider class that derives directly from the System.Management.Automation.Provider.CmdletProvider class.

Example

This sample shows how to define a provider class and how to declare the `CmdletProvider` attribute.

C#

```
using System.Management.Automation;
using System.Management.Automation.Provider;
using System.ComponentModel;


namespace Microsoft.Samples.PowerShell.Providers
{
   #region AccessDBProvider

    /// <summary>
   /// Simple provider.
   /// </summary>
   [CmdletProvider("AccessDB", ProviderCapabilities.None)]
   public class AccessDBProvider : CmdletProvider
   {

   }

   #endregion AccessDBProvider
}
```

See Also

System.Management.Automation.Provider.ItemCmdletProvider

System.Management.Automation.Provider.ContainerCmdletProvider

System.Management.Automation.Provider.NavigationCmdletProvider

Designing Your Windows PowerShell Provider

Last updated on 05/20/2025

AccessDBProviderSample02

This sample shows how to overwrite the System.Management.Automation.Provider.DriveCmdletProvider.NewDrive* and System.Management.Automation.Provider.DriveCmdletProvider.RemoveDrive* methods to support calls to the `New-PSDrive` and `Remove-PSDrive` cmdlets. The provider class in this sample derives from the System.Management.Automation.Provider.DriveCmdletProvider class.

Demonstrates

> **Important**
>
> Your provider class will most likely derive from one of the following classes and possibly implement other provider interfaces:
>
> - System.Management.Automation.Provider.ItemCmdletProvider class. See AccessDBProviderSample03.
> - System.Management.Automation.Provider.ContainerCmdletProvider class. See AccessDBProviderSample04.
> - System.Management.Automation.Provider.NavigationCmdletProvider class. See AccessDBProviderSample05.
>
> For more information about choosing which provider class to derive from based on provider features, see Designing Your Windows PowerShell Provider.

This sample demonstrates the following:

- Declaring the `CmdletProvider` attribute.
- Defining a provider class that drives from the System.Management.Automation.Provider.DriveCmdletProvider class.
- Overwriting the System.Management.Automation.Provider.DriveCmdletProvider.NewDrive* method to support creating new drives. (This sample does not show how to add dynamic parameters to the `New-PSDrive` cmdlet.)

- Overwriting the System.Management.Automation.Provider.DriveCmdletProvider.RemoveDrive* method to support removing existing drives.

Example

This sample shows how to overwrite the System.Management.Automation.Provider.DriveCmdletProvider.NewDrive* and System.Management.Automation.Provider.DriveCmdletProvider.RemoveDrive* methods. For this sample provider, when a drive is created its connection information is stored in an `AccessDBPsDriveInfo` object.

C#

```
using System;
using System.IO;
using System.Data;
using System.Data.Odbc;
using System.Management.Automation;
using System.Management.Automation.Provider;
using System.ComponentModel;

namespace Microsoft.Samples.PowerShell.Providers
{
   #region AccessDBProvider

    /// <summary>
   /// A PowerShell Provider which acts upon a access data store.
   /// </summary>
   /// <remarks>
   /// This example only demonstrates the drive overrides
   /// </remarks>
   [CmdletProvider("AccessDB", ProviderCapabilities.None)]
   public class AccessDBProvider : DriveCmdletProvider
   {
       #region Drive Manipulation

       /// <summary>
       /// Create a new drive.  Create a connection to the database file and set
       /// the Connection property in the PSDriveInfo.
       /// </summary>
       /// <param name="drive">
       /// Information describing the drive to add.
       /// </param>
       /// <returns>The added drive.</returns>
       protected override PSDriveInfo NewDrive(PSDriveInfo drive)
       {
           // check if drive object is null
           if (drive == null)
```

```
            {
                WriteError(new ErrorRecord(
                    new ArgumentNullException("drive"),
                    "NullDrive",
                    ErrorCategory.InvalidArgument,
                    null)
                );

                return null;
            }

            // check if drive root is not null or empty
            // and if its an existing file
            if (String.IsNullOrEmpty(drive.Root) || (File.Exists(drive.Root) ==
false))
            {
                WriteError(new ErrorRecord(
                    new ArgumentException("drive.Root"),
                    "NoRoot",
                    ErrorCategory.InvalidArgument,
                    drive)
                );

                return null;
            }

            // create a new drive and create an ODBC connection to the new drive
            AccessDBPSDriveInfo accessDBPSDriveInfo = new
AccessDBPSDriveInfo(drive);

            OdbcConnectionStringBuilder builder = new OdbcConnectionStringBuilder();

            builder.Driver = "Microsoft Access Driver (*.mdb)";
            builder.Add("DBQ", drive.Root);

            OdbcConnection conn = new OdbcConnection(builder.ConnectionString);
            conn.Open();
            accessDBPSDriveInfo.Connection = conn;

            return accessDBPSDriveInfo;
        } // NewDrive

        /// <summary>
        /// Removes a drive from the provider.
        /// </summary>
        /// <param name="drive">The drive to remove.</param>
        /// <returns>The drive removed.</returns>
        protected override PSDriveInfo RemoveDrive(PSDriveInfo drive)
        {
            // check if drive object is null
            if (drive == null)
            {
                WriteError(new ErrorRecord(
                    new ArgumentNullException("drive"),
                    "NullDrive",
```

```
                    ErrorCategory.InvalidArgument,
                    drive)
                );

                return null;
            }

            // close ODBC connection to the drive
            AccessDBPSDriveInfo accessDBPSDriveInfo = drive as AccessDBPSDriveInfo;

            if (accessDBPSDriveInfo == null)
            {
                return null;
            }
            accessDBPSDriveInfo.Connection.Close();

            return accessDBPSDriveInfo;
        } // RemoveDrive

        #endregion Drive Manipulation

    } // AccessDBProvider

    #endregion AccessDBProvider

    #region AccessDBPSDriveInfo

    /// <summary>
    /// Any state associated with the drive should be held here.
    /// In this case, it's the connection to the database.
    /// </summary>
    internal class AccessDBPSDriveInfo : PSDriveInfo
    {
        private OdbcConnection connection;

        /// <summary>
        /// ODBC connection information.
        /// </summary>
        public OdbcConnection Connection
        {
            get { return connection; }
            set { connection = value; }
        }

        /// <summary>
        /// Constructor that takes one argument
        /// </summary>
        /// <param name="driveInfo">Drive provided by this provider</param>
        public AccessDBPSDriveInfo(PSDriveInfo driveInfo)
            : base(driveInfo)
        { }

    } // class AccessDBPSDriveInfo
```

```
    #endregion AccessDBPSDriveInfo
}
```

See Also

System.Management.Automation.Provider.ItemCmdletProvider

System.Management.Automation.Provider.ContainerCmdletProvider

System.Management.Automation.Provider.NavigationCmdletProvider

Designing Your Windows PowerShell Provider

Last updated on 05/20/2025

AccessDBProviderSample03

This sample shows how to overwrite the System.Management.Automation.Provider.ItemCmdletProvider.GetItem* and System.Management.Automation.Provider.ItemCmdletProvider.SetItem* methods to support calls to the `Get-Item` and `Set-Item` cmdlets. The provider class in this sample derives from the System.Management.Automation.Provider.ItemCmdletProvider class.

Demonstrates

> **Important**
>
> Your provider class will most likely derive from one of the following classes and possibly implement other provider interfaces:
>
> - System.Management.Automation.Provider.ItemCmdletProvider class.
> - System.Management.Automation.Provider.ContainerCmdletProvider class. See AccessDBProviderSample04.
> - System.Management.Automation.Provider.NavigationCmdletProvider class. See AccessDBProviderSample05.
>
> For more information about choosing which provider class to derive from based on provider features, see Designing Your Windows PowerShell Provider.

This sample demonstrates the following:

- Declaring the `CmdletProvider` attribute.
- Defining a provider class that derives from the System.Management.Automation.Provider.ItemCmdletProvider class.
- Overwriting the System.Management.Automation.Provider.DriveCmdletProvider.NewDrive* method to change the behavior of the `New-PSDrive` cmdlet, allowing the user to create new drives. (This sample does not show how to add dynamic parameters to the `New-PSDrive` cmdlet.)
- Overwriting the System.Management.Automation.Provider.DriveCmdletProvider.RemoveDrive* method to support removing existing drives.

- Overwriting the System.Management.Automation.Provider.ItemCmdletProvider.GetItem* method to change the behavior of the `Get-Item` cmdlet, allowing the user to retrieve items from the data store. (This sample does not show how to add dynamic parameters to the `Get-Item` cmdlet.)
- Overwriting the System.Management.Automation.Provider.ItemCmdletProvider.SetItem* method to change the behavior of the `Set-Item` cmdlet, allowing the user to update the items in the data store. (This sample does not show how to add dynamic parameters to the `Get-Item` cmdlet.)
- Overwriting the System.Management.Automation.Provider.ItemCmdletProvider.ItemExists* method to change the behavior of the `Test-Path` cmdlet. (This sample does not show how to add dynamic parameters to the `Test-Path` cmdlet.)
- Overwriting the System.Management.Automation.Provider.ItemCmdletProvider.IsValidPath* method to determine if the provided path is valid.

Example

This sample shows how to overwrite the methods needed to get and set items in a Microsoft Access data base.

C#

```
using System;
using System.IO;
using System.Data;
using System.Data.Odbc;
using System.Collections.ObjectModel;
using System.Text;
using System.Diagnostics;
using System.Text.RegularExpressions;
using System.Management.Automation;
using System.Management.Automation.Provider;
using System.ComponentModel;
using System.Globalization;

namespace Microsoft.Samples.PowerShell.Providers
{
   #region AccessDBProvider

    /// <summary>
    /// A PowerShell Provider which acts upon a access database.
    /// </summary>
    /// <remarks>
    /// This example implements the item overloads.
```

```
    /// </remarks>
   [CmdletProvider("AccessDB", ProviderCapabilities.None)]

   public class AccessDBProvider : ItemCmdletProvider
   {
      #region Drive Manipulation

       /// <summary>
       /// Create a new drive.  Create a connection to the database file and set
       /// the Connection property in the PSDriveInfo.
       /// </summary>
       /// <param name="drive">
       /// Information describing the drive to add.
       /// </param>
       /// <returns>The added drive.</returns>
       protected override PSDriveInfo NewDrive(PSDriveInfo drive)
       {
           // check if drive object is null
           if (drive == null)
           {
               WriteError(new ErrorRecord(
                   new ArgumentNullException("drive"),
                   "NullDrive",
                   ErrorCategory.InvalidArgument,
                   null)
               );

               return null;
           }

           // check if drive root is not null or empty
           // and if its an existing file
           if (String.IsNullOrEmpty(drive.Root) || (File.Exists(drive.Root) ==
false))
           {
               WriteError(new ErrorRecord(
                   new ArgumentException("drive.Root"),
                   "NoRoot",
                   ErrorCategory.InvalidArgument,
                   drive)
               );

               return null;
           }

           // create a new drive and create an ODBC connection to the new drive
           AccessDBPSDriveInfo accessDBPSDriveInfo = new
AccessDBPSDriveInfo(drive);

           OdbcConnectionStringBuilder builder = new OdbcConnectionStringBuilder();

           builder.Driver = "Microsoft Access Driver (*.mdb)";
           builder.Add("DBQ", drive.Root);

           OdbcConnection conn = new OdbcConnection(builder.ConnectionString);
```

```
            conn.Open();
            accessDBPSDriveInfo.Connection = conn;

            return accessDBPSDriveInfo;
        } // NewDrive

        /// <summary>
        /// Removes a drive from the provider.
        /// </summary>
        /// <param name="drive">The drive to remove.</param>
        /// <returns>The drive removed.</returns>
        protected override PSDriveInfo RemoveDrive(PSDriveInfo drive)
        {
            // check if drive object is null
            if (drive == null)
            {
                WriteError(new ErrorRecord(
                    new ArgumentNullException("drive"),
                    "NullDrive",
                    ErrorCategory.InvalidArgument,
                    drive)
                );

                return null;
            }

            // close ODBC connection to the drive
            AccessDBPSDriveInfo accessDBPSDriveInfo = drive as AccessDBPSDriveInfo;

            if (accessDBPSDriveInfo == null)
            {
                return null;
            }
            accessDBPSDriveInfo.Connection.Close();

            return accessDBPSDriveInfo;
        } // RemoveDrive

        #endregion Drive Manipulation

        #region Item Methods

    /// <summary>
    /// Retrieves an item using the specified path.
    /// </summary>
    /// <param name="path">The path to the item to return.</param>
    protected override void GetItem(string path)
    {
        // check if the path represented is a drive
        if (PathIsDrive(path))
        {
            WriteItemObject(this.PSDriveInfo, path, true);
            return;
        }// if (PathIsDrive...
```

```
        // Get table name and row information from the path and do
        // necessary actions
        string tableName;
        int rowNumber;

        PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

        if (type == PathType.Table)
        {
            DatabaseTableInfo table = GetTable(tableName);
            WriteItemObject(table, path, true);
        }
        else if (type == PathType.Row)
        {
            DatabaseRowInfo row = GetRow(tableName, rowNumber);
            WriteItemObject(row, path, false);
        }
        else
        {
            ThrowTerminatingInvalidPathException(path);
        }

    } // GetItem

    /// <summary>
    /// Set the content of a row of data specified by the supplied path
    /// parameter.
    /// </summary>
    /// <param name="path">Specifies the path to the row whose columns
    /// will be updated.</param>
    /// <param name="values">Comma separated string of values</param>
    protected override void SetItem(string path, object values)
    {
        // Get type, table name and row number from the path specified
        string tableName;
        int rowNumber;

        PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

        if (type != PathType.Row)
        {
            WriteError(new ErrorRecord(new NotSupportedException(
                  "SetNotSupported"), "",
               ErrorCategory.InvalidOperation, path));

            return;
        }

        // Get in-memory representation of table
        OdbcDataAdapter da = GetAdapterForTable(tableName);

        if (da == null)
        {
            return;
        }
```

```
            DataSet ds = GetDataSetForTable(da, tableName);
            DataTable table = GetDataTable(ds, tableName);

            if (rowNumber >= table.Rows.Count)
            {
                // The specified row number has to be available. If not
                // NewItem has to be used to add a new row
                throw new ArgumentException("Row specified is not available");
            } // if (rowNum...

            string[] colValues = (values as string).Split(',');

            // set the specified row
            DataRow row = table.Rows[rowNumber];

            for (int i = 0; i < colValues.Length; i++)
            {
                row[i] = colValues[i];
            }

            // Update the table
            if (ShouldProcess(path, "SetItem"))
            {
                da.Update(ds, tableName);
            }

        } // SetItem

        /// <summary>
        /// Test to see if the specified item exists.
        /// </summary>
        /// <param name="path">The path to the item to verify.</param>
        /// <returns>True if the item is found.</returns>
        protected override bool ItemExists(string path)
        {
            // check if the path represented is a drive
            if (PathIsDrive(path))
            {
                return true;
            }

            // Obtain type, table name and row number from path
            string tableName;
            int rowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            DatabaseTableInfo table = GetTable(tableName);

            if (type == PathType.Table)
            {
                // if specified path represents a table then DatabaseTableInfo
                // object for the same should exist
                if (table != null)
                {
```

```
                return true;
            }
        }
        else if (type == PathType.Row)
        {
            // if specified path represents a row then DatabaseTableInfo should
            // exist for the table and then specified row number must be within
            // the maximum row count in the table
            if (table != null && rowNumber < table.RowCount)
            {
                return true;
            }
        }

        return false;

    } // ItemExists

    /// <summary>
    /// Test to see if the specified path is syntactically valid.
    /// </summary>
    /// <param name="path">The path to validate.</param>
    /// <returns>True if the specified path is valid.</returns>
    protected override bool IsValidPath(string path)
    {
        bool result = true;

        // check if the path is null or empty
        if (String.IsNullOrEmpty(path))
        {
            result = false;
        }

        // convert all separators in the path to a uniform one
        path = NormalizePath(path);

        // split the path into individual chunks
        string[] pathChunks = path.Split(pathSeparator.ToCharArray());

        foreach (string pathChunk in pathChunks)
        {
            if (pathChunk.Length == 0)
            {
                result = false;
            }
        }
        return result;
    } // IsValidPath

    #endregion Item Overloads

#region Helper Methods

/// <summary>
/// Checks if a given path is actually a drive name.
```

```
        /// </summary>
        /// <param name="path">The path to check.</param>
        /// <returns>
        /// True if the path given represents a drive, false otherwise.
        /// </returns>
        private bool PathIsDrive(string path)
        {
            // Remove the drive name and first path separator.  If the
            // path is reduced to nothing, it is a drive. Also if its
            // just a drive then there wont be any path separators
            if (String.IsNullOrEmpty(
                        path.Replace(this.PSDriveInfo.Root, "")) ||
                String.IsNullOrEmpty(
                        path.Replace(this.PSDriveInfo.Root + pathSeparator, ""))

               )
            {
                return true;
            }
            else
            {
                return false;
            }
        } // PathIsDrive

        /// <summary>
        /// Breaks up the path into individual elements.
        /// </summary>
        /// <param name="path">The path to split.</param>
        /// <returns>An array of path segments.</returns>
        private string[] ChunkPath(string path)
        {
            // Normalize the path before splitting
            string normalPath = NormalizePath(path);

            // Return the path with the drive name and first path
            // separator character removed, split by the path separator.
            string pathNoDrive = normalPath.Replace(this.PSDriveInfo.Root
                                        + pathSeparator, "");

            return pathNoDrive.Split(pathSeparator.ToCharArray());
        } // ChunkPath

        /// <summary>
        /// Adapts the path, making sure the correct path separator
        /// character is used.
        /// </summary>
        /// <param name="path"></param>
        /// <returns></returns>
        private string NormalizePath(string path)
        {
            string result = path;

            if (!String.IsNullOrEmpty(path))
            {
```

```
            result = path.Replace("/", pathSeparator);
        }

        return result;
    } // NormalizePath

    /// <summary>
    /// Chunks the path and returns the table name and the row number
    /// from the path
    /// </summary>
    /// <param name="path">Path to chunk and obtain information</param>
    /// <param name="tableName">Name of the table as represented in the
    /// path</param>
    /// <param name="rowNumber">Row number obtained from the path</param>
    /// <returns>what the path represents</returns>
    private PathType GetNamesFromPath(string path, out string tableName, out int
rowNumber)
    {
        PathType retVal = PathType.Invalid;
        rowNumber = -1;
        tableName = null;

        // Check if the path specified is a drive
        if (PathIsDrive(path))
        {
            return PathType.Database;
        }

        // chunk the path into parts
        string[] pathChunks = ChunkPath(path);

        switch (pathChunks.Length)
        {
            case 1:
                {
                    string name = pathChunks[0];

                    if (TableNameIsValid(name))
                    {
                        tableName = name;
                        retVal = PathType.Table;
                    }
                }
                break;

            case 2:
                {
                    string name = pathChunks[0];

                    if (TableNameIsValid(name))
                    {
                        tableName = name;
                    }

                    int number = SafeConvertRowNumber(pathChunks[1]);
```

```
                    if (number >= 0)
                    {
                        rowNumber = number;
                        retVal = PathType.Row;
                    }
                    else
                    {
                        WriteError(new ErrorRecord(
                            new ArgumentException("Row number is not valid"),
                            "RowNumberNotValid",
                            ErrorCategory.InvalidArgument,
                            path));
                    }
                }
                break;

            default:
                {
                    WriteError(new ErrorRecord(
                        new ArgumentException("The path supplied has too many
segments"),
                        "PathNotValid",
                        ErrorCategory.InvalidArgument,
                        path));
                }
                break;
        } // switch(pathChunks...

        return retVal;
    } // GetNamesFromPath

    /// <summary>
    /// Throws an argument exception stating that the specified path does
    /// not represent either a table or a row
    /// </summary>
    /// <param name="path">path which is invalid</param>
    private void ThrowTerminatingInvalidPathException(string path)
    {
        StringBuilder message = new StringBuilder("Path must represent either a
table or a row :");
        message.Append(path);

        throw new ArgumentException(message.ToString());
    }

    /// <summary>
    /// Retrieve the list of tables from the database.
    /// </summary>
    /// <returns>
    /// Collection of DatabaseTableInfo objects, each object representing
    /// information about one database table
    /// </returns>
    private Collection<DatabaseTableInfo> GetTables()
    {
```

```
        Collection<DatabaseTableInfo> results =
                new Collection<DatabaseTableInfo>();

        // using ODBC connection to the database and get the schema of tables
        AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;

        if (di == null)
        {
            return null;
        }

        OdbcConnection connection = di.Connection;
        DataTable dt = connection.GetSchema("Tables");
        int count;

        // iterate through all rows in the schema and create DatabaseTableInfo
        // objects which represents a table
        foreach (DataRow dr in dt.Rows)
        {
            String tableName = dr["TABLE_NAME"] as String;
            DataColumnCollection columns = null;

            // find the number of rows in the table
            try
            {
                String cmd = "Select count(*) from \"" + tableName + "\"";
                OdbcCommand command = new OdbcCommand(cmd, connection);

                count = (Int32)command.ExecuteScalar();
            }
            catch
            {
                count = 0;
            }

            // create DatabaseTableInfo object representing the table
            DatabaseTableInfo table =
                    new DatabaseTableInfo(dr, tableName, count, columns);

            results.Add(table);
        } // foreach (DataRow...

        return results;
    } // GetTables

    /// <summary>
    /// Return row information from a specified table.
    /// </summary>
    /// <param name="tableName">The name of the database table from
    /// which to retrieve rows.</param>
    /// <returns>Collection of row information objects.</returns>
    private Collection<DatabaseRowInfo> GetRows(string tableName)
    {
        Collection<DatabaseRowInfo> results =
                    new Collection<DatabaseRowInfo>();
```

```
            // Obtain rows in the table and add it to the collection
            try
            {
                OdbcDataAdapter da = GetAdapterForTable(tableName);

                if (da == null)
                {
                    return null;
                }

                DataSet ds = GetDataSetForTable(da, tableName);
                DataTable table = GetDataTable(ds, tableName);

                int i = 0;
                foreach (DataRow row in table.Rows)
                {
                    results.Add(new DatabaseRowInfo(row,
i.ToString(CultureInfo.CurrentCulture)));
                    i++;
                } // foreach (DataRow...
            }
            catch (Exception e)
            {
                WriteError(new ErrorRecord(e, "CannotAccessSpecifiedRows",
                    ErrorCategory.InvalidOperation, tableName));
            }

            return results;

        } // GetRows

        /// <summary>
        /// Retrieve information about a single table.
        /// </summary>
        /// <param name="tableName">The table for which to retrieve
        /// data.</param>
        /// <returns>Table information.</returns>
        private DatabaseTableInfo GetTable(string tableName)
        {
            foreach (DatabaseTableInfo table in GetTables())
            {
                if (String.Equals(tableName, table.Name,
StringComparison.OrdinalIgnoreCase))
                {
                    return table;
                }
            }

            return null;
        } // GetTable

        /// <summary>
        /// Obtain a data adapter for the specified Table
        /// </summary>
```

```
        /// <param name="tableName">Name of the table to obtain the
        /// adapter for</param>
        /// <returns>Adapter object for the specified table</returns>
        /// <remarks>An adapter serves as a bridge between a DataSet (in memory
        /// representation of table) and the data source</remarks>
        private OdbcDataAdapter GetAdapterForTable(string tableName)
        {
            OdbcDataAdapter da = null;
            AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;

            if (di == null || !TableNameIsValid(tableName) ||
!TableIsPresent(tableName))
            {
                return null;
            }

            OdbcConnection connection = di.Connection;

            try
            {
                // Create a odbc data adpater. This can be sued to update the
                // data source with the records that will be created here
                // using data sets
                string sql = "Select * from " + tableName;
                da = new OdbcDataAdapter(new OdbcCommand(sql, connection));

                // Create a odbc command builder object. This will create sql
                // commands automatically for a single table, thus
                // eliminating the need to create new sql statements for
                // every operation to be done.
                OdbcCommandBuilder cmd = new OdbcCommandBuilder(da);

                // Open the connection if its not already open
                if (connection.State != ConnectionState.Open)
                {
                    connection.Open();
                }
            }
            catch (Exception e)
            {
                WriteError(new ErrorRecord(e, "CannotAccessSpecifiedTable",
                  ErrorCategory.InvalidOperation, tableName));
            }

            return da;
        } // GetAdapterForTable

        /// <summary>
        /// Gets the DataSet (in memory representation) for the table
        /// for the specified adapter
        /// </summary>
        /// <param name="adapter">Adapter to be used for obtaining
        /// the table</param>
        /// <param name="tableName">Name of the table for which a
        /// DataSet is required</param>
```

```
        /// <returns>The DataSet with the filled in schema</returns>
        private DataSet GetDataSetForTable(OdbcDataAdapter adapter, string tableName)
        {
            Debug.Assert(adapter != null);

            // Create a dataset object which will provide an in-memory
            // representation of the data being worked upon in the
            // data source.
            DataSet ds = new DataSet();

            // Create a table named "Table" which will contain the same
            // schema as in the data source.
            //adapter.FillSchema(ds, SchemaType.Source);
            adapter.Fill(ds, tableName);
            ds.Locale = CultureInfo.InvariantCulture;

            return ds;
        } //GetDataSetForTable

        /// <summary>
        /// Get the DataTable object which can be used to operate on
        /// for the specified table in the data source
        /// </summary>
        /// <param name="ds">DataSet object which contains the tables
        /// schema</param>
        /// <param name="tableName">Name of the table</param>
        /// <returns>Corresponding DataTable object representing
        /// the table</returns>
        ///
        private DataTable GetDataTable(DataSet ds, string tableName)
        {
            Debug.Assert(ds != null);
            Debug.Assert(tableName != null);

            DataTable table = ds.Tables[tableName];
            table.Locale = CultureInfo.InvariantCulture;

            return table;
        } // GetDataTable

        /// <summary>
        /// Retrieves a single row from the named table.
        /// </summary>
        /// <param name="tableName">The table that contains the
        /// numbered row.</param>
        /// <param name="row">The index of the row to return.</param>
        /// <returns>The specified table row.</returns>
        private DatabaseRowInfo GetRow(string tableName, int row)
        {
            Collection<DatabaseRowInfo> di = GetRows(tableName);

            // if the row is invalid write an appropriate error else return the
            // corresponding row information
            if (row < di.Count && row >= 0)
            {
```

```
                return di[row];
            }
            else
            {
                WriteError(new ErrorRecord(
                   new ItemNotFoundException(),
                   "RowNotFound",
                   ErrorCategory.ObjectNotFound,
                   row.ToString(CultureInfo.CurrentCulture))
                );
            }

            return null;
        } // GetRow

        /// <summary>
        /// Method to safely convert a string representation of a row number
        /// into its Int32 equivalent
        /// </summary>
        /// <param name="rowNumberAsStr">String representation of the row
        /// number</param>
        /// <remarks>If there is an exception, -1 is returned</remarks>
        private int SafeConvertRowNumber(string rowNumberAsStr)
        {
            int rowNumber = -1;
            try
            {
                rowNumber = Convert.ToInt32(rowNumberAsStr,
CultureInfo.CurrentCulture);
            }
            catch (FormatException fe)
            {
                WriteError(new ErrorRecord(fe, "RowStringFormatNotValid",
                    ErrorCategory.InvalidData, rowNumberAsStr));
            }
            catch (OverflowException oe)
            {
                WriteError(new ErrorRecord(oe, "RowStringConversionToNumberFailed",
                    ErrorCategory.InvalidData, rowNumberAsStr));
            }

            return rowNumber;
        } // SafeConvertRowNumber

        /// <summary>
        /// Check if a table name is valid
        /// </summary>
        /// <param name="tableName">Table name to validate</param>
        /// <remarks>Helps to check for SQL injection attacks</remarks>
        private bool TableNameIsValid(string tableName)
        {
            Regex exp = new Regex(pattern, RegexOptions.Compiled |
RegexOptions.IgnoreCase);

            if (exp.IsMatch(tableName))
```

```
            {
                return true;
            }
            WriteError(new ErrorRecord(
                new ArgumentException("Table name not valid"), "TableNameNotValid",
                     ErrorCategory.InvalidArgument, tableName));
            return false;
        } // TableNameIsValid

        /// <summary>
        /// Checks to see if the specified table is present in the
        /// database
        /// </summary>
        /// <param name="tableName">Name of the table to check</param>
        /// <returns>true, if table is present, false otherwise</returns>
        private bool TableIsPresent(string tableName)
        {
            // using ODBC connection to the database and get the schema of tables
            AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;
            if (di == null)
            {
                return false;
            }

            OdbcConnection connection = di.Connection;
            DataTable dt = connection.GetSchema("Tables");

            // check if the specified tableName is available
            // in the list of tables present in the database
            foreach (DataRow dr in dt.Rows)
            {
                string name = dr["TABLE_NAME"] as string;
                if (name.Equals(tableName, StringComparison.OrdinalIgnoreCase))
                {
                    return true;
                }
            }

            WriteError(new ErrorRecord(
                new ArgumentException("Specified Table is not present in database"),
"TableNotAvailable",
                     ErrorCategory.InvalidArgument, tableName));

            return false;
        }// TableIsPresent

        #endregion Helper Methods

        #region Private Properties

        private string pathSeparator = "\\";
        private static string pattern = @"^[a-z]+[0-9]*_*$";

        private enum PathType { Database, Table, Row, Invalid };
```

```
    #endregion Private Properties
}

 #endregion AccessDBProvider

 #region Helper Classes

#region AccessDBPSDriveInfo

/// <summary>
 /// Any state associated with the drive should be held here.
 /// In this case, it's the connection to the database.
 /// </summary>
 internal class AccessDBPSDriveInfo : PSDriveInfo
 {
     private OdbcConnection connection;

     /// <summary>
     /// ODBC connection information.
     /// </summary>
     public OdbcConnection Connection
     {
         get { return connection; }
         set { connection = value; }
     }

     /// <summary>
     /// Constructor that takes one argument
     /// </summary>
     /// <param name="driveInfo">Drive provided by this provider</param>
     public AccessDBPSDriveInfo(PSDriveInfo driveInfo)
         : base(driveInfo)
     { }

 } // class AccessDBPSDriveInfo

 #endregion AccessDBPSDriveInfo

 #region DatabaseTableInfo

 /// <summary>
 /// Contains information specific to the database table.
 /// Similar to the DirectoryInfo class.
 /// </summary>
 public class DatabaseTableInfo
 {
     /// <summary>
     /// Row from the "tables" schema
     /// </summary>
     public DataRow Data
     {
         get
         {
             return data;
         }
```

```
            set
            {
                data = value;
            }
        }
        private DataRow data;

        /// <summary>
        /// The table name.
        /// </summary>
        public string Name
        {
            get
            {
                return name;
            }
            set
            {
                name = value;
            }
        }
        private String name;

        /// <summary>
        /// The number of rows in the table.
        /// </summary>
        public int RowCount
        {
            get
            {
                return rowCount;
            }
            set
            {
                rowCount = value;
            }
        }
        private int rowCount;

        /// <summary>
        /// The column definitions for the table.
        /// </summary>
        public DataColumnCollection Columns
        {
            get
            {
                return columns;
            }
            set
            {
                columns = value;
            }
        }
        private DataColumnCollection columns;
```

```
        /// <summary>
        /// Constructor.
        /// </summary>
        /// <param name="row">The row definition.</param>
        /// <param name="name">The table name.</param>
        /// <param name="rowCount">The number of rows in the table.</param>
        /// <param name="columns">Information on the column tables.</param>
        public DatabaseTableInfo(DataRow row, string name, int rowCount,
                        DataColumnCollection columns)
        {
            Name = name;
            Data = row;
            RowCount = rowCount;
            Columns = columns;
        } // DatabaseTableInfo
    } // class DatabaseTableInfo

    #endregion DatabaseTableInfo

    #region DatabaseRowInfo

    /// <summary>
    /// Contains information specific to an individual table row.
    /// Analogous to the FileInfo class.
    /// </summary>
    public class DatabaseRowInfo
    {
        /// <summary>
        /// Row data information.
        /// </summary>
        public DataRow Data
        {
            get
            {
                return data;
            }
            set
            {
                data = value;
            }
        }
        private DataRow data;

        /// <summary>
        /// The row index.
        /// </summary>
        public string RowNumber
        {
            get
            {
                return rowNumber;
            }
            set
            {
                rowNumber = value;
```

```
            }
        }
        private string rowNumber;

        /// <summary>
        /// Constructor.
        /// </summary>
        /// <param name="row">The row information.</param>
        /// <param name="name">The row index.</param>
        public DatabaseRowInfo(DataRow row, string name)
        {
            RowNumber = name;
            Data = row;
        } // DatabaseRowInfo
    } // class DatabaseRowInfo

    #endregion DatabaseRowInfo

    #endregion Helper Classes
}
```

See Also

System.Management.Automation.Provider.ItemCmdletProvider

System.Management.Automation.Provider.ContainerCmdletProvider

System.Management.Automation.Provider.NavigationCmdletProvider

Designing Your Windows PowerShell Provider

Last updated on 05/20/2025

AccessDBProviderSample04

This sample shows how to overwrite container methods to support calls to the `Copy-Item`, `Get-ChildItem`, `New-Item`, and `Remove-Item` cmdlets. These methods should be implemented when the data store contains items that are containers. A container is a group of child items under a common parent item. The provider class in this sample derives from the System.Management.Automation.Provider.ContainerCmdletProvider class.

Demonstrates

> **Important**
>
> Your provider class will most likely derive from the **System.Management.Automation.Provider.NavigationCmdletProvider**

This sample demonstrates the following:

- Declaring the `CmdletProvider` attribute.
- Defining a provider class that derives from the System.Management.Automation.Provider.ContainerCmdletProvider class.
- Overwriting the System.Management.Automation.Provider.ContainerCmdletProvider.CopyItem method to change the behavior of the `Copy-Item` cmdlet which allows the user to copy items from one location to another. (This sample does not show how to add dynamic parameters to the `Copy-Item` cmdlet.)
- Overwriting the System.Management.Automation.Provider.ContainerCmdletProvider.GetChildItems* method to change the behavior of the Get-ChildItems cmdlet, which allows the user to retrieve the child items of the parent item. (This sample does not show how to add dynamic parameters to the Get-ChildItems cmdlet.)
- Overwriting the System.Management.Automation.Provider.ContainerCmdletProvider.GetChildNames* method to change the behavior of the Get-ChildItems cmdlet when the `Name` parameter of the cmdlet is specified.
- Overwriting the System.Management.Automation.Provider.ContainerCmdletProvider.NewItem* method to

change the behavior of the `New-Item` cmdlet, which allows the user to add items to the data store. (This sample does not show how to add dynamic parameters to the `New-Item` cmdlet.)

- Overwriting the System.Management.Automation.Provider.ContainerCmdletProvider.RemoveItem* method to change the behavior of the `Remove-Item` cmdlet. (This sample does not show how to add dynamic parameters to the `Remove-Item` cmdlet.)

Example

This sample shows how to overwrite the methods needed to copy, create, and remove items, as well as methods for getting the child items of a parent item.

C#

```
using System;
using System.IO;
using System.Data;
using System.Data.Odbc;
using System.Data.OleDb;
using System.Diagnostics;
using System.Collections.ObjectModel;
using System.Text;
using System.Text.RegularExpressions;
using System.Management.Automation;
using System.Management.Automation.Provider;
using System.ComponentModel;
using System.Globalization;

namespace Microsoft.Samples.PowerShell.Providers
{
   #region AccessDBProvider

    /// <summary>
   /// A PowerShell Provider which acts upon an Access database
   /// </summary>
   /// <remarks>
   /// This example implements the container overloads</remarks>
   [CmdletProvider("AccessDB", ProviderCapabilities.None)]
   public class AccessDBProvider : ContainerCmdletProvider
   {

       #region Drive Manipulation

       /// <summary>
       /// Create a new drive.  Create a connection to the database file and set
       /// the Connection property in the PSDriveInfo.
       /// </summary>
       /// <param name="drive">
```

```
        /// Information describing the drive to add.
        /// </param>
        /// <returns>The added drive.</returns>
        protected override PSDriveInfo NewDrive(PSDriveInfo drive)
        {
            // check if drive object is null
            if (drive == null)
            {
                WriteError(new ErrorRecord(
                    new ArgumentNullException("drive"),
                    "NullDrive",
                    ErrorCategory.InvalidArgument,
                    null)
                );

                return null;
            }

            // check if drive root is not null or empty
            // and if its an existing file
            if (String.IsNullOrEmpty(drive.Root) || (File.Exists(drive.Root) ==
false))
            {
                WriteError(new ErrorRecord(
                    new ArgumentException("drive.Root"),
                    "NoRoot",
                    ErrorCategory.InvalidArgument,
                    drive)
                );

                return null;
            }


            // create a new drive and create an ODBC connection to the new drive
            AccessDBPSDriveInfo accessDBPSDriveInfo = new
AccessDBPSDriveInfo(drive);

            OdbcConnectionStringBuilder builder = new OdbcConnectionStringBuilder();

            builder.Driver = "Microsoft Access Driver (*.mdb)";
            builder.Add("DBQ", drive.Root);

            OdbcConnection conn = new OdbcConnection(builder.ConnectionString);
            conn.Open();
            accessDBPSDriveInfo.Connection = conn;

            return accessDBPSDriveInfo;
        } // NewDrive

        /// <summary>
        /// Removes a drive from the provider.
        /// </summary>
        /// <param name="drive">The drive to remove.</param>
        /// <returns>The drive removed.</returns>
```

```
        protected override PSDriveInfo RemoveDrive(PSDriveInfo drive)
        {
            // check if drive object is null
            if (drive == null)
            {
                WriteError(new ErrorRecord(
                    new ArgumentNullException("drive"),
                    "NullDrive",
                    ErrorCategory.InvalidArgument,
                    drive)
                );

                return null;
            }

            // close ODBC connection to the drive
            AccessDBPSDriveInfo accessDBPSDriveInfo = drive as AccessDBPSDriveInfo;

            if (accessDBPSDriveInfo == null)
            {
                return null;
            }
            accessDBPSDriveInfo.Connection.Close();

            return accessDBPSDriveInfo;
        } // RemoveDrive

        #endregion Drive Manipulation

        #region Item Methods

        /// <summary>
        /// Retrieves an item using the specified path.
        /// </summary>
        /// <param name="path">The path to the item to return.</param>
        protected override void GetItem(string path)
        {
            // check if the path represented is a drive
            if (PathIsDrive(path))
            {
                WriteItemObject(this.PSDriveInfo, path, true);
                return;
            }// if (PathIsDrive...

            // Get table name and row information from the path and do
            // necessary actions
            string tableName;
            int rowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            if (type == PathType.Table)
            {
                DatabaseTableInfo table = GetTable(tableName);
                WriteItemObject(table, path, true);
```

```
        }
        else if (type == PathType.Row)
        {
            DatabaseRowInfo row = GetRow(tableName, rowNumber);
            WriteItemObject(row, path, false);
        }
        else
        {
            ThrowTerminatingInvalidPathException(path);
        }

    } // GetItem

    /// <summary>
    /// Set the content of a row of data specified by the supplied path
    /// parameter.
    /// </summary>
    /// <param name="path">Specifies the path to the row whose columns
    /// will be updated.</param>
    /// <param name="values">Comma separated string of values</param>
    protected override void SetItem(string path, object values)
    {
        // Get type, table name and row number from the path specified
        string tableName;
        int rowNumber;

        PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

        if (type != PathType.Row)
        {
            WriteError(new ErrorRecord(new NotSupportedException(
                  "SetNotSupported"), "",
               ErrorCategory.InvalidOperation, path));

            return;
        }

        // Get in-memory representation of table
        OdbcDataAdapter da = GetAdapterForTable(tableName);

        if (da == null)
        {
            return;
        }
        DataSet ds = GetDataSetForTable(da, tableName);
        DataTable table = GetDataTable(ds, tableName);

        if (rowNumber >= table.Rows.Count)
        {
            // The specified row number has to be available. If not
            // NewItem has to be used to add a new row
            throw new ArgumentException("Row specified is not available");
        } // if (rowNum...

        string[] colValues = (values as string).Split(',');
```

```
        // set the specified row
        DataRow row = table.Rows[rowNumber];

        for (int i = 0; i < colValues.Length; i++)
        {
            row[i] = colValues[i];
        }

        // Update the table
        if (ShouldProcess(path, "SetItem"))
        {
            da.Update(ds, tableName);
        }

    } // SetItem

    /// <summary>
    /// Test to see if the specified item exists.
    /// </summary>
    /// <param name="path">The path to the item to verify.</param>
    /// <returns>True if the item is found.</returns>
    protected override bool ItemExists(string path)
    {
        // check if the path represented is a drive
        if (PathIsDrive(path))
        {
            return true;
        }

        // Obtain type, table name and row number from path
        string tableName;
        int rowNumber;

        PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

        DatabaseTableInfo table = GetTable(tableName);

        if (type == PathType.Table)
        {
            // if specified path represents a table then DatabaseTableInfo
            // object for the same should exist
            if (table != null)
            {
                return true;
            }
        }
        else if (type == PathType.Row)
        {
            // if specified path represents a row then DatabaseTableInfo should
            // exist for the table and then specified row number must be within
            // the maximum row count in the table
            if (table != null && rowNumber < table.RowCount)
            {
                return true;
```

```
            }
        }

        return false;

    } // ItemExists

    /// <summary>
    /// Test to see if the specified path is syntactically valid.
    /// </summary>
    /// <param name="path">The path to validate.</param>
    /// <returns>True if the specified path is valid.</returns>
    protected override bool IsValidPath(string path)
    {
        bool result = true;

        // check if the path is null or empty
        if (String.IsNullOrEmpty(path))
        {
            result = false;
        }

        // convert all separators in the path to a uniform one
        path = NormalizePath(path);

        // split the path into individual chunks
        string[] pathChunks = path.Split(pathSeparator.ToCharArray());

        foreach (string pathChunk in pathChunks)
        {
            if (pathChunk.Length == 0)
            {
                result = false;
            }
        }
        return result;
    } // IsValidPath

    #endregion Item Overloads

    #region Container Overloads

    /// <summary>
    /// Return either the tables in the database or the datarows
    /// </summary>
    /// <param name="path">The path to the parent</param>
    /// <param name="recurse">True to return all child items recursively.
    /// </param>
    protected override void GetChildItems(string path, bool recurse)
    {
        // If path represented is a drive then the children in the path are
        // tables. Hence all tables in the drive represented will have to be
        // returned
        if (PathIsDrive(path))
        {
```

```
                foreach (DatabaseTableInfo table in GetTables())
                {
                    WriteItemObject(table, path, true);

                    // if the specified item exists and recurse has been set then
                    // all child items within it have to be obtained as well
                    if (ItemExists(path) && recurse)
                    {
                        GetChildItems(path + pathSeparator + table.Name, recurse);
                    }
                } // foreach (DatabaseTableInfo...
            } // if (PathIsDrive...
            else
            {
                // Get the table name, row number and type of path from the
                // path specified
                string tableName;
                int rowNumber;

                PathType type = GetNamesFromPath(path, out tableName, out
rowNumber);

                if (type == PathType.Table)
                {
                    // Obtain all the rows within the table
                    foreach (DatabaseRowInfo row in GetRows(tableName))
                    {
                        WriteItemObject(row, path + pathSeparator + row.RowNumber,
                                false);
                    } // foreach (DatabaseRowInfo...
                }
                else if (type == PathType.Row)
                {
                    // In this case the user has directly specified a row, hence
                    // just give that particular row
                    DatabaseRowInfo row = GetRow(tableName, rowNumber);
                    WriteItemObject(row, path + pathSeparator + row.RowNumber,
                                false);
                }
                else
                {
                    // In this case, the path specified is not valid
                    ThrowTerminatingInvalidPathException(path);
                }
            } // else
        } // GetChildItems

        /// <summary>
        /// Return the names of all child items.
        /// </summary>
        /// <param name="path">The root path.</param>
        /// <param name="returnContainers">Not used.</param>
        protected override void GetChildNames(string path,
                                    ReturnContainers returnContainers)
        {
```

```
            // If the path represented is a drive, then the child items are
            // tables. get the names of all the tables in the drive.
            if (PathIsDrive(path))
            {
                foreach (DatabaseTableInfo table in GetTables())
                {
                    WriteItemObject(table.Name, path, true);
                } // foreach (DatabaseTableInfo...
            } // if (PathIsDrive...
            else
            {
                // Get type, table name and row number from path specified
                string tableName;
                int rowNumber;

                PathType type = GetNamesFromPath(path, out tableName, out
rowNumber);

                if (type == PathType.Table)
                {
                    // Get all the rows in the table and then write out the
                    // row numbers.
                    foreach (DatabaseRowInfo row in GetRows(tableName))
                    {
                        WriteItemObject(row.RowNumber, path, false);
                    } // foreach (DatabaseRowInfo...
                }
                else if (type == PathType.Row)
                {
                    // In this case the user has directly specified a row, hence
                    // just give that particular row
                    DatabaseRowInfo row = GetRow(tableName, rowNumber);

                    WriteItemObject(row.RowNumber, path, false);
                }
                else
                {
                    ThrowTerminatingInvalidPathException(path);
                }
            } // else
        } // GetChildNames

        /// <summary>
        /// Determines if the specified path has child items.
        /// </summary>
        /// <param name="path">The path to examine.</param>
        /// <returns>
        /// True if the specified path has child items.
        /// </returns>
        protected override bool HasChildItems(string path)
        {
            if (PathIsDrive(path))
            {
                return true;
            }
```

```
            return (ChunkPath(path).Length == 1);
        } // HasChildItems

        /// <summary>
        /// Creates a new item at the specified path.
        /// </summary>
        ///
        /// <param name="path">
        /// The path to the new item.
        /// </param>
        ///
        /// <param name="type">
        /// Type for the object to create. "Table" for creating a new table and
        /// "Row" for creating a new row in a table.
        /// </param>
        ///
        /// <param name="newItemValue">
        /// Object for creating new instance of a type at the specified path. For
        /// creating a "Table" the object parameter is ignored and for creating
        /// a "Row" the object must be of type string which will contain comma
        /// separated values of the rows to insert.
        /// </param>
        protected override void NewItem(string path, string type,
                                        object newItemValue)
        {
            string tableName;
            int rowNumber;

            PathType pt = GetNamesFromPath(path, out tableName, out rowNumber);

            if (pt == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(path);
            }

            // Check if type is either "table" or "row", if not throw an
            // exception
            if (!String.Equals(type, "table", StringComparison.OrdinalIgnoreCase)
                && !String.Equals(type, "row", StringComparison.OrdinalIgnoreCase))
            {
                WriteError(new ErrorRecord
                                    (new ArgumentException("Type must be either a
table or row"),
                                        "CannotCreateSpecifiedObject",
                                           ErrorCategory.InvalidArgument,
                                                path
                                     )
                           );

                throw new ArgumentException("This provider can only create items of
type \"table\" or \"row\"");
            }

            // Path type is the type of path of the container. So if a drive
```

```
            // is specified, then a table can be created under it and if a table
            // is specified, then a row can be created under it. For the sake of
            // completeness, if a row is specified, then if the row specified by
            // the path does not exist, a new row is created. However, the row
            // number may not match as the row numbers only get incremented based
            // on the number of rows

            if (PathIsDrive(path))
            {
                if (String.Equals(type, "table",
StringComparison.OrdinalIgnoreCase))
                {
                    // Execute command using ODBC connection to create a table
                    try
                    {
                        // create the table using an sql statement
                        string newTableName = newItemValue.ToString();

                        if (!TableNameIsValid(newTableName))
                        {
                            return;
                        }
                        string sql = "create table " + newTableName
                                              + " (ID INT)";

                        // Create the table using the Odbc connection from the
                        // drive.
                        AccessDBPSDriveInfo di = this.PSDriveInfo as
AccessDBPSDriveInfo;

                        if (di == null)
                        {
                            return;
                        }
                        OdbcConnection connection = di.Connection;

                        if (ShouldProcess(newTableName, "create"))
                        {
                            OdbcCommand cmd = new OdbcCommand(sql, connection);
                            cmd.ExecuteScalar();
                        }
                    }
                    catch (Exception ex)
                    {
                        WriteError(new ErrorRecord(ex, "CannotCreateSpecifiedTable",
                                  ErrorCategory.InvalidOperation, path)
                                  );
                    }
                } // if (String...
                else if (String.Equals(type, "row",
StringComparison.OrdinalIgnoreCase))
                {
                    throw new
                        ArgumentException("A row cannot be created under a database,
specify a path that represents a Table");
```

```
                }
            }// if (PathIsDrive...
            else
            {
                if (String.Equals(type, "table",
StringComparison.OrdinalIgnoreCase))
                {
                    if (rowNumber < 0)
                    {
                        throw new
                            ArgumentException("A table cannot be created within
another table, specify a path that represents a database");
                    }
                    else
                    {
                        throw new
                            ArgumentException("A table cannot be created inside a
row, specify a path that represents a database");
                    }
                } //if (String.Equals....
                // if path specified is a row, create a new row
                else if (String.Equals(type, "row",
StringComparison.OrdinalIgnoreCase))
                {
                    // The user is required to specify the values to be inserted
                    // into the table in a single string separated by commas
                    string value = newItemValue as string;

                    if (String.IsNullOrEmpty(value))
                    {
                        throw new
                            ArgumentException("Value argument must have comma
separated values of each column in a row");
                    }
                    string[] rowValues = value.Split(',');

                    OdbcDataAdapter da = GetAdapterForTable(tableName);

                    if (da == null)
                    {
                        return;
                    }

                    DataSet ds = GetDataSetForTable(da, tableName);
                    DataTable table = GetDataTable(ds, tableName);

                    if (rowValues.Length != table.Columns.Count)
                    {
                        string message =
                             String.Format(CultureInfo.CurrentCulture,
                                           "The table has {0} columns and the
value specified must have so many comma separated values",
                                                table.Columns.Count);

                        throw new ArgumentException(message);
```

```
                    }

                    if (!Force && (rowNumber >=0 && rowNumber < table.Rows.Count))
                    {
                        string message = String.Format(CultureInfo.CurrentCulture,
                                                       "The row {0} already
exists. To create a new row specify row number as {1}, or specify path to a table,
or use the -Force parameter",
                                                                rowNumber,
table.Rows.Count);

                        throw new ArgumentException(message);
                    }

                    if (rowNumber > table.Rows.Count)
                    {
                        string message = String.Format(CultureInfo.CurrentCulture,
                                                "To create a new row specify row number
as {0}, or specify path to a table",
                                                    table.Rows.Count);

                        throw new ArgumentException(message);
                    }

                    // Create a new row and update the row with the input
                    // provided by the user
                    DataRow row = table.NewRow();
                    for (int i = 0; i < rowValues.Length; i++)
                    {
                        row[i] = rowValues[i];
                    }
                    table.Rows.Add(row);

                    if (ShouldProcess(tableName, "update rows"))
                    {
                        // Update the table from memory back to the data source
                        da.Update(ds, tableName);
                    }

                }// else if (String...
            }// else ...

        } // NewItem

        /// <summary>
        /// Copies an item at the specified path to the location specified
        /// </summary>
        ///
        /// <param name="path">
        /// Path of the item to copy
        /// </param>
        ///
        /// <param name="copyPath">
        /// Path of the item to copy to
        /// </param>
```

```
        ///
        /// <param name="recurse">
        /// Tells the provider to recurse subcontainers when copying
        /// </param>
        ///
        protected override void CopyItem(string path, string copyPath, bool recurse)
        {
            string tableName, copyTableName;
            int rowNumber, copyRowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);
            PathType copyType = GetNamesFromPath(copyPath, out copyTableName, out
copyRowNumber);

            if (type == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(path);
            }

            if (type == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(copyPath);
            }

            // Get the table and the table to copy to
            OdbcDataAdapter da = GetAdapterForTable(tableName);
            if (da == null)
            {
                return;
            }

            DataSet ds = GetDataSetForTable(da, tableName);
            DataTable table = GetDataTable(ds, tableName);

            OdbcDataAdapter cda = GetAdapterForTable(copyTableName);
            if (cda == null)
            {
                return;
            }

            DataSet cds = GetDataSetForTable(cda, copyTableName);
            DataTable copyTable = GetDataTable(cds, copyTableName);

            // if source represents a table
            if (type == PathType.Table)
            {
                // if copyPath does not represent a table
                if (copyType != PathType.Table)
                {
                    ArgumentException e = new ArgumentException("Table can only be
copied on to another table location");

                    WriteError(new ErrorRecord(e, "PathNotValid",
                        ErrorCategory.InvalidArgument, copyPath));
```

```
                throw e;
            }

            // if table already exists then force parameter should be set
            // to force a copy
            if (!Force && GetTable(copyTableName) != null)
            {
                throw new ArgumentException("Specified path already exists");
            }

            for (int i = 0; i < table.Rows.Count; i++)
            {
                DataRow row = table.Rows[i];
                DataRow copyRow = copyTable.NewRow();

                copyRow.ItemArray = row.ItemArray;
                copyTable.Rows.Add(copyRow);
            }
        } // if (type == ...
        // if source represents a row
        else
        {
            if (copyType == PathType.Row)
            {
                if (!Force && (copyRowNumber < copyTable.Rows.Count))
                {
                    throw new ArgumentException("Specified path already
exists.");
                }

                DataRow row = table.Rows[rowNumber];
                DataRow copyRow = null;

                if (copyRowNumber < copyTable.Rows.Count)
                {
                    // copy to an existing row
                    copyRow = copyTable.Rows[copyRowNumber];
                    copyRow.ItemArray = row.ItemArray;
                    copyRow[0] = GetNextID(copyTable);
                }
                else if (copyRowNumber == copyTable.Rows.Count)
                {
                    // copy to the next row in the table that will
                    // be created
                    copyRow = copyTable.NewRow();
                    copyRow.ItemArray = row.ItemArray;
                    copyRow[0] = GetNextID(copyTable);
                    copyTable.Rows.Add(copyRow);
                }
                else
                {
                    // attempting to copy to a nonexistent row or a row
                    // that cannot be created now - throw an exception
                    string message = String.Format(CultureInfo.CurrentCulture,
                                                   "The item cannot be specified to the
```

```
copied row. Specify row number as {0}, or specify a path to the table.",
                                                    table.Rows.Count);

                    throw new ArgumentException(message);
                }
            }
            else
            {
                // destination path specified represents a table,
                // create a new row and copy the item
                DataRow copyRow = copyTable.NewRow();
                copyRow.ItemArray = table.Rows[rowNumber].ItemArray;
                copyRow[0] = GetNextID(copyTable);
                copyTable.Rows.Add(copyRow);
            }
        }

        if (ShouldProcess(copyTableName, "CopyItems"))
        {
            cda.Update(cds, copyTableName);
        }

    } //CopyItem

    /// <summary>
    /// Removes (deletes) the item at the specified path
    /// </summary>
    ///
    /// <param name="path">
    /// The path to the item to remove.
    /// </param>
    ///
    /// <param name="recurse">
    /// True if all children in a subtree should be removed, false if only
    /// the item at the specified path should be removed. Is applicable
    /// only for container (table) items. Its ignored otherwise (even if
    /// specified).
    /// </param>
    ///
    /// <remarks>
    /// There are no elements in this store which are hidden from the user.
    /// Hence this method will not check for the presence of the Force
    /// parameter
    /// </remarks>
    ///
    protected override void RemoveItem(string path, bool recurse)
    {
        string tableName;
        int rowNumber = 0;

        PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

        if (type == PathType.Table)
        {
            // if recurse flag has been specified, delete all the rows as well
```

```
            if (recurse)
            {
                OdbcDataAdapter da = GetAdapterForTable(tableName);
                if (da == null)
                {
                    return;
                }

                DataSet ds = GetDataSetForTable(da, tableName);
                DataTable table = GetDataTable(ds, tableName);

                for (int i = 0; i < table.Rows.Count; i++)
                {
                    table.Rows[i].Delete();
                }

                if (ShouldProcess(path, "RemoveItem"))
                {
                    da.Update(ds, tableName);
                    RemoveTable(tableName);
                }
            }//if (recurse...
            else
            {
                // Remove the table
                if (ShouldProcess(path, "RemoveItem"))
                {
                    RemoveTable(tableName);
                }
            }
        }
        else if (type == PathType.Row)
        {
            OdbcDataAdapter da = GetAdapterForTable(tableName);
            if (da == null)
            {
                return;
            }

            DataSet ds = GetDataSetForTable(da, tableName);
            DataTable table = GetDataTable(ds, tableName);

            table.Rows[rowNumber].Delete();

            if (ShouldProcess(path, "RemoveItem"))
            {
                da.Update(ds, tableName);
            }
        }
        else
        {
            ThrowTerminatingInvalidPathException(path);
        }

    } // RemoveItem
```

```
        #endregion Container Overloads

        #region Helper Methods

       /// <summary>
       /// Checks if a given path is actually a drive name.
       /// </summary>
       /// <param name="path">The path to check.</param>
       /// <returns>
       /// True if the path given represents a drive, false otherwise.
       /// </returns>
       private bool PathIsDrive(string path)
       {
           // Remove the drive name and first path separator.  If the
           // path is reduced to nothing, it is a drive. Also if its
           // just a drive then there wont be any path separators
           if (String.IsNullOrEmpty(
                       path.Replace(this.PSDriveInfo.Root, "")) ||
               String.IsNullOrEmpty(
                       path.Replace(this.PSDriveInfo.Root + pathSeparator, ""))

               )
           {
               return true;
           }
           else
           {
               return false;
           }
        } // PathIsDrive

        /// <summary>
        /// Breaks up the path into individual elements.
        /// </summary>
        /// <param name="path">The path to split.</param>
        /// <returns>An array of path segments.</returns>
        private string[] ChunkPath(string path)
        {
            // Normalize the path before splitting
            string normalPath = NormalizePath(path);

            // Return the path with the drive name and first path
            // separator character removed, split by the path separator.
            string pathNoDrive = normalPath.Replace(this.PSDriveInfo.Root
                                           + pathSeparator, "");

            return pathNoDrive.Split(pathSeparator.ToCharArray());
        } // ChunkPath

        /// <summary>
        /// Adapts the path, making sure the correct path separator
        /// character is used.
        /// </summary>
        /// <param name="path"></param>
```

```
        /// <returns></returns>
        private string NormalizePath(string path)
        {
            string result = path;

            if (!String.IsNullOrEmpty(path))
            {
                result = path.Replace("/", pathSeparator);
            }

            return result;
        } // NormalizePath

        /// <summary>
        /// Chunks the path and returns the table name and the row number
        /// from the path
        /// </summary>
        /// <param name="path">Path to chunk and obtain information</param>
        /// <param name="tableName">Name of the table as represented in the
        /// path</param>
        /// <param name="rowNumber">Row number obtained from the path</param>
        /// <returns>what the path represents</returns>
        private PathType GetNamesFromPath(string path, out string tableName, out int
rowNumber)
        {
            PathType retVal = PathType.Invalid;
            rowNumber = -1;
            tableName = null;

            // Check if the path specified is a drive
            if (PathIsDrive(path))
            {
                return PathType.Database;
            }

            // chunk the path into parts
            string[] pathChunks = ChunkPath(path);

            switch (pathChunks.Length)
            {
                case 1:
                    {
                        string name = pathChunks[0];

                        if (TableNameIsValid(name))
                        {
                            tableName = name;
                            retVal = PathType.Table;
                        }
                    }
                    break;

                case 2:
                    {
                        string name = pathChunks[0];
```

```
                    if (TableNameIsValid(name))
                    {
                        tableName = name;
                    }

                    int number = SafeConvertRowNumber(pathChunks[1]);

                    if (number >= 0)
                    {
                        rowNumber = number;
                        retVal = PathType.Row;
                    }
                    else
                    {
                        WriteError(new ErrorRecord(
                            new ArgumentException("Row number is not valid"),
                            "RowNumberNotValid",
                            ErrorCategory.InvalidArgument,
                            path));
                    }
                }
                break;

            default:
                {
                    WriteError(new ErrorRecord(
                        new ArgumentException("The path supplied has too many
segments"),
                        "PathNotValid",
                        ErrorCategory.InvalidArgument,
                        path));
                }
                break;
        } // switch(pathChunks...

        return retVal;
    } // GetNamesFromPath

    /// <summary>
    /// Throws an argument exception stating that the specified path does
    /// not represent either a table or a row
    /// </summary>
    /// <param name="path">path which is invalid</param>
    private void ThrowTerminatingInvalidPathException(string path)
    {
        StringBuilder message = new StringBuilder("Path must represent either a
table or a row :");
        message.Append(path);

        throw new ArgumentException(message.ToString());
    }

    /// <summary>
    /// Retrieve the list of tables from the database.
```

```
        /// </summary>
        /// <returns>
        /// Collection of DatabaseTableInfo objects, each object representing
        /// information about one database table
        /// </returns>
        private Collection<DatabaseTableInfo> GetTables()
        {
            Collection<DatabaseTableInfo> results =
                    new Collection<DatabaseTableInfo>();

            // using ODBC connection to the database and get the schema of tables
            AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;

            if (di == null)
            {
                return null;
            }

            OdbcConnection connection = di.Connection;
            DataTable dt = connection.GetSchema("Tables");
            int count;

            // iterate through all rows in the schema and create DatabaseTableInfo
            // objects which represents a table
            foreach (DataRow dr in dt.Rows)
            {
                String tableName = dr["TABLE_NAME"] as String;
                DataColumnCollection columns = null;

                // find the number of rows in the table
                try
                {
                    String cmd = "Select count(*) from \"" + tableName + "\"";
                    OdbcCommand command = new OdbcCommand(cmd, connection);

                    count = (Int32)command.ExecuteScalar();
                }
                catch
                {
                    count = 0;
                }

                // create DatabaseTableInfo object representing the table
                DatabaseTableInfo table =
                        new DatabaseTableInfo(dr, tableName, count, columns);

                results.Add(table);
            } // foreach (DataRow...

            return results;
        } // GetTables

        /// <summary>
        /// Return row information from a specified table.
        /// </summary>
```

```
        /// <param name="tableName">The name of the database table from
        /// which to retrieve rows.</param>
        /// <returns>Collection of row information objects.</returns>
        private Collection<DatabaseRowInfo> GetRows(string tableName)
        {
            Collection<DatabaseRowInfo> results =
                        new Collection<DatabaseRowInfo>();

            // Obtain rows in the table and add it to the collection
            try
            {
                OdbcDataAdapter da = GetAdapterForTable(tableName);

                if (da == null)
                {
                    return null;
                }

                DataSet ds = GetDataSetForTable(da, tableName);
                DataTable table = GetDataTable(ds, tableName);

                int i = 0;
                foreach (DataRow row in table.Rows)
                {
                    results.Add(new DatabaseRowInfo(row,
i.ToString(CultureInfo.CurrentCulture)));
                    i++;
                } // foreach (DataRow...
            }
            catch (Exception e)
            {
                WriteError(new ErrorRecord(e, "CannotAccessSpecifiedRows",
                    ErrorCategory.InvalidOperation, tableName));
            }

            return results;

        } // GetRows

        /// <summary>
        /// Retrieve information about a single table.
        /// </summary>
        /// <param name="tableName">The table for which to retrieve
        /// data.</param>
        /// <returns>Table information.</returns>
        private DatabaseTableInfo GetTable(string tableName)
        {
            foreach (DatabaseTableInfo table in GetTables())
            {
                if (String.Equals(tableName, table.Name,
StringComparison.OrdinalIgnoreCase))
                {
                    return table;
                }
            }
```

```
            return null;
        } // GetTable

        /// <summary>
        /// Removes the specified table from the database
        /// </summary>
        /// <param name="tableName">Name of the table to remove</param>
        private void RemoveTable(string tableName)
        {
            // validate if tablename is valid and if table is present
            if (String.IsNullOrEmpty(tableName) || !TableNameIsValid(tableName) ||
!TableIsPresent(tableName))
            {
                return;
            }

            // Execute command using ODBC connection to remove a table
            try
            {
                // delete the table using an sql statement
                string sql = "drop table " + tableName;

                AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;

                if (di == null)
                {
                    return;
                }
                OdbcConnection connection = di.Connection;

                OdbcCommand cmd = new OdbcCommand(sql, connection);
                cmd.ExecuteScalar();
            }
            catch (Exception ex)
            {
                WriteError(new ErrorRecord(ex, "CannotRemoveSpecifiedTable",
                          ErrorCategory.InvalidOperation, null)
                          );
            }

        } // RemoveTable

        /// <summary>
        /// Obtain a data adapter for the specified Table
        /// </summary>
        /// <param name="tableName">Name of the table to obtain the
        /// adapter for</param>
        /// <returns>Adapter object for the specified table</returns>
        /// <remarks>An adapter serves as a bridge between a DataSet (in memory
        /// representation of table) and the data source</remarks>
        private OdbcDataAdapter GetAdapterForTable(string tableName)
        {
            OdbcDataAdapter da = null;
            AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;
```

```
            if (di == null || !TableNameIsValid(tableName)
||!TableIsPresent(tableName))
            {
                return null;
            }

            OdbcConnection connection = di.Connection;

            try
            {
                // Create a odbc data adpater. This can be sued to update the
                // data source with the records that will be created here
                // using data sets
                string sql = "Select * from " + tableName;
                da = new OdbcDataAdapter(new OdbcCommand(sql, connection));

                // Create a odbc command builder object. This will create sql
                // commands automatically for a single table, thus
                // eliminating the need to create new sql statements for
                // every operation to be done.
                OdbcCommandBuilder cmd = new OdbcCommandBuilder(da);

                // Set the delete cmd for the table here
                sql = "Delete from " + tableName + " where ID = ?";
                da.DeleteCommand = new OdbcCommand(sql, connection);

                // Specify a DeleteCommand parameter based on the "ID"
                // column
                da.DeleteCommand.Parameters.Add(new OdbcParameter());
                da.DeleteCommand.Parameters[0].SourceColumn = "ID";

                // Create an InsertCommand based on the sql string
                // Insert into "tablename" values (?,?,?)" where
                // ? represents a column in the table. Note that
                // the number of ? will be equal to the number of
                // columnds
                DataSet ds = new DataSet();

                da.FillSchema(ds, SchemaType.Source);
                ds.Locale = CultureInfo.InvariantCulture;

                sql = "Insert into " + tableName + " values ( ";
                for (int i = 0; i < ds.Tables["Table"].Columns.Count; i++)
                {
                    sql += "?, ";
                }
                sql = sql.Substring(0, sql.Length - 2);
                sql += ")";
                da.InsertCommand = new OdbcCommand(sql, connection);

                // Create parameters for the InsertCommand based on the
                // captions of each column
                for (int i = 0; i < ds.Tables["Table"].Columns.Count; i++)
                {
```

```
                da.InsertCommand.Parameters.Add(new OdbcParameter());
                da.InsertCommand.Parameters[i].SourceColumn =
                                ds.Tables["Table"].Columns[i].Caption;

            }

            // Open the connection if its not already open
            if (connection.State != ConnectionState.Open)
            {
                connection.Open();
            }
        }
        catch (Exception e)
        {
            WriteError(new ErrorRecord(e, "CannotAccessSpecifiedTable",
              ErrorCategory.InvalidOperation, tableName));
        }

        return da;
    } // GetAdapterForTable

    /// <summary>
    /// Gets the DataSet (in memory representation) for the table
    /// for the specified adapter
    /// </summary>
    /// <param name="adapter">Adapter to be used for obtaining
    /// the table</param>
    /// <param name="tableName">Name of the table for which a
    /// DataSet is required</param>
    /// <returns>The DataSet with the filled in schema</returns>
    private DataSet GetDataSetForTable(OdbcDataAdapter adapter, string
tableName)
    {
        Debug.Assert(adapter != null);

        // Create a dataset object which will provide an in-memory
        // representation of the data being worked upon in the
        // data source.
        DataSet ds = new DataSet();

        // Create a table named "Table" which will contain the same
        // schema as in the data source.
        //adapter.FillSchema(ds, SchemaType.Source);
        adapter.Fill(ds, tableName);
        ds.Locale = CultureInfo.InvariantCulture;

        return ds;
    } //GetDataSetForTable

    /// <summary>
    /// Get the DataTable object which can be used to operate on
    /// for the specified table in the data source
    /// </summary>
    /// <param name="ds">DataSet object which contains the tables
    /// schema</param>
```

```
        /// <param name="tableName">Name of the table</param>
        /// <returns>Corresponding DataTable object representing
        /// the table</returns>
        ///
        private DataTable GetDataTable(DataSet ds, string tableName)
        {
            Debug.Assert(ds != null);
            Debug.Assert(tableName != null);

            DataTable table = ds.Tables[tableName];
            table.Locale = CultureInfo.InvariantCulture;

            return table;
        } // GetDataTable

        /// <summary>
        /// Retrieves a single row from the named table.
        /// </summary>
        /// <param name="tableName">The table that contains the
        /// numbered row.</param>
        /// <param name="row">The index of the row to return.</param>
        /// <returns>The specified table row.</returns>
        private DatabaseRowInfo GetRow(string tableName, int row)
        {
            Collection<DatabaseRowInfo> di = GetRows(tableName);

            // if the row is invalid write an appropriate error else return the
            // corresponding row information
            if (row < di.Count && row >= 0)
            {
                return di[row];
            }
            else
            {
                WriteError(new ErrorRecord(
                   new ItemNotFoundException(),
                   "RowNotFound",
                   ErrorCategory.ObjectNotFound,
                   row.ToString(CultureInfo.CurrentCulture))
                );
            }

            return null;
        } // GetRow

        /// <summary>
        /// Method to safely convert a string representation of a row number
        /// into its Int32 equivalent
        /// </summary>
        /// <param name="rowNumberAsStr">String representation of the row
        /// number</param>
        /// <remarks>If there is an exception, -1 is returned</remarks>
        private int SafeConvertRowNumber(string rowNumberAsStr)
        {
            int rowNumber = -1;
```

```
            try
            {
                rowNumber = Convert.ToInt32(rowNumberAsStr,
CultureInfo.CurrentCulture);
            }
            catch (FormatException fe)
            {
                WriteError(new ErrorRecord(fe, "RowStringFormatNotValid",
                    ErrorCategory.InvalidData, rowNumberAsStr));
            }
            catch (OverflowException oe)
            {
                WriteError(new ErrorRecord(oe, "RowStringConversionToNumberFailed",
                    ErrorCategory.InvalidData, rowNumberAsStr));
            }

            return rowNumber;
        } // SafeConvertRowNumber

        /// <summary>
        /// Check if a table name is valid
        /// </summary>
        /// <param name="tableName">Table name to validate</param>
        /// <remarks>Helps to check for SQL injection attacks</remarks>
        private bool TableNameIsValid(string tableName)
        {
            Regex exp = new Regex(pattern, RegexOptions.Compiled |
RegexOptions.IgnoreCase);

            if (exp.IsMatch(tableName))
            {
                return true;
            }
            WriteError(new ErrorRecord(
                new ArgumentException("Table name not valid"), "TableNameNotValid",
                     ErrorCategory.InvalidArgument, tableName));
            return false;
        } // TableNameIsValid

        /// <summary>
        /// Checks to see if the specified table is present in the
        /// database
        /// </summary>
        /// <param name="tableName">Name of the table to check</param>
        /// <returns>true, if table is present, false otherwise</returns>
        private bool TableIsPresent(string tableName)
        {
            // using ODBC connection to the database and get the schema of tables
            AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;
            if (di == null)
            {
                return false;
            }

            OdbcConnection connection = di.Connection;
```

```
            DataTable dt = connection.GetSchema("Tables");

            // check if the specified tableName is available
            // in the list of tables present in the database
            foreach (DataRow dr in dt.Rows)
            {
                string name = dr["TABLE_NAME"] as string;
                if (name.Equals(tableName, StringComparison.OrdinalIgnoreCase))
                {
                    return true;
                }
            }

            WriteError(new ErrorRecord(
                new ArgumentException("Specified Table is not present in database"),
"TableNotAvailable",
                     ErrorCategory.InvalidArgument, tableName));

            return false;
        }// TableIsPresent

        /// <summary>
        /// Gets the next available ID in the table
        /// </summary>
        /// <param name="table">DataTable object representing the table to
        /// search for ID</param>
        /// <returns>next available id</returns>
        private int GetNextID(DataTable table)
        {
            int big = 0;
            int id = 0;

            for (int i = 0; i < table.Rows.Count; i++)
            {
                DataRow row = table.Rows[i];

                object o = row["ID"];

                if (o.GetType().Name.Equals("Int16"))
                {
                    id = (int)(short)o;
                }
                else
                {
                    id = (int)o;
                }

                if (big < id)
                {
                    big = id;
                }
            }

            big++;
            return big;
```

```
        }

        #endregion Helper Methods

        #region Private Properties

        private string pathSeparator = "\\";
        private static string pattern = @"^[a-z]+[0-9]*_*$";

        private enum PathType { Database, Table, Row, Invalid };

        #endregion Private Properties
    }

    #endregion AccessDBProvider

    #region Helper Classes

    #region AccessDBPSDriveInfo

    /// <summary>
    /// Any state associated with the drive should be held here.
    /// In this case, it's the connection to the database.
    /// </summary>
    internal class AccessDBPSDriveInfo : PSDriveInfo
    {
        private OdbcConnection connection;

        /// <summary>
        /// ODBC connection information.
        /// </summary>
        public OdbcConnection Connection
        {
            get { return connection; }
            set { connection = value; }
        }

        /// <summary>
        /// Constructor that takes one argument
        /// </summary>
        /// <param name="driveInfo">Drive provided by this provider</param>
        public AccessDBPSDriveInfo(PSDriveInfo driveInfo)
            : base(driveInfo)
        { }

    } // class AccessDBPSDriveInfo

    #endregion AccessDBPSDriveInfo

    #region DatabaseTableInfo

    /// <summary>
    /// Contains information specific to the database table.
    /// Similar to the DirectoryInfo class.
    /// </summary>
```

```
public class DatabaseTableInfo
{
    /// <summary>
    /// Row from the "tables" schema
    /// </summary>
    public DataRow Data
    {
        get
        {
            return data;
        }
        set
        {
            data = value;
        }
    }
    private DataRow data;

    /// <summary>
    /// The table name.
    /// </summary>
    public string Name
    {
        get
        {
            return name;
        }
        set
        {
            name = value;
        }
    }
    private String name;

    /// <summary>
    /// The number of rows in the table.
    /// </summary>
    public int RowCount
    {
        get
        {
            return rowCount;
        }
        set
        {
            rowCount = value;
        }
    }
    private int rowCount;

    /// <summary>
    /// The column definitions for the table.
    /// </summary>
    public DataColumnCollection Columns
    {
```

```
            get
            {
                return columns;
            }
            set
            {
                columns = value;
            }
        }
        private DataColumnCollection columns;

        /// <summary>
        /// Constructor.
        /// </summary>
        /// <param name="row">The row definition.</param>
        /// <param name="name">The table name.</param>
        /// <param name="rowCount">The number of rows in the table.</param>
        /// <param name="columns">Information on the column tables.</param>
        public DatabaseTableInfo(DataRow row, string name, int rowCount,
                        DataColumnCollection columns)
        {
            Name = name;
            Data = row;
            RowCount = rowCount;
            Columns = columns;
        } // DatabaseTableInfo
    } // class DatabaseTableInfo

    #endregion DatabaseTableInfo

    #region DatabaseRowInfo

    /// <summary>
    /// Contains information specific to an individual table row.
    /// Analogous to the FileInfo class.
    /// </summary>
    public class DatabaseRowInfo
    {
        /// <summary>
        /// Row data information.
        /// </summary>
        public DataRow Data
        {
            get
            {
                return data;
            }
            set
            {
                data = value;
            }
        }
        private DataRow data;

        /// <summary>
```

```
        /// The row index.
        /// </summary>
        public string RowNumber
        {
            get
            {
                return rowNumber;
            }
            set
            {
                rowNumber = value;
            }
        }
        private string rowNumber;

        /// <summary>
        /// Constructor.
        /// </summary>
        /// <param name="row">The row information.</param>
        /// <param name="name">The row index.</param>
        public DatabaseRowInfo(DataRow row, string name)
        {
            RowNumber = name;
            Data = row;
        } // DatabaseRowInfo
    } // class DatabaseRowInfo

    #endregion DatabaseRowInfo

    #endregion Helper Classes
}
```

See Also

System.Management.Automation.Provider.ItemCmdletProvider

System.Management.Automation.Provider.ContainerCmdletProvider

System.Management.Automation.Provider.NavigationCmdletProvider

Designing Your Windows PowerShell Provider

Last updated on 05/20/2025

AccessDBProviderSample05

This sample shows how to overwrite container methods to support calls to the `Move-Item` and `Join-Path` cmdlets. These methods should be implemented when the user needs to move items within a container and if the data store contains nested containers. The provider class in this sample derives from the System.Management.Automation.Provider.NavigationCmdletProvider class.

Demonstrates

> **Important**
>
> Your provider class will most likely derive from one of the following classes and possibly implement other provider interfaces:
>
> - System.Management.Automation.Provider.ItemCmdletProvider class. See AccessDBProviderSample03.
> - System.Management.Automation.Provider.ContainerCmdletProvider class. See AccessDBProviderSample04.
> - System.Management.Automation.Provider.NavigationCmdletProvider class.
>
> For more information about choosing which provider class to derive from based on provider features, see Designing Your Windows PowerShell Provider.

This sample demonstrates the following:

- Declaring the `CmdletProvider` attribute.
- Defining a provider class that derives from the System.Management.Automation.Provider.NavigationCmdletProvider class.
- Overwriting the System.Management.Automation.Provider.NavigationCmdletProvider.MoveItem* method to change the behavior of the `Move-Item` cmdlet, allowing the user to move items from one location to another. (This sample does not show how to add dynamic parameters to the `Move-Item` cmdlet.)

- Overwriting the System.Management.Automation.Provider.NavigationCmdletProvider.MakePath* method to change the behavior of the `Join-Path` cmdlet.
- Overwriting the System.Management.Automation.Provider.NavigationCmdletProvider.IsItemContainer* method.
- Overwriting the System.Management.Automation.Provider.NavigationCmdletProvider.GetChildName* method.
- Overwriting the System.Management.Automation.Provider.NavigationCmdletProvider.GetParentPath* method.
- Overwriting the System.Management.Automation.Provider.NavigationCmdletProvider.NormalizeRelativePath* method.

Example

This sample shows how to overwrite the methods needed to move items in a Microsoft Access data base.

C#

```
using System;
using System.IO;
using System.Data;
using System.Data.Odbc;
using System.Diagnostics;
using System.Collections.ObjectModel;
using System.Text;
using System.Text.RegularExpressions;
using System.Management.Automation;
using System.Management.Automation.Provider;
using System.ComponentModel;
using System.Globalization;

namespace Microsoft.Samples.PowerShell.Providers
{
   #region AccessDBProvider

    /// <summary>
   /// This example implements the navigation methods.
```

```
    /// </summary>
    [CmdletProvider("AccessDB", ProviderCapabilities.None)]
    public class AccessDBProvider : NavigationCmdletProvider
    {

        #region Drive Manipulation

        /// <summary>
        /// Create a new drive.  Create a connection to the database file and set
        /// the Connection property in the PSDriveInfo.
        /// </summary>
        /// <param name="drive">
        /// Information describing the drive to add.
        /// </param>
        /// <returns>The added drive.</returns>
        protected override PSDriveInfo NewDrive(PSDriveInfo drive)
        {
            // check if drive object is null
            if (drive == null)
            {
                WriteError(new ErrorRecord(
                    new ArgumentNullException("drive"),
                    "NullDrive",
                    ErrorCategory.InvalidArgument,
                    null)
                );

                return null;
            }

            // check if drive root is not null or empty
            // and if its an existing file
            if (String.IsNullOrEmpty(drive.Root) || (File.Exists(drive.Root) ==
false))
            {
                WriteError(new ErrorRecord(
                    new ArgumentException("drive.Root"),
                    "NoRoot",
                    ErrorCategory.InvalidArgument,
                    drive)
                );

                return null;
            }


            // create a new drive and create an ODBC connection to the new drive
            AccessDBPSDriveInfo accessDBPSDriveInfo = new
AccessDBPSDriveInfo(drive);

            OdbcConnectionStringBuilder builder = new OdbcConnectionStringBuilder();

            builder.Driver = "Microsoft Access Driver (*.mdb)";
            builder.Add("DBQ", drive.Root);
```

```
            OdbcConnection conn = new OdbcConnection(builder.ConnectionString);
            conn.Open();
            accessDBPSDriveInfo.Connection = conn;

            return accessDBPSDriveInfo;
        } // NewDrive

        /// <summary>
        /// Removes a drive from the provider.
        /// </summary>
        /// <param name="drive">The drive to remove.</param>
        /// <returns>The drive removed.</returns>
        protected override PSDriveInfo RemoveDrive(PSDriveInfo drive)
        {
            // check if drive object is null
            if (drive == null)
            {
                WriteError(new ErrorRecord(
                    new ArgumentNullException("drive"),
                    "NullDrive",
                    ErrorCategory.InvalidArgument,
                    drive)
                );

                return null;
            }

            // close ODBC connection to the drive
            AccessDBPSDriveInfo accessDBPSDriveInfo = drive as AccessDBPSDriveInfo;

            if (accessDBPSDriveInfo == null)
            {
                return null;
            }
            accessDBPSDriveInfo.Connection.Close();

            return accessDBPSDriveInfo;
        } // RemoveDrive

        #endregion Drive Manipulation

    #region Item Methods

        /// <summary>
        /// Retrieves an item using the specified path.
        /// </summary>
        /// <param name="path">The path to the item to return.</param>
        protected override void GetItem(string path)
        {
            // check if the path represented is a drive
            if (PathIsDrive(path))
            {
                WriteItemObject(this.PSDriveInfo, path, true);
                return;
            }// if (PathIsDrive...
```

```
        // Get table name and row information from the path and do
        // necessary actions
        string tableName;
        int rowNumber;

        PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

        if (type == PathType.Table)
        {
            DatabaseTableInfo table = GetTable(tableName);
            WriteItemObject(table, path, true);
        }
        else if (type == PathType.Row)
        {
            DatabaseRowInfo row = GetRow(tableName, rowNumber);
            WriteItemObject(row, path, false);
        }
        else
        {
            ThrowTerminatingInvalidPathException(path);
        }

    } // GetItem

    /// <summary>
    /// Set the content of a row of data specified by the supplied path
    /// parameter.
    /// </summary>
    /// <param name="path">Specifies the path to the row whose columns
    /// will be updated.</param>
    /// <param name="values">Comma separated string of values</param>
    protected override void SetItem(string path, object values)
    {
        // Get type, table name and row number from the path specified
        string tableName;
        int rowNumber;

        PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

        if (type != PathType.Row)
        {
            WriteError(new ErrorRecord(new NotSupportedException(
                  "SetNotSupported"), "",
               ErrorCategory.InvalidOperation, path));

            return;
        }

        // Get in-memory representation of table
        OdbcDataAdapter da = GetAdapterForTable(tableName);

        if (da == null)
        {
            return;
```

```
        }
        DataSet ds = GetDataSetForTable(da, tableName);
        DataTable table = GetDataTable(ds, tableName);

        if (rowNumber >= table.Rows.Count)
        {
            // The specified row number has to be available. If not
            // NewItem has to be used to add a new row
            throw new ArgumentException("Row specified is not available");
        } // if (rowNum...

        string[] colValues = (values as string).Split(',');

        // set the specified row
        DataRow row = table.Rows[rowNumber];

        for (int i = 0; i < colValues.Length; i++)
        {
            row[i] = colValues[i];
        }

        // Update the table
        if (ShouldProcess(path, "SetItem"))
        {
            da.Update(ds, tableName);
        }

    } // SetItem

    /// <summary>
    /// Test to see if the specified item exists.
    /// </summary>
    /// <param name="path">The path to the item to verify.</param>
    /// <returns>True if the item is found.</returns>
    protected override bool ItemExists(string path)
    {
        // check if the path represented is a drive
        if (PathIsDrive(path))
        {
            return true;
        }

        // Obtain type, table name and row number from path
        string tableName;
        int rowNumber;

        PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

        DatabaseTableInfo table = GetTable(tableName);

        if (type == PathType.Table)
        {
            // if specified path represents a table then DatabaseTableInfo
            // object for the same should exist
            if (table != null)
```

```
            {
                return true;
            }
        }
        else if (type == PathType.Row)
        {
            // if specified path represents a row then DatabaseTableInfo should
            // exist for the table and then specified row number must be within
            // the maximum row count in the table
            if (table != null && rowNumber < table.RowCount)
            {
                return true;
            }
        }

        return false;

    } // ItemExists

    /// <summary>
    /// Test to see if the specified path is syntactically valid.
    /// </summary>
    /// <param name="path">The path to validate.</param>
    /// <returns>True if the specified path is valid.</returns>
    protected override bool IsValidPath(string path)
    {
        bool result = true;

        // check if the path is null or empty
        if (String.IsNullOrEmpty(path))
        {
            result = false;
        }

        // convert all separators in the path to a uniform one
        path = NormalizePath(path);

        // split the path into individual chunks
        string[] pathChunks = path.Split(pathSeparator.ToCharArray());

        foreach (string pathChunk in pathChunks)
        {
            if (pathChunk.Length == 0)
            {
                result = false;
            }
        }
        return result;
    } // IsValidPath

    #endregion Item Overloads

    #region Container Overloads

    /// <summary>
```

```
        /// Return either the tables in the database or the datarows
        /// </summary>
        /// <param name="path">The path to the parent</param>
        /// <param name="recurse">True to return all child items recursively.
        /// </param>
        protected override void GetChildItems(string path, bool recurse)
        {
            // If path represented is a drive then the children in the path are
            // tables. Hence all tables in the drive represented will have to be
            // returned
            if (PathIsDrive(path))
            {
                foreach (DatabaseTableInfo table in GetTables())
                {
                    WriteItemObject(table, path, true);

                    // if the specified item exists and recurse has been set then
                    // all child items within it have to be obtained as well
                    if (ItemExists(path) && recurse)
                    {
                        GetChildItems(path + pathSeparator + table.Name, recurse);
                    }
                } // foreach (DatabaseTableInfo...
            } // if (PathIsDrive...
            else
            {
                // Get the table name, row number and type of path from the
                // path specified
                string tableName;
                int rowNumber;

                PathType type = GetNamesFromPath(path, out tableName, out
rowNumber);

                if (type == PathType.Table)
                {
                    // Obtain all the rows within the table
                    foreach (DatabaseRowInfo row in GetRows(tableName))
                    {
                        WriteItemObject(row, path + pathSeparator + row.RowNumber,
                                false);
                    } // foreach (DatabaseRowInfo...
                }
                else if (type == PathType.Row)
                {
                    // In this case the user has directly specified a row, hence
                    // just give that particular row
                    DatabaseRowInfo row = GetRow(tableName, rowNumber);
                    WriteItemObject(row, path + pathSeparator + row.RowNumber,
                                false);
                }
                else
                {
                    // In this case, the path specified is not valid
                    ThrowTerminatingInvalidPathException(path);
```

```
            }
        } // else
    } // GetChildItems

    /// <summary>
    /// Return the names of all child items.
    /// </summary>
    /// <param name="path">The root path.</param>
    /// <param name="returnContainers">Not used.</param>
    protected override void GetChildNames(string path,
                                  ReturnContainers returnContainers)
    {
        // If the path represented is a drive, then the child items are
        // tables. get the names of all the tables in the drive.
        if (PathIsDrive(path))
        {
            foreach (DatabaseTableInfo table in GetTables())
            {
                WriteItemObject(table.Name, path, true);
            } // foreach (DatabaseTableInfo...
        } // if (PathIsDrive...
        else
        {
            // Get type, table name and row number from path specified
            string tableName;
            int rowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out
rowNumber);

            if (type == PathType.Table)
            {
                // Get all the rows in the table and then write out the
                // row numbers.
                foreach (DatabaseRowInfo row in GetRows(tableName))
                {
                    WriteItemObject(row.RowNumber, path, false);
                } // foreach (DatabaseRowInfo...
            }
            else if (type == PathType.Row)
            {
                // In this case the user has directly specified a row, hence
                // just give that particular row
                DatabaseRowInfo row = GetRow(tableName, rowNumber);

                WriteItemObject(row.RowNumber, path, false);
            }
            else
            {
                ThrowTerminatingInvalidPathException(path);
            }
        } // else
    } // GetChildNames

    /// <summary>
```

```
        /// Determines if the specified path has child items.
        /// </summary>
        /// <param name="path">The path to examine.</param>
        /// <returns>
        /// True if the specified path has child items.
        /// </returns>
        protected override bool HasChildItems(string path)
        {
            if (PathIsDrive(path))
            {
                return true;
            }

            return (ChunkPath(path).Length == 1);
        } // HasChildItems

        /// <summary>
        /// Creates a new item at the specified path.
        /// </summary>
        ///
        /// <param name="path">
        /// The path to the new item.
        /// </param>
        ///
        /// <param name="type">
        /// Type for the object to create. "Table" for creating a new table and
        /// "Row" for creating a new row in a table.
        /// </param>
        ///
        /// <param name="newItemValue">
        /// Object for creating new instance of a type at the specified path. For
        /// creating a "Table" the object parameter is ignored and for creating
        /// a "Row" the object must be of type string which will contain comma
        /// separated values of the rows to insert.
        /// </param>
        protected override void NewItem(string path, string type,
                                        object newItemValue)
        {
            string tableName;
            int rowNumber;

            PathType pt = GetNamesFromPath(path, out tableName, out rowNumber);

            if (pt == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(path);
            }

            // Check if type is either "table" or "row", if not throw an
            // exception
            if (!String.Equals(type, "table", StringComparison.OrdinalIgnoreCase)
                && !String.Equals(type, "row", StringComparison.OrdinalIgnoreCase))
            {
                WriteError(new ErrorRecord
                                    (new ArgumentException("Type must be either a
```

```
table or row"),
                                    "CannotCreateSpecifiedObject",
                                       ErrorCategory.InvalidArgument,
                                            path
                                    )
                          );

                throw new ArgumentException("This provider can only create items of
type \"table\" or \"row\"");
            }

            // Path type is the type of path of the container. So if a drive
            // is specified, then a table can be created under it and if a table
            // is specified, then a row can be created under it. For the sake of
            // completeness, if a row is specified, then if the row specified by
            // the path does not exist, a new row is created. However, the row
            // number may not match as the row numbers only get incremented based
            // on the number of rows

            if (PathIsDrive(path))
            {
                if (String.Equals(type, "table",
StringComparison.OrdinalIgnoreCase))
                {
                    // Execute command using ODBC connection to create a table
                    try
                    {
                        // create the table using an sql statement
                        string newTableName = newItemValue.ToString();
                        string sql = "create table " + newTableName
                                              + " (ID INT)";

                        // Create the table using the Odbc connection from the
                        // drive.
                        AccessDBPSDriveInfo di = this.PSDriveInfo as
AccessDBPSDriveInfo;

                        if (di == null)
                        {
                            return;
                        }
                        OdbcConnection connection = di.Connection;

                        if (ShouldProcess(newTableName, "create"))
                        {
                            OdbcCommand cmd = new OdbcCommand(sql, connection);
                            cmd.ExecuteScalar();
                        }
                    }
                    catch (Exception ex)
                    {
                        WriteError(new ErrorRecord(ex, "CannotCreateSpecifiedTable",
                                  ErrorCategory.InvalidOperation, path)
                                  );
                    }
```

```
                } // if (String...
                else if (String.Equals(type, "row",
StringComparison.OrdinalIgnoreCase))
                {
                    throw new
                        ArgumentException("A row cannot be created under a database,
specify a path that represents a Table");
                }
            }// if (PathIsDrive...
            else
            {
                if (String.Equals(type, "table",
StringComparison.OrdinalIgnoreCase))
                {
                    if (rowNumber < 0)
                    {
                        throw new
                            ArgumentException("A table cannot be created within
another table, specify a path that represents a database");
                    }
                    else
                    {
                        throw new
                            ArgumentException("A table cannot be created inside a
row, specify a path that represents a database");
                    }
                } //if (String.Equals....
                // if path specified is a row, create a new row
                else if (String.Equals(type, "row",
StringComparison.OrdinalIgnoreCase))
                {
                    // The user is required to specify the values to be inserted
                    // into the table in a single string separated by commas
                    string value = newItemValue as string;

                    if (String.IsNullOrEmpty(value))
                    {
                        throw new
                            ArgumentException("Value argument must have comma
separated values of each column in a row");
                    }
                    string[] rowValues = value.Split(',');

                    OdbcDataAdapter da = GetAdapterForTable(tableName);

                    if (da == null)
                    {
                        return;
                    }

                    DataSet ds = GetDataSetForTable(da, tableName);
                    DataTable table = GetDataTable(ds, tableName);

                    if (rowValues.Length != table.Columns.Count)
                    {
```

```
                        string message = String.Format(CultureInfo.CurrentCulture,
                                                       "The table has {0} columns and the
value specified must have so many comma separated values",
                                                           table.Columns.Count);

                        throw new ArgumentException(message);
                    }

                    if (!Force && (rowNumber >= 0 && rowNumber < table.Rows.Count))
                    {
                        string message = String.Format(CultureInfo.CurrentCulture,
                                                       "The row {0} already exists. To create
a new row specify row number as {1}, or specify path to a table, or use the -Force
parameter",
                                                           rowNumber, table.Rows.Count);

                        throw new ArgumentException(message);
                    }

                    if (rowNumber > table.Rows.Count)
                    {
                        string message = String.Format(CultureInfo.CurrentCulture,
                                                       "To create a new row specify row number
as {0}, or specify path to a table",
                                                           table.Rows.Count);

                        throw new ArgumentException(message);
                    }

                    // Create a new row and update the row with the input
                    // provided by the user
                    DataRow row = table.NewRow();
                    for (int i = 0; i < rowValues.Length; i++)
                    {
                        row[i] = rowValues[i];
                    }
                    table.Rows.Add(row);

                    if (ShouldProcess(tableName, "update rows"))
                    {
                        // Update the table from memory back to the data source
                        da.Update(ds, tableName);
                    }

                }// else if (String...
            }// else ...

        } // NewItem

        /// <summary>
        /// Copies an item at the specified path to the location specified
        /// </summary>
        ///
        /// <param name="path">
        /// Path of the item to copy
```

```
        /// </param>
        ///
        /// <param name="copyPath">
        /// Path of the item to copy to
        /// </param>
        ///
        /// <param name="recurse">
        /// Tells the provider to recurse subcontainers when copying
        /// </param>
        ///
        protected override void CopyItem(string path, string copyPath, bool recurse)
        {
            string tableName, copyTableName;
            int rowNumber, copyRowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);
            PathType copyType = GetNamesFromPath(copyPath, out copyTableName, out
copyRowNumber);

            if (type == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(path);
            }

            if (type == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(copyPath);
            }

            // Get the table and the table to copy to
            OdbcDataAdapter da = GetAdapterForTable(tableName);
            if (da == null)
            {
                return;
            }

            DataSet ds = GetDataSetForTable(da, tableName);
            DataTable table = GetDataTable(ds, tableName);

            OdbcDataAdapter cda = GetAdapterForTable(copyTableName);
            if (cda == null)
            {
                return;
            }

            DataSet cds = GetDataSetForTable(cda, copyTableName);
            DataTable copyTable = GetDataTable(cds, copyTableName);

            // if source represents a table
            if (type == PathType.Table)
            {
                // if copyPath does not represent a table
                if (copyType != PathType.Table)
                {
                    ArgumentException e = new ArgumentException("Table can only be
```

```
copied on to another table location");

                    WriteError(new ErrorRecord(e, "PathNotValid",
                        ErrorCategory.InvalidArgument, copyPath));

                    throw e;
                }

                // if table already exists then force parameter should be set
                // to force a copy
                if (!Force && GetTable(copyTableName) != null)
                {
                    throw new ArgumentException("Specified path already exists");
                }

                for (int i = 0; i < table.Rows.Count; i++)
                {
                    DataRow row = table.Rows[i];
                    DataRow copyRow = copyTable.NewRow();

                    copyRow.ItemArray = row.ItemArray;
                    copyTable.Rows.Add(copyRow);
                }
            } // if (type == ...
            // if source represents a row
            else
            {
                if (copyType == PathType.Row)
                {
                    if (!Force && (copyRowNumber < copyTable.Rows.Count))
                    {
                        throw new ArgumentException("Specified path already
exists.");
                    }

                    DataRow row = table.Rows[rowNumber];
                    DataRow copyRow = null;

                    if (copyRowNumber < copyTable.Rows.Count)
                    {
                        // copy to an existing row
                        copyRow = copyTable.Rows[copyRowNumber];
                        copyRow.ItemArray = row.ItemArray;
                        copyRow[0] = GetNextID(copyTable);
                    }
                    else if (copyRowNumber == copyTable.Rows.Count)
                    {
                        // copy to the next row in the table that will
                        // be created
                        copyRow = copyTable.NewRow();
                        copyRow.ItemArray = row.ItemArray;
                        copyRow[0] = GetNextID(copyTable);
                        copyTable.Rows.Add(copyRow);
                    }
                    else
```

```
                    {
                        // attempting to copy to a nonexistent row or a row
                        // that cannot be created now - throw an exception
                        string message = String.Format(CultureInfo.CurrentCulture,
                                                       "The item cannot be specified to the
copied row. Specify row number as {0}, or specify a path to the table.",
                                                           table.Rows.Count);

                        throw new ArgumentException(message);
                    }
                }
                else
                {
                    // destination path specified represents a table,
                    // create a new row and copy the item
                    DataRow copyRow = copyTable.NewRow();
                    copyRow.ItemArray = table.Rows[rowNumber].ItemArray;
                    copyRow[0] = GetNextID(copyTable);
                    copyTable.Rows.Add(copyRow);
                }
            }

            if (ShouldProcess(copyTableName, "CopyItems"))
            {
                cda.Update(cds, copyTableName);
            }

        } //CopyItem

        /// <summary>
        /// Removes (deletes) the item at the specified path
        /// </summary>
        ///
        /// <param name="path">
        /// The path to the item to remove.
        /// </param>
        ///
        /// <param name="recurse">
        /// True if all children in a subtree should be removed, false if only
        /// the item at the specified path should be removed. Is applicable
        /// only for container (table) items. Its ignored otherwise (even if
        /// specified).
        /// </param>
        ///
        /// <remarks>
        /// There are no elements in this store which are hidden from the user.
        /// Hence this method will not check for the presence of the Force
        /// parameter
        /// </remarks>
        ///
        protected override void RemoveItem(string path, bool recurse)
        {
            string tableName;
            int rowNumber = 0;
```

```
            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            if (type == PathType.Table)
            {
                // if recurse flag has been specified, delete all the rows as well
                if (recurse)
                {
                    OdbcDataAdapter da = GetAdapterForTable(tableName);
                    if (da == null)
                    {
                        return;
                    }

                    DataSet ds = GetDataSetForTable(da, tableName);
                    DataTable table = GetDataTable(ds, tableName);

                    for (int i = 0; i < table.Rows.Count; i++)
                    {
                        table.Rows[i].Delete();
                    }

                    if (ShouldProcess(path, "RemoveItem"))
                    {
                        da.Update(ds, tableName);
                        RemoveTable(tableName);
                    }
                }//if (recurse...
                else
                {
                    // Remove the table
                    if (ShouldProcess(path, "RemoveItem"))
                    {
                        RemoveTable(tableName);
                    }
                }
            }
            else if (type == PathType.Row)
            {
                OdbcDataAdapter da = GetAdapterForTable(tableName);
                if (da == null)
                {
                    return;
                }

                DataSet ds = GetDataSetForTable(da, tableName);
                DataTable table = GetDataTable(ds, tableName);

                table.Rows[rowNumber].Delete();

                if (ShouldProcess(path, "RemoveItem"))
                {
                    da.Update(ds, tableName);
                }
            }
            else
```

```
            {
                ThrowTerminatingInvalidPathException(path);
            }

        } // RemoveItem

        #endregion Container Overloads

       #region Navigation

       /// <summary>
       /// Determine if the path specified is that of a container.
       /// </summary>
       /// <param name="path">The path to check.</param>
       /// <returns>True if the path specifies a container.</returns>
       protected override bool IsItemContainer(string path)
       {
          if (PathIsDrive(path))
          {
              return true;
          }

          string[] pathChunks = ChunkPath(path);
          string tableName;
          int rowNumber;

          PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

          if (type == PathType.Table)
          {
             foreach (DatabaseTableInfo ti in GetTables())
             {
                 if (string.Equals(ti.Name, tableName,
StringComparison.OrdinalIgnoreCase))
                 {
                     return true;
                 }
             } // foreach (DatabaseTableInfo...
          } // if (pathChunks...

          return false;
       } // IsItemContainer

        /// <summary>
        /// Get the name of the leaf element in the specified path
        /// </summary>
        ///
        /// <param name="path">
        /// The full or partial provider specific path
        /// </param>
        ///
        /// <returns>
        /// The leaf element in the path
        /// </returns>
        protected override string GetChildName(string path)
```

```
        {
            if (PathIsDrive(path))
            {
                return path;
            }

            string tableName;
            int rowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            if (type == PathType.Table)
            {
                return tableName;
            }
            else if (type == PathType.Row)
            {
                return rowNumber.ToString(CultureInfo.CurrentCulture);
            }
            else
            {
                ThrowTerminatingInvalidPathException(path);
            }

            return null;
        }

        /// <summary>
        /// Removes the child segment of the path and returns the remaining
        /// parent portion
        /// </summary>
        ///
        /// <param name="path">
        /// A full or partial provider specific path. The path may be to an
        /// item that may or may not exist.
        /// </param>
        ///
        /// <param name="root">
        /// The fully qualified path to the root of a drive. This parameter
        /// may be null or empty if a mounted drive is not in use for this
        /// operation.  If this parameter is not null or empty the result
        /// of the method should not be a path to a container that is a
        /// parent or in a different tree than the root.
        /// </param>
        ///
        /// <returns></returns>

        protected override string GetParentPath(string path, string root)
        {
            // If root is specified then the path has to contain
            // the root. If not nothing should be returned
            if (!String.IsNullOrEmpty(root))
            {
                if (!path.Contains(root))
                {
```

```
                return null;
            }
        }

        return path.Substring(0, path.LastIndexOf(pathSeparator,
StringComparison.OrdinalIgnoreCase));
    }

    /// <summary>
    /// Joins two strings with a provider specific path separator.
    /// </summary>
    ///
    /// <param name="parent">
    /// The parent segment of a path to be joined with the child.
    /// </param>
    ///
    /// <param name="child">
    /// The child segment of a path to be joined with the parent.
    /// </param>
    ///
    /// <returns>
    /// A string that represents the parent and child segments of the path
    /// joined by a path separator.
    /// </returns>

    protected override string MakePath(string parent, string child)
    {
        string result;

        string normalParent = NormalizePath(parent);
        normalParent = RemoveDriveFromPath(normalParent);
        string normalChild = NormalizePath(child);
        normalChild = RemoveDriveFromPath(normalChild);

        if (String.IsNullOrEmpty(normalParent) &&
String.IsNullOrEmpty(normalChild))
        {
            result = String.Empty;
        }
        else if (String.IsNullOrEmpty(normalParent) &&
!String.IsNullOrEmpty(normalChild))
        {
            result = normalChild;
        }
        else if (!String.IsNullOrEmpty(normalParent) &&
String.IsNullOrEmpty(normalChild))
        {
            if (normalParent.EndsWith(pathSeparator,
StringComparison.OrdinalIgnoreCase))
            {
                result = normalParent;
            }
            else
            {
                result = normalParent + pathSeparator;
```

```
                }
            } // else if (!String...
            else
            {
                if (!normalParent.Equals(String.Empty) &&
                    !normalParent.EndsWith(pathSeparator,
StringComparison.OrdinalIgnoreCase))
                {
                    result = normalParent + pathSeparator;
                }
                else
                {
                    result = normalParent;
                }

                if (normalChild.StartsWith(pathSeparator,
StringComparison.OrdinalIgnoreCase))
                {
                    result += normalChild.Substring(1);
                }
                else
                {
                    result += normalChild;
                }
            } // else

            return result;
        } // MakePath

        /// <summary>
        /// Normalizes the path that was passed in and returns the normalized
        /// path as a relative path to the basePath that was passed.
        /// </summary>
        ///
        /// <param name="path">
        /// A fully qualified provider specific path to an item.  The item
        /// should exist or the provider should write out an error.
        /// </param>
        ///
        /// <param name="basepath">
        /// The path that the return value should be relative to.
        /// </param>
        ///
        /// <returns>
        /// A normalized path that is relative to the basePath that was
        /// passed.  The provider should parse the path parameter, normalize
        /// the path, and then return the normalized path relative to the
        /// basePath.
        /// </returns>

        protected override string NormalizeRelativePath(string path,
                                                         string basepath)
        {
            // Normalize the paths first
            string normalPath = NormalizePath(path);
```

```
            normalPath = RemoveDriveFromPath(normalPath);
            string normalBasePath = NormalizePath(basepath);
            normalBasePath = RemoveDriveFromPath(normalBasePath);

            if (String.IsNullOrEmpty(normalBasePath))
            {
                return normalPath;
            }
            else
            {
                if (!normalPath.Contains(normalBasePath))
                {
                    return null;
                }

                return normalPath.Substring(normalBasePath.Length +
pathSeparator.Length);
            }
        }

        /// <summary>
        /// Moves the item specified by the path to the specified destination
        /// </summary>
        ///
        /// <param name="path">
        /// The path to the item to be moved
        /// </param>
        ///
        /// <param name="destination">
        /// The path of the destination container
        /// </param>

        protected override void MoveItem(string path, string destination)
        {
            // Get type, table name and rowNumber from the path
            string tableName, destTableName;
            int rowNumber, destRowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            PathType destType = GetNamesFromPath(destination, out destTableName,
                                    out destRowNumber);

            if (type == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(path);
            }

            if (destType == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(destination);
            }

            if (type == PathType.Table)
            {
```

```
                ArgumentException e = new ArgumentException("Move not supported for
tables");

                WriteError(new ErrorRecord(e, "MoveNotSupported",
                    ErrorCategory.InvalidArgument, path));

                throw e;
            }
            else
            {
                OdbcDataAdapter da = GetAdapterForTable(tableName);
                if (da == null)
                {
                    return;
                }

                DataSet ds = GetDataSetForTable(da, tableName);
                DataTable table = GetDataTable(ds, tableName);

                OdbcDataAdapter dda = GetAdapterForTable(destTableName);
                if (dda == null)
                {
                    return;
                }

                DataSet dds = GetDataSetForTable(dda, destTableName);
                DataTable destTable = GetDataTable(dds, destTableName);
                DataRow row = table.Rows[rowNumber];

                if (destType == PathType.Table)
                {
                    DataRow destRow = destTable.NewRow();

                    destRow.ItemArray = row.ItemArray;
                }
                else
                {
                    DataRow destRow = destTable.Rows[destRowNumber];

                    destRow.ItemArray = row.ItemArray;
                }

                // Update the changes
                if (ShouldProcess(path, "MoveItem"))
                {
                    WriteItemObject(row, path, false);
                    dda.Update(dds, destTableName);
                }
            }
        }

        #endregion Navigation

        #region Helper Methods
```

```
        /// <summary>
        /// Checks if a given path is actually a drive name.
        /// </summary>
        /// <param name="path">The path to check.</param>
        /// <returns>
        /// True if the path given represents a drive, false otherwise.
        /// </returns>
        private bool PathIsDrive(string path)
        {
            // Remove the drive name and first path separator.  If the
            // path is reduced to nothing, it is a drive. Also if its
            // just a drive then there wont be any path separators
            if (String.IsNullOrEmpty(
                        path.Replace(this.PSDriveInfo.Root, "")) ||
                String.IsNullOrEmpty(
                        path.Replace(this.PSDriveInfo.Root + pathSeparator, ""))

               )
            {
                return true;
            }
            else
            {
                return false;
            }
        } // PathIsDrive

        /// <summary>
        /// Breaks up the path into individual elements.
        /// </summary>
        /// <param name="path">The path to split.</param>
        /// <returns>An array of path segments.</returns>
        private string[] ChunkPath(string path)
        {
            // Normalize the path before splitting
            string normalPath = NormalizePath(path);

            // Return the path with the drive name and first path
            // separator character removed, split by the path separator.
            string pathNoDrive = normalPath.Replace(this.PSDriveInfo.Root
                                        + pathSeparator, "");

            return pathNoDrive.Split(pathSeparator.ToCharArray());
        } // ChunkPath

        /// <summary>
        /// Adapts the path, making sure the correct path separator
        /// character is used.
        /// </summary>
        /// <param name="path"></param>
        /// <returns></returns>
        private string NormalizePath(string path)
        {
            string result = path;
```

```
            if (!String.IsNullOrEmpty(path))
            {
                result = path.Replace("/", pathSeparator);
            }

            return result;
        } // NormalizePath

        /// <summary>
        /// Ensures that the drive is removed from the specified path
        /// </summary>
        ///
        /// <param name="path">Path from which drive needs to be removed</param>
        /// <returns>Path with drive information removed</returns>
        private string RemoveDriveFromPath(string path)
        {
            string result = path;
            string root;

            if (this.PSDriveInfo == null)
            {
                root = String.Empty;
            }
            else
            {
                root = this.PSDriveInfo.Root;
            }

            if (result == null)
            {
                result = String.Empty;
            }

            if (result.Contains(root))
            {
                result = result.Substring(result.IndexOf(root,
StringComparison.OrdinalIgnoreCase) + root.Length);
            }

            return result;
        }

        /// <summary>
        /// Chunks the path and returns the table name and the row number
        /// from the path
        /// </summary>
        /// <param name="path">Path to chunk and obtain information</param>
        /// <param name="tableName">Name of the table as represented in the
        /// path</param>
        /// <param name="rowNumber">Row number obtained from the path</param>
        /// <returns>what the path represents</returns>
        private PathType GetNamesFromPath(string path, out string tableName, out int
rowNumber)
        {
            PathType retVal = PathType.Invalid;
```

```
            rowNumber = -1;
            tableName = null;

            // Check if the path specified is a drive
            if (PathIsDrive(path))
            {
                return PathType.Database;
            }

            // chunk the path into parts
            string[] pathChunks = ChunkPath(path);

            switch (pathChunks.Length)
            {
                case 1:
                    {
                        string name = pathChunks[0];

                        if (TableNameIsValid(name))
                        {
                            tableName = name;
                            retVal = PathType.Table;
                        }
                    }
                    break;

                case 2:
                    {
                        string name = pathChunks[0];

                        if (TableNameIsValid(name))
                        {
                            tableName = name;
                        }

                        int number = SafeConvertRowNumber(pathChunks[1]);

                        if (number >= 0)
                        {
                            rowNumber = number;
                            retVal = PathType.Row;
                        }
                        else
                        {
                            WriteError(new ErrorRecord(
                                new ArgumentException("Row number is not valid"),
                                "RowNumberNotValid",
                                ErrorCategory.InvalidArgument,
                                path));
                        }
                    }
                    break;

                default:
                    {
```

```
                    WriteError(new ErrorRecord(
                        new ArgumentException("The path supplied has too many
segments"),
                        "PathNotValid",
                        ErrorCategory.InvalidArgument,
                        path));
                }
                break;
            } // switch(pathChunks...

            return retVal;
        } // GetNamesFromPath

        /// <summary>
        /// Throws an argument exception stating that the specified path does
        /// not represent either a table or a row
        /// </summary>
        /// <param name="path">path which is invalid</param>
        private void ThrowTerminatingInvalidPathException(string path)
        {
            StringBuilder message = new StringBuilder("Path must represent either a
table or a row :");
            message.Append(path);

            throw new ArgumentException(message.ToString());
        }

        /// <summary>
        /// Retrieve the list of tables from the database.
        /// </summary>
        /// <returns>
        /// Collection of DatabaseTableInfo objects, each object representing
        /// information about one database table
        /// </returns>
        private Collection<DatabaseTableInfo> GetTables()
        {
            Collection<DatabaseTableInfo> results =
                    new Collection<DatabaseTableInfo>();

            // using ODBC connection to the database and get the schema of tables
            AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;

            if (di == null)
            {
                return null;
            }

            OdbcConnection connection = di.Connection;
            DataTable dt = connection.GetSchema("Tables");
            int count;

            // iterate through all rows in the schema and create DatabaseTableInfo
            // objects which represents a table
            foreach (DataRow dr in dt.Rows)
            {
```

```
            String tableName = dr["TABLE_NAME"] as String;
            DataColumnCollection columns = null;

            // find the number of rows in the table
            try
            {
                String cmd = "Select count(*) from \"" + tableName + "\"";
                OdbcCommand command = new OdbcCommand(cmd, connection);

                count = (Int32)command.ExecuteScalar();
            }
            catch
            {
                count = 0;
            }

            // create DatabaseTableInfo object representing the table
            DatabaseTableInfo table =
                    new DatabaseTableInfo(dr, tableName, count, columns);

            results.Add(table);
        } // foreach (DataRow...

        return results;
    } // GetTables

    /// <summary>
    /// Return row information from a specified table.
    /// </summary>
    /// <param name="tableName">The name of the database table from
    /// which to retrieve rows.</param>
    /// <returns>Collection of row information objects.</returns>
    private Collection<DatabaseRowInfo> GetRows(string tableName)
    {
        Collection<DatabaseRowInfo> results =
                    new Collection<DatabaseRowInfo>();

        // Obtain rows in the table and add it to the collection
        try
        {
            OdbcDataAdapter da = GetAdapterForTable(tableName);

            if (da == null)
            {
                return null;
            }

            DataSet ds = GetDataSetForTable(da, tableName);
            DataTable table = GetDataTable(ds, tableName);

            int i = 0;
            foreach (DataRow row in table.Rows)
            {
                results.Add(new DatabaseRowInfo(row,
i.ToString(CultureInfo.CurrentCulture)));
```

```
                i++;
            } // foreach (DataRow...
        }
        catch (Exception e)
        {
            WriteError(new ErrorRecord(e, "CannotAccessSpecifiedRows",
                ErrorCategory.InvalidOperation, tableName));
        }

        return results;

    } // GetRows

    /// <summary>
    /// Retrieve information about a single table.
    /// </summary>
    /// <param name="tableName">The table for which to retrieve
    /// data.</param>
    /// <returns>Table information.</returns>
    private DatabaseTableInfo GetTable(string tableName)
    {
        foreach (DatabaseTableInfo table in GetTables())
        {
            if (String.Equals(tableName, table.Name,
StringComparison.OrdinalIgnoreCase))
            {
                return table;
            }
        }

        return null;
    } // GetTable

    /// <summary>
    /// Removes the specified table from the database
    /// </summary>
    /// <param name="tableName">Name of the table to remove</param>
    private void RemoveTable(string tableName)
    {
        // validate if tablename is valid and if table is present
        if (String.IsNullOrEmpty(tableName) || !TableNameIsValid(tableName) ||
!TableIsPresent(tableName))
        {
            return;
        }

        // Execute command using ODBC connection to remove a table
        try
        {
            // delete the table using an sql statement
            string sql = "drop table " + tableName;

            AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;

            if (di == null)
```

```
            {
                return;
            }
            OdbcConnection connection = di.Connection;

            OdbcCommand cmd = new OdbcCommand(sql, connection);
            cmd.ExecuteScalar();
        }
        catch (Exception ex)
        {
            WriteError(new ErrorRecord(ex, "CannotRemoveSpecifiedTable",
                      ErrorCategory.InvalidOperation, null)
                      );
        }

    } // RemoveTable

    /// <summary>
    /// Obtain a data adapter for the specified Table
    /// </summary>
    /// <param name="tableName">Name of the table to obtain the
    /// adapter for</param>
    /// <returns>Adapter object for the specified table</returns>
    /// <remarks>An adapter serves as a bridge between a DataSet (in memory
    /// representation of table) and the data source</remarks>
    private OdbcDataAdapter GetAdapterForTable(string tableName)
    {
        OdbcDataAdapter da = null;
        AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;

        if (di == null || !TableNameIsValid(tableName) ||
!TableIsPresent(tableName))
        {
            return null;
        }

        OdbcConnection connection = di.Connection;

        try
        {
            // Create a odbc data adpater. This can be sued to update the
            // data source with the records that will be created here
            // using data sets
            string sql = "Select * from " + tableName;
            da = new OdbcDataAdapter(new OdbcCommand(sql, connection));

            // Create a odbc command builder object. This will create sql
            // commands automatically for a single table, thus
            // eliminating the need to create new sql statements for
            // every operation to be done.
            OdbcCommandBuilder cmd = new OdbcCommandBuilder(da);

            // Set the delete cmd for the table here
            sql = "Delete from " + tableName + " where ID = ?";
            da.DeleteCommand = new OdbcCommand(sql, connection);
```

```
            // Specify a DeleteCommand parameter based on the "ID"
            // column
            da.DeleteCommand.Parameters.Add(new OdbcParameter());
            da.DeleteCommand.Parameters[0].SourceColumn = "ID";

            // Create an InsertCommand based on the sql string
            // Insert into "tablename" values (?,?,?)" where
            // ? represents a column in the table. Note that
            // the number of ? will be equal to the number of
            // columnds
            DataSet ds = new DataSet();

            da.FillSchema(ds, SchemaType.Source);
            ds.Locale = CultureInfo.InvariantCulture;

            sql = "Insert into " + tableName + " values ( ";
            for (int i = 0; i < ds.Tables["Table"].Columns.Count; i++)
            {
                sql += "?, ";
            }
            sql = sql.Substring(0, sql.Length - 2);
            sql += ")";
            da.InsertCommand = new OdbcCommand(sql, connection);

            // Create parameters for the InsertCommand based on the
            // captions of each column
            for (int i = 0; i < ds.Tables["Table"].Columns.Count; i++)
            {
                da.InsertCommand.Parameters.Add(new OdbcParameter());
                da.InsertCommand.Parameters[i].SourceColumn =
                                ds.Tables["Table"].Columns[i].Caption;

            }

            // Open the connection if its not already open
            if (connection.State != ConnectionState.Open)
            {
                connection.Open();
            }
        }
        catch (Exception e)
        {
            WriteError(new ErrorRecord(e, "CannotAccessSpecifiedTable",
              ErrorCategory.InvalidOperation, tableName));
        }

        return da;
    } // GetAdapterForTable

    /// <summary>
    /// Gets the DataSet (in memory representation) for the table
    /// for the specified adapter
    /// </summary>
    /// <param name="adapter">Adapter to be used for obtaining
```

```
        /// the table</param>
        /// <param name="tableName">Name of the table for which a
        /// DataSet is required</param>
        /// <returns>The DataSet with the filled in schema</returns>
        private DataSet GetDataSetForTable(OdbcDataAdapter adapter, string tableName)
        {
            Debug.Assert(adapter != null);

            // Create a dataset object which will provide an in-memory
            // representation of the data being worked upon in the
            // data source.
            DataSet ds = new DataSet();

            // Create a table named "Table" which will contain the same
            // schema as in the data source.
            //adapter.FillSchema(ds, SchemaType.Source);
            adapter.Fill(ds, tableName);
            ds.Locale = CultureInfo.InvariantCulture;

            return ds;
        } //GetDataSetForTable

        /// <summary>
        /// Get the DataTable object which can be used to operate on
        /// for the specified table in the data source
        /// </summary>
        /// <param name="ds">DataSet object which contains the tables
        /// schema</param>
        /// <param name="tableName">Name of the table</param>
        /// <returns>Corresponding DataTable object representing
        /// the table</returns>
        ///
        private DataTable GetDataTable(DataSet ds, string tableName)
        {
            Debug.Assert(ds != null);
            Debug.Assert(tableName != null);

            DataTable table = ds.Tables[tableName];
            table.Locale = CultureInfo.InvariantCulture;

            return table;
        } // GetDataTable

        /// <summary>
        /// Retrieves a single row from the named table.
        /// </summary>
        /// <param name="tableName">The table that contains the
        /// numbered row.</param>
        /// <param name="row">The index of the row to return.</param>
        /// <returns>The specified table row.</returns>
        private DatabaseRowInfo GetRow(string tableName, int row)
        {
            Collection<DatabaseRowInfo> di = GetRows(tableName);

            // if the row is invalid write an appropriate error else return the
```

```
            // corresponding row information
            if (row < di.Count && row >= 0)
            {
                return di[row];
            }
            else
            {
                WriteError(new ErrorRecord(
                   new ItemNotFoundException(),
                   "RowNotFound",
                   ErrorCategory.ObjectNotFound,
                   row.ToString(CultureInfo.CurrentCulture))
                );
            }

            return null;
        } // GetRow

        /// <summary>
        /// Method to safely convert a string representation of a row number
        /// into its Int32 equivalent
        /// </summary>
        /// <param name="rowNumberAsStr">String representation of the row
        /// number</param>
        /// <remarks>If there is an exception, -1 is returned</remarks>
        private int SafeConvertRowNumber(string rowNumberAsStr)
        {
            int rowNumber = -1;
            try
            {
                rowNumber = Convert.ToInt32(rowNumberAsStr,
CultureInfo.CurrentCulture);
            }
            catch (FormatException fe)
            {
                WriteError(new ErrorRecord(fe, "RowStringFormatNotValid",
                    ErrorCategory.InvalidData, rowNumberAsStr));
            }
            catch (OverflowException oe)
            {
                WriteError(new ErrorRecord(oe, "RowStringConversionToNumberFailed",
                    ErrorCategory.InvalidData, rowNumberAsStr));
            }

            return rowNumber;
        } // SafeConvertRowNumber

        /// <summary>
        /// Check if a table name is valid
        /// </summary>
        /// <param name="tableName">Table name to validate</param>
        /// <remarks>Helps to check for SQL injection attacks</remarks>
        private bool TableNameIsValid(string tableName)
        {
            Regex exp = new Regex(pattern, RegexOptions.Compiled |
```

```
RegexOptions.IgnoreCase);

            if (exp.IsMatch(tableName))
            {
                return true;
            }
            WriteError(new ErrorRecord(
                new ArgumentException("Table name not valid"), "TableNameNotValid",
                     ErrorCategory.InvalidArgument, tableName));
            return false;
        } // TableNameIsValid

        /// <summary>
        /// Checks to see if the specified table is present in the
        /// database
        /// </summary>
        /// <param name="tableName">Name of the table to check</param>
        /// <returns>true, if table is present, false otherwise</returns>
        private bool TableIsPresent(string tableName)
        {
            // using ODBC connection to the database and get the schema of tables
            AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;
            if (di == null)
            {
                return false;
            }

            OdbcConnection connection = di.Connection;
            DataTable dt = connection.GetSchema("Tables");

            // check if the specified tableName is available
            // in the list of tables present in the database
            foreach (DataRow dr in dt.Rows)
            {
                string name = dr["TABLE_NAME"] as string;
                if (name.Equals(tableName, StringComparison.OrdinalIgnoreCase))
                {
                    return true;
                }
            }

            WriteError(new ErrorRecord(
                new ArgumentException("Specified Table is not present in database"),
"TableNotAvailable",
                     ErrorCategory.InvalidArgument, tableName));

            return false;
        }// TableIsPresent

        /// <summary>
        /// Gets the next available ID in the table
        /// </summary>
        /// <param name="table">DataTable object representing the table to
        /// search for ID</param>
        /// <returns>next available id</returns>
```

```
    private int GetNextID(DataTable table)
    {
        int big = 0;

        for (int i = 0; i < table.Rows.Count; i++)
        {
            DataRow row = table.Rows[i];

            int id = (int)row["ID"];

            if (big < id)
            {
                big = id;
            }
        }

        big++;
        return big;
    }

    #endregion Helper Methods

    #region Private Properties

    private string pathSeparator = "\\";
    private static string pattern = @"^[a-z]+[0-9]*_*$";

     private enum PathType { Database, Table, Row, Invalid };

    #endregion Private Properties

} // AccessDBProvider

 #endregion AccessDBProvider

#region Helper Classes

#region AccessDBPSDriveInfo

/// <summary>
/// Any state associated with the drive should be held here.
/// In this case, it's the connection to the database.
/// </summary>
internal class AccessDBPSDriveInfo : PSDriveInfo
{
    private OdbcConnection connection;

    /// <summary>
    /// ODBC connection information.
    /// </summary>
    public OdbcConnection Connection
    {
        get { return connection; }
        set { connection = value; }
    }
```

```
    /// <summary>
    /// Constructor that takes one argument
    /// </summary>
    /// <param name="driveInfo">Drive provided by this provider</param>
    public AccessDBPSDriveInfo(PSDriveInfo driveInfo)
        : base(driveInfo)
    { }

} // class AccessDBPSDriveInfo

#endregion AccessDBPSDriveInfo

#region DatabaseTableInfo

/// <summary>
/// Contains information specific to the database table.
/// Similar to the DirectoryInfo class.
/// </summary>
public class DatabaseTableInfo
{
    /// <summary>
    /// Row from the "tables" schema
    /// </summary>
    public DataRow Data
    {
        get
        {
            return data;
        }
        set
        {
            data = value;
        }
    }
    private DataRow data;

    /// <summary>
    /// The table name.
    /// </summary>
    public string Name
    {
        get
        {
            return name;
        }
        set
        {
            name = value;
        }
    }
    private String name;

    /// <summary>
    /// The number of rows in the table.
```

```
        /// </summary>
        public int RowCount
        {
            get
            {
                return rowCount;
            }
            set
            {
                rowCount = value;
            }
        }
        private int rowCount;

        /// <summary>
        /// The column definitions for the table.
        /// </summary>
        public DataColumnCollection Columns
        {
            get
            {
                return columns;
            }
            set
            {
                columns = value;
            }
        }
        private DataColumnCollection columns;

        /// <summary>
        /// Constructor.
        /// </summary>
        /// <param name="row">The row definition.</param>
        /// <param name="name">The table name.</param>
        /// <param name="rowCount">The number of rows in the table.</param>
        /// <param name="columns">Information on the column tables.</param>
        public DatabaseTableInfo(DataRow row, string name, int rowCount,
                                 DataColumnCollection columns)
        {
            Name = name;
            Data = row;
            RowCount = rowCount;
            Columns = columns;
        } // DatabaseTableInfo
    } // class DatabaseTableInfo

    #endregion DatabaseTableInfo

    #region DatabaseRowInfo

    /// <summary>
    /// Contains information specific to an individual table row.
    /// Analogous to the FileInfo class.
    /// </summary>
```

```
    public class DatabaseRowInfo
    {
        /// <summary>
        /// Row data information.
        /// </summary>
        public DataRow Data
        {
            get
            {
                return data;
            }
            set
            {
                data = value;
            }
        }
        private DataRow data;

        /// <summary>
        /// The row index.
        /// </summary>
        public string RowNumber
        {
            get
            {
                return rowNumber;
            }
            set
            {
                rowNumber = value;
            }
        }
        private string rowNumber;

        /// <summary>
        /// Constructor.
        /// </summary>
        /// <param name="row">The row information.</param>
        /// <param name="name">The row index.</param>
        public DatabaseRowInfo(DataRow row, string name)
        {
            RowNumber = name;
            Data = row;
        } // DatabaseRowInfo
    } // class DatabaseRowInfo

    #endregion DatabaseRowInfo

    #endregion Helper Classes
}
```

See Also

System.Management.Automation.Provider.ItemCmdletProvider

System.Management.Automation.Provider.ContainerCmdletProvider

System.Management.Automation.Provider.NavigationCmdletProvider

Designing Your Windows PowerShell Provider

Last updated on 05/20/2025

AccessDBProviderSample06

This sample shows how to overwrite content methods to support calls to the `Clear-Content`, `Get-Content`, and `Set-Content` cmdlets. These methods should be implemented when the user needs to manage the content of the items in the data store. The provider class in this sample derives from the System.Management.Automation.Provider.NavigationCmdletProvider class, and it implements the System.Management.Automation.Provider.IContentCmdletProvider interface.

Demonstrates

> **Important**
>
> Your provider class will most likely derive from one of the following classes and possibly implement other provider interfaces:
>
> - System.Management.Automation.Provider.ItemCmdletProvider class. See AccessDBProviderSample03.
> - System.Management.Automation.Provider.ContainerCmdletProvider class. See AccessDBProviderSample04.
> - System.Management.Automation.Provider.NavigationCmdletProvider class.
>
> For more information about choosing which provider class to derive from based on provider features, see Designing Your Windows PowerShell Provider.

This sample demonstrates the following:

- Declaring the `CmdletProvider` attribute.
- Defining a provider class that derives from the System.Management.Automation.Provider.NavigationCmdletProvider class and that declares the System.Management.Automation.Provider.IContentCmdletProvider interface.
- Overwriting the System.Management.Automation.Provider.IContentCmdletProvider.ClearContent* method to change the behavior of the `Clear-Content` cmdlet, allowing the user to remove the content from an item. (This sample does not show how to add dynamic parameters to the `Clear-Content` cmdlet.)

- Overwriting the System.Management.Automation.Provider.IContentCmdletProvider.GetContentReader* method to change the behavior of the `Get-Content` cmdlet, allowing the user to retrieve the content of an item. (This sample does not show how to add dynamic parameters to the `Get-Content` cmdlet.).
- Overwriting the Microsoft.PowerShell.Commands.FileSystemProvider.GetContentWriter* method to change the behavior of the `Set-Content` cmdlet, allowing the user to update the content of an item. (This sample does not show how to add dynamic parameters to the `Set-Content` cmdlet.)

Example

This sample shows how to overwrite the methods needed to clear, get, and set the content of items in a Microsoft Access data base.

C#

```
using System;
using System.IO;
using System.Data;
using System.Data.Odbc;
using System.Diagnostics;
using System.Collections;
using System.Collections.ObjectModel;
using System.Management.Automation;
using System.Management.Automation.Provider;
using System.Text;
using System.Text.RegularExpressions;
using System.ComponentModel;
using System.Globalization;

namespace Microsoft.Samples.PowerShell.Providers
{
   #region AccessDBProvider

   /// <summary>
   /// This example implements the content methods.
   /// </summary>
   [CmdletProvider("AccessDB", ProviderCapabilities.None)]
   public class AccessDBProvider : NavigationCmdletProvider, IContentCmdletProvider
   {

       #region Drive Manipulation

        /// <summary>
        /// Create a new drive.  Create a connection to the database file and set
        /// the Connection property in the PSDriveInfo.
        /// </summary>
```

```
        /// <param name="drive">
        /// Information describing the drive to add.
        /// </param>
        /// <returns>The added drive.</returns>
        protected override PSDriveInfo NewDrive(PSDriveInfo drive)
        {
            // check if drive object is null
            if (drive == null)
            {
                WriteError(new ErrorRecord(
                    new ArgumentNullException("drive"),
                    "NullDrive",
                    ErrorCategory.InvalidArgument,
                    null)
                );

                return null;
            }

            // check if drive root is not null or empty
            // and if its an existing file
            if (String.IsNullOrEmpty(drive.Root) || (File.Exists(drive.Root) ==
false))
            {
                WriteError(new ErrorRecord(
                    new ArgumentException("drive.Root"),
                    "NoRoot",
                    ErrorCategory.InvalidArgument,
                    drive)
                );

                return null;
            }

            // create a new drive and create an ODBC connection to the new drive
            AccessDBPSDriveInfo accessDBPSDriveInfo = new
AccessDBPSDriveInfo(drive);

            OdbcConnectionStringBuilder builder = new
OdbcConnectionStringBuilder();

            builder.Driver = "Microsoft Access Driver (*.mdb)";
            builder.Add("DBQ", drive.Root);

            OdbcConnection conn = new OdbcConnection(builder.ConnectionString);
            conn.Open();
            accessDBPSDriveInfo.Connection = conn;

            return accessDBPSDriveInfo;
        } // NewDrive

        /// <summary>
        /// Removes a drive from the provider.
        /// </summary>
        /// <param name="drive">The drive to remove.</param>
```

```
        /// <returns>The drive removed.</returns>
        protected override PSDriveInfo RemoveDrive(PSDriveInfo drive)
        {
            // check if drive object is null
            if (drive == null)
            {
                WriteError(new ErrorRecord(
                    new ArgumentNullException("drive"),
                    "NullDrive",
                    ErrorCategory.InvalidArgument,
                    drive)
                );

                return null;
            }

            // close ODBC connection to the drive
            AccessDBPSDriveInfo accessDBPSDriveInfo = drive as AccessDBPSDriveInfo;

            if (accessDBPSDriveInfo == null)
            {
                return null;
            }
            accessDBPSDriveInfo.Connection.Close();

            return accessDBPSDriveInfo;
        } // RemoveDrive

        #endregion Drive Manipulation

        #region Item Methods

        /// <summary>
        /// Retrieves an item using the specified path.
        /// </summary>
        /// <param name="path">The path to the item to return.</param>
        protected override void GetItem(string path)
        {
            // check if the path represented is a drive
            if (PathIsDrive(path))
            {
                WriteItemObject(this.PSDriveInfo, path, true);
                return;
            }// if (PathIsDrive...

            // Get table name and row information from the path and do
            // necessary actions
            string tableName;
            int rowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            if (type == PathType.Table)
            {
                DatabaseTableInfo table = GetTable(tableName);
```

```
            WriteItemObject(table, path, true);
        }
        else if (type == PathType.Row)
        {
            DatabaseRowInfo row = GetRow(tableName, rowNumber);
            WriteItemObject(row, path, false);
        }
        else
        {
            ThrowTerminatingInvalidPathException(path);
        }

    } // GetItem

    /// <summary>
    /// Set the content of a row of data specified by the supplied path
    /// parameter.
    /// </summary>
    /// <param name="path">Specifies the path to the row whose columns
    /// will be updated.</param>
    /// <param name="values">Comma separated string of values</param>
    protected override void SetItem(string path, object values)
    {
        // Get type, table name and row number from the path specified
        string tableName;
        int rowNumber;

        PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

        if (type != PathType.Row)
        {
            WriteError(new ErrorRecord(new NotSupportedException(
                  "SetNotSupported"), "",
               ErrorCategory.InvalidOperation, path));

            return;
        }

        // Get in-memory representation of table
        OdbcDataAdapter da = GetAdapterForTable(tableName);

        if (da == null)
        {
            return;
        }
        DataSet ds = GetDataSetForTable(da, tableName);
        DataTable table = GetDataTable(ds, tableName);

        if (rowNumber >= table.Rows.Count)
        {
            // The specified row number has to be available. If not
            // NewItem has to be used to add a new row
            throw new ArgumentException("Row specified is not available");
        } // if (rowNum...
```

```
            string[] colValues = (values as string).Split(',');

            // set the specified row
            DataRow row = table.Rows[rowNumber];

            for (int i = 0; i < colValues.Length; i++)
            {
                row[i] = colValues[i];
            }

            // Update the table
            if (ShouldProcess(path, "SetItem"))
            {
                da.Update(ds, tableName);
            }

        } // SetItem

        /// <summary>
        /// Test to see if the specified item exists.
        /// </summary>
        /// <param name="path">The path to the item to verify.</param>
        /// <returns>True if the item is found.</returns>
        protected override bool ItemExists(string path)
        {
            // check if the path represented is a drive
            if (PathIsDrive(path))
            {
                return true;
            }

            // Obtain type, table name and row number from path
            string tableName;
            int rowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            DatabaseTableInfo table = GetTable(tableName);

            if (type == PathType.Table)
            {
                // if specified path represents a table then DatabaseTableInfo
                // object for the same should exist
                if (table != null)
                {
                    return true;
                }
            }
            else if (type == PathType.Row)
            {
                // if specified path represents a row then DatabaseTableInfo should
                // exist for the table and then specified row number must be within
                // the maximum row count in the table
                if (table != null && rowNumber < table.RowCount)
                {
```

```
                return true;
            }
        }

        return false;

    } // ItemExists

    /// <summary>
    /// Test to see if the specified path is syntactically valid.
    /// </summary>
    /// <param name="path">The path to validate.</param>
    /// <returns>True if the specified path is valid.</returns>
    protected override bool IsValidPath(string path)
    {
        bool result = true;

        // check if the path is null or empty
        if (String.IsNullOrEmpty(path))
        {
            result = false;
        }

        // convert all separators in the path to a uniform one
        path = NormalizePath(path);

        // split the path into individual chunks
        string[] pathChunks = path.Split(pathSeparator.ToCharArray());

        foreach (string pathChunk in pathChunks)
        {
            if (pathChunk.Length == 0)
            {
                result = false;
            }
        }
        return result;
    } // IsValidPath

    #endregion Item Overloads

#region Container Overloads

    /// <summary>
    /// Return either the tables in the database or the datarows
    /// </summary>
    /// <param name="path">The path to the parent</param>
    /// <param name="recurse">True to return all child items recursively.
    /// </param>
    protected override void GetChildItems(string path, bool recurse)
    {
        // If path represented is a drive then the children in the path are
        // tables. Hence all tables in the drive represented will have to be
        // returned
        if (PathIsDrive(path))
```

```
            {
                foreach (DatabaseTableInfo table in GetTables())
                {
                    WriteItemObject(table, path, true);

                    // if the specified item exists and recurse has been set then
                    // all child items within it have to be obtained as well
                    if (ItemExists(path) && recurse)
                    {
                        GetChildItems(path + pathSeparator + table.Name, recurse);
                    }
                } // foreach (DatabaseTableInfo...
            } // if (PathIsDrive...
            else
            {
                // Get the table name, row number and type of path from the
                // path specified
                string tableName;
                int rowNumber;

                PathType type = GetNamesFromPath(path, out tableName, out
rowNumber);

                if (type == PathType.Table)
                {
                    // Obtain all the rows within the table
                    foreach (DatabaseRowInfo row in GetRows(tableName))
                    {
                        WriteItemObject(row, path + pathSeparator + row.RowNumber,
                                false);
                    } // foreach (DatabaseRowInfo...
                }
                else if (type == PathType.Row)
                {
                    // In this case the user has directly specified a row, hence
                    // just give that particular row
                    DatabaseRowInfo row = GetRow(tableName, rowNumber);
                    WriteItemObject(row, path + pathSeparator + row.RowNumber,
                                false);
                }
                else
                {
                    // In this case, the path specified is not valid
                    ThrowTerminatingInvalidPathException(path);
                }
            } // else
        } // GetChildItems

        /// <summary>
        /// Return the names of all child items.
        /// </summary>
        /// <param name="path">The root path.</param>
        /// <param name="returnContainers">Not used.</param>
        protected override void GetChildNames(string path,
                                      ReturnContainers returnContainers)
```

```
        {
            // If the path represented is a drive, then the child items are
            // tables. get the names of all the tables in the drive.
            if (PathIsDrive(path))
            {
                foreach (DatabaseTableInfo table in GetTables())
                {
                    WriteItemObject(table.Name, path, true);
                } // foreach (DatabaseTableInfo...
            } // if (PathIsDrive...
            else
            {
                // Get type, table name and row number from path specified
                string tableName;
                int rowNumber;

                PathType type = GetNamesFromPath(path, out tableName, out
rowNumber);

                if (type == PathType.Table)
                {
                    // Get all the rows in the table and then write out the
                    // row numbers.
                    foreach (DatabaseRowInfo row in GetRows(tableName))
                    {
                        WriteItemObject(row.RowNumber, path, false);
                    } // foreach (DatabaseRowInfo...
                }
                else if (type == PathType.Row)
                {
                    // In this case the user has directly specified a row, hence
                    // just give that particular row
                    DatabaseRowInfo row = GetRow(tableName, rowNumber);

                    WriteItemObject(row.RowNumber, path, false);
                }
                else
                {
                    ThrowTerminatingInvalidPathException(path);
                }
            } // else
        } // GetChildNames

        /// <summary>
        /// Determines if the specified path has child items.
        /// </summary>
        /// <param name="path">The path to examine.</param>
        /// <returns>
        /// True if the specified path has child items.
        /// </returns>
        protected override bool HasChildItems(string path)
        {
            if (PathIsDrive(path))
            {
                return true;
```

```
            }

            return (ChunkPath(path).Length == 1);
        } // HasChildItems

        /// <summary>
        /// Creates a new item at the specified path.
        /// </summary>
        ///
        /// <param name="path">
        /// The path to the new item.
        /// </param>
        ///
        /// <param name="type">
        /// Type for the object to create. "Table" for creating a new table and
        /// "Row" for creating a new row in a table.
        /// </param>
        ///
        /// <param name="newItemValue">
        /// Object for creating new instance of a type at the specified path. For
        /// creating a "Table" the object parameter is ignored and for creating
        /// a "Row" the object must be of type string which will contain comma
        /// separated values of the rows to insert.
        /// </param>
        protected override void NewItem(string path, string type,
                                        object newItemValue)
        {
            string tableName;
            int rowNumber;

            PathType pt = GetNamesFromPath(path, out tableName, out rowNumber);

            if (pt == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(path);
            }

            // Check if type is either "table" or "row", if not throw an
            // exception
            if (!String.Equals(type, "table", StringComparison.OrdinalIgnoreCase)
                && !String.Equals(type, "row", StringComparison.OrdinalIgnoreCase))
            {
                WriteError(new ErrorRecord
                                    (new ArgumentException("Type must be either a
table or row"),
                                       "CannotCreateSpecifiedObject",
                                          ErrorCategory.InvalidArgument,
                                               path
                                     )
                                );

                throw new ArgumentException("This provider can only create items of
type \"table\" or \"row\"");
            }
```

```
            // Path type is the type of path of the container. So if a drive
            // is specified, then a table can be created under it and if a table
            // is specified, then a row can be created under it. For the sake of
            // completeness, if a row is specified, then if the row specified by
            // the path does not exist, a new row is created. However, the row
            // number may not match as the row numbers only get incremented based
            // on the number of rows

            if (PathIsDrive(path))
            {
                if (String.Equals(type, "table",
StringComparison.OrdinalIgnoreCase))
                {
                    // Execute command using ODBC connection to create a table
                    try
                    {
                        // create the table using an sql statement
                        string newTableName = newItemValue.ToString();
                        string sql = "create table " + newTableName
                                            + " (ID INT)";

                        // Create the table using the Odbc connection from the
                        // drive.
                        AccessDBPSDriveInfo di = this.PSDriveInfo as
AccessDBPSDriveInfo;

                        if (di == null)
                        {
                            return;
                        }
                        OdbcConnection connection = di.Connection;

                        if (ShouldProcess(newTableName, "create"))
                        {
                            OdbcCommand cmd = new OdbcCommand(sql, connection);
                            cmd.ExecuteScalar();
                        }
                    }
                    catch (Exception ex)
                    {
                        WriteError(new ErrorRecord(ex,
"CannotCreateSpecifiedTable",
                                    ErrorCategory.InvalidOperation, path)
                                    );
                    }
                } // if (String...
                else if (String.Equals(type, "row",
StringComparison.OrdinalIgnoreCase))
                {
                    throw new
                        ArgumentException("A row cannot be created under a
database, specify a path that represents a Table");
                }
            }// if (PathIsDrive...
            else
```

```
            {
                if (String.Equals(type, "table",
StringComparison.OrdinalIgnoreCase))
                {
                    if (rowNumber < 0)
                    {
                        throw new
                            ArgumentException("A table cannot be created within
another table, specify a path that represents a database");
                    }
                    else
                    {
                        throw new
                            ArgumentException("A table cannot be created inside a
row, specify a path that represents a database");
                    }
                } //if (String.Equals....
                // if path specified is a row, create a new row
                else if (String.Equals(type, "row",
StringComparison.OrdinalIgnoreCase))
                {
                    // The user is required to specify the values to be inserted
                    // into the table in a single string separated by commas
                    string value = newItemValue as string;

                    if (String.IsNullOrEmpty(value))
                    {
                        throw new
                            ArgumentException("Value argument must have comma
separated values of each column in a row");
                    }
                    string[] rowValues = value.Split(',');

                    OdbcDataAdapter da = GetAdapterForTable(tableName);

                    if (da == null)
                    {
                        return;
                    }

                    DataSet ds = GetDataSetForTable(da, tableName);
                    DataTable table = GetDataTable(ds, tableName);

                    if (rowValues.Length != table.Columns.Count)
                    {
                        string message = String.Format(CultureInfo.CurrentCulture,
                                                       "The table has {0} columns and the
value specified must have so many comma separated values",
                                                       table.Columns.Count);

                        throw new ArgumentException(message);
                    }

                    if (!Force && (rowNumber >= 0 && rowNumber < table.Rows.Count))
                    {
```

```
                        string message = String.Format(CultureInfo.CurrentCulture,
                                                       "The row {0} already exists. To create
a new row specify row number as {1}, or specify path to a table, or use the -Force
parameter",
                                                       rowNumber, table.Rows.Count);

                        throw new ArgumentException(message);
                    }

                    if (rowNumber > table.Rows.Count)
                    {
                        string message = String.Format(CultureInfo.CurrentCulture,
                                                       "To create a new row specify row number
as {0}, or specify path to a table",
                                                        table.Rows.Count);

                        throw new ArgumentException(message);
                    }

                    // Create a new row and update the row with the input
                    // provided by the user
                    DataRow row = table.NewRow();
                    for (int i = 0; i < rowValues.Length; i++)
                    {
                        row[i] = rowValues[i];
                    }
                    table.Rows.Add(row);

                    if (ShouldProcess(tableName, "update rows"))
                    {
                        // Update the table from memory back to the data source
                        da.Update(ds, tableName);
                    }

                }// else if (String...
            }// else ...

        } // NewItem

        /// <summary>
        /// Copies an item at the specified path to the location specified
        /// </summary>
        ///
        /// <param name="path">
        /// Path of the item to copy
        /// </param>
        ///
        /// <param name="copyPath">
        /// Path of the item to copy to
        /// </param>
        ///
        /// <param name="recurse">
        /// Tells the provider to recurse subcontainers when copying
        /// </param>
        ///
```

```
        protected override void CopyItem(string path, string copyPath, bool
recurse)
        {
            string tableName, copyTableName;
            int rowNumber, copyRowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);
            PathType copyType = GetNamesFromPath(copyPath, out copyTableName, out
copyRowNumber);

            if (type == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(path);
            }

            if (type == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(copyPath);
            }

            // Get the table and the table to copy to
            OdbcDataAdapter da = GetAdapterForTable(tableName);
            if (da == null)
            {
                return;
            }

            DataSet ds = GetDataSetForTable(da, tableName);
            DataTable table = GetDataTable(ds, tableName);

            OdbcDataAdapter cda = GetAdapterForTable(copyTableName);
            if (cda == null)
            {
                return;
            }

            DataSet cds = GetDataSetForTable(cda, copyTableName);
            DataTable copyTable = GetDataTable(cds, copyTableName);

            // if source represents a table
            if (type == PathType.Table)
            {
                // if copyPath does not represent a table
                if (copyType != PathType.Table)
                {
                    ArgumentException e = new ArgumentException("Table can only be
copied on to another table location");

                    WriteError(new ErrorRecord(e, "PathNotValid",
                        ErrorCategory.InvalidArgument, copyPath));

                    throw e;
                }

                // if table already exists then force parameter should be set
```

```
                // to force a copy
                if (!Force && GetTable(copyTableName) != null)
                {
                    throw new ArgumentException("Specified path already exists");
                }

                for (int i = 0; i < table.Rows.Count; i++)
                {
                    DataRow row = table.Rows[i];
                    DataRow copyRow = copyTable.NewRow();

                    copyRow.ItemArray = row.ItemArray;
                    copyTable.Rows.Add(copyRow);
                }
            } // if (type == ...
            // if source represents a row
            else
            {
                if (copyType == PathType.Row)
                {
                    if (!Force && (copyRowNumber < copyTable.Rows.Count))
                    {
                        throw new ArgumentException("Specified path already
exists.");
                    }

                    DataRow row = table.Rows[rowNumber];
                    DataRow copyRow = null;

                    if (copyRowNumber < copyTable.Rows.Count)
                    {
                        // copy to an existing row
                        copyRow = copyTable.Rows[copyRowNumber];
                        copyRow.ItemArray = row.ItemArray;
                        copyRow[0] = GetNextID(copyTable);
                    }
                    else if (copyRowNumber == copyTable.Rows.Count)
                    {
                        // copy to the next row in the table that will
                        // be created
                        copyRow = copyTable.NewRow();
                        copyRow.ItemArray = row.ItemArray;
                        copyRow[0] = GetNextID(copyTable);
                        copyTable.Rows.Add(copyRow);
                    }
                    else
                    {
                        // attempting to copy to a nonexistent row or a row
                        // that cannot be created now - throw an exception
                        string message = String.Format(CultureInfo.CurrentCulture,
                                                       "The item cannot be specified to the
copied row. Specify row number as {0}, or specify a path to the table.",
                                                           table.Rows.Count);

                        throw new ArgumentException(message);
```

```
                }
            }
            else
            {
                // destination path specified represents a table,
                // create a new row and copy the item
                DataRow copyRow = copyTable.NewRow();
                copyRow.ItemArray = table.Rows[rowNumber].ItemArray;
                copyRow[0] = GetNextID(copyTable);
                copyTable.Rows.Add(copyRow);
            }
        }

        if (ShouldProcess(copyTableName, "CopyItems"))
        {
            cda.Update(cds, copyTableName);
        }

    } //CopyItem

    /// <summary>
    /// Removes (deletes) the item at the specified path
    /// </summary>
    ///
    /// <param name="path">
    /// The path to the item to remove.
    /// </param>
    ///
    /// <param name="recurse">
    /// True if all children in a subtree should be removed, false if only
    /// the item at the specified path should be removed. Is applicable
    /// only for container (table) items. Its ignored otherwise (even if
    /// specified).
    /// </param>
    ///
    /// <remarks>
    /// There are no elements in this store which are hidden from the user.
    /// Hence this method will not check for the presence of the Force
    /// parameter
    /// </remarks>
    ///
    protected override void RemoveItem(string path, bool recurse)
    {
        string tableName;
        int rowNumber = 0;

        PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

        if (type == PathType.Table)
        {
            // if recurse flag has been specified, delete all the rows as well
            if (recurse)
            {
                OdbcDataAdapter da = GetAdapterForTable(tableName);
                if (da == null)
```

```
                {
                    return;
                }

                DataSet ds = GetDataSetForTable(da, tableName);
                DataTable table = GetDataTable(ds, tableName);

                for (int i = 0; i < table.Rows.Count; i++)
                {
                    table.Rows[i].Delete();
                }

                if (ShouldProcess(path, "RemoveItem"))
                {
                    da.Update(ds, tableName);
                    RemoveTable(tableName);
                }
            }//if (recurse...
            else
            {
                // Remove the table
                if (ShouldProcess(path, "RemoveItem"))
                {
                    RemoveTable(tableName);
                }
            }
        }
        else if (type == PathType.Row)
        {
            OdbcDataAdapter da = GetAdapterForTable(tableName);
            if (da == null)
            {
                return;
            }

            DataSet ds = GetDataSetForTable(da, tableName);
            DataTable table = GetDataTable(ds, tableName);

            table.Rows[rowNumber].Delete();

            if (ShouldProcess(path, "RemoveItem"))
            {
                da.Update(ds, tableName);
            }
        }
        else
        {
            ThrowTerminatingInvalidPathException(path);
        }

    } // RemoveItem

    #endregion Container Overloads

#region Navigation
```

```
        /// <summary>
        /// Determine if the path specified is that of a container.
        /// </summary>
        /// <param name="path">The path to check.</param>
        /// <returns>True if the path specifies a container.</returns>
        protected override bool IsItemContainer(string path)
        {
            if (PathIsDrive(path))
            {
                return true;
            }

            string[] pathChunks = ChunkPath(path);
            string tableName;
            int rowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            if (type == PathType.Table)
            {
                foreach (DatabaseTableInfo ti in GetTables())
                {
                    if (string.Equals(ti.Name, tableName,
StringComparison.OrdinalIgnoreCase))
                    {
                        return true;
                    }
                } // foreach (DatabaseTableInfo...
            } // if (pathChunks...

            return false;
        } // IsItemContainer

        /// <summary>
        /// Get the name of the leaf element in the specified path
        /// </summary>
        ///
        /// <param name="path">
        /// The full or partial provider specific path
        /// </param>
        ///
        /// <returns>
        /// The leaf element in the path
        /// </returns>
        protected override string GetChildName(string path)
        {
            if (PathIsDrive(path))
            {
                return path;
            }

            string tableName;
            int rowNumber;
```

```
            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            if (type == PathType.Table)
            {
                return tableName;
            }
            else if (type == PathType.Row)
            {
                return rowNumber.ToString(CultureInfo.CurrentCulture);
            }
            else
            {
                ThrowTerminatingInvalidPathException(path);
            }

            return null;
        }

        /// <summary>
        /// Removes the child segment of the path and returns the remaining
        /// parent portion
        /// </summary>
        ///
        /// <param name="path">
        /// A full or partial provider specific path. The path may be to an
        /// item that may or may not exist.
        /// </param>
        ///
        /// <param name="root">
        /// The fully qualified path to the root of a drive. This parameter
        /// may be null or empty if a mounted drive is not in use for this
        /// operation.  If this parameter is not null or empty the result
        /// of the method should not be a path to a container that is a
        /// parent or in a different tree than the root.
        /// </param>
        ///
        /// <returns></returns>

        protected override string GetParentPath(string path, string root)
        {
            // If root is specified then the path has to contain
            // the root. If not nothing should be returned
            if (!String.IsNullOrEmpty(root))
            {
                if (!path.Contains(root))
                {
                    return null;
                }
            }

            return path.Substring(0, path.LastIndexOf(pathSeparator,
StringComparison.OrdinalIgnoreCase));
        }

        /// <summary>
```

```
        /// Joins two strings with a provider specific path separator.
        /// </summary>
        ///
        /// <param name="parent">
        /// The parent segment of a path to be joined with the child.
        /// </param>
        ///
        /// <param name="child">
        /// The child segment of a path to be joined with the parent.
        /// </param>
        ///
        /// <returns>
        /// A string that represents the parent and child segments of the path
        /// joined by a path separator.
        /// </returns>

        protected override string MakePath(string parent, string child)
        {
            string result;

            string normalParent = NormalizePath(parent);
            normalParent = RemoveDriveFromPath(normalParent);
            string normalChild = NormalizePath(child);
            normalChild = RemoveDriveFromPath(normalChild);

            if (String.IsNullOrEmpty(normalParent) &&
String.IsNullOrEmpty(normalChild))
            {
                result = String.Empty;
            }
            else if (String.IsNullOrEmpty(normalParent) &&
!String.IsNullOrEmpty(normalChild))
            {
                result = normalChild;
            }
            else if (!String.IsNullOrEmpty(normalParent) &&
String.IsNullOrEmpty(normalChild))
            {
                if (normalParent.EndsWith(pathSeparator,
StringComparison.OrdinalIgnoreCase))
                {
                    result = normalParent;
                }
                else
                {
                    result = normalParent + pathSeparator;
                }
            } // else if (!String...
            else
            {
                if (!normalParent.Equals(String.Empty,
StringComparison.OrdinalIgnoreCase) &&
                    !normalParent.EndsWith(pathSeparator,
StringComparison.OrdinalIgnoreCase))
                {
```

```
                    result = normalParent + pathSeparator;
                }
                else
                {
                    result = normalParent;
                }

                if (normalChild.StartsWith(pathSeparator,
StringComparison.OrdinalIgnoreCase))
                {
                    result += normalChild.Substring(1);
                }
                else
                {
                    result += normalChild;
                }
            } // else

            return result;
        } // MakePath

        /// <summary>
        /// Normalizes the path that was passed in and returns the normalized
        /// path as a relative path to the basePath that was passed.
        /// </summary>
        ///
        /// <param name="path">
        /// A fully qualified provider specific path to an item.  The item
        /// should exist or the provider should write out an error.
        /// </param>
        ///
        /// <param name="basepath">
        /// The path that the return value should be relative to.
        /// </param>
        ///
        /// <returns>
        /// A normalized path that is relative to the basePath that was
        /// passed.  The provider should parse the path parameter, normalize
        /// the path, and then return the normalized path relative to the
        /// basePath.
        /// </returns>

        protected override string NormalizeRelativePath(string path,
                                                         string basepath)
        {
            // Normalize the paths first
            string normalPath = NormalizePath(path);
            normalPath = RemoveDriveFromPath(normalPath);
            string normalBasePath = NormalizePath(basepath);
            normalBasePath = RemoveDriveFromPath(normalBasePath);

            if (String.IsNullOrEmpty(normalBasePath))
            {
                return normalPath;
            }
```

```
            else
            {
                if (!normalPath.Contains(normalBasePath))
                {
                    return null;
                }

                return normalPath.Substring(normalBasePath.Length +
pathSeparator.Length);
            }
        }

        /// <summary>
        /// Moves the item specified by the path to the specified destination
        /// </summary>
        ///
        /// <param name="path">
        /// The path to the item to be moved
        /// </param>
        ///
        /// <param name="destination">
        /// The path of the destination container
        /// </param>

        protected override void MoveItem(string path, string destination)
        {
            // Get type, table name and rowNumber from the path
            string tableName, destTableName;
            int rowNumber, destRowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            PathType destType = GetNamesFromPath(destination, out destTableName,
                                    out destRowNumber);

            if (type == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(path);
            }

            if (destType == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(destination);
            }

            if (type == PathType.Table)
            {
                ArgumentException e = new ArgumentException("Move not supported for
tables");

                WriteError(new ErrorRecord(e, "MoveNotSupported",
                    ErrorCategory.InvalidArgument, path));

                throw e;
            }
```

```
            else
            {
                OdbcDataAdapter da = GetAdapterForTable(tableName);
                if (da == null)
                {
                    return;
                }

                DataSet ds = GetDataSetForTable(da, tableName);
                DataTable table = GetDataTable(ds, tableName);

                OdbcDataAdapter dda = GetAdapterForTable(destTableName);
                if (dda == null)
                {
                    return;
                }

                DataSet dds = GetDataSetForTable(dda, destTableName);
                DataTable destTable = GetDataTable(dds, destTableName);
                DataRow row = table.Rows[rowNumber];

                if (destType == PathType.Table)
                {
                    DataRow destRow = destTable.NewRow();

                    destRow.ItemArray = row.ItemArray;
                }
                else
                {
                    DataRow destRow = destTable.Rows[destRowNumber];

                    destRow.ItemArray = row.ItemArray;
                }

                // Update the changes
                if (ShouldProcess(path, "MoveItem"))
                {
                    WriteItemObject(row, path, false);
                    dda.Update(dds, destTableName);
                }
            }
        }

        #endregion Navigation

        #region Helper Methods

        /// <summary>
        /// Checks if a given path is actually a drive name.
        /// </summary>
        /// <param name="path">The path to check.</param>
        /// <returns>
        /// True if the path given represents a drive, false otherwise.
        /// </returns>
        private bool PathIsDrive(string path)
```

```
        {
            // Remove the drive name and first path separator.  If the
            // path is reduced to nothing, it is a drive. Also if its
            // just a drive then there wont be any path separators
            if (String.IsNullOrEmpty(
                        path.Replace(this.PSDriveInfo.Root, "")) ||
                String.IsNullOrEmpty(
                        path.Replace(this.PSDriveInfo.Root + pathSeparator, ""))

               )
            {
                return true;
            }
            else
            {
                return false;
            }
        } // PathIsDrive

        /// <summary>
        /// Breaks up the path into individual elements.
        /// </summary>
        /// <param name="path">The path to split.</param>
        /// <returns>An array of path segments.</returns>
        private string[] ChunkPath(string path)
        {
            // Normalize the path before splitting
            string normalPath = NormalizePath(path);

            // Return the path with the drive name and first path
            // separator character removed, split by the path separator.
            string pathNoDrive = normalPath.Replace(this.PSDriveInfo.Root
                                        + pathSeparator, "");

            return pathNoDrive.Split(pathSeparator.ToCharArray());
        } // ChunkPath

        /// <summary>
        /// Adapts the path, making sure the correct path separator
        /// character is used.
        /// </summary>
        /// <param name="path"></param>
        /// <returns></returns>
        private string NormalizePath(string path)
        {
            string result = path;

            if (!String.IsNullOrEmpty(path))
            {
                result = path.Replace("/", pathSeparator);
            }

            return result;
        } // NormalizePath
```

```
        /// <summary>
        /// Ensures that the drive is removed from the specified path
        /// </summary>
        ///
        /// <param name="path">Path from which drive needs to be removed</param>
        /// <returns>Path with drive information removed</returns>
        private string RemoveDriveFromPath(string path)
        {
            string result = path;
            string root;

            if (this.PSDriveInfo == null)
            {
                root = String.Empty;
            }
            else
            {
                root = this.PSDriveInfo.Root;
            }

            if (result == null)
            {
                result = String.Empty;
            }

            if (result.Contains(root))
            {
                result = result.Substring(result.IndexOf(root,
StringComparison.OrdinalIgnoreCase) + root.Length);
            }

            return result;
        }

        /// <summary>
        /// Chunks the path and returns the table name and the row number
        /// from the path
        /// </summary>
        /// <param name="path">Path to chunk and obtain information</param>
        /// <param name="tableName">Name of the table as represented in the
        /// path</param>
        /// <param name="rowNumber">Row number obtained from the path</param>
        /// <returns>what the path represents</returns>
        public PathType GetNamesFromPath(string path, out string tableName, out int
rowNumber)
        {
            PathType retVal = PathType.Invalid;
            rowNumber = -1;
            tableName = null;

            // Check if the path specified is a drive
            if (PathIsDrive(path))
            {
                return PathType.Database;
            }
```

```
            // chunk the path into parts
            string[] pathChunks = ChunkPath(path);

            switch (pathChunks.Length)
            {
                case 1:
                    {
                        string name = pathChunks[0];

                        if (TableNameIsValid(name))
                        {
                            tableName = name;
                            retVal = PathType.Table;
                        }
                    }
                    break;

                case 2:
                    {
                        string name = pathChunks[0];

                        if (TableNameIsValid(name))
                        {
                            tableName = name;
                        }

                        int number = SafeConvertRowNumber(pathChunks[1]);

                        if (number >= 0)
                        {
                            rowNumber = number;
                            retVal = PathType.Row;
                        }
                        else
                        {
                            WriteError(new ErrorRecord(
                                new ArgumentException("Row number is not valid"),
                                "RowNumberNotValid",
                                ErrorCategory.InvalidArgument,
                                path));
                        }
                    }
                    break;

                default:
                    {
                        WriteError(new ErrorRecord(
                            new ArgumentException("The path supplied has too many
segments"),
                            "PathNotValid",
                            ErrorCategory.InvalidArgument,
                            path));
                    }
                    break;
```

```
            } // switch(pathChunks...

            return retVal;
        } // GetNamesFromPath

        /// <summary>
        /// Throws an argument exception stating that the specified path does
        /// not represent either a table or a row
        /// </summary>
        /// <param name="path">path which is invalid</param>
        private void ThrowTerminatingInvalidPathException(string path)
        {
            StringBuilder message = new StringBuilder("Path must represent either a
table or a row :");
            message.Append(path);

            throw new ArgumentException(message.ToString());
        }

        /// <summary>
        /// Retrieve the list of tables from the database.
        /// </summary>
        /// <returns>
        /// Collection of DatabaseTableInfo objects, each object representing
        /// information about one database table
        /// </returns>
        internal Collection<DatabaseTableInfo> GetTables()
        {
            Collection<DatabaseTableInfo> results =
                    new Collection<DatabaseTableInfo>();

            // using ODBC connection to the database and get the schema of tables
            AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;

            if (di == null)
            {
                return null;
            }

            OdbcConnection connection = di.Connection;
            DataTable dt = connection.GetSchema("Tables");
            int count;

            // iterate through all rows in the schema and create DatabaseTableInfo
            // objects which represents a table
            foreach (DataRow dr in dt.Rows)
            {
                String tableName = dr["TABLE_NAME"] as String;
                DataColumnCollection columns = null;

                // find the number of rows in the table
                try
                {
                    String cmd = "Select count(*) from \"" + tableName + "\"";
                    OdbcCommand command = new OdbcCommand(cmd, connection);
```

```
                    count = (Int32)command.ExecuteScalar();
                }
                catch
                {
                    count = 0;
                }

                // create DatabaseTableInfo object representing the table
                DatabaseTableInfo table =
                        new DatabaseTableInfo(dr, tableName, count, columns);

                results.Add(table);
            } // foreach (DataRow...

            return results;
        } // GetTables

        /// <summary>
        /// Return row information from a specified table.
        /// </summary>
        /// <param name="tableName">The name of the database table from
        /// which to retrieve rows.</param>
        /// <returns>Collection of row information objects.</returns>
        public Collection<DatabaseRowInfo> GetRows(string tableName)
        {
            Collection<DatabaseRowInfo> results =
                        new Collection<DatabaseRowInfo>();

            // Obtain rows in the table and add it to the collection
            try
            {
                OdbcDataAdapter da = GetAdapterForTable(tableName);

                if (da == null)
                {
                    return null;
                }

                DataSet ds = GetDataSetForTable(da, tableName);
                DataTable table = GetDataTable(ds, tableName);

                int i = 0;
                foreach (DataRow row in table.Rows)
                {
                    results.Add(new DatabaseRowInfo(row,
i.ToString(CultureInfo.CurrentCulture)));
                    i++;
                } // foreach (DataRow...
            }
            catch (Exception e)
            {
                WriteError(new ErrorRecord(e, "CannotAccessSpecifiedRows",
                    ErrorCategory.InvalidOperation, tableName));
            }
```

```
            return results;

        } // GetRows

        /// <summary>
        /// Retrieve information about a single table.
        /// </summary>
        /// <param name="tableName">The table for which to retrieve
        /// data.</param>
        /// <returns>Table information.</returns>
        private DatabaseTableInfo GetTable(string tableName)
        {
            foreach (DatabaseTableInfo table in GetTables())
            {
                if (String.Equals(tableName, table.Name,
StringComparison.OrdinalIgnoreCase))
                {
                    return table;
                }
            }

            return null;
        } // GetTable

        /// <summary>
        /// Removes the specified table from the database
        /// </summary>
        /// <param name="tableName">Name of the table to remove</param>
        private void RemoveTable(string tableName)
        {
            // validate if tablename is valid and if table is present
            if (String.IsNullOrEmpty(tableName) || !TableNameIsValid(tableName) ||
!TableIsPresent(tableName))
            {
                return;
            }

            // Execute command using ODBC connection to remove a table
            try
            {
                // delete the table using an sql statement
                string sql = "drop table " + tableName;

                AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;

                if (di == null)
                {
                    return;
                }
                OdbcConnection connection = di.Connection;

                OdbcCommand cmd = new OdbcCommand(sql, connection);
                cmd.ExecuteScalar();
            }
```

```
            catch (Exception ex)
            {
                WriteError(new ErrorRecord(ex, "CannotRemoveSpecifiedTable",
                          ErrorCategory.InvalidOperation, null)
                          );
            }

        } // RemoveTable

        /// <summary>
        /// Obtain a data adapter for the specified Table
        /// </summary>
        /// <param name="tableName">Name of the table to obtain the
        /// adapter for</param>
        /// <returns>Adapter object for the specified table</returns>
        /// <remarks>An adapter serves as a bridge between a DataSet (in memory
        /// representation of table) and the data source</remarks>
        internal OdbcDataAdapter GetAdapterForTable(string tableName)
        {
            OdbcDataAdapter da = null;
            AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;

            if (di == null || !TableNameIsValid(tableName) ||
!TableIsPresent(tableName))
            {
                return null;
            }

            OdbcConnection connection = di.Connection;

            try
            {
                // Create a odbc data adpater. This can be sued to update the
                // data source with the records that will be created here
                // using data sets
                string sql = "Select * from " + tableName;
                da = new OdbcDataAdapter(new OdbcCommand(sql, connection));

                // Create a odbc command builder object. This will create sql
                // commands automatically for a single table, thus
                // eliminating the need to create new sql statements for
                // every operation to be done.
                OdbcCommandBuilder cmd = new OdbcCommandBuilder(da);

                // Set the delete cmd for the table here
                sql = "Delete from " + tableName + " where ID = ?";
                da.DeleteCommand = new OdbcCommand(sql, connection);

                // Specify a DeleteCommand parameter based on the "ID"
                // column
                da.DeleteCommand.Parameters.Add(new OdbcParameter());
                da.DeleteCommand.Parameters[0].SourceColumn = "ID";

                // Create an InsertCommand based on the sql string
                // Insert into "tablename" values (?,?,?)" where
```

```
                // ? represents a column in the table. Note that
                // the number of ? will be equal to the number of
                // columnds
                DataSet ds = new DataSet();
                ds.Locale = CultureInfo.InvariantCulture;

                da.FillSchema(ds, SchemaType.Source);

                sql = "Insert into " + tableName + " values ( ";
                for (int i = 0; i < ds.Tables["Table"].Columns.Count; i++)
                {
                    sql += "?, ";
                }
                sql = sql.Substring(0, sql.Length - 2);
                sql += ")";
                da.InsertCommand = new OdbcCommand(sql, connection);

                // Create parameters for the InsertCommand based on the
                // captions of each column
                for (int i = 0; i < ds.Tables["Table"].Columns.Count; i++)
                {
                    da.InsertCommand.Parameters.Add(new OdbcParameter());
                    da.InsertCommand.Parameters[i].SourceColumn =
                                    ds.Tables["Table"].Columns[i].Caption;

                }

                // Open the connection if its not already open
                if (connection.State != ConnectionState.Open)
                {
                    connection.Open();
                }
            }
            catch (Exception e)
            {
                WriteError(new ErrorRecord(e, "CannotAccessSpecifiedTable",
                  ErrorCategory.InvalidOperation, tableName));
            }

            return da;
        } // GetAdapterForTable

        /// <summary>
        /// Gets the DataSet (in memory representation) for the table
        /// for the specified adapter
        /// </summary>
        /// <param name="adapter">Adapter to be used for obtaining
        /// the table</param>
        /// <param name="tableName">Name of the table for which a
        /// DataSet is required</param>
        /// <returns>The DataSet with the filled in schema</returns>
        internal DataSet GetDataSetForTable(OdbcDataAdapter adapter, string
tableName)
        {
            Debug.Assert(adapter != null);
```

```
        // Create a dataset object which will provide an in-memory
        // representation of the data being worked upon in the
        // data source.
        DataSet ds = new DataSet();

        // Create a table named "Table" which will contain the same
        // schema as in the data source.
        //adapter.FillSchema(ds, SchemaType.Source);
        adapter.Fill(ds, tableName);
        ds.Locale = CultureInfo.InvariantCulture;

        return ds;
    } //GetDataSetForTable

    /// <summary>
    /// Get the DataTable object which can be used to operate on
    /// for the specified table in the data source
    /// </summary>
    /// <param name="ds">DataSet object which contains the tables
    /// schema</param>
    /// <param name="tableName">Name of the table</param>
    /// <returns>Corresponding DataTable object representing
    /// the table</returns>
    ///
    internal DataTable GetDataTable(DataSet ds, string tableName)
    {
        Debug.Assert(ds != null);
        Debug.Assert(tableName != null);

        DataTable table = ds.Tables[tableName];
        table.Locale = CultureInfo.InvariantCulture;

        return table;
    } // GetDataTable

   /// <summary>
    /// Retrieves a single row from the named table.
    /// </summary>
    /// <param name="tableName">The table that contains the
    /// numbered row.</param>
    /// <param name="row">The index of the row to return.</param>
    /// <returns>The specified table row.</returns>
    private DatabaseRowInfo GetRow(string tableName, int row)
    {
        Collection<DatabaseRowInfo> di = GetRows(tableName);

        // if the row is invalid write an appropriate error else return the
        // corresponding row information
        if (row < di.Count && row >= 0)
        {
            return di[row];
        }
        else
        {
```

```
                WriteError(new ErrorRecord(
                    new ItemNotFoundException(),
                    "RowNotFound",
                    ErrorCategory.ObjectNotFound,
                    row.ToString(CultureInfo.CurrentCulture))
                );
            }

            return null;
        } // GetRow

        /// <summary>
        /// Method to safely convert a string representation of a row number
        /// into its Int32 equivalent
        /// </summary>
        /// <param name="rowNumberAsStr">String representation of the row
        /// number</param>
        /// <remarks>If there is an exception, -1 is returned</remarks>
        private int SafeConvertRowNumber(string rowNumberAsStr)
        {
            int rowNumber = -1;
            try
            {
                rowNumber = Convert.ToInt32(rowNumberAsStr,
CultureInfo.CurrentCulture);
            }
            catch (FormatException fe)
            {
                WriteError(new ErrorRecord(fe, "RowStringFormatNotValid",
                    ErrorCategory.InvalidData, rowNumberAsStr));
            }
            catch (OverflowException oe)
            {
                WriteError(new ErrorRecord(oe, "RowStringConversionToNumberFailed",
                    ErrorCategory.InvalidData, rowNumberAsStr));
            }

            return rowNumber;
        } // 1


        /// <summary>
        /// Check if a table name is valid
        /// </summary>
        /// <param name="tableName">Table name to validate</param>
        /// <remarks>Helps to check for SQL injection attacks</remarks>
        private bool TableNameIsValid(string tableName)
        {
            Regex exp = new Regex(pattern, RegexOptions.Compiled |
RegexOptions.IgnoreCase);

            if (exp.IsMatch(tableName))
            {
                return true;
            }
```

```
            WriteError(new ErrorRecord(
                new ArgumentException("Table name not valid"), "TableNameNotValid",
                    ErrorCategory.InvalidArgument, tableName));
            return false;
        } // TableNameIsValid

        /// <summary>
        /// Checks to see if the specified table is present in the
        /// database
        /// </summary>
        /// <param name="tableName">Name of the table to check</param>
        /// <returns>true, if table is present, false otherwise</returns>
        private bool TableIsPresent(string tableName)
        {
            // using ODBC connection to the database and get the schema of tables
            AccessDBPSDriveInfo di = this.PSDriveInfo as AccessDBPSDriveInfo;
            if (di == null)
            {
                return false;
            }

            OdbcConnection connection = di.Connection;
            DataTable dt = connection.GetSchema("Tables");

            // check if the specified tableName is available
            // in the list of tables present in the database
            foreach (DataRow dr in dt.Rows)
            {
                string name = dr["TABLE_NAME"] as string;
                if (name.Equals(tableName, StringComparison.OrdinalIgnoreCase))
                {
                    return true;
                }
            }

            WriteError(new ErrorRecord(
                new ArgumentException("Specified Table is not present in database"),
"TableNotAvailable",
                     ErrorCategory.InvalidArgument, tableName));

            return false;
        }// TableIsPresent

        /// <summary>
        /// Gets the next available ID in the table
        /// </summary>
        /// <param name="table">DataTable object representing the table to
        /// search for ID</param>
        /// <returns>next available id</returns>
        private int GetNextID(DataTable table)
        {
            int big = 0;

            for (int i = 0; i < table.Rows.Count; i++)
            {
```

```
                DataRow row = table.Rows[i];

                int id = (int)row["ID"];

                if (big < id)
                {
                    big = id;
                }
            }

            big++;
            return big;
        }
        #endregion Helper Methods

        #region Content Methods

        /// <summary>
        /// Clear the contents at the specified location. In this case, clearing
        /// the item amounts to clearing a row
        /// </summary>
        /// <param name="path">The path to the content to clear.</param>
        public void ClearContent(string path)
        {
            string tableName;
            int rowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            if (type != PathType.Table)
            {
                WriteError(new ErrorRecord(
                    new InvalidOperationException("Operation not supported. Content
can be cleared only for table"),
                        "NotValidRow", ErrorCategory.InvalidArgument,
                            path));
                return;
            }

            OdbcDataAdapter da = GetAdapterForTable(tableName);

            if (da == null)
            {
                return;
            }

            DataSet ds = GetDataSetForTable(da, tableName);
            DataTable table = GetDataTable(ds, tableName);

            // Clear contents at the specified location
            for (int i = 0; i < table.Rows.Count; i++)
            {
                table.Rows[i].Delete();
            }
```

```
            if (ShouldProcess(path, "ClearContent"))
            {
                da.Update(ds, tableName);
            }

        } // ClearContent

        /// <summary>
        /// Not implemented.
        /// </summary>
        /// <param name="path"></param>
        /// <returns></returns>
        public object ClearContentDynamicParameters(string path)
        {
            return null;
        }

        /// <summary>
        /// Get a reader at the path specified.
        /// </summary>
        /// <param name="path">The path from which to read.</param>
        /// <returns>A content reader used to read the data.</returns>
        public IContentReader GetContentReader(string path)
        {
            string tableName;
            int rowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            if (type == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(path);
            }
            else if (type == PathType.Row)
            {
                throw new InvalidOperationException("contents can be obtained only
for tables");
            }

            return new AccessDBContentReader(path, this);
        } // GetContentReader

        /// <summary>
        /// Not implemented.
        /// </summary>
        /// <param name="path"></param>
        /// <returns></returns>
        public object GetContentReaderDynamicParameters(string path)
        {
            return null;
        }

        /// <summary>
        /// Get an object used to write content.
        /// </summary>
```

```
        /// <param name="path">The root path at which to write.</param>
        /// <returns>A content writer for writing.</returns>
        public IContentWriter GetContentWriter(string path)
        {
            string tableName;
            int rowNumber;

            PathType type = GetNamesFromPath(path, out tableName, out rowNumber);

            if (type == PathType.Invalid)
            {
                ThrowTerminatingInvalidPathException(path);
            }
            else if (type == PathType.Row)
            {
                throw new InvalidOperationException("contents can be added only to
tables");
            }

            return new AccessDBContentWriter(path, this);
        }

        /// <summary>
        /// Not implemented.
        /// </summary>
        /// <param name="path"></param>
        /// <returns></returns>
        public object GetContentWriterDynamicParameters(string path)
        {
            return null;
        }

        #endregion Content Methods

        #region Private Properties

        private string pathSeparator = "\\";
        private static string pattern = @"^[a-z]+[0-9]*_*$";

        #endregion Private Properties

    } // AccessDBProvider

    #endregion AccessDBProvider

     #region Helper Classes

    #region Public Enumerations

    /// <summary>
    /// Type of item represented by the path
    /// </summary>
    public enum PathType
    {
        /// <summary>
```

```
        /// Represents a database
        /// </summary>
        Database,
        /// <summary>
        /// Represents a table
        /// </summary>
        Table,
        /// <summary>
        /// Represents a row
        /// </summary>
        Row,
        /// <summary>
        /// Represents an invalid path
        /// </summary>
        Invalid
    };

    #endregion Public Enumerations

     #region AccessDBPSDriveInfo

     /// <summary>
     /// Any state associated with the drive should be held here.
     /// In this case, it's the connection to the database.
     /// </summary>
     internal class AccessDBPSDriveInfo : PSDriveInfo
     {
         private OdbcConnection connection;

         /// <summary>
         /// ODBC connection information.
         /// </summary>
         public OdbcConnection Connection
         {
             get { return connection; }
             set { connection = value; }
         }

         /// <summary>
         /// Constructor that takes one argument
         /// </summary>
         /// <param name="driveInfo">Drive provided by this provider</param>
         public AccessDBPSDriveInfo(PSDriveInfo driveInfo)
             : base(driveInfo)
         { }

     } // class AccessDBPSDriveInfo

     #endregion AccessDBPSDriveInfo

     #region DatabaseTableInfo

     /// <summary>
     /// Contains information specific to the database table.
     /// Similar to the DirectoryInfo class.
```

```
    /// </summary>
    public class DatabaseTableInfo
    {
        /// <summary>
        /// Row from the "tables" schema
        /// </summary>
        public DataRow Data
        {
            get
            {
                return data;
            }
            set
            {
                data = value;
            }
        }
        private DataRow data;

        /// <summary>
        /// The table name.
        /// </summary>
        public string Name
        {
            get
            {
                return name;
            }
            set
            {
                name = value;
            }
        }
        private String name;

        /// <summary>
        /// The number of rows in the table.
        /// </summary>
        public int RowCount
        {
            get
            {
                return rowCount;
            }
            set
            {
                rowCount = value;
            }
        }
        private int rowCount;

        /// <summary>
        /// The column definitions for the table.
        /// </summary>
        public DataColumnCollection Columns
```

```
        {
            get
            {
                return columns;
            }
            set
            {
                columns = value;
            }
        }
        private DataColumnCollection columns;

        /// <summary>
        /// Constructor.
        /// </summary>
        /// <param name="row">The row definition.</param>
        /// <param name="name">The table name.</param>
        /// <param name="rowCount">The number of rows in the table.</param>
        /// <param name="columns">Information on the column tables.</param>
        public DatabaseTableInfo(DataRow row, string name, int rowCount,
                        DataColumnCollection columns)
        {
            Name = name;
            Data = row;
            RowCount = rowCount;
            Columns = columns;
        } // DatabaseTableInfo
    } // class DatabaseTableInfo

    #endregion DatabaseTableInfo

    #region DatabaseRowInfo

    /// <summary>
    /// Contains information specific to an individual table row.
    /// Analogous to the FileInfo class.
    /// </summary>
    public class DatabaseRowInfo
    {
        /// <summary>
        /// Row data information.
        /// </summary>
        public DataRow Data
        {
            get
            {
                return data;
            }
            set
            {
                data = value;
            }
        }
        private DataRow data;
```

```
        /// <summary>
        /// The row index.
        /// </summary>
        public string RowNumber
        {
            get
            {
                return rowNumber;
            }
            set
            {
                rowNumber = value;
            }
        }
        private string rowNumber;

        /// <summary>
        /// Constructor.
        /// </summary>
        /// <param name="row">The row information.</param>
        /// <param name="name">The row index.</param>
        public DatabaseRowInfo(DataRow row, string name)
        {
            RowNumber = name;
            Data = row;
        } // DatabaseRowInfo
    } // class DatabaseRowInfo

    #endregion DatabaseRowInfo

    #region AccessDBContentReader

    /// <summary>
    /// Content reader used to retrieve data from this provider.
    /// </summary>
    public class AccessDBContentReader : IContentReader
    {
        // A provider instance is required so as to get "content"
        private AccessDBProvider provider;
        private string path;
        private long currentOffset;

        internal AccessDBContentReader(string path, AccessDBProvider provider)
        {
            this.path = path;
            this.provider = provider;
        }

        /// <summary>
        /// Read the specified number of rows from the source.
        /// </summary>
        /// <param name="readCount">The number of items to
        /// return.</param>
        /// <returns>An array of elements read.</returns>
        public IList Read(long readCount)
```

```
        {
            // Read the number of rows specified by readCount and increment
            // offset
            string tableName;
            int rowNumber;
            PathType type = provider.GetNamesFromPath(path, out tableName, out
rowNumber);

            Collection<DatabaseRowInfo> rows =
                provider.GetRows(tableName);
            Collection<DataRow> results = new Collection<DataRow>();

            if (currentOffset < 0 || currentOffset >= rows.Count)
            {
                return null;
            }

            int rowsRead = 0;

            while (rowsRead < readCount && currentOffset < rows.Count)
            {
                results.Add(rows[(int)currentOffset].Data);
                rowsRead++;
                currentOffset++;
            }

            return results;
        } // Read

        /// <summary>
        /// Moves the content reader specified number of rows from the
        /// origin
        /// </summary>
        /// <param name="offset">Number of rows to offset</param>
        /// <param name="origin">Starting row from which to offset</param>
        public void Seek(long offset, System.IO.SeekOrigin origin)
        {
            // get the number of rows in the table which will help in
            // calculating current position
            string tableName;
            int rowNumber;

            PathType type = provider.GetNamesFromPath(path, out tableName, out
rowNumber);

            if (type == PathType.Invalid)
            {
                throw new ArgumentException("Path specified must represent a table
or a row :" + path);
            }

            if (type == PathType.Table)
            {
                Collection<DatabaseRowInfo> rows = provider.GetRows(tableName);
```

```
                int numRows = rows.Count;

                if (offset > rows.Count)
                {
                    throw new
                           ArgumentException(
                               "Offset cannot be greater than the number of rows
available"
                                                );
                }

                if (origin == System.IO.SeekOrigin.Begin)
                {
                    // starting from Beginning with an index 0, the current offset
                    // has to be advanced to offset - 1
                    currentOffset = offset - 1;
                }
                else if (origin == System.IO.SeekOrigin.End)
                {
                    // starting from the end which is numRows - 1, the current
                    // offset is so much less than numRows - 1
                    currentOffset = numRows - 1 - offset;
                }
                else
                {
                    // calculate from the previous value of current offset
                    // advancing forward always
                    currentOffset += offset;
                }
            } // if (type...
            else
            {
                // for row, the offset will always be set to 0
                currentOffset = 0;
            }

        } // Seek

        /// <summary>
        /// Closes the content reader, so all members are reset
        /// </summary>
        public void Close()
        {
            Dispose();
        } // Close

        /// <summary>
        /// Dispose any resources being used
        /// </summary>
        public void Dispose()
        {
            Seek(0, System.IO.SeekOrigin.Begin);

            GC.SuppressFinalize(this);
        } // Dispose
```

```
    } // AccessDBContentReader

    #endregion AccessDBContentReader

    #region AccessDBContentWriter

    /// <summary>
    /// Content writer used to write data in this provider.
    /// </summary>
    public class AccessDBContentWriter : IContentWriter
    {
        // A provider instance is required so as to get "content"
        private AccessDBProvider provider;
        private string path;
        private long currentOffset;

        internal AccessDBContentWriter(string path, AccessDBProvider provider)
        {
            this.path = path;
            this.provider = provider;
        }

        /// <summary>
        /// Write the specified row contents in the source
        /// </summary>
        /// <param name="content"> The contents to be written to the source.
        /// </param>
        /// <returns>An array of elements which were successfully written to
        /// the source</returns>
        ///
        public IList Write(IList content)
        {
            if (content == null)
            {
                return null;
            }

            // Get the total number of rows currently available it will
            // determine how much to overwrite and how much to append at
            // the end
            string tableName;
            int rowNumber;
            PathType type = provider.GetNamesFromPath(path, out tableName, out
rowNumber);

            if (type == PathType.Table)
            {
                OdbcDataAdapter da = provider.GetAdapterForTable(tableName);
                if (da == null)
                {
                    return null;
                }

                DataSet ds = provider.GetDataSetForTable(da, tableName);
                DataTable table = provider.GetDataTable(ds, tableName);
```

```
                string[] colValues = (content[0] as string).Split(',');

                // set the specified row
                DataRow row = table.NewRow();

                for (int i = 0; i < colValues.Length; i++)
                {
                    if (!String.IsNullOrEmpty(colValues[i]))
                    {
                        row[i] = colValues[i];
                    }
                }

                //table.Rows.InsertAt(row, rowNumber);
                // Update the table
                table.Rows.Add(row);
                da.Update(ds, tableName);

            }
            else
            {
                throw new InvalidOperationException("Operation not supported.
Content can be added only for tables");
            }

            return null;
        } // Write

        /// <summary>
        /// Moves the content reader specified number of rows from the
        /// origin
        /// </summary>
        /// <param name="offset">Number of rows to offset</param>
        /// <param name="origin">Starting row from which to offset</param>
        public void Seek(long offset, System.IO.SeekOrigin origin)
        {
            // get the number of rows in the table which will help in
            // calculating current position
            string tableName;
            int rowNumber;

            PathType type = provider.GetNamesFromPath(path, out tableName, out
rowNumber);

            if (type == PathType.Invalid)
            {
                throw new ArgumentException("Path specified should represent either
a table or a row : " + path);
            }

            Collection<DatabaseRowInfo> rows =
                   provider.GetRows(tableName);

            int numRows = rows.Count;
```

```
            if (offset > rows.Count)
            {
                throw new
                       ArgumentException(
                           "Offset cannot be greater than the number of rows
available"
                                                );
            }

            if (origin == System.IO.SeekOrigin.Begin)
            {
                // starting from Beginning with an index 0, the current offset
                // has to be advanced to offset - 1
                currentOffset = offset - 1;
            }
            else if (origin == System.IO.SeekOrigin.End)
            {
                // starting from the end which is numRows - 1, the current
                // offset is so much less than numRows - 1
                currentOffset = numRows - 1 - offset;
            }
            else
            {
                // calculate from the previous value of current offset
                // advancing forward always
                currentOffset += offset;
            }

        } // Seek

        /// <summary>
        /// Closes the content reader, so all members are reset
        /// </summary>
        public void Close()
        {
            Dispose();
        } // Close

        /// <summary>
        /// Dispose any resources being used
        /// </summary>
        public void Dispose()
        {
            Seek(0, System.IO.SeekOrigin.Begin);

            GC.SuppressFinalize(this);
        } // Dispose
    } // AccessDBContentWriter

    #endregion AccessDBContentWriter

    #endregion Helper Classes
} // namespace Microsoft.Samples.PowerShell.Providers
```

See Also

System.Management.Automation.Provider.ItemCmdletProvider

System.Management.Automation.Provider.ContainerCmdletProvider

System.Management.Automation.Provider.NavigationCmdletProvider

Designing Your Windows PowerShell Provider

Last updated on 05/20/2025

Windows PowerShell Host Quickstart

To host Windows PowerShell in your application, you use the System.Management.Automation.PowerShell class. This class provides methods that create a pipeline of commands and then execute those commands in a runspace. The simplest way to create a host application is to use the default runspace. The default runspace contains all of the core Windows PowerShell commands. If you want your application to expose only a subset of the Windows PowerShell commands, you must create a custom runspace.

> **Note**
>
> To run the following samples, you need to have the `Microsoft.PowerShell.SDK` NuGet package installed.

Using the default runspace

To start, we'll use the default runspace, and use the methods of the System.Management.Automation.PowerShell class to add commands, parameters, statements, and scripts to a pipeline.

AddCommand

You use the System.Management.Automation.PowerShell.AddCommand method to add commands to the pipeline. For example, suppose you want to get the list of running processes on the machine. The way to run this command is as follows.

1. Create a System.Management.Automation.PowerShell object.

   C#

   ```
   PowerShell ps = PowerShell.Create();
   ```

2. Add the command that you want to execute.

   C#

   ```
   ps.AddCommand("Get-Process");
   ```

3. Invoke the command.

```
C#

ps.Invoke();
```

If you call the AddCommand method more than once before you call the System.Management.Automation.PowerShell.Invoke method, the result of the first command is piped to the second, and so on. If you do not want to pipe the result of a previous command to a command, add it by calling the System.Management.Automation.PowerShell.AddStatement instead.

AddParameter

The previous example executes a single command without any parameters. You can add parameters to the command by using the System.Management.Automation.PSCommand.AddParameter method. For example, the following code gets a list of all of the processes that are named `powershell` running on the machine.

```
C#

PowerShell.Create().AddCommand("Get-Process")
                   .AddParameter("Name", "powershell")
                   .Invoke();
```

You can add additional parameters by calling the AddParameter method repeatedly.

```
C#

PowerShell.Create().AddCommand("Get-ChildItem")
                   .AddParameter("Path", @"C:\Windows")
                   .AddParameter("Filter", "*.exe")
                   .Invoke();
```

You can also add a dictionary of parameter names and values by calling the System.Management.Automation.PowerShell.AddParameters method.

```
C#

var parameters = new Dictionary<string, string>
{
    { "Path", @"C:\Windows" },
    { "Filter", "*.exe" }
```

```
};

PowerShell.Create().AddCommand("Get-Process")
                   .AddParameters(parameters)
                   .Invoke()
```

AddStatement

You can simulate batching by using the System.Management.Automation.PowerShell.AddStatement method, which adds an additional statement to the end of the pipeline. The following code gets a list of running processes with the name `powershell`, and then gets the list of running services.

C#

```
PowerShell ps = PowerShell.Create();
ps.AddCommand("Get-Process").AddParameter("Name", "powershell");
ps.AddStatement().AddCommand("Get-Service");
ps.Invoke();
```

AddScript

You can run an existing script by calling the System.Management.Automation.PowerShell.AddScript method. The following example adds a script to the pipeline and runs it. This example assumes there is already a script named `MyScript.ps1` in a folder named `D:\PSScripts`.

C#

```
PowerShell ps = PowerShell.Create();
ps.AddScript(@"D:\PSScripts\MyScript.ps1").Invoke();
```

There is also a version of the AddScript method that takes a boolean parameter named `useLocalScope`. If this parameter is set to `true`, then the script is run in the local scope. The following code will run the script in the local scope.

C#

```
PowerShell ps = PowerShell.Create();
ps.AddScript(@"D:\PSScripts\MyScript.ps1", true).Invoke();
```

Creating a custom runspace

While the default runspace used in the previous examples loads all of the core Windows PowerShell commands, you can create a custom runspace that loads only a specified subset of all commands. You might want to do this to improve performance (loading a larger number of commands is a performance hit), or to restrict the capability of the user to perform operations. A runspace that exposes only a limited number of commands is called a constrained runspace. To create a constrained runspace, you use the System.Management.Automation.Runspaces.Runspace and System.Management.Automation.Runspaces.InitialSessionState classes.

Creating an InitialSessionState object

To create a custom runspace, you must first create a System.Management.Automation.Runspaces.InitialSessionState object. In the following example, we use the System.Management.Automation.Runspaces.RunspaceFactory to create a runspace after creating a default InitialSessionState object.

C#

```
InitialSessionState iss = InitialSessionState.CreateDefault();

Runspace rs = RunspaceFactory.CreateRunspace(iss);
rs.Open();

PowerShell ps = PowerShell.Create();
ps.Runspace = rs;
ps.AddCommand("Get-Command");
ps.Invoke();

rs.Close();
```

Constraining the runspace

In the previous example, we created a default System.Management.Automation.Runspaces.InitialSessionState object that loads all of the built-in core Windows PowerShell. We could also have called the System.Management.Automation.Runspaces.InitialSessionState.CreateDefault2 method to create an InitialSessionState object that would load only the commands in the Microsoft.PowerShell.Core snapin. To create a more constrained runspace, you must create an empty InitialSessionState object by calling the System.Management.Automation.Runspaces.InitialSessionState.Create method, and then add commands to the InitialSessionState.

Using a runspace that loads only the commands that you specify provides significantly improved performance.

You use the methods of the System.Management.Automation.Runspaces.SessionStateCmdletEntry class to define cmdlets for the initial session state. The following example creates an empty initial session state, then defines and adds the `Get-Command` and `Import-Module` commands to the initial session state. We then create a runspace constrained by that initial session state, and execute the commands in that runspace.

Create the initial session state.

C#

```
InitialSessionState iss = InitialSessionState.Create();
```

Define and add commands to the initial session state.

C#

```
SessionStateCmdletEntry getCommand = new SessionStateCmdletEntry(
    "Get-Command", typeof(Microsoft.PowerShell.Commands.GetCommandCommand), "");
SessionStateCmdletEntry importModule = new SessionStateCmdletEntry(
    "Import-Module", typeof(Microsoft.PowerShell.Commands.ImportModuleCommand),
"");

iss.Commands.Add(getCommand);
iss.Commands.Add(importModule);
```

Create and open the runspace.

C#

```
Runspace rs = RunspaceFactory.CreateRunspace(iss);
rs.Open();
```

Execute a command and show the result.

C#

```
PowerShell ps = PowerShell.Create();
ps.Runspace = rs;
ps.AddCommand("Get-Command");

Collection<CommandInfo> result = ps.Invoke<CommandInfo>();
```

```
foreach (CommandInfo entry in result)
{
    Console.WriteLine(entry.Name);
}
```

Close the runspace.

C#

```
rs.Close();
```

When run, the output of this code will look as follows.

PowerShell

```
Get-Command
Import-Module
```

Last updated on 10/16/2025

Creating Runspaces

A runspace is the operating environment for the commands that are invoked by a host application. This environment includes the commands and data that are currently present, and any language restrictions that currently apply.

Host applications can use the default runspace that is provided by Windows PowerShell, which includes all available core commands, or create a custom runspace that includes only a subset of the available commands. To create a customized runspace, you create a System.Management.Automation.Runspaces.InitialSessionState object and assign it to your runspace.

Runspace tasks

1. Creating an InitialSessionState
2. Creating a constrained runspace
3. Creating multiple runspaces

See Also

Last updated on 05/20/2025

Creating an InitialSessionState

PowerShell commands run in a runspace. To host PowerShell in your application, you must create a System.Management.Automation.Runspaces.Runspace object. Every runspace has a System.Management.Automation.Runspaces.InitialSessionState object associated with it. The InitialSessionState specifies characteristics of the runspace, such as which commands, variables, and modules are available for that runspace.

Create a default InitialSessionState

The CreateDefault and CreateDefault2 methods of the **InitialSessionState** class can be used to create an **InitialSessionState** object. The **CreateDefault** method creates an **InitialSessionState** with all of the built-in commands loaded, while the **CreateDefault2** method loads only the commands required to host PowerShell (the commands from the Microsoft.PowerShell.Core module).

If you want to further limit the commands available in your host application you need to create a constrained runspace. For information, see Creating a constrained runspace.

The following code shows how to create an **InitialSessionState**, assign it to a runspace, add commands to the pipeline in that runspace, and invoke the commands. For more information about adding and invoking commands, see Adding and invoking commands.

C#

```
namespace SampleHost
{
  using System;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;

  class HostP4b
  {
    static void Main(string[] args)
    {
      // Call InitialSessionState.CreateDefault() to create an empty
      // InitialSessionState object, then add the variables that will be
      // available when the runspace is opened.
      InitialSessionState iss = InitialSessionState.CreateDefault();
      SessionStateVariableEntry var1 =
        new SessionStateVariableEntry("test1",
                                      "MyVar1",
                                      "Initial session state MyVar1 test");
```

```
      iss.Variables.Add(var1);

      SessionStateVariableEntry var2 =
        new SessionStateVariableEntry("test2",
                                      "MyVar2",
                                      "Initial session state MyVar2 test");
      iss.Variables.Add(var2);

      // Call RunspaceFactory.CreateRunspace(InitialSessionState) to
      // create the runspace where the pipeline is run.
      Runspace rs = RunspaceFactory.CreateRunspace(iss);
      rs.Open();

      // Call PowerShell.Create() to create the PowerShell object, then
      // specify the runspace and pipeline commands.
      PowerShell ps = PowerShell.Create();
      ps.Runspace = rs;
      ps.AddCommand("Get-Variable");
      ps.AddArgument("test*");

      Console.WriteLine("Variable             Value");
      Console.WriteLine("-------------------------");

      // Call ps.Invoke() to run the pipeline synchronously.
      foreach (PSObject result in ps.Invoke())
      {
        Console.WriteLine("{0,-20}{1}",
            result.Members["Name"].Value,
            result.Members["Value"].Value);
      } // End foreach.

      // Close the runspace to free resources.
      rs.Close();

    } // End Main.
  } // End SampleHost.
}
```

See Also

Creating a constrained runspace

Adding and invoking commands

Last updated on 05/20/2025

Creating a constrained runspace

For performance or security reasons, you might want to restrict the Windows PowerShell commands available to your host application. To do this you create an empty System.Management.Automation.Runspaces.InitialSessionState by calling the System.Management.Automation.Runspaces.InitialSessionState.Create* method, and then add only the commands you want available.

Using a runspace that loads only the commands that you specify provides significantly improved performance.

You use the methods of the System.Management.Automation.Runspaces.SessionStateCmdletEntry class to define cmdlets for the initial session state.

You can also make commands private. Private commands can be used by the host application, but not by users of the application.

Adding commands to an empty runspace

The following example demonstrates how to create an empty InitialSessionState and add commands to it.

C#

```
namespace Microsoft.Samples.PowerShell.Runspaces
{
  using System;
  using System.Collections.ObjectModel;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;
  using Microsoft.PowerShell.Commands;
  using PowerShell = System.Management.Automation.PowerShell;

  /// <summary>
  /// This class contains the Main entry point for the application.
  /// </summary>
  internal class Runspace10b
  {
    /// <summary>
    /// This sample shows how to create an empty initial session state,
    /// how to add commands to the session state, and then how to create a
    /// runspace that has only those two commands. A PowerShell object
    /// is used to run the Get-Command cmdlet to show that only two commands
```

```
    /// are available.
    /// </summary>
    /// <param name="args">Parameter not used.</param>
    private static void Main(string[] args)
    {
      // Create an empty InitialSessionState and then add two commands.
      InitialSessionState iss = InitialSessionState.Create();

      // Add the Get-Process and Get-Command cmdlets to the session state.
      SessionStateCmdletEntry ssce1 = new SessionStateCmdletEntry(
                                                          "Get-Process",

typeof(GetProcessCommand),
                                                          null);
      iss.Commands.Add(ssce1);

      SessionStateCmdletEntry ssce2 = new SessionStateCmdletEntry(
                                                          "Get-Command",

typeof(GetCommandCommand),
                                                          null);
      iss.Commands.Add(ssce2);

      // Create a runspace.
      using (Runspace myRunSpace = RunspaceFactory.CreateRunspace(iss))
      {
        myRunSpace.Open();
        using (PowerShell powershell = PowerShell.Create())
        {
          powershell.Runspace = myRunSpace;

          // Create a pipeline with the Get-Command command.
          powershell.AddCommand("Get-Command");

          Collection<PSObject> results = powershell.Invoke();

          Console.WriteLine("Verb                 Noun");
          Console.WriteLine("---------------------------");

          // Display each result object.
          foreach (PSObject result in results)
          {
            Console.WriteLine(
                          "{0,-20} {1}",
                          result.Members["verb"].Value,
                          result.Members["Noun"].Value);
          }
        }

        // Close the runspace and release any resources.
        myRunSpace.Close();
      }

      System.Console.WriteLine("Hit any key to exit...");
      System.Console.ReadKey();
```

```
    }
  }
}
```

Making commands private

You can also make a command private, by setting it's System.Management.Automation.CommandInfo.Visibility property to System.Management.Automation.SessionStateEntryVisibility **Private**. The host application and other commands can call that command, but the user of the application cannot. In the following example, the Get-ChildItem command is private.

C#

```
defaultSessionState = InitialSessionState.CreateDefault();
commandIndex = GetIndexOfEntry(defaultSessionState.Commands, "Get-ChildItem");
defaultSessionState.Commands[commandIndex].Visibility =
SessionStateEntryVisibility.Private;

this.runspace = RunspaceFactory.CreateRunspace(defaultSessionState);
this.runspace.Open();
```

See Also

Creating an InitialSessionState

Last updated on 05/20/2025

Creating multiple runspaces

If you create a large number of runspaces, you might consider creating a runspace pool. Using a System.Management.Automation.Runspaces.RunspacePool object, rather than creating a large number of individual runspaces with the same characteristics, can improve performance.

Creating and using a runspace pool.

The following example shows how to create a runspace pool and how to run a command asynchronously in a runspace of the pool.

C#

```
namespace HostRunspacePool
{
  using System;
  using System.Collections.ObjectModel;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;

  /// <summary>
  /// This class provides the Main entry point for the Host application.
  /// </summary>
  internal class HostRunspacePool
  {
    /// <summary>
    /// This sample demonstrates the following.
    /// 1. Creating and opening a runspace pool.
    /// 2. Creating a PowerShell object.
    /// 3. Adding commands and arguments to the PowerShell object.
    /// 4. Running the commands asynchronously using the runspace
    ///    of the runspace pool.
    /// </summary>
    /// <param name="args">Parameter is not used.</param>
    private static void Main(string[] args)
    {
      // Create a pool of runspaces.
      using (RunspacePool rsp = RunspaceFactory.CreateRunspacePool())
      {
        rsp.Open();

        // Create a PowerShell object to run the following command.
        //  Get-Process wmi*
        PowerShell gpc = PowerShell.Create();
        // Specify the runspace to use and add commands.
        gpc.RunspacePool = rsp;
        gpc.AddCommand("Get-Process").AddArgument("wmi*");
```

```
        // Invoke the command asynchronously.
        IAsyncResult gpcAsyncResult = gpc.BeginInvoke();
        // Get the results of running the command.
        PSDataCollection<PSObject> gpcOutput = gpc.EndInvoke(gpcAsyncResult);

        // Process the output.
        Console.WriteLine("The output from running the command: Get-Process wmi*");
        for (int i= 0; i < gpcOutput.Count; i++)
        {
         Console.WriteLine(
                           "Process Name: {0} Process Id: {1}",
                           gpcOutput[i].Properties["ProcessName"].Value,
                           gpcOutput[i].Properties["Id"].Value);
        }
      } // End using.
    } // End Main entry point.
  } // End HostPs5 class.
}
```

See Also

Creating an InitialSessionState

Last updated on 05/20/2025

Adding and invoking commands

After creating a runspace, you can add Windows PowerShell commands and scripts to a pipeline, and then invoke the pipeline synchronously or asynchronously.

Creating a pipeline

The System.Management.Automation.PowerShell class provides several methods to add commands, parameters, and scripts to the pipeline. You can invoke the pipeline synchronously by calling an overload of the System.Management.Automation.PowerShell.Invoke* method, or asynchronously by calling an overload of the System.Management.Automation.PowerShell.BeginInvoke* and then the System.Management.Automation.PowerShell.EndInvoke* method.

AddCommand

1. Create a System.Management.Automation.PowerShell object.

   ```
   C#
   PowerShell ps = PowerShell.Create();
   ```

2. Add the command that you want to execute.

   ```
   C#
   ps.AddCommand("Get-Process");
   ```

3. Invoke the command.

   ```
   C#
   ps.Invoke();
   ```

If you call the System.Management.Automation.PowerShell.AddCommand* method more than once before you call the System.Management.Automation.PowerShell.Invoke* method, the result of the first command is piped to the second, and so on. If you do not want to pipe the result of a previous command to a command, add it by calling the System.Management.Automation.PowerShell.AddStatement* instead.

AddParameter

The previous example executes a single command without any parameters. You can add parameters to the command by using the System.Management.Automation.PSCommand.AddParameter* method For example, the following code gets a list of all of the processes that are named `powershell` running on the machine.

C#

```
PowerShell.Create().AddCommand("Get-Process")
                   .AddParameter("Name", "powershell")
                   .Invoke();
```

You can add additional parameters by calling System.Management.Automation.PSCommand.AddParameter* repeatedly.

C#

```
PowerShell.Create().AddCommand("Get-Command")
                   .AddParameter("Name", "Get-VM")
                   .AddParameter("Module", "Hyper-V")
                   .Invoke();
```

You can also add a dictionary of parameter names and values by calling the System.Management.Automation.PowerShell.AddParameters* method.

C#

```
IDictionary parameters = new Dictionary<String, String>();
parameters.Add("Name", "Get-VM");

parameters.Add("Module", "Hyper-V");
PowerShell.Create().AddCommand("Get-Command")
   .AddParameters(parameters)
      .Invoke()
```

AddStatement

You can simulate batching by using the System.Management.Automation.PowerShell.AddStatement* method, which adds an additional statement to the end of the pipeline The following code gets a list of running processes with the name `powershell`, and then gets the list of running services.

C#

```
PowerShell ps = PowerShell.Create();
ps.AddCommand("Get-Process").AddParameter("Name", "powershell");
ps.AddStatement().AddCommand("Get-Service");
ps.Invoke();
```

AddScript

You can run an existing script by calling the System.Management.Automation.PowerShell.AddScript* method. The following example adds a script to the pipeline and runs it. This example assumes there is already a script named `MyScript.ps1` in a folder named `D:\PSScripts`.

C#

```
PowerShell ps = PowerShell.Create();
ps.AddScript(File.ReadAllText(@"D:\PSScripts\MyScript.ps1")).Invoke();
```

There is also a version of the System.Management.Automation.PowerShell.AddScript* method that takes a boolean parameter named `useLocalScope`. If this parameter is set to `true`, then the script is run in the local scope. The following code will run the script in the local scope.

C#

```
PowerShell ps = PowerShell.Create();
ps.AddScript(File.ReadAllText(@"D:\PSScripts\MyScript.ps1"), true).Invoke();
```

Invoking a pipeline synchronously

After you add elements to the pipeline, you invoke it. To invoke the pipeline synchronously, you call an overload of the System.Management.Automation.PowerShell.Invoke* method. The following example shows how to synchronously invoke a pipeline.

C#

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Management.Automation;

namespace HostPS1e
{
  class HostPS1e
```

```
  {
    static void Main(string[] args)
    {
      // Using the PowerShell.Create and AddCommand
      // methods, create a command pipeline.
      PowerShell ps = PowerShell.Create().AddCommand ("Sort-Object");

      // Using the PowerShell.Invoke method, run the command
      // pipeline using the supplied input.
      foreach (PSObject result in ps.Invoke(new int[] { 3, 1, 6, 2, 5, 4 }))
      {
          Console.WriteLine("{0}", result);
      } // End foreach.
    } // End Main.
  } // End HostPS1e.
}
```

Invoking a pipeline asynchronously

You invoke a pipeline asynchronously by calling an overload of the System.Management.Automation.PowerShell.BeginInvoke* to create an IAsyncResult object, and then calling the System.Management.Automation.PowerShell.EndInvoke* method.

The following example shows how to invoke a pipeline asynchronously.

C#

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Management.Automation;

namespace HostPS3
{
  class HostPS3
  {
    static void Main(string[] args)
    {
      // Use the PowerShell.Create and PowerShell.AddCommand
      // methods to create a command pipeline that includes
      // Get-Process cmdlet. Do not include spaces immediately
      // before or after the cmdlet name as that will cause
      // the command to fail.
      PowerShell ps = PowerShell.Create().AddCommand("Get-Process");

      // Create an IAsyncResult object and call the
      // BeginInvoke method to start running the
      // command pipeline asynchronously.
      IAsyncResult asyncpl = ps.BeginInvoke();
```

```
      // Using the PowerShell.Invoke method, run the command
      // pipeline using the default runspace.
      foreach (PSObject result in ps.EndInvoke(asyncpl))
      {
        Console.WriteLine("{0,-20}{1}",
                result.Members["ProcessName"].Value,
                result.Members["Id"].Value);
      } // End foreach.
      System.Console.WriteLine("Hit any key to exit.");
      System.Console.ReadKey();
    } // End Main.
  } // End HostPS3.
}
```

See Also

Creating an InitialSessionState

Creating a constrained runspace

Last updated on 05/20/2025

Creating remote runspaces

PowerShell commands that take a **ComputerName** parameter can be run on any computer that runs PowerShell. To run commands that don't take a **ComputerName** parameter, you can use WS-Management to configure a runspace that connects to a specified computer, and run commands on that computer.

Using a WSManConnection to create a remote runspace

To create a runspace that connects to a remote computer, you create a System.Management.Automation.Runspaces.WSManConnectionInfo object. You specify the target endpoint for the connection by setting the System.Management.Automation.Runspaces.WSManConnectionInfo.ConnectionUri property of the object. You then create a runspace by calling the System.Management.Automation.Runspaces.RunspaceFactory.CreateRunspace method, specifying the System.Management.Automation.Runspaces.WSManConnectionInfo object as the `connectionInfo` parameter.

The following example shows how to create a runspace that connects to a remote computer. In the example, `RemoteComputerUri` is used as a placeholder for the actual URI of a remote computer.

C#

```
namespace Samples
{
  using System;
  using System.Collections.ObjectModel;
  using System.Management.Automation;            // PowerShell namespace.
  using System.Management.Automation.Runspaces;  // PowerShell namespace.

  /// <summary>
  /// This class contains the Main entry point for this host application.
  /// </summary>
  internal class RemoteRunspace02
  {
    /// <summary>
    /// This sample shows how to create a remote runspace that
    /// runs commands on the local computer.
    /// </summary>
    /// <param name="args">Parameter not used.</param>
```

```
    private static void Main(string[] args)
    {
      // Create a WSManConnectionInfo object using the default constructor
      // to connect to the "localHost". The WSManConnectionInfo object can
      // also be used to specify connections to remote computers.
      Uri RemoteComputerUri = new Uri("http://Server01:5985/WSMAN");
      WSManConnectionInfo connectionInfo = new
WSManConnectionInfo(RemoteComputerUri);

      // Set the OperationTimeout property and OpenTimeout properties.
      // The OperationTimeout property is used to tell PowerShell
      // how long to wait (in milliseconds) before timing out for an
      // operation. The OpenTimeout property is used to tell Windows
      // PowerShell how long to wait (in milliseconds) before timing out
      // while establishing a remote connection.
      connectionInfo.OperationTimeout = 4 * 60 * 1000; // 4 minutes.
      connectionInfo.OpenTimeout = 1 * 60 * 1000; // 1 minute.

      // Create a remote runspace using the connection information.
      //using (Runspace remoteRunspace = RunspaceFactory.CreateRunspace())
      using (Runspace remoteRunspace =
RunspaceFactory.CreateRunspace(connectionInfo))
      {
        // Establish the connection by calling the Open() method to open the
runspace.
        // The OpenTimeout value set previously will be applied while establishing
        // the connection. Establishing a remote connection involves sending and
        // receiving some data, so the OperationTimeout will also play a role in
this process.
          remoteRunspace.Open();

        // Create a PowerShell object to run commands in the remote runspace.
        using (PowerShell powershell = PowerShell.Create())
        {
          powershell.Runspace = remoteRunspace;
          powershell.AddCommand("Get-Process");
          powershell.Invoke();

          Collection<PSObject> results = powershell.Invoke();

          Console.WriteLine("Process             HandleCount");
          Console.WriteLine("-------------------------------");

          // Display the results.
          foreach (PSObject result in results)
          {
            Console.WriteLine(
                          "{0,-20} {1}",
                          result.Members["ProcessName"].Value,
                          result.Members["HandleCount"].Value);
          }
        }

        // Close the connection. Call the Close() method to close the remote
        // runspace. The Dispose() method (called by using primitive) will call
```

```
        // the Close() method if it is not already called.
        remoteRunspace.Close();
      }
    }
  }
}
```

Last updated on 05/20/2025

Creating a custom user interface

Windows PowerShell provides abstract classes and interfaces that allow you to create a custom interactive UI that hosts the Windows PowerShell engine. To create a custom UI, you must implement the System.Management.Automation.Host.PSHost class. Optionally, you can also implement the System.Management.Automation.Host.PSHostRawUserInterface and System.Management.Automation.Host.PSHostUserInterface classes, and the System.Management.Automation.Host.IHostSupportsInteractiveSession and System.Management.Automation.Host.IHostUISupportsMultipleChoiceSelection interfaces.

Last updated on 05/20/2025

Host Application Samples

This section includes sample code that is provided in the Windows PowerShell 2.0 SDK.

In This Section

PowerShell API Samples This section includes sample code that shows how to create runspaces that restrict functionality, and how to asynchronously run commands using a runspace pool to supply the runspaces.

Custom Host Samples Includes sample code for writing a custom host. The host is the component of Windows PowerShell that provides communications between the user and the Windows PowerShell engine. For more information about custom hosts, see Custom Host.

Runspace Samples Includes sample code for creating runspaces. For more information about how runspaces are used, see Host Application Runspaces.

Remote Runspace Samples This section includes sample code that shows how to create runspaces that can be used to connect to a computer by using WS-Management-based Windows PowerShell remoting.

See Also

Last updated on 05/20/2025

Windows PowerShell API Samples

This section includes sample code that shows how to create runspaces that restrict functionality, and how to asynchronously run commands by using a runspace pool to supply the runspaces. You can use Microsoft Visual Studio to create a console application and then copy the code from the topics in this section into your host application.

In This Section

PowerShell01 Sample This sample shows how to use a System.Management.Automation.Runspaces.InitialSessionState object to limit the functionality of a runspace. The output of this sample demonstrates how to restrict the language mode of the runspace, how to mark a cmdlet as private, how to add and remove cmdlets and providers, how to add a proxy command, and more.

PowerShell02 Sample This sample shows how to run commands asynchronously by using the runspaces of a runspace pool. The sample generates a list of commands, and then runs those commands while the Windows PowerShell engine opens a runspace from the pool when it is needed.

Last updated on 05/20/2025

Windows PowerShell01 Sample

This sample shows how to use a System.Management.Automation.Runspaces.InitialSessionState object to limit the functionality of a runspace. The output of this sample demonstrates how to restrict the language mode of the runspace, how to mark a cmdlet as private, how to add and remove cmdlets and providers, how to add a proxy command, and more. This sample concentrates on how to restrict the runspace programmatically. Scripting alternatives to restricting the runspace include the $ExecutionContext.SessionState.LanguageMode and PSSessionConfiguration commands.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following:

- Restricting the language by setting the System.Management.Automation.Runspaces.InitialSessionState.LanguageMode property.
- Adding aliases to the initial session state by using a System.Management.Automation.Runspaces.SessionStateAliasEntry object.
- Marking commands as private.
- Removing providers from the initial session state by using the System.Management.Automation.Runspaces.InitialSessionState.Providers property.
- Removing commands from the initial session state by using the System.Management.Automation.Runspaces.InitialSessionState.Commands property.
- Adding commands and providers to the System.Management.Automation.Runspaces.InitialSessionState object.

Example

This sample shows several ways to limit the functionality of a runspace.

C#

```
namespace Sample
{
  using System;
  using System.Collections.ObjectModel;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;

  /// <summary>
  /// This class contains the Main entry point for the application.
  /// </summary>
  internal class PowerShell01
  {
    /// <summary>
    /// The runspace used to run commands.
    /// </summary>
    private Runspace runspace;

    /// <summary>
    /// Return the first index of the entry in <paramref name="entries"/>
    /// with the name <paramref name="name"/>. Return -1 if it is not found.
    /// </summary>
    /// <typeparam name="T">Type of ConstrainedSessionStateEntry</typeparam>
    /// <param name="entries">Collection of entries to search for <paramref
name="name"/> in.</param>
    /// <param name="name">Named of the entry we are looking for</param>
    /// <returns>
    /// The first index of the entry in <paramref name="entries"/> with the
    /// name <paramref name="name"/>, or return -1 if it is not found.
    /// </returns>
    private static int GetIndexOfEntry<T>(
            InitialSessionStateEntryCollection<T> entries,
            string name) where T : ConstrainedSessionStateEntry
    {
      int foundIndex = 0;
      foreach (T entry in entries)
      {
        if (entry.Name.Equals(name, StringComparison.OrdinalIgnoreCase))
        {
          return foundIndex;
        }

        foundIndex++;
      }

      return -1;
    }

    /// <summary>
    /// Run commands to demonstrate the ways to constrain the runspace.
    /// </summary>
    /// <param name="args">This parameter is unused.</param>
    private static void Main(string[] args)
    {
```

```
      new PowerShell01().RunCommands();
    }

    /// <summary>
    /// Run a script to display the results and errors.
    /// </summary>
    /// <param name="script">Script to be run.</param>
    /// <param name="scriptComment">Comment to be printed about
    /// the script.</param>
    private void RunScript(string script, string scriptComment)
    {
      Console.WriteLine("Running '{0}'\n{1}.\n\nPowerShell Output:", script,
scriptComment);

      // Using a PowerShell object, create a pipeline, add the script to the
      //  pipeline, and specify the runspace where the pipeline is invoked.
      PowerShell powerShellCommand = PowerShell.Create();
      powerShellCommand.AddScript(script);
      powerShellCommand.Runspace = this.runspace;

      try
      {
        Collection<PSObject> results = powerShellCommand.Invoke();

        // Display the results.
        foreach (PSObject result in results)
        {
          Console.WriteLine(result);
        }

        // Display any non-terminating errors.
        foreach (ErrorRecord error in powerShellCommand.Streams.Error)
        {
          Console.WriteLine("PowerShell Error: {0}", error);
        }
      }
      catch (RuntimeException ex)
      {
        Console.WriteLine("PowerShell Error: {0}", ex.Message);
        Console.WriteLine();
      }

      Console.WriteLine("\n----------------------------\n");
    }

    /// <summary>
    /// Run some commands to demonstrate the script capabilities.
    /// </summary>
    private void RunCommands()
    {
      this.runspace =
RunspaceFactory.CreateRunspace(InitialSessionState.CreateDefault());
      this.runspace.Open();
      this.RunScript("$a=0;$a", "Assigning to a variable will work for a default
InitialSessionState");
```

```
      this.runspace.Close();

      this.runspace =
RunspaceFactory.CreateRunspace(InitialSessionState.CreateDefault());
      this.runspace.InitialSessionState.LanguageMode =
PSLanguageMode.RestrictedLanguage;
      this.runspace.Open();
      this.RunScript("$a=0;$a", "Assigning to a variable will not work in
RestrictedLanguage LanguageMode");
      this.runspace.Close();

      this.runspace =
RunspaceFactory.CreateRunspace(InitialSessionState.CreateDefault());
      this.runspace.InitialSessionState.LanguageMode = PSLanguageMode.NoLanguage;
      this.runspace.Open();
      this.RunScript("10/2", "A script will not work in NoLanguage LanguageMode.");
      this.runspace.Close();

      this.runspace =
RunspaceFactory.CreateRunspace(InitialSessionState.CreateDefault());
      this.runspace.Open();
      string scriptComment = "Get-ChildItem with a default InitialSessionState will
work since the standard \n" +
           "PowerShell cmdlets are included in the default InitialSessionState";
      this.RunScript("Get-ChildItem", scriptComment);
      this.runspace.Close();

      InitialSessionState defaultSessionState =
InitialSessionState.CreateDefault();
      defaultSessionState.Commands.Add(new SessionStateAliasEntry("dir2", "Get-
ChildItem"));
      this.runspace = RunspaceFactory.CreateRunspace(defaultSessionState);
      this.runspace.Open();
      this.RunScript("dir2", "An alias, like dir2, can be added to
InitialSessionState");
      this.runspace.Close();

      defaultSessionState = InitialSessionState.CreateDefault();
      int commandIndex = GetIndexOfEntry(defaultSessionState.Commands, "Get-
ChildItem");
      defaultSessionState.Commands.RemoveItem(commandIndex);
      this.runspace = RunspaceFactory.CreateRunspace(defaultSessionState);
      this.runspace.Open();
      scriptComment = "Get-ChildItem was removed from the list of commands so
it\nwill no longer be found";
      this.RunScript("Get-ChildItem", scriptComment);
      this.runspace.Close();

      defaultSessionState = InitialSessionState.CreateDefault();
      defaultSessionState.Providers.Clear();
      this.runspace = RunspaceFactory.CreateRunspace(defaultSessionState);
      this.runspace.Open();
      this.RunScript("Get-ChildItem", "There are no providers so Get-ChildItem will
not work");
      this.runspace.Close();
```

```
      // Marks a command as private, and then defines a proxy command
      // that uses the private command.  One reason to define a proxy for
      // a command is to remove a parameter of the original command.
      // For a more complete sample of a proxy command, see the Runspace11
      // sample.
      defaultSessionState = InitialSessionState.CreateDefault();
      commandIndex = GetIndexOfEntry(defaultSessionState.Commands, "Get-
ChildItem");
      defaultSessionState.Commands[commandIndex].Visibility =
SessionStateEntryVisibility.Private;
      CommandMetadata getChildItemMetadata = new CommandMetadata(
           typeof(Microsoft.PowerShell.Commands.GetChildItemCommand));
      getChildItemMetadata.Parameters.Remove("Recurse");
      string getChildItemBody = ProxyCommand.Create(getChildItemMetadata);
      defaultSessionState.Commands.Add(new SessionStateFunctionEntry("Get-
ChildItem2", getChildItemBody));
      this.runspace = RunspaceFactory.CreateRunspace(defaultSessionState);
      this.runspace.Open();
      this.RunScript("Get-ChildItem", "Get-ChildItem is private so it will not be
available");
      scriptComment = "Get-ChildItem2 is a proxy to Get-ChildItem. \n" +
                      "It works even when Get-ChildItem is private.";
      this.RunScript("Get-ChildItem2", scriptComment);
      scriptComment = "This will fail. Unlike Get-ChildItem, Get-ChildItem2 does
not have -Recurse";
      this.RunScript("Get-ChildItem2 -Recurse", scriptComment);

      InitialSessionState cleanSessionState = InitialSessionState.Create();
      this.runspace = RunspaceFactory.CreateRunspace(cleanSessionState);
      this.runspace.Open();
      scriptComment = "A script will not work because \n" +
                     "InitialSessionState.Create() will have the default LanguageMode
of NoLanguage";
      this.RunScript("10/2", scriptComment);
      this.runspace.Close();

      cleanSessionState = InitialSessionState.Create();
      cleanSessionState.LanguageMode = PSLanguageMode.FullLanguage;
      this.runspace = RunspaceFactory.CreateRunspace(cleanSessionState);
      this.runspace.Open();
      scriptComment = "Get-ChildItem, standard cmdlets and providers are not
present \n" +
                      "in an InitialSessionState returned from
InitialSessionState.Create()";
      this.RunScript("Get-ChildItem", scriptComment);
      this.runspace.Close();

      cleanSessionState = InitialSessionState.Create();
      cleanSessionState.Commands.Add(
                new SessionStateCmdletEntry(
                    "Get-ChildItem",
                    typeof(Microsoft.PowerShell.Commands.GetChildItemCommand),
                    null));
      cleanSessionState.Providers.Add(
```

```
                new SessionStateProviderEntry(
                    "FileSystem",
                    typeof(Microsoft.PowerShell.Commands.FileSystemProvider),
                    null));
      cleanSessionState.LanguageMode = PSLanguageMode.FullLanguage;
      this.runspace = RunspaceFactory.CreateRunspace(cleanSessionState);
      this.runspace.Open();
      scriptComment = "Get-ChildItem and the FileSystem provider were explicitly
added\n" +
                "so Get-ChildItem will work";
      this.RunScript("Get-ChildItem", scriptComment);
      this.runspace.Close();

      Console.Write("Done...");
      Console.ReadLine();
    }
  }
}
```

See Also

Writing a Windows PowerShell Host Application

Last updated on 05/20/2025

Windows PowerShell02 Sample

This sample shows how to run commands asynchronously using the runspaces of a runspace pool. The sample generates a list of commands, and then runs those commands while the Windows PowerShell engine opens a runspace from the pool when it is needed.

Requirements

- This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following:

- Creating a RunspacePool object with a minimum and maximum number of runspaces allowed to be open at the same time.
- Creating a list of commands.
- Running the commands asynchronously.
- Calling the System.Management.Automation.Runspaces.RunspacePool.GetAvailableRunspaces* method to see how many runspaces are free.
- Capturing the command output with the System.Management.Automation.PowerShell.EndInvoke* method.

Example

This sample shows how to open the runspaces of a runspace pool, and how to asynchronously run commands in those runspaces.

C#

```
namespace Sample
{
  using System;
  using System.Collections;
  using System.Collections.Generic;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;

  /// <summary>
```

```
  /// This class contains the Main entry point for the application.
  /// </summary>
  internal class PowerShell02
  {
    /// <summary>
    /// Runs many commands with the help of a RunspacePool.
    /// </summary>
    /// <param name="args">This parameter is unused.</param>
    private static void Main(string[] args)
    {
      // Creating and opening runspace pool. Use a minimum of 1 runspace and a
maximum of
      // 5 runspaces can be opened at the same time.
      RunspacePool runspacePool = RunspaceFactory.CreateRunspacePool(1, 5);
      runspacePool.Open();

      using (runspacePool)
      {
        // Define the commands to be run.
        List<PowerShell> powerShellCommands = new List<PowerShell>();

        // The command results.
        List<IAsyncResult> powerShellCommandResults = new List<IAsyncResult>();

        // The maximum number of runspaces that can be opened at one time is
        // 5, but we can queue up many more commands that will use the
        // runspace pool.
        for (int i = 0; i < 100; i++)
        {
          // Using a PowerShell object, run the commands.
          PowerShell powershell = PowerShell.Create();

          // Instead of setting the Runspace property of powershell,
          // the RunspacePool property is used. That is the only difference
          // between running commands with a runspace and running commands
          // with a runspace pool.
          powershell.RunspacePool = runspacePool;

          // The script to be run outputs a sequence number and the number of
available runspaces
          // in the pool.
          string script = String.Format(
                        "write-output ' Command: {0}, Available Runspaces: {1}'",
                        i,
                        runspacePool.GetAvailableRunspaces());

          // The three lines below look the same running with a runspace or
          // with a runspace pool.
          powershell.AddScript(script);
          powerShellCommands.Add(powershell);
          powerShellCommandResults.Add(powershell.BeginInvoke());
        }

        // Collect the results.
        for (int i = 0; i < 100; i++)
```

```
        {
          // EndInvoke will wait for each command to finish, so we will be getting
the commands
          // in the same 0 to 99 order that they have been invoked withy
BeginInvoke.
          PSDataCollection<PSObject> results =
powerShellCommands[i].EndInvoke(powerShellCommandResults[i]);

          // Print all the results. One PSObject with a plain string is the
expected result.
          PowerShell02.PrintCollection(results);
        }
      }
    }

    /// <summary>
    /// Iterates through a collection printing all items.
    /// </summary>
    /// <param name="collection">collection to be printed</param>
    private static void PrintCollection(IList collection)
    {
      foreach (object obj in collection)
      {
        Console.WriteLine("PowerShell Result: {0}", obj);
      }
    }
  }
}
```

See Also

Writing a Windows PowerShell Host Application

Last updated on 05/20/2025

Custom Host Samples

This section includes sample code for writing a custom host. You can use Microsoft Visual Studio to create a console application and then copy the code from the topics in this section into your host application.

In This Section

Host01 Sample This sample shows how to implement a host application that uses a basic custom host.

Host02 Sample This sample shows how to write a host application that uses the Windows PowerShell runtime along with a custom host implementation. The host application sets the host culture to German, runs the Get-Process cmdlet and displays the results as you would see them using pwrsh.exe, and then prints out the current data and time in German.

Host03 Sample This sample shows how to build an interactive console-based host application that reads commands from the command line, executes the commands, and then displays the results to the console.

Host04 Sample This sample shows how to build an interactive console-based host application that reads commands from the command line, executes the commands, and then displays the results to the console. This host application also supports displaying prompts that allow the user to specify multiple choices.

Host05 Sample This sample shows how to build an interactive console-based host application that reads commands from the command line, executes the commands, and then displays the results to the console. This host application also supports calls to remote computers by using the Enter-PSSession and Exit-PSSession cmdlets

Host06 Sample This sample shows how to build an interactive console-based host application that reads commands from the command line, executes the commands, and then displays the results to the console. In addition, this sample uses the Tokenizer APIs to specify the color of the text that is entered by the user.

See Also

Last updated on 05/20/2025

Host01 Sample

This sample shows how to implement a host application that uses a custom host. In this sample a runspace is created that uses the custom host, and then the System.Management.Automation.PowerShell API is used to run a script that calls "exit." The host application then looks at the output of the script and prints out the results.

This sample uses the default UI features provided by Windows PowerShell. For more information about implementing the UI features of a custom host, see Host02 Sample.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

- Creating a custom host class that derives from the System.Management.Automation.Host.PSHost class.
- Creating a runspace that uses the custom host class.
- Creating a System.Management.Automation.PowerShell object that runs a script that calls exit.
- Verifying that the correct exit code was used in the exit process.

Example 1

The following code shows an implementation of a host application that uses a simple custom host interface.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{

    using System;
    using System.Management.Automation;
    using System.Management.Automation.Runspaces;
    using PowerShell = System.Management.Automation.PowerShell;
```

```
/// <summary>
/// This class contains the Main entry point for this host application.
/// </summary>
internal class Host01
{

    /// <summary>
    /// Indicator to tell the host application that it should exit.
    /// </summary>
    private bool shouldExit;

    /// <summary>
    /// The exit code that the host application will use to exit.
    /// </summary>
    private int exitCode;

    /// <summary>
    /// Gets or sets a value indicating whether the
    /// host application should exit.
    /// </summary>
    public bool ShouldExit
    {
        get { return this.shouldExit; }
        set { this.shouldExit = value; }
    }

    /// <summary>
    /// Gets or sets the PSHost implementation that is
    /// used to tell the host application what code to use
    /// when exiting.
    /// </summary>
    public int ExitCode
    {
        get { return this.exitCode; }
        set { this.exitCode = value; }
    }

    /// <summary>
    /// This sample uses a PowerShell object to run
    /// a script that calls exit. The host application looks at
    /// this and prints out the result.
    /// </summary>
    /// <param name="args">Parameter not used.</param>
    private static void Main(string[] args)
    {
        // Create an instance of this host application class so that
        // the Windows PowerShell engine will have access to the
        // ShouldExit and ExitCode parameters.
        Host01 me = new Host01();

        // Create the host instance to use.
        MyHost myHost = new MyHost(me);

        // Create a runspace that uses the host object and run the
        // script using a PowerShell object.
```

```
            using (Runspace myRunSpace = RunspaceFactory.CreateRunspace(myHost))
            {
                // Open the runspace.
                myRunSpace.Open();

                // Create a PowerShell object to run the script.
                using (PowerShell powershell = PowerShell.Create())
                {
                    powershell.Runspace = myRunSpace;

                    // Create the pipeline and run the script
                    // "exit (2+2)".
                    string script = "exit (2+2)";
                    powershell.AddScript(script);
                    powershell.Invoke(script);
                }

                // Check the flags and see if they were set properly.
                Console.WriteLine(
                    "ShouldExit={0} (should be True); ExitCode={1} (should be 4)",
                    me.ShouldExit,
                    me.ExitCode);

                // close the runspace to free resources.
                myRunSpace.Close();
            }

            Console.WriteLine("Hit any key to exit...");
            Console.ReadKey();
        }
    }
}
```

Example 2

The following code is the implementation of the System.Management.Automation.Host.PSHost class that is used by this host application. Those elements that are not implemented throw an exception or return nothing.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
    using System;
    using System.Globalization;
    using System.Management.Automation.Host;

    /// <summary>
    /// This is a sample implementation of the PSHost abstract class for
    /// console applications. Not all members are implemented. Those that
    /// are not implemented throw a NotImplementedException exception or
```

```
    /// return nothing.
    /// </summary>
    internal class MyHost : PSHost
    {
        /// <summary>
        /// A reference to the PSHost implementation.
        /// </summary>
        private Host01 program;

        /// <summary>
        /// The culture information of the thread that created
        /// this object.
        /// </summary>
        private CultureInfo originalCultureInfo =
            System.Threading.Thread.CurrentThread.CurrentCulture;

        /// <summary>
        /// The UI culture information of the thread that created
        /// this object.
        /// </summary>
        private CultureInfo originalUICultureInfo =
            System.Threading.Thread.CurrentThread.CurrentUICulture;

        /// <summary>
        /// The identifier of this PSHost implementation.
        /// </summary>
        private Guid myId = Guid.NewGuid();

        /// <summary>
        /// Initializes a new instance of the MyHost class. Keep
        /// a reference to the host application object so that it
        /// can be informed of when to exit.
        /// </summary>
        /// <param name="program">
        /// A reference to the host application object.
        /// </param>
        public MyHost(Host01 program)
        {
            this.program = program;
        }

        /// <summary>
        /// Return the culture information to use. This implementation
        /// returns a snapshot of the culture information of the thread
        /// that created this object.
        /// </summary>
        public override System.Globalization.CultureInfo CurrentCulture
        {
            get { return this.originalCultureInfo; }
        }

        /// <summary>
        /// Return the UI culture information to use. This implementation
        /// returns a snapshot of the UI culture information of the thread
        /// that created this object.
```

```
        /// </summary>
        public override System.Globalization.CultureInfo CurrentUICulture
        {
            get { return this.originalUICultureInfo; }
        }

        /// <summary>
        /// This implementation always returns the GUID allocated at
        /// instantiation time.
        /// </summary>
        public override Guid InstanceId
        {
            get { return this.myId; }
        }

        /// <summary>
        /// Return a string that contains the name of the host implementation.
        /// Keep in mind that this string may be used by script writers to
        /// identify when your host is being used.
        /// </summary>
        public override string Name
        {
            get { return "MySampleConsoleHostImplementation"; }
        }

        /// <summary>
        /// This sample does not implement a PSHostUserInterface component so
        /// this property simply returns null.
        /// </summary>
        public override PSHostUserInterface UI
        {
            get { return null; }
        }

        /// <summary>
        /// Return the version object for this application. Typically this
        /// should match the version resource in the application.
        /// </summary>
        public override Version Version
        {
            get { return new Version(1, 0, 0, 0); }
        }

        /// <summary>
        /// Not implemented by this example class. The call fails with
        /// a NotImplementedException exception.
        /// </summary>
        public override void EnterNestedPrompt()
        {
            throw new NotImplementedException(
                "The method or operation is not implemented.");
        }

        /// <summary>
        /// Not implemented by this example class. The call fails
```

```
        /// with a NotImplementedException exception.
        /// </summary>
        public override void ExitNestedPrompt()
        {
            throw new NotImplementedException(
                "The method or operation is not implemented.");
        }

        /// <summary>
        /// This API is called before an external application process is
        /// started. Typically it is used to save state so the parent can
        /// restore state that has been modified by a child process (after
        /// the child exits). In this example, this functionality is not
        /// needed so the method returns nothing.
        /// </summary>
        public override void NotifyBeginApplication()
        {
            return;
        }

        /// <summary>
        /// This API is called after an external application process finishes.
        /// Typically it is used to restore state that a child process may
        /// have altered. In this example, this functionality is not
        /// needed so the method returns nothing.
        /// </summary>
        public override void NotifyEndApplication()
        {
           return;
        }

        /// <summary>
        /// Indicate to the host application that exit has
        /// been requested. Pass the exit code that the host
        /// application should use when exiting the process.
        /// </summary>
        /// <param name="exitCode">The exit code to use.</param>
        public override void SetShouldExit(int exitCode)
        {
            this.program.ShouldExit = true;
            this.program.ExitCode = exitCode;
        }
    }
}
```

See Also

Last updated on 05/20/2025

Host02 Sample

This sample shows how to write a host application that uses the Windows PowerShell runtime along with a custom host implementation. The host application sets the host culture to German, runs the Get-Process cmdlet and displays the results as you would see them by using pwrsh.exe, and then prints out the current data and time in German.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

- Creating a custom host whose classes derive from the System.Management.Automation.Host.PSHost class, the System.Management.Automation.Host.PSHostUserInterface class, and the System.Management.Automation.Host.PSHostRawUserInterface class.
- Creating a runspace that uses the custom host.
- Setting the host culture to German.
- Creating a System.Management.Automation.PowerShell object that runs a script to retrieve and sort the processes, then retrieves the current date which is displayed in German.

Example 1

The following code shows an implementation of a host application that uses the custom host.

C#

```
// Copyright (c) 2006 Microsoft Corporation. All rights reserved.
//
// THIS CODE AND INFORMATION IS PROVIDED "AS IS" WITHOUT WARRANTY OF
// ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO
// THE IMPLIED WARRANTIES OF MERCHANTABILITY AND/OR FITNESS FOR A
// PARTICULAR PURPOSE.
//
using System;
using System.Collections.Generic;
```

```
using System.Text;
using System.Management.Automation;
using System.Management.Automation.Host;
using System.Management.Automation.Runspaces;
using System.Globalization;

namespace Microsoft.Samples.PowerShell.Host
{
  class Host02
  {
    /// <summary>
    /// Define the property that the PSHost implementation will
    /// use to tell the host application that it should exit.
    /// </summary>
    public bool ShouldExit
    {
      get { return shouldExit; }
      set { shouldExit = value; }
    }
    private bool shouldExit;

    /// <summary>
    /// Define the property that the PSHost implementation will
    /// use to tell the host application what exit code to use
    /// when exiting.
    /// </summary>
    public int ExitCode
    {
      get { return exitCode; }
      set { exitCode = value; }
    }
    private int exitCode;

    /// <summary>
    /// This sample uses the PowerShell runtime along with a host
    /// implementation to call Get-Process and display the results
    /// as you would see them in powershell.exe.
    /// </summary>
    /// <param name="args">Ignored</param>
    static void Main(string[] args)
    {
      // Set the current culture to German. We want this to be picked up when the
MyHost
      // instance is created...
      System.Threading.Thread.CurrentThread.CurrentCulture =
CultureInfo.GetCultureInfo("de-de");

      // Create the runspace so that you can access the pipeline.
      MyHost myHost = new MyHost(new Host02());

      Runspace myRunSpace = RunspaceFactory.CreateRunspace(myHost);
      myRunSpace.Open();

      // Create the pipeline.
      Pipeline pipe = myRunSpace.CreatePipeline();
```

```
      // Add the script we want to run. This script does two things.
      // First, it runs the Get-Process cmdlet with the cmdlet output
      // sorted by handle count. Second, the GetDate cmdlet is piped
      // to the Out-String cmdlet so that we can see the
      // date displayed in German.

      pipe.Commands.AddScript(@"
                  Get-Process | sort HandleCount
                  # This should display the date in German...
                  Get-Date | Out-String
                  ");

      // Add the default outputter to the end of the pipe and indicate
      // that it should handle both output and errors from the previous
      // commands. This will result in the output being written using the PSHost
      // and PSHostUserInterface classes instead of returning objects to the
hosting
      // application.
      pipe.Commands.Add("Out-Default");

pipe.Commands[0].MergeMyResults(PipelineResultTypes.Error,PipelineResultTypes.Outpu
t);

      // Invoke the pipeline. There will not be any objects
      // returned. The Out-Default cmdlet consumes the objects.
      pipe.Invoke();

      System.Console.WriteLine("Hit any key to exit...");
      System.Console.ReadKey();
    }
  }
}
```

Example 2

The following code is the implementation of the System.Management.Automation.Host.PSHost class that is used by this host application. Those elements that are not implemented throw an exception or return nothing.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Globalization;
  using System.Management.Automation.Host;

  /// <summary>
  /// This is a sample implementation of the PSHost abstract class for
  /// console applications. Not all members are implemented. Those that
```

```
/// are not implemented throw a NotImplementedException exception or
/// return nothing.
/// </summary>
internal class MyHost : PSHost
{
  /// <summary>
  /// A reference to the PSHost implementation.
  /// </summary>
  private Host02 program;

  /// <summary>
  /// The culture information of the thread that created
  /// this object.
  /// </summary>
  private CultureInfo originalCultureInfo =
      System.Threading.Thread.CurrentThread.CurrentCulture;

  /// <summary>
  /// The UI culture information of the thread that created
  /// this object.
  /// </summary>
  private CultureInfo originalUICultureInfo =
      System.Threading.Thread.CurrentThread.CurrentUICulture;

  /// <summary>
  /// The identifier of this PSHost implementation.
  /// </summary>
  private Guid myId = Guid.NewGuid();

  /// <summary>
  /// Initializes a new instance of the MyHost class. Keep
  /// a reference to the host application object so that it
  /// can be informed of when to exit.
  /// </summary>
  /// <param name="program">
  /// A reference to the host application object.
  /// </param>
  public MyHost(Host02 program)
  {
    this.program = program;
  }

  /// <summary>
  /// A reference to the implementation of the PSHostUserInterface
  /// class for this application.
  /// </summary>
  private MyHostUserInterface myHostUserInterface = new MyHostUserInterface();

  /// <summary>
  /// Gets the culture information to use. This implementation
  /// returns a snapshot of the culture information of the thread
  /// that created this object.
  /// </summary>
  public override System.Globalization.CultureInfo CurrentCulture
  {
```

```
        get { return this.originalCultureInfo; }
    }

    /// <summary>
    /// Gets the UI culture information to use. This implementation
    /// returns a snapshot of the UI culture information of the thread
    /// that created this object.
    /// </summary>
    public override System.Globalization.CultureInfo CurrentUICulture
    {
        get { return this.originalUICultureInfo; }
    }

    /// <summary>
    /// Gets an identifier for this host. This implementation always
    /// returns the GUID allocated at instantiation time.
    /// </summary>
    public override Guid InstanceId
    {
        get { return this.myId; }
    }

    /// <summary>
    /// Gets a string that contains the name of this host implementation.
    /// Keep in mind that this string may be used by script writers to
    /// identify when your host is being used.
    /// </summary>
    public override string Name
    {
        get { return "MySampleConsoleHostImplementation"; }
    }

    /// <summary>
    /// Gets an instance of the implementation of the PSHostUserInterface
    /// class for this application. This instance is allocated once at startup time
    /// and returned every time thereafter.
    /// </summary>
    public override PSHostUserInterface UI
    {
        get { return this.myHostUserInterface; }
    }

    /// <summary>
    /// Gets the version object for this application. Typically this
    /// should match the version resource in the application.
    /// </summary>
    public override Version Version
    {
        get { return new Version(1, 0, 0, 0); }
    }

    /// <summary>
    /// This API Instructs the host to interrupt the currently running
    /// pipeline and start a new nested input loop. In this example this
    /// functionality is not needed so the method throws a
```

```
    /// NotImplementedException exception.
    /// </summary>
    public override void EnterNestedPrompt()
    {
      throw new NotImplementedException(
            "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API instructs the host to exit the currently running input loop.
    /// In this example this functionality is not needed so the method
    /// throws a NotImplementedException exception.
    /// </summary>
    public override void ExitNestedPrompt()
    {
      throw new NotImplementedException(
            "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API is called before an external application process is
    /// started. Typically it is used to save state so that the parent
    /// can restore state that has been modified by a child process (after
    /// the child exits). In this example this functionality is not
    /// needed so the method returns nothing.
    /// </summary>
    public override void NotifyBeginApplication()
    {
      return;
    }

    /// <summary>
    /// This API is called after an external application process finishes.
    /// Typically it is used to restore state that a child process has
    /// altered. In this example, this functionality is not needed so
    /// the method returns nothing.
    /// </summary>
    public override void NotifyEndApplication()
    {
      return;
    }

    /// <summary>
    /// Indicate to the host application that exit has
    /// been requested. Pass the exit code that the host
    /// application should use when exiting the process.
    /// </summary>
    /// <param name="exitCode">The exit code that the
    /// host application should use.</param>
    public override void SetShouldExit(int exitCode)
    {
      this.program.ShouldExit = true;
      this.program.ExitCode = exitCode;
    }
```

```
  }
}
```

Example 3

The following code is the implementation of the System.Management.Automation.Host.PSHostUserInterface class that is used by this host application.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Collections.Generic;
  using System.Globalization;
  using System.Management.Automation;
  using System.Management.Automation.Host;

  /// <summary>
  /// A sample implementation of the PSHostUserInterface abstract class for
  /// console applications. Not all members are implemented. Those that are
  /// not implemented throw a NotImplementedException exception. Members that
  /// are implemented include those that map easily to Console APIs.
  /// </summary>
  internal class MyHostUserInterface : PSHostUserInterface
  {
    /// <summary>
    /// An instance of the PSRawUserInterface class.
    /// </summary>
    private MyRawUserInterface myRawUi = new MyRawUserInterface();

    /// <summary>
    /// Gets an instance of the PSRawUserInterface class for this host
    /// application.
    /// </summary>
    public override PSHostRawUserInterface RawUI
    {
      get { return this.myRawUi; }
    }

    /// <summary>
    /// Prompts the user for input. In this example this functionality is not
    /// needed so the method throws a NotImplementException exception.
    /// </summary>
    /// <param name="caption">The caption or title of the prompt.</param>
    /// <param name="message">The text of the prompt.</param>
    /// <param name="descriptions">A collection of FieldDescription objects that
    /// describe each field of the prompt.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override Dictionary<string, PSObject> Prompt(
```

```
                                                    string caption,
                                                    string message,

System.Collections.ObjectModel.Collection<FieldDescription> descriptions)
    {
       throw new NotImplementedException(
           "The method or operation is not implemented.");
    }

    /// <summary>
    /// Provides a set of choices that enable the user to choose a
    /// single option from a set of options. In this example this
    /// functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="caption">Text that proceeds (a title) the choices.</param>
    /// <param name="message">A message that describes the choice.</param>
    /// <param name="choices">A collection of ChoiceDescription objects that
describes
    /// each choice.</param>
    /// <param name="defaultChoice">The index of the label in the Choices parameter
    /// collection. To indicate no default choice, set to -1.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override int PromptForChoice(string caption, string message,
System.Collections.ObjectModel.Collection<ChoiceDescription> choices, int
defaultChoice)
    {
      throw new NotImplementedException("The method or operation is not
implemented.");
    }

    /// <summary>
    /// Prompts the user for credentials with a specified prompt window caption,
    /// prompt message, user name, and target name. In this example this
    /// functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="caption">The caption for the message window.</param>
    /// <param name="message">The text of the message.</param>
    /// <param name="userName">The user name whose credential is to be prompted
for.</param>
    /// <param name="targetName">The name of the target for which the credential is
collected.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override PSCredential PromptForCredential(
                                                    string caption,
                                                    string message,
                                                    string userName,
                                                    string targetName)
    {
      throw new NotImplementedException("The method or operation is not
implemented.");
    }

    /// <summary>
```

```
    /// Prompts the user for credentials by using a specified prompt window
caption,
    /// prompt message, user name and target name, credential types allowed to be
    /// returned, and UI behavior options. In this example this functionality
    /// is not needed so the method throws a NotImplementExcepti on exception.
    /// </summary>
    /// <param name="caption">The caption for the message window.</param>
    /// <param name="message">The text of the message.</param>
    /// <param name="userName">The user name whose credential is to be prompted
for.</param>
    /// <param name="targetName">The name of the target for which the credential is
collected.</param>
    /// <param name="allowedCredentialTypes">A PSCredentialTypes constant that
    /// identifies the type of credentials that can be returned.</param>
    /// <param name="options">A PSCredentialUIOptions constant that identifies the
UI
    /// behavior when it gathers the credentials.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override PSCredential PromptForCredential(
                                                    string caption,
                                                    string message,
                                                    string userName,
                                                    string targetName,
                                                    PSCredentialTypes
allowedCredentialTypes,
                                                    PSCredentialUIOptions options)
    {
      throw new NotImplementedException("The method or operation is not
implemented.");
    }

    /// <summary>
    /// Reads characters that are entered by the user until a newline
    /// (carriage return) is encountered.
    /// </summary>
    /// <returns>The characters that are entered by the user.</returns>
    public override string ReadLine()
    {
      return Console.ReadLine();
    }

    /// <summary>
    /// Reads characters entered by the user until a newline (carriage return)
    /// is encountered and returns the characters as a secure string. In this
    /// example this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <returns>Throws a NotImplemented exception.</returns>
    public override System.Security.SecureString ReadLineAsSecureString()
    {
      throw new NotImplementedException("The method or operation is not
implemented.");
    }

    /// <summary>
```

```
    /// Writes characters to the output display of the host.
    /// </summary>
    /// <param name="value">The characters to be written.</param>
    public override void Write(string value)
    {
      System.Console.Write(value);
    }

    /// <summary>
    /// Writes characters to the output display of the host and specifies the
    /// foreground and background colors of the characters. This implementation
    /// ignores the colors.
    /// </summary>
    /// <param name="foregroundColor">The color of the characters.</param>
    /// <param name="backgroundColor">The background color to use.</param>
    /// <param name="value">The characters to be written.</param>
    public override void Write(
                               ConsoleColor foregroundColor,
                               ConsoleColor backgroundColor,
                               string value)
    {
       // Colors are ignored.
       System.Console.Write(value);
    }

    /// <summary>
    /// Writes a debug message to the output display of the host.
    /// </summary>
    /// <param name="message">The debug message that is displayed.</param>
    public override void WriteDebugLine(string message)
    {
      Console.WriteLine(String.Format(
                                      CultureInfo.CurrentCulture,
                                      "DEBUG: {0}",
                                      message));
    }

    /// <summary>
    /// Writes an error message to the output display of the host.
    /// </summary>
    /// <param name="value">The error message that is displayed.</param>
    public override void WriteErrorLine(string value)
    {
      Console.WriteLine(String.Format(
                                      CultureInfo.CurrentCulture,
                                      "ERROR: {0}",
                                      value));
    }

    /// <summary>
    /// Writes a newline character (carriage return)
    /// to the output display of the host.
    /// </summary>
    public override void WriteLine()
    {
```

```
      System.Console.WriteLine();
    }

    /// <summary>
    /// Writes a line of characters to the output display of the host
    /// and appends a newline character(carriage return).
    /// </summary>
    /// <param name="value">The line to be written.</param>
    public override void WriteLine(string value)
    {
      System.Console.WriteLine(value);
    }

    /// <summary>
    /// Writes a line of characters to the output display of the host
    /// with foreground and background colors and appends a newline (carriage
return).
    /// </summary>
    /// <param name="foregroundColor">The foreground color of the display. </param>
    /// <param name="backgroundColor">The background color of the display. </param>
    /// <param name="value">The line to be written.</param>
    public override void WriteLine(ConsoleColor foregroundColor, ConsoleColor
backgroundColor, string value)
    {
      // Write to the output stream, ignore the colors
      System.Console.WriteLine(value);
    }

    /// <summary>
    /// Writes a progress report to the output display of the host.
    /// </summary>
    /// <param name="sourceId">Unique identifier of the source of the record.
</param>
    /// <param name="record">A ProgressReport object.</param>
    public override void WriteProgress(long sourceId, ProgressRecord record)
    {
    }

    /// <summary>
    /// Writes a verbose message to the output display of the host.
    /// </summary>
    /// <param name="message">The verbose message that is displayed.</param>
    public override void WriteVerboseLine(string message)
    {
      Console.WriteLine(String.Format(CultureInfo.CurrentCulture, "VERBOSE: {0}",
message));
    }

    /// <summary>
    /// Writes a warning message to the output display of the host.
    /// </summary>
    /// <param name="message">The warning message that is displayed.</param>
    public override void WriteWarningLine(string message)
    {
      Console.WriteLine(String.Format(CultureInfo.CurrentCulture, "WARNING: {0}",
```

```
message));
    }
  }
}
```

Example 4

The following code is the implementation of the System.Management.Automation.Host.PSHostRawUserInterface class that is used by this host application. Those elements that are not implemented throw an exception or return nothing.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Management.Automation.Host;

  /// <summary>
  /// A sample implementation of the PSHostRawUserInterface for console
  /// applications. Members of this class that easily map to the .NET
  /// console class are implemented. More complex methods are not
  /// implemented and throw a NotImplementedException exception.
  /// </summary>
  internal class MyRawUserInterface : PSHostRawUserInterface
  {
    /// <summary>
    /// Gets or sets the background color of the displayed text.
    /// This maps to the corresponding Console.Background property.
    /// </summary>
    public override ConsoleColor BackgroundColor
    {
      get { return Console.BackgroundColor; }
      set { Console.BackgroundColor = value; }
    }

    /// <summary>
    /// Gets or sets the size of the host buffer. In this example the
    /// buffer size is adapted from the Console buffer size members.
    /// </summary>
    public override Size BufferSize
    {
      get { return new Size(Console.BufferWidth, Console.BufferHeight); }
      set { Console.SetBufferSize(value.Width, value.Height); }
    }

    /// <summary>
    /// Gets or sets the cursor position. In this example this
    /// functionality is not needed so the property throws a
    /// NotImplementException exception.
    /// </summary>
```

```
    public override Coordinates CursorPosition
    {
      get { throw new NotImplementedException(
                  "The method or operation is not implemented."); }
      set { throw new NotImplementedException(
                  "The method or operation is not implemented."); }
    }

    /// <summary>
    /// Gets or sets the size of the displayed cursor. In this example
    /// the cursor size is taken directly from the Console.CursorSize
    /// property.
    /// </summary>
    public override int CursorSize
    {
      get { return Console.CursorSize; }
      set { Console.CursorSize = value; }
    }

    /// <summary>
    /// Gets or sets the foreground color of the displayed text.
    /// This maps to the corresponding Console.ForegroundColor property.
    /// </summary>
    public override ConsoleColor ForegroundColor
    {
      get { return Console.ForegroundColor; }
      set { Console.ForegroundColor = value; }
    }

    /// <summary>
    /// Gets a value indicating whether the user has pressed a key. This maps
    /// to the corresponding Console.KeyAvailable property.
    /// </summary>
    public override bool KeyAvailable
    {
      get { return Console.KeyAvailable; }
    }

    /// <summary>
    /// Gets the dimensions of the largest window that could be
    /// rendered in the current display, if the buffer was at the least
    /// that large. This example uses the Console.LargestWindowWidth and
    /// Console.LargestWindowHeight properties to determine the returned
    /// value of this property.
    /// </summary>
    public override Size MaxPhysicalWindowSize
    {
      get { return new Size(Console.LargestWindowWidth,
Console.LargestWindowHeight); }
    }

    /// <summary>
    /// Gets the dimensions of the largest window size that can be
    /// displayed. This example uses the Console.LargestWindowWidth and
    /// console.LargestWindowHeight properties to determine the returned
```

```
    /// value of this property.
    /// </summary>
    public override Size MaxWindowSize
    {
      get { return new Size(Console.LargestWindowWidth,
Console.LargestWindowHeight); }
    }

    /// <summary>
    /// Gets or sets the position of the displayed window. This example
    /// uses the Console window position APIs to determine the returned
    /// value of this property.
    /// </summary>
    public override Coordinates WindowPosition
    {
      get { return new Coordinates(Console.WindowLeft, Console.WindowTop); }
      set { Console.SetWindowPosition(value.X, value.Y); }
    }

    /// <summary>
    /// Gets or sets the size of the displayed window. This example
    /// uses the corresponding Console window size APIs to determine the
    /// returned value of this property.
    /// </summary>
    public override Size WindowSize
    {
      get { return new Size(Console.WindowWidth, Console.WindowHeight); }
      set { Console.SetWindowSize(value.Width, value.Height); }
    }

    /// <summary>
    /// Gets or sets the title of the displayed window. The example
    /// maps the Console.Title property to the value of this property.
    /// </summary>
    public override string WindowTitle
    {
      get { return Console.Title; }
      set { Console.Title = value; }
    }

    /// <summary>
    /// This API resets the input buffer. In this example this
    /// functionality is not needed so the method returns nothing.
    /// </summary>
    public override void FlushInputBuffer()
    {
    }

    /// <summary>
    /// This API returns a rectangular region of the screen buffer. In
    /// this example this functionality is not needed so the method throws
    /// a NotImplementException exception.
    /// </summary>
    /// <param name="rectangle">Defines the size of the rectangle.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
```

```
    public override BufferCell[,] GetBufferContents(Rectangle rectangle)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API reads a pressed, released, or pressed and released keystroke
    /// from the keyboard device, blocking processing until a keystroke is
    /// typed that matches the specified keystroke options. In this example
    /// this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="options">Options, such as IncludeKeyDown,  used when
    /// reading the keyboard.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override KeyInfo ReadKey(ReadKeyOptions options)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API crops a region of the screen buffer. In this example
    /// this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="source">The region of the screen to be scrolled.</param>
    /// <param name="destination">The region of the screen to receive the
    /// source region contents.</param>
    /// <param name="clip">The region of the screen to include in the operation.
</param>
    /// <param name="fill">The character and attributes to be used to fill all
cell.</param>
    public override void ScrollBufferContents(Rectangle source, Coordinates
destination, Rectangle clip, BufferCell fill)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }

    /// <summary>
    /// This method copies an array of buffer cells into the screen buffer
    /// at a specified location. In this example this functionality is
    /// not needed so the method throws a NotImplementedException exception.
    /// </summary>
    /// <param name="origin">The parameter is not used.</param>
    /// <param name="contents">The parameter is not used.</param>
    public override void SetBufferContents(Coordinates origin,
                                           BufferCell[,] contents)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }
```

```
    /// <summary>
    /// This method copies a given character, foreground color, and background
    /// color to a region of the screen buffer. In this example this
    /// functionality is not needed so the method throws a
    /// NotImplementException exception./// </summary>
    /// <param name="rectangle">Defines the area to be filled. </param>
    /// <param name="fill">Defines the fill character.</param>
    public override void SetBufferContents(Rectangle rectangle, BufferCell fill)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }
  }
}
```

See Also

System.Management.Automation.PowerShell

System.Management.Automation.Host.PSHost

System.Management.Automation.Host.PSHostUserInterface

System.Management.Automation.Host.PSHostRawUserInterface

Last updated on 05/20/2025

Host03 Sample

This sample shows how to build an interactive console-based host application that reads commands from the command line, executes the commands, and then displays the results to the console.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

- Creating a custom host whose classes derive from the System.Management.Automation.Host.PSHost class, the System.Management.Automation.Host.PSHostUserInterface class, and the System.Management.Automation.Host.PSHostRawUserInterface class.
- Building a console application that uses these host classes to build an interactive Windows PowerShell shell.

Example 1

This example allows the user to enter commands at a command line, processes those commands, and then prints out the results.

C#

```
// Copyright (c) 2006 Microsoft Corporation. All rights reserved.
//
// THIS CODE AND INFORMATION IS PROVIDED "AS IS" WITHOUT WARRANTY OF
// ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO
// THE IMPLIED WARRANTIES OF MERCHANTABILITY AND/OR FITNESS FOR A
// PARTICULAR PURPOSE.
//

namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Collections.ObjectModel;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;
  using PowerShell = System.Management.Automation.PowerShell;
```

```
/// This class contains the Main entry point for this host application.
internal class PSListenerConsoleSample
{
  /// Indicator to tell the host application that it should exit.
  private bool shouldExit;

  /// The exit code that the host application will use to exit.
  private int exitCode;

  /// Holds the instance of the PSHost implementation for this interpreter.
  private MyHost myHost;

  /// Holds the runspace for this interpreter.
  private Runspace myRunSpace;

  /// Holds a reference to the currently executing pipeline so it can be
  /// stopped by the control-C handler.
  private PowerShell currentPowerShell;

  /// Used to serialize access to instance data.
  private object instanceLock = new object();

  /// Create this instance of the console listener.
  private PSListenerConsoleSample()
  {
    // Create the host and runspace instances for this interpreter.
    // Note that this application does not support console files so
    // only the default snapins will be available.
    this.myHost = new MyHost(this);
    this.myRunSpace = RunspaceFactory.CreateRunspace(this.myHost);
    this.myRunSpace.Open();
  }

  /// Gets or sets a value indicating whether the host application
  /// should exit.
  public bool ShouldExit
  {
    get { return this.shouldExit; }
    set { this.shouldExit = value; }
  }

  /// Gets or sets the exit code that the host application will use
  /// when exiting.
  public int ExitCode
  {
    get { return this.exitCode; }
    set { this.exitCode = value; }
  }

  /// Creates and initiates the listener instance.
  /// param name="args";This parameter is not used.
  private static void Main(string[] args)
  {
    // Display the welcome message...
```

```
      Console.Title = "PowerShell Console Host Sample Application";
      ConsoleColor oldFg = Console.ForegroundColor;
      Console.ForegroundColor = ConsoleColor.Cyan;
      Console.WriteLine("    PowerShell Console Host Interactive Sample");
      Console.WriteLine("    ====================================");
      Console.WriteLine(string.Empty);
      Console.WriteLine("This is an example of a simple interactive console host
that uses the ");
      Console.WriteLine("Windows PowerShell engine to interpret commands. Type
'exit' to exit.");
      Console.WriteLine(string.Empty);
      Console.ForegroundColor = oldFg;

      // Create the listener and run it - this never returns...
      PSListenerConsoleSample listener = new PSListenerConsoleSample();
      listener.Run();
    }

    /// A helper class that builds and executes a pipeline that writes to the
    /// default output path. Any exceptions that are thrown are just passed to
    /// the caller. Since all output goes to the default outputter, this method()
    /// won't return anything.
    /// param name="cmd"; The script to run.
    /// param name="input";Any input arguments to pass to the script. If null
    /// then nothing is passed in.
    private void executeHelper(string cmd, object input)
    {
      // Ignore empty command lines.
      if (String.IsNullOrEmpty(cmd))
      {
        return;
      }

      // Create the pipeline object and make it available
      // to the ctrl-C handle through the currentPowerShell instance
      // variable
      lock (this.instanceLock)
      {
        this.currentPowerShell = PowerShell.Create();
      }

      this.currentPowerShell.Runspace = this.myRunSpace;

      // Create a pipeline for this execution. Place the result in the
      // currentPowerShell instance variable so that it is available
      // to be stopped.
      try
      {
        this.currentPowerShell.AddScript(cmd);

        // Now add the default outputter to the end of the pipe and indicate
        // that it should handle both output and errors from the previous
        // commands. This will result in the output being written using the PSHost
        // and PSHostUserInterface classes instead of returning objects to the
hosting
```

```
        // application.
        this.currentPowerShell.AddCommand("Out-Default");

this.currentPowerShell.Commands.Commands[0].MergeMyResults(PipelineResultTypes.Erro
r, PipelineResultTypes.Output);

        // If there was any input specified, pass it in, otherwise just
        // execute the pipeline.
        if (input != null)
        {
          this.currentPowerShell.Invoke(new object[] { input });
        }
        else
        {
          this.currentPowerShell.Invoke();
        }
      }
      finally
      {
        // Dispose of the pipeline line and set it to null, locked because
        // currentPowerShell may be accessed by the ctrl-C handler.
        lock (this.instanceLock)
        {
          this.currentPowerShell.Dispose();
          this.currentPowerShell = null;
        }
      }
    }

    /// An exception occurred that we want to display
    /// using the display formatter. To do this we run
    /// a second pipeline passing in the error record.
    /// The runtime will bind this to the $input variable
    /// which is why $input is being piped to Out-String.
    /// We then call WriteErrorLine to make sure the error
    /// gets displayed in the correct error color.

    /// param name="e"; The exception to display.
    private void ReportException(Exception e)
    {
      if (e != null)
      {
        object error;
        IContainsErrorRecord icer = e as IContainsErrorRecord;
        if (icer != null)
        {
          error = icer.ErrorRecord;
        }
        else
        {
          error = (object)new ErrorRecord(e, "Host.ReportException",
ErrorCategory.NotSpecified, null);
        }

        lock (this.instanceLock)
```

```
        {
          this.currentPowerShell = PowerShell.Create();
        }

        this.currentPowerShell.Runspace = this.myRunSpace;

        try
        {
          this.currentPowerShell.AddScript("$input").AddCommand("Out-String");

          // Do not merge errors, this function will swallow errors.
          Collection<PSObject> result;
          PSDataCollection<object> inputCollection = new PSDataCollection<object>
();
          inputCollection.Add(error);
          inputCollection.Complete();
          result = this.currentPowerShell.Invoke(inputCollection);

          if (result.Count > 0)
          {
            string str = result[0].BaseObject as string;
            if (!string.IsNullOrEmpty(str))
            {
              // Remove \r\n that is added by Out-String.
              this.myHost.UI.WriteErrorLine(str.Substring(0, str.Length - 2));
            }
          }
        }
        finally
        {
          // Dispose of the pipeline line and set it to null, locked because
currentPowerShell
          // may be accessed by the ctrl-C handler.
          lock (this.instanceLock)
          {
            this.currentPowerShell.Dispose();
            this.currentPowerShell = null;
          }
        }
      }
    }

    /// Basic script execution routine - any runtime exceptions are
    /// caught and passed back into the engine to display.

    /// param name="cmd"; The parameter is not used.
    private void Execute(string cmd)
    {
      try
      {
        // Execute the command with no input.
        this.executeHelper(cmd, null);
      }
      catch (RuntimeException rte)
      {
```

```
        this.ReportException(rte);
      }
    }

    /// Method used to handle control-C's from the user. It calls the
    /// pipeline Stop() method to stop execution. If any exceptions occur
    /// they are printed to the console but otherwise ignored.

    /// param name="sender"; See sender property of ConsoleCancelEventHandler
documentation.
    /// param name="e"; See e property of ConsoleCancelEventHandler documentation.
    private void HandleControlC(object sender, ConsoleCancelEventArgs e)
    {
      try
      {
        lock (this.instanceLock)
        {
          if (this.currentPowerShell != null &&
this.currentPowerShell.InvocationStateInfo.State == PSInvocationState.Running)
          {
            this.currentPowerShell.Stop();
          }
        }

        e.Cancel = true;
      }
      catch (Exception exception)
      {
        this.myHost.UI.WriteErrorLine(exception.ToString());
      }
    }

    /// Implements the basic listener loop. It sets up the ctrl-C handler, then
    /// reads a command from the user, executes it and repeats until the ShouldExit
    /// flag is set.
    private void Run()
    {
      // Set up the control-C handler.
      Console.CancelKeyPress += new ConsoleCancelEventHandler(this.HandleControlC);
      Console.TreatControlCAsInput = false;

      // Read commands to execute until ShouldExit is set by
      // the user calling "exit".
      while (!this.ShouldExit)
      {
        this.myHost.UI.Write(ConsoleColor.Cyan, ConsoleColor.Black,
"\nPSConsoleSample: ");
        string cmd = Console.ReadLine();
        this.Execute(cmd);
      }

      // Exit with the desired exit code that was set by exit command.
      // This is set in the host by the MyHost.SetShouldExit() implementation.
      Environment.Exit(this.ExitCode);
    }
```

```
  }
}
```

Example 2

The following code is the implementation of the System.Management.Automation.Host.PSHost class that is used by this host application. Those elements that are not implemented throw an exception or return nothing.

C#

```
// Copyright (c) 2006 Microsoft Corporation. All rights reserved.
//
// THIS CODE AND INFORMATION IS PROVIDED "AS IS" WITHOUT WARRANTY OF
// ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO
// THE IMPLIED WARRANTIES OF MERCHANTABILITY AND/OR FITNESS FOR A
// PARTICULAR PURPOSE.
//
using System;
using System.Collections.Generic;
using System.Text;
using System.Management.Automation;
using System.Management.Automation.Host;
using System.Management.Automation.Runspaces;

namespace Microsoft.Samples.PowerShell.Host
{
  /// <summary>
  /// Simple PowerShell interactive console host listener implementation. This
class
  /// implements a basic read-evaluate-print loop or 'listener' allowing you to
  /// interactively work with the PowerShell runtime.
  /// </summary>
  class PSListenerConsoleSample
  {
    /// <summary>
    /// Define the property that the PSHost implementation will use to tell the
host
    /// application that it should exit.
    /// </summary>
    public bool ShouldExit
    {
      get { return shouldExit; }
      set { shouldExit = value; }
    }
    private bool shouldExit;

    /// <summary>
    /// Define the property that the PSHost implementation will use to tell the
host
    /// application what code to use when exiting.
```

```
    /// </summary>
    public int ExitCode
    {
      get { return exitCode; }
      set { exitCode = value; }
    }
    private int exitCode;
    /// <summary>
    /// Holds the instance of the PSHost implementation for this interpreter.
    /// </summary>
    private MyHost myHost;

    /// <summary>
    /// Holds the runspace for this interpreter.
    /// </summary>
    private Runspace myRunSpace;

    /// <summary>
    /// Holds a reference to the currently executing pipeline so it can be
    /// stopped by the control-C handler.
    /// </summary>
    private Pipeline currentPipeline;

    /// <summary>
    /// Used to serialize access to instance data...
    /// </summary>
    private object instanceLock = new object();

    /// <summary>
    /// Create this instance of the console listener.
    /// </summary>
    PSListenerConsoleSample()
    {
      // Create the host and runspace instances for this interpreter. Note that
      // this application doesn't support console files so only the default snapins
      // will be available.
      myHost = new MyHost(this);
      myRunSpace = RunspaceFactory.CreateRunspace(myHost);
      myRunSpace.Open();
    }

    /// <summary>
    /// A helper class that builds and executes a pipeline that writes to the
    /// default output path. Any exceptions that are thrown are just passed to
    /// the caller. Since all output goes to the default outputter, this method()
    /// won't return anything.
    /// </summary>
    /// <param name="cmd">The script to run</param>
    /// <param name="input">Any input arguments to pass to the script. If null
    /// then nothing is passed in.</param>
    void executeHelper(string cmd, object input)
    {
      // Ignore empty command lines.
      if (String.IsNullOrEmpty(cmd))
        return;
```

```
      // Create the pipeline object and make it available
      // to the ctrl-C handle through the currentPipeline instance
      // variable.
      lock (instanceLock)
      {
        currentPipeline = myRunSpace.CreatePipeline();
      }

      // Create a pipeline for this execution. Place the result in the
currentPipeline
      // instance variable so that it is available to be stopped.
      try
      {
        currentPipeline.Commands.AddScript(cmd);

        // Now add the default outputter to the end of the pipe and indicate
        // that it should handle both output and errors from the previous
        // commands. This will result in the output being written using the PSHost
        // and PSHostUserInterface classes instead of returning objects to the
hosting
        // application.
        currentPipeline.Commands.Add("Out-Default");
        currentPipeline.Commands[0].MergeMyResults(PipelineResultTypes.Error,
PipelineResultTypes.Output);

        // If there was any input specified, pass it in, otherwise just
        // execute the pipeline.
        if (input != null)
        {
          currentPipeline.Invoke(new object[] { input });
        }
        else
        {
          currentPipeline.Invoke();
        }
      }
      finally
      {
        // Dispose of the pipeline line and set it to null, locked because
currentPipeline
        // may be accessed by the ctrl-C handler.
        lock (instanceLock)
        {
          currentPipeline.Dispose();
          currentPipeline = null;
        }
      }
    }

    /// <summary>
    /// Basic script execution routine - any runtime exceptions are
    /// caught and passed back into the runtime to display.
    /// </summary>
    /// <param name="cmd"></param>
```

```
    void Execute(string cmd)
    {
      try
      {
        // execute the command with no input...
        executeHelper(cmd, null);
      }
      catch (RuntimeException rte)
      {
        // An exception occurred that we want to display
        // using the display formatter. To do this we run
        // a second pipeline passing in the error record.
        // The runtime will bind this to the $input variable
        // which is why $input is being piped to Out-Default
        executeHelper("$input | Out-Default", rte.ErrorRecord);
      }
    }

    /// <summary>
    /// Method used to handle control-C's from the user. It calls the
    /// pipeline Stop() method to stop execution. If any exceptions occur,
    /// they are printed to the console; otherwise they are ignored.
    /// </summary>
    /// <param name="sender">See ConsoleCancelEventHandler documentation</param>
    /// <param name="e">See ConsoleCancelEventHandler documentation</param>
    void HandleControlC(object sender, ConsoleCancelEventArgs e)
    {
      try
      {
        lock (instanceLock)
        {
          if (currentPipeline != null && currentPipeline.PipelineStateInfo.State ==
PipelineState.Running)
                    currentPipeline.Stop();
        }
        e.Cancel = true;
      }
      catch (Exception exception)
      {
        this.myHost.UI.WriteErrorLine(exception.ToString());
      }
    }

    /// <summary>
    /// Implements the basic listener loop. It sets up the ctrl-C handler, then
    /// reads a command from the user, executes it and repeats until the ShouldExit
    /// flag is set.
    /// </summary>
    private void Run()
    {
      // Set up the control-C handler.
      Console.CancelKeyPress += new ConsoleCancelEventHandler(HandleControlC);
      Console.TreatControlCAsInput = false;

      // Loop reading commands to execute until ShouldExit is set by
```

```
      // the user calling "exit".
      while (!ShouldExit)
      {
        myHost.UI.Write(ConsoleColor.Cyan, ConsoleColor.Black, "\nPSConsoleSample:
");
        string cmd = Console.ReadLine();
        Execute(cmd);
      }

      // Exit with the desired exit code that was set by exit command.
      // This is set in the host by the MyHost.SetShouldExit() implementation.
      Environment.Exit(ExitCode);
    }

    /// <summary>
    /// Creates and initiates the listener instance.
    /// </summary>
    /// <param name="args">Ignored for now.</param>
    static void Main(string[] args)
    {
      // Display the welcome message.
      Console.Title = "PowerShell Console Host Sample Application";
      ConsoleColor oldFg = Console.ForegroundColor;
      Console.ForegroundColor = ConsoleColor.Cyan;
      Console.WriteLine("    PowerShell Console Host Interactive Sample");
      Console.WriteLine("    ======================================");
      Console.WriteLine("");
      Console.WriteLine("This is an example of a simple interactive console host
using the PowerShell");
      Console.WriteLine("engine to interpret commands. Type 'exit' to exit.");
      Console.WriteLine("");
      Console.ForegroundColor = oldFg;

      // Create the listener and run it - this never returns.
      PSListenerConsoleSample listener = new PSListenerConsoleSample();
      listener.Run();
    }
  }
}
```

Example 3

The following code is the implementation of the System.Management.Automation.Host.PSHostUserInterface class that is used by this host application.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
```

```
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Globalization;
using System.Management.Automation;
using System.Management.Automation.Host;
using System.Text;

/// <summary>
/// A sample implementation of the PSHostUserInterface abstract class for
/// console applications. Not all members are implemented. Those that are
/// not implemented throw a NotImplementedException exception or return
/// nothing. Members that are implemented include those that map easily to
/// Console APIs and a basic implementation of the prompt API provided.
/// </summary>
internal class MyHostUserInterface : PSHostUserInterface
{
  /// <summary>
  /// An instance of the PSRawUserInterface object.
  /// </summary>
  private MyRawUserInterface myRawUi = new MyRawUserInterface();

  /// <summary>
  /// Gets an instance of the PSRawUserInterface object for this host
  /// application.
  /// </summary>
  public override PSHostRawUserInterface RawUI
  {
    get { return this.myRawUi; }
  }

  /// <summary>
  /// Prompts the user for input.
  /// <param name="caption">The caption or title of the prompt.</param>
  /// <param name="message">The text of the prompt.</param>
  /// <param name="descriptions">A collection of FieldDescription objects that
  /// describe each field of the prompt.</param>
  /// <returns>A dictionary object that contains the results of the user
  /// prompts.</returns>
  public override Dictionary<string, PSObject> Prompt(
                                    string caption,
                                    string message,
                                    Collection<FieldDescription> descriptions)
  {
    this.Write(
               ConsoleColor.Blue,
               ConsoleColor.Black,
               caption + "\n" + message + " ");
    Dictionary<string, PSObject> results =
             new Dictionary<string, PSObject>();
    foreach (FieldDescription fd in descriptions)
    {
      string[] label = GetHotkeyAndLabel(fd.Label);
      this.WriteLine(label[1]);
      string userData = Console.ReadLine();
      if (userData == null)
```

```
        {
          return null;
        }

        results[fd.Name] = PSObject.AsPSObject(userData);
      }

      return results;
    }

    /// <summary>

/// Provides a set of choices that enable the user to choose a
    /// single option from a set of options.
    /// </summary>
    /// <param name="caption">Text that proceeds (a title) the choices.</param>
    /// <param name="message">A message that describes the choice.</param>
    /// <param name="choices">A collection of ChoiceDescription objects that
describe
    /// each choice.</param>
    /// <param name="defaultChoice">The index of the label in the Choices parameter
    /// collection. To indicate no default choice, set to -1.</param>
    /// <returns>The index of the Choices parameter collection element that
corresponds
    /// to the option that is selected by the user.</returns>
    public override int PromptForChoice(
                                        string caption,
                                        string message,
                                        Collection<ChoiceDescription> choices,
                                        int defaultChoice)
    {
      // Write the caption and message strings in Blue.
      this.WriteLine(
                    ConsoleColor.Blue,
                    ConsoleColor.Black,
                    caption + "\n" + message + "\n");

      // Convert the choice collection into something that is easier to
      // work with. See the BuildHotkeysAndPlainLabels method for details.
      Dictionary<string, PSObject> results =
          new Dictionary<string, PSObject>();
      string[,] promptData = BuildHotkeysAndPlainLabels(choices);

      // Format the overall choice prompt string to display...
      StringBuilder sb = new StringBuilder();
      for (int element = 0; element < choices.Count; element++)
      {
        sb.Append(String.Format(
                                CultureInfo.CurrentCulture,
                                "|{0}> {1} ",
                                promptData[0, element],
                                promptData[1, element]));
      }

      sb.Append(String.Format(
```

```
                                CultureInfo.CurrentCulture,
                                "[Default is ({0})]",
                                promptData[0, defaultChoice]));

      // Read prompts until a match is made, the default is
      // chosen, or the loop is interrupted with ctrl-C.
      while (true)
      {
        this.WriteLine(ConsoleColor.Cyan, ConsoleColor.Black, sb.ToString());
        string data =
Console.ReadLine().Trim().ToUpper(CultureInfo.CurrentCulture);

        // If the choice string was empty, use the default selection.
        if (data.Length == 0)
        {
          return defaultChoice;
        }

        // See if the selection matched and return the
        // corresponding index if it did.
        for (int i = 0; i < choices.Count; i++)
        {
          if (promptData[0, i] == data)
          {
            return i;
          }
        }

        this.WriteErrorLine("Invalid choice: " + data);
      }
    }

    /// <summary>
    /// Prompts the user for credentials with a specified prompt window caption,
    /// prompt message, user name, and target name. In this example this
    /// functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="caption">The caption for the message window.</param>
    /// <param name="message">The text of the message.</param>
    /// <param name="userName">The user name whose credential is to be prompted
for.</param>
    /// <param name="targetName">The name of the target for which the credential is
collected.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override PSCredential PromptForCredential(
                                                    string caption,
                                                    string message,
                                                    string userName,
                                                    string targetName)
    {
      throw new NotImplementedException("The method or operation is not
implemented.");
    }
```

```
    /// <summary>
    /// Prompts the user for credentials by using a specified prompt window
caption,
    /// prompt message, user name and target name, credential types allowed to be
    /// returned, and UI behavior options. In this example this functionality
    /// is not needed so the method throws a NotImplementException exception.
    /// </summary>
    /// <param name="caption">The caption for the message window.</param>
    /// <param name="message">The text of the message.</param>
    /// <param name="userName">The user name whose credential is to be prompted
for.</param>
    /// <param name="targetName">The name of the target for which the credential is
collected.</param>
    /// <param name="allowedCredentialTypes">A PSCredentialTypes constant that
    /// identifies the type of credentials that can be returned.</param>
    /// <param name="options">A PSCredentialUIOptions constant that identifies the
UI
    /// behavior when it gathers the credentials.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override PSCredential PromptForCredential(
                                                    string caption,
                                                    string message,
                                                    string userName,
                                                    string targetName,
                                                    PSCredentialTypes
allowedCredentialTypes,
                                                    PSCredentialUIOptions options)
    {
      throw new NotImplementedException("The method or operation is not
implemented.");
    }

    /// <summary>
    /// Reads characters that are entered by the user until a newline
    /// (carriage return) is encountered.
    /// </summary>
    /// <returns>The characters that are entered by the user.</returns>
    public override string ReadLine()
    {
      return Console.ReadLine();
    }

    /// <summary>
    /// Reads characters entered by the user until a newline (carriage return)
    /// is encountered and returns the characters as a secure string. In this
    /// example this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <returns>Throws a NotImplemented exception.</returns>
    public override System.Security.SecureString ReadLineAsSecureString()
    {
      throw new NotImplementedException("The method or operation is not
implemented.");
    }
```

```
    /// <summary>
    /// Writes characters to the output display of the host.
    /// </summary>
    /// <param name="value">The characters to be written.</param>
    public override void Write(string value)
    {
      Console.Write(value);
    }

    /// <summary>
    /// Writes characters to the output display of the host with possible
    /// foreground and background colors.
    /// </summary>
    /// <param name="foregroundColor">The color of the characters.</param>
    /// <param name="backgroundColor">The background color to use.</param>
    /// <param name="value">The characters to be written.</param>
    public override void Write(
                              ConsoleColor foregroundColor,
                              ConsoleColor backgroundColor,
                              string value)
    {
      ConsoleColor oldFg = Console.ForegroundColor;
      ConsoleColor oldBg = Console.BackgroundColor;
      Console.ForegroundColor = foregroundColor;
      Console.BackgroundColor = backgroundColor;
      Console.Write(value);
      Console.ForegroundColor = oldFg;
      Console.BackgroundColor = oldBg;
    }

    /// <summary>
    /// Writes a line of characters to the output display of the host
    /// with foreground and background colors and appends a newline (carriage
return).
    /// </summary>
    /// <param name="foregroundColor">The foreground color of the display. </param>
    /// <param name="backgroundColor">The background color of the display. </param>
    /// <param name="value">The line to be written.</param>
    public override void WriteLine(
                                  ConsoleColor foregroundColor,
                                  ConsoleColor backgroundColor,
                                  string value)
    {
      ConsoleColor oldFg = Console.ForegroundColor;
      ConsoleColor oldBg = Console.BackgroundColor;
      Console.ForegroundColor = foregroundColor;
      Console.BackgroundColor = backgroundColor;
      Console.WriteLine(value);
      Console.ForegroundColor = oldFg;
      Console.BackgroundColor = oldBg;
    }

    /// <summary>
    /// Writes a debug message to the output display of the host.
    /// </summary>
```

```
    /// <param name="message">The debug message that is displayed.</param>
    public override void WriteDebugLine(string message)
    {
      this.WriteLine(
                     ConsoleColor.DarkYellow,
                     ConsoleColor.Black,
                     String.Format(CultureInfo.CurrentCulture, "DEBUG: {0}",
message));
    }

    /// <summary>
    /// Writes an error message to the output display of the host.
    /// </summary>
    /// <param name="value">The error message that is displayed.</param>
    public override void WriteErrorLine(string value)
    {
      this.WriteLine(
                     ConsoleColor.Red,
                     ConsoleColor.Black,
                     value);
    }

    /// <summary>
    /// Writes a newline character (carriage return)
    /// to the output display of the host.
    /// </summary>
    public override void WriteLine()
    {
      Console.WriteLine();
    }

    /// <summary>
    /// Writes a line of characters to the output display of the host
    /// and appends a newline character(carriage return).
    /// </summary>
    /// <param name="value">The line to be written.</param>
    public override void WriteLine(string value)
    {
      Console.WriteLine(value);
    }

    /// <summary>
    /// Writes a progress report to the output display of the host.
    /// </summary>
    /// <param name="sourceId">Unique identifier of the source of the record.
</param>
    /// <param name="record">A ProgressReport object.</param>
    public override void WriteProgress(long sourceId, ProgressRecord record)
    {

    }

    /// <summary>
    /// Writes a verbose message to the output display of the host.
    /// </summary>
```

```
    /// <param name="message">The verbose message that is displayed.</param>
    public override void WriteVerboseLine(string message)
    {
      this.WriteLine(
                    ConsoleColor.Green,
                    ConsoleColor.Black,
                    String.Format(CultureInfo.CurrentCulture, "VERBOSE: {0}",
message));
    }

    /// <summary>
    /// Writes a warning message to the output display of the host.
    /// </summary>
    /// <param name="message">The warning message that is displayed.</param>
    public override void WriteWarningLine(string message)
    {
      this.WriteLine(
                    ConsoleColor.Yellow,
                    ConsoleColor.Black,
                    String.Format(CultureInfo.CurrentCulture, "WARNING: {0}",
message));
    }

    /// <summary>
    /// This is a private worker function splits out the
    /// accelerator keys from the menu and builds a two
    /// dimensional array with the first access containing the
    /// accelerator and the second containing the label string
    /// with the & removed.
    /// </summary>
    /// <param name="choices">The choice collection to process</param>
    /// <returns>
    /// A two dimensional array containing the accelerator characters
    /// and the cleaned-up labels</returns>
    private static string[,] BuildHotkeysAndPlainLabels(
        Collection<ChoiceDescription> choices)
    {
      // Allocate the result array.
      string[,] hotkeysAndPlainLabels = new string[2, choices.Count];
      for (int i = 0; i < choices.Count; ++i)
      {
        string[] hotkeyAndLabel = GetHotkeyAndLabel(choices[i].Label);
        hotkeysAndPlainLabels[0, i] = hotkeyAndLabel[0];
        hotkeysAndPlainLabels[1, i] = hotkeyAndLabel[1];
      }

      return hotkeysAndPlainLabels;
    }

    /// <summary>
    /// Parse a string containing a hotkey character.
    /// Take a string of the form
    ///    Yes to &all
    /// and returns a two-dimensional array split out as
    ///    "A", "Yes to all".
```

```
    /// </summary>
    /// <param name="input">The string to process</param>
    /// <returns>
    /// A two dimensional array containing the parsed components.
    /// </returns>
    private static string[] GetHotkeyAndLabel(string input)
    {
      string[] result = new string[] { String.Empty, String.Empty };
      string[] fragments = input.Split('&');
      if (fragments.Length == 2)
      {
        if (fragments[1].Length > 0)
        {
          result[0] = fragments[1][0].ToString().
          ToUpper(CultureInfo.CurrentCulture);
        }

        result[1] = (fragments[0] + fragments[1]).Trim();
      }
      else
      {
        result[1] = input;
      }

      return result;
    }
  }
}
```

Example 4

The following code is the implementation of the System.Management.Automation.Host.PSHostRawUserInterface class that is used by this host application. Those elements that are not implemented throw an exception or return nothing.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Management.Automation.Host;

  /// <summary>
  /// A sample implementation of the PSHostRawUserInterface for console
  /// applications. Members of this class that easily map to the .NET
  /// console class are implemented. More complex methods are not
  /// implemented and throw a NotImplementedException exception.
  /// </summary>
  internal class MyRawUserInterface : PSHostRawUserInterface
  {
    /// <summary>
```

```
    /// Gets or sets the background color of text to be written.
    /// This maps to the corresponding Console.Background property.
    /// </summary>
    public override ConsoleColor BackgroundColor
    {
      get { return Console.BackgroundColor; }
      set { Console.BackgroundColor = value; }
    }

    /// <summary>
    /// Gets or sets the host buffer size adapted from the Console buffer
    /// size members.
    /// </summary>
    public override Size BufferSize
    {
      get { return new Size(Console.BufferWidth, Console.BufferHeight); }
      set { Console.SetBufferSize(value.Width, value.Height); }
    }

    /// <summary>
    /// Gets or sets the cursor position. In this example this
    /// functionality is not needed so the property throws a
    /// NotImplementException exception.
    /// </summary>
    public override Coordinates CursorPosition
    {
      get { throw new NotImplementedException(
                "The method or operation is not implemented."); }
      set { throw new NotImplementedException(
                "The method or operation is not implemented."); }
    }

    /// <summary>
    /// Gets or sets the cursor size taken directly from the
    /// Console.CursorSize property.
    /// </summary>
    public override int CursorSize
    {
      get { return Console.CursorSize; }
      set { Console.CursorSize = value; }
    }

    /// <summary>
    /// Gets or sets the foreground color of the text to be written.
    /// This maps to the corresponding Console.ForegroundColor property.
    /// </summary>
    public override ConsoleColor ForegroundColor
    {
      get { return Console.ForegroundColor; }
      set { Console.ForegroundColor = value; }
    }

    /// <summary>
    /// Gets a value indicating whether a key is available. This maps to
    /// the corresponding Console.KeyAvailable property.
```

```
    /// </summary>
    public override bool KeyAvailable
    {
      get { return Console.KeyAvailable; }
    }

    /// <summary>
    /// Gets the maximum physical size of the window adapted from the
    ///  Console.LargestWindowWidth and Console.LargestWindowHeight
    ///  properties.
    /// </summary>
    public override Size MaxPhysicalWindowSize
    {
      get { return new Size(Console.LargestWindowWidth,
Console.LargestWindowHeight); }
    }

    /// <summary>
    /// Gets the maximum window size adapted from the
    /// Console.LargestWindowWidth and console.LargestWindowHeight
    /// properties.
    /// </summary>
    public override Size MaxWindowSize
    {
      get { return new Size(Console.LargestWindowWidth,
Console.LargestWindowHeight); }
    }

    /// <summary>
    /// Gets or sets the window position adapted from the Console window position
    /// members.
    /// </summary>
    public override Coordinates WindowPosition
    {
      get { return new Coordinates(Console.WindowLeft, Console.WindowTop); }
      set { Console.SetWindowPosition(value.X, value.Y); }
    }

    /// <summary>
    /// Gets or sets the window size adapted from the corresponding Console
    /// calls.
    /// </summary>
    public override Size WindowSize
    {
      get { return new Size(Console.WindowWidth, Console.WindowHeight); }
      set { Console.SetWindowSize(value.Width, value.Height); }
    }

    /// <summary>
    /// Gets or sets the title of the window mapped to the Console.Title
    /// property.
    /// </summary>
    public override string WindowTitle
    {
      get { return Console.Title; }
```

```
      set { Console.Title = value; }
    }

    /// <summary>
    /// This API resets the input buffer. In this example this
    /// functionality is not needed so the method returns nothing.
    /// </summary>
    public override void FlushInputBuffer()
    {
    }

    /// <summary>
    /// This API returns a rectangular region of the screen buffer. In
    /// this example this functionality is not needed so the method throws
    /// a NotImplementException exception.
    /// </summary>
    /// <param name="rectangle">Defines the size of the rectangle.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override BufferCell[,] GetBufferContents(Rectangle rectangle)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API Reads a pressed, released, or pressed and released keystroke
    /// from the keyboard device, blocking processing until a keystroke is
    /// typed that matches the specified keystroke options. In this example
    /// this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="options">Options, such as IncludeKeyDown,  used when
    /// reading the keyboard.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override KeyInfo ReadKey(ReadKeyOptions options)
    {
      throw new NotImplementedException(
                 "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API crops a region of the screen buffer. In this example
    /// this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="source">The region of the screen to be scrolled.</param>
    /// <param name="destination">The region of the screen to receive the
    /// source region contents.</param>
    /// <param name="clip">The region of the screen to include in the operation.
</param>
    /// <param name="fill">The character and attributes to be used to fill all
cell.</param>
    public override void ScrollBufferContents(Rectangle source, Coordinates
destination, Rectangle clip, BufferCell fill)
    {
```

```
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }

     /// <summary>
    /// This API copies an array of buffer cells into the screen buffer
    /// at a specified location. In this example this  functionality is
    /// not needed si the method  throws a NotImplementedException exception.
    /// </summary>
    /// <param name="origin">The parameter is not used.</param>
    /// <param name="contents">The parameter is not used.</param>
    public override void SetBufferContents(Coordinates origin, BufferCell[,]
contents)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API Copies a given character, foreground color, and background
    /// color to a region of the screen buffer. In this example this
    /// functionality is not needed so the method throws a
    /// NotImplementException exception./// </summary>
    /// <param name="rectangle">Defines the area to be filled. </param>
    /// <param name="fill">Defines the fill character.</param>
    public override void SetBufferContents(Rectangle rectangle, BufferCell fill)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }
  }
}
```

See Also

System.Management.Automation.Host.PSHost

System.Management.Automation.Host.PSHostUserInterface

System.Management.Automation.Host.PSHostRawUserInterface

Last updated on 05/20/2025

Host04 Sample

This sample shows how to build an interactive console-based host application that reads commands from the command line, executes the commands, and then displays the results to the console. This host application also supports displaying prompts that allow the user to specify multiple choices.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

- Creating a custom host whose classes derive from the System.Management.Automation.Host.PSHost class, the System.Management.Automation.Host.PSHostUserInterface class, and the System.Management.Automation.Host.PSHostRawUserInterface class.
- Building a console application that uses these host classes to build an interactive Windows PowerShell shell.
- Creating a `$PROFILE` variable and loading the following profiles.
  - current user, current host
  - current user, all hosts
  - all users, current host
  - all users, all hosts
- Implement the System.Management.Automation.Host.IHostUISupportsMultipleChoiceSelection interface.

Example 1

This example allows the user to enter commands at a command line, processes those commands, and then prints out the results.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
```

```
using System;
using System.Collections.ObjectModel;
using System.Management.Automation;
using System.Management.Automation.Runspaces;
using PowerShell = System.Management.Automation.PowerShell;

/// <summary>
/// This class contains the Main entry point for this host application.
/// </summary>
internal class PSListenerConsoleSample
{
  /// <summary>
  /// Indicator to tell the host application that it should exit.
  /// </summary>
  private bool shouldExit;

  /// <summary>
  /// The exit code that the host application will use to exit.
  /// </summary>
  private int exitCode;

  /// <summary>
  /// Holds a reference to  the PSHost object for this interpreter.
  /// </summary>
  private MyHost myHost;

  /// <summary>
  /// Holds a reference to the runspace for this interpreter.
  /// </summary>
  private Runspace myRunSpace;

  /// <summary>
  /// Holds a reference to the currently executing pipeline so that
  /// it can be stopped by the control-C handler.
  /// </summary>
  private PowerShell currentPowerShell;

  /// <summary>
  /// Used to serialize access to instance data.
  /// </summary>
  private object instanceLock = new object();

  /// <summary>
  /// Gets or sets a value indicating whether the host application
  /// should exit.
  /// </summary>
  public bool ShouldExit
  {
    get { return this.shouldExit; }
    set { this.shouldExit = value; }
  }

  /// <summary>
  /// Gets or sets the exit code that the host application will use
  /// when exiting.
```

```
    /// </summary>
    public int ExitCode
    {
      get { return this.exitCode; }
      set { this.exitCode = value; }
    }

    /// <summary>
    /// Creates and initiates the listener.
    /// </summary>
    /// <param name="args">The parameter is not used</param>
    private static void Main(string[] args)
    {
      // Display the welcome message.
      Console.Title = "Windows PowerShell Console Host Application Sample";
      ConsoleColor oldFg = Console.ForegroundColor;
      Console.ForegroundColor = ConsoleColor.Cyan;
      Console.WriteLine("    Windows PowerShell Console Host Interactive Sample");
      Console.WriteLine("    ================================================");
      Console.WriteLine(string.Empty);
      Console.WriteLine("This is an example of a simple interactive console host
that uses ");
      Console.WriteLine("the Windows PowerShell engine to interpret commands.");
      Console.WriteLine("Type 'exit' to exit.");
      Console.WriteLine(string.Empty);
      Console.ForegroundColor = oldFg;

      // Create the listener and run it. This never returns.
      PSListenerConsoleSample listener = new PSListenerConsoleSample();
      listener.Run();
    }

    /// <summary>
    /// Create this instance of the console listener.
    /// </summary>
    private PSListenerConsoleSample()
    {
      // Create the host and runspace instances for this interpreter. Note that
      // this application doesn't support console files so only the default snapins
      // will be available.
      this.myHost = new MyHost(this);
      this.myRunSpace = RunspaceFactory.CreateRunspace(this.myHost);
      this.myRunSpace.Open();

      // Create a PowerShell object that will be used to execute the commands
      // to create $PROFILE and load the profiles.
      lock (this.instanceLock)
      {
        this.currentPowerShell = PowerShell.Create();
      }

      try
      {
        this.currentPowerShell.Runspace = this.myRunSpace;
```

```
        PSCommand[] profileCommands =
Microsoft.Samples.PowerShell.Host.HostUtilities.GetProfileCommands("SampleHost04");
        foreach (PSCommand command in profileCommands)
        {
          this.currentPowerShell.Commands = command;
          this.currentPowerShell.Invoke();
        }
      }
      finally
      {
        // Dispose of the pipeline line and set it to null, locked because
currentPowerShell
        // may be accessed by the ctrl-C handler...
        lock (this.instanceLock)
        {
          this.currentPowerShell.Dispose();
          this.currentPowerShell = null;
        }
      }
    }

    /// <summary>
    /// A helper class that builds and executes a pipeline that writes to the
    /// default output path. Any exceptions that are thrown are just passed to
    /// the caller. Since all output goes to the default outputter, this method
    /// returns nothing.
    /// </summary>
    /// <param name="cmd">The script to run</param>
    /// <param name="input">Any input arguments to pass to the script. If null
    /// then nothing is passed in.</param>
    private void executeHelper(string cmd, object input)
    {
      // Ignore empty command lines.
      if (String.IsNullOrEmpty(cmd))
      {
        return;
      }

      // Create the pipeline object and make it available
      // to the ctrl-C handle through the currentPowerShell instance
      // variable.
      lock (this.instanceLock)
      {
        this.currentPowerShell = PowerShell.Create();
      }

      this.currentPowerShell.Runspace = this.myRunSpace;

      // Create a pipeline for this execution. Place the result in the
      // currentPowerShell instance variable so that it is available to
      // be stopped.
      try
      {
        this.currentPowerShell.AddScript(cmd);
```

```
        // Now add the default outputter to the end of the pipe and indicate
        // that it should handle both output and errors from the previous
        // commands. This will result in the output being written using the PSHost
        // and PSHostUserInterface classes instead of returning objects to the
hosting
        // application.
        this.currentPowerShell.AddCommand("Out-Default");

this.currentPowerShell.Commands.Commands[0].MergeMyResults(PipelineResultTypes.Erro
r, PipelineResultTypes.Output);

        // If there was any input specified, pass it in, otherwise just
        // execute the pipeline.
        if (input != null)
        {
          this.currentPowerShell.Invoke(new object[] { input });
        }
        else
        {
          this.currentPowerShell.Invoke();
        }
      }
      finally
      {
        // Dispose of the pipeline line and set it to null, locked because
currentPowerShell
        // may be accessed by the ctrl-C handler.
        lock (this.instanceLock)
        {
          this.currentPowerShell.Dispose();
          this.currentPowerShell = null;
        }
      }
    }

    /// <summary>
    /// An exception occurred that we want to display
    /// using the display formatter. To do this we run
    /// a second pipeline passing in the error record.
    /// The runtime will bind this to the $input variable
    /// which is why $input is being piped to Out-String.
    /// We then call WriteErrorLine to make sure the error
    /// gets displayed in the correct error color.
    /// </summary>
    /// <param name="e">The exception to display</param>
    private void ReportException(Exception e)
    {
      if (e != null)
      {
        object error;
        IContainsErrorRecord icer = e as IContainsErrorRecord;
        if (icer != null)
        {
          error = icer.ErrorRecord;
        }
```

```
        else
        {
          error = (object)new ErrorRecord(e, "Host.ReportException",
ErrorCategory.NotSpecified, null);
        }

        lock (this.instanceLock)
        {
          this.currentPowerShell = PowerShell.Create();
        }

        this.currentPowerShell.Runspace = this.myRunSpace;

        try
        {
          this.currentPowerShell.AddScript("$input").AddCommand("Out-String");

          // Do not merge errors, this function will swallow errors.
          Collection<PSObject> result;
          PSDataCollection<object> inputCollection = new PSDataCollection<object>
();
          inputCollection.Add(error);
          inputCollection.Complete();
          result = this.currentPowerShell.Invoke(inputCollection);

          if (result.Count > 0)
          {
            string str = result[0].BaseObject as string;
            if (!string.IsNullOrEmpty(str))
            {
              // Remove \r\n that is added by Out-String.
              this.myHost.UI.WriteErrorLine(str.Substring(0, str.Length - 2));
            }
          }
        }
        finally
        {
          // Dispose of the pipeline line and set it to null, locked because
currentPowerShell
          // may be accessed by the ctrl-C handler.
          lock (this.instanceLock)
          {
            this.currentPowerShell.Dispose();
            this.currentPowerShell = null;
          }
        }
      }
    }

    /// <summary>
    /// Basic script execution routine - any runtime exceptions are
    /// caught and passed back into the engine to display.
    /// </summary>
    /// <param name="cmd">The parameter is not used.</param>
    private void Execute(string cmd)
```

```
    {
      try
      {
        // Execute the command with no input.
        this.executeHelper(cmd, null);
      }
      catch (RuntimeException rte)
      {
        this.ReportException(rte);
      }
    }

    /// <summary>
    /// Method used to handle control-C's from the user. It calls the
    /// pipeline Stop() method to stop execution. If any exceptions occur
    /// they are printed to the console but otherwise ignored.
    /// </summary>
    /// <param name="sender">See sender property of ConsoleCancelEventHandler
documentation.</param>
    /// <param name="e">See e property of ConsoleCancelEventHandler
documentation</param>
    private void HandleControlC(object sender, ConsoleCancelEventArgs e)
    {
      try
      {
        lock (this.instanceLock)
        {
          if (this.currentPowerShell != null &&
this.currentPowerShell.InvocationStateInfo.State == PSInvocationState.Running)
          {
            this.currentPowerShell.Stop();
          }
        }

        e.Cancel = true;
      }
      catch (Exception exception)
      {
        this.myHost.UI.WriteErrorLine(exception.ToString());
      }
    }

    /// <summary>
    /// Implements the basic listener loop. It sets up the ctrl-C handler, then
    /// reads a command from the user, executes it and repeats until the ShouldExit
    /// flag is set.
    /// </summary>
    private void Run()
    {
      // Set up the control-C handler.
      Console.CancelKeyPress += new ConsoleCancelEventHandler(this.HandleControlC);
      Console.TreatControlCAsInput = false;

      // Read commands to execute until ShouldExit is set by
      // the user calling "exit".
```

```
      while (!this.ShouldExit)
      {
        this.myHost.UI.Write(ConsoleColor.Cyan, ConsoleColor.Black,
"\nPSConsoleSample: ");
        string cmd = Console.ReadLine();
        this.Execute(cmd);
      }

      // Exit with the desired exit code that was set by exit command.
      // This is set in the host by the MyHost.SetShouldExit() implementation.
      Environment.Exit(this.ExitCode);
    }
  }
}
```

Example 2

The following code is the implementation of the System.Management.Automation.Host.PSHost class that is used by this host application. Those elements that are not implemented throw an exception or return nothing.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Globalization;
  using System.Management.Automation.Host;

  /// <summary>
  /// This is a sample implementation of the PSHost abstract class for
  /// console applications. Not all members are implemented. Those that
  /// are not implemented throw a NotImplementedException exception or
  /// return nothing.
  /// </summary>
  internal class MyHost : PSHost
  {
    /// <summary>
    /// A reference to the PSHost implementation.
    /// </summary>
    private PSListenerConsoleSample program;

    /// <summary>
    /// The culture information of the thread that created
    /// this object.
    /// </summary>
    private CultureInfo originalCultureInfo =
        System.Threading.Thread.CurrentThread.CurrentCulture;

    /// <summary>
    /// The UI culture information of the thread that created
```

```
    /// this object.
    /// </summary>
    private CultureInfo originalUICultureInfo =
        System.Threading.Thread.CurrentThread.CurrentUICulture;

    /// <summary>
    /// The identifier of this PSHost implementation.
    /// </summary>
    private static Guid instanceId = Guid.NewGuid();

    /// <summary>
    /// Initializes a new instance of the MyHost class. Keep
    /// a reference to the host application object so that it
    /// can be informed of when to exit.
    /// </summary>
    /// <param name="program">
    /// A reference to the host application object.
    /// </param>
    public MyHost(PSListenerConsoleSample program)
    {
      this.program = program;
    }

    /// <summary>
    /// A reference to the implementation of the PSHostUserInterface
    /// class for this application.
    /// </summary>
    private MyHostUserInterface myHostUserInterface = new MyHostUserInterface();

    /// <summary>
    /// Gets the culture information to use. This implementation
    /// returns a snapshot of the culture information of the thread
    /// that created this object.
    /// </summary>
    public override CultureInfo CurrentCulture
    {
      get { return this.originalCultureInfo; }
    }

    /// <summary>
    /// Gets the UI culture information to use. This implementation
    /// returns a snapshot of the UI culture information of the thread
    /// that created this object.
    /// </summary>
    public override CultureInfo CurrentUICulture
    {
      get { return this.originalUICultureInfo; }
    }

    /// <summary>
    /// Gets an identifier for this host. This implementation always
    /// returns the GUID allocated at instantiation time.
    /// </summary>
    public override Guid InstanceId
    {
```

```
    get { return instanceId; }
  }

  /// <summary>
  /// Gets a string that contains the name of this host implementation.
  /// Keep in mind that this string may be used by script writers to
  /// identify when your host is being used.
  /// </summary>
  public override string Name
  {
    get { return "MySampleConsoleHostImplementation"; }
  }

  /// <summary>
  /// Gets an instance of the implementation of the PSHostUserInterface
  /// class for this application. This instance is allocated once at startup time
  /// and returned every time thereafter.
  /// </summary>
  public override PSHostUserInterface UI
  {
    get { return this.myHostUserInterface; }
  }

  /// <summary>
  /// Gets the version object for this application. Typically this
  /// should match the version resource in the application.
  /// </summary>
  public override Version Version
  {
    get { return new Version(1, 0, 0, 0); }
  }

  /// <summary>
  /// This API Instructs the host to interrupt the currently running
  /// pipeline and start a new nested input loop. In this example this
  /// functionality is not needed so the method throws a
  /// NotImplementedException exception.
  /// </summary>
  public override void EnterNestedPrompt()
  {
    throw new NotImplementedException(
          "The method or operation is not implemented.");
  }

  /// <summary>
  /// This API instructs the host to exit the currently running input loop.
  /// In this example this functionality is not needed so the method
  /// throws a NotImplementedException exception.
  /// </summary>
  public override void ExitNestedPrompt()
  {
    throw new NotImplementedException(
          "The method or operation is not implemented.");
  }
```

```
    /// <summary>
    /// This API is called before an external application process is
    /// started. Typically it is used to save state so that the parent
    /// can restore state that has been modified by a child process (after
    /// the child exits). In this example this functionality is not
    /// needed so the method returns nothing.
    /// </summary>
    public override void NotifyBeginApplication()
    {
      return;
    }

    /// <summary>
    /// This API is called after an external application process finishes.
    /// Typically it is used to restore state that a child process has
    /// altered. In this example, this functionality is not needed so
    /// the method returns nothing.
    /// </summary>
    public override void NotifyEndApplication()
    {
      return;
    }

    /// <summary>
    /// Indicate to the host application that exit has
    /// been requested. Pass the exit code that the host
    /// application should use when exiting the process.
    /// </summary>
    /// <param name="exitCode">The exit code that the
    /// host application should use.</param>
    public override void SetShouldExit(int exitCode)
    {
      this.program.ShouldExit = true;
      this.program.ExitCode = exitCode;
    }
  }
}
```

Example 3

The following code is the implementation of the System.Management.Automation.Host.PSHostUserInterface class that is used by this host application.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Collections.Generic;
  using System.Collections.ObjectModel;
```

```
  using System.Globalization;
  using System.Management.Automation;
  using System.Management.Automation.Host;
  using System.Text;

  /// <summary>
  /// A sample implementation of the PSHostUserInterface abstract class for
  /// console applications. Not all members are implemented. Those that are
  /// not implemented throw a NotImplementedException exception or return
  /// nothing. Members that are implemented include those that map easily to
  /// Console APIs and a basic implementation of the prompt API provided.
  /// </summary>
  internal class MyHostUserInterface : PSHostUserInterface,
IHostUISupportsMultipleChoiceSelection
  {
    /// <summary>
    /// A reference to the PSRawUserInterface implementation.
    /// </summary>
    private MyRawUserInterface myRawUi = new MyRawUserInterface();

    /// <summary>
    /// Gets an instance of the PSRawUserInterface object for this host
    /// application.
    /// </summary>
    public override PSHostRawUserInterface RawUI
    {
      get { return this.myRawUi; }
    }

    /// <summary>
    /// Prompts the user for input.
    /// <param name="caption">The caption or title of the prompt.</param>
    /// <param name="message">The text of the prompt.</param>
    /// <param name="descriptions">A collection of FieldDescription objects
    /// that describe each field of the prompt.</param>
    /// <returns>A dictionary object that contains the results of the user
    /// prompts.</returns>
    public override Dictionary<string, PSObject> Prompt(
                              string caption,
                              string message,
                              Collection<FieldDescription> descriptions)
    {
      this.Write(
                 ConsoleColor.Blue,
                 ConsoleColor.Black,
                 caption + "\n" + message + " ");
      Dictionary<string, PSObject> results =
                new Dictionary<string, PSObject>();
      foreach (FieldDescription fd in descriptions)
      {
        string[] label = GetHotkeyAndLabel(fd.Label);
        this.WriteLine(label[1]);
        string userData = Console.ReadLine();
        if (userData == null)
        {
```

```
          return null;
        }

        results[fd.Name] = PSObject.AsPSObject(userData);
      }

      return results;
    }

    /// <summary>

/// Provides a set of choices that enable the user to choose a
    /// single option from a set of options.
    /// </summary>
    /// <param name="caption">Text that proceeds (a title) the choices.</param>
    /// <param name="message">A message that describes the choice.</param>
    /// <param name="choices">A collection of ChoiceDescription objects that
    /// describe each choice.</param>
    /// <param name="defaultChoice">The index of the label in the Choices
    /// parameter collection. To indicate no default choice, set to -1.</param>
    /// <returns>The index of the Choices parameter collection element that
    /// corresponds to the option that is selected by the user.</returns>
    public override int PromptForChoice(
                                        string caption,
                                        string message,
                                        Collection<ChoiceDescription> choices,
                                        int defaultChoice)
    {
      // Write the caption and message strings in Blue.
      this.WriteLine(
                    ConsoleColor.Blue,
                    ConsoleColor.Black,
                    caption + "\n" + message + "\n");

      // Convert the choice collection into something that is
      // easier to work with. See the BuildHotkeysAndPlainLabels
      // method for details.
      string[,] promptData = BuildHotkeysAndPlainLabels(choices);

      // Format the overall choice prompt string to display.
      StringBuilder sb = new StringBuilder();
      for (int element = 0; element < choices.Count; element++)
      {
        sb.Append(String.Format(
                                CultureInfo.CurrentCulture,
                                "|{0}> {1} ",
                                promptData[0, element],
                                promptData[1, element]));
      }

      sb.Append(String.Format(
                              CultureInfo.CurrentCulture,
                              "[Default is ({0})]",
                              promptData[0, defaultChoice]));
```

```
      // Read prompts until a match is made, the default is
      // chosen, or the loop is interrupted with ctrl-C.
      while (true)
      {
        this.WriteLine(ConsoleColor.Cyan, ConsoleColor.Black, sb.ToString());
        string data =
Console.ReadLine().Trim().ToUpper(CultureInfo.CurrentCulture);

        // If the choice string was empty, use the default selection.
        if (data.Length == 0)
        {
          return defaultChoice;
        }

        // See if the selection matched and return the
        // corresponding index if it did.
        for (int i = 0; i < choices.Count; i++)
        {
          if (promptData[0, i] == data)
          {
            return i;
          }
        }

        this.WriteErrorLine("Invalid choice: " + data);
      }
    }

    #region IHostUISupportsMultipleChoiceSelection Members

    /// <summary>
    /// Provides a set of choices that enable the user to choose a one or
    /// more options from a set of options.
    /// </summary>
    /// <param name="caption">Text that proceeds (a title) the choices.</param>
    /// <param name="message">A message that describes the choice.</param>
    /// <param name="choices">A collection of ChoiceDescription objects that
    /// describe each choice.</param>
    /// <param name="defaultChoices">The index of the label in the Choices
    /// parameter collection. To indicate no default choice, set to -1.</param>
    /// <returns>The index of the Choices parameter collection element that
    /// corresponds to the option that is selected by the user.</returns>
    public Collection<int> PromptForChoice(
                                           string caption,
                                           string message,
                                           Collection<ChoiceDescription> choices,
                                           IEnumerable<int> defaultChoices)
    {
      // Write the caption and message strings in Blue.
      this.WriteLine(
                     ConsoleColor.Blue,
                     ConsoleColor.Black,
                     caption + "\n" + message + "\n");

      // Convert the choice collection into something that is
```

```
      // easier to work with. See the BuildHotkeysAndPlainLabels
      // method for details.
      string[,] promptData = BuildHotkeysAndPlainLabels(choices);

      // Format the overall choice prompt string to display.
      StringBuilder sb = new StringBuilder();
      for (int element = 0; element < choices.Count; element++)
      {
        sb.Append(String.Format(
                                CultureInfo.CurrentCulture,
                                "|{0}> {1} ",
                                promptData[0, element],
                                promptData[1, element]));
      }

      Collection<int> defaultResults = new Collection<int>();
      if (defaultChoices != null)
      {
        int countDefaults = 0;
        foreach (int defaultChoice in defaultChoices)
        {
          ++countDefaults;
          defaultResults.Add(defaultChoice);
        }

        if (countDefaults != 0)
        {
          sb.Append(countDefaults == 1 ? "[Default choice is " : "[Default choices
are ");
          foreach (int defaultChoice in defaultChoices)
          {
            sb.AppendFormat(
                            CultureInfo.CurrentCulture,
                            "\"{0}\",",
                            promptData[0, defaultChoice]);
          }

          sb.Remove(sb.Length - 1, 1);
          sb.Append("]");
        }
      }

      this.WriteLine(
                     ConsoleColor.Cyan,
                     ConsoleColor.Black,
                     sb.ToString());
      // Read prompts until a match is made, the default is
      // chosen, or the loop is interrupted with ctrl-C.
      Collection<int> results = new Collection<int>();
      while (true)
      {
        ReadNext:
        string prompt = string.Format(CultureInfo.CurrentCulture, "Choice[{0}]:",
results.Count);
        this.Write(ConsoleColor.Cyan, ConsoleColor.Black, prompt);
```

```
        string data =
Console.ReadLine().Trim().ToUpper(CultureInfo.CurrentCulture);

        // If the choice string was empty, no more choices have been made.
        // If there were no choices made, return the defaults
        if (data.Length == 0)
        {
          return (results.Count == 0) ? defaultResults : results;
        }

        // See if the selection matched and return the
        // corresponding index if it did.
        for (int i = 0; i < choices.Count; i++)
        {
          if (promptData[0, i] == data)
          {
            results.Add(i);
            goto ReadNext;
          }
        }

        this.WriteErrorLine("Invalid choice: " + data);
      }
    }

    #endregion

    /// <summary>
    /// Prompts the user for credentials with a specified prompt window
    /// caption, prompt message, user name, and target name. In this
    /// example this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="caption">The caption for the message window.</param>
    /// <param name="message">The text of the message.</param>
    /// <param name="userName">The user name whose credential is to be
    /// prompted for.</param>
    /// <param name="targetName">The name of the target for which the
    /// credential is collected.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override PSCredential PromptForCredential(
                                                    string caption,
                                                    string message,
                                                    string userName,
                                                    string targetName)
    {
      throw new NotImplementedException(
                          "The method or operation is not implemented.");
    }

    /// <summary>
    /// Prompts the user for credentials by using a specified prompt window
    /// caption, prompt message, user name and target name, credential
    /// types allowed to be returned, and UI behavior options. In this
    /// example this functionality is not needed so the method throws a
```

```
    /// NotImplementException exception.
    /// </summary>
    /// <param name="caption">The caption for the message window.</param>
    /// <param name="message">The text of the message.</param>
    /// <param name="userName">The user name whose credential is to be
    /// prompted for.</param>
    /// <param name="targetName">The name of the target for which the
    /// credential is collected.</param>
    /// <param name="allowedCredentialTypes">A PSCredentialTypes constant
    /// that identifies the type of credentials that can be returned.</param>
    /// <param name="options">A PSCredentialUIOptions constant that
    /// identifies the UI behavior when it gathers the credentials.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override PSCredential PromptForCredential(
                                        string caption,
                                        string message,
                                        string userName,
                                        string targetName,
                                        PSCredentialTypes allowedCredentialTypes,
                                        PSCredentialUIOptions options)
    {
      throw new NotImplementedException(
                                "The method or operation is not implemented.");
    }

    /// <summary>
    /// Reads characters that are entered by the user until a newline
    /// (carriage return) is encountered.
    /// </summary>
    /// <returns>The characters that are entered by the user.</returns>
    public override string ReadLine()
    {
      return Console.ReadLine();
    }

    /// <summary>
    /// Reads characters entered by the user until a newline (carriage return)
    /// is encountered and returns the characters as a secure string. In this
    /// example this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <returns>Throws a NotImplemented exception.</returns>
    public override System.Security.SecureString ReadLineAsSecureString()
    {
      throw new NotImplementedException("The method or operation is not
implemented.");
    }

    /// <summary>
    /// Writes characters to the output display of the host.
    /// </summary>
    /// <param name="value">The characters to be written.</param>
    public override void Write(string value)
    {
      Console.Write(value);
```

```
    }

    /// <summary>
    /// Writes characters to the output display of the host with possible
    /// foreground and background colors.
    /// </summary>
    /// <param name="foregroundColor">The color of the characters.</param>
    /// <param name="backgroundColor">The background color to use.</param>
    /// <param name="value">The characters to be written.</param>
    public override void Write(
                               ConsoleColor foregroundColor,
                               ConsoleColor backgroundColor,
                               string value)
    {
      ConsoleColor oldFg = Console.ForegroundColor;
      ConsoleColor oldBg = Console.BackgroundColor;
      Console.ForegroundColor = foregroundColor;
      Console.BackgroundColor = backgroundColor;
      Console.Write(value);
      Console.ForegroundColor = oldFg;
      Console.BackgroundColor = oldBg;
    }

    /// <summary>
    /// Writes a line of characters to the output display of the host
    /// with foreground and background colors and appends a newline (carriage
return).
    /// </summary>
    /// <param name="foregroundColor">The foreground color of the display. </param>
    /// <param name="backgroundColor">The background color of the display. </param>
    /// <param name="value">The line to be written.</param>
    public override void WriteLine(
                                   ConsoleColor foregroundColor,
                                   ConsoleColor backgroundColor,
                                   string value)
    {
      ConsoleColor oldFg = Console.ForegroundColor;
      ConsoleColor oldBg = Console.BackgroundColor;
      Console.ForegroundColor = foregroundColor;
      Console.BackgroundColor = backgroundColor;
      Console.WriteLine(value);
      Console.ForegroundColor = oldFg;
      Console.BackgroundColor = oldBg;
    }

    /// <summary>
    /// Writes a debug message to the output display of the host.
    /// </summary>
    /// <param name="message">The debug message that is displayed.</param>
    public override void WriteDebugLine(string message)
    {
      this.WriteLine(
                     ConsoleColor.DarkYellow,
                     ConsoleColor.Black,
                     String.Format(CultureInfo.CurrentCulture, "DEBUG: {0}",
```

```
message));
    }

    /// <summary>
    /// Writes an error message to the output display of the host.
    /// </summary>
    /// <param name="value">The error message that is displayed.</param>
    public override void WriteErrorLine(string value)
    {
      this.WriteLine(
                    ConsoleColor.Red,
                    ConsoleColor.Black,
                    value);
    }

    /// <summary>
    /// Writes a newline character (carriage return)
    /// to the output display of the host.
    /// </summary>
    public override void WriteLine()
    {
      Console.WriteLine();
    }

       /// <summary>
    /// Writes a line of characters to the output display of the host
    /// and appends a newline character(carriage return).
    /// </summary>
    /// <param name="value">The line to be written.</param>
    public override void WriteLine(string value)
    {
      Console.WriteLine(value);
    }

    /// <summary>
    /// Writes a progress report to the output display of the host.
    /// </summary>
    /// <param name="sourceId">Unique identifier of the source of the record.
</param>
    /// <param name="record">A ProgressReport object.</param>
    public override void WriteProgress(long sourceId, ProgressRecord record)
    {

    }

    /// <summary>
    /// Writes a verbose message to the output display of the host.
    /// </summary>
    /// <param name="message">The verbose message that is displayed.</param>
    public override void WriteVerboseLine(string message)
    {
      this.WriteLine(
                    ConsoleColor.Green,
                    ConsoleColor.Black,
                    String.Format(CultureInfo.CurrentCulture, "VERBOSE: {0}",
```

```
message));
    }

    /// <summary>
    /// Writes a warning message to the output display of the host.
    /// </summary>
    /// <param name="message">The warning message that is displayed.</param>
    public override void WriteWarningLine(string message)
    {
      this.WriteLine(
                    ConsoleColor.Yellow,
                    ConsoleColor.Black,
                    String.Format(CultureInfo.CurrentCulture, "WARNING: {0}",
message));
    }

    /// <summary>
    /// Parse a string containing a hotkey character.
    /// Take a string of the form
    ///    Yes to &all
    /// and returns a two-dimensional array split out as
    ///    "A", "Yes to all".
    /// </summary>
    /// <param name="input">The string to process</param>
    /// <returns>
    /// A two dimensional array containing the parsed components.
    /// </returns>
    private static string[] GetHotkeyAndLabel(string input)
    {
      string[] result = new string[] { String.Empty, String.Empty };
      string[] fragments = input.Split('&');
      if (fragments.Length == 2)
      {
        if (fragments[1].Length > 0)
        {
          result[0] = fragments[1][0].ToString().
          ToUpper(CultureInfo.CurrentCulture);
        }

        result[1] = (fragments[0] + fragments[1]).Trim();
      }
      else
      {
        result[1] = input;
      }

      return result;
    }

    /// <summary>
    /// This is a private worker function splits out the
    /// accelerator keys from the menu and builds a two
    /// dimensional array with the first access containing the
    /// accelerator and the second containing the label string
    /// with the & removed.
```

```
    /// </summary>
    /// <param name="choices">The choice collection to process</param>
    /// <returns>
    /// A two dimensional array containing the accelerator characters
    /// and the cleaned-up labels</returns>
    private static string[,] BuildHotkeysAndPlainLabels(
         Collection<ChoiceDescription> choices)
    {
      // Allocate the result array
      string[,] hotkeysAndPlainLabels = new string[2, choices.Count];

      for (int i = 0; i < choices.Count; ++i)
      {
        string[] hotkeyAndLabel = GetHotkeyAndLabel(choices[i].Label);
        hotkeysAndPlainLabels[0, i] = hotkeyAndLabel[0];
        hotkeysAndPlainLabels[1, i] = hotkeyAndLabel[1];
      }

      return hotkeysAndPlainLabels;
    }
  }
}
```

Example 4

The following code is the implementation of the System.Management.Automation.Host.PSHostRawUserInterface class that is used by this host application. Those elements that are not implemented throw an exception or return nothing.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Management.Automation.Host;

  /// <summary>
  /// A sample implementation of the PSHostRawUserInterface for console
  /// applications. Members of this class that easily map to the .NET
  /// console class are implemented. More complex methods are not
  /// implemented and throw a NotImplementedException exception.
  /// </summary>
  internal class MyRawUserInterface : PSHostRawUserInterface
  {
    /// <summary>
    /// Gets or sets the background color of text to be written.
    /// This maps to the corresponding Console.Background property.
    /// </summary>
    public override ConsoleColor BackgroundColor
    {
      get { return Console.BackgroundColor; }
```

```
    set { Console.BackgroundColor = value; }
  }

  /// <summary>
  /// Gets or sets the host buffer size adapted from the Console buffer
  /// size members.
  /// </summary>
  public override Size BufferSize
  {
    get { return new Size(Console.BufferWidth, Console.BufferHeight); }
    set { Console.SetBufferSize(value.Width, value.Height); }
  }

   /// <summary>
  /// Gets or sets the cursor position. In this example this
  /// functionality is not needed so the property throws a
  /// NotImplementException exception.
  /// </summary>
  public override Coordinates CursorPosition
  {
    get { throw new NotImplementedException(
                "The method or operation is not implemented."); }
    set { throw new NotImplementedException(
                "The method or operation is not implemented."); }
  }

  /// <summary>
  /// Gets or sets the cursor size taken directly from the
  /// Console.CursorSize property.
  /// </summary>
  public override int CursorSize
  {
    get { return Console.CursorSize; }
    set { Console.CursorSize = value; }
  }

  /// <summary>
  /// Gets or sets the foreground color of the text to be written.
  /// This maps to the corresponding Console.ForegroundColor property.
  /// </summary>
  public override ConsoleColor ForegroundColor
  {
    get { return Console.ForegroundColor; }
    set { Console.ForegroundColor = value; }
  }

  /// <summary>
  /// Gets a value indicating whether a key is available. This maps to
  /// the corresponding Console.KeyAvailable property.
  /// </summary>
  public override bool KeyAvailable
  {
    get { return Console.KeyAvailable; }
  }
```

```
    /// <summary>
    /// Gets the maximum physical size of the window adapted from the
    /// Console.LargestWindowWidth and Console.LargestWindowHeight
    /// properties.
    /// </summary>
    public override Size MaxPhysicalWindowSize
    {
      get { return new Size(Console.LargestWindowWidth,
Console.LargestWindowHeight); }
    }

    /// <summary>
    /// Gets the maximum window size adapted from the
    /// Console.LargestWindowWidth and console.LargestWindowHeight
    /// properties.
    /// </summary>
    public override Size MaxWindowSize
    {
      get { return new Size(Console.LargestWindowWidth,
Console.LargestWindowHeight); }
    }

    /// <summary>
    /// Gets or sets the window position adapted from the Console window position
    /// members.
    /// </summary>
    public override Coordinates WindowPosition
    {
      get { return new Coordinates(Console.WindowLeft, Console.WindowTop); }
      set { Console.SetWindowPosition(value.X, value.Y); }
    }

    /// <summary>
    /// Gets or sets the window size adapted from the corresponding Console
    /// calls.
    /// </summary>
    public override Size WindowSize
    {
      get { return new Size(Console.WindowWidth, Console.WindowHeight); }
      set { Console.SetWindowSize(value.Width, value.Height); }
    }

    /// <summary>
    /// Gets or sets the title of the window mapped to the Console.Title
    /// property.
    /// </summary>
    public override string WindowTitle
    {
      get { return Console.Title; }
      set { Console.Title = value; }
    }

    /// <summary>
    /// This API resets the input buffer. In this example this
    /// functionality is not needed so the method returns nothing.
```

```
    /// </summary>
    public override void FlushInputBuffer()
    {
    }

    /// <summary>
    /// This API returns a rectangular region of the screen buffer. In
    /// this example this functionality is not needed so the method throws
    /// a NotImplementException exception.
    /// </summary>
    /// <param name="rectangle">Defines the size of the rectangle.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override BufferCell[,] GetBufferContents(Rectangle rectangle)
    {
      throw new NotImplementedException(
               "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API Reads a pressed, released, or pressed and released keystroke
    /// from the keyboard device, blocking processing until a keystroke is
    /// typed that matches the specified keystroke options. In this example
    /// this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="options">Options, such as IncludeKeyDown,  used when
    /// reading the keyboard.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override KeyInfo ReadKey(ReadKeyOptions options)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API crops a region of the screen buffer. In this example
    /// this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="source">The region of the screen to be scrolled.</param>
    /// <param name="destination">The region of the screen to receive the
    /// source region contents.</param>
    /// <param name="clip">The region of the screen to include in the operation.
</param>
    /// <param name="fill">The character and attributes to be used to fill all
cell.</param>
    public override void ScrollBufferContents(Rectangle source, Coordinates
destination, Rectangle clip, BufferCell fill)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API copies an array of buffer cells into the screen buffer
```

```
    /// at a specified location. In this example this  functionality is
    /// not needed si the method  throws a NotImplementedException exception.
    /// </summary>
    /// <param name="origin">The parameter is not used.</param>
    /// <param name="contents">The parameter is not used.</param>
    public override void SetBufferContents(Coordinates origin, BufferCell[,]
contents)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API Copies a given character, foreground color, and background
    /// color to a region of the screen buffer. In this example this
    /// functionality is not needed so the method throws a
    /// NotImplementException exception./// </summary>
    /// <param name="rectangle">Defines the area to be filled. </param>
    /// <param name="fill">Defines the fill character.</param>
    public override void SetBufferContents(Rectangle rectangle, BufferCell fill)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }
  }
}
```

See Also

System.Management.Automation.Host.PSHost

System.Management.Automation.Host.PSHostUserInterface

System.Management.Automation.Host.PSHostRawUserInterface

Last updated on 05/20/2025

Host05 Sample

This sample shows how to build an interactive console-based host application that reads commands from the command line, executes the commands, and then displays the results to the console. This host application also supports calls to remote computers by using the Enter-PSSession and Exit-PSSession cmdlets.

Requirements

- This sample requires Windows PowerShell 2.0.
- This application must be run in elevated mode (Run as administrator).

Demonstrates

- Creating a custom host whose classes derive from the System.Management.Automation.Host.PSHost class, the System.Management.Automation.Host.PSHostUserInterface class, and the System.Management.Automation.Host.PSHostRawUserInterface class.
- Building a console application that uses these host classes to build an interactive Windows PowerShell shell.
- Creating a `$PROFILE` variable and loading the following profiles.
  - current user, current host
  - current user, all hosts
  - all users, current host
  - all users, all hosts
- Implement the System.Management.Automation.Host.IHostUISupportsMultipleChoiceSelection interface.
- Implement the System.Management.Automation.Host.IHostSupportsInteractiveSession interface to support interactive remoting by using the Enter-PSSession and Exit-PSSession cmdlets.

Example 1

This example allows the user to enter commands at a command line, processes those commands, and then prints out the results.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Collections.Generic;
  using System.Collections.ObjectModel;
  using System.Management.Automation;
  using System.Management.Automation.Host;
  using System.Management.Automation.Runspaces;
  using System.Text;
  using PowerShell = System.Management.Automation.PowerShell;

  /// <summary>
  /// Simple PowerShell interactive console host listener implementation. This
class
  /// implements a basic read-evaluate-print loop or 'listener' allowing you to
  /// interactively work with the PowerShell engine.
  /// </summary>
  internal class PSListenerConsoleSample
  {
    /// <summary>
    /// Holds a reference to the runspace for this interpreter.
    /// </summary>
    internal Runspace myRunSpace;

    /// <summary>
    /// Indicator to tell the host application that it should exit.
    /// </summary>
    private bool shouldExit;

    /// <summary>
    /// The exit code that the host application will use to exit.
    /// </summary>
    private int exitCode;

    /// <summary>
    /// Holds a reference to  the PSHost object for this interpreter.
    /// </summary>
    private MyHost myHost;

    /// <summary>
    /// Holds a reference to the currently executing pipeline so that
    /// it can be stopped by the control-C handler.
    /// </summary>
    private PowerShell currentPowerShell;

    /// <summary>
    /// Used to serialize access to instance data.
    /// </summary>
```

```
    private object instanceLock = new object();

    /// <summary>
    /// Gets or sets a value indicating whether the host application
    /// should exit.
    /// </summary>
    public bool ShouldExit
    {
      get { return this.shouldExit; }
      set { this.shouldExit = value; }
    }

    /// <summary>
    /// Gets or sets the exit code that the host application will use
    /// when exiting.
    /// </summary>
    public int ExitCode
    {
      get { return this.exitCode; }
      set { this.exitCode = value; }
    }

    /// <summary>
    /// Creates and initiates the listener.
    /// </summary>
    /// <param name="args">The parameter is not used.</param>
    private static void Main(string[] args)
    {
      // Display the welcome message.
      Console.Title = "Windows PowerShell Console Host Application Sample";
      ConsoleColor oldFg = Console.ForegroundColor;
      Console.ForegroundColor = ConsoleColor.Cyan;
      Console.WriteLine("    Windows PowerShell Console Host Interactive Sample");
      Console.WriteLine("    ================================================");
      Console.WriteLine(string.Empty);
      Console.WriteLine("This is an example of a simple interactive console host
that uses ");
      Console.WriteLine("the Windows PowerShell engine to interpret commands.");
      Console.WriteLine("Type 'exit' to exit.");
      Console.WriteLine(string.Empty);
      Console.ForegroundColor = oldFg;

      // Create the listener and runs it. This method never returns.
      PSListenerConsoleSample listener = new PSListenerConsoleSample();
      listener.Run();
    }

    /// <summary>
    /// Create an instance of the console listener.
    /// </summary>
    private PSListenerConsoleSample()
    {
      // Create the host and runspace instances for this interpreter. Note
      // that this application doesn't support console files so only the
      // default snap-ins will be available.
```

```
      this.myHost = new MyHost(this);
      this.myRunSpace = RunspaceFactory.CreateRunspace(this.myHost);
      this.myRunSpace.Open();

      // Create a PowerShell object to run the commands used to create
      // $PROFILE and load the profiles.
      lock (this.instanceLock)
      {
        this.currentPowerShell = PowerShell.Create();
      }

      try
      {
        this.currentPowerShell.Runspace = this.myRunSpace;

        PSCommand[] profileCommands =
Microsoft.Samples.PowerShell.Host.HostUtilities.GetProfileCommands("SampleHost05");
        foreach (PSCommand command in profileCommands)
        {
          this.currentPowerShell.Commands = command;
          this.currentPowerShell.Invoke();
        }
      }
      finally
      {
        // Dispose of the pipeline line and set it to null, locked because
currentPowerShell
        // may be accessed by the ctrl-C handler...
        lock (this.instanceLock)
        {
          this.currentPowerShell.Dispose();
          this.currentPowerShell = null;
        }
      }
    }

    /// <summary>
    /// A helper class that builds and executes a pipeline that writes to the
    /// default output path. Any exceptions that are thrown are just passed to
    /// the caller. Since all output goes to the default
    /// outputter, this method does not return anything.
    /// </summary>
    /// <param name="cmd">The script to run.</param>
    /// <param name="input">Any input arguments to pass to the script.
    /// If null then nothing is passed in.</param>
    private void executeHelper(string cmd, object input)
    {
      // Ignore empty command lines.
      if (String.IsNullOrEmpty(cmd))
      {
        return;
      }

      // Create the pipeline object and make it available to the
      // ctrl-C handle through the currentPowerShell instance
```

```
      // variable.
      lock (this.instanceLock)
      {
        this.currentPowerShell = PowerShell.Create();
      }

      // Create a pipeline for this execution, and then place the
      // result in the currentPowerShell variable so it is available
      // to be stopped.
      try
      {
        this.currentPowerShell.Runspace = this.myRunSpace;
        this.currentPowerShell.AddScript(cmd);

        // Add the default outputter to the end of the pipe and then
        // call the MergeMyResults method to merge the output and
        // error streams from the pipeline. This will result in the
        // output being written using the PSHost and PSHostUserInterface
        // classes instead of returning objects to the host application.
        this.currentPowerShell.AddCommand("Out-Default");

this.currentPowerShell.Commands.Commands[0].MergeMyResults(PipelineResultTypes.Erro
r, PipelineResultTypes.Output);

        // If there is any input pass it in, otherwise just invoke the
        // the pipeline.
        if (input != null)
        {
          this.currentPowerShell.Invoke(new object[] { input });
        }
        else
        {
          this.currentPowerShell.Invoke();
        }
      }
      finally
      {
        // Dispose the PowerShell object and set currentPowerShell to null.
        // It is locked because currentPowerShell may be accessed by the
        // ctrl-C handler.
        lock (this.instanceLock)
        {
          this.currentPowerShell.Dispose();
          this.currentPowerShell = null;
        }
      }
    }

    /// <summary>
    /// To display an exception using the display formatter,
    /// run a second pipeline passing in the error record.
    /// The runtime will bind this to the $input variable,
    /// which is why $input is being piped to the Out-String
    /// cmdlet. The WriteErrorLine method is called to make sure
    /// the error gets displayed in the correct error color.
```

```
    /// </summary>
    /// <param name="e">The exception to display.</param>
    private void ReportException(Exception e)
    {
      if (e != null)
      {
        object error;
        IContainsErrorRecord icer = e as IContainsErrorRecord;
        if (icer != null)
        {
          error = icer.ErrorRecord;
        }
        else
        {
          error = (object)new ErrorRecord(e, "Host.ReportException",
ErrorCategory.NotSpecified, null);
        }

        lock (this.instanceLock)
        {
          this.currentPowerShell = PowerShell.Create();
        }

        this.currentPowerShell.Runspace = this.myRunSpace;

        try
        {
          this.currentPowerShell.AddScript("$input").AddCommand("Out-String");

          // Do not merge errors, this function will swallow errors.
          Collection<PSObject> result;
          PSDataCollection<object> inputCollection = new PSDataCollection<object>
();
          inputCollection.Add(error);
          inputCollection.Complete();
          result = this.currentPowerShell.Invoke(inputCollection);

          if (result.Count > 0)
          {
            string str = result[0].BaseObject as string;
            if (!string.IsNullOrEmpty(str))
            {
              // Remove \r\n, which is added by the Out-String cmdlet.
              this.myHost.UI.WriteErrorLine(str.Substring(0, str.Length - 2));
            }
          }
        }
        finally
        {
          // Dispose of the pipeline and set it to null, locking it  because
          // currentPowerShell may be accessed by the ctrl-C handler.
          lock (this.instanceLock)
          {
            this.currentPowerShell.Dispose();
            this.currentPowerShell = null;
```

```
        }
      }
    }
  }

  /// <summary>
  /// Basic script execution routine. Any runtime exceptions are
  /// caught and passed back to the Windows PowerShell engine to
  /// display.
  /// </summary>
  /// <param name="cmd">Script to run.</param>
  private void Execute(string cmd)
  {
    try
    {
      // Execute the command with no input.
      this.executeHelper(cmd, null);
    }
    catch (RuntimeException rte)
    {
      this.ReportException(rte);
    }
  }

  /// <summary>
  /// Method used to handle control-C's from the user. It calls the
  /// pipeline Stop() method to stop execution. If any exceptions occur
  /// they are printed to the console but otherwise ignored.
  /// </summary>
  /// <param name="sender">See sender property documentation of
  /// ConsoleCancelEventHandler.</param>
  /// <param name="e">See e property documentation of
  /// ConsoleCancelEventHandler.</param>
  private void HandleControlC(object sender, ConsoleCancelEventArgs e)
  {
    try
    {
      lock (this.instanceLock)
      {
        if (this.currentPowerShell != null &&
this.currentPowerShell.InvocationStateInfo.State == PSInvocationState.Running)
        {
          this.currentPowerShell.Stop();
        }
      }

      e.Cancel = true;
    }
    catch (Exception exception)
    {
      this.myHost.UI.WriteErrorLine(exception.ToString());
    }
  }

  /// <summary>
```

```
    /// Implements the basic listener loop. It sets up the ctrl-C handler, then
    /// reads a command from the user, executes it and repeats until the ShouldExit
    /// flag is set.
    /// </summary>
    private void Run()
    {
      // Set up the control-C handler.
      Console.CancelKeyPress += new ConsoleCancelEventHandler(this.HandleControlC);
      Console.TreatControlCAsInput = false;

      // Read commands to execute until ShouldExit is set by
      // the user calling "exit".
      while (!this.ShouldExit)
      {
        string prompt;
        if (this.myHost.IsRunspacePushed)
        {
          prompt = string.Format("\n[{0}] PSConsoleSample: ",
this.myRunSpace.ConnectionInfo.ComputerName);
        }
        else
        {
          prompt = "\nPSConsoleSample: ";
        }

        this.myHost.UI.Write(ConsoleColor.Cyan, ConsoleColor.Black, prompt);
        string cmd = Console.ReadLine();
        this.Execute(cmd);
      }

      // Exit with the desired exit code that was set by exit command.
      // This is set in the host by the MyHost.SetShouldExit() implementation.
      Environment.Exit(this.ExitCode);
    }
  }
}
```

Example 2

The following code is the implementation of the System.Management.Automation.Host.PSHost class that is used by this host application. Those elements that are not implemented throw an exception or return nothing.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Globalization;
  using System.Management.Automation.Host;
  using System.Management.Automation.Runspaces;
```

```
/// <summary>
/// This is a sample implementation of the PSHost abstract class for
/// console applications. Not all members are implemented. Those that
/// are not implemented throw a NotImplementedException exception or
/// return nothing.
/// </summary>
internal class MyHost : PSHost, IHostSupportsInteractiveSession
{

  /// <summary>
  /// A reference to the PSHost implementation.
  /// </summary>
  private PSListenerConsoleSample program;

  /// <summary>
  /// The culture information of the thread that created
  /// this object.
  /// </summary>
  private CultureInfo originalCultureInfo =
      System.Threading.Thread.CurrentThread.CurrentCulture;

  /// <summary>
  /// The UI culture information of the thread that created
  /// this object.
  /// </summary>
  private CultureInfo originalUICultureInfo =
      System.Threading.Thread.CurrentThread.CurrentUICulture;

  /// <summary>
  /// The identifier of this PSHost implementation.
  /// </summary>
  private static Guid instanceId = Guid.NewGuid();

  /// <summary>
  /// Initializes a new instance of the MyHost class. Keep
  /// a reference to the host application object so that it
  /// can be informed of when to exit.
  /// </summary>
  /// <param name="program">
  /// A reference to the host application object.
  /// </param>
  public MyHost(PSListenerConsoleSample program)
  {
    this.program = program;
  }

  /// <summary>
  /// A reference to the implementation of the PSHostUserInterface
  /// class for this application.
  /// </summary>
  private MyHostUserInterface myHostUserInterface = new MyHostUserInterface();

  /// <summary>
  /// A reference to the runspace used to start an interactive session.
```

```
    /// </summary>
    public Runspace pushedRunspace = null;

    /// <summary>
    /// Gets the culture information to use. This implementation
    /// returns a snapshot of the culture information of the thread
    /// that created this object.
    /// </summary>
    public override CultureInfo CurrentCulture
    {
      get { return this.originalCultureInfo; }
    }

    /// <summary>
    /// Gets the UI culture information to use. This implementation
    /// returns a snapshot of the UI culture information of the thread
    /// that created this object.
    /// </summary>
    public override CultureInfo CurrentUICulture
    {
      get { return this.originalUICultureInfo; }
    }

    /// <summary>
    /// Gets an identifier for this host. This implementation always
    /// returns the GUID allocated at instantiation time.
    /// </summary>
    public override Guid InstanceId
    {
      get { return instanceId; }
    }

    /// <summary>
    /// Gets a string that contains the name of this host implementation.
    /// Keep in mind that this string may be used by script writers to
    /// identify when your host is being used.
    /// </summary>
    public override string Name
    {
      get { return "MySampleConsoleHostImplementation"; }
    }

    /// <summary>
    /// Gets an instance of the implementation of the PSHostUserInterface
    /// class for this application. This instance is allocated once at startup time
    /// and returned every time thereafter.
    /// </summary>
    public override PSHostUserInterface UI
    {
      get { return this.myHostUserInterface; }
    }

    /// <summary>
    /// Gets the version object for this application. Typically this
    /// should match the version resource in the application.
```

```
    /// </summary>
    public override Version Version
    {
      get { return new Version(1, 0, 0, 0); }
    }

    #region IHostSupportsInteractiveSession Properties

    /// <summary>
    /// Gets a value indicating whether a request
    /// to open a PSSession has been made.
    /// </summary>
    public bool IsRunspacePushed
    {
      get { return this.pushedRunspace != null; }
    }

    /// <summary>
    /// Gets or sets the runspace used by the PSSession.
    /// </summary>
    public Runspace Runspace
    {
      get { return this.program.myRunSpace; }
      internal set { this.program.myRunSpace = value; }
    }
    #endregion IHostSupportsInteractiveSession Properties

    /// <summary>
    /// This API Instructs the host to interrupt the currently running
    /// pipeline and start a new nested input loop. In this example this
    /// functionality is not needed so the method throws a
    /// NotImplementedException exception.
    /// </summary>
    public override void EnterNestedPrompt()
    {
      throw new NotImplementedException(
            "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API instructs the host to exit the currently running input loop.
    /// In this example this functionality is not needed so the method
    /// throws a NotImplementedException exception.
    /// </summary>
    public override void ExitNestedPrompt()
    {
      throw new NotImplementedException(
            "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API is called before an external application process is
    /// started. Typically it is used to save state so that the parent
    /// can restore state that has been modified by a child process (after
    /// the child exits). In this example this functionality is not
```

```
    /// needed so the method returns nothing.
    /// </summary>
    public override void NotifyBeginApplication()
    {
      return;
    }

    /// <summary>
    /// This API is called after an external application process finishes.
    /// Typically it is used to restore state that a child process has
    /// altered. In this example, this functionality is not needed so
    /// the method returns nothing.
    /// </summary>
    public override void NotifyEndApplication()
    {
      return;
    }

    /// <summary>
    /// Indicate to the host application that exit has
    /// been requested. Pass the exit code that the host
    /// application should use when exiting the process.
    /// </summary>
    /// <param name="exitCode">The exit code that the
    /// host application should use.</param>
    public override void SetShouldExit(int exitCode)
    {
      this.program.ShouldExit = true;
      this.program.ExitCode = exitCode;
    }

    #region IHostSupportsInteractiveSession Methods

    /// <summary>
    /// Requests to close a PSSession.
    /// </summary>
    public void PopRunspace()
    {
      Runspace = this.pushedRunspace;
      this.pushedRunspace = null;
    }

    /// <summary>
    /// Requests to open a PSSession.
    /// </summary>
    /// <param name="runspace">Runspace to use.</param>
    public void PushRunspace(Runspace runspace)
    {
      this.pushedRunspace = Runspace;
      Runspace = runspace;
    }

    #endregion IHostSupportsInteractiveSession Methods
  }
}
```

Example 3

The following code is the implementation of the System.Management.Automation.Host.PSHostUserInterface class that is used by this host application.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Collections.Generic;
  using System.Collections.ObjectModel;
  using System.Globalization;
  using System.Management.Automation;
  using System.Management.Automation.Host;
  using System.Text;

  /// <summary>
  /// A sample implementation of the PSHostUserInterface abstract class for
  /// console applications. Not all members are implemented. Those that are
  /// not implemented throw a NotImplementedException exception or return
  /// nothing. Members that are implemented include those that map easily to
  /// Console APIs and a basic implementation of the prompt API provided.
  /// </summary>
  internal class MyHostUserInterface : PSHostUserInterface,
IHostUISupportsMultipleChoiceSelection
  {
    /// <summary>
    /// A reference to the PSRawUserInterface implementation.
    /// </summary>
    private MyRawUserInterface myRawUi = new MyRawUserInterface();

    /// <summary>
    /// Gets an instance of the PSRawUserInterface object for this host
    /// application.
    /// </summary>
    public override PSHostRawUserInterface RawUI
    {
      get { return this.myRawUi; }
    }

    /// <summary>
    /// Prompts the user for input.
    /// <param name="caption">The caption or title of the prompt.</param>
    /// <param name="message">The text of the prompt.</param>
    /// <param name="descriptions">A collection of FieldDescription objects
    /// that describe each field of the prompt.</param>
    /// <returns>A dictionary object that contains the results of the user
    /// prompts.</returns>
```

```
    public override Dictionary<string, PSObject> Prompt(
                            string caption,
                            string message,
                            Collection<FieldDescription> descriptions)
    {
      this.Write(
                ConsoleColor.Blue,
                ConsoleColor.Black,
                caption + "\n" + message + " ");
      Dictionary<string, PSObject> results =
                new Dictionary<string, PSObject>();
      foreach (FieldDescription fd in descriptions)
      {
        string[] label = GetHotkeyAndLabel(fd.Label);
        this.WriteLine(label[1]);
        string userData = Console.ReadLine();
        if (userData == null)
        {
          return null;
        }

        results[fd.Name] = PSObject.AsPSObject(userData);
      }

      return results;
    }

    /// <summary>

    /// Provides a set of choices that enable the user to choose a
    /// single option from a set of options.
    /// </summary>
    /// <param name="caption">Text that proceeds (a title) the choices.</param>
    /// <param name="message">A message that describes the choice.</param>
    /// <param name="choices">A collection of ChoiceDescription objects that
    /// describe each choice.</param>
    /// <param name="defaultChoice">The index of the label in the Choices
    /// parameter collection. To indicate no default choice, set to -1.</param>
    /// <returns>The index of the Choices parameter collection element that
    /// corresponds to the option that is selected by the user.</returns>
    public override int PromptForChoice(
                                        string caption,
                                        string message,
                                        Collection<ChoiceDescription> choices,
                                        int defaultChoice)
    {
      // Write the caption and message strings in Blue.
      this.WriteLine(
                    ConsoleColor.Blue,
                    ConsoleColor.Black,
                    caption + "\n" + message + "\n");

      // Convert the choice collection into something that is
      // easier to work with. See the BuildHotkeysAndPlainLabels
      // method for details.
```

```
      string[,] promptData = BuildHotkeysAndPlainLabels(choices);

      // Format the overall choice prompt string to display.
      StringBuilder sb = new StringBuilder();
      for (int element = 0; element < choices.Count; element++)
      {
        sb.Append(String.Format(
                                CultureInfo.CurrentCulture,
                                "|{0}> {1} ",
                                promptData[0, element],
                                promptData[1, element]));
      }

      sb.Append(String.Format(
                              CultureInfo.CurrentCulture,
                              "[Default is ({0})]",
                              promptData[0, defaultChoice]));

      // Read prompts until a match is made, the default is
      // chosen, or the loop is interrupted with ctrl-C.
      while (true)
      {
        this.WriteLine(ConsoleColor.Cyan, ConsoleColor.Black, sb.ToString());
        string data =
Console.ReadLine().Trim().ToUpper(CultureInfo.CurrentCulture);

        // If the choice string was empty, use the default selection.
        if (data.Length == 0)
        {
          return defaultChoice;
        }

        // See if the selection matched and return the
        // corresponding index if it did.
        for (int i = 0; i < choices.Count; i++)
        {
          if (promptData[0, i] == data)
          {
            return i;
          }
        }

        this.WriteErrorLine("Invalid choice: " + data);
      }
    }

    #region IHostUISupportsMultipleChoiceSelection Members

    /// <summary>
    /// Provides a set of choices that enable the user to choose a one or
    /// more options from a set of options.
    /// </summary>
    /// <param name="caption">Text that proceeds (a title) the choices.</param>
    /// <param name="message">A message that describes the choice.</param>
    /// <param name="choices">A collection of ChoiceDescription objects that
```

```
    /// describe each choice.</param>
    /// <param name="defaultChoices">The index of the label in the Choices
    /// parameter collection. To indicate no default choice, set to -1.</param>
    /// <returns>The index of the Choices parameter collection element that
    /// corresponds to the option that is selected by the user.</returns>
    public Collection<int> PromptForChoice(
                                            string caption,
                                            string message,
                                            Collection<ChoiceDescription> choices,
                                            IEnumerable<int> defaultChoices)
    {
      // Write the caption and message strings in Blue.
      this.WriteLine(
                    ConsoleColor.Blue,
                    ConsoleColor.Black,
                    caption + "\n" + message + "\n");

      // Convert the choice collection into something that is
      // easier to work with. See the BuildHotkeysAndPlainLabels
      // method for details.
      string[,] promptData = BuildHotkeysAndPlainLabels(choices);

      // Format the overall choice prompt string to display.
      StringBuilder sb = new StringBuilder();
      for (int element = 0; element < choices.Count; element++)
      {
        sb.Append(String.Format(
                                CultureInfo.CurrentCulture,
                                "|{0}> {1} ",
                                promptData[0, element],
                                promptData[1, element]));
      }

      Collection<int> defaultResults = new Collection<int>();
      if (defaultChoices != null)
      {
        int countDefaults = 0;
        foreach (int defaultChoice in defaultChoices)
        {
          ++countDefaults;
          defaultResults.Add(defaultChoice);
        }

        if (countDefaults != 0)
        {
          sb.Append(countDefaults == 1 ? "[Default choice is " : "[Default choices
are ");
          foreach (int defaultChoice in defaultChoices)
          {
            sb.AppendFormat(
                            CultureInfo.CurrentCulture,
                            "\"{0}\",",
                            promptData[0, defaultChoice]);
          }
```

```
            sb.Remove(sb.Length - 1, 1);
            sb.Append("]");
          }
        }

        this.WriteLine(
                        ConsoleColor.Cyan,
                        ConsoleColor.Black,
                        sb.ToString());

        // Read prompts until a match is made, the default is
        // chosen, or the loop is interrupted with ctrl-C.
        Collection<int> results = new Collection<int>();
        while (true)
        {
          ReadNext:
          string prompt = string.Format(CultureInfo.CurrentCulture, "Choice[{0}]:",
results.Count);
          this.Write(ConsoleColor.Cyan, ConsoleColor.Black, prompt);
          string data =
Console.ReadLine().Trim().ToUpper(CultureInfo.CurrentCulture);

          // If the choice string was empty, no more choices have been made.
          // If there were no choices made, return the defaults
          if (data.Length == 0)
          {
            return (results.Count == 0) ? defaultResults : results;
          }

          // See if the selection matched and return the
          // corresponding index if it did.
          for (int i = 0; i < choices.Count; i++)
          {
            if (promptData[0, i] == data)
            {
              results.Add(i);
              goto ReadNext;
            }
          }

          this.WriteErrorLine("Invalid choice: " + data);
        }
      }

      #endregion

      /// <summary>
      /// Prompts the user for credentials with a specified prompt window
      /// caption, prompt message, user name, and target name. In this
      /// example this functionality is not needed so the method throws a
      /// NotImplementException exception.
      /// </summary>
      /// <param name="caption">The caption for the message window.</param>
      /// <param name="message">The text of the message.</param>
      /// <param name="userName">The user name whose credential is to be
```

```
    /// prompted for.</param>
    /// <param name="targetName">The name of the target for which the
    /// credential is collected.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override PSCredential PromptForCredential(
                                                    string caption,
                                                    string message,
                                                    string userName,
                                                    string targetName)
    {
      throw new NotImplementedException(
                          "The method or operation is not implemented.");
    }

    /// <summary>
    /// Prompts the user for credentials by using a specified prompt window
    /// caption, prompt message, user name and target name, credential
    /// types allowed to be returned, and UI behavior options. In this
    /// example this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="caption">The caption for the message window.</param>
    /// <param name="message">The text of the message.</param>
    /// <param name="userName">The user name whose credential is to be
    /// prompted for.</param>
    /// <param name="targetName">The name of the target for which the
    /// credential is collected.</param>
    /// <param name="allowedCredentialTypes">A PSCredentialTypes constant
    /// that identifies the type of credentials that can be returned.</param>
    /// <param name="options">A PSCredentialUIOptions constant that
    /// identifies the UI behavior when it gathers the credentials.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override PSCredential PromptForCredential(
                                       string caption,
                                       string message,
                                       string userName,
                                       string targetName,
                                       PSCredentialTypes allowedCredentialTypes,
                                       PSCredentialUIOptions options)
    {
      throw new NotImplementedException(
                              "The method or operation is not implemented.");
    }

    /// <summary>
    /// Reads characters that are entered by the user until a newline
    /// (carriage return) is encountered.
    /// </summary>
    /// <returns>The characters that are entered by the user.</returns>
    public override string ReadLine()
    {
      return Console.ReadLine();
    }

    /// <summary>
```

```
    /// Reads characters entered by the user until a newline (carriage return)
    /// is encountered and returns the characters as a secure string. In this
    /// example this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <returns>Throws a NotImplemented exception.</returns>
    public override System.Security.SecureString ReadLineAsSecureString()
    {
      throw new NotImplementedException("The method or operation is not
implemented.");
    }

    /// <summary>
    /// Writes characters to the output display of the host.
    /// </summary>
    /// <param name="value">The characters to be written.</param>
    public override void Write(string value)
    {
      Console.Write(value);
    }

    /// <summary>
    /// Writes characters to the output display of the host with possible
    /// foreground and background colors.
    /// </summary>
    /// <param name="foregroundColor">The color of the characters.</param>
    /// <param name="backgroundColor">The background color to use.</param>
    /// <param name="value">The characters to be written.</param>
    public override void Write(
                               ConsoleColor foregroundColor,
                               ConsoleColor backgroundColor,
                               string value)
    {
      ConsoleColor oldFg = Console.ForegroundColor;
      ConsoleColor oldBg = Console.BackgroundColor;
      Console.ForegroundColor = foregroundColor;
      Console.BackgroundColor = backgroundColor;
      Console.Write(value);
      Console.ForegroundColor = oldFg;
      Console.BackgroundColor = oldBg;
    }

    /// <summary>
    /// Writes a line of characters to the output display of the host
    /// with foreground and background colors and appends a newline (carriage
return).
    /// </summary>
    /// <param name="foregroundColor">The foreground color of the display. </param>
    /// <param name="backgroundColor">The background color of the display. </param>
    /// <param name="value">The line to be written.</param>
    public override void WriteLine(
                                   ConsoleColor foregroundColor,
                                   ConsoleColor backgroundColor,
                                   string value)
    {
```

```
      ConsoleColor oldFg = Console.ForegroundColor;
      ConsoleColor oldBg = Console.BackgroundColor;
      Console.ForegroundColor = foregroundColor;
      Console.BackgroundColor = backgroundColor;
      Console.WriteLine(value);
      Console.ForegroundColor = oldFg;
      Console.BackgroundColor = oldBg;
    }

    /// <summary>
    /// Writes a debug message to the output display of the host.
    /// </summary>
    /// <param name="message">The debug message that is displayed.</param>
    public override void WriteDebugLine(string message)
    {
      this.WriteLine(
                    ConsoleColor.DarkYellow,
                    ConsoleColor.Black,
                    String.Format(CultureInfo.CurrentCulture, "DEBUG: {0}",
message));
    }

    /// <summary>
    /// Writes an error message to the output display of the host.
    /// </summary>
    /// <param name="value">The error message that is displayed.</param>
    public override void WriteErrorLine(string value)
    {
      this.WriteLine(
                    ConsoleColor.Red,
                    ConsoleColor.Black,
                    value);
    }

    /// <summary>
    /// Writes a newline character (carriage return)
    /// to the output display of the host.
    /// </summary>
    public override void WriteLine()
    {
      Console.WriteLine();
    }

       /// <summary>
    /// Writes a line of characters to the output display of the host
    /// and appends a newline character(carriage return).
    /// </summary>
    /// <param name="value">The line to be written.</param>
    public override void WriteLine(string value)
    {
      Console.WriteLine(value);
    }

    /// <summary>
    /// Writes a progress report to the output display of the host.
```

```
    /// </summary>
    /// <param name="sourceId">Unique identifier of the source of the record.
</param>
    /// <param name="record">A ProgressReport object.</param>
    public override void WriteProgress(long sourceId, ProgressRecord record)
    {

    }

    /// <summary>
    /// Writes a verbose message to the output display of the host.
    /// </summary>
    /// <param name="message">The verbose message that is displayed.</param>
    public override void WriteVerboseLine(string message)
    {
      this.WriteLine(
                    ConsoleColor.Green,
                    ConsoleColor.Black,
                    String.Format(CultureInfo.CurrentCulture, "VERBOSE: {0}",
message));
    }

    /// <summary>
    /// Writes a warning message to the output display of the host.
    /// </summary>
    /// <param name="message">The warning message that is displayed.</param>
    public override void WriteWarningLine(string message)
    {
      this.WriteLine(
                    ConsoleColor.Yellow,
                    ConsoleColor.Black,
                    String.Format(CultureInfo.CurrentCulture, "WARNING: {0}",
message));
    }

    /// <summary>
    /// Parse a string containing a hotkey character.
    /// Take a string of the form
    ///    Yes to &all
    /// and returns a two-dimensional array split out as
    ///    "A", "Yes to all".
    /// </summary>
    /// <param name="input">The string to process</param>
    /// <returns>
    /// A two dimensional array containing the parsed components.
    /// </returns>
    private static string[] GetHotkeyAndLabel(string input)
    {
      string[] result = new string[] { String.Empty, String.Empty };
      string[] fragments = input.Split('&');
      if (fragments.Length == 2)
      {
        if (fragments[1].Length > 0)
        {
          result[0] = fragments[1][0].ToString().
```

```
          ToUpper(CultureInfo.CurrentCulture);
        }

        result[1] = (fragments[0] + fragments[1]).Trim();
      }
      else
      {
        result[1] = input;
      }

      return result;
    }

    /// <summary>
    /// This is a private worker function splits out the
    /// accelerator keys from the menu and builds a two
    /// dimensional array with the first access containing the
    /// accelerator and the second containing the label string
    /// with the & removed.
    /// </summary>
    /// <param name="choices">The choice collection to process</param>
    /// <returns>
    /// A two dimensional array containing the accelerator characters
    /// and the cleaned-up labels</returns>
    private static string[,] BuildHotkeysAndPlainLabels(
         Collection<ChoiceDescription> choices)
    {
      // Allocate the result array
      string[,] hotkeysAndPlainLabels = new string[2, choices.Count];

      for (int i = 0; i < choices.Count; ++i)
      {
        string[] hotkeyAndLabel = GetHotkeyAndLabel(choices[i].Label);
        hotkeysAndPlainLabels[0, i] = hotkeyAndLabel[0];
        hotkeysAndPlainLabels[1, i] = hotkeyAndLabel[1];
      }

      return hotkeysAndPlainLabels;
    }
  }
}
```

Example 4

The following code is the implementation of the System.Management.Automation.Host.PSHostRawUserInterface class that is used by this host application. Those elements that are not implemented throw an exception or return nothing.

```
C#
```

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Management.Automation.Host;

  /// <summary>
  /// A sample implementation of the PSHostRawUserInterface for console
  /// applications. Members of this class that easily map to the .NET
  /// console class are implemented. More complex methods are not
  /// implemented and throw a NotImplementedException exception.
  /// </summary>
  internal class MyRawUserInterface : PSHostRawUserInterface
  {
    /// <summary>
    /// Gets or sets the background color of text to be written.
    /// This maps to the corresponding Console.Background property.
    /// </summary>
    public override ConsoleColor BackgroundColor
    {
      get { return Console.BackgroundColor; }
      set { Console.BackgroundColor = value; }
    }

    /// <summary>
    /// Gets or sets the host buffer size adapted from the Console buffer
    /// size members.
    /// </summary>
    public override Size BufferSize
    {
      get { return new Size(Console.BufferWidth, Console.BufferHeight); }
      set { Console.SetBufferSize(value.Width, value.Height); }
    }

    /// <summary>
    /// Gets or sets the cursor position. In this example this
    /// functionality is not needed so the property throws a
    /// NotImplementException exception.
    /// </summary>
    public override Coordinates CursorPosition
    {
      get { throw new NotImplementedException(
                  "The method or operation is not implemented."); }
      set { throw new NotImplementedException(
                  "The method or operation is not implemented."); }
    }

    /// <summary>
    /// Gets or sets the cursor size taken directly from the
    /// Console.CursorSize property.
    /// </summary>
    public override int CursorSize
    {
      get { return Console.CursorSize; }
      set { Console.CursorSize = value; }
```

```
    }

    /// <summary>
    /// Gets or sets the foreground color of the text to be written.
    /// This maps to the corresponding Console.ForegroundColor property.
    /// </summary>
    public override ConsoleColor ForegroundColor
    {
      get { return Console.ForegroundColor; }
      set { Console.ForegroundColor = value; }
    }

    /// <summary>
    /// Gets a value indicating whether a key is available. This maps to
    /// the corresponding Console.KeyAvailable property.
    /// </summary>
    public override bool KeyAvailable
    {
      get { return Console.KeyAvailable; }
    }

    /// <summary>
    /// Gets the maximum physical size of the window adapted from the
    /// Console.LargestWindowWidth and Console.LargestWindowHeight
    /// properties.
    /// </summary>
    public override Size MaxPhysicalWindowSize
    {
      get { return new Size(Console.LargestWindowWidth,
Console.LargestWindowHeight); }
    }

    /// <summary>
    /// Gets the maximum window size adapted from the
    /// Console.LargestWindowWidth and console.LargestWindowHeight
    /// properties.
    /// </summary>
    public override Size MaxWindowSize
    {
      get { return new Size(Console.LargestWindowWidth,
Console.LargestWindowHeight); }
    }

    /// <summary>
    /// Gets or sets the window position adapted from the Console window position
    /// members.
    /// </summary>
    public override Coordinates WindowPosition
    {
      get { return new Coordinates(Console.WindowLeft, Console.WindowTop); }
      set { Console.SetWindowPosition(value.X, value.Y); }
    }

    /// <summary>
    /// Gets or sets the window size adapted from the corresponding Console
```

```
    /// calls.
    /// </summary>
    public override Size WindowSize
    {
      get { return new Size(Console.WindowWidth, Console.WindowHeight); }
      set { Console.SetWindowSize(value.Width, value.Height); }
    }

    /// <summary>
    /// Gets or sets the title of the window mapped to the Console.Title
    /// property.
    /// </summary>
    public override string WindowTitle
    {
      get { return Console.Title; }
      set { Console.Title = value; }
    }

    /// <summary>
    /// This API resets the input buffer. In this example this
    /// functionality is not needed so the method returns nothing.
    /// </summary>
    public override void FlushInputBuffer()
    {
    }

    /// <summary>
    /// This API returns a rectangular region of the screen buffer. In
    /// this example this functionality is not needed so the method throws
    /// a NotImplementException exception.
    /// </summary>
    /// <param name="rectangle">Defines the size of the rectangle.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override BufferCell[,] GetBufferContents(Rectangle rectangle)
    {
      throw new NotImplementedException(
               "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API Reads a pressed, released, or pressed and released keystroke
    /// from the keyboard device, blocking processing until a keystroke is
    /// typed that matches the specified keystroke options. In this example
    /// this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="options">Options, such as IncludeKeyDown,  used when
    /// reading the keyboard.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override KeyInfo ReadKey(ReadKeyOptions options)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }
```

```
    /// <summary>
    /// This API crops a region of the screen buffer. In this example
    /// this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="source">The region of the screen to be scrolled.</param>
    /// <param name="destination">The region of the screen to receive the
    /// source region contents.</param>
    /// <param name="clip">The region of the screen to include in the operation.
</param>
    /// <param name="fill">The character and attributes to be used to fill all
cell.</param>
    public override void ScrollBufferContents(Rectangle source, Coordinates
destination, Rectangle clip, BufferCell fill)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API copies an array of buffer cells into the screen buffer
    /// at a specified location. In this example this  functionality is
    /// not needed si the method  throws a NotImplementedException exception.
    /// </summary>
    /// <param name="origin">The parameter is not used.</param>
    /// <param name="contents">The parameter is not used.</param>
    public override void SetBufferContents(Coordinates origin, BufferCell[,]
contents)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API Copies a given character, foreground color, and background
    /// color to a region of the screen buffer. In this example this
    /// functionality is not needed so the method throws a
    /// NotImplementException exception./// </summary>
    /// <param name="rectangle">Defines the area to be filled. </param>
    /// <param name="fill">Defines the fill character.</param>
    public override void SetBufferContents(Rectangle rectangle, BufferCell fill)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }
  }
}
```

See Also

System.Management.Automation.Host.PSHost

System.Management.Automation.Host.PSHostUserInterface

System.Management.Automation.Host.PSHostRawUserInterface

Last updated on 05/20/2025

Host06 Sample

This sample shows how to build an interactive console-based host application that reads commands from the command line, executes the commands, and then displays the results to the console. In addition, this sample uses the Tokenizer APIs to specify the color of the text that is entered by the user.

Requirements

- This sample requires Windows PowerShell 2.0.
- This application must be run in elevated mode (Run as administrator).

Demonstrates

- Creating a custom host whose classes derive from the System.Management.Automation.Host.PSHost class, the System.Management.Automation.Host.PSHostUserInterface class, and the System.Management.Automation.Host.PSHostRawUserInterface class.
- Building a console application that uses these host classes to build an interactive Windows PowerShell shell.
- Creating a `$PROFILE` variable and loading the following profiles.
  - current user, current host
  - current user, all hosts
  - all users, current host
  - all users, all hosts
- Implement the System.Management.Automation.Host.IHostUISupportsMultipleChoiceSelection interface.
- Implement the System.Management.Automation.Host.IHostSupportsInteractiveSession interface to support interactive remoting by using the Enter-PSSession and Exit-PSSession cmdlets.
- Use the Tokenize API to colorize the command line as it is typed.

Example 1

This example allows the user to enter commands at a command line, processes those commands, and then prints out the results.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Collections.Generic;
  using System.Collections.ObjectModel;
  using System.Management.Automation;
  using System.Management.Automation.Host;
  using System.Management.Automation.Runspaces;
  using System.Text;
  using PowerShell = System.Management.Automation.PowerShell;

  /// <summary>
  /// This sample shows how to implement a basic read-evaluate-print
  /// loop (or 'listener') that allowing you to interactively work
  /// with the Windows PowerShell engine.
  /// </summary>
  internal class PSListenerConsoleSample
  {
    /// <summary>
    /// Used to read user input.
    /// </summary>
        internal ConsoleReadLine consoleReadLine = new ConsoleReadLine();

    /// <summary>
    /// Holds a reference to the runspace for this interpreter.
    /// </summary>
    internal Runspace myRunSpace;

    /// <summary>
    /// Indicator to tell the host application that it should exit.
    /// </summary>
    private bool shouldExit;

    /// <summary>
    /// The exit code that the host application will use to exit.
    /// </summary>
    private int exitCode;

    /// <summary>
    /// Holds a reference to the PSHost implementation for this interpreter.
    /// </summary>
    private MyHost myHost;

    /// <summary>
    /// Holds a reference to the currently executing pipeline so that it can be
    /// stopped by the control-C handler.
    /// </summary>
    private PowerShell currentPowerShell;
```

```
    /// <summary>
    /// Used to serialize access to instance data.
    /// </summary>
    private object instanceLock = new object();

    /// <summary>
    /// Gets or sets a value indicating whether the host application
    /// should exit.
    /// </summary>
    public bool ShouldExit
    {
      get { return this.shouldExit; }
      set { this.shouldExit = value; }
    }

    /// <summary>
    /// Gets or sets a value indicating whether the host application
    /// should exit.
    /// </summary>
    public int ExitCode
    {
      get { return this.exitCode; }
      set { this.exitCode = value; }
    }

    /// <summary>
    /// Creates and initiates the listener instance.
    /// </summary>
    /// <param name="args">This parameter is not used.</param>
    private static void Main(string[] args)
    {
      // Display the welcome message.
      Console.Title = "PowerShell Console Host Sample Application";
      ConsoleColor oldFg = Console.ForegroundColor;
      Console.ForegroundColor = ConsoleColor.Cyan;
      Console.WriteLine("    Windows PowerShell Console Host Application Sample");
      Console.WriteLine("    ================================================");
      Console.WriteLine(string.Empty);
      Console.WriteLine("This is an example of a simple interactive console host
uses ");
      Console.WriteLine("the Windows PowerShell engine to interpret commands.");
      Console.WriteLine("Type 'exit' to exit.");
      Console.WriteLine(string.Empty);
      Console.ForegroundColor = oldFg;

      // Create the listener and run it. This method never returns.
      PSListenerConsoleSample listener = new PSListenerConsoleSample();
      listener.Run();
    }

    /// <summary>
    /// Initializes a new instance of the PSListenerConsoleSample class.
    /// </summary>
    public PSListenerConsoleSample()
```

```
    {
      // Create the host and runspace instances for this interpreter.
      // Note that this application does not support console files so
      // only the default snap-ins will be available.
      this.myHost = new MyHost(this);
      this.myRunSpace = RunspaceFactory.CreateRunspace(this.myHost);
      this.myRunSpace.Open();

      // Create a PowerShell object to run the commands used to create
      // $PROFILE and load the profiles.
      lock (this.instanceLock)
      {
        this.currentPowerShell = PowerShell.Create();
      }

      try
      {
        this.currentPowerShell.Runspace = this.myRunSpace;

        PSCommand[] profileCommands =
Microsoft.Samples.PowerShell.Host.HostUtilities.GetProfileCommands("SampleHost06");
        foreach (PSCommand command in profileCommands)
        {
          this.currentPowerShell.Commands = command;
          this.currentPowerShell.Invoke();
        }
      }
      finally
      {
        // Dispose the PowerShell object and set currentPowerShell
        // to null. It is locked because currentPowerShell may be
        // accessed by the ctrl-C handler.
        lock (this.instanceLock)
        {
          this.currentPowerShell.Dispose();
          this.currentPowerShell = null;
        }
      }
    }

    /// <summary>
    /// A helper class that builds and executes a pipeline that writes
    /// to the default output path. Any exceptions that are thrown are
    /// just passed to the caller. Since all output goes to the default
    /// outputter, this method does not return anything.
    /// </summary>
    /// <param name="cmd">The script to run.</param>
    /// <param name="input">Any input arguments to pass to the script.
    /// If null then nothing is passed in.</param>
    private void executeHelper(string cmd, object input)
    {
      // Ignore empty command lines.
      if (String.IsNullOrEmpty(cmd))
      {
        return;
```

```
      }

      // Create the pipeline object and make it available to the
      // ctrl-C handle through the currentPowerShell instance
      // variable.
      lock (this.instanceLock)
      {
        this.currentPowerShell = PowerShell.Create();
      }

      // Add a script and command to the pipeline and then run the pipeline. Place
      // the results in the currentPowerShell variable so that the pipeline can be
      // stopped.
      try
      {
        this.currentPowerShell.Runspace = this.myRunSpace;

        this.currentPowerShell.AddScript(cmd);

        // Add the default outputter to the end of the pipe and then call the
        // MergeMyResults method to merge the output and error streams from the
        // pipeline. This will result in the output being written using the PSHost
        // and PSHostUserInterface classes instead of returning objects to the host
        // application.
        this.currentPowerShell.AddCommand("Out-Default");

this.currentPowerShell.Commands.Commands[0].MergeMyResults(PipelineResultTypes.Erro
r, PipelineResultTypes.Output);

        // If there is any input pass it in, otherwise just invoke the
        // the pipeline.
        if (input != null)
        {
          this.currentPowerShell.Invoke(new object[] { input });
        }
        else
        {
          this.currentPowerShell.Invoke();
        }
      }
      finally
      {
        // Dispose the PowerShell object and set currentPowerShell to null.
        // It is locked because currentPowerShell may be accessed by the
        // ctrl-C handler.
        lock (this.instanceLock)
        {
          this.currentPowerShell.Dispose();
          this.currentPowerShell = null;
        }
      }
    }

    /// <summary>
    /// To display an exception using the display formatter,
```

```
    /// run a second pipeline passing in the error record.
    /// The runtime will bind this to the $input variable,
    /// which is why $input is being piped to the Out-String
    /// cmdlet. The WriteErrorLine method is called to make sure
    /// the error gets displayed in the correct error color.
    /// </summary>
    /// <param name="e">The exception to display.</param>
    private void ReportException(Exception e)
    {
      if (e != null)
      {
        object error;
        IContainsErrorRecord icer = e as IContainsErrorRecord;
        if (icer != null)
        {
          error = icer.ErrorRecord;
        }
        else
        {
          error = (object)new ErrorRecord(e, "Host.ReportException",
ErrorCategory.NotSpecified, null);
        }

        lock (this.instanceLock)
        {
          this.currentPowerShell = PowerShell.Create();
        }

        this.currentPowerShell.Runspace = this.myRunSpace;

        try
        {
          this.currentPowerShell.AddScript("$input").AddCommand("Out-String");

          // Do not merge errors, this function will swallow errors.
          Collection<PSObject> result;
          PSDataCollection<object> inputCollection = new PSDataCollection<object>
();
          inputCollection.Add(error);
          inputCollection.Complete();
          result = this.currentPowerShell.Invoke(inputCollection);

          if (result.Count > 0)
          {
            string str = result[0].BaseObject as string;
            if (!string.IsNullOrEmpty(str))
            {
              // Remove \r\n, which is added by the Out-String cmdlet.
              this.myHost.UI.WriteErrorLine(str.Substring(0, str.Length - 2));
            }
          }
        }
        finally
        {
          // Dispose of the pipeline and set it to null, locking it  because
```

```
          // currentPowerShell may be accessed by the ctrl-C handler.
          lock (this.instanceLock)
          {
            this.currentPowerShell.Dispose();
            this.currentPowerShell = null;
          }
        }
      }
    }

    /// <summary>
    /// Basic script execution routine. Any runtime exceptions are
    /// caught and passed back to the Windows PowerShell engine to
    /// display.
    /// </summary>
    /// <param name="cmd">Script to run.</param>
    private void Execute(string cmd)
    {
      try
      {
        // Run the command with no input.
        this.executeHelper(cmd, null);
      }
      catch (RuntimeException rte)
      {
        this.ReportException(rte);
      }
    }

    /// <summary>
    /// Method used to handle control-C's from the user. It calls the
    /// pipeline Stop() method to stop execution. If any exceptions occur
    /// they are printed to the console but otherwise ignored.
    /// </summary>
    /// <param name="sender">See sender property documentation of
    /// ConsoleCancelEventHandler.</param>
    /// <param name="e">See e property documentation of
    /// ConsoleCancelEventHandler.</param>
    private void HandleControlC(object sender, ConsoleCancelEventArgs e)
    {
      try
      {
        lock (this.instanceLock)
        {
          if (this.currentPowerShell != null &&
this.currentPowerShell.InvocationStateInfo.State == PSInvocationState.Running)
          {
            this.currentPowerShell.Stop();
          }
        }

        e.Cancel = true;
      }
      catch (Exception exception)
      {
```

```
        this.myHost.UI.WriteErrorLine(exception.ToString());
      }
    }

    /// <summary>
    /// Implements the basic listener loop. It sets up the ctrl-C handler, then
    /// reads a command from the user, executes it and repeats until the ShouldExit
    /// flag is set.
    /// </summary>
    private void Run()
    {
      // Set up the control-C handler.
      Console.CancelKeyPress += new ConsoleCancelEventHandler(this.HandleControlC);
      Console.TreatControlCAsInput = false;

      // Read commands and run them until the ShouldExit flag is set by
      // the user calling "exit".
      while (!this.ShouldExit)
      {
        string prompt;
        if (this.myHost.IsRunspacePushed)
        {
          prompt = string.Format("\n[{0}] PSConsoleSample: ",
this.myRunSpace.ConnectionInfo.ComputerName);
        }
        else
        {
          prompt = "\nPSConsoleSample: ";
        }

        this.myHost.UI.Write(ConsoleColor.Cyan, ConsoleColor.Black, prompt);
        string cmd = this.consoleReadLine.Read();
        this.Execute(cmd);
      }

      // Exit with the desired exit code that was set by the exit command.
      // The exit code is set in the host by the MyHost.SetShouldExit() method.
      Environment.Exit(this.ExitCode);
    }
  }
}
```

Example 2

The following code is the implementation of the System.Management.Automation.Host.PSHost class that is used by this host application. Those elements that are not implemented throw an exception or return nothing.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Globalization;
  using System.Management.Automation.Host;
  using System.Management.Automation.Runspaces;

  /// <summary>
  /// This is a sample implementation of the PSHost abstract class for
  /// console applications. Not all members are implemented. Those that
  /// are not implemented throw a NotImplementedException exception or
  /// return nothing.
  /// </summary>
  internal class MyHost : PSHost, IHostSupportsInteractiveSession
  {
        public MyHost(PSListenerConsoleSample program)
        {
            this.program = program;
        }

    /// <summary>
    /// A reference to the PSHost implementation.
    /// </summary>
    private PSListenerConsoleSample program;

    /// <summary>
    /// The culture information of the thread that created
    /// this object.
    /// </summary>
    private CultureInfo originalCultureInfo =
        System.Threading.Thread.CurrentThread.CurrentCulture;

    /// <summary>
    /// The UI culture information of the thread that created
    /// this object.
    /// </summary>
    private CultureInfo originalUICultureInfo =
        System.Threading.Thread.CurrentThread.CurrentUICulture;

    /// <summary>
    /// The identifier of this PSHost implementation.
    /// </summary>
    private static Guid instanceId = Guid.NewGuid();

    /// <summary>
    /// A reference to the implementation of the PSHostUserInterface
    /// class for this application.
    /// </summary>
    private MyHostUserInterface myHostUserInterface = new MyHostUserInterface();

    /// <summary>
    /// A reference to the runspace used to start an interactive session.
    /// </summary>
    public Runspace pushedRunspace = null;
```

```
    /// <summary>
    /// Gets the culture information to use. This implementation
    /// returns a snapshot of the culture information of the thread
    /// that created this object.
    /// </summary>
    public override CultureInfo CurrentCulture
    {
      get { return this.originalCultureInfo; }
    }

    /// <summary>
    /// Gets the UI culture information to use. This implementation
    /// returns a snapshot of the UI culture information of the thread
    /// that created this object.
    /// </summary>
    public override CultureInfo CurrentUICulture
    {
      get { return this.originalUICultureInfo; }
    }

    /// <summary>
    /// Gets an identifier for this host. This implementation always
    /// returns the GUID allocated at instantiation time.
    /// </summary>
    public override Guid InstanceId
    {
      get { return instanceId; }
    }

    /// <summary>
    /// Gets a string that contains the name of this host implementation.
    /// Keep in mind that this string may be used by script writers to
    /// identify when your host is being used.
    /// </summary>
    public override string Name
    {
      get { return "MySampleConsoleHostImplementation"; }
    }

    /// <summary>
    /// Gets an instance of the implementation of the PSHostUserInterface
    /// class for this application. This instance is allocated once at startup time
    /// and returned every time thereafter.
    /// </summary>
    public override PSHostUserInterface UI
    {
      get { return this.myHostUserInterface; }
    }

    /// <summary>
    /// Gets the version object for this application. Typically this
    /// should match the version resource in the application.
    /// </summary>
    public override Version Version
```

```
    {
      get { return new Version(1, 0, 0, 0); }
    }

    #region IHostSupportsInteractiveSession Properties

    /// <summary>
    /// Gets a value indicating whether a request
    /// to open a PSSession has been made.
    /// </summary>
    public bool IsRunspacePushed
    {
      get { return this.pushedRunspace != null; }
    }

    /// <summary>
    /// Gets or sets the runspace used by the PSSession.
    /// </summary>
    public Runspace Runspace
    {
      get { return this.program.myRunSpace; }
      internal set { this.program.myRunSpace = value; }
    }
    #endregion IHostSupportsInteractiveSession Properties

    /// <summary>
    /// This API Instructs the host to interrupt the currently running
    /// pipeline and start a new nested input loop. In this example this
    /// functionality is not needed so the method throws a
    /// NotImplementedException exception.
    /// </summary>
    public override void EnterNestedPrompt()
    {
      throw new NotImplementedException(
            "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API instructs the host to exit the currently running input loop.
    /// In this example this functionality is not needed so the method
    /// throws a NotImplementedException exception.
    /// </summary>
    public override void ExitNestedPrompt()
    {
      throw new NotImplementedException(
            "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API is called before an external application process is
    /// started. Typically it is used to save state so that the parent
    /// can restore state that has been modified by a child process (after
    /// the child exits). In this example this functionality is not
    /// needed so the method returns nothing.
    /// </summary>
```

```
    public override void NotifyBeginApplication()
    {
      return;
    }

    /// <summary>
    /// This API is called after an external application process finishes.
    /// Typically it is used to restore state that a child process has
    /// altered. In this example, this functionality is not needed so
    /// the method returns nothing.
    /// </summary>
    public override void NotifyEndApplication()
    {
      return;
    }

    /// <summary>
    /// Indicate to the host application that exit has
    /// been requested. Pass the exit code that the host
    /// application should use when exiting the process.
    /// </summary>
    /// <param name="exitCode">The exit code that the
    /// host application should use.</param>
    public override void SetShouldExit(int exitCode)
    {
      this.program.ShouldExit = true;
      this.program.ExitCode = exitCode;
    }

    #region IHostSupportsInteractiveSession Methods

    /// <summary>
    /// Requests to close a PSSession.
    /// </summary>
    public void PopRunspace()
    {
      Runspace = this.pushedRunspace;
      this.pushedRunspace = null;
    }

    /// <summary>
    /// Requests to open a PSSession.
    /// </summary>
    /// <param name="runspace">Runspace to use.</param>
    public void PushRunspace(Runspace runspace)
    {
      this.pushedRunspace = Runspace;
      Runspace = runspace;
    }

    #endregion IHostSupportsInteractiveSession Methods
  }
}
```

Example 3

The following code is the implementation of the System.Management.Automation.Host.PSHostUserInterface class that is used by this host application.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Collections.Generic;
  using System.Collections.ObjectModel;
  using System.Globalization;
  using System.Management.Automation;
  using System.Management.Automation.Host;
  using System.Text;

  /// <summary>
  /// A sample implementation of the PSHostUserInterface abstract class for
  /// console applications. Not all members are implemented. Those that are
  /// not implemented throw a NotImplementedException exception or return
  /// nothing. Members that are implemented include those that map easily to
  /// Console APIs and a basic implementation of the prompt API provided.
  /// </summary>
  internal class MyHostUserInterface : PSHostUserInterface,
IHostUISupportsMultipleChoiceSelection
  {
    /// <summary>
    /// A reference to the PSRawUserInterface implementation.
    /// </summary>
    private MyRawUserInterface myRawUi = new MyRawUserInterface();

    /// <summary>
    /// Gets an instance of the PSRawUserInterface class for this host
    /// application.
    /// </summary>
    public override PSHostRawUserInterface RawUI
    {
      get { return this.myRawUi; }
    }

    /// <summary>
    /// Prompts the user for input.
    /// <param name="caption">The caption or title of the prompt.</param>
    /// <param name="message">The text of the prompt.</param>
    /// <param name="descriptions">A collection of FieldDescription objects
    /// that describe each field of the prompt.</param>
    /// <returns>A dictionary object that contains the results of the user
    /// prompts.</returns>
    public override Dictionary<string, PSObject> Prompt(
                            string caption,
```

```
                              string message,
                              Collection<FieldDescription> descriptions)
    {
      this.Write(
                 ConsoleColor.DarkCyan,
                 ConsoleColor.Black,
                 caption + "\n" + message + " ");
      Dictionary<string, PSObject> results =
               new Dictionary<string, PSObject>();
      foreach (FieldDescription fd in descriptions)
      {
        string[] label = GetHotkeyAndLabel(fd.Label);
        this.WriteLine(label[1]);
        string userData = Console.ReadLine();
        if (userData == null)
        {
          return null;
        }

        results[fd.Name] = PSObject.AsPSObject(userData);
      }

      return results;
    }

    /// <summary>

/// Provides a set of choices that enable the user to choose a
    /// single option from a set of options.
    /// </summary>
    /// <param name="caption">Text that proceeds (a title) the choices.</param>
    /// <param name="message">A message that describes the choice.</param>
    /// <param name="choices">A collection of ChoiceDescription objects that
    /// describe each choice.</param>
    /// <param name="defaultChoice">The index of the label in the Choices
    /// parameter collection. To indicate no default choice, set to -1.</param>
    /// <returns>The index of the Choices parameter collection element that
    /// corresponds to the option that is selected by the user.</returns>
    public override int PromptForChoice(
                                        string caption,
                                        string message,
                                        Collection<ChoiceDescription> choices,
                                        int defaultChoice)
    {
      // Write the caption and message strings in Blue.
      this.WriteLine(
                     ConsoleColor.Blue,
                     ConsoleColor.Black,
                     caption + "\n" + message + "\n");

      // Convert the choice collection into something that is
      // easier to work with. See the BuildHotkeysAndPlainLabels
      // method for details.
      string[,] promptData = BuildHotkeysAndPlainLabels(choices);
```

```
      // Format the overall choice prompt string to display.
      StringBuilder sb = new StringBuilder();
      for (int element = 0; element < choices.Count; element++)
      {
        sb.Append(String.Format(
                                CultureInfo.CurrentCulture,
                                "|{0}> {1} ",
                                promptData[0, element],
                                promptData[1, element]));
      }

      sb.Append(String.Format(
                               CultureInfo.CurrentCulture,
                               "[Default is ({0}]",
                               promptData[0, defaultChoice]));

      // Read prompts until a match is made, the default is
      // chosen, or the loop is interrupted with ctrl-C.
      while (true)
      {
        this.WriteLine(ConsoleColor.Cyan, ConsoleColor.Black, sb.ToString());
        string data =
Console.ReadLine().Trim().ToUpper(CultureInfo.CurrentCulture);

        // If the choice string was empty, use the default selection.
        if (data.Length == 0)
        {
          return defaultChoice;
        }

        // See if the selection matched and return the
        // corresponding index if it did.
        for (int i = 0; i < choices.Count; i++)
        {
          if (promptData[0, i] == data)
          {
            return i;
          }
        }

        this.WriteErrorLine("Invalid choice: " + data);
      }
    }

    #region IHostUISupportsMultipleChoiceSelection Members

        /// <summary>
    /// Provides a set of choices that enable the user to choose a one or
    /// more options from a set of options.
    /// </summary>
    /// <param name="caption">Text that proceeds (a title) the choices.</param>
    /// <param name="message">A message that describes the choice.</param>
    /// <param name="choices">A collection of ChoiceDescription objects that
    /// describe each choice.</param>
    /// <param name="defaultChoices">The index of the label in the Choices
```

```
    /// parameter collection. To indicate no default choice, set to -1.</param>
    /// <returns>The index of the Choices parameter collection element that
    /// corresponds to the option that is selected by the user.</returns>
    public Collection<int> PromptForChoice(
                                            string caption,
                                            string message,
                                            Collection<ChoiceDescription> choices,
                                            IEnumerable<int> defaultChoices)
    {
      // Write the caption and message strings in Blue.
      this.WriteLine(
                    ConsoleColor.Blue,
                    ConsoleColor.Black,
                    caption + "\n" + message + "\n");

      // Convert the choice collection into something that is
      // easier to work with. See the BuildHotkeysAndPlainLabels
      // method for details.
      string[,] promptData = BuildHotkeysAndPlainLabels(choices);

      // Format the overall choice prompt string to display.
      StringBuilder sb = new StringBuilder();
      for (int element = 0; element < choices.Count; element++)
      {
        sb.Append(String.Format(
                                CultureInfo.CurrentCulture,
                                "|{0}> {1} ",
                                promptData[0, element],
                                promptData[1, element]));
      }

      Collection<int> defaultResults = new Collection<int>();
      if (defaultChoices != null)
      {
        int countDefaults = 0;
        foreach (int defaultChoice in defaultChoices)
        {
          ++countDefaults;
          defaultResults.Add(defaultChoice);
        }

        if (countDefaults != 0)
        {
          sb.Append(countDefaults == 1 ? "[Default choice is " : "[Default choices
are ");
          foreach (int defaultChoice in defaultChoices)
          {
            sb.AppendFormat(
                            CultureInfo.CurrentCulture,
                            "\"{0}\",",
                            promptData[0, defaultChoice]);
          }

          sb.Remove(sb.Length - 1, 1);
          sb.Append("]");
```

```
        }
      }

      this.WriteLine(
                    ConsoleColor.Cyan,
                    ConsoleColor.Black,
                    sb.ToString());
      // Read prompts until a match is made, the default is
      // chosen, or the loop is interrupted with ctrl-C.
      Collection<int> results = new Collection<int>();
      while (true)
      {
        ReadNext:
        string prompt = string.Format(CultureInfo.CurrentCulture, "Choice[{0}]:",
results.Count);
        this.Write(ConsoleColor.Cyan, ConsoleColor.Black, prompt);
        string data =
Console.ReadLine().Trim().ToUpper(CultureInfo.CurrentCulture);

        // If the choice string was empty, no more choices have been made.
        // If there were no choices made, return the defaults
        if (data.Length == 0)
        {
          return (results.Count == 0) ? defaultResults : results;
        }

        // See if the selection matched and return the
        // corresponding index if it did.
        for (int i = 0; i < choices.Count; i++)
        {
          if (promptData[0, i] == data)
          {
            results.Add(i);
            goto ReadNext;
          }
        }

        this.WriteErrorLine("Invalid choice: " + data);
      }
    }

    #endregion

    /// <summary>
    /// Prompts the user for credentials with a specified prompt window
    /// caption, prompt message, user name, and target name. In this
    /// example this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="caption">The caption for the message window.</param>
    /// <param name="message">The text of the message.</param>
    /// <param name="userName">The user name whose credential is to be
    /// prompted for.</param>
    /// <param name="targetName">The name of the target for which the
    /// credential is collected.</param>
```

```
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override PSCredential PromptForCredential(
                                                    string caption,
                                                    string message,
                                                    string userName,
                                                    string targetName)
    {
      throw new NotImplementedException(
                          "The method or operation is not implemented.");
    }

    /// <summary>
    /// Prompts the user for credentials by using a specified prompt window
    /// caption, prompt message, user name and target name, credential
    /// types allowed to be returned, and UI behavior options. In this
    /// example this functionality is not needed so the method throws a
    /// NotImplementException exception.
    /// </summary>
    /// <param name="caption">The caption for the message window.</param>
    /// <param name="message">The text of the message.</param>
    /// <param name="userName">The user name whose credential is to be
    /// prompted for.</param>
    /// <param name="targetName">The name of the target for which the
    /// credential is collected.</param>
    /// <param name="allowedCredentialTypes">A PSCredentialTypes constant
    /// that identifies the type of credentials that can be returned.</param>
    /// <param name="options">A PSCredentialUIOptions constant that
    /// identifies the UI behavior when it gathers the credentials.</param>
    /// <returns>Throws a NotImplementedException exception.</returns>
    public override PSCredential PromptForCredential(
                                       string caption,
                                       string message,
                                       string userName,
                                       string targetName,
                                       PSCredentialTypes allowedCredentialTypes,
                                       PSCredentialUIOptions options)
    {
      throw new NotImplementedException(
                              "The method or operation is not implemented.");
    }

    /// <summary>
    /// Reads characters that are entered by the user until a newline
    /// (carriage return) is encountered.
    /// </summary>
    /// <returns>The characters that are entered by the user.</returns>
    public override string ReadLine()
    {
      return Console.ReadLine();
    }

    /// <summary>
    /// Reads characters entered by the user until a newline (carriage return)
    /// is encountered and returns the characters as a secure string. In this
    /// example this functionality is not needed so the method throws a
```

```
    /// NotImplementException exception.
    /// </summary>
    /// <returns>Throws a NotImplemented exception.</returns>
    public override System.Security.SecureString ReadLineAsSecureString()
    {
      throw new NotImplementedException("The method or operation is not
implemented.");
    }

    /// <summary>
    /// Writes characters to the output display of the host.
    /// </summary>
    /// <param name="value">The characters to be written.</param>
    public override void Write(string value)
    {
      Console.Write(value);
    }

    /// <summary>
    /// Writes characters to the output display of the host with possible
    /// foreground and background colors.
    /// </summary>
    /// <param name="foregroundColor">The color of the characters.</param>
    /// <param name="backgroundColor">The background color to use.</param>
    /// <param name="value">The characters to be written.</param>
    public override void Write(
                               ConsoleColor foregroundColor,
                               ConsoleColor backgroundColor,
                               string value)
    {
      ConsoleColor oldFg = Console.ForegroundColor;
      ConsoleColor oldBg = Console.BackgroundColor;
      Console.ForegroundColor = foregroundColor;
      Console.BackgroundColor = backgroundColor;
      Console.Write(value);
      Console.ForegroundColor = oldFg;
      Console.BackgroundColor = oldBg;
    }

    /// <summary>
    /// Writes a line of characters to the output display of the host
    /// with foreground and background colors and appends a newline (carriage
return).
    /// </summary>
    /// <param name="foregroundColor">The foreground color of the display. </param>
    /// <param name="backgroundColor">The background color of the display. </param>
    /// <param name="value">The line to be written.</param>
    public override void WriteLine(
                                   ConsoleColor foregroundColor,
                                   ConsoleColor backgroundColor,
                                   string value)
    {
      ConsoleColor oldFg = Console.ForegroundColor;
      ConsoleColor oldBg = Console.BackgroundColor;
      Console.ForegroundColor = foregroundColor;
```

```
      Console.BackgroundColor = backgroundColor;
      Console.WriteLine(value);
      Console.ForegroundColor = oldFg;
      Console.BackgroundColor = oldBg;
    }

    /// <summary>
    /// Writes a debug message to the output display of the host.
    /// </summary>
    /// <param name="message">The debug message that is displayed.</param>
    public override void WriteDebugLine(string message)
    {
      this.WriteLine(
                    ConsoleColor.DarkYellow,
                    ConsoleColor.Black,
                    String.Format(CultureInfo.CurrentCulture, "DEBUG: {0}",
message));
    }

    /// <summary>
    /// Writes an error message to the output display of the host.
    /// </summary>
    /// <param name="value">The error message that is displayed.</param>
    public override void WriteErrorLine(string value)
    {
      this.WriteLine(
                    ConsoleColor.Red,
                    ConsoleColor.Black,
                    value);
    }

    /// <summary>
    /// Writes a newline character (carriage return)
    /// to the output display of the host.
    /// </summary>
    public override void WriteLine()
    {
      Console.WriteLine();
    }

       /// <summary>
    /// Writes a line of characters to the output display of the host
    /// and appends a newline character(carriage return).
    /// </summary>
    /// <param name="value">The line to be written.</param>
    public override void WriteLine(string value)
    {
      Console.WriteLine(value);
    }

    /// <summary>
    /// Writes a progress report to the output display of the host.
    /// </summary>
    /// <param name="sourceId">Unique identifier of the source of the record.
</param>
```

```
    /// <param name="record">A ProgressReport object.</param>
    public override void WriteProgress(long sourceId, ProgressRecord record)
    {

    }

    /// <summary>
    /// Writes a verbose message to the output display of the host.
    /// </summary>
    /// <param name="message">The verbose message that is displayed.</param>
    public override void WriteVerboseLine(string message)
    {
      this.WriteLine(
                    ConsoleColor.Green,
                    ConsoleColor.Black,
                    String.Format(CultureInfo.CurrentCulture, "VERBOSE: {0}",
message));
    }

    /// <summary>
    /// Writes a warning message to the output display of the host.
    /// </summary>
    /// <param name="message">The warning message that is displayed.</param>
    public override void WriteWarningLine(string message)
    {
      this.WriteLine(
                    ConsoleColor.Yellow,
                    ConsoleColor.Black,
                    String.Format(CultureInfo.CurrentCulture, "WARNING: {0}",
message));
    }

    /// <summary>
    /// Parse a string containing a hotkey character.
    /// Take a string of the form
    ///    Yes to &all
    /// and returns a two-dimensional array split out as
    ///    "A", "Yes to all".
    /// </summary>
    /// <param name="input">The string to process</param>
    /// <returns>
    /// A two dimensional array containing the parsed components.
    /// </returns>
    private static string[] GetHotkeyAndLabel(string input)
    {
      string[] result = new string[] { String.Empty, String.Empty };
      string[] fragments = input.Split('&');
      if (fragments.Length == 2)
      {
        if (fragments[1].Length > 0)
        {
          result[0] = fragments[1][0].ToString().
          ToUpper(CultureInfo.CurrentCulture);
        }
```

```
        result[1] = (fragments[0] + fragments[1]).Trim();
      }
      else
      {
        result[1] = input;
      }

      return result;
    }

    /// <summary>
    /// This is a private worker function splits out the
    /// accelerator keys from the menu and builds a two
    /// dimensional array with the first access containing the
    /// accelerator and the second containing the label string
    /// with the & removed.
    /// </summary>
    /// <param name="choices">The choice collection to process</param>
    /// <returns>
    /// A two dimensional array containing the accelerator characters
    /// and the cleaned-up labels</returns>
    private static string[,] BuildHotkeysAndPlainLabels(
         Collection<ChoiceDescription> choices)
    {
      // Allocate the result array
      string[,] hotkeysAndPlainLabels = new string[2, choices.Count];

      for (int i = 0; i < choices.Count; ++i)
      {
        string[] hotkeyAndLabel = GetHotkeyAndLabel(choices[i].Label);
        hotkeysAndPlainLabels[0, i] = hotkeyAndLabel[0];
        hotkeysAndPlainLabels[1, i] = hotkeyAndLabel[1];
      }

      return hotkeysAndPlainLabels;
    }
  }
}
```

Example 4

The following code is the implementation of the System.Management.Automation.Host.PSHostRawUserInterface class that is used by this host application. Those elements that are not implemented throw an exception or return nothing.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
  using System;
  using System.Management.Automation.Host;
```

```
/// <summary>
/// A sample implementation of the PSHostRawUserInterface for console
/// applications. Members of this class that easily map to the .NET
/// console class are implemented. More complex methods are not
/// implemented and throw a NotImplementedException exception.
/// </summary>
internal class MyRawUserInterface : PSHostRawUserInterface
{
  /// <summary>
  /// Gets or sets the background color of text to be written.
  /// This maps to the corresponding Console.Background property.
  /// </summary>
  public override ConsoleColor BackgroundColor
  {
    get { return Console.BackgroundColor; }
    set { Console.BackgroundColor = value; }
  }

  /// <summary>
  /// Gets or sets the host buffer size adapted from the Console buffer
  /// size members.
  /// </summary>
  public override Size BufferSize
  {
    get { return new Size(Console.BufferWidth, Console.BufferHeight); }
    set { Console.SetBufferSize(value.Width, value.Height); }
  }

  /// <summary>
  /// Gets or sets the cursor position. In this example this
  /// functionality is not needed so the property throws a
  /// NotImplementException exception.
  /// </summary>
  public override Coordinates CursorPosition
  {
    get { throw new NotImplementedException(
                "The method or operation is not implemented."); }
    set { throw new NotImplementedException(
                "The method or operation is not implemented."); }
  }

  /// <summary>
  /// Gets or sets the cursor size taken directly from the
  /// Console.CursorSize property.
  /// </summary>
  public override int CursorSize
  {
    get { return Console.CursorSize; }
    set { Console.CursorSize = value; }
  }

  /// <summary>
  /// Gets or sets the foreground color of the text to be written.
  /// This maps to the corresponding Console.ForegroundColor property.
```

```
    /// </summary>
    public override ConsoleColor ForegroundColor
    {
      get { return Console.ForegroundColor; }
      set { Console.ForegroundColor = value; }
    }

    /// <summary>
    /// Gets a value indicating whether a key is available. This maps to
    /// the corresponding Console.KeyAvailable property.
    /// </summary>
    public override bool KeyAvailable
    {
      get { return Console.KeyAvailable; }
    }

    /// <summary>
    /// Gets the maximum physical size of the window adapted from the
    /// Console.LargestWindowWidth and Console.LargestWindowHeight
    /// properties.
    /// </summary>
    public override Size MaxPhysicalWindowSize
    {
      get { return new Size(Console.LargestWindowWidth,
Console.LargestWindowHeight); }
    }

    /// <summary>
    /// Gets the maximum window size adapted from the
    /// Console.LargestWindowWidth and console.LargestWindowHeight
    /// properties.
    /// </summary>
    public override Size MaxWindowSize
    {
      get { return new Size(Console.LargestWindowWidth,
Console.LargestWindowHeight); }
    }

    /// <summary>
    /// Gets or sets the window position adapted from the Console window position
    /// members.
    /// </summary>
    public override Coordinates WindowPosition
    {
      get { return new Coordinates(Console.WindowLeft, Console.WindowTop); }
      set { Console.SetWindowPosition(value.X, value.Y); }
    }

    /// <summary>
    /// Gets or sets the window size adapted from the corresponding Console
    /// calls.
    /// </summary>
    public override Size WindowSize
    {
      get { return new Size(Console.WindowWidth, Console.WindowHeight); }
```

```
    set { Console.SetWindowSize(value.Width, value.Height); }
  }

  /// <summary>
  /// Gets or sets the title of the window mapped to the Console.Title
  /// property.
  /// </summary>
  public override string WindowTitle
  {
    get { return Console.Title; }
    set { Console.Title = value; }
  }

  /// <summary>
  /// This API resets the input buffer. In this example this
  /// functionality is not needed so the method returns nothing.
  /// </summary>
  public override void FlushInputBuffer()
  {
  }

  /// <summary>
  /// This API returns a rectangular region of the screen buffer. In
  /// this example this functionality is not needed so the method throws
  /// a NotImplementException exception.
  /// </summary>
  /// <param name="rectangle">Defines the size of the rectangle.</param>
  /// <returns>Throws a NotImplementedException exception.</returns>
  public override BufferCell[,] GetBufferContents(Rectangle rectangle)
  {
    throw new NotImplementedException(
              "The method or operation is not implemented.");
  }

  /// <summary>
  /// This API Reads a pressed, released, or pressed and released keystroke
  /// from the keyboard device, blocking processing until a keystroke is
  /// typed that matches the specified keystroke options. In this example
  /// this functionality is not needed so the method throws a
  /// NotImplementException exception.
  /// </summary>
  /// <param name="options">Options, such as IncludeKeyDown,  used when
  /// reading the keyboard.</param>
  /// <returns>Throws a NotImplementedException exception.</returns>
  public override KeyInfo ReadKey(ReadKeyOptions options)
  {
    throw new NotImplementedException(
               "The method or operation is not implemented.");
  }

  /// <summary>
  /// This API crops a region of the screen buffer. In this example
  /// this functionality is not needed so the method throws a
  /// NotImplementException exception.
  /// </summary>
```

```
    /// <param name="source">The region of the screen to be scrolled.</param>
    /// <param name="destination">The region of the screen to receive the
    /// source region contents.</param>
    /// <param name="clip">The region of the screen to include in the operation.
</param>
    /// <param name="fill">The character and attributes to be used to fill all
cell.</param>
    public override void ScrollBufferContents(Rectangle source, Coordinates
destination, Rectangle clip, BufferCell fill)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API copies an array of buffer cells into the screen buffer
    /// at a specified location. In this example this  functionality is
    /// not needed si the method  throws a NotImplementedException exception.
    /// </summary>
    /// <param name="origin">The parameter is not used.</param>
    /// <param name="contents">The parameter is not used.</param>
    public override void SetBufferContents(Coordinates origin, BufferCell[,]
contents)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }

    /// <summary>
    /// This API Copies a given character, foreground color, and background
    /// color to a region of the screen buffer. In this example this
    /// functionality is not needed so the method throws a
    /// NotImplementException exception./// </summary>
    /// <param name="rectangle">Defines the area to be filled. </param>
    /// <param name="fill">Defines the fill character.</param>
    public override void SetBufferContents(Rectangle rectangle, BufferCell fill)
    {
      throw new NotImplementedException(
                "The method or operation is not implemented.");
    }
  }
}
```

Example 5

The following code reads the command line and colors the text as it is entered. Tokens are determined by using the System.Management.Automation.PSParser.Tokenize* method.

C#

```
namespace Microsoft.Samples.PowerShell.Host
{
```

```
using System;
using System.Collections.ObjectModel;
using System.Management.Automation;
using System.Text;

/// <summary>
/// This class is used to read the command line and color the text as
/// it is entered. Tokens are determined using the PSParser.Tokenize
/// method.
/// </summary>
internal class ConsoleReadLine
{
  /// <summary>
  /// The buffer used to edit.
  /// </summary>
  private StringBuilder buffer = new StringBuilder();

  /// <summary>
  /// The position of the cursor within the buffer.
  /// </summary>
  private int current;

  /// <summary>
  /// The count of characters in buffer rendered.
  /// </summary>
  private int rendered;

  /// <summary>
  /// Store the anchor and handle cursor movement
  /// </summary>
  private Cursor cursor;

  /// <summary>
  /// The array of colors for tokens, indexed by PSTokenType
  /// </summary>
  private ConsoleColor[] tokenColors;

  /// <summary>
  /// We do not pick different colors for every token, those tokens
  /// use this default.
  /// </summary>
  private ConsoleColor defaultColor = Console.ForegroundColor;

  /// <summary>
  /// Initializes a new instance of the ConsoleReadLine class.
  /// </summary>
  public ConsoleReadLine()
  {
    this.tokenColors = new ConsoleColor[]
    {
      this.defaultColor,       // Unknown
      ConsoleColor.Yellow,     // Command
      ConsoleColor.Green,      // CommandParameter
      ConsoleColor.Cyan,       // CommandArgument
      ConsoleColor.Cyan,       // Number
```

```
        ConsoleColor.Cyan,       // String
        ConsoleColor.Green,      // Variable
        this.defaultColor,            // Member
        this.defaultColor,            // LoopLabel
        ConsoleColor.DarkYellow, // Attribute
        ConsoleColor.DarkYellow, // Type
        ConsoleColor.DarkCyan,   // Operator
        this.defaultColor,            // GroupStart
        this.defaultColor,            // GroupEnd
        ConsoleColor.Magenta,    // Keyword
        ConsoleColor.Red,        // Comment
        ConsoleColor.DarkCyan,   // StatementSeparator
                this.defaultColor,            // NewLine
                this.defaultColor,            // LineContinuation
                this.defaultColor,            // Position
      };
    }

    /// <summary>
    /// Read a line of text, colorizing while typing.
    /// </summary>
    /// <returns>The command line read</returns>
    public string Read()
    {
      this.Initialize();

      while (true)
      {
        ConsoleKeyInfo key = Console.ReadKey(true);

        switch (key.Key)
        {
          case ConsoleKey.Backspace:
               this.OnBackspace();
               break;
          case ConsoleKey.Delete:
               this.OnDelete();
               break;
          case ConsoleKey.Enter:
               return this.OnEnter();
          case ConsoleKey.RightArrow:
               this.OnRight(key.Modifiers);
               break;
          case ConsoleKey.LeftArrow:
               this.OnLeft(key.Modifiers);
               break;
          case ConsoleKey.Escape:
               this.OnEscape();
               break;
          case ConsoleKey.Home:
               this.OnHome();
               break;
          case ConsoleKey.End:
               this.OnEnd();
               break;
```

```
        case ConsoleKey.UpArrow:
        case ConsoleKey.DownArrow:
        case ConsoleKey.LeftWindows:
        case ConsoleKey.RightWindows:
        // ignore these
        continue;

        default:
        if (key.KeyChar == '\x0D')
        {
          goto case ConsoleKey.Enter;      // Ctrl-M
        }

        if (key.KeyChar == '\x08')
        {
          goto case ConsoleKey.Backspace;  // Ctrl-H
        }

        this.Insert(key);
        break;
      }
    }
  }

  /// <summary>
  /// Initializes the buffer.
  /// </summary>
  private void Initialize()
  {
    this.buffer.Length = 0;
    this.current = 0;
    this.rendered = 0;
    this.cursor = new Cursor();
  }

  /// <summary>
  /// Inserts a key.
  /// </summary>
  /// <param name="key">The key to insert.</param>
  private void Insert(ConsoleKeyInfo key)
  {
    this.buffer.Insert(this.current, key.KeyChar);
    this.current++;
    this.Render();
  }

  /// <summary>
  /// The End key was entered..
  /// </summary>
  private void OnEnd()
  {
    this.current = this.buffer.Length;
    this.cursor.Place(this.rendered);
  }
```

```
    /// <summary>
    /// The Home key was entered.
    /// </summary>
    private void OnHome()
    {
      this.current = 0;
      this.cursor.Reset();
    }

    /// <summary>
    /// The Escape key was entered.
    /// </summary>
    private void OnEscape()
    {
      this.buffer.Length = 0;
      this.current = 0;
      this.Render();
    }

    /// <summary>
    /// Moves to the left of the cursor position.
    /// </summary>
    /// <param name="consoleModifiers">Enumeration for Alt, Control,
    /// and Shift keys.</param>
    private void OnLeft(ConsoleModifiers consoleModifiers)
    {
      if ((consoleModifiers & ConsoleModifiers.Control) != 0)
      {
        // Move back to the start of the previous word.
        if (this.buffer.Length > 0 && this.current != 0)
        {
          bool nonLetter = IsSeparator(this.buffer[this.current - 1]);
          while (this.current > 0 && (this.current - 1 < this.buffer.Length))
          {
            this.MoveLeft();

            if (IsSeparator(this.buffer[this.current]) != nonLetter)
            {
              if (!nonLetter)
              {
                this.MoveRight();
                break;
              }

              nonLetter = false;
            }
          }
        }
      }
      else
      {
        this.MoveLeft();
      }
    }
```

```
    /// <summary>
    /// Determines if a character is a separator.
    /// </summary>
    /// <param name="ch">Character to investigate.</param>
    /// <returns>A value that indicates whether the character
    /// is a separator.</returns>
    private static bool IsSeparator(char ch)
    {
      return !Char.IsLetter(ch);
    }

    /// <summary>
    /// Moves to what is to the right of the cursor position.
    /// </summary>
    /// <param name="consoleModifiers">Enumeration for Alt, Control,
    /// and Shift keys.</param>
    private void OnRight(ConsoleModifiers consoleModifiers)
    {
      if ((consoleModifiers & ConsoleModifiers.Control) != 0)
      {
        // Move to the next word.
        if (this.buffer.Length != 0 && this.current < this.buffer.Length)
        {
          bool nonLetter = IsSeparator(this.buffer[this.current]);
          while (this.current < this.buffer.Length)
          {
            this.MoveRight();

            if (this.current == this.buffer.Length)
            {
              break;
            }

            if (IsSeparator(this.buffer[this.current]) != nonLetter)
            {
              if (nonLetter)
              {
                break;
              }

              nonLetter = true;
            }
          }
        }
      }
      else
      {
        this.MoveRight();
      }
    }

    /// <summary>
    /// Moves the cursor one character to the right.
    /// </summary>
    private void MoveRight()
```

```
    {
      if (this.current < this.buffer.Length)
      {
        char c = this.buffer[this.current];
        this.current++;
        Cursor.Move(1);
      }
    }

    /// <summary>
    /// Moves the cursor one character to the left.
    /// </summary>
    private void MoveLeft()
    {
      if (this.current > 0 && (this.current - 1 < this.buffer.Length))
      {
        this.current--;
        char c = this.buffer[this.current];
        Cursor.Move(-1);
      }
    }

    /// <summary>
    /// The Enter key was entered.
    /// </summary>
    /// <returns>A newline character.</returns>
    private string OnEnter()
    {
      Console.Out.Write("\n");
      return this.buffer.ToString();
    }

    /// <summary>
    /// The delete key was entered.
    /// </summary>
    private void OnDelete()
    {
      if (this.buffer.Length > 0 && this.current < this.buffer.Length)
      {
        this.buffer.Remove(this.current, 1);
        this.Render();
      }
    }

    /// <summary>
    /// The Backspace key was entered.
    /// </summary>
    private void OnBackspace()
    {
      if (this.buffer.Length > 0 && this.current > 0)
      {
        this.buffer.Remove(this.current - 1, 1);
        this.current--;
        this.Render();
      }
```

```
    }

    /// <summary>
    /// Displays the line.
    /// </summary>
    private void Render()
    {
      string text = this.buffer.ToString();

      // The PowerShell tokenizer is used to decide how to colorize
      // the input.  Any errors in the input are returned in 'errors',
      // but we won't be looking at those here.
      Collection<PSParseError> errors = null;
      Collection<PSToken> tokens = PSParser.Tokenize(text, out errors);

      if (tokens.Count > 0)
      {
        // We can skip rendering tokens that end before the cursor.
        int i;
        for (i = 0; i < tokens.Count; ++i)
        {
          if (this.current >= tokens[i].Start)
          {
            break;
          }
        }

        // Place the cursor at the start of the first token to render.  The
        // last edit may require changes to the colorization of characters
        // preceding the cursor.
        this.cursor.Place(tokens[i].Start);

        for (; i < tokens.Count; ++i)
        {
          // Write out the token.  We don't use tokens[i].Content, instead we
          // use the actual text from our input because the content sometimes
          // excludes part of the token, e.g. the quote characters of a string.
          Console.ForegroundColor = this.tokenColors[(int)tokens[i].Type];
          Console.Out.Write(text.Substring(tokens[i].Start, tokens[i].Length));

          // Whitespace doesn't show up in the array of tokens.  Write it out here.
          if (i != (tokens.Count - 1))
          {
            Console.ForegroundColor = this.defaultColor;
            for (int j = (tokens[i].Start + tokens[i].Length); j < tokens[i +
1].Start; ++j)
            {
              Console.Out.Write(text[j]);
            }
          }
        }

        // It's possible there is text left over to output.  This happens when
there is
        // some error during tokenization, e.g. a string literal is missing a
```

```
closing quote.
        Console.ForegroundColor = this.defaultColor;
        for (int j = tokens[i - 1].Start + tokens[i - 1].Length; j < text.Length;
++j)
        {
          Console.Out.Write(text[j]);
        }
      }
      else
      {
        // If tokenization completely failed, just redraw the whole line.  This
        // happens most frequently when the first token is incomplete, like a
string
        // literal missing a closing quote.
        this.cursor.Reset();
        Console.Out.Write(text);
      }

      // If characters were deleted, we must write over previously written
characters
      if (text.Length < this.rendered)
      {
        Console.Out.Write(new string(' ', this.rendered - text.Length));
      }

      this.rendered = text.Length;
      this.cursor.Place(this.current);
    }

    /// <summary>
    /// A helper class for maintaining the cursor while editing the command line.
    /// </summary>
    internal class Cursor
    {
      /// <summary>
      /// The top anchor for reposition the cursor.
      /// </summary>
      private int anchorTop;

      /// <summary>
      /// The left anchor for repositioning the cursor.
      /// </summary>
      private int anchorLeft;

      /// <summary>
      /// Initializes a new instance of the Cursor class.
      /// </summary>
      public Cursor()
      {
        this.anchorTop = Console.CursorTop;
        this.anchorLeft = Console.CursorLeft;
      }

      /// <summary>
      /// Moves the cursor.
```

```
      /// </summary>
      /// <param name="delta">The number of characters to move.</param>
      internal static void Move(int delta)
      {
        int position = Console.CursorTop * Console.BufferWidth + Console.CursorLeft
+ delta;

        Console.CursorLeft = position % Console.BufferWidth;
        Console.CursorTop = position / Console.BufferWidth;
      }

      /// <summary>
      /// Resets the cursor position.
      /// </summary>
      internal void Reset()
      {
        Console.CursorTop = this.anchorTop;
        Console.CursorLeft = this.anchorLeft;
      }

      /// <summary>
      /// Moves the cursor to a specific position.
      /// </summary>
      /// <param name="position">The new position.</param>
      internal void Place(int position)
      {
        Console.CursorLeft = (this.anchorLeft + position) % Console.BufferWidth;
        int cursorTop = this.anchorTop + (this.anchorLeft + position) /
Console.BufferWidth;
        if (cursorTop >= Console.BufferHeight)
        {
          this.anchorTop -= cursorTop - Console.BufferHeight + 1;
          cursorTop = Console.BufferHeight - 1;
        }

        Console.CursorTop = cursorTop;
      }
    } // End Cursor
  }
}
```

See Also

System.Management.Automation.Host.PSHost

System.Management.Automation.Host.PSHostUserInterface

System.Management.Automation.Host.PSHostRawUserInterface

Last updated on 05/20/2025

Runspace Samples

This section includes sample code that shows how to use different types of runspaces to run commands synchronously and asynchronously. You can use Microsoft Visual Studio to create a console application and then copy the code from the topics in this section into your host application.

In This Section

> **Note**
>
> For samples of host applications that create custom host interfaces, see Custom Host Samples.

Runspace01 Sample This sample shows how to use the System.Management.Automation.PowerShell class to run the Get-Process cmdlet synchronously and display its output in a console window.

Runspace02 Sample This sample shows how to use the System.Management.Automation.PowerShell class to run the Get-Process and Sort-Object cmdlets synchronously. The results of these commands is displayed by using a System.Windows.Forms.DataGridView control.

Runspace03 Sample This sample shows how to use the System.Management.Automation.PowerShell class to run a script synchronously, and how to handle non-terminating errors. The script receives a list of process names and then retrieves those processes. The results of the script, including any non-terminating errors that were generated when running the script, are displayed in a console window.

Runspace04 Sample This sample shows how to use the System.Management.Automation.PowerShell class to run commands, and how to catch terminating errors that are thrown when running the commands. Two commands are run, and the last command is passed a parameter argument that is not valid. As a result no objects are returned and a terminating error is thrown.

Runspace05 Sample This sample shows how to add a snap-in to a System.Management.Automation.Runspaces.InitialSessionState object so that the cmdlet of the

snap-in is available when the runspace is opened. The snap-in provides a Get-Proc cmdlet (defined by the GetProcessSample01 Sample) that is run synchronously using a System.Management.Automation.PowerShell object.

Runspace06 Sample This sample shows how to add a module to a System.Management.Automation.Runspaces.InitialSessionState object so that the module is loaded when the runspace is opened. The module provides a Get-Proc cmdlet (defined by the GetProcessSample02 Sample) that is run synchronously using a System.Management.Automation.PowerShell object.

Runspace07 Sample This sample shows how to create a runspace, and then use that runspace to run two cmdlets synchronously by using a System.Management.Automation.PowerShell object.

Runspace08 Sample This sample shows how to add commands and arguments to the pipeline of a System.Management.Automation.PowerShell object and how to run the commands synchronously.

Runspace09 Sample This sample shows how to add a script to the pipeline of a System.Management.Automation.PowerShell object and how to run the script asynchronously. Events are used to handle the output of the script.

Runspace10 Sample This sample shows how to create a default initial session state, how to add a cmdlet to the System.Management.Automation.Runspaces.InitialSessionState, how to create a runspace that uses the initial session state, and how to run the command by using a System.Management.Automation.PowerShell object.

Runspace11 Sample This shows how to use the System.Management.Automation.ProxyCommand class to create a proxy command that calls an existing cmdlet, but restricts the set of available parameters. The proxy command is then added to an initial session state that is used to create a constrained runspace. This means that the user can access the functionality of the cmdlet only through the proxy command.

See Also

Last updated on 05/20/2025

Runspace01 Sample

This sample shows how to use the System.Management.Automation.PowerShell class to run the Get-Process cmdlet synchronously. The Get-Process cmdlet returns System.Diagnostics.Process objects for each process running on the local computer. The values of the System.Diagnostics.Process.ProcessName* and System.Diagnostics.Process.HandleCount* properties are then extracted from the returned objects and displayed in a console window.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

- Creating a System.Management.Automation.PowerShell object to run a command.
- Adding a command to the pipeline of the System.Management.Automation.PowerShell object.
- Running the command synchronously.
- Using System.Management.Automation.PSObject objects to extract properties from the objects returned by the command.

Example

This sample runs the Get-Process cmdlet synchronously in the default runspace provided by Windows PowerShell.

C#

```
namespace Microsoft.Samples.PowerShell.Runspaces
{
  using System;
  using System.Management.Automation;
  using PowerShell = System.Management.Automation.PowerShell;

  /// <summary>
  /// This class contains the Main entry point for this host application.
  /// </summary>
  internal class Runspace01
  {
```

```
    /// <summary>
    /// This sample uses the PowerShell class to execute
    /// the Get-Process cmdlet synchronously. The name and
    /// handlecount are then extracted from the PSObjects
    /// returned and displayed.
    /// </summary>
    /// <param name="args">Parameter not used.</param>
    /// <remarks>
    /// This sample demonstrates the following:
    /// 1. Creating a PowerShell object to run a command.
    /// 2. Adding a command to the pipeline of the PowerShell object.
    /// 3. Running the command synchronously.
    /// 4. Using PSObject objects to extract properties from the objects
    ///    returned by the command.
    /// </remarks>
    private static void Main(string[] args)
    {
      // Create a PowerShell object. Creating this object takes care of
      // building all of the other data structures needed to run the command.
      using (PowerShell powershell = PowerShell.Create().AddCommand("Get-Process"))
      {
        Console.WriteLine("Process              HandleCount");
        Console.WriteLine("-------------------------------");

        // Invoke the command synchronously and display the
        // ProcessName and HandleCount properties of the
        // objects that are returned.
        foreach (PSObject result in powershell.Invoke())
        {
          Console.WriteLine(
                      "{0,-20} {1}",
                      result.Members["ProcessName"].Value,
                      result.Members["HandleCount"].Value);
        }
      }

      System.Console.WriteLine("Hit any key to exit...");
      System.Console.ReadKey();
    }
  }
}
```

See Also

Last updated on 05/20/2025

Runspace02 Sample

This sample shows how to use the System.Management.Automation.PowerShell class to run the Get-Process and Sort-Object cmdlets synchronously. The Get-Process cmdlet returns System.Diagnostics.Process objects for each process running on the local computer, and the `Sort-Object` sorts the objects based on their System.Diagnostics.Process.Id* property. The results of these commands is displayed by using a System.Windows.Forms.DataGridView control.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Creating a System.Management.Automation.PowerShell object to run commands.
- Adding commands to the pipeline of System.Management.Automation.PowerShell object.
- Running the commands synchronously.
- Using a System.Windows.Forms.DataGridView control to display the output of the commands in a Windows Forms application.

Example

This sample runs the Get-Process and Sort-Object cmdlets synchronously in the default runspace provided by Windows PowerShell. The output is displayed in a form using a System.Windows.Forms.DataGridView control.

C#

```
namespace Microsoft.Samples.PowerShell.Runspaces
{
  using System;
  using System.Collections;
  using System.Collections.ObjectModel;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;
```

```
using System.Windows.Forms;
using PowerShell = System.Management.Automation.PowerShell;

/// <summary>
/// This class contains the Main entry point for this host application.
/// </summary>
internal class Runspace02
{
  /// <summary>
  /// This method creates the form where the output is displayed.
  /// </summary>
  private static void CreateForm()
  {
    Form form = new Form();
    DataGridView grid = new DataGridView();
    form.Controls.Add(grid);
    grid.Dock = DockStyle.Fill;

    // Create a PowerShell object. Creating this object takes care of
    // building all of the other data structures needed to run the command.
    using (PowerShell powershell = PowerShell.Create())
    {
      powershell.AddCommand("Get-Process").AddCommand("Sort-
Object").AddArgument("ID");
      if (Runspace.DefaultRunspace == null)
      {
        Runspace.DefaultRunspace = powershell.Runspace;
      }

      Collection<PSObject> results = powershell.Invoke();

      // The generic collection needs to be re-wrapped in an ArrayList
      // for data-binding to work.
      ArrayList objects = new ArrayList();
      objects.AddRange(results);

      // The DataGridView will use the PSObjectTypeDescriptor type
      // to retrieve the properties.
      grid.DataSource = objects;
    }

    form.ShowDialog();
  }

  /// <summary>
  /// This sample uses a PowerShell object to run the Get-Process
  /// and Sort-Object cmdlets synchronously. Windows Forms and
  /// data binding are then used to display the results in a
  /// DataGridView control.
  /// </summary>
  /// <param name="args">The parameter is not used.</param>
  /// <remarks>
  /// This sample demonstrates the following:
  /// 1. Creating a PowerShell object.
  /// 2. Adding commands and arguments to the pipeline of
```

```
    ///    the PowerShell object.
    /// 3. Running the commands synchronously.
    /// 4. Using a DataGridView control to display the output
    ///    of the commands in a Windows Forms application.
    /// </remarks>
    private static void Main(string[] args)
    {
      Runspace02.CreateForm();
    }
  }
}
```

See Also

Writing a Windows PowerShell Host Application

Last updated on 05/20/2025

Runspace03 Sample

This sample shows how to use the System.Management.Automation.PowerShell class to run a script synchronously, and how to handle non-terminating errors. The script receives a list of process names and then retrieves those processes. The results of the script, including any non-terminating errors that were generated when running the script, are displayed in a console window.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Creating a System.Management.Automation.PowerShell object to run a script.
- Adding a script to the pipeline of the System.Management.Automation.PowerShell object.
- Passing input objects to the script from the calling program.
- Running the script synchronously.
- Using System.Management.Automation.PSObject objects to extract and display properties from the objects returned by the script.
- Retrieving and displaying error records that were generated when the script was run.

Example

This sample runs a script synchronously in the default runspace provided by Windows PowerShell. The output of the script and any non-terminating errors that were generated are displayed in a console window.

C#

```
namespace Microsoft.Samples.PowerShell.Runspaces
{
  using System;
  using System.Collections;
```

```
using System.Management.Automation;
using System.Management.Automation.Runspaces;
using PowerShell = System.Management.Automation.PowerShell;

/// <summary>
/// This class contains the Main entry point for this host application.
/// </summary>
internal class Runspace03
{
  /// <summary>
  /// This sample shows how to use the PowerShell class to run a
  /// script that retrieves process information for the list of
  /// process names passed to the script. It shows how to pass input
  /// objects to a script and how to retrieve error objects as well
  /// as the output objects.
  /// </summary>
  /// <param name="args">Parameter not used.</param>
  /// <remarks>
  /// This sample demonstrates the following:
  /// 1. Creating a PowerShell object to run a script.
  /// 2. Adding a script to the pipeline of the PowerShell object.
  /// 3. Passing input objects to the script from the calling program.
  /// 4. Running the script synchronously.
  /// 5. Using PSObject objects to extract and display properties from
  ///    the objects returned by the script.
  /// 6. Retrieving and displaying error records that were generated
  ///    when the script was run.
  /// </remarks>
  private static void Main(string[] args)
  {
    // Define a list of processes to look for.
    string[] processNames = new string[]
    {
      "lsass", "nosuchprocess", "services", "nosuchprocess2"
    };

    // The script to run to get these processes. Input passed
    // to the script will be available in the $input variable.
    string script = "$input | Get-Process -Name {$_}";

    // Create a PowerShell object. Creating this object takes care of
    // building all of the other data structures needed to run the script.
    using (PowerShell powershell = PowerShell.Create())
    {
      powershell.AddScript(script);

      Console.WriteLine("Process             HandleCount");
      Console.WriteLine("-------------------------------");

      // Invoke the script synchronously and display the
      // ProcessName and HandleCount properties of the
      // objects that are returned.
      foreach (PSObject result in powershell.Invoke(processNames))
      {
        Console.WriteLine(
```

```
                            "{0,-20} {1}",
                            result.Members["ProcessName"].Value,
                            result.Members["HandleCount"].Value);
        }

        // Process any error records that were generated while running
        //  the script.
        Console.WriteLine("\nThe following non-terminating errors occurred:\n");
        PSDataCollection<ErrorRecord> errors = powershell.Streams.Error;
        if (errors != null && errors.Count > 0)
        {
          foreach (ErrorRecord err in errors)
          {
            System.Console.WriteLine("    error: {0}", err.ToString());
          }
        }
      }

      System.Console.WriteLine("\nHit any key to exit...");
      System.Console.ReadKey();
    }
  }
}
```

See Also

Writing a Windows PowerShell Host Application

Last updated on 05/20/2025

Runspace04 Sample

This sample shows how to use the System.Management.Automation.PowerShell class to run commands, and how to catch terminating errors that are thrown when running the commands. Two commands are run, and the last command is passed a parameter argument that is not valid. As a result, no objects are returned and a terminating error is thrown.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Creating a System.Management.Automation.PowerShell object.
- Adding commands to the pipeline of the System.Management.Automation.PowerShell object.
- Adding parameter arguments to the pipeline.
- Invoking the commands synchronously.
- Using System.Management.Automation.PSObject objects to extract and display properties from the objects returned by the commands.
- Retrieving and displaying error records that were generated during the running of the commands.
- Catching and displaying terminating exceptions thrown by the commands.

Example

This sample runs commands synchronously in the default runspace provided by Windows PowerShell. The last command throws a terminating error because a parameter argument that is not valid is passed to the command. The terminating error is trapped and displayed.

```
C#
```

```
namespace Microsoft.Samples.PowerShell.Runspaces
{
  using System;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;
  using PowerShell = System.Management.Automation.PowerShell;

  /// <summary>
  /// This class contains the Main entry point for this host application.
  /// </summary>
  internal class Runspace04
  {
    /// <summary>
    /// This sample shows how to use a PowerShell object to run commands.
    /// The commands generate a terminating exception that the caller
    /// should catch and process.
    /// </summary>
    /// <param name="args">The parameter is not used.</param>
    /// <remarks>
    /// This sample demonstrates the following:
    /// 1. Creating a PowerShell object to run commands.
    /// 2. Adding commands to the pipeline of  the PowerShell object.
    /// 3. Passing input objects to the commands from the calling program.
    /// 4. Using PSObject objects to extract and display properties from the
    ///    objects returned by the commands.
    /// 5. Retrieving and displaying error records that were generated
    ///    while running the commands.
    /// 6. Catching and displaying terminating exceptions generated
    ///    while running the commands.
    /// </remarks>
    private static void Main(string[] args)
    {
      // Create a PowerShell object.
      using (PowerShell powershell = PowerShell.Create())
      {
        // Add the commands to the PowerShell object.
        powershell.AddCommand("Get-ChildItem").AddCommand("Select-
String").AddArgument("*");

        // Run the commands synchronously. Because of the bad regular expression,
        // no objects will be returned. Instead, an exception will be thrown.
        try
        {
          foreach (PSObject result in powershell.Invoke())
          {
            Console.WriteLine("'{0}'", result.ToString());
          }

          // Process any error records that were generated while running the
commands.
          Console.WriteLine("\nThe following non-terminating errors occurred:\n");
          PSDataCollection<ErrorRecord> errors = powershell.Streams.Error;
          if (errors != null && errors.Count > 0)
          {
```

```
            foreach (ErrorRecord err in errors)
            {
              System.Console.WriteLine("    error: {0}", err.ToString());
            }
          }
        }
        catch (RuntimeException runtimeException)
        {
          // Trap any exception generated by the commands. These exceptions
          // will all be derived from the RuntimeException exception.
          System.Console.WriteLine(
                        "Runtime exception: {0}: {1}\n{2}",
                        runtimeException.ErrorRecord.InvocationInfo.InvocationName,
                        runtimeException.Message,

runtimeException.ErrorRecord.InvocationInfo.PositionMessage);
        }
      }

      System.Console.WriteLine("\nHit any key to exit...");
      System.Console.ReadKey();
    }
  }
}
```

See Also

Writing a Windows PowerShell Host Application

Last updated on 05/20/2025

Runspace05 Sample

This sample shows how to add a snap-in to a System.Management.Automation.Runspaces.InitialSessionState object so that the cmdlet of the snap-in is available when the runspace is opened. The snap-in provides a Get-Proc cmdlet (defined by the GetProcessSample01 Sample) that is run synchronously by using a System.Management.Automation.PowerShell object.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Creating a System.Management.Automation.Runspaces.InitialSessionState object.
- Adding the snap-in to the System.Management.Automation.Runspaces.InitialSessionState object.
- Creating a System.Management.Automation.Runspaces.Runspace object that uses the System.Management.Automation.Runspaces.InitialSessionState object.
- Creating a System.Management.Automation.PowerShell object that uses the runspace.
- Adding the snap-in's Get-Proc cmdlet to the pipeline of the System.Management.Automation.PowerShell object.
- Running the command synchronously.
- Extracting properties from the System.Management.Automation.PSObject objects returned by the command.

Example

This sample creates a runspace that uses a System.Management.Automation.Runspaces.InitialSessionState object to define the elements

that are available when the runspace is opened. In this sample, a snap-in that defines a Get-Proc cmdlet is added to the initial session state.

C#

```
namespace Microsoft.Samples.PowerShell.Runspaces
{
  using System;
  using System.Collections.ObjectModel;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;
  using PowerShell = System.Management.Automation.PowerShell;

  /// <summary>
  /// This class contains the Main entry point for this host application.
  /// </summary>
  internal class Runspace05
  {
    /// <summary>
    /// This sample shows how to define an initial session state that is
    /// used when creating a runspace. The sample invokes a command from
    /// a Windows PowerShell snap-in that is present in the console file.
    /// </summary>
    /// <param name="args">The parameter is not used.</param>
    /// <remarks>
    /// This sample assumes that user has copied the GetProcessSample01.dll
    /// that is produced by the GetProcessSample01 sample to the current
    /// directory.
    /// This sample demonstrates the following:
    /// 1. Creating a default initial session state.
    /// 2. Adding a snap-in to the initial session state.
    /// 3. Creating a runspace that uses the initial session state.
    /// 4. Creating a PowerShell object that uses the runspace.
    /// 5. Adding the snap-in's Get-Proc cmdlet to the PowerShell object.
    /// 6. Using PSObject objects to extract and display properties from
    ///    the objects returned by the cmdlet.
    /// </remarks>
    private static void Main(string[] args)
    {
      // Create the default initial session state. The default initial
      // session state contains all the elements provided by Windows
      // PowerShell.
      InitialSessionState iss = InitialSessionState.CreateDefault();
      PSSnapInException warning;
      iss.ImportPSSnapIn("GetProcPSSnapIn01", out warning);

      // Create a runspace. Notice that no PSHost object is supplied to the
      // CreateRunspace method so the default host is used. See the Host
      // samples for more information on creating your own custom host.
      using (Runspace myRunSpace = RunspaceFactory.CreateRunspace(iss))
      {
        myRunSpace.Open();
```

```
        // Create a PowerShell object.
        using (PowerShell powershell = PowerShell.Create())
        {
          // Add the snap-in cmdlet and specify the runspace.
          powershell.AddCommand("GetProcPSSnapIn01\\Get-Proc");
          powershell.Runspace = myRunSpace;

          // Run the cmdlet synchronously.
          Collection<PSObject> results = powershell.Invoke();

          Console.WriteLine("Process             HandleCount");
          Console.WriteLine("--------------------------------");

          // Display the results.
          foreach (PSObject result in results)
          {
            Console.WriteLine(
                            "{0,-20} {1}",
                            result.Members["ProcessName"].Value,
                            result.Members["HandleCount"].Value);
          }
        }

        // Close the runspace to release any resources.
        myRunSpace.Close();
      }
      System.Console.WriteLine("Hit any key to exit...");
      System.Console.ReadKey();
    }
  }
}
```

See Also

Writing a Windows PowerShell Host Application

Last updated on 05/20/2025

Runspace06 Sample

This sample shows how to add a module to a System.Management.Automation.Runspaces.InitialSessionState object so that the module is loaded when the runspace is opened. The module provides a Get-Proc cmdlet (defined by the GetProcessSample02 Sample) that is run synchronously by using a System.Management.Automation.PowerShell object.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Creating a System.Management.Automation.Runspaces.InitialSessionState object.
- Adding the module to the System.Management.Automation.Runspaces.InitialSessionState object.
- Creating a System.Management.Automation.Runspaces.Runspace object that uses the System.Management.Automation.Runspaces.InitialSessionState object.
- Creating a System.Management.Automation.PowerShell object that uses the runspace.
- Adding the module's Get-Proc cmdlet to the pipeline of the System.Management.Automation.PowerShell object.
- Running the command synchronously.
- Extracting properties from the System.Management.Automation.PSObject objects returned by the command.

Example

This sample creates a runspace that uses a System.Management.Automation.Runspaces.InitialSessionState object to define the elements

that are available when the runspace is opened. In this sample, a module that defines a Get-Proc cmdlet is added to the initial session state.

C#

```
namespace Microsoft.Samples.PowerShell.Runspaces
{
  using System;
  using System.Collections.ObjectModel;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;
  using PowerShell = System.Management.Automation.PowerShell;

  /// <summary>
  /// This class contains the Main entry point for this host application.
  /// </summary>
  internal class Runspace06
  {
    /// <summary>
    /// This sample shows how to define an initial session state that is
    /// used when creating a runspace. The sample invokes a command from
    /// a binary module that is loaded by the initial session state.
    /// </summary>
    /// <param name="args">Parameter not used.</param>
    /// <remarks>
    /// This sample assumes that user has copied the GetProcessSample02.dll
    /// that is produced by the GetProcessSample02 sample to the current
    /// directory.
    /// This sample demonstrates the following:
    /// 1. Creating a default initial session state.
    /// 2. Adding a module to the initial session state.
    /// 3. Creating a runspace that uses the initial session state.
    /// 4. Creating a PowerShell object that uses the runspace.
    /// 5. Adding the module's Get-Proc cmdlet to the PowerShell object.
    /// 6. Running the command synchronously.
    /// 7. Using PSObject objects to extract and display properties from
    ///    the objects returned by the cmdlet.
    /// </remarks>
    private static void Main(string[] args)
    {
        // Create the default initial session state and add the module.
      InitialSessionState iss = InitialSessionState.CreateDefault();
      iss.ImportPSModule(new string[] { @".\GetProcessSample02.dll" });

      // Create a runspace. Notice that no PSHost object is supplied to the
      // CreateRunspace method so the default host is used. See the Host
      // samples for more information on creating your own custom host.
      using (Runspace myRunSpace = RunspaceFactory.CreateRunspace(iss))
      {
        myRunSpace.Open();

        // Create a PowerShell object.
        using (PowerShell powershell = PowerShell.Create())
```

```
        {
          // Add the cmdlet and specify the runspace.
          powershell.AddCommand(@"GetProcessSample02\Get-Proc");
          powershell.Runspace = myRunSpace;

          Collection<PSObject> results = powershell.Invoke();

          Console.WriteLine("Process             HandleCount");
          Console.WriteLine("-------------------------------");

          // Display the results.
          foreach (PSObject result in results)
          {
            Console.WriteLine(
                            "{0,-20} {1}",
                            result.Members["ProcessName"].Value,
                            result.Members["HandleCount"].Value);
          }
        }

        // Close the runspace to release any resources.
        myRunSpace.Close();
      }

      System.Console.WriteLine("Hit any key to exit...");
      System.Console.ReadKey();
    }
  }
}
```

See Also

Writing a Windows PowerShell Host Application

Last updated on 05/20/2025

Runspace07 Sample

This sample shows how to create a runspace, and then use that runspace to run two cmdlets synchronously by using a System.Management.Automation.PowerShell object.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Creating a System.Management.Automation.Runspaces.Runspace object by using the System.Management.Automation.Runspaces.RunspaceFactory class.
- Creating a System.Management.Automation.PowerShell object that uses the runspace.
- Adding cmdlets to the pipeline of the System.Management.Automation.PowerShell object.
- Running the cmdlets synchronously.
- Extracting properties from the System.Management.Automation.PSObject objects returned by the command.

Example

This sample creates a runspace that used by a System.Management.Automation.PSObject object to run the Get-Process and Measure-Object cmdlets.

C#

```
namespace Microsoft.Samples.PowerShell.Runspaces
{
  using System;
  using System.Collections.ObjectModel;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;
  using PowerShell = System.Management.Automation.PowerShell;

  /// <summary>
```

```
/// This class contains the Main entry point for this host application.
/// </summary>
internal class Runspace07
{
  /// <summary>
  /// This sample shows how to create a runspace and how to run commands
  /// using a PowerShell object. It builds a pipeline that runs the
  /// Get-Process cmdlet, which is piped to the Measure-Object
  /// cmdlet to count the number of processes running on the system.
  /// </summary>
  /// <param name="args">The parameter is not used.</param>
  /// <remarks>
  /// This sample demonstrates the following:
  /// 1. Creating a runspace using the RunspaceFactory class.
  /// 2. Creating a PowerShell object that uses the runspace.
  /// 3. Adding cmdlets to the pipeline of the PowerShell object.
  /// 4. Running the cmdlets synchronously.
  /// 5. Working with PSObject objects to extract properties
  ///    from the objects returned by the cmdlets.
  /// </remarks>
  private static void Main(string[] args)
  {
    Collection<PSObject> result;     // Will hold the result
                                     // of running the cmdlets.

    // Create a runspace. We can't use the RunspaceInvoke class
    // because we need to get at the underlying runspace to
    // explicitly add the commands. Notice that no PSHost object is
    // supplied to the CreateRunspace method so the default host is
    // used. See the Host samples for more information on creating
    // your own custom host.
    using (Runspace myRunSpace = RunspaceFactory.CreateRunspace())
    {
      myRunSpace.Open();

      // Create a PowerShell object and specify the runspace.
      PowerShell powershell = PowerShell.Create();
      powershell.Runspace = myRunSpace;

      // Use the using statement so we dispose of the PowerShell object
      // when we're done.
      using (powershell)
      {
        // Add the Get-Process cmdlet to the PowerShell object. Notice
        // we are specify the name of the cmdlet, not a script.
        powershell.AddCommand("Get-Process");

        // Add the Measure-Object cmdlet to count the number
        // of objects being returned. Commands are always added to the end
        // of the pipeline.
        powershell.AddCommand("Measure-Object");

        // Run the cmdlets synchronously and save the objects returned.
        result = powershell.Invoke();
      }
```

```
        // Even after disposing of the pipeLine, we still need to set
        // the powershell variable to null so that the garbage collector
        // can clean it up.
        powershell = null;

        // Display the results of running the commands (checking that
        // everything is ok first.
        if (result == null || result.Count != 1)
        {
          throw new InvalidOperationException(
                    "pipeline.Invoke() returned the wrong number of objects");
        }

        PSMemberInfo count = result[0].Properties["Count"];
        if (count == null)
        {
          throw new InvalidOperationException(
                    "The object returned doesn't have a 'count' property");
        }

        Console.WriteLine(
                   "Runspace07: The Get-Process cmdlet returned {0} objects",
                   count.Value);

        // Close the runspace to release any resources.
        myRunSpace.Close();
      }

      System.Console.WriteLine("Hit any key to exit...");
      System.Console.ReadKey();
    }
  }
}
```

See Also

Writing a Windows PowerShell Host Application

Last updated on 05/20/2025

Runspace08 Sample

This sample shows how to add commands and arguments to the pipeline of a System.Management.Automation.PowerShell object and how to run the commands synchronously.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Creating a System.Management.Automation.Runspaces.Runspace object by using the System.Management.Automation.Runspaces.RunspaceFactory class.
- Creating a System.Management.Automation.PowerShell object that uses the runspace.
- Adding cmdlets to the pipeline of the System.Management.Automation.PowerShell object.
- Running the cmdlets synchronously.
- Extracting properties from the System.Management.Automation.PSObject objects returned by the command.

Example

This sample runs the Get-Process and Sort-Object cmdlets by using a System.Management.Automation.PowerShell object.

C#

```
namespace Microsoft.Samples.PowerShell.Runspaces
{
  using System;
  using System.Collections.Generic;
  using System.Collections.ObjectModel;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;
```

```
  using PowerShell = System.Management.Automation.PowerShell;

  /// <summary>
  /// This class contains the Main entry point for this host application.
  /// </summary>
  internal class Runspace08
  {
    /// <summary>
    /// This sample shows how to use a PowerShell object to run commands. The
    /// PowerShell object builds a pipeline that include the Get-Process cmdlet,
    /// which is then piped to the Sort-Object cmdlet. Parameters are added to the
    /// Sort-Object cmdlet to sort the HandleCount property in descending order.
    /// </summary>
    /// <param name="args">Parameter is not used.</param>
    /// <remarks>
    /// This sample demonstrates:
    /// 1. Creating a PowerShell object
    /// 2. Adding individual commands to the PowerShell object.
    /// 3. Adding parameters to the commands.
    /// 4. Running the pipeline of the PowerShell object synchronously.
    /// 5. Working with PSObject objects to extract properties
    ///    from the objects returned by the commands.
    /// </remarks>
    private static void Main(string[] args)
    {
      Collection<PSObject> results; // Holds the result of the pipeline execution.

      // Create the PowerShell object. Notice that no runspace is specified so a
      // new default runspace is used.
      PowerShell powershell = PowerShell.Create();

      // Use the using statement so that we can dispose of the PowerShell object
      // when we are done.
      using (powershell)
      {
        // Add the Get-Process cmdlet to the pipeline of the PowerShell object.
        powershell.AddCommand("Get-Process");

        // Add the Sort-Object cmdlet and its parameters to the pipeline of
        // the PowerShell object so that we can sort the HandleCount property
        //  in descending order.
        powershell.AddCommand("Sort-
Object").AddParameter("Descending").AddParameter("Property", "HandleCount");

        // Run the commands of the pipeline synchronously.
        results = powershell.Invoke();
      }

      // Even after disposing of the PowerShell object, we still
      // need to set the powershell variable to null so that the
      // garbage collector can clean it up.
      powershell = null;

      Console.WriteLine("Process              HandleCount");
      Console.WriteLine("-------------------------------");
```

```
      // Display the results returned by the commands.
      foreach (PSObject result in results)
      {
        Console.WriteLine(
                          "{0,-20} {1}",
                          result.Members["ProcessName"].Value,
                          result.Members["HandleCount"].Value);
      }

      System.Console.WriteLine("Hit any key to exit...");
      System.Console.ReadKey();
    }
  }
}
```

See Also

Writing a Windows PowerShell Host Application

Last updated on 05/20/2025

Runspace09 Sample

This sample shows how to add a script to the pipeline of a System.Management.Automation.PowerShell object and how to run the script asynchronously. Events are used to handle the output of the script.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Creating a System.Management.Automation.PowerShell object that uses the runspace.
- Adding a script the pipeline of the System.Management.Automation.PowerShell object.
- Using the System.Management.Automation.PowerShell.BeginInvoke* method to run the pipeline asynchronously.
- Using the events of the System.Management.Automation.PowerShell object to process the output of the script.
- Using the System.Management.Automation.PowerShell.Stop* method to interrupt the invocation of the pipeline.

Example

This sample runs to run a script that generates the numbers from 1 to 10 with delays between each number. The script is run asynchronously and events are used to handle the output.

C#

```
namespace Microsoft.Samples.PowerShell.Runspaces
{
  using System;
  using System.Collections.Generic;
  using System.Collections.ObjectModel;
  using System.Diagnostics;
  using System.Management.Automation;
```

```
  using System.Management.Automation.Runspaces;
  using PowerShell = System.Management.Automation.PowerShell;

  /// <summary>
  /// This class contains the Main entry point for this host application.
  /// </summary>
  internal class Runspace09
  {
    /// <summary>
    /// This sample shows how to use a PowerShell object to run a
    /// script that generates the numbers from 1 to 10 with delays
    /// between each number. The pipeline of the PowerShell object
    /// is run asynchronously and events are used to handle the output.
    /// </summary>
    /// <param name="args">The parameter is not used.</param>
    /// <remarks>
    /// This sample demonstrates the following:
    /// 1. Creating a PowerShell object.
    /// 2. Adding a script to the pipeline of the PowerShell object.
    /// 3. Using the BeginInvoke method to run the pipeline asynchronously.
    /// 4. Using the events of the PowerShell object to process the
    ///    output of the script.
    /// 5. Using the PowerShell.Stop() method to interrupt the invocation of
    ///    the pipeline.
    /// </remarks>
    private static void Main(string[] args)
    {
      Console.WriteLine("Print the numbers from 1 to 10. Hit any key to halt
processing\n");

      using (PowerShell powershell = PowerShell.Create())
      {
        // Add a script to the PowerShell object. The script generates the
        // numbers from 1 to 10 in half second intervals.
        powershell.AddScript("1..10 | foreach {$_ ; Start-Sleep -Milli 500}");

        // Add the event handlers.  If we did not care about hooking the DataAdded
        // event, we would let BeginInvoke create the output stream for us.
        PSDataCollection<PSObject> output = new PSDataCollection<PSObject>();
        output.DataAdded += new EventHandler<DataAddedEventArgs>(Output_DataAdded);
        powershell.InvocationStateChanged += new
EventHandler<PSInvocationStateChangedEventArgs>(Powershell_InvocationStateChanged);

        // Invoke the pipeline asynchronously.
        IAsyncResult asyncResult = powershell.BeginInvoke<PSObject, PSObject>(null,
output);

        // Wait for things to happen. If the user hits a key before the
        // script has completed, then call the PowerShell Stop() method
        // to halt processing.
        Console.ReadKey();
        if (powershell.InvocationStateInfo.State != PSInvocationState.Completed)
        {
          // Stop the invocation of the pipeline.
          Console.WriteLine("\nStopping the pipeline!\n");
```

```
            powershell.Stop();

            // Wait for the Windows PowerShell state change messages to be displayed.
            System.Threading.Thread.Sleep(500);
            Console.WriteLine("\nPress a key to exit");
            Console.ReadKey();
          }
        }
      }

      /// <summary>
      /// The output data added event handler. This event is called when
      /// data is added to the output pipe. It reads the data that is
      /// available and displays it on the console.
      /// </summary>
      /// <param name="sender">The output pipe this event is associated with.</param>
      /// <param name="e">Parameter is not used.</param>
      private static void Output_DataAdded(object sender, DataAddedEventArgs e)
      {
        PSDataCollection<PSObject> myp = (PSDataCollection<PSObject>)sender;

        Collection<PSObject> results = myp.ReadAll();
        foreach (PSObject result in results)
        {
          Console.WriteLine(result.ToString());
        }
      }

      /// <summary>
      /// This event handler is called when the pipeline state is changed.
      /// If the state change is to Completed, the handler issues a message
      /// asking the user to exit the program.
      /// </summary>
      /// <param name="sender">This parameter is not used.</param>
      /// <param name="e">The PowerShell state information.</param>
      private static void Powershell_InvocationStateChanged(object sender,
PSInvocationStateChangedEventArgs e)
      {
        Console.WriteLine("PowerShell object state changed: state: {0}\n",
e.InvocationStateInfo.State);
        if (e.InvocationStateInfo.State == PSInvocationState.Completed)
        {
          Console.WriteLine("Processing completed, press a key to exit!");
        }
      }
    }
}
```

See Also

Writing a Windows PowerShell Host Application

Last updated on 05/20/2025

Runspace10 Sample

This sample shows how to create a default initial session state, how to add a cmdlet to the System.Management.Automation.Runspaces.InitialSessionState, how to create a runspace that uses the initial session state, and how to run the command by using a System.Management.Automation.PowerShell object.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Creating a System.Management.Automation.Runspaces.InitialSessionState object.
- Adding a cmdlet (defined by the Host application) to the System.Management.Automation.Runspaces.InitialSessionState object.
- Creating a System.Management.Automation.Runspaces.Runspace object that uses the object.
- Creating a System.Management.Automation.PowerShell object that uses the System.Management.Automation.Runspaces.Runspace object.
- Adding the command to the pipeline of the System.Management.Automation.PowerShell object.
- Extracting properties from the System.Management.Automation.PSObject objects returned by the command.

Example

This sample creates a runspace that uses a System.Management.Automation.Runspaces.InitialSessionState object to define the elements that are available when the runspace is opened. In this sample, the Get-Proc cmdlet (defined by the Host application) is added to the initial session state, and the cmdlet is run synchronously by using a System.Management.Automation.PowerShell object.

C#

```
namespace Microsoft.Samples.PowerShell.Runspaces
{
  using System;
  using System.Collections.Generic;
  using System.Collections.ObjectModel;
  using System.Diagnostics;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;
  using PowerShell = System.Management.Automation.PowerShell;

  #region GetProcCommand

  /// <summary>
  /// Class that implements the GetProcCommand.
  /// </summary>
  [Cmdlet(VerbsCommon.Get, "Proc")]
  public class GetProcCommand : Cmdlet
  {
    #region Cmdlet Overrides

    /// <summary>
    /// For each of the requested process names, retrieve and write
    /// the associated processes.
    /// </summary>
    protected override void ProcessRecord()
    {
      // Get the current processes.
      Process[] processes = Process.GetProcesses();

      // Write the processes to the pipeline making them available
      // to the next cmdlet. The second argument (true) tells the
      // system to enumerate the array, and send one process object
      // at a time to the pipeline.
      WriteObject(processes, true);
    }

    #endregion Overrides
  } // End GetProcCommand class.

  #endregion GetProcCommand

  /// <summary>
  /// This class contains the Main entry point for this host application.
  /// </summary>
  internal class Runspace10
  {
    /// <summary>
    /// This sample shows how to create a default initial session state, how to add
    /// add a cmdlet to the InitialSessionState object, and then how to create
    /// a Runspace object.
    /// </summary>
    /// <param name="args">Parameter is not used.</param>
```

```
    /// This sample demonstrates:
    /// 1. Creating an InitialSessionState object.
    /// 2. Adding a cmdlet to the InitialSessionState object.
    /// 3. Creating a runspace that uses the InitialSessionState object.
    /// 4. Creating a PowerShell object that uses the Runspace object.
    /// 5. Running the added command synchronously.
    /// 6. Working with PSObject objects to extract properties
    ///    from the objects returned by the pipeline.
    private static void Main(string[] args)
    {
      // Create a default InitialSessionState object. The default
      // InitialSessionState object contains all the elements provided
      // by Windows PowerShell.
      InitialSessionState iss = InitialSessionState.CreateDefault();

      // Add the Get-Proc cmdlet to the InitialSessionState object.
      SessionStateCmdletEntry ssce = new SessionStateCmdletEntry("Get-Proc",
typeof(GetProcCommand), null);
      iss.Commands.Add(ssce);

      // Create a Runspace object that uses the InitialSessionState object.
      // Notice that no PSHost object is specified, so the default host is used.
      // See the Hosting samples for information on creating your own custom host.
      using (Runspace myRunSpace = RunspaceFactory.CreateRunspace(iss))
      {
        myRunSpace.Open();

        using (PowerShell powershell = PowerShell.Create())
        {
          powershell.Runspace = myRunSpace;

          // Add the Get-Proc cmdlet to the pipeline of the PowerShell object.
          powershell.AddCommand("Get-Proc");

          Collection<PSObject> results = powershell.Invoke();

          Console.WriteLine("Process             HandleCount");
          Console.WriteLine("-------------------------------");

          // Display the output of the pipeline.
          foreach (PSObject result in results)
          {
             Console.WriteLine(
                            "{0,-20} {1}",
                            result.Members["ProcessName"].Value,
                            result.Members["HandleCount"].Value);
          }
        }

        // Close the runspace to release resources.
        myRunSpace.Close();
      }

      System.Console.WriteLine("Hit any key to exit...");
      System.Console.ReadKey();
```

```
    }
  }
}
```

See Also

Writing a Windows PowerShell Host Application

Last updated on 05/20/2025

Runspace11 Sample

This sample shows how to use the System.Management.Automation.ProxyCommand class to create a proxy command that calls an existing cmdlet, but restricts the set of available parameters. The proxy command is then added to an initial session state that is used to create a constrained runspace. This means that the user can access the functionality of the cmdlet only through the proxy command.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

This sample demonstrates the following.

- Creating a System.Management.Automation.CommandMetadata object that describes the metadata of an existing cmdlet.
- Creating a System.Management.Automation.Runspaces.InitialSessionState object.
- Modifying the cmdlet metadata to remove a parameter of the cmdlet.
- Adding the cmdlet to the System.Management.Automation.Runspaces.InitialSessionState object and making the cmdlet private.
- Creating a proxy function that calls the existing cmdlet, but exposes only a restricted set of parameters.
- Adding the proxy function to the initial session state.
- Creating a System.Management.Automation.PowerShell object that uses the System.Management.Automation.Runspaces.Runspace object.
- Calling the private cmdlet and the proxy function using a System.Management.Automation.PowerShell object to demonstrate the constrained runspace.

Example

This creates a proxy command for a private cmdlet to demonstrate a constrained runspace.

C#

```
namespace Microsoft.Samples.PowerShell.Runspaces
{
  using System;
  using System.Collections.Generic;
  using System.Diagnostics;
  using System.Management.Automation;
  using System.Management.Automation.Runspaces;
  using PowerShell = System.Management.Automation.PowerShell;

  #region GetProcCommand

  /// <summary>
  /// This class implements the Get-Proc cmdlet. It has been copied
  /// verbatim from the GetProcessSample02.cs sample.
  /// </summary>
  [Cmdlet(VerbsCommon.Get, "Proc")]
  public class GetProcCommand : Cmdlet
  {
    #region Parameters

    /// <summary>
    /// The names of the processes to act on.
    /// </summary>
    private string[] processNames;

    /// <summary>
    /// Gets or sets the list of process names on which
    /// the Get-Proc cmdlet will work.
    /// </summary>
    [Parameter(Position = 0)]
    [ValidateNotNullOrEmpty]
    public string[] Name
    {
      get { return this.processNames; }
      set { this.processNames = value; }
    }

    #endregion Parameters

    #region Cmdlet Overrides

    /// <summary>
    /// The ProcessRecord method calls the Process.GetProcesses
    /// method to retrieve the processes specified by the Name
    /// parameter. Then, the WriteObject method writes the
    /// associated processes to the pipeline.
    /// </summary>
    protected override void ProcessRecord()
    {
      // If no process names are passed to the cmdlet, get all
```

```
      // processes.
      if (this.processNames == null)
      {
        WriteObject(Process.GetProcesses(), true);
      }
      else
      {
        // If process names are passed to cmdlet, get and write
        // the associated processes.
        foreach (string name in this.processNames)
        {
          WriteObject(Process.GetProcessesByName(name), true);
        }
      } // if (processNames...
    } // ProcessRecord

    #endregion Cmdlet Overrides
  } // GetProcCommand

  #endregion GetProcCommand

  /// <summary>
  /// This class contains the Main entry point for this host application.
  /// </summary>
  internal class Runspace11
  {
    /// <summary>
    /// This shows how to use the ProxyCommand class to create a proxy
    /// command that calls an existing cmdlet, but restricts the set of
    /// available parameters. The proxy command is then added to an initial
    /// session state that is used to create a constrained runspace. This
    /// means that the user can access the cmdlet only through the proxy
    /// command.
    /// </summary>
    /// <remarks>
    /// This sample demonstrates the following:
    /// 1. Creating a CommandMetadata object that describes the metadata of an
    ///    existing cmdlet.
    /// 2. Modifying the cmdlet metadata to remove a parameter of the cmdlet.
    /// 3. Adding the cmdlet to an initial session state and making it private.
    /// 4. Creating a proxy function that calls the existing cmdlet, but exposes
    ///    only a restricted set of parameters.
    /// 6. Adding the proxy function to the initial session state.
    /// 7. Calling the private cmdlet and the proxy function to demonstrate the
    ///    constrained runspace.
    /// </remarks>
    private static void Main()
    {
      // Create a default initial session state. The default initial session state
      // includes all the elements that are provided by Windows PowerShell.
      InitialSessionState iss = InitialSessionState.CreateDefault();

      // Add the Get-Proc cmdlet to the initial session state.
      SessionStateCmdletEntry cmdletEntry = new SessionStateCmdletEntry("Get-Proc",
typeof(GetProcCommand), null);
```

```
        iss.Commands.Add(cmdletEntry);

        // Make the cmdlet private so that it is not accessible.
        cmdletEntry.Visibility = SessionStateEntryVisibility.Private;

        // Set the language mode of the initial session state to NoLanguage to
        //prevent users from using language features. Only the invocation of
        // public commands is allowed.
        iss.LanguageMode = PSLanguageMode.NoLanguage;

        // Create the proxy command using cmdlet metadata to expose the
        // Get-Proc cmdlet.
        CommandMetadata cmdletMetadata = new CommandMetadata(typeof(GetProcCommand));

        // Remove one of the parameters from the command metadata.
        cmdletMetadata.Parameters.Remove("Name");

        // Generate the body of a proxy function that calls the original cmdlet,
        // but does not have the removed parameter.
        string bodyOfProxyFunction = ProxyCommand.Create(cmdletMetadata);

        // Add the proxy function to the initial session state. The name of the proxy
        // function can be the same as the name of the cmdlet, but to clearly
        // demonstrate that the original cmdlet is not available a different name is
        // used for the proxy function.
        iss.Commands.Add(new SessionStateFunctionEntry("Get-ProcProxy",
bodyOfProxyFunction));

        // Create the constrained runspace using the initial session state.
        using (Runspace myRunspace = RunspaceFactory.CreateRunspace(iss))
        {
          myRunspace.Open();

          // Call the private cmdlet to demonstrate that it is not available.
          try
          {
            using (PowerShell powershell = PowerShell.Create())
            {
              powershell.Runspace = myRunspace;
              powershell.AddCommand("Get-Proc").AddParameter("Name", "*explore*");
              powershell.Invoke();
            }
          }
          catch (CommandNotFoundException e)
          {
            System.Console.WriteLine(
                          "Invoking 'Get-Proc' failed as expected: {0}: {1}",
                          e.GetType().FullName,
                          e.Message);
          }

          // Call the proxy function to demonstrate that the -Name parameter is
          // not available.
          try
          {
```

```
          using (PowerShell powershell = PowerShell.Create())
          {
            powershell.Runspace = myRunspace;
            powershell.AddCommand("Get-ProcProxy").AddParameter("Name", "idle");
            powershell.Invoke();
          }
        }
        catch (ParameterBindingException e)
        {
          System.Console.WriteLine(
                        "\nInvoking 'Get-ProcProxy -Name idle' failed as expected:
{0}: {1}",
                        e.GetType().FullName,
                        e.Message);
        }

        // Call the proxy function to demonstrate that it calls into the
        // private cmdlet to retrieve the processes.
        using (PowerShell powershell = PowerShell.Create())
        {
          powershell.Runspace = myRunspace;
          powershell.AddCommand("Get-ProcProxy");
          List<Process> processes = new List<Process>(powershell.Invoke<Process>
());
          System.Console.WriteLine(
                        "\nInvoking the Get-ProcProxy function called into the Get-
Proc cmdlet and returned {0} processes",
                        processes.Count);
        }

        // Close the runspace to release resources.
        myRunspace.Close();
      }

      System.Console.WriteLine("Hit any key to exit...");
      System.Console.ReadKey();
    }
  }
}
```

See Also

Writing a Windows PowerShell Host Application

Last updated on 05/20/2025

Remote Runspace Samples

This section includes sample code that shows how to create runspaces that can be used to connect to a computer by using WS-Management-based Windows PowerShell remoting. You can use Microsoft Visual Studio to create a console application and then copy the code from the topics in this section into your host application.

In This Section

> **Note**
>
> For more information about running commands on a remote computer, see Windows PowerShell Remoting.

RemoteRunspace01 Sample This sample shows how to create a remote runspace that is used to establish a remote connection.

RemoteRunspacePool01 Sample This sample shows how to construct a remote runspace pool and how to run multiple commands concurrently by using this pool.

Last updated on 05/20/2025

RemoteRunspace01 Sample

This sample shows how to create a remote runspace that is used to establish a remote connection.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

- Creating a System.Management.Automation.Runspaces.WSManConnectionInfo object.
- Setting the System.Management.Automation.Runspaces.RunspaceConnectionInfo.OperationTimeout* and System.Management.Automation.Runspaces.RunspaceConnectionInfo.OpenTimeout* properties of the System.Management.Automation.Runspaces.WSManConnectionInfo object.
- Creating a remote runspace that uses the System.Management.Automation.Runspaces.WSManConnectionInfo object to establish the remote connection.
- Closing the remote runspace to release the remote connection.

Example

This sample defines a remote connection and then uses that connection information to establish a remote connection.

C#

```
namespace Microsoft.Samples.PowerShell.Runspaces
{
  using System;
  using System.Management.Automation;             // Windows PowerShell namespace.
  using System.Management.Automation.Runspaces;   // Windows PowerShell namespace.

  /// <summary>
  /// This class contains the Main entry point for the application.
  /// </summary>
```

```
  internal class RemoteRunspace01
  {
    /// <summary>
    /// This sample shows how to use a WSManConnectionInfo object to set
    /// various timeouts and how to establish a remote connection.
    /// </summary>
    /// <param name="args">This parameter is not used.</param>
    public static void Main(string[] args)
    {
      // Create a WSManConnectionInfo object using the default constructor
      // to connect to the "localHost". The WSManConnectionInfo object can
      // also specify connections to remote computers.
      WSManConnectionInfo connectionInfo = new WSManConnectionInfo();

      // Set the OperationTimeout property. The OperationTimeout is used to tell
      // Windows PowerShell how long to wait (in milliseconds) before timing out
      // for any operation. This includes sending input data to the remote
computer,
      // receiving output data from the remote computer, and more. The user can
      // change this timeout depending on whether the connection is to a computer
      // in the data center or across a slow WAN.
      connectionInfo.OperationTimeout = 4 * 60 * 1000; // 4 minutes.

      // Set the OpenTimeout property. OpenTimeout is used to tell Windows
PowerShell
      // how long to wait (in milliseconds) before timing out while establishing a
      // remote connection. The user can change this timeout depending on whether
the
      // connection is to a computer in the data center or across a slow WAN.
      connectionInfo.OpenTimeout = 1 * 60 * 1000; // 1 minute.

      // Create a remote runspace using the connection information.
      using (Runspace remoteRunspace =
RunspaceFactory.CreateRunspace(connectionInfo))
      {
        // Establish the connection by calling the Open() method to open the
runspace.
        // The OpenTimeout value set previously will be applied while establishing
        // the connection. Establishing a remote connection involves sending and
        // receiving some data, so the OperationTimeout will also play a role in
this process.
        remoteRunspace.Open();

        // Add the code to run commands in the remote runspace here. The
        // OperationTimeout value set previously will play a role here because
        // running commands involves sending and receiving data.

        // Close the connection. Call the Close() method to close the remote
        // runspace. The Dispose() method (called by using primitive) will call
        // the Close() method if it is not already called.
        remoteRunspace.Close();
      }
    }
  }
}
```

Last updated on 05/20/2025

RemoteRunspacePool01 Sample

This sample shows how to construct a remote runspace pool and how to run multiple commands concurrently by using this pool.

Requirements

This sample requires Windows PowerShell 2.0.

Demonstrates

- Creating a System.Management.Automation.Runspaces.WSManConnectionInfo object.
- Setting the System.Management.Automation.Runspaces.RunspaceConnectionInfo.OperationTimeout* and System.Management.Automation.Runspaces.RunspaceConnectionInfo.OpenTimeout* properties of the System.Management.Automation.Runspaces.WSManConnectionInfo object.
- Creating a remote runspace that uses the System.Management.Automation.Runspaces.WSManConnectionInfo object to establish the remote connection.
- Running the Get-Process and Get-Service cmdlets concurrently by using the remote runspace pool.
- Closing the remote runspace pool to release the remote connection.

Example

This sample shows how to construct a remote runspace pool and how to run multiple commands concurrently by using this pool.

C#

```
namespace Samples
{
  using System;
  using System.Management.Automation;            // Windows PowerShell namespace.
  using System.Management.Automation.Runspaces;  // Windows PowerShell namespace.
```

```
  /// <summary>
  /// This class contains the Main entry point for the application.
  /// </summary>
  internal class RemoteRunspacePool01
  {
    /// <summary>
    /// This sample shows how to construct a remote RunspacePool and how to
    /// concurrently run the Get-Process and Get-Service commands using the
    /// runspaces of the pool.
    /// </summary>
    /// <param name="args">Parameter is not used.</param>
    public static void Main(string[] args)
    {
      // Create a WSManConnectionInfo object using the default constructor to
      // connect to the "localhost". The WSManConnectionInfo object can also
      // specify connections to remote computers.
      WSManConnectionInfo connectionInfo = new WSManConnectionInfo();

      // Create a remote runspace pool that uses the WSManConnectionInfo object.
      // The minimum runspaces value of 1 specifies that Windows PowerShell will
      // keep at least 1 runspace open. The maximum runspaces value of 2 specifies
      // that Windows PowerShell will allows 2 runspaces to be opened at the
      // same time so that two commands can be run concurrently.
      using (RunspacePool remoteRunspacePool =
             RunspaceFactory.CreateRunspacePool(1, 2, connectionInfo))
      {
        // Call the Open() method to open the runspace pool and establish
        // the connection.
        remoteRunspacePool.Open();

        // Call the Create() method to create a pipeline, call the
AddCommand(string)
        // method to add the "Get-Process" command, and then call the BeginInvoke()
        // method to run the command asynchronously using a runspace of the pool.
        PowerShell gpsCommand = PowerShell.Create().AddCommand("Get-Process");
        gpsCommand.RunspacePool = remoteRunspacePool;
        IAsyncResult gpsCommandAsyncResult = gpsCommand.BeginInvoke();

        // The previous call does not block the current thread because it is
        // running asynchronously. Because the remote runspace pool can open two
        // runspaces, the second command can be run.
        PowerShell getServiceCommand = PowerShell.Create().AddCommand("Get-
Service");
        getServiceCommand.RunspacePool = remoteRunspacePool;
        IAsyncResult getServiceCommandAsyncResult =
getServiceCommand.BeginInvoke();

        // When you are ready to handle the output, wait for the command to
complete
        // before extracting results. A call to the EndInvoke() method will block
and return
        // the output.
        PSDataCollection<PSObject> gpsCommandOutput =
gpsCommand.EndInvoke(gpsCommandAsyncResult);
```

```
        // Process the output from the first command.
        if ((gpsCommandOutput != null) && (gpsCommandOutput.Count > 0))
        {
          Console.WriteLine("The first output from running Get-Process command: ");
          Console.WriteLine(
                            "Process Name: {0} Process Id: {1}",
                            gpsCommandOutput[0].Properties["ProcessName"].Value,
                            gpsCommandOutput[0].Properties["Id"].Value);
          Console.WriteLine();
        }

        // Now process the output from the second command. As discussed previously,
wait
        // for the command to complete before extracting the results.
        PSDataCollection<PSObject> getServiceCommandOutput =
getServiceCommand.EndInvoke(
                                    getServiceCommandAsyncResult);

        // Process the output of the second command as needed.
        if ((getServiceCommandOutput != null) && (getServiceCommandOutput.Count >
0))
        {
          Console.WriteLine("The first output from running Get-Service command: ");
          Console.WriteLine(
                            "Service Name: {0} Description: {1} State: {2}",

getServiceCommandOutput[0].Properties["ServiceName"].Value,

getServiceCommandOutput[0].Properties["DisplayName"].Value,
                            getServiceCommandOutput[0].Properties["Status"].Value);
        }

        // Once done with running all the commands, close the remote runspace pool.
        // The Dispose() method (called by using primitive) will call Close(), if
it
        // is not already called.
        remoteRunspacePool.Close();
      } // End Using.
    } // End Main.
  } // End RemoteRunspacePool01 class
}
```

See Also

Last updated on 05/20/2025

Formatting File Overview

The display format for the objects that are returned by commands (cmdlets, functions, and scripts) are defined by using formatting files (`format.ps1xml`). Several of these files are provided by PowerShell to define the display format for those objects returned by PowerShell-provided commands, such as the System.Diagnostics.Process object returned by the `Get-Process` cmdlet. However, you can also create your own custom formatting files to overwrite the default display formats or you can write a custom formatting file to define the display of objects returned by your own commands.

> **Important**
>
> Formatting files do not determine the elements of an object that are returned to the pipeline. When an object is returned to the pipeline, all members of that object are available even if some are not displayed.

PowerShell uses the data in these formatting files to determine what is displayed and how the displayed data is formatted. The displayed data can include the properties of an object or the value of a script. Scripts are used if you want to display some value that is not available directly from the properties of an object, such as adding the value of two properties of an object and then displaying the sum as a piece of data. Formatting of the displayed data is done by defining views for the objects that you want to display. You can define a single view for each object, you can define a single view for multiple objects, or you can define multiple views for the same object. There is no limit to the number of views that you can define.

Common Features of Formatting Files

Each formatting file can define the following components that can be shared across all the views defined by the file:

- Default configuration setting, such as whether the data displayed in the rows of tables will be displayed on the next line if the data is longer than the width of the column. For more information about these settings, see Wrap Element for TableRowEntry.
- Sets of objects that can be displayed by any of the views of the formatting file. For more information about these sets (referred to as **selection sets**), see Defining Sets of Objects.

- Common controls that can be used by all the views of the formatting file. Controls give you finer control on how data is displayed. For more information about controls, see Defining Custom Controls.

Formatting Views

Formatting views can display objects in a table format, list format, wide format, and custom format. For the most part, each formatting definition is described by a set of XML tags that describe the view. Each view contains the name of the view, the objects that use the view, and the elements of the view, such as the column and row information for a table view.

Table View

Lists the properties of an object or a script block value in one or more columns. Each column represents a single property of the object or a script value. You can define a table view that displays all the properties of an object, a subset of the properties of an object, or a combination of properties and script values. Each row of the table represents a returned object. Creating a table view is very similar to when you pipe an object to the `Format-Table` cmdlet. For more information about this view, see Table View.

List View

Lists the properties of an object or a script value in a single column. Each row of the list displays an optional label or the property name followed by the value of the property or script. Creating a list view is very similar to piping an object to the `Format-List` cmdlet. For more information about this view, see List View.

Wide View

Lists a single property of an object or a script value in one or more columns. There is no label or header for this view. Creating a wide view is very similar to piping an object to the `Format-Wide` cmdlet. For more information about this view, see Wide View.

Custom View

Displays a customizable view of object properties or script values that does not adhere to the rigid structure of table views, list views, or wide views. You can define a stand-alone custom view, or you can define a custom view that is used by another view, such as a table view or list

view. Creating a custom view is very similar to piping an object to the `Format-Custom` cmdlet. For more information about this view, see Custom View.

Components of a View

The following XML examples show the basic XML components of a view. The individual XML elements vary depending on which view you want to create, but the basic components of the views are all the same.

To start with, each view has a `Name` element that specifies a user friendly name that is used to reference the view. a `ViewSelectedBy` element that defines which .NET objects are displayed by the view, and a **control** element that defines the view.

XML

```
<ViewDefinitions>
  <View>
    <Name>NameOfView</Name>
    <ViewSelectedBy>...</ViewSelectedBy>
    <TableControl>...</TableControl>
  </View>
  <View>
    <Name>NameOfView</Name>
    <ViewSelectedBy>...</ViewSelectedBy>
    <ListControl>...</ListControl>
  <View>
  <View>
    <Name>NameOfView</Name>
    <ViewSelectedBy>...</ViewSelectedBy>
    <WideControl>...</WideControl>
  <View>
  <View>
    <Name>NameOfView</Name>
    <ViewSelectedBy>...</ViewSelectedBy>
    <CustomControl>...</CustomControl>
  </View>
</ViewDefinitions>
```

Within the control element, you can define one or more **entry** elements. If you use multiple definitions, you must specify which .NET objects use each definition. Typically only one entry, with only one definition, is needed for each control.

XML

```
<ListControl>
  <ListEntries>
```

```
    <ListEntry>
      <EntrySelectedBy>...</EntrySelectedBy>
      <ListItems>...</ListItems>
    </ListEntry>
    <ListEntry>
        <EntrySelectedBy>...</EntrySelectedBy>
      <ListItems>...</ListItems>
    </ListEntry>
    <ListEntry>
        <EntrySelectedBy>...</EntrySelectedBy>
      <ListItems>...</ListItems>
    </ListEntry>
  </ListEntries>
</ListControl>
```

Within each entry element of a view, you specify the **item** elements that define the .NET properties or scripts that are displayed by that view.

XML

```
<ListItems>
  <ListItem>...</ListItem>
  <ListItem>...</ListItem>
  <ListItem>...</ListItem>
</ListItems>
```

As shown in the preceding examples, the formatting file can contain multiple views, a view can contain multiple definitions, and each definition can contain multiple items.

Example of a Table View

The following example shows the XML tags used to define a table view that contains two columns. The ViewDefinitions element is the container element for all the views defined in the formatting file. The View element defines the specific table, list, wide, or custom view. Within each View element, the Name element specifies the name of the view, the ViewSelectedBy element defines the objects that use the view, and the different control elements (such as the `TableControl` element shown in the following example) define the type of the view.

XML

```
<ViewDefinitions>
  <View>
    <Name>Name of View</Name>
    <ViewSelectedBy>
      <TypeName>Object to display using this view</TypeName>
```

```
      <TypeName>Object to display using this view</TypeName>
    </ViewSelectedBy>
    <TableControl>
      <TableHeaders>
        <TableColumnHeader>
          <Width></Width>
        </TableColumnHeader>
        <TableColumnHeader>
          <Width></Width>
        </TableColumnHeader>
      </TableHeaders>
      <TableRowEntries>
        <TableRowEntry>
          <TableColumnItems>
            <TableColumnItem>
              <PropertyName>Header for column 1</PropertyName>
            </TableColumnItem>
            <TableColumnItem>
              <PropertyName>Header for column 2</PropertyName>
            </TableColumnItem>
          </TableColumnItems>
        </TableRowEntry>
      </TableRowEntries>
    </TableControl>
  </View>
</ViewDefinitions>
```

See Also

Creating a List View

Creating a Table View

Creating a Wide View

Creating Custom Controls

Writing a PowerShell Formatting and Types File

Last updated on 05/20/2025

Formatting File Concepts

The topics in this section provide information that you might need to know when creating your own formatting files, such as the different types of views that you can define and the special components of those views.

In This Section

Creating a Table View Provides an example of a displayed table view and the XML elements used to define the view.

Creating a List View Provides an example of a displayed list view and the XML elements used to define the view.

Creating a Wide View Provides an example of a displayed wide view and the XML elements used to define the view.

Creating Custom Controls Provides an example of a custom control.

Defining Selection Sets Provides information, an example, and describes the XML elements used to create a selection set.

Defining Conditions for Displaying Data When defining what data is displayed by a view or a control, you can specify a condition that must exist for the data to be displayed.

Formatting Displayed Data You can specify how the individual data points in your List, Table, or Wide view are displayed.

PowerShell Formatting Files Lists the available formatting files provided by PowerShell.

Last updated on 05/20/2025

Creating a Table View

A table view displays data in one or more columns. Each row in the table represents a .NET object, and each column of the table represents a property of the object or a script value. You can define a table view that displays all the properties of an object or a subset of the properties of an object.

A Table View Display

The following example shows how Windows PowerShell displays the System.ServiceProcess.ServiceController object that is returned by the Get-Service cmdlet. For this object, Windows PowerShell has defined a table view that displays the `Status` property, the `Name` property (this property is an alias property for the `ServiceName` property), and the `DisplayName` property. Each row in the table represents an object returned by the cmdlet.

Output

```
Status   Name               DisplayName
------   ----               -----------
Stopped  AJRouter           AllJoyn Router Service
Stopped  ALG                Application Layer Gateway Service
Stopped  AppIDSvc           Application Identity
Running  Appinfo            Application Information
```

Defining the Table View

The following XML shows the table view schema for displaying the System.ServiceProcess.ServiceController object. You must specify each property that you want displayed in the table view.

XML

```
<View>
  <Name>service</Name>
  <ViewSelectedBy>
    <TypeName>System.ServiceProcess.ServiceController</TypeName>
  </ViewSelectedBy>
  <TableControl>
    <TableHeaders>
      <TableColumnHeader>
        <Width>8</Width>
      </TableColumnHeader>
```

```
      <TableColumnHeader>
        <Width>18</Width>
      </TableColumnHeader>
      <TableColumnHeader>
        <Width>38</Width>
      </TableColumnHeader>
    </TableHeaders>
    <TableRowEntries>
      <TableRowEntry>
        <TableColumnItems>
          <TableColumnItem>
           <PropertyName>Status</PropertyName>
          </TableColumnItem>
          <TableColumnItem>
            <PropertyName>Name</PropertyName>
          </TableColumnItem>
          <TableColumnItem>
            <PropertyName>DisplayName</PropertyName>
          </TableColumnItem>
        </TableColumnItems>
      </TableRowEntry>
    </TableRowEntries>
  </TableControl>
</View>
```

The following XML elements are used to define a list view:

- The View element is the parent element of the table view. (This is the same parent element for the list, wide, and custom control views.)
- The Name element specifies the name of the view. This element is required for all views.
- The ViewSelectedBy element defines the objects that use the view. This element is required.
- The GroupBy element (not shown in this example) defines when a new group of objects is displayed. A new group is started whenever the value of a specific property or script changes. This element is optional.
- The Controls element (not shown in this example) defines the custom controls that are defined by the table view. Controls give you a way to further specify how the data is displayed. This element is optional. A view can define its own custom controls, or it can use common controls that can be used by any view in the formatting file. For more information about custom controls, see Creating Custom Controls.
- The HideTableHeaders element (not show in this example) specifies that the table will not show any labels at the top of the table. This element is optional.

- The TableControl element that defines the header and row information of the table. Similar to all other views, a table view can display the values of object properties or values generated by scripts.

Defining Column Headers

1. The TableHeaders element and its child elements define what is displayed at the top of the table.

2. The TableColumnHeader element defines what is displayed at the top of a column of the table. Specify these elements in the order that you want the headers displayed.

   There is no limit to the number of these element that you can use, but the number of TableColumnHeader elements in your table view must equal the number of TableRowEntry elements that you use.

3. The Label element specifies the text that is displayed. This element is optional.

4. The Width element specifies the width (in characters) of the column. This element is optional.

5. The Alignment element specifies how the label is displayed. The label can be aligned to the left, to the right, or centered. This element is optional.

Defining the Table Rows

Table views can provide one or more definitions that specify what data is displayed in the rows of the table by using the child elements of the TableRowEntries element. Notice that you can specify multiple definitions for the rows of the table, but the headers for the rows remains the same, regardless of what row definition is used. Typically, a table will have only one definition.

In the following example, the view provides a single definition that displays the values of several properties of the System.Diagnostics.Process object. A table view can display the value of a property or the value of a script (not shown in the example) in its rows.

XML

```
<TableRowEntries>
      <TableRowEntry>
        <TableColumnItems>
          <TableColumnItem>
           <PropertyName>Status</PropertyName>
          </TableColumnItem>
```

```
        <TableColumnItem>
          <PropertyName>Name</PropertyName>
        </TableColumnItem>
        <TableColumnItem>
          <PropertyName>DisplayName</PropertyName>
        </TableColumnItem>
      </TableColumnItems>
    </TableRowEntry>
  </TableRowEntries>
```

The following XML elements can be used to provide definitions for a row:

- The TableRowEntries element and its child elements define what is displayed in the rows of the table.
- The TableRowEntry element provides a definition of the row. At least one TableRowEntry is required; however, there is no maximum limit to the number of elements that you can add. In most cases, a view will have only one definition.
- The EntrySelectedBy element specifies the objects that are displayed by a specific definition. This element is optional and is needed only when you define multiple TableRowEntry elements that display different objects.
- The Wrap element specifies that text that exceeds the column width is displayed on the next line. By default, text that exceeds the column width is truncated.
- The TableColumnItems element defines the properties or scripts whose values are displayed in the row.
- The TableColumnItem element defines the property or script whose value is displayed in the column of the row. A TableColumnItem element is required for each column of the row. The first entry is displayed in first column, the second entry in the second column, and so on.
- The PropertyName element specifies the property whose value is displayed in the row. You must specify either a property or a script, but you cannot specify both.
- The ScriptBlock element specifies the script whose value is displayed in the row. You must specify either a script or a property, but you cannot specify both.
- The FormatString element specifies a format pattern that defines how the property or script value is displayed. This element is optional.

- The Alignment element specifies how the value of the property or script is displayed. The value can be aligned to the left, to the right, or centered. This element is optional.

Defining the Objects That Use the Table View

There are two ways to define which .NET objects use the table view. You can use the ViewSelectedBy element to define the objects that can be displayed by all the definitions of the view, or you can use the EntrySelectedBy element to define which objects are displayed by a specific definition of the view. In most cases, a view has only one definition, so objects are typically defined by the ViewSelectedBy element.

The following example shows how to define the objects that are displayed by the table view using the ViewSelectedBy and TypeName elements. There is no limit to the number of TypeName elements that you can specify, and their order is not significant.

XML

```
<View>
  <Name>System.ServiceProcess.ServiceController</Name>
  <ViewSelectedBy>
    <TypeName>System.ServiceProcess.ServiceController</TypeName>
  </ViewSelectedBy>
  <TableControl>...</TableControl>
</View>
```

The following XML elements can be used to specify the objects that are used by the table view:

- The ViewSelectedBy element defines which objects are displayed by the list view.
- The TypeName element specifies the .NET object that is displayed by the view. The fully qualified .NET type name is required. You must specify at least one type or selection set for the view, but there is no maximum number of elements that can be specified.

The following example uses the ViewSelectedBy and SelectionSetName elements. Use selection sets where you have a related set of objects that are displayed using multiple views, such as when you define a list view and a table view for the same objects. For more information about how to create a selection set, see Defining Selection Sets.

XML

```
<View>
  <Name>System.ServiceProcess.ServiceController</Name>
  <ViewSelectedBy>
    <SelectionSetName>.NET Type Set</SelectionSetName>
```

```
  </ViewSelectedBy>
  <TableControl>...</TableControl>
</View>
```

The following XML elements can be used to specify the objects that are used by the list view:

- The ViewSelectedBy element defines which objects are displayed by the list view.
- The SelectionSetName element specifies a set of objects that can be displayed by the view. You must specify at least one selection set or type for the view, but there is no maximum number of elements that can be specified.

The following example shows how to define the objects displayed by a specific definition of the table view using the EntrySelectedBy element. Using this element, you can specify the .NET type name of the object, a selection set of objects, or a selection condition that specifies when the definition is used. For more information about how to create a selection conditions, see Defining Conditions for Displaying Data.

> **Note**
>
> When creating multiple definitions of the table view you cannot specify different column headers. You can specify only what is displayed in the rows of the table, such as what objects are displayed.

XML

```
<TableRowEntry>
  <EntrySelectedBy>
    <TypeName>.NET Type</TypeName>
  </EntrySelectedBy>
</TableRowEntry>
```

The following XML elements can be used to specify the objects that are used by a specific definition of the list view:

- The EntrySelectedBy element defines which objects are displayed by the definition.
- The TypeName element specifies the .NET object that is displayed by the definition. When using this element, the fully qualified .NET type name is required. You must specify at least one type, selection set, or selection condition for the definition, but there is no maximum number of elements that can be specified.

- The SelectionSetName element (not shown) specifies a set of objects that can be displayed by this definition. You must specify at least one type, selection set, or selection condition for the definition, but there is no maximum number of elements that can be specified.

- The SelectionCondition element (not shown) specifies a condition that must exist for this definition to be used. You must specify at least one type, selection set, or selection condition for the definition, but there is no maximum number of elements that can be specified. For more information about defining selection conditions, see Defining Conditions for Displaying Data.

Using Format Strings

Formatting strings can be added to a view to further define how the data is displayed. The following example shows how to define a formatting string for the value of the `StartTime` property.

XML

```
<TableColumnItem>
  <PropertyName>StartTime</PropertyName>
  <FormatString>{0:MMM} {0:DD} {0:HH}:{0:MM}</FormatString>
</TableColumnItem>
```

The following XML elements can be used to specify a format pattern:

- The TableColumnItem element defines the property or script whose value is displayed in the column of the row. A TableColumnItem element is required for each column of the row. The first entry is displayed in first column, the second entry in the second column, and so on.

- The PropertyName element specifies the property whose value is displayed in the row. You must specify either a property or a script, but you cannot specify both.

- The FormatString element specifies a format pattern that defines how the property or script value is displayed.

In the following example, the `ToString` method is called to format the value of the script. Scripts can call any method of an object. Therefore, if an object has a method, such as `ToString`, that has formatting parameters, the script can call that method to format the output value of the script.

XML

```
<ListItem>
  <ScriptBlock>
    [string]::Format("{0,-10} {1,-8}", $_.LastWriteTime.ToString("d"),
$_.LastWriteTime.ToString("t"))
  </ScriptBlock>
</ListItem>
```

The following XML element can be used to calling the `ToString` method:

- The TableColumnItem element defines the property or script whose value is displayed in the column of the row. A TableColumnItem element is required for each column of the row. The first entry is displayed in first column, the second entry in the second column, and so on.
- The ScriptBlock element specifies the script whose value is displayed in the row. You must specify either a script or a property, but you cannot specify both.

See Also

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Creating a List View

A list view displays data in a single column (in sequential order). The data displayed in the list can be the value of a .NET property or the value of a script.

A List View Display

The following output shows how Windows PowerShell displays the properties of System.ServiceProcess.ServiceController objects that are returned by the Get-Service cmdlet. In this example, three objects were returned, with each object separated from the preceding object by a blank line.

PowerShell

```
Get-Service | Format-List
```

Output

```
Name                : AEADIFilters
DisplayName         : Andrea ADI Filters Service
Status              : Running
DependentServices   : {}
ServicesDependedOn  : {}
CanPauseAndContinue : False
CanShutdown         : False
CanStop             : True
ServiceType         : Win32OwnProcess

Name                : AeLookupSvc
DisplayName         : Application Experience
Status              : Running
DependentServices   : {}
ServicesDependedOn  : {}
CanPauseAndContinue : False
CanShutdown         : False
CanStop             : True
ServiceType         : Win32ShareProcess

Name                : AgereModemAudio
DisplayName         : Agere Modem Call Progress Audio
Status              : Running
DependentServices   : {}
ServicesDependedOn  : {}
CanPauseAndContinue : False
CanShutdown         : False
CanStop             : True
```

```
ServiceType         : Win32OwnProcess
...
```

Defining the List View

The following XML shows the list view schema for displaying several properties of the System.ServiceProcess.ServiceController object. You must specify each property that you want displayed in the list view.

XML

```xml
<View>
  <Name>System.ServiceProcess.ServiceController</Name>
  <ViewSelectedBy>
    <TypeName>System.ServiceProcess.ServiceController</TypeName>
  </ViewSelectedBy>
  <ListControl>
    <ListEntries>
      <ListEntry>
        <ListItems>
          <ListItem>
            <PropertyName>Name</PropertyName>
          </ListItem>
          <ListItem>
            <PropertyName>DisplayName</PropertyName>
          </ListItem>
          <ListItem>
            <PropertyName>Status</PropertyName>
          </ListItem>
          <ListItem>
            <PropertyName>ServiceType</PropertyName>
          </ListItem>
        </ListItems>
      </ListEntry>
    </ListEntries>
  </ListControl>
</View>
```

The following XML elements are used to define a list view:

- The View element is the parent element of the list view. (This is the same parent element for the table, wide, and custom control views.)
- The Name element specifies the name of the view. This element is required for all views.
- The ViewSelectedBy element defines the objects that use the view. This element is required.

- The GroupBy element defines when a new group of objects is displayed. A new group is started whenever the value of a specific property or script changes. This element is optional.
- The Controls element defines the custom controls that are defined by the list view. Controls give you a way to further specify how the data is displayed. This element is optional. A view can define its own custom controls, or it can use common controls that can be used by any view in the formatting file. For more information about custom controls, see Creating Custom Controls.
- The ListControl element defines what is displayed in the view and how it is formatted. Similar to all other views, a list view can display the values of object properties or values generated by script.

For an example of a complete formatting file that defines a simple list view, see List View (Basic).

Providing Definitions for Your List View

List views can provide one or more definitions by using the child elements of the ListControl element. Typically, a view will have only one definition. In the following example, the view provides a single definition that displays several properties of the System.Diagnostics.Process object. A list view can display the value of a property or the value of a script (not shown in the example).

XML

```
<ListControl>
    <ListEntries>
      <ListEntry>
        <ListItems>
          <ListItem>
            <PropertyName>Name</PropertyName>
          </ListItem>
          <ListItem>
            <PropertyName>DisplayName</PropertyName>
          </ListItem>
          <ListItem>
            <PropertyName>Status</PropertyName>
          </ListItem>
          <ListItem>
            <PropertyName>ServiceType</PropertyName>
          </ListItem>
        </ListItems>
      </ListEntry>
```

```
    </ListEntries>
  </ListControl>
```

The following XML elements can be used to provide definitions for a list view:

- The ListControl element and its child elements define what is displayed in the view.
- The ListEntries element provides the definitions of the view. In most cases, a view will have only one definition. This element is required.
- The ListEntry element provides a definition of the view. At least one ListEntry is required; however, there is no maximum limit to the number of elements that you can add. In most cases, a view will have only one definition.
- The EntrySelectedBy element specifies the objects that are displayed by a specific definition. This element is optional and is needed only when you define multiple ListEntry elements that display different objects.
- The ListItems element specifies the properties and scripts whose values are displayed in the rows of the list view.
- The ListItem element specifies a property or script whose value is displayed in a row of the list view. A list view must specify at least one property or script. There is no maximum limit to the number of rows that can be specified.
- The PropertyName element specifies the property whose value is displayed in the row. You must specify either a property or a script, but you cannot specify both.
- The ScriptBlock element specifies the script whose value is displayed in the row. You must specify either a script or a property, but you cannot specify both.
- The Label element specifies the label that is displayed to the left of the property or script value in the row. This element is optional. If a label is not specified, the name of the property or the script is displayed. For a complete example, see List View (Labels).
- The ItemSelectionCondition element specifies a condition that must exist for the row to be displayed. For more information about adding conditions to the list view, see Defining Conditions for Displaying Data. This element is optional.
- The FormatString element specifies a pattern that is used to display the value of the property or script. This element is optional.

For an example of a complete formatting file that defines a simple list view, see List View (Basic).

Defining the Objects That Use the List View

There are two ways to define which .NET objects use the list view. You can use the ViewSelectedBy element to define the objects that can be displayed by all the definitions of the view, or you can use the EntrySelectedBy element to define which objects are displayed by a specific definition of the view. In most cases, a view has only one definition, so objects are typically defined by the ViewSelectedBy element.

The following example shows how to define the objects that are displayed by the list view using the ViewSelectedBy and TypeName elements. There is no limit to the number of TypeName elements that you can specify, and their order is not significant.

XML

```
<View>
  <Name>System.ServiceProcess.ServiceController</Name>
  <ViewSelectedBy>
    <TypeName>System.Diagnostics.Process</TypeName>
  </ViewSelectedBy>
  <ListControl>...</ListControl>
</View>
```

The following XML elements can be used to specify the objects that are used by the list view:

- The ViewSelectedBy element defines which objects are displayed by the list view.
- The TypeName element specifies the .NET object that is displayed by the view. The fully qualified .NET type name is required. You must specify at least one type or selection set for the view, but there is no maximum number of elements that can be specified.

For an example of a complete formatting file, see List View (Basic).

The following example uses the ViewSelectedBy and SelectionSetName elements. Use selection sets where you have a related set of objects that are displayed using multiple views, such as when you define a list view and a table view for the same objects. For more information about how to create a selection set, see Defining Selection Sets.

XML

```
<View>
  <Name>System.ServiceProcess.ServiceController</Name>
  <ViewSelectedBy>
    <SelectionSetName>.NET Type Set</SelectionSetName>
  </ViewSelectedBy>
  <ListControl>...</ListControl>
</View>
```

The following XML elements can be used to specify the objects that are used by the list view:

- The ViewSelectedBy element defines which objects are displayed by the list view.
- The SelectionSetName element specifies a set of objects that can be displayed by the view. You must specify at least one selection set or type for the view, but there is no maximum number of elements that can be specified.

The following example shows how to define the objects displayed by a specific definition of the list view using the EntrySelectedBy element. Using this element, you can specify the .NET type name of the object, a selection set of objects, or a selection condition that specifies when the definition is used. For more information about how to create a selection conditions, see Defining Conditions for Displaying Data.

XML

```
<ListEntry>
  <EntrySelectedBy>
    <TypeName>.NET Type</TypeName>
  </EntrySelectedBy>
</ListEntry>
```

The following XML elements can be used to specify the objects that are used by a specific definition of the list view:

- The EntrySelectedBy element defines which objects are displayed by the definition.
- The TypeName element specifies the .NET object that is displayed by the definition. When using this element, the fully qualified .NET type name is required. You must specify at least one type, selection set, or selection condition for the definition, but there is no maximum number of elements that can be specified.
- The SelectionSetName element (not shown) specifies a set of objects that can be displayed by this definition. You must specify at least one type, selection set, or selection condition for the definition, but there is no maximum number of elements that can be specified.

- The SelectionCondition element (not shown) specifies a condition that must exist for this definition to be used. You must specify at least one type, selection set, or selection condition for the definition, but there is no maximum number of elements that can be specified. For more information about defining selection conditions, see Defining Conditions for Displaying Data.

Displaying Groups of Objects in a List View

You can separate the objects that are displayed by the list view into groups. This does not mean that you define a group, only that Windows PowerShell starts a new group whenever the value of a specific property or script changes. In the following example, a new group is started whenever the value of the System.ServiceProcess.ServiceController.ServiceType property changes.

XML

```
<GroupBy>
  <Label>Service Type</Label>
  <PropertyName>ServiceType</PropertyName>
</GroupBy>
```

The following XML elements are used to define when a group is started:

- The GroupBy element defines the property or script that starts the new group and defines how the group is displayed.
- The PropertyName element specifies the property that starts a new group whenever its value changes. You must specify a property or script to start the group, but you cannot specify both.
- The ScriptBlock element specifies the script that starts a new group whenever its value changes. You must specify a script or property to start the group, but you cannot specify both.
- The Label element defines a label that is displayed at the beginning of each group. In addition to the text specified by this element, Windows PowerShell displays the value that triggered the new group and adds a blank line before and after the label. This element is optional.
- The CustomControl element defines a control that is used to display the data. This element is optional.

- The CustomControlName element specifies a common or view control that is used to display the data. This element is optional.

For an example of a complete formatting file that defines groups, see List View (GroupBy).

Using Format Strings

Formatting strings can be added to a view to further define how the data is displayed. The following example shows how to define a formatting string for the value of the `StartTime` property.

XML

```
<ListItem>
  <PropertyName>StartTime</PropertyName>
  <FormatString>{0:MMM} {0:DD} {0:HH}:{0:MM}</FormatString>
</ListItem>
```

The following XML elements can be used to specify a format pattern:

- The ListItem element specifies the data that is displayed by the view.
- The PropertyName element specifies the property whose value is displayed by the view. You must specify either a property or a script, but you cannot specify both.
- The FormatString element specifies a format pattern that defines how the property or script value is displayed in the view.
- The ScriptBlock element (not shown) specifies the script whose value is displayed by the view. You must specify either a script or a property, but you cannot specify both.

In the following example, the `ToString` method is called to format the value of the script. Scripts can call any method of an object. Therefore, if an object has a method, such as `ToString`, that has formatting parameters, the script can call that method to format the output value of the script.

XML

```
<ListItem>
  <ScriptBlock>
    [string]::Format("{0,-10} {1,-8}", $_.LastWriteTime.ToString("d"),
$_.LastWriteTime.ToString("t"))
  </ScriptBlock>
</ListItem>
```

The following XML element can be used to calling the `ToString` method:

- The ListItem element specifies the data that is displayed by the view.
- The ScriptBlock element (not shown) specifies the script whose value is displayed by the view. You must specify either a script or a property, but you cannot specify both.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

Creating a Wide View

A wide view displays a single value for each object that's displayed. The displayed value can be the value of a .NET object property or the value of a script. By default, there is no label or header for this view.

A Wide View Display

The following example shows how Windows PowerShell displays the System.Diagnostics.Process object that's returned by the Get-Process cmdlet when its output is piped to the Format-Wide cmdlet. (By default, the Get-Process cmdlet returns a table view.) In this example, the two columns are used to display the name of the process for each returned object. The name of the object's property isn't displayed, only the value of the property.

PowerShell

```
Get-Process | Format-Wide
```

Output

```
AEADISRV                    agrsmsvc
Ati2evxx                    Ati2evxx
audiodg                     CCC
CcmExec                     communicator
Crypserv                    csrss
csrss                       DevDtct2
DM1Service                  dpupdchk
dwm                         DxStudio
EXCEL                       explorer
GoogleToolbarNotifier       GrooveMonitor
hpqwmiex                    hpservice
Idle                        InoRpc
InoRT                       InoTask
ipoint                      lsass
lsm                         MOM
MSASCui                     notepad
...                         ...
```

Defining the Wide View

The following XML shows the wide view schema for the System.Diagnostics.Process object.

XML

```
<View>
  <Name>process</Name>
  <ViewSelectedBy>
    <TypeName>System.Diagnostics.Process</TypeName>
  </ViewSelectedBy>
  <GroupBy>...</GroupBy>
  <Controls>...</Controls>
  <WideControl>
    <WideEntries>
      <WideEntry>
        <WideItem>
          <PropertyName>ProcessName</PropertyName>
        </WideItem>
      </WideEntry>
    </WideEntries>
  </WideControl>
</View>
```

The following XML elements are used to define a wide view:

- The View element is the parent element of the wide view. (This is the same parent element for the table, list, and custom control views.)
- The Name element specifies the name of the view. This element is required for all views.
- The ViewSelectedBy element defines the objects that use the view. This element is required.
- The GroupBy element defines when a new group of objects is displayed. A new group is started whenever the value of a specific property or script changes. This element is optional.
- The Controls elements defines the custom controls that are defined by the wide view. Controls give you a way to further specify how the data is displayed. This element is optional. A view can define its own custom controls, or it can use common controls that can be used by any view in the formatting file. For more information about custom controls, see Creating Custom Controls.
- The WideControl element and its child elements define what's displayed in the view. In the preceding example, the view is designed to display the System.Diagnostics.Process.ProcessName property.

For an example of a complete formatting file that defines a simple wide view, see Wide View (Basic).

Providing Definitions for Your Wide View

Wide views can provide one or more definitions by using the child elements of the WideControl element. Typically, a view will have only one definition. In the following example, the view provides a single definition that displays the System.Diagnostics.Process.ProcessName property. A wide view can display the value of a property or the value of a script (not shown in the example).

XML

```
<WideControl>
  <AutoSize/>
  <ColumnNumber></ColumnNumber>
  <WideEntries>
    <WideEntry>
      <WideItem>
        <PropertyName>ProcessName</PropertyName>
      </WideItem>
    </WideEntry>
  </WideEntries>
</WideControl>
```

The following XML elements can be used to provide definitions for a wide view:

- The WideControl element and its child elements define what's displayed in the view.
- The AutoSize element specifies whether the column size and the number of columns are adjusted based on the size of the data. This element is optional.
- The ColumnNumber element specifies the number of columns displayed in the wide view. This element is optional.
- The WideEntries element provides the definitions of the view. In most cases, a view will have only one definition. This element is required.
- The WideEntry element provides a definition of the view. At least one WideEntry is required; however, there is no maximum limit to the number of elements that you can add. In most cases, a view will have only one definition.
- The EntrySelectedBy element specifies the objects that are displayed by a specific definition. This element is optional and is needed only when you define multiple WideEntry elements that display different objects.
- The WideItem element specifies the data that's displayed by the view. In contrast to other types of views, a wide control can display only one item.
- The PropertyName element specifies the property whose value is displayed by the view. You must specify either a property or a script, but you can't specify both.
- The ScriptBlock element specifies the script whose value is displayed by the view. You must specify either a script or a property, but you can't specify both.

- The FormatString element specifies a pattern that's used to display the data. This element is optional.

For an example of a complete formatting file that defines a wide view definition, see Wide View (Basic).

Defining the Objects That Use the Wide View

There are two ways to define which .NET objects use the wide view. You can use the ViewSelectedBy element to define the objects that can be displayed by all the definitions of the view, or you can use the EntrySelectedBy element to define which objects are displayed by a specific definition of the view. In most cases, a view has only one definition, so objects are typically defined by the ViewSelectedBy element.

The following example shows how to define the objects that are displayed by the wide view using the ViewSelectedBy and TypeName elements. There is no limit to the number of TypeName elements that you can specify, and their order isn't significant.

XML

```
<View>
  <Name>System.ServiceProcess.ServiceController</Name>
  <ViewSelectedBy>
    <TypeName>System.Diagnostics.Process</TypeName>
  </ViewSelectedBy>
  <WideControl>...</WideControl>
</View>
```

The following XML elements can be used to specify the objects that are used by the wide view:

- The ViewSelectedBy element defines which objects are displayed by the wide view.
- The TypeName element specifies the .NET that's displayed by the view. The fully qualified .NET type name is required. You must specify at least one type or selection set for the view, but there is no maximum number of elements that can be specified.

For an example of a complete formatting file, see Wide View (Basic).

The following example uses the ViewSelectedBy and SelectionSetName elements. Use selection sets where you have a related set of objects that are displayed using multiple views, such as when you define a wide view and a table view for the same objects. For more information about how to create a selection set, see Defining Selection Sets.

XML

```
<View>
  <Name>System.ServiceProcess.ServiceController</Name>
  <ViewSelectedBy>
    <SelectionSetName>.NET Type Set</SelectionSetName>
  </ViewSelectedBy>
  <WideControl>...</WideControl>
</View>
```

The following XML elements can be used to specify the objects that are used by the wide view:

- The ViewSelectedBy element defines which objects are displayed by the wide view.
- The SelectionSetName element specifies a set of objects that can be displayed by the view. You must specify at least one selection set or type for the view, but there is no maximum number of elements that can be specified.

The following example shows how to define the objects displayed by a specific definition of the wide view using the EntrySelectedBy element. Using this element, you can specify the .NET type name of the object, a selection set of objects, or a selection condition that specifies when the definition is used. For more information about how to create a selection conditions, see Defining Conditions for Displaying Data.

XML

```
<WideEntry>
  <EntrySelectedBy>
    <TypeName>.NET Type</TypeName>
  </EntrySelectedBy>
</WideEntry>
```

The following XML elements can be used to specify the objects that are used by a specific definition of the wide view:

- The EntrySelectedBy element defines which objects are displayed by the definition.
- The TypeName element specifies the .NET that's displayed by the definition. When using this element the fully qualified .NET type name is required. You must specify at least one type, selection set, or selection condition for the definition, but there is no maximum number of elements that can be specified.
- The SelectionSetName element (not shown) specifies a set of objects that can be displayed by this definition. You must specify at least one type, selection set, or selection condition for the definition, but there is no maximum number of elements that can be specified.

- The SelectionCondition element (not shown) specifies a condition that must exist for this definition to be used. You must specify at least one type, selection set, or selection condition for the definition, but there is no maximum number of elements that can be specified. For more information about defining selection conditions, see Defining Conditions for Displaying Data.

Displaying Groups of objects in a Wide View

You can separate the objects that are displayed by the wide view into groups. This doesn't mean that you define a group, only that Windows PowerShell starts a new group whenever the value of a specific property or script changes. In the following example, a new group is started whenever the value of the System.ServiceProcess.ServiceController.ServiceType property changes.

XML

```
<GroupBy>
  <Label>Service Type</Label>
  <PropertyName>ServiceType</PropertyName>
</GroupBy>
```

The following XML elements are used to define when a group is started:

- The GroupBy element defines the property or script that starts the new group and defines how the group is displayed.
- The PropertyName element specifies the property that starts a new group whenever its value changes. You must specify a property or script to start the group, but you can't specify both.
- The ScriptBlock element specifies the script that starts a new group whenever its value changes. You must specify a script or property to start the group, but you can't specify both.
- The Label element defines a label that's displayed at the beginning of each group. In addition to the text specified by this element, Windows PowerShell displays the value that triggered the new group and adds a blank line before and after the label. This element is optional.
- The CustomControl element defines a control that's used to display the data. This element is optional.
- The CustomControlName element specifies a common or view control that's used to display the data. This element is optional.

For an example of a complete formatting file that defines groups, see Wide View (GroupBy).

Using Format Strings

Formatting strings can be added to a wide view to further define how the data is displayed. The following example shows how to define a formatting string for the value of the `StartTime` property.

XML

```
<WideItem>
  <PropertyName>StartTime</PropertyName>
  <FormatString>{0:MMM} {0:DD} {0:HH}:{0:MM}</FormatString>
</WideItem>
```

The following XML elements can be used to specify a format pattern:

- The WideItem element specifies the data that's displayed by the view.
- The PropertyName element specifies the property whose value is displayed by the view. You must specify either a property or a script, but you can't specify both.
- The FormatString element specifies a format pattern that defines how the property or script value is displayed in the view
- The ScriptBlock element (not shown) specifies the script whose value is displayed by the view. You must specify either a script or a property, but you can't specify both.

In the following example, the `ToString` method is called to format the value of the script. Scripts can call any method of an object. Therefore, if an object has a method, such as `ToString`, that has formatting parameters, the script can call that method to format the output value of the script.

XML

```
<WideItem>
  <ScriptBlock>
    [string]::Format("{0,-10} {1,-8}", $_.LastWriteTime.ToString("d"),
$_.LastWriteTime.ToString("t"))
  </ScriptBlock>
</WideItem>
```

The following XML element can be used to calling the `ToString` method:

- The WideItem element specifies the data that's displayed by the view.

- The ScriptBlock element (not shown) specifies the script whose value is displayed by the view. You must specify either a script or a property, but you can't specify both.

See Also

- Wide View (Basic)
- Wide View (GroupBy)
- Writing a PowerShell Formatting File

Last updated on 05/20/2025

Creating Custom Controls

Custom controls are the most flexible components of a formatting file. Unlike table, list, and wide views that define a formal structure of data, such as a table of data, custom controls allow you to define how an individual piece of data is displayed. You can define a common set of custom controls that are available to all the views of the formatting file, you can define custom controls that are available to a specific view, or you can define a set of controls that are available to a group of objects.

Custom Control Example

The following example shows a custom control that is defined in the Certificates.Format.ps1xml file. This custom control is used to separate the System.Management.Automation.Signature objects displayed in a table view.

XML

```
<Controls>
  <Control>
    <Name>SignatureTypes-GroupingFormat</Name>
    <CustomControl>
      <CustomEntries>
        <CustomEntry>
          <CustomItem>
            <Frame>
              <LeftIndent>4</LeftIndent>
              <CustomItem>
                <Text AssemblyName="System.Management.Automation"
BaseName="FileSystemProviderStrings"
                  ResourceId="DirectoryDisplayGrouping"/>
              <ExpressionBinding>
                <ScriptBlock>Split-Path $_.Path</ScriptBlock>
              </ExpressionBinding>
              <NewLine/>
            </CustomItem>
          </Frame>
        </CustomItem>
      </CustomEntry>
    </CustomEntries>
  </CustomControl>
</Control>
</Controls>
```

See Also

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Loading and Exporting Formatting Data

Once you've created your formatting file, you need to update the format data of the session by loading your files into the current session. PowerShell loads a predefined set of formats. Once the format data of the current session is updated, PowerShell uses that data to display the .NET objects associated with the views defined in the loaded formats. There's no limit to the number of formats that you can load into the current session. You can also export the format data in the current session back to a formatting file.

Loading format data

Formatting files can be loaded into the current session using the following methods:

- You can import the formatting file into the current session from the command line. Use the Update-FormatData cmdlet as described in the following procedure.
- You can create a module manifest that references your formatting file. Modules allow you to package your formatting files for distribution. Use the New-ModuleManifest cmdlet to create the manifest, and the Import-Module cmdlet to load the module into the current session. For more information about modules, see Writing a Windows PowerShell Module.
- You can create a snap-in that references your formatting file. Use the System.Management.Automation.PSSnapIn.Formats to reference your formatting files. However, best practice recommendation is to use modules to package cmdlets and associated formatting and types files.
- If you're invoking commands programmatically, you can add formatting files to the initial session state of the runspace where the commands are run. For more information, see the System.Management.Automation.Runspaces.SessionStateFormatEntry class.

When a formatting file is loaded, it's added to an internal list that PowerShell uses to choose the view used when displaying objects in the host. You can prepend your formatting file to the beginning of the list, or you can append it to the end of the list.

Knowing where your formatting file is added to this list is important.

- If you're loading a formatting file that defines the only view for an object, you can use any of the methods described previously.

- If you're loading a formatting file that defines a view for an object that has an existing view defined, it must be added to the beginning of the list. You must use the Update-FormatData cmdlet and prepend your file to the beginning of the list.

Storing Your Formatting File

You can store formatting files anywhere on disk. However, it's recommended that you store them in the same folder as your profile script.

Use the following command to determine the location of your profile script.

PowerShell

```
Split-Path -Path $PROFILE -Parent
```

Loading a format file

1. Store your formatting file to disk.
2. Run the Update-FormatData cmdlet using one of the following commands.

   If you're changing how an object is displayed, use the following command to add your formatting file to the front of the list.

   PowerShell

   ```
   Update-FormatData -PrependPath PathToFormattingFile
   ```

   Use the following command to add your formatting file to the end of the list.

   PowerShell

   ```
   Update-FormatData -AppendPath PathToFormattingFile
   ```

   > ⓘ Note
   >
   > Once format data has been loaded in a session it can't be removed. You must open a new session without the format data loaded.

Exporting format data

PowerShell includes format definitions for many .NET types. You can use the Get-FormatData cmdlet to view the format data that's loaded in the current session. You can export the format data for a type to a file using the Export-FormatData cmdlet.

The following commands export the format data for the `System.Guid` type to a file named `System.Guid.format.ps1xml` in the current directory.

PowerShell

```powershell
Get-FormatData System.Guid | Export-FormatData -Path ./System.Guid.format.ps1xml
Get-Content ./System.Guid.format.ps1xml
```

Output

```
<?xml version="1.0" encoding="utf-8"?>
<Configuration>
  <ViewDefinitions>
    <View>
      <Name>System.Guid</Name>
      <ViewSelectedBy>
        <TypeName>System.Guid</TypeName>
      </ViewSelectedBy>
      <TableControl>
        <TableHeaders />
        <TableRowEntries>
          <TableRowEntry>
            <TableColumnItems>
              <TableColumnItem>
                <PropertyName>Guid</PropertyName>
              </TableColumnItem>
            </TableColumnItems>
          </TableRowEntry>
        </TableRowEntries>
      </TableControl>
    </View>
  </ViewDefinitions>
</Configuration>
```

You can edit the exported file create a custom format definition for that type.

Last updated on 05/20/2025

Defining Selection Sets

When creating multiple views and controls, you can define sets of objects that are referred to as selection sets. A selection set enables you to define the objects one time, without having to define them repeatedly for each view or control. Typically, selection sets are used when you have a set of related .NET objects. For example, The `FileSystem` formatting file (FileSystem.format.ps1xml) defines a selection set of the file system types that several views use.

Where Selection Sets are Defined and Referenced

You define selection sets as part of the common data that can be used by all the views and controls defined in the formatting file. The following example shows how to define three selection sets.

XML

```
<Configuration>
  <SelectionSets>
    <SelectionSet>...</SelectionSet>
    <SelectionSet>...</SelectionSet>
    <SelectionSet>...</SelectionSet>
  </SelectionSets>
</Configuration>
```

You can reference a selection sets in the following ways:

- Each view has a `ViewSelectedBy` element that defines which objects are displayed by using the view. The `ViewSelectedBy` element has a `SelectionSetName` child element that specifies the selection set that all the definitions of the view use. There is no restriction on the number of selection sets that you can reference from a view.

- In each definition of a view or control, the `EntrySelectedBy` element defines which objects are displayed by using that definition. Typically a view or control has only one definition so the objects are defined by the `ViewSelectedBy` element. The `EntrySelectedBy` element of the definition has a `SelectionSetName` child element that specifies the selection set. If you specify the selection set for a definition, you cannot specify any of the other child elements of the `EntrySelectedBy` element.

- In each definition of a view or control, the `SelectionCondition` element can be used to specify a condition for when the definition is used. The `SelectionCondition` element has a `SelectionSetName` child element that specifies the selection set that triggers the condition. The condition is triggered when any of the objects defined in the selection set are displayed. For more information about how to set these conditions, see Defining Conditions for when Data is Displayed.

Selection Set Example

The following example shows a selection set that is taken directly from the `FileSystem` formatting file provided by Windows PowerShell. For more information about other Windows PowerShell formatting files, see Windows PowerShell Formatting Files.

XML

```
<SelectionSets>
  <SelectionSet>
    <Name>FileSystemTypes</Name>
    <Types>
     <TypeName>System.IO.DirectoryInfo</TypeName>
     <TypeName>System.IO.FileInfo</TypeName>
     <TypeName>Deserialized.System.IO.DirectoryInfo</TypeName>
     <TypeName>Deserialized.System.IO.FileInfo</TypeName>
    </Types>
  </SelectionSet>
</SelectionSets>
```

The previous selection set is referenced in the `ViewSelectedBy` element of a table view.

XML

```
<ViewDefinitions>
  <View>
    <Name>Files</Name>
    <ViewSelectedBy>
      <SelectionSetName>FileSystemTypes</SelectionSetName>
    </ViewSelectedBy>
    <TableControl>...</TableControl>
  </View>
</ViewDefinitions>
```

XML Elements

There is no limit to the number of selection sets that you can define. The following XML elements are used to create a selection set.

- The SelectionSets element defines the sets of .NET objects that are referenced by the views and controls of the formatting file.
- The SelectionSet element defines a single set of .NET objects.
- The Name element specifies the name that is used to reference the selection set.
- The Types element specifies the .NET types of the objects of the selection set. (Within formatting files, objects are specified by their .NET type.)

The following XML elements are used to specify a selection set.

- The following element specifies the selection set to use in all the definitions of the view:
  - SelectionSetName Element for ViewSelectedBy (Format)
  - SelectionSetName Element for EntrySelectedBy for GroupBy (Format)
- The following elements specify the selection set used by a single view definition:
  - SelectionSetName Element for EntrySelectedBy for ListControl (Format)
  - SelectionSetName Element for EntrySelectedBy for TableControl (Format)
  - SelectionSetName Element for EntrySelectedBy for WideControl (Format)
  - SelectionSetName Element for EntrySelectedBy for CustomControl for View (Format)
- The following elements specify the selection set used by common and view control definitions:
  - SelectionSetName Element for EntrySelectedBy for Controls for View (Format)
  - SelectionSetName Element for EntrySelectedBy for Controls for Configuration (Format)
- The following elements specify the selection set used when you define which object to expand:
  - SelectionSetName Element for EntrySelectedBy for EnumerableExpansion (Format)
- The following elements specify the selection set used by selection conditions.

- SelectionSetName Element for SelectionCondition for Controls for Configuration (Format)
- SelectionSetName Element for SelectionCondition for Controls for View (Format)
- SelectionSetName Element for SelectionCondition for CustomControl for View (Format)
- SelectionSetName Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion (Format)
- SelectionSetName Element for SelectionCondition for EntrySelectedBy for ListEntry (Format)
- SelectionSetName Element for SelectionCondition for EntrySelectedBy for TableControl (Format)
- SelectionSetName Element for SelectionCondition for EntrySelectedBy for WideEntry (Format)
- SelectionSetName Element for SelectionCondition for GroupBy (Format)

See Also

SelectionSets

SelectionSet

Name

Types

PowerShell Formatting Files

Defining Conditions for when Data is Displayed

Writing a PowerShell Formatting and Types File

Last updated on 05/20/2025

Defining Conditions for Displaying Data

When defining what data is displayed by a view or a control, you can specify a condition that must exist for the data to be displayed. The condition can be triggered by a specific property, or when a script or property value evaluates to `true`. When the selection condition is met, the definition of the view or control is used.

Specifying a Selection Condition for a Definition

When creating a definition for a view or control, the `EntrySelectedBy` element is used to specify which objects will use the definition or what condition must exist for the definition to be used. The condition is specified by the `SelectionCondition` element.

In the following example, a selection condition is specified for a definition of a table view. In this example, the definition is used only when the specified script is evaluated to `true`.

XML

```
<TableRowEntry>
  <EntrySelectedBy>
    <SelectionCondition>
      <ScriptBlock>ScriptToEvaluate</ScriptBlock>
    </SelectionCondition>
  </EntrySelectedBy>
  <TableColumnItems>
  </TableColumnItems>
</TableRowEntry>
```

There is no limit to the number of selection conditions that you can specify for a definition of a view or control. The only requirements are the following:

- The selection condition must specify one property name or script to trigger the condition, but cannot specify both.
- The selection condition can specify any number of .NET types or selection sets, but cannot specify both.

Specifying a Selection Condition for an Item

You can also specify when an item of a list view or control is used by including the `ItemSelectionCondition` element in the item definition. In the following example, a selection condition is specified for an item of a list view. In this example, the item is used only when the script is evaluated to `true`.

XML

```
<ListItem>
  <ItemSelectionCondition>
    <ScriptBlock>ScriptToEvaluate</ScriptBlock>
  </ItemSelectionCondition>
</ListItem>
```

You can specify only one selection condition for an item. And the condition must specify one property name or script to trigger the condition, but cannot specify both.

XML Elements

The following XML elements are used to create a selection condition.

- The following elements specify selection conditions for view definitions:
  - SelectionCondition Element for EntrySelectedBy for TableControl (Format)
  - SelectionCondition Element for EntrySelectedBy for ListControl (Format)
  - SelectionCondition Element for EntrySelectedBy for WideControl (Format)
  - SelectionCondition Element for EntrySelectedBy for CustomControl (Format)
- The following elements specify selection conditions for common and view control definitions:
  - SelectionCondition Element for EntrySelectedBy for Controls for Configuration (Format)
  - SelectionCondition Element for EntrySelectedBy for Controls for View (Format)
- The following element specifies the selection condition for expanding collection objects:
  - SelectionCondition Element for EntrySelectedBy for EnumerableExpansion (Format)
- The following element specifies the selection condition for displaying a new group of data:

    - SelectionCondition Element for EntrySelectedBy for GroupBy (Format)

- The following element specifies an item selection condition for a list view:
    - ItemSelectionCondition Element for ListItem for ListControl (Format)

- The following elements specify an item selection condition for controls:
    - ItemSelectionCondition Element for ExpressionBinding for Controls for Configuration (Format)
    - ItemSelectionCondition Element for ExpressionBinding for Controls for View (Format)
    - ItemSelectionCondition Element for ExpressionBinding for CustomControl (Format)

See Also

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Formatting Displayed Data

You can specify how the individual data points in your List, Table, or Wide view are displayed. You can use the `FormatString` element when defining the items of your view, or you can use the `ScriptBlock` element to call the `FormatString` method on the data.

Using the FormatString Element

In the following example the value of the **TotalProcessorTime** property of the System.Diagnostics.Process object is formatted using the FormatString element. the **TotalProcessorTime** property

XML

```
<TableColumnItem>
  <PropertyName>TotalProcessorTime</PropertyName>
  <FormatString>{0:MMM}{0:dd}{0:HH}:{0:mm}</FormatString>
</TableColumnItem>
```

Last updated on 05/20/2025

Windows PowerShell Formatting Files

Windows PowerShell provides several formatting files (.format.ps1xml) that are located in the installation directory (`$PSHOME`). Each of these files defines the default display for a specific set of .NET objects. These files should never be changed. However, you can use them as a reference for creating your own custom formatting files.

`Certificate.Format.ps1xml` Defines the display of objects in the Certificate store such as x.509 certificates and certificate stores.

`DotNetTypes.Format.ps1xml` Defines the display of miscellaneous .NET objects such as CultureInfo, FileVersionInfo, and EventLogEntry objects.

`FileSystem.Format.ps1xml` Defines the display of file system objects such as file and directory objects.

`Help.Format.ps1xml` Defines the different views used by the Get-Help cmdlet, such as the detailed, full, parameters, and example views.

`PowerShellCore.Format.ps1xml` Defines the display of the objects generated by Windows PowerShell core cmdlets, such as the objects returned by the Get-Member and Get-History cmdlets.

`PowerShellTrace.Format.ps1xml` Defines the display of trace objects such as those generated by the Trace-Command cmdlet.

`Registry.Format.ps1xml` Defines the display of registry objects such as key and entry objects.

See Also

Writing a Windows PowerShell Cmdlet

Last updated on 05/20/2025

How to Create a Formatting File (Format.ps1xml)

This topic describes how to create a formatting file (`Format.ps1xml`).

> **Note**
>
> You can also create a formatting file by making a copy of one of the files provided by Windows PowerShell. To protect the users of your `Format.ps1xml` file, sign the file using a digital signature. For more information, see about_Signing.

Create a Format.ps1xml file

1. Open a new text file using a text editor such as Visual Studio Code.
2. Copy the following lines into the formatting file.

   XML

   ```xml
   <?xml version="1.0" encoding="utf-8"?>
   <Configuration>
   <ViewDefinitions>
   </ViewDefinitions>
   </Configuration>
   ```

   - The `<Configuration></Configuration>` tags define the root `Configuration` node. All additional XML tags will be enclosed within this node.
   - The `<ViewDefinitions></ViewDefinitions>` tags define the `ViewDefinitions` node. All views are defined within this node.
3. Save the file to a folder of your choice. If you are writing a module, save the file to a subfolder of the module folder. Use the following name format when you save the file: `MyFile.Format.ps1xml`. Formatting files must use the `.ps1xml` extension.

   You are now ready to add views to the formatting file. There is no limit to the number of views that can be defined in a formatting file. You can add a single view for each object, multiple views for the same object, or a single view that is used by multiple objects.

See also

- Formatting File Overview
- Formatting File Concepts

Last updated on 01/02/2026

Wide View (Basic)

This example shows how to implement a basic wide view that displays the System.ServiceProcess.ServiceController objects returned by the `Get-Service` cmdlet. For more information about the components of a wide view, see Creating a Wide View.

Load this formatting file

1. Copy the XML from the Example section of this topic into a text file.
2. Save the text file. Be sure to add the `format.ps1xml` extension to the file to identify it as a formatting file.
3. Open Windows PowerShell, and run the following command to load the formatting file into the current session: `Update-FormatData -PrependPath <PathToFormattingFile>`.

   > ⚠ **Warning**
   >
   > This formatting file defines the display of an object that is already defined by a Windows PowerShell formatting file. You must use the **PrependPath** parameter when you run the cmdlet, and you cannot load this formatting file as a module.

Demonstrates

This formatting file demonstrates the following XML elements:

- The Name element for the view.
- The ViewSelectedBy element that defines what objects are displayed by the view.
- The WideItem element that defines what property is displayed by the view.

Example

The following XML defines a wide view that displays the value of the System.ServiceProcess.ServiceController.ServiceName property.

XML

```
<?xml version="1.0" encoding="utf-8" ?>

<Configuration>
  <ViewDefinitions>
    <View>
      <Name>ServiceWideView</Name>
      <ViewSelectedBy>
        <TypeName>System.ServiceProcess.ServiceController</TypeName>
      </ViewSelectedBy>
      <WideControl>
        <WideEntries>
          <WideEntry>
            <WideItem>
              <PropertyName>ServiceName</PropertyName>
            </WideItem>
          </WideEntry>
        </WideEntries>
      </WideControl>
    </View>
  </ViewDefinitions>
</Configuration>
```

The following example shows how Windows PowerShell displays the System.ServiceProcess.ServiceController objects after this format file is loaded.

PowerShell

```
Get-Service f*
```

Output

```
Fax                 FCSAM
fdPHost             FDResPub
FontCache           FontCache3.0.0.0
FSysAgent           FwcAgent
```

See Also

Examples of Formatting Files

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Wide View (GroupBy)

This example shows how to implement a wide view that displays groups of System.ServiceProcess.ServiceController objects returned by the `Get-Service` cmdlet. For more information about the components of a wide view, see Creating a Wide View.

Load this formatting file

1. Copy the XML from the Example section of this topic into a text file.
2. Save the text file. Be sure to add the `format.ps1xml` extension to the file to identify it as a formatting file.
3. Open Windows PowerShell, and run the following command to load the formatting file into the current session: `Update-FormatData -PrependPath <Path to file>`.

   > ⚠ **Warning**
   >
   > This formatting file defines the display of an object that is already defined by a Windows PowerShell formatting files. You must use the **PrependPath** parameter when you run the cmdlet, and you cannot load this formatting file as a module.

Demonstrates

This formatting file demonstrates the following XML elements:

- The Name element for the view.
- The ViewSelectedBy element that defines what objects are displayed by the view.
- The GroupBy element that defines when a new group is displayed.
- The WideItem element that defines what property is displayed by the view.

Example

The following XML defines a wide view that displays groups of objects. Each new group is started when the value of the System.ServiceProcess.ServiceController.ServiceType property

changes.

XML

```
<?xml version="1.0" encoding="utf-8" ?>

<Configuration>
  <ViewDefinitions>
    <View>
      <Name>ServiceWideView</Name>
      <ViewSelectedBy>
        <TypeName>System.ServiceProcess.ServiceController</TypeName>
      </ViewSelectedBy>
      <GroupBy>
        <Label>Service Type</Label>
        <PropertyName>ServiceType</PropertyName>
      </GroupBy>
      <WideControl>
        <WideEntries>
          <WideEntry>
            <WideItem>
              <PropertyName>ServiceName</PropertyName>
            </WideItem>
          </WideEntry>
        </WideEntries>
      </WideControl>
    </View>
  </ViewDefinitions>
</Configuration>
```

The following example shows how Windows PowerShell displays the System.ServiceProcess.ServiceController objects after this format file is loaded.

PowerShell

```
Get-Service f*
```

Output

```
   Service Type: Win32OwnProcess

Fax                           FCSAM

   Service Type: Win32ShareProcess

fdPHost                       FDResPub
FontCache

   Service Type: Win32OwnProcess
```

```
FontCache3.0.0.0                FSysAgent
FwcAgent
```

See Also

Examples of Formatting Files

Writing a PowerShell Formatting File

Last updated on 05/20/2025

List View (Basic)

This example shows how to implement a basic list view that displays the System.ServiceProcess.ServiceController objects returned by the Get-Service cmdlet. For more information about the components of a list view, see Creating a List View.

Load this formatting file

1. Copy the XML from the Example section of this topic into a text file.
2. Save the text file. Be sure to add the `format.ps1xml` extension to the file to identify it as a formatting file.
3. Open Windows PowerShell, and run the following command to load the formatting file into the current session: `Update-FormatData -PrependPath PathToFormattingFile`.

> ⚠ **Warning**
>
> This formatting file defines the display of an object that is already defined by a Windows PowerShell formatting file. You must use the `PrependPath` parameter when you run the cmdlet, and you cannot load this formatting file as a module.

Demonstrates

This formatting file demonstrates the following XML elements:

- The Name element for the view.
- The ViewSelectedBy element that defines what objects are displayed by the view.
- The ListControl element that defines what property is displayed by the view.
- The ListItem element that defines what is displayed in a row of the list view.
- The PropertyName element that defines which property is displayed.

Example

The following XML defines a list view that displays four properties of the System.ServiceProcess.ServiceController object. In each row, the name of the property is displayed followed by the value of the property.

XML

```
<Configuration>
  <View>
    <Name>System.ServiceProcess.ServiceController</Name>
    <ViewSelectedBy>
      <TypeName>System.ServiceProcess.ServiceController</TypeName>
    </ViewSelectedBy>
    <ListControl>
      <ListEntries>
        <ListEntry>
          <ListItems>
            <ListItem>
              <PropertyName>Name</PropertyName>
            </ListItem>
            <ListItem>
              <PropertyName>DisplayName</PropertyName>
            </ListItem>
            <ListItem>
              <PropertyName>Status</PropertyName>
            </ListItem>
            <ListItem>
              <PropertyName>ServiceType</PropertyName>
            </ListItem>
          </ListItems>
        </ListEntry>
      </ListEntries>
    </ListControl>
  </View>
</Configuration>
```

The following example shows how Windows PowerShell displays the System.ServiceProcess.ServiceController objects after this format file is loaded.

PowerShell

```
Get-Service f*
```

Output

```
Name        : Fax
DisplayName : Fax
Status      : Stopped
ServiceType : Win32OwnProcess
```

```
Name        : FCSAM
DisplayName : Microsoft Antimalware Service
Status      : Running
ServiceType : Win32OwnProcess

Name        : fdPHost
DisplayName : Function Discovery Provider Host
Status      : Stopped
ServiceType : Win32ShareProcess

Name        : FDResPub
DisplayName : Function Discovery Resource Publication
Status      : Running
ServiceType : Win32ShareProcess

Name        : FontCache
DisplayName : Windows Font Cache Service
Status      : Running
ServiceType : Win32ShareProcess

Name        : FontCache3.0.0.0
DisplayName : Windows Presentation Foundation Font Cache 3.0.0.0
Status      : Stopped
ServiceType : Win32OwnProcess

Name        : FSysAgent
DisplayName : Microsoft Forefront System Agent
Status      : Running
ServiceType : Win32OwnProcess

Name        : FwcAgent
DisplayName : Firewall Client Agent
Status      : Running
ServiceType : Win32OwnProcess
```

See Also

Examples of Formatting Files

Writing a PowerShell Formatting File

Last updated on 05/20/2025

List View (Labels)

This example shows how to implement a list view that displays a custom label for each row of the list. This list view displays the properties of the System.ServiceProcess.ServiceController object that is returned by the Get-Service cmdlet. For more information about the components of a list view, see Creating a List View.

Load this formatting file

1. Copy the XML from the Example section of this topic into a text file.
2. Save the text file. Be sure to add the `format.ps1xml` extension to the file to identify it as a formatting file.
3. Open Windows PowerShell, and run the following command to load the formatting file into the current session: `Update-FormatData -PrependPath PathToFormattingFile`.

> ⚠ **Warning**
>
> This formatting file defines the display of an object that is already defined by a Windows PowerShell formatting file. You must use the `PrependPath` parameter when you run the cmdlet, and you cannot load this formatting file as a module.

Demonstrates

This formatting file demonstrates the following XML elements:

- The Name element for the view.
- The ViewSelectedBy element that defines what objects are displayed by the view.
- The ListControl element that defines what property is displayed by the view.
- The ListItem element that defines what is displayed in a row of the list view.
- The Label element that defines what is displayed in a row of the list view.
- The PropertyName element that defines which property is displayed.

Example

The following XML defines a list view that displays a custom label in each row. In this case, the label includes the property name with each letter capitalized and the word "property". In each row, the name of the property is displayed followed by the value of the property.

XML

```
<Configuration>
  <ViewDefinitions>
    <View>
  <Name>System.ServiceProcess.ServiceController</Name>
  <ViewSelectedBy>
    <TypeName>System.ServiceProcess.ServiceController</TypeName>
  </ViewSelectedBy>
  <ListControl>
    <ListEntries>
      <ListEntry>
        <ListItems>
          <ListItem>
            <Label>NAME property</Label>
            <PropertyName>Name</PropertyName>
          </ListItem>
          <ListItem>
            <Label>DISPLAYNAME property</Label>
            <PropertyName>DisplayName</PropertyName>
          </ListItem>
          <ListItem>
            <Label>STATUS property</Label>
            <PropertyName>Status</PropertyName>
          </ListItem>
          <ListItem>
            <Label>SERVICETYPE property</Label>
            <PropertyName>ServiceType</PropertyName>
          </ListItem>
        </ListItems>
      </ListEntry>
    </ListEntries>
  </ListControl>
</View>

  </ViewDefinitions>
</Configuration>
```

The following example shows how Windows PowerShell displays the System.ServiceProcess.ServiceController objects after this format file is loaded.

PowerShell

```
Get-Service f*
```

Output

```
NAME property        : Fax
DISPLAYNAME property : Fax
STATUS property      : Stopped
SERVICETYPE property : Win32OwnProcess

NAME property        : FCSAM
DISPLAYNAME property : Microsoft Antimalware Service
STATUS property      : Running
SERVICETYPE property : Win32OwnProcess

NAME property        : fdPHost
DISPLAYNAME property : Function Discovery Provider Host
STATUS property      : Stopped
SERVICETYPE property : Win32ShareProcess

NAME property        : FDResPub
DISPLAYNAME property : Function Discovery Resource Publication
STATUS property      : Running
SERVICETYPE property : Win32ShareProcess

NAME property        : FontCache
DISPLAYNAME property : Windows Font Cache Service
STATUS property      : Running
SERVICETYPE property : Win32ShareProcess

NAME property        : FontCache3.0.0.0
DISPLAYNAME property : Windows Presentation Foundation Font Cache 3.0.0.0
STATUS property      : Stopped
SERVICETYPE property : Win32OwnProcess

NAME property        : FSysAgent
DISPLAYNAME property : Microsoft Forefront System Agent
STATUS property      : Running
SERVICETYPE property : Win32OwnProcess

NAME property        : FwcAgent
DISPLAYNAME property : Firewall Client Agent
STATUS property      : Running
SERVICETYPE property : Win32OwnProcess
```

See Also

Examples of Formatting Files

Writing a PowerShell Formatting File

Last updated on 05/20/2025

List View (GroupBy)

This example shows how to implement a list view that separates the rows of the list into groups. This list view displays the properties of the System.ServiceProcess.ServiceController objects returned by the Get-Service cmdlet. For more information about the components of a list view, see Creating a List View.

Load this formatting file

1. Copy the XML from the Example section of this topic into a text file.
2. Save the text file. Be sure to add the `format.ps1xml` extension to the file to identify it as a formatting file.
3. Open Windows PowerShell, and run the following command to load the formatting file into the current session: `Update-FormatData -PrependPath PathToFormattingFile`.

> ⚠ **Warning**
>
> This formatting file defines the display of an object that is already defined by a Windows PowerShell formatting file. You must use the `PrependPath` parameter when you run the cmdlet, and you cannot load this formatting file as a module.

Demonstrates

This formatting file demonstrates the following XML elements:

- The Name element for the view.
- The ViewSelectedBy element that defines what objects are displayed by the view.
- The GroupBy element that defines how a new group of objects is displayed.
- The ListControl element that defines what property is displayed by the view.
- The ListItem element that defines what is displayed in a row of the list view.
- The PropertyName element that defines which property is displayed.

Example

The following XML defines a list view that starts a new group whenever the value of the System.ServiceProcess.ServiceController.Status property changes. When each group is started, a custom label is displayed that includes the new value of the property.

XML

```
<Configuration>
  <ViewDefinitions>
    <View>
      <Name>System.ServiceProcess.ServiceController</Name>
      <ViewSelectedBy>
        <TypeName>System.ServiceProcess.ServiceController</TypeName>
      </ViewSelectedBy>
      <GroupBy>
        <PropertyName>Status</PropertyName>
        <Label>New Service Status</Label>
      </GroupBy>
      <ListControl>
        <ListEntries>
          <ListEntry>
            <ListItems>
              <ListItem>
                <PropertyName>Name</PropertyName>
              </ListItem>
              <ListItem>
                <PropertyName>DisplayName</PropertyName>
              </ListItem>
              <ListItem>
                <PropertyName>ServiceType</PropertyName>
              </ListItem>
            </ListItems>
          </ListEntry>
        </ListEntries>
      </ListControl>
    </View>
  </ViewDefinitions>
</Configuration>
```

The following example shows how Windows PowerShell displays the System.ServiceProcess.ServiceController objects after this format file is loaded. The blank lines added before and after the group label are automatically added by Windows PowerShell.

PowerShell

```
Get-Service f*
```

Output

```
   New Service Status: Stopped

Name        : Fax
DisplayName : Fax
ServiceType : Win32OwnProcess

   New Service Status: Running

Name        : FCSAM
DisplayName : Microsoft Antimalware Service
ServiceType : Win32OwnProcess

   New Service Status: Stopped

Name        : fdPHost
DisplayName : Function Discovery Provider Host
ServiceType : Win32ShareProcess

   New Service Status: Running

Name        : FDResPub
DisplayName : Function Discovery Resource Publication
ServiceType : Win32ShareProcess

Name        : FontCache
DisplayName : Windows Font Cache Service
ServiceType : Win32ShareProcess

   New Service Status: Stopped

Name        : FontCache3.0.0.0
DisplayName : Windows Presentation Foundation Font Cache 3.0.0.0
ServiceType : Win32OwnProcess

   New Service Status: Running

Name        : FSysAgent
DisplayName : Microsoft Forefront System Agent
ServiceType : Win32OwnProcess

Name        : FwcAgent
DisplayName : Firewall Client Agent
ServiceType : Win32OwnProcess
```

See Also

Examples of Formatting Files

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Format Schema XML Reference

The topics in this section describe the XML elements used by formatting files (Format.ps1xml files). Formatting files define how the .NET object is displayed; they do not change the object itself.

In This Section

Alignment Element for TableColumnHeader for TableControl (Format) Defines how the data in a column header is displayed.

Alignment Element for TableColumnItem for TableControl (Format) Defines how the data in the row is displayed.

AutoSize Element for TableControl (Format) Specifies whether the column size and the number of columns are adjusted based on the size of the data.

Autosize Element for WideControl (Format) Specifies whether the column size and the number of columns are adjusted based on the size of the data.

ColumnNumber Element for WideControl (Format) Specifies the number of columns displayed in the wide view.

Configuration Element (Format) Represents the top-level element of the formatting file.

Control Element for Controls for Configuration (Format) Defines a common control that can be used by all the views of the formatting file and the name that is used to reference the control.

Control Element for Controls for View (Format) Defines a control that can be used by the view and the name that is used to reference the control.

Controls Element for Configuration (Format) Defines the common controls that can be used by all views of the formatting file.

Controls Element for View (Format) Defines the view controls that can be used by a specific view.

CustomControl Element for Control for Configuration (Format) Defines a control. This element is used when defining a common control that can be used by all the views in the formatting file.

CustomControl Element for Control for Controls for View (Format) Defines a control that is used by the view.

CustomControl Element for GroupBy (Format) Defines the custom control that displays the new group.

CustomControl Element (Format) Defines a custom control format for the view.

CustomControlName Element for ExpressionBinding for Controls for Configuration (Format) Specifies the name of a common control. This element is used when defining a common control that can be used by all the views in the formatting file.

CustomControlName Element for ExpressionBindine for Controls for View (Format) Specifies the name of a common control or a view control. This element is used when defining controls that can be used by a view.

CustomControlName Element of GroupBy (Format) Specifies the name of a custom control that is used to display the new group. This element is used when defining a table, list, wide or custom control view.

CustomEntry Element for CustomControl for Controls for Configuration (Format) Provides a definition of the common control. This element is used when defining a common control that can be used by all the views in the formatting file.

CustomEntry Element for CustomEntries for Controls for View (Format) Provides a definition of the control. This element is used when defining controls that can be used by a view.

CustomEntry Element for CustomEntries for View (Format) Provides a definition of the custom control view.

CustomEntry Element for CustomControl for GroupBy (Format) Provides a definition of the control. This element is used when defining how a new group of objects is displayed.

CustomEntries Element for CustomControl for Configuration (Format) Provides the definitions of a common control. This element is used when defining a common control that can be used by all the views in the formatting file.

CustomEntries Element for CustomControl for Controls for View (Format) Provides the definitions for the control. This element is used when defining controls that can be used by a view.

CustomEntries Element for CustomControl for GroupBy (Format) Provides the definitions for the control. This element is used when defining how a new group of objects is displayed.

CustomEntries Element for CustomControl for View (Format) Provides the definitions of the custom control view. The custom control view must specify one or more definitions.

CustomItem Element for CustomEntry for Controls for Configuration Defines what data is displayed by the control and how it is displayed. This element is used when defining a common control that can be used by all the views in the formatting file.

CustomItem Element for CustomEntry for Controls for View (Format) Defines what data is displayed by the control and how it is displayed. This element is used when defining controls that can be used by a view.

CustomItem Element for CustomEntry for View (Format) Defines what data is displayed by the custom control view and how it is displayed. This element is used when defining a custom control view.

CustomItem Element for CustomEntry for GroupBy (Format) Defines what data is displayed by the custom control view and how it is displayed. This element is used when defining how a new group of objects is displayed.

DefaultSettings Element (Format) Defines common settings that apply to all the views of the formatting file. Common settings include displaying errors, wrapping text in tables, defining how collections are expanded, and more.

DisplayError Element (Format) Specifies that the string #ERR is displayed when an error occurs displaying a piece of data.

EntrySelectedBy Element for CustomEntry for Controls for Configuration (Format) Defines the .NET types that use the definition of the common control or the condition that must exist for this control to be used. This element is used when defining a common control that can be used by all the views in the formatting file.

EntrySelectedBy Element for CustomEntry for Controls for View (Format) Defines the .NET types that use this control definition or the condition that must exist for this definition to be used. This element is used when defining controls that can be used by a view.

EntrySelectedBy Element for CustomEntry for View (Format) Defines the .NET types that use this custom entry or the condition that must exist for this entry to be used.

EntrySelectedBy Element for EnumerableExpansion (Format) Defines the .NET types that use this definition or the condition that must exist for this definition to be used.

EntrySelectedBy Element for CustomEntry for GroupBy (Format) Defines the .NET types that use this control definition or the condition that must exist for this definition to be used. This element is used when defining how a new group of objects is displayed.

EntrySelectedBy Element for ListEntry for ListControl (Format) Defines the .NET types that use this list view definition or the condition that must exist for this definition to be used. In most cases only one definition is needed for a list view. However, you can provide multiple definitions for the list view if you want to use the same list view to display different data for different objects.

EntrySelectedBy Element for TableRowEntry (Format) Defines the .NET types whose property values are displayed in the row.

EntrySelectedBy Element for WideEntry (Format) Defines the .NET types that use this definition of the wide view or the condition that must exist for this definition to be used.

EnumerableExpansion Element (Format) Defines how specific .NET collection objects are expanded when they are displayed in a view.

EnumerableExpansions Element (Format) Defines how .NET collection objects are expanded when they are displayed in a view.

EnumerateCollection Element for ExpressionBinding for Controls for Configuration (Format) Specified that the elements of collections are displayed by the control. This element is used when defining a common control that can be used by all the views in the formatting file.

EnumerateCollection Element for ExpressionBinding for Controls for View (Format) Specified that the elements of collections are displayed. This element is used when defining controls that can be used by a view.

EnumerateCollection Element for Expression Binding for CustomControl for View (Format) Specifies that the elements of collections are displayed. This element is used when defining a custom control view.

EnumerateCollection Element for ExpressionBinding for GroupBy (Format) Specifies that the elements of collections are displayed. This element is used when defining how a new group of objects is displayed.

Expand Element (Format) Specifies how the collection object is expanded for this definition.

ExpressionBinding Element for CustomItem for Controls for Configuration (Format) Defines the data that is displayed by the control. This element is used when defining a common control that can be used by all the views in the formatting file.

ExpressionBinding Element for CustomItem for Controls for View (Format) Defines the data that is displayed by the control. This element is used when defining controls that can be used by a view.

ExpressionBinding Element for CustomItem for CustomControl for View (Format) Defines the data that is displayed by the control. This element is used when defining a custom control view.

ExpressionBinding Element for CustomItem for GroupBy (Format) Defines the data that is displayed by the control. This element is used when defining how a new group of objects is displayed.

FirstLineHanging Element for Frame for Controls for Configuration (Format) Specifies how many characters the first line of data is shifted to the left. This element is used when defining a common control that can be used by all the views in the formatting file.

FirstLineHanging Element of Frame of Controls of View (Format) Specifies how many characters the first line of data is shifted to the left. This element is used when defining controls that can be used by a view.

FirstLineHanging Element for Frame for CustomControl for View (Format) Specifies how many characters the first line of data is shifted to the left. This element is used when defining a custom control view.

FirstLineHanging Element for Frame for GroupBy (Format) Specifies how many characters the first line of data is shifted to the left. This element is used when defining how a new group of objects is displayed.

FirstLineIndent Element for Frame for Controls for Configuration (Format) Specifies how many characters the first line of data is shifted to the right. This element is used when defining a common control that can be used by all the views in the formatting file.

FirstLineIndent Element of Frame of Controls of View (Format) Specifies how many characters the first line of data is shifted to the right. This element is used when defining controls that can be used by a view.

FirstLineIndent Element Specifies how many characters the first line of data is shifted to the right. This element is used when defining a custom control view.

FirstLineIndent Element for Frame for GroupBy (Format) Specifies how many characters the first line of data is shifted to the right. This element is used when defining how a new group of objects is displayed.

FormatString Element for ListItem (Format) Specifies a format pattern that defines how the property or script value is displayed.

FormatString Element for TableColumnItem (Format) Specifies a format pattern that defines how the property or script value of the table is displayed.

FormatString Element for WideItem for WideControl (Format) Specifies a format pattern that defines how the property or script value is displayed in the view.

Frame Element for CustomItem for Controls for Configuration (Format) Defines how the data is displayed, such as shifting the data to the left or right. This element is used when defining a common control that can be used by all the views in the formatting file.

Frame Element for CustomItem for Controls for View (Format) Defines how the data is displayed, such as shifting the data to the left or right. This element is used when defining controls that can be used by a view.

Frame Element for CustomItem for CustomControl for View (Format) Defines how the data is displayed, such as shifting the data to the left or right. This element is used when defining a custom control view.

Frame Element for CustomItem for GroupBy (Format) Defines how the data is displayed, such as shifting the data to the left or right. This element is used when defining how a new group of objects is displayed.

GroupBy Element for View (Format) Defines how Windows PowerShell displays a new group of objects.

HideTableHeaders Element (Format) Specifies that the headers of the table are not displayed.

ItemSelectionCondition Element for ExpressionBinding for Controls for Configuration (Format) Defines the condition that must exist for this control to be used. This element is used when defining a common control that can be used by all the views in the formatting file.

ItemSelectionCondition Element of ExpressionBinding for Controls for View (Format) Defines the condition that must exist for this control to be used. This element is used when defining controls that can be used by a view.

ItemSelectionCondition Element for Expression Binding for CustomControl for View (Format) Defines the condition that must exist for this control to be used. There is no limit to the number of selection conditions that can be specified for a control item. This element is used when defining a custom control view.

ItemSelectionCondition Element for ExpressionBinding for GroupBy (Format) Defines the condition that must exist for this control to be used. There is no limit to the number of selection conditions that can be specified for a control item. This element is used when defining how a new group of objects is displayed.

ItemSelectionCondition Element for ListItem (Format) Defines the condition that must exist for this list item to be used.

Label Element for ListItem for ListControl(Format) Specifies the label for the property or script value in the row.

Label Element for GroupBy (Format) Specifies a label that is displayed when a new group is encountered.

Label Element for TableColumnHeader (Format) Defines the label that is displayed at the top of a column.

LeftIndent Element for Frame for Controls for Configuration (Format) Specifies how many characters the data is shifted away from the left margin. This element is used when defining a common control that can be used by all the views in the formatting file.

LeftIndent Element of Frame of Controls of View (Format) Specifies how many characters the data is shifted away from the left margin. This element is used when defining controls that can be used by a view.

LeftIndent Element for Frame for CustomControl for View (Format) Specifies how many characters the data is shifted away from the left margin. This element is used when defining a custom control view.

LeftIndent Element for Frame for GroupBy (Format) Specifies how many characters the data is shifted away from the left margin. This element is used when defining how a new group of objects is displayed.

ListControl Element (Format) Defines a list format for the view.

ListEntry Element (Format) Provides a definition of the list view.

ListEntries Element (Format) Defines how the rows of the list view are displayed.

ListItem Element (Format) Defines the property or script whose value is displayed in a row of the list view.

ListItems Element (Format) Defines the properties and scripts that are displayed in the list view.

Name Element for Control for Controls for Configuration (Format) Specifies the name of the control. This element is used when defining a common control that can be used by all the views in the formatting file.

Name Element for SelectionSet (Format) Specifies the name used to reference the selection set.

Name Element for View (Format) Specifies the name that is used to identify the view.

NewLine Element for CustomItem for Controls for Configuration (Format) Adds a blank line to the display of the control. This element is used when defining a common control that can be used by all the views in the formatting file.

NewLine Element for CustomItem for Controls for View (Format) Adds a blank line to the display of the control. This element is used when defining controls that can be used by a view.

NewLine Element for CustomItem for CustomControl for View (Format) Adds a blank line to the display of the control. This element is used when defining a custom control view.

NewLine Element for CustomItem for GroupBy (Format) Adds a blank line to the display of the control. This element is used when defining how a new group of objects is displayed.

PropertyName Element for ExpressionBinding for Controls for Configuration (Format) Specifies the .NET property whose value is displayed by the common control. This element is used when defining a common control that can be used by all the views in the formatting file.

PropertyName Element for ExpressionBinding for Controls for View (Format) Specifies the .NET property whose value is displayed by the control. This element is used when defining controls that can be used by a view.

PropertyName Element for ExpressionBinding for CustomControl for View (Format) Specifies the .NET property whose value is displayed by the control. This element is used when defining a custom control view

PropertyName Element for ExpressionBinding for GroupBy (Format) Specifies the .NET property whose value is displayed by the control. This element is used when defining how a new group of objects is displayed.

PropertyName Element for GroupBy (Format) Specifies the .NET property that starts a new group whenever its value changes.

PropertyName Element for ItemSelectionCondition for Controls for Configuration (Format) Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the control is used. This element is used when defining a common control that can be used by all the views in the formatting file.

PropertyName Element for ItemSelectionCondition for Controls for View (Format) Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the control is used. This element is used when defining controls that can be used by a view.

PropertyName Element for ItemSelectionCondition for CustomControl for View (Format Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the control is used. This element is used when defining a custom control view.

PropertyName Element for ItemSelectionCondition for GroupBy (Format) Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the control is used. This element is used when defining how a new group of objects is displayed.

PropertyName Element for ItemSelectionCondition for ListItem (Format) Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the view is used. This element is used when defining a list view.

PropertyName Element for ListItem for ListControl (Format) Specifies the .NET property whose value is displayed in the list.

PropertyName Element for SelectionCondition for EntrySelectedBy for ListEntry (Format) Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the entry is used. This element is used when defining a common control that can be used by all the views in the formatting file.

PropertyName Element for SelectionCondition for Controls for View (Format) Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the entry is used. This element is used when defining controls that can be used by a view.

PropertyName Element for SelectionCondition for CustomControl for View (Format) Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the definition is used. This element is used when defining a custom control view.

PropertyName Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion (Format) Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the definition is used.

PropertyName Element for SelectionCondition for GroupBy (Format) Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the definition is used. This element is used when defining how a new group of objects is displayed.

PropertyName Element for SelectionCondition for EntrySelectedBy for ListEntry (Format) Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the list entry is used.

PropertyName Element for SelectionCondition for EntrySelectedBy for TableRowEntry (Format) Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the table entry is used.

PropertyName Element for SelectionCondition for EntrySelectedBy for WideEntry (Format) Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the definition is used.

PropertyName Element for TableColumnItem (Format) Specifies the property whose value is displayed in the column of the row.

PropertyName Element for WideItem (Format) Specifies the property of the object whose value is displayed in the wide view.

RightIndent Element for Frame for Controls for Configuration (Format) Specifies how many characters the data is shifted away from the right margin. This element is used when defining a common control that can be used by all the views in the formatting file.

RightIndent Element of Frame of Controls of View (Format) Specifies how many characters the data is shifted away from the right margin. This element is used when defining controls that can be used by a view.

RightIndent Element Specifies how many characters the data is shifted away from the right margin. This element is used when defining a custom control view.

RightIndent Element for Frame for GroupBy (Format) Specifies how many characters the data is shifted away from the right margin. This element is used when defining how a new group of objects is displayed.

ScriptBlock Element for ExpressionBinding for Controls for Configuration (Format) Specifies the script whose value is displayed by the common control. This element is used when defining a common control that can be used by all the views in the formatting file.

ScriptBlock Element for ExpressionBinding for Controls for View (Format) Specifies the script whose value is displayed by the control. This element is used when defining controls that can be used by a view.

ScriptBlock Element for ExpressionBinding for CustomCustomControl for View (Format) Specifies the script whose value is displayed by the control. This element is used when defining a custom control view.

ScriptBlock Element for ExpressionBinding for GroupBy (Format) Specifies the script whose value is displayed by the control. This element is used when defining how a new group of objects is displayed.

ScriptBlock Element for GroupBy (Format) Specifies the script that starts a new group whenever its value changes.

ScriptBlock Element for ItemSelectionCondition for Controls for Configuration (Format) Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the control is used. This element is used when defining a common control that can be used by all the views in the formatting file.

ScriptBlock Element for ItemSelectionCondition for Controls for View (Format) Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the control is used. This element is used when defining controls that can be used by a view.

ScriptBlock Element for ItemSelectionCondition for CustomControl for View (Format) Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the control is used. This element is used when defining a custom control view.

ScriptBlock Element for ItemSelectionCondition for GroupBy (Format) Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the control is used. This element is used when defining how a new group of objects is displayed.

ScriptBlock Element for ItemSelectionCondition for ListControl (Format) Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the list item is used. This element is used when defining a list view.

ScriptBlock Element for ListItem (Format) Specifies the script whose value is displayed in the row of the list.

ScriptBlock Element for SelectionCondition for Controls for Configuration (Format) Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the definition is used. This element is used when defining a common control that can be used by all the views in the formatting file.

ScriptBlock Element for SelectionCondition for Controls for View (Format) Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the definition is used. This element is used when defining controls that can be used by a view.

ScriptBlock Element for SelectionCondition for CustomControl for View (Format) Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the definition is used. This element is used when defining a custom control view.

ScriptBlock Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion (Format) Specifies the script that triggers the condition.

ScriptBlock Element for SelectionCondition for GroupBy (Format) Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the definition is used. This element is used when defining how a new group of objects is displayed.

ScriptBlock Element for SelectionCondition for EntrySelectedBy for ListEntry (Format) Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the list entry is used.

ScriptBlock Element for SelectionCondition for EntrySelectedBy for TableRowEntry (Format) Specifies the script block that triggers the condition. When this script is evaluated to `true`, the condition is met, and the table entry is used.

ScriptBlock Element for SelectionCondition for EntrySelectedBy for WideEntry (Format) Specifies the script that triggers the condition. When this script is evaluated to `true`, the

condition is met, and the wide entry definition is used.

ScriptBlock Element for TableColumnItem (Format) Specifies the script whose value is displayed in the column of the row.

ScriptBlock Element for WideItem (Format) Specifies the script whose value is displayed in the wide view.

SelectionCondition Element for EntrySelectedBy for CustomEntry for Configuration (Format) Defines a condition that must exist for a common control definition to be used. This element is used when defining a common control that can be used by all the views in the formatting file.

SelectionCondition Element for EntrySelectedBy for Controls for View (Format) Defines a condition that must exist for the control definition to be used. This element is used when defining controls that can be used by a view.

SelectionCondition Element for EntrySelectedBy for CustomControl for View (Format) Defines a condition that must exist for a control definition to be used. This element is used when defining a custom control view.

SelectionCondition Element for EntrySelectedBy for EnumerableExpansion (Format) Defines the condition that must exist to expand the collection objects of this definition.

SelectionCondition Element for EntrySelectedBy for GroupBy (Format) Defines a condition that must exist for a control definition to be used. This element is used when defining how a new group of objects is displayed.

SelectionCondition Element for EntrySelectedBy for ListEntry (Format) Defines the condition that must exist to use this definition of the list view. There is no limit to the number of selection conditions that can be specified for a list definition.

SelectionCondition Element for EntrySelectedBy for TableRowEntry (Format) Defines the condition that must exist to use for this definition of the table view. There is no limit to the number of selection conditions that can be specified for a table definition.

SelectionCondition Element for EntrySelectedBy for WideEntry (Format) Defines the condition that must exist for this definition to be used. There is no limit to the number of selection conditions that can be specified for a wide entry definition.

SelectionSet Element (Format) Defines a set of .NET objects that can be referenced by the name of the set.

SelectionSetName Element for EntrySelectedBy for Controls for Configuration (Format) Specifies a set of .NET types that use this definition of the control. This element is used when defining a common control that can be used by all the views in the formatting file.

SelectionSetName Element for EntrySelectedBy for Controls for View (Format) Specifies a set of .NET types that use this definition of the control. This element is used when defining controls that can be used by a view.

SelectionSetName Element for EntrySelectedBy for CustomEntry (Format) Specifies a set of .NET objects for the list entry. There is no limit to the number of selection sets that can be specified for an entry.

SelectionSetName Element for EntrySelectedBy for EnumerableExpansion (Format) Specifies the set of .NET types that are expanded by this definition.

SelectionSetName Element for EntrySelectedBy for GroupBy (Format) Specifies a set of .NET objects for the list entry. There is no limit to the number of selection sets that can be specified for an entry. This element is used when defining how a new group of objects is displayed.

SelectionSetName Element for EntrySelectedBy for ListEntry (Format) Specifies a set of .NET objects for the list entry. There is no limit to the number of selection sets that can be specified for an entry.

SelectionSetName Element for EntrySelectedBy for TableRowEntry (Format) Specifies a set of .NET types the use this entry of the table view. There is no limit to the number of selection sets that can be specified for an entry.

SelectionSetName Element for EntrySelectedBy for WideEntry (Format) Specifies a set of .NET objects for the definition. The definition is used whenever one of these objects is displayed.

SelectionSetName Element for SelectionCondition for Controls for Configuration (Format) Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met, and the object is displayed by using this control. This element is used when defining a common control that can be used by all the views in the formatting file.

SelectionSetName Element for SelectionCondition for Controls for View (Format) Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met and the object is displayed using this control. This element is used when defining controls that can be used by a view.

EntrySelectedBy Element for CustomEntry for View (Format) Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met and the object is displayed using this control. This element is used when defining a custom control view.

SelectionSetName Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion (Format) Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met.

SelectionSetName Element for SelectionCondition for GroupBy (Format) Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met, and the object is displayed by using this control. This element is used when defining how a new group of objects is displayed.

SelectionSetName Element for SelectionCondition for EntrySelectedBy for ListEntry (Format) Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met, and the object is displayed by using this definition of the list view.

SelectionSetName Element for SelectionCondition for EntrySelectedBy for TableRowEntry (Format) Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met, and the object is displayed by using this definition of the table view.

SelectionSetName Element for SelectionCondition for EntrySelectedBy for WideEntry (Format) Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met, and the object is displayed by using this definition of the wide view.

SelectionSetName Element for ViewSelectedBy (Format) Specifies a set of .NET objects that are displayed by the view.

SelectionSets Element (Format) Defines the sets of .NET objects that can be used by individual format views.

ShowError Element (Format) Specifies that the full error record is displayed when an error occurs while displaying a piece of data.

TableColumnHeader Element for TableHeaders for TableControl (Format) Defines the label, the width of the column, and the alignment of the label for a column of the table.

TableColumnItem Element (Format) Defines the property or script whose value is displayed in the column of the row.

TableColumnItems Element (Format) Defines the properties or scripts whose values are displayed in the row.

TableControl Element (Format) Defines a table format for a view.

TableHeaders Element (Format) Defines the headers for the columns of a table.

TableRowEntries Element (Format) Defines the rows of the table.

TableRowEntry Element (Format) Defines the data that is displayed in a row of the table.

Text Element for CustomItem for Controls for Configuration (Format) Specifies text that is added to the data that is displayed by the control, such as a label, brackets to enclose the data, and spaces to indent the data. This element is used when defining a common control that can be used by all the views in the formatting file.

Text Element for CustomItem for Controls for View (Format) Specifies text that is added to the data that is displayed by the control, such as a label, brackets to enclose the data, and spaces to indent the data. This element is used when defining controls that can be used by a view.

Text Element for CustomItem (Format) Specifies text that is added to the data that is displayed by the control, such as a label, brackets to enclose the data, and spaces to indent the data. This element is used when defining a custom control view.

Text Element for CustomItem for GroupBy (Format) Specifies text that is added to the data that is displayed by the control, such as a label, brackets to enclose the data, and spaces to indent the data. This element is used when defining how a new group of objects is displayed.

TypeName Element for EntrySelectedBy for Controls for Configuration (Format) Specifies a .NET type that uses this definition of the control. This element is used when defining a common control that can be used by all the views in the formatting file.

TypeName Element for EntrySelectedBy for Controls for View (Format) Specifies a .NET type that uses this definition of the control. This element is used when defining controls that can be used by a view.

TypeName Element for EntrySelectedBy for CustomEntry for View (Format) Specifies a .NET type that uses this definition of the custom control view. There is no limit to the number of types that can be specified for a definition.

TypeName Element for EntrySelectedBy for EnumerableExpansion (Format) Specifies a .NET type that is expanded by this definition. This element is used when defining a default settings.

TypeName Element for EntrySelectedBy for GroupBy (Format) Specifies a .NET type that uses this definition of the custom control. This element is used when defining how a new group of objects is displayed.

TypeName Element for EntrySelectedBy for ListControl (Format) Specifies a .NET type that uses this entry of the list view. There is no limit to the number of types that can be specified for a list entry.

TypeName Element for EntrySelectedBy for TableRowEntry (Format) Specifies a .NET type that uses this entry of the table view. There is no limit to the number of types that can be specified for a table entry.

TypeName Element for EntrySelectedBy for WideEntry (Format) Specifies a .NET type for the definition. The definition is used whenever this object is displayed.

TypeName Element for SelectionCondition for Controls for Configuration (Format) Specifies a .NET type that triggers the condition. This element is used when defining a common control that can be used by all the views in the formatting file.

TypeName Element for SelectionCondition for Controls for View (Format) Specifies a .NET type that triggers the condition. This element is used when defining controls that can be used by a view.

TypeName Element for SelectionCondition for CustomControl for View (Format) Specifies a .NET type that triggers the condition. This element is used when defining a custom control view.

TypeName Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion (Format) Specifies a .NET type that triggers the condition.

TypeName Element for SelectionCondition for GroupBy (Format) Specifies a .NET type that triggers the condition. This element is used when defining how a new group of objects is displayed.

TypeName Element for SelectionCondition for EntrySelectedBy for ListControl (Format) Specifies a .NET type that triggers the condition. When this type is present, the list entry is used.

TypeName Element for SelectionCondition for EntrySelectedBy for TableRowEntry (Format) Specifies a .NET type that triggers the condition. When this type is present, the condition is met, and the table row is used.

TypeName Element for SelectionCondition for EntrySelectedBy for WideEntry (Format) Specifies a .NET type that triggers the condition. When this type is present, the definition is used.

TypeName Element for Types (Format) Specifies the .NET type of an object that belongs to the selection set.

TypeName Element for ViewSelectedBy (Format) Specifies a .NET object that is displayed by the view.

Types Element (Format) Defines the .NET objects that are in the selection set.

View Element (Format) Defines a view that is used to display one or more .NET objects.

ViewDefinitions Element (Format) Defines the views used to display objects.

ViewSelectedBy Element (Format) Defines the .NET objects that are displayed by the view.

WideControl Element (Format) Defines a wide (single value) list format for the view. This view displays a single property value or script value for each object.

WideEntries Element (Format) Provides the definitions of the wide view. The wide view must specify one or more definitions.

WideEntry Element (Format) Provides a definition of the wide view.

WideItem Element (Format) Defines the property or script whose value is displayed.

Width Element (Format) Defines the width (in characters) of a column.

Wrap Element (Format) Specifies that text that exceeds the column width is displayed on the next line.

WrapTables Element (Format) Specifies that data in a table cell is moved to the next line if the data is longer than the width of the column.

See Also

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Configuration Element

Represents the top-level element of a formatting file.

Schema

- Configuration Element

Syntax

XML

```
<Configuration>
  <DefaultSettings>...</DefaultSettings>
  <SelectionSets>...</SelectionSets>
  <Controls>...</Controls>
  <ViewDefinitions>...</ViewDefinitions>
</Configuration>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `Configuration` element. This element must be the root element for each formatting file, and this element must contain at least one child element.

Attributes

None.

Child Elements

Expand table

Element	Description
Controls Element for Configuration	Optional element. Defines the common controls that can be used by all views of the formatting file.

Element	Description
DefaultSettings Element	Optional element. Defines common settings that apply to all the views of the formatting file.
SelectionSets Element Format	Optional element. Defines the common sets of .NET objects that can be used by all views of the formatting file.
ViewDefinitions Element	Optional element. Defines the views used to display objects.

Parent Elements

None.

Remarks

Formatting files define how objects are displayed. In most cases, this root element contains a ViewDefinitions element that defines the table, list, and wide views of the formatting file. In addition to the view definitions, the formatting file can define common selection sets, settings, and controls that those views can use.

See Also

Controls Element for Configuration

DefaultSettings Element

SelectionSets Element

ViewDefinitions Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Controls Element for Configuration

Defines the common controls that can be used by all views of the formatting file.

Schema

- Configuration Element
- Controls Element

Syntax

XML

```
<Controls>
  <Control>...</Control>
</Controls>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `Controls` element.

Attributes

None.

Child Elements

Expand table

Element	Description
Control Element for Controls for Configuration	Required element. Defines a common control that can be used by all views of the formatting file.

Parent Elements

Expand table

Element	Description
Configuration Element	Represents the top-level element of a formatting file.

Remarks

You can create any number of common controls. For each control, you must specify the name that is used to reference the control and the components of the control.

See Also

Configuration Element

Control Element for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Control Element for Controls for Configuration

Defines a common control that can be used by all the views of the formatting file and the name that is used to reference the control.

Schema

- Configuration Element
- Controls Element
- Control Element

Syntax

XML

```
<Control>
  <Name>NameOfControl</Name>
  <CustomControl>...</CustomControl>
</Control>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element for the `Control` element. You must specify only one of each child element.

Attributes

None.

Child Elements

Expand table

Element	Description
CustomControl Element for Control for Controls for Configuration	Required element. Defines the control.

Element	Description
Name Element for Control for Configuration	Required element. Specifies the name used to reference the control.

Parent Elements

Expand table

Element	Description
Controls Element of Configuration	Defines the common controls that can be used by all views of the formatting file or by other controls.

Remarks

The name given to this control can be referenced in the following elements:

- ExpressionBinding Element for CustomItem
- GroupBy Element for View

See Also

Controls Element of Configuration

CustomControl element for Control for Configuration

ExpressionBinding Element for CustomItem

GroupBy Element for View(Format)

Name Element for Control for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomControl Element for Control for Controls for Configuration

Defines a control. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element

Syntax

XML

```
<CustomControl>
  <CustomEntries>...</CustomEntries>
</CustomControl>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `CustomControl` element. This element must have at least one child element. There is no maximum limit to the number of child elements that can be specified.

Attributes

None.

Child Elements

Expand table

Element	Description
CustomEntries Element for CustomControl for Configuration	Required element.

Element	Description
	Provides the definitions of a control.

Parent Elements

Expand table

Element	Description
Control Element for Controls for Configuration	Defines a common control that can be used by all the views of the formatting file and the name that is used to reference the control.

Remarks

See Also

Control Element for Controls for Configuration

CustomEntries Element for CustomControl for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomEntries Element for CustomControl for Controls for Configuration

Provides the definitions of a common control. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element

Syntax

XML

```
<CustomEntries>
  <CustomEntry>...</CustomEntry>
</CustomEntries>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomEntries` element. You must specify one or more child elements.

Attributes

None.

Child Elements

Expand table

Element	Description
CustomEntry Element for CustomControl for Controls for Configuration	Provides a definition of the common control.

Parent Elements

Expand table

Element	Description
CustomControl Element for Control for Configuration	Defines a common control.

Remarks

In most cases, a control has only one definition, which is defined in a single `CustomEntry` element. However it is possible to have multiple definitions if you want to use the same control to display different .NET objects. In those cases, you can define a `CustomEntry` element for each object or set of objects.

See Also

CustomControl Element for Control for Configuration

CustomEntry Element for CustomControl for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomEntry Element for CustomControl for Controls for Configuration

Provides a definition of the common control. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element

Syntax

XML

```
<CustomEntry>
  <EntrySelectedBy>...</EntrySelectedBy>
  <CustomItem>...</CustomItem>
</CustomEntry>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomEntry` element. You must specify the items displayed by the definition.

Attributes

None.

Child Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for Controls for Configuration	Optional element. Defines the .NET types that use the definition of the common control or the condition that must exist for this control to be used.
CustomItem Element for CustomEntry for Controls for Configuration	Required element. Defines what data is displayed by the control and how it is displayed.

Parent Elements

Expand table

Element	Description
CustomEntries Element for CustomControl for Configuration	Provides the definitions of the common control.

Remarks

In most cases, only one definition is required for each common custom control, but it is possible to have multiple definitions if you want to use the same control to display different .NET objects. In those cases, you can provide a separate definition for each object or set of objects.

See Also

CustomEntries Element for CustomControl for Configuration

CustomItem Element for CustomEntry for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomItem Element for CustomEntry for Controls for Configuration

Defines what data is displayed by the control and how it is displayed. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element

Syntax

XML

```xml
<CustomItem>
  <ExpressionBinding>...</ExpressionBinding>
  <NewLine/>
  <Text>TextToDisplay</Text>
  <Frame>...</Frame>
</CustomItem>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomItem` element. For more information, see Remarks.

Attributes

None.

Child Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for Controls for Configuration	Optional element. Defines the data that is displayed by the control.
Frame Element for CustomItem for Controls for Configuration	Optional element. Defines how the data is displayed, such as shifting the data to the left or right.
NewLine Element for CustomItem for Controls for Configuration	Optional element. Adds a blank line to the display of the control.
Text Element for CustomItem for Controls for Configuration	Optional element. Adds text, such as parentheses or brackets, to the display of the control.

Parent Elements

Expand table

Element	Description
CustomEntry Element for CustomControl for Controls for Configuration	Provides a definition of the control.

Remarks

When specifying the child elements of the `CustomItem` element, keep the following in mind:

- The child elements must be added in the following sequence: `ExpressionBinding`, `NewLine`, `Text`, and `Frame`.
- There is no maximum limit to the number of sequences that you can specify.
- In each sequence, there is no maximum limit to the number of `ExpressionBinding` elements that you can use.

See Also

ExpressionBinding Element for CustomItem for Controls for Configuration

Frame Element for CustomItem for Controls for Configuration

NewLine Element for CustomItem for Controls for Configuration

Text Element for CustomItem for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ExpressionBinding Element for CustomItem for Controls for Configuration

Defines the data that is displayed by the control. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element

Syntax

XML

```
<ExpressionBinding>
  <CustomControl>...</CustomControl>
  <CustomControlName>NameofCommonCustomControl</CustomControlName>
  <EnumerateCollection/>
  <ItemSelectionCondition>...</ItemSelectionCondition>
  <PropertyName>Nameof.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate></ScriptBlock>
</ExpressionBinding>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ExpressionBinding` element.

Attributes

None.

Child Elements

Expand table

Element	Description
`CustomControl Element`	Optional element. Defines a control that is used by this control.
CustomControlName Element for ExpressionBinding for Controls for Configuration	Optional element. Specifies the name of a common control or a view control.
EnumerateCollection Element for ExpressionBinding for Controls for Configuration	Optional element. Specified that the elements of collections are displayed by the control.
ItemSelectionCondition Element for ExpressionBinding for Controls for Configuration	Optional element. Defines the condition that must exist for this common control to be used.
PropertyName Element for ExpressionBinding for Controls for Configuration	Optional element. Specifies the .NET property whose value is displayed by the common control.
ScriptBlock Element for ExpressionBinding for Controls for Configuration	Optional element. Specifies the script whose value is displayed by the common control.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for Controls for Configuration	Defines what data is displayed by the custom control view and how it is displayed.

Remarks

See Also

CustomItem Element for CustomEntry for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomControlName Element for ExpressionBinding for Controls for Configuration

Specifies the name of a common control. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- CustomControlName Element

Syntax

XML

```
<CustomControlName>NameofCustomControl</CustomControlName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomControlName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for Controls for Configuration	Defines the data that is displayed by the control.

Text Value

Specify the name of the control.

Remarks

You can create common controls that can be used by all the views of a formatting file, and you can create view controls that can be used by a specific view. The following elements specify the names of these controls:

- Name Element for Control for Controls for Configuration
- Name Element for Control for Controls for View

See Also

Name Element for Control for Controls for Configuration

Name Element for Control for Controls for View

ExpressionBinding Element for CustomItem for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

EnumerateCollection Element for ExpressionBinding for Controls for Configuration

Specified that the elements of collections are displayed by the control. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- EnumerateCollection Element

Syntax

XML

```
<EnumerateCollection/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `EnumerateCollection` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for Controls for Configuration	Defines the data that is displayed by the control.

Remarks

See Also

ExpressionBinding Element for CustomItem for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ItemSelectionCondition Element for ExpressionBinding for Controls for Configuration

Defines the condition that must exist for this control to be used. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ItemSelectionCondition Element

Syntax

XML

```
<ItemSelectionCondition>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</ItemSelectionCondition>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ItemSelectionCondition` element.

Attributes

None.

Child Elements

Expand table

Element	Description
PropertyName Element for ItemSelectionCondition for Controls for Configuration	Optional element. Specifies the .NET property that triggers the condition.
ScriptBlock Element for ItemSelectionCondition for Controls for Configuration	Optional element. Specifies the script that triggers the condition.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for Controls for Configuration	Defines the data that is displayed by the control.

Remarks

You can specify one property name or a script for this condition but cannot specify both.

See Also

PropertyName Element for ItemSelectionCondition for Controls for Configuration

ScriptBlock Element for ItemSelectionCondition for Controls for Configuration

ExpressionBinding Element for CustomItem for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for ItemSelectionCondition for Controls for Configuration

Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the control is used. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ItemSelectionCondition Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ItemSelectionCondition Element for ExpressionBinding for Controls for Configuration	Defines the condition that must exist for this control to be used.

Text Value

Specify the name of the .NET property that triggers the condition.

Remarks

If this element is used, you cannot specify the ScriptBlock element when defining the selection condition.

See Also

ScriptBlock Element for ItemSelectionCondition for Controls for Configuration

ItemSelectionCondition Element for ExpressionBinding for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for ItemSelectionCondition for Controls for Configuration

Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the control is used. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ItemSelectionCondition Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ItemSelectionCondition Element for ExpressionBinding for Controls for Configuration	Defines the condition that must exist for this control to be used.

Text Value

Specify the script that is evaluated.

Remarks

If this element is used, you cannot specify the PropertyName element when defining the selection condition.

See Also

PropertyName Element for ItemSelectionCondition for Controls for Configuration

ItemSelectionCondition Element for ExpressionBinding for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for ExpressionBinding for Controls for Configuration

Specifies the .NET property whose value is displayed by the common control. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for Controls for Configuration	Defines the data that is displayed by the control.

Text Value

Specify the name of the .NET property whose value is displayed by the control.

Remarks

See Also

ExpressionBinding Element for CustomItem for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for ExpressionBinding for Controls for Configuration

Specifies the script whose value is displayed by the common control. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for Controls for Configuration	Defines the data that is displayed by the common control.

Text Value

Specify the script whose value is displayed by the control.

Remarks

See Also

ExpressionBinding Element for CustomItem for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Frame Element for CustomItem for Controls for Configuration

Defines how the data is displayed, such as shifting the data to the left or right. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element

Syntax

XML

```
<Frame>
  <LeftIndent>NumberOfCharactersToShift</LeftIndent>
  <RightIndent>NumberOfCharactersToShift</RightIndent>
  <FirstLineHanging>NumberOfCharactersToShift</FirstLineHanging>
  <FirstLineIndent>NumberOfCharactersToShift</FirstLineIndent>
  <CustomItem>...</CustomItem>
</Frame>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `Frame` element.

Attributes

None.

Child Elements

Expand table

Element	Description
`CustomItem Element`	Required Element
FirstLineHanging Element for Frame for Controls for Configuration	Optional element. Specifies how many characters the first line of data is shifted to the left.
FirstLineIndent Element for Frame for Controls for Configuration	Optional element. Specifies how many characters the first line of data is shifted to the right.
LeftIndent Element for Frame for Controls for Configuration	Optional element. Specifies how many characters the data is shifted away from the left margin.
RightIndent Element for Frame for Controls for Configuration	Optional element. Specifies how many characters the data is shifted away from the right margin.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for Controls for Configuration	Defines what data is displayed by the control and how it is displayed.

Remarks

You cannot specify the FirstLineHanging and the FirstLineIndent elements in the same `Frame` element.

See Also

FirstLineHanging Element for Frame for Controls for Configuration

FirstLineIndent Element for Frame for Controls for Configuration

LeftIndent Element for Frame for Controls for Configuration

RightIndent Element for Frame for Controls for Configuration

CustomItem Element for CustomEntry for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

FirstLineHanging Element for Frame for Controls for Configuration

Specifies how many characters the first line of data is shifted to the left. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- FirstLineHanging Element

Syntax

XML

```
<FirstLineHanging>NumberOfCharactersToShift</FirstLineHanging>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `FirstLineHanging` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for Controls for Configuration	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the first line of the data.

Remarks

If this element is specified, you cannot specify the `FirstLineIndent` element.

See Also

Frame Element for CustomItem for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

FirstLineIndent Element for Frame for Controls for Configuration

Specifies how many characters the first line of data is shifted to the right. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- FirstLineIndent Element

Syntax

XML

```
<FirstLineIndent>NumberOfCharactersToShift</FirstLineIndent>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `FirstLineIndent` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for Controls for Configuration	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the first line of the data.

Remarks

If this element is specified, you cannot specify the FirstLineHanging element.

See Also

FirstLineHanging Element for Frame for Controls for Configuration

Frame Element for CustomItem for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

LeftIndent Element for Frame for Controls for Configuration

Specifies how many characters the data is shifted away from the left margin. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- LeftIndent Element

Syntax

XML

```
<LeftIndent>CharactersToShift</LeftIndent>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `LeftIndent` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for Controls for Configuration	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the data to the left.

Remarks

See Also

Frame Element for CustomItem for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

RightIndent Element for Frame for Controls for Configuration

Specifies how many characters the data is shifted away from the right margin. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- RightIndent Element

Syntax

XML

```
<RightIndent>CharactersToShift</RightIndent>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `RightIndent` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for Controls for Configuration	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the data to the right.

Remarks

See Also

Frame Element for CustomItem for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

NewLine Element for CustomItem for Controls for Configuration

Adds a blank line to the display of the control. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- NewLine Element

Syntax

XML

```
<NewLine/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `NewLine` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for Controls for Configuration	Defines a control for the custom control view.

Remarks

See Also

CustomItem Element for CustomEntry for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Text Element for CustomItem for Controls for Configuration

Specifies text that is added to the data that is displayed by the control, such as a label, brackets to enclose the data, and spaces to indent the data. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Text Element

Syntax

XML

```
<Text>TextToDisplay</Text>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `Text` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for Controls for Configuration	Defines what data is displayed by the control and how it is displayed.

Text Value

Specify the text of a control for data that you want to display.

Remarks

See Also

CustomItem Element for CustomEntry for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

EntrySelectedBy Element for CustomEntry for Controls

Defines the .NET types that use the definition of the common control or the condition that must exist for this control to be used. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element

Syntax

XML

```xml
<EntrySelectedBy>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>SelectionSet</SelectionSetName>
  <SelectionCondition>...</SelectionCondition>
</EntrySelectedBy>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `EntrySelectedBy` element.

Attributes

None.

Child Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for Controls for Configuration	Optional element. Defines the condition that must exist for the common control definition to be used.
SelectionSetName Element for EntrySelectedBy for Controls for Configuration	Optional element. Specifies a set of .NET types that use this definition of the common control.
TypeName Element for EntrySelectedBy for Controls for Configuration	Optional element. Specifies a .NET type that uses this definition of the common control.

Parent Elements

Expand table

Element	Description
CustomEntry Element for CustomControl for Controls for Configuration	Provides a definition of the common control.

Remarks

At a minimum, each definition must have at least one .NET type, selection set, or selection condition specified. There is no maximum limit to the number of types, selection sets, or selection conditions that you can specify.

See Also

SelectionCondition Element for EntrySelectedBy for Controls for Configuration

SelectionSetName Element for EntrySelectedBy for Controls for Configuration

CustomEntry Element for CustomControl for Controls for Configuration

TypeName Element for EntrySelectedBy for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionCondition Element for EntrySelectedBy for Controls for Configuration

Defines a condition that must exist for a common control definition to be used. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element

Syntax

XML

```xml
<SelectionCondition>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</SelectionCondition>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionCondition` element.

Attributes

None.

Child Elements

Expand table

Element	Description
PropertyName Element for SelectionCondition for Controls for Configuration	Optional element. Specifies a .NET property that triggers the condition.
ScriptBlock Element for SelectionCondition for Controls for Configuration	Optional element. Specifies the script that triggers the condition.
SelectionSetName Element for SelectionCondition for Controls for Configuration	Optional element. Specifies the set of .NET types that triggers the condition.
TypeName Element for SelectionCondition for Controls for Configuration	Optional element. Specifies a .NET type that triggers the condition.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for Controls for Configuration	Defines the .NET types that use this entry of the common control definition.

Remarks

The following guidelines must be followed when defining a selection condition:

- The selection condition must specify a least one property name or a script block, but cannot specify both.
- The selection condition can specify any number of .NET types or selection sets, but cannot specify both.

For more information about how selection conditions can be used, see Defining Conditions for when Data is Displayed.

See Also

PropertyName Element for SelectionCondition for Controls for Configuration

ScriptBlock Element for SelectionCondition for Controls for Configuration

SelectionSetName Element for SelectionCondition for Controls for Configuration

TypeName Element for SelectionCondition for Controls for Configuration

EntrySelectedBy Element for CustomEntry for Controls for Configuration

Writing a Windows PowerShell Formatting and Types File

Last updated on 05/20/2025

PropertyName Element for SelectionCondition for Controls for Configuration

Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the entry is used. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for Controls for Configuration	Defines a condition that must exist for a common control definition to be used.

Text Value

Specify the .NET property name.

Remarks

The selection condition must specify a least one property name or a script, but cannot specify both. For more information about how selection conditions can be used, see Defining Conditions for Displaying Data.

See Also

SelectionCondition Element for EntrySelectedBy for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for SelectionCondition for Controls for Configuration

Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the definition is used. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for Controls for Configuration	Defines a condition that must exist for the common control definition to be used.

Text Value

Specify the script that is evaluated.

Remarks

The selection condition must specify a least one script or property name to evaluate, but cannot specify both. For more information about how selection conditions can be used, see Defining Conditions for Displaying Data.

See Also

SelectionCondition Element for EntrySelectedBy for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for SelectionCondition for Controls for Configuration

Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met, and the object is displayed by using this control. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for Controls for Configuration	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the name of the selection set.

Remarks

Selection sets are common groups of .NET objects that can be used by any view that the formatting file defines. For more information about creating and referencing selection sets, see Defining Sets of Objects.

The selection condition can specify a selection set or .NET type, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for Displaying Data.

See Also

SelectionCondition Element for EntrySelectedBy for Controls for Configuration

Defining Conditions for When Data Is Displayed

Defining Selection Sets

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for SelectionCondition for Controls for Configuration

Specifies a .NET type that triggers the condition. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` Element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for CustomEntry for Configuration	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

See Also

SelectionCondition Element for EntrySelectedBy for CustomEntry for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for EntrySelectedBy for Controls for Configuration

Specifies a set of .NET types that use this definition of the control. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element of Configuration
- Control Element for Controls for Configuration
- CustomControl Element for Control for Configuration
- CustomEntries Element for CustomControl for Configuration
- CustomEntry Element for CustomControl for Controls for Configuration
- EntrySelectedBy Element for CustomEntry for Controls for Configuration
- SelectionSetName Element for EntrySelectedBy for Controls for Configuration

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for Controls for Configuration	Defines the .NET types that use this control definition or the condition that must exist for this definition to be used.

Text Value

Specify the name of the selection set.

Remarks

Each control definition must have at least one type name, selection set, or selection condition defined.

Selection sets are typically used when you want to define a group of objects that are used in multiple views. For more information about defining selection sets, see Defining Selection Sets.

See Also

EntrySelectedBy Element for CustomEntry for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for EntrySelectedBy for Controls for Configuration

Specifies a .NET type that uses this definition of the control. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for Controls for Configuration	Defines the .NET types that use this control definition or the condition that must exist for this definition to be used.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

See Also

EntrySelectedBy Element for CustomEntry for Controls for Configuration

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Name Element for Control for Controls for Configuration

Specifies the name of the control. This element is used when defining a common control that can be used by all the views in the formatting file.

Schema

- Configuration Element
- Controls Element
- Control Element
- Name Element

Syntax

XML

```
<Name>NameOfControl</Name>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `Name` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Control Element for Controls for Configuration	Defines a common control that can be used by all the views of the formatting file and the name that is used to reference the control.

Text Value

Specify the name that is used to reference this control.

Remarks

The name specified here can be used in the following elements to reference this control.

- When creating a table, list, wide or custom control view, the control can be specified by the following element: GroupBy Element for View
- When creating another common control, this control can be specified by the following element: ExpressionBinding Element for CustomItem for Controls for Configuration
- When creating a control that can be used by a view, this control can be specified by the following element: ExpressionBinding Element for CustomItem for Controls for View

See Also

Control Element for Controls for Configuration

ExpressionBinding Element for CustomItem for Controls for Configuration

ExpressionBinding Element for CustomItem for Controls for View

GroupBy Element for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

DefaultSettings Element

Defines common settings that apply to all the views of the formatting file. Common settings include displaying errors, wrapping text in tables, defining how collections are expanded, and more.

Schema

- Configuration Element
- DefaultSettings Element

Syntax

XML

```
<DefaultSettings>
  <ShowError/>
  <DisplayError/>
 <PropertyCountForTable>NumberOfProperties</PropertyCountFortable>
  <WrapTables/>
  <EnumerableExpansions>...</EnumerableExpansions>
</DefaultSettings>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `DefaultSettings` element.

Attributes

None.

Child Elements

Expand table

Element	Description
DisplayError Element	Optional element.

Element	Description
	Specifies that the string #ERR is displayed when an error occurs while displaying a piece of data.
EnumerableExpansions Element	Optional element. Defines the different ways that .NET objects are expanded when they are displayed in a view.
PropertyCountForTable	Optional element. Specifies the minimum number of properties that an object must have to display the object in a table view.
ShowError Element	Optional element. Specifies that the full error record is displayed when an error occurs while displaying a piece of data.
WrapTables Element	Optional element. Specifies that data in a table is moved to the next line if it does not fit into the width of the column.

Parent Elements

Expand table

Element	Description
Configuration Element	Represents the top-level element of a formatting file.

Remarks

See Also

Configuration Element

DisplayError Element

EnumerableExpansions Element

PropertyCountForTable

ShowError Element

WrapTables Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

DisplayError Element

Specifies that the string #ERR is displayed when an error occurs displaying a piece of data.

Schema

- Configuration Element
- DefaultSettings Element
- DisplayError Element

Syntax

XML

```
<DisplayError/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `DisplayError` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
DefaultSettings Element	Defines common settings that apply to all the views of the formatting file.

Remarks

By default, when an error occurs while trying to display a piece of data, the location of the data is left blank. When this element is set to true, the #ERR string will be displayed.

See Also

DefaultSettings Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

EnumerableExpansions Element

Defines how .NET collection objects are expanded when they are displayed in a view.

Schema

- Configuration Element
- DefaultSettings Element
- EnumerableExpansions Element

Syntax

XML

```xml
<EnumerableExpansions>
  <EnumerableExpansion>...</EnumerableExpansion>
</EnumerableExpansions>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `EnumerableExpansions` element. There is no limit to the number of child elements that you can use.

Attributes

None.

Child Elements

Expand table

Element	Description
EnumerableExpansion Element	Optional element. Defines the specific .NET collection objects that are expanded when they are displayed in a view.

Parent Elements

Expand table

Element	Description
DefaultSettings Element	Defines common settings that apply to all the views of the formatting file.

Remarks

This element is used to define how collection objects and the objects in the collection are displayed. In this case, a collection object refers to any object that supports the **System.Collections.ICollection** interface.

See Also

Writing a PowerShell Formatting File

Last updated on 05/20/2025

EnumerableExpansion Element

Defines how specific .NET collection objects are expanded when they are displayed in a view.

Schema

- Configuration Element
- DefaultSettings Element
- EnumerableExpansions Element
- EnumerableExpansion Element

Syntax

XML

```
<EnumerableExpansion>
  <EntrySelectedBy>...</EntrySelectedBy>
  <Expand>EnumOnly, CoreOnly, Both</Expand>
</EnumerableExpansion>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `EnumerableExpansion` element.

Attributes

None.

Child Elements

Expand table

Element	Description
EntrySelectedBy Element for EnumerableExpansion	Optional element. Defines which .NET collection objects are expanded by this definition.

Element	Description
Expand Element	Specifies how the collection object is expanded for this definition.

Parent Elements

Expand table

Element	Description
EnumerableExpansions Element	Defines the different ways that .NET collection objects are expanded when they are displayed in a view.

Remarks

This element is used to define how collection objects and the objects in the collection are displayed. In this case, a collection object refers to any object that supports the **System.Collections.ICollection** interface.

The default behavior is to display only the properties of the objects in the collection.

See Also

Writing a PowerShell Formatting File

Last updated on 05/20/2025

EntrySelectedBy Element for EnumerableExpansion

Defines the .NET types that use this definition or the condition that must exist for this definition to be used.

Schema

- Configuration Element
- DefaultSettings Element
- EnumerableExpansions Element
- EnumerableExpansion Element
- EntrySelectedBy Element for EnumerableExpansion

Syntax

XML

```xml
<EntrySelectedBy>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <SelectionCondition>...</SelectionCondition>
</EntrySelectedBy>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `EntrySelectedBy` element.

Attributes

None.

Child Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for EnumerableExpansion	Optional element. Defines the condition that must exist to expand the collection objects of this definition.
SelectionSetName Element for EntrySelectedBy for EnumerableExpansion	Optional element. Specifies a set of .NET types that use this definition of how collection objects are expanded.
TypeName Element for EntrySelectedBy for EnumerableExpansion	Optional element. Specifies a .NET type that uses this definition of how collection objects are expanded.

Parent Elements

Expand table

Element	Description
EnumerableExpansion Element	Defines how specific .NET collection objects are expanded when they are displayed in a view.

Remarks

You must specify at least one type, selection set, or selection condition for a definition entry. There is no maximum limit to the number of child elements that you can use.

Selection conditions are used to define a condition that must exist for the definition to be used, such as when an object has a specific property or that a specific property value or script evaluates to `true`. For more information about selection conditions, see Defining Conditions for Displaying Data.

See Also

Defining Conditions for Displaying Data

EnumerableExpansion Element

SelectionCondition Element for EntrySelectedBy for EnumerableExpansion

SelectionSetName Element for EntrySelectedBy for EnumerableExpansion

TypeName Element for EntrySelectedBy for EnumerableExpansion

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionCondition Element for EntrySelectedBy for EnumerableExpansion

Defines the condition that must exist to expand the collection objects of this definition.

Schema

- Configuration Element
- DefaultSettings Element
- EnumerableExpansions Element
- EnumerableExpansion Element
- EntrySelectedBy Element
- SelectionCondition Element

Syntax

XML

```xml
<SelectionCondition>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</SelectionCondition>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionCondition` element. You must specify a single `PropertyName` or `ScriptBlock` element. The `SelectionSetName` and `TypeName` elements are optional. You can specify one of either element.

Attributes

None.

Child Elements

Expand table

Element	Description
PropertyName Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion	Optional element. Specifies the .NET property that triggers the condition.
ScriptBlock Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion	Optional element. Specifies the script that triggers the condition.
SelectionSetName Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion	Optional element. Specifies the set of .NET types that triggers the condition.
TypeName Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion	Optional element. Specifies a .NET type that triggers the condition.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for EnumerableExpansion	Defines which .NET collection objects are expanded by this definition.

Remarks

Each definition must have at least one type name, selection set, or selection condition defined.

When you are defining a selection condition, the following requirements apply:

- The selection condition must specify a least one property name or a script block, but cannot specify both.
- The selection condition can specify any number of .NET types or selection sets, but cannot specify both.

For more information about how to use selection conditions, see Defining Conditions for Displaying Data.

For more information about other components of a wide view, see Wide View.

See Also

Defining Conditions for When Data Is Displayed

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for SelectionCondition for Controls for View

Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the entry is used. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element for Controls for View
- CustomControl Element for Control for Controls for View
- CustomEntries Element for CustomControl for Controls for View
- CustomEntry Element for CustomEntries for Controls for View
- EntrySelectedBy Element for CustomEntry for Controls for View
- SelectionCondition Element for EntrySelectedBy for Controls for View
- PropertyName Element for SelectionCondition for Controls for View

Syntax

XML

```xml
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for Controls for View	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the .NET property name.

Remarks

The selection condition must specify a least one property name or a script, but cannot specify both. For more information about how selection conditions can be used, see Defining Conditions for Displaying Data.

See Also

SelectionCondition Element for EntrySelectedBy for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for SelectionCondition for Controls for View

Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the definition is used. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for Controls for View	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the script that is evaluated.

Remarks

The selection condition must specify a least one script or property name to evaluate, but cannot specify both. For more information about how selection conditions can be used, see Defining Conditions for Displaying Data.

See Also

SelectionCondition Element for EntrySelectedBy for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion

Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met.

Schema

- Configuration Element
- DefaultSettings Element
- EnumerableExpansions Element
- EnumerableExpansions Element
- EntrySelectedBy Element
- SelectionCondition Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for EnumerableExpansion	Defines the condition that must exist to expand the collection objects of this definition.

Text Value

Specify the name of the selection set.

Remarks

The selection condition can specify a selection set or .NET type, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for Displaying Data.

Selection sets are common groups of .NET objects that can be used by any view that the formatting file defines. For more information about creating and referencing selection sets, see Defining Selection Sets.

See Also

Defining Selection Sets

SelectionCondition Element for EntrySelectedBy for EnumerableExpansion

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion

Specifies a .NET type that triggers the condition.

Schema

- Configuration Element
- DefaultSettings Element
- EnumerableExpansions Element
- EnumerableExpansions Element
- EntrySelectedBy Element
- SelectionCondition Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for EnumerableExpansion	Defines the condition that must exist to expand the collection objects of this definition.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

See Also

SelectionCondition Element for EntrySelectedBy for EnumerableExpansion

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for EntrySelectedBy for EnumerableExpansion

Specifies the set of .NET types that are expanded by this definition.

Schema

- Configuration Element
- DefaultSettings Element
- EnumerableExpansions Element
- EnumerableExpansion Element
- EntrySelectedBy Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for EnumerableExpansion	Defines the .NET collection objects that are expanded by this definition.

Text Value

Specify the name of the selection set.

Remarks

Each definition must specify one or more type names, a selection set, or a selection condition.

Selection sets are typically used when you want to define a group of objects that are used in multiple views. For example, you might want to create a table view and a list view for the same set of objects. For more information about defining selection sets, see Defining Sets of Objects for a View.

See Also

Defining Selection Sets

EntrySelectedBy Element for EnumerableExpansion

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for EntrySelectedBy for EnumerableExpansion

Specifies a .NET type that is expanded by this definition. This element is used when defining a default settings.

Schema

- Configuration Element
- DefaultSettings Element
- EnumerableExpansions Element
- EnumerableExpansion Element
- EntrySelectedBy Element
- TypeName Element

Syntax

XML

```xml
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for EnumerableExpansion	Defines the .NET types that use this definition or the condition that must exist for this definition to be used.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

See Also

EntrySelectedBy Element for EnumerableExpansion

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Expand Element

Specifies how the collection object is expanded for this definition.

Schema

- Configuration Element
- DefaultSettings Element
- EnumerableExpansions Element
- EnumerableExpansion Element
- Expand Element

Syntax

XML

```
<Expand>EnumOnly, CoreOnly, Both</Expand>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `Expand` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EnumerableExpansion Element	Defines how specific .NET collection objects are expanded when they are displayed in a view.

Text Value

Specify one of the following values:

- EnumOnly: Displays only the properties of the objects in the collection.
- CoreOnly: Displays only the properties of the collection object.
- Both: Displays the properties of the objects in the collection and the properties of the collection object.

Remarks

This element is used to define how collection objects and the objects in the collection are displayed. In this case, a collection object refers to any object that supports the **System.Collections.ICollection** interface.

The default behavior is to display only the properties of the objects in the collection.

See Also

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyCountForTable Element

Optional element. Specifies the minimum number of properties that an object must have to display the object in a table view.

Schema

- DefaultSettings Element
- PropertyCountForTable Element

Syntax

XML

```
<PropertyCountForTable>NumberOfProperties</PropertyCountFortable>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyCountForTable` element. The default value for this element is `4`.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
DefaultSettings Element	Defines common settings that apply to all the views of the formatting file.

Remarks

See Also

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ShowError Element

Specifies that the full error record is displayed when an error occurs while displaying a piece of data.

Schema

- Configuration Element
- DefaultSettings Element
- ShowError Element

Syntax

```
scr
```

```
<ShowError/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ShowError` element. The default value for this element is `false`.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
DefaultSettings Element	Defines common settings that apply to all the views of the formatting file.

Remarks

See Also

Writing a PowerShell Formatting File

Last updated on 05/20/2025

WrapTables Element

Specifies that data in a table cell is moved to the next line if the data is longer than the width of the column.

Schema

- Configuration Element
- DefaultSettings Element
- WrapTables Element

Syntax

XML

```
<WrapTables/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `WrapTables` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
DefaultSettings Element	Defines common settings that apply to all the views of the formatting file.

Remarks

See Also

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSets Element

Defines the common sets of .NET objects that can be used by all views of the formatting file. The views and controls of the formatting file can reference the complete set of objects by using only the name of the selection set.

Schema

- Configuration Element
- SelectionSets Element

Syntax

XML

```
<SelectionSets>
  <SelectionSet>...</SelectionSet>
</SelectionSets>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `SelectionSets` element. Each child element defines a set of objects that can be referenced by the name of the set. The order of the child elements is not significant.

Attributes

None.

Child Elements

Expand table

Element	Description
SelectionSet Element	Required element. Defines a single set of .NET objects that can be referenced by the name of the set.

Parent Elements

Expand table

Element	Description
Configuration Element	Represents the top-level element of a formatting file.

Remarks

You can use selection sets when you have a set of related objects that you want to reference by using a single name, such as a set of objects that are related through inheritance. When defining your views, you can specify the set of objects by using the name of the selection set instead of listing all the objects within each view.

Common selection sets are specified by their name when defining the views of the formatting file or the definitions of the views. In these cases, the `SelectionSetName` child element of the `ViewSelectedBy` and `EntrySelectedBy` elements specifies the set to be used. For more information about selection sets, see Defining Sets of Objects.

See Also

Configuration Element

Defining Selection Sets

SelectionSet Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSet Element

Defines a set of .NET objects that can be referenced by the name of the set.

Schema

- Configuration Element
- SelectionSets Element
- SelectionSet Element

Syntax

XML

```xml
<SelectionSet>
  <Name>SelectionSetName</Name>
  <Types>...</Types>
</SelectionSet>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `SelectionSet` element. Each selection set must have a name, and it must specify the .NET objects of the set.

Attributes

None.

Child Elements

Expand table

Element	Description
Name Element for SelectionSet	Required element. Specifies the name used to reference the selection set.
Types Element	Required element.

Element	Description
	Defines the .NET objects that are in the selection set.

Parent Elements

Expand table

Element	Description
SelectionSets Element Format	Defines the common sets of .NET objects that can be used by all views of the formatting file.

Remarks

You can use selection sets when you have a set of related objects that you want to reference by using a single name, such as a set of objects that are related through inheritance. When defining your views, you can specify the set of objects by using the name of the selection set instead of listing all the objects within each view.

Common selection sets are specified by their name when defining the views of the formatting file or the definitions of the views. In these cases, the `SelectionSetName` child element of the `ViewSelectedBy` and `EntrySelectedBy` elements specifies the set to be used. For more information about selection sets, see Defining Sets of Objects.

Example

The following example shows a `SelectionSet` element that defines four .NET types.

XML

```
<SelectionSets>
  <SelectionSet>
    <Name>FileSystemTypes</Name>
    <Types>
     <TypeName>System.IO.DirectoryInfo</TypeName>
     <TypeName>System.IO.FileInfo</TypeName>
     <TypeName>Deserialized.System.IO.DirectoryInfo</TypeName>
     <TypeName>Deserialized.System.IO.FileInfo</TypeName>
    </Types>
  </SelectionSet>
</SelectionSets>
```

See Also

Defining Selection Sets

Name Element of SelectionSet

SelectionSets Element

Types Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Name Element for SelectionSet

Specifies the name used to reference the selection set.

Schema

- Configuration Element
- SelectionSets Element
- SelectionSet Element
- Name Element

Syntax

XML

```
<Name>Name of selection set</Name>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `Name` Element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionSet Element	Defines a single set of .NET objects that can be referenced by the name of the set.

Text Value

Specify the name to reference the selection set. There are no restrictions as to what characters can be used.

Remarks

The name specified here is used in the `SelectionSetName` element. The selection set that can be used by a view, by a definition of a view (views can have multiple definitions), or when specifying a selection condition. For more information about selection sets, see Defining Sets of Objects.

Example

This example shows a `SelectionSet` element that defines four .NET types. The name of the selection set is "FileSystemTypes".

XML

```xml
<SelectionSets>
  <SelectionSet>
    <Name>FileSystemTypes</Name>
    <Types>
     <TypeName>System.IO.DirectoryInfo</TypeName>
     <TypeName>System.IO.FileInfo</TypeName>
     <TypeName>Deserialized.System.IO.DirectoryInfo</TypeName>
     <TypeName>Deserialized.System.IO.FileInfo</TypeName>
    </Types>
  </SelectionSet>
</SelectionSets>
```

See Also

Defining Selection Sets

SelectionSet Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Types Element for SelectionSet

Defines the .NET objects that are in the selection set.

Schema

- Configuration Element
- SelectionSets Element
- SelectionSet Element
- Types Element

Syntax

XML

```xml
<Types>
  <TypeName>Nameof.NetType</TypeName>
</Types>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `Types` element. There must be at least one child element, but there is no maximum limit to the number of child elements that can be added.

Attributes

None.

Child Elements

Expand table

Element	Description
TypeName Element of Types	Required element. Specifies the .NET object that belongs to the selection set.

Parent Elements

Expand table

Element	Description
SelectionSet Element	Defines a set of .NET objects that can be referenced by the name of the set.

Remarks

The objects defined by this element make up a selection set that can be used by a view, by a definition of a view (views can have multiple definitions), or when specifying a selection condition. For more information about selection sets, see Defining Sets of Objects.

Example

This example shows a `SelectionSet` element that defines four .NET types.

XML

```
<SelectionSets>
  <SelectionSet>
    <Name>FileSystemTypes</Name>
    <Types>
     <TypeName>System.IO.DirectoryInfo</TypeName>
     <TypeName>System.IO.FileInfo</TypeName>
     <TypeName>Deserialized.System.IO.DirectoryInfo</TypeName>
     <TypeName>Deserialized.System.IO.FileInfo</TypeName>
    </Types>
  </SelectionSet>
</SelectionSets>
```

See Also

Defining Sets of Objects

SelectionSet Element

TypeName Element of Types

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for Types

Specifies the .NET type of an object that belongs to the selection set.

Schema

- Configuration Element
- SelectionSets Element
- SelectionSet Element
- Types Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</Name>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `TypeName` element. At least one `TypeName` element must be included in the selection set.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Types Element	Defines the .NET objects that are in the selection set.

Text Value

Specify the fully qualified name for the .NET type.

Remarks

You can use selection sets when you have a set of related objects that you want to reference by using a single name, such as a set of objects that are related through inheritance. When defining your views, you can specify the set of objects by using the name of the selection set instead of listing all the objects within each view.

Common selection sets are specified by their name when defining the views of the formatting file. In these cases, the `SelectionSetName` child element of the `ViewSelectedBy` element for the view specifies the set. However, different entries of a view can also specify a selection set that applies to only that entry of the view. For more information about selection sets, see Defining Sets of Objects.

Example

The following example shows a `SelectionSet` element that defines four .NET types.

```
<SelectionSets>
  <SelectionSet>
    <Name>FileSystemTypes</Name>
    <Types>
     <TypeName>System.IO.DirectoryInfo</TypeName>
     <TypeName>System.IO.FileInfo</TypeName>
     <TypeName>Deserialized.System.IO.DirectoryInfo</TypeName>
     <TypeName>Deserialized.System.IO.FileInfo</TypeName>
    </Types>
  </SelectionSet>
</SelectionSets>
```

See Also

Defining Selection Sets

SelectionSet Element

SelectionSets Element

Types Element

Writing a Windows PowerShell Formatting File

Last updated on 05/20/2025

ViewDefinitions Element

Defines the views used to display .NET objects. These views can display the properties and script values of an object in a table format, list format, wide format, and custom control format.

Schema

- Configuration Element
- ViewDefinitions

Syntax

XML

```
<ViewDefinitions>
  <View>...</View>
</ViewDefinitions>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `ViewDefinitions` element. There is no limit to the number of views that can be defined in a formatting file, and they can be added in any order.

Attributes

None.

Child Elements

Expand table

Element	Description
View Element	Defines a view that is used to display one or more .NET objects.

Parent Elements

Expand table

Element	Description
Configuration Element	Represents the top-level element of a formatting file.

Remarks

For more information about the components of the different types of views, see the following topics:

- Creating a Table View
- Creating a List View
- Creating a Wide View
- Custom Controls

Example

This example shows a `ViewDefinitions` element that contains the parent elements for a table view and a list view.

XML

```
<Configuration>
  <ViewDefinitions>
    <View>
      <TableControl>...</TableControl>
    </View>
    <View>
      <ListControl>...</ListControl>
    </View>
  </ViewDefinitions>
</Configuration>
```

See Also

Configuration Element

View Element

Creating a Table View

Creating a List View

Creating a Wide View

Custom Controls

Writing a PowerShell Formatting File

Last updated on 05/20/2025

View Element

Defines a view that displays one or more .NET objects. There is no limit to the number of views that can be defined in a formatting file.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element

Syntax

XML

```
<View>
  <Name>Friendly name of view.</Name>
  <OutOfBand />
  <ViewSelectedBy>...</ViewSelectedBy>
  <Controls>...</Controls>
  <GroupBy>...</GroupBy>
  <TableControl>...</TableControl>
  <ListControl>...</ListControl>
  <WideControl>...</WideControl>
  <CustomControl>...</CustomControl>
</View>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `View` element. You must specify one and only one of the control child elements, and you must specify the name of the view and the objects that use the view. Defining custom controls, how to group objects, and specifying if the view is out-of-band are optional.

Attributes

None.

Child Elements

Expand table

Element	Description
Controls Element for View	Optional element. Defines a set of controls that can be referenced by their name from within the view.
CustomControl Element	Optional element. Defines a custom control format for the view.
GroupBy Element for View	Optional element. Defines how the members of the .NET objects are grouped.
ListControl Element	Optional element. Defines a list format for the view.
Name Element for View	Required element. Specifies the name used to reference the view.
OutOfBand	Optional element When OutOfBand is true, the view applies regardless of previous objects that may have selected a different view.
TableControl Element	Optional element. Defines a table format for the view.
ViewSelectedBy Element for View	Required element. Defines the .NET objects that this view displays.
WideControl Element	Optional element. Defines a wide (single value) list format for the view.

Parent Elements

Expand table

Element	Description
ViewDefinitions Element	Defines the views used to display objects.

Remarks

For more information about the components of different views and custom controls, see the following topics:

- Table View Components
- List View Components
- Wide View Components
- Custom Controls

Example

This example shows a `View` element that defines a table view for the System.ServiceProcess.ServiceController object.

XML

```
<ViewDefinitions>
  <View>
    <Name>service</Name>
    <ViewSelectedBy>
      <TypeName>System.ServiceProcess.ServiceController</TypeName>
    </ViewSelectedBy>
    <TableControl>...</TableControl>
  </View>
</ViewDefinitions>
```

See Also

ViewDefinitions Element

Name Element for View

ViewSelectedBy Element

Controls Element for View

GroupBy Element for View

TableControl Element

ListControl Element

WideControl Element

CustomControl Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Controls Element for View

Defines the view controls that can be used by a specific view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element

Syntax

XML

```xml
<Controls>
  <Control>...</Control>
</Controls>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent elements of the `Controls` element. This element must have at least one child element. There is no maximum number of child elements, nor is their order significant.

Attributes

None.

Child Elements

Expand table

Element	Description
Control Element for Controls for View	Defines a control that can be used by the view.

Parent Elements

Expand table

Element	Description
View Element	Defines a view that is used to display the members of one or more .NET objects.

Remarks

See Also

Control Element

View Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Control Element for Controls for View

Defines a control that can be used by the view and the name that is used to reference the control.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element

Syntax

XML

```xml
<Control>
  <Name>NameOfControl</Name>
  <CustomControl>...</CustomControl>
</Control>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `Control` element.

Attributes

None.

Child Elements

Expand table

Element	Description
Name Element for Control for View	Required element.

Element	Description
	Specifies the name of the control.
CustomControl Element for Control for Controls for View	Required element. Defines the control used by this view.

Parent Elements

Expand table

Element	Description
Controls Element	Defines the view controls that can be used by a specific view.

Remarks

This control can be specified by the following elements:

- CustomControlName Element for ExpressionBinding for Controls for View
- CustomControlName Element for ExpressionBinding for CustomControl for View
- CustomControlName Element for ExpressionBinding for GroupBy
- CustomControlName Element for GroupBy

See Also

CustomControl Element for Control for Controls for View

CustomControlName Element for ExpressionBinding for Controls for View

CustomControlName Element for ExpressionBinding for CustomControl for View

CustomControlName Element for ExpressionBinding for GroupBy

CustomControlName Element for ExpressionBinding for GroupBy

Controls Element

Name Element for Control for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomControl Element for Control for Controls for View

Defines a control that is used by the view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element

Syntax

XML

```
<CustomControl>
  <CustomEntries>...</CustomEntries>
</CustomControl>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomControl` element. You must specify only one child element.

Attributes

None.

Child Elements

Expand table

Element	Description
CustomEntries Element for CustomControl for Controls for View	Required element.

Element	Description
	Provides the definitions for the control.

Parent Elements

Expand table

Element	Description
Control Element for Controls for View	Defines a control that can be used by the view and the name that is used to reference the control.

Remarks

See Also

CustomEntries Element for CustomControl for View

Control Element for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomEntries Element for CustomControl for Controls for View

Provides the definitions for the control. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element

Syntax

XML

```
<CustomEntries>
  <CustomEntry>...</CustomEntry>
</CustomEntries>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements of the `CustomEntries` element. There is no maximum limit to the number of child elements that can be specified.

Attributes

None.

Child Elements

Expand table

Element	Description
CustomEntry Element for CustomEntries for Controls for View	Required element. Provides a definition of the control.

Parent Elements

Expand table

Element	Description
CustomControl Element for Control for Controls for View	Defines the control used by the view.

Remarks

In most cases, a control has only one definition, which is specified in a single `CustomEntry` element. However, it is possible to provide multiple definitions if you want to use the same control to display different .NET objects. In those cases, you can define a `CustomEntry` element for each object or set of objects.

See Also

CustomEntry Element for CustomEntries for Controls for View

CustomControl Element for Control for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomEntry Element for CustomEntries for Controls for View

Provides a definition of the control. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element

Syntax

XML

```
<CustomEntry>
  <EntrySelectedBy>...</EntrySelectedBy>
  <CustomItem>...</CustomItem>
</CustomEntry>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent elements of the `CustomEntry` element.

Attributes

None.

Child Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for Controls for View	Optional element. Defines the .NET types that use this control definition or the condition that must exist for this definition to be used.
CustomItem Element for CustomEntry for Controls for View	Required element. Defines how the control displays the data.

Parent Elements

Expand table

Element	Description
CustomEntries Element for CustomControl for View	Provides the definitions for the control.

Remarks

See Also

CustomEntries Element for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomItem Element for CustomEntry for Controls for View

Defines what data is displayed by the control and how it is displayed. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element

Syntax

XML

```xml
<CustomItem>
  <ExpressionBinding>...</ExpressionBinding>
  <NewLine/>
  <Text>TextToDisplay</Text>
  <Frame>...<Frame>
</CustomItem>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomItem` element. For more information, see Remarks.

Attributes

None.

Child Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for Controls for View	Optional element. Defines the data that is displayed by the control.
Frame Element for CustomItem for Controls for View	Optional element. Defines how the data is displayed, such as shifting the data to the left or right.
NewLine Element for CustomItem for Controls for View	Optional element. Adds a blank line to the display of the control.
Text Element for CustomItem for Controls for View	Optional element. Adds text, such as parentheses or brackets, to the display of the control.

Parent Elements

Expand table

Element	Description
CustomEntry Element for CustomEntries for Controls for View	Provides a definition of the control.

Remarks

When specifying the child elements of the `CustomItem` element, keep the following in mind:

- The child elements must be added in the following sequence: `ExpressionBinding`, `NewLine`, `Text`, and `Frame`.
- There is no maximum limit to the number of sequences that you can specify.
- In each sequence, there is no maximum limit to the number of `ExpressionBinding` elements that you can use.

See Also

ExpressionBinding Element for CustomItem for Controls for View

Frame Element for CustomItem for Controls for View

NewLine Element for CustomItem for Controls for View

Text Element for CustomItem for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ExpressionBinding Element for CustomItem for Controls for View

Defines the data that is displayed by the control. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element

Syntax

XML

```xml
<ExpressionBinding>
  <CustomControl>...</CustomControl>
  <CustomControlName>NameofCommonCustomControl</CustomControlName>
  <EnumerateCollection/>
  <ItemSelectionCondition>...</ItemSelectionCondition>
  <PropertyName>Nameof.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate></ScriptBlock>
</ExpressionBinding>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ExpressionBinding` element.

Attributes

None.

Child Elements

Expand table

Element	Description
`CustomControl Element`	Optional element. Defines a control that is used by this control.
CustomControlName Element for ExpressionBinding for Controls for View	Optional element. Specifies the name of a common control or a view control.
EnumerateCollection Element for ExpressionBinding for Controls for View	Optional element. Specifies that the elements of collections are displayed.
ItemSelectionCondition Element of ExpressionBinding for Controls for View	Optional element. Defines the condition that must exist for this control to be used.
PropertyName Element for ExpressionBinding for Controls for View	Optional element. Specifies the .NET property whose value is displayed by the control.
ScriptBlock Element for ExpressionBinding for Controls for View	Optional element. Specifies the script whose value is displayed by the control.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for Controls for View	Defines what data is displayed by the control and how it is displayed.

Remarks

See Also

CustomItem Element for CustomEntry for Controls for View

CustomControlName Element for ExpressionBinding for Controls for View

EnumerateCollection Element for ExpressionBinding for Controls for View

ItemSelectionCondition Element of ExpressionBinding for Controls for View

PropertyName Element for ExpressionBinding for Controls for View

ScriptBlock Element for ExpressionBinding for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomControlName Element for ExpressionBinding for Controls for View

Specifies the name of a common control or a view control. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- CustomControlName Element

Syntax

XML

```
<CustomControlName>NameofCustomControl</CustomControlName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomControlName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for Controls for View	Defines the data that is displayed by the control.

Text Value

Specify the name of the control.

Remarks

You can create common controls that can be used by all the views of a formatting file, and you can create view controls that can be used by a specific view. The following elements specify the names of these controls:

- Name Element for Control for Controls for Configuration
- Name Element for Control for Controls for View

See Also

Name Element for Control for Controls for Configuration

Name Element for Control for Controls for View

ExpressionBinding Element for CustomItem for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

EnumerateCollection Element for ExpressionBinding for Controls for View

Specified that the elements of collections are displayed. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- EnumerateCollection Element

Syntax

XML

```
<EnumerateCollection/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `EnumerateCollection` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for Controls for View	Defines the data that is displayed by the control.

Remarks

See Also

ExpressionBinding Element for CustomItem for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ItemSelectionCondition Element for ExpressionBinding for Controls for View

Defines the condition that must exist for this control to be used. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ItemSelectionCondition Element

Syntax

XML

```xml
<ItemSelectionCondition>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</ItemSelectionCondition>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ItemSelectionCondition` element.

Attributes

None.

Child Elements

Expand table

Element	Description
PropertyName Element for ItemSelectionCondition for Controls for View	Optional element. Specifies the .NET property that triggers the condition.
ScriptBlock Element for ItemSelectionCondition for Controls for View	Optional element. Specifies the script that triggers the condition.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for Controls for View	Defines the data that is displayed by the control.

Remarks

You can specify one property name or a script for this condition but cannot specify both.

See Also

PropertyName Element for ItemSelectionCondition for Controls for View

ScriptBlock Element for ItemSelectionCondition for Controls for View

ExpressionBinding Element for CustomItem for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for ItemSelectionCondition for Controls for View

Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the control is used. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ItemSelectionCondition Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ItemSelectionCondition Element of ExpressionBinding for Controls for View	Defines the condition that must exist for this control to be used.

Text Value

Specify the name of the .NET property that triggers the condition.

Remarks

If this element is used, you cannot specify the ScriptBlock element when defining the selection condition.

See Also

ScriptBlock Element for ItemSelectionCondition for Controls for View

ItemSelectionCondition Element of ExpressionBinding for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for ItemSelectionCondition for Controls for View

Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the control is used. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ItemSelectionCondition Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ItemSelectionCondition Element of ExpressionBinding for Controls for View	Defines the condition that must exist for this control to be used.

Text Value

Specify the script that is evaluated.

Remarks

If this element is used, you cannot specify the PropertyName element when defining the selection condition.

See Also

PropertyName Element for ItemSelectionCondition for Controls for View

ItemSelectionCondition Element of ExpressionBinding for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for ExpressionBinding for Controls for View

Specifies the .NET property whose value is displayed by the control. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for Controls for View	Defines the data that is displayed by the control.

Text Value

Specify the name of the .NET property whose value is displayed by the control.

Remarks

See Also

ExpressionBinding Element for CustomItem for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for ExpressionBinding for Controls for View

Specifies the script whose value is displayed by the control. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements